

모델링 팀 실무 교안

실제 판매 IP 로 배우는 아키텍처 · 설계 · 모델링 · 검증

이 교안은 실제로 팔리거나 팔렸던 **IP** 의 소스를 근거로 쓰였다. 인용하는 코드는 전부 파일과 줄번호로 가리키며, 손으로 옮겨 적은 코드는 없다 — 모든 코드 인용은 디스크의 파일을 읽어 줄번호와 함께 찍는다.

수집한 실물: 55,409개 파일, **19,632,596**줄. 설계 IP(OpenTitan · Caliptra · CVA6 · Corundum · PULP · verilog-ethernet/pcie)와 모델링 자산(Spike · Sail · SystemC · Verilator · gem5 · aff3ct · OpenSSL · liquid-dsp)을 모두 받아 두었다.

읽는 법. 초록 상자는 학부 수준 복습, 보라 상자는 석사 수준 상세, 붉은 상자는 실무에서 사고가 났던 자리다.

차례

1장. 세 팀의 삼각형 — 당신은 어디에 서 있나	3
2장. 모델링 팀이 실제로 쓰는 코드.....	5
3장. 참조모델을 쓰는 규율 — 무엇이 ‘골든’인가.....	7
4장. DV 와의 계약 — 실제 비교 코드를 읽는다.....	9
5장. 첫 주 — 무엇을 묻고 무엇을 만드나	12
수 체계 — 모델과 RTL 이 갈라지는 1번 원인.....	14
타이밍 · 클럭 · 파이프라인 — 모델에 없는 세계	16
검증 이론 — ‘다 돌렸다’를 어떻게 말하나.....	18
AES 블록 — 암호 IP 의 표준 사례.....	20
AXI 인터커넥트 — 모든 SoC 의 등뼈.....	26
FIFO · CDC 기본 셀 — 가장 많이 쓰이고 가장 많이 틀린다	28
RISC-V 코어 — 모델링 팀의 주무대.....	29
RS-FEC — 통신 IP 의 필수 블록.....	30
SoC 하나를 만들려면 어떤 IP 가 필요한가.....	31
업계 표준 — 무엇을 알고 있어야 하나	33
논문 활용법 — 실무에서 어떻게 쓰나.....	35
IP 아이디어 — 무엇을 제안할 것인가.....	36
부록 A 본문이 인용한 소스 전문.....	38

1장. 세 팀의 삼각형 — 당신은 어디에 서 있나

반도체 IP 개발 조직은 세 팀으로 나뉜다. 이 구분은 관습이 아니라 버그를 정의하기 위한 구조다. 세 팀이 없으면 "무엇이 맞는가"를 정할 수 없다.

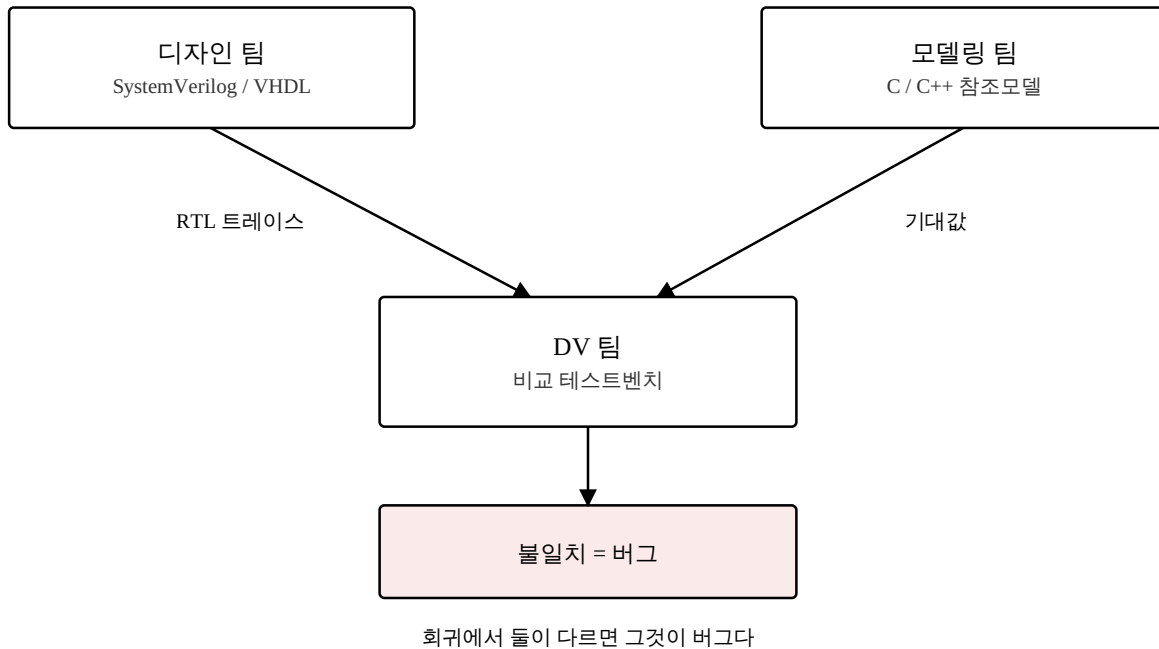


그림 1. 세 팀의 관계. 디자인은 하드웨어를, 모델링은 정답을, DV는 둘을 맞대는 장치를 만든다. 회귀(regression)에서 디자인과 모델이 다르면 그것이 버그다 — 이것이 버그의 조작적 정의다.

1.1 각 팀이 실제로 만드는 것

표 1. 세 팀의 산출물 — 이 교안이 실제로 받아 놓은 코드로

팀	만드는 것	언어	이 교안의 실물	규모
디자인	합성 가능한 RTL, 파이프라인 · FSM · 데이터패스	SystemVerilog / Verilog / VHDL	opentitan/hw/ip/aes/rtl	17,254줄
모델링	참조모델(골든). 명세의 의미를 코드로	C / C++ (SystemC · TLM)	riscv-isa-sim (Spike)	87,547줄
DV	테스트벤치 · 스코어보드 · 커버리지 · 자극 생성	SystemVerilog(UVM) Python(cocotb)	core-v-verif	637,962줄

학부 복습 · 세 팀이 같은 명세를 서로 다른 두 가지 방법으로 구현하고, 세 번째 팀이 둘을 비교한다. 두 독립 구현이 같은 답을 내면 명세를 제대로 읽었을 확률이 높다. 이것이 다중 구현에 의한 검증이고, 우주선 제어기부터 CPU까지 같은 원리다.

석사 수준 · 두 구현이 독립이 아니면 이 논증은 무너진다. 모델러와 설계자가 같은 사람에게 같은 설명을 듣고, 같은 오해를 하면, 회귀는 초록인데 칩은 틀린다. 실무에서 이것을 막는 장치가 명세를 문서가 아니라 실행 가능한 형식

모델로 두는 것이다 — RISC-V 의 Sail 모델 (riscv/sail-riscv, 100,524줄)이 그 예다. Sail 은 규격 문서와 C 예물 레이터를 같은 원천에서 낸다.

1.2 왜 모델이 RTL 보다 훨씬 작은가

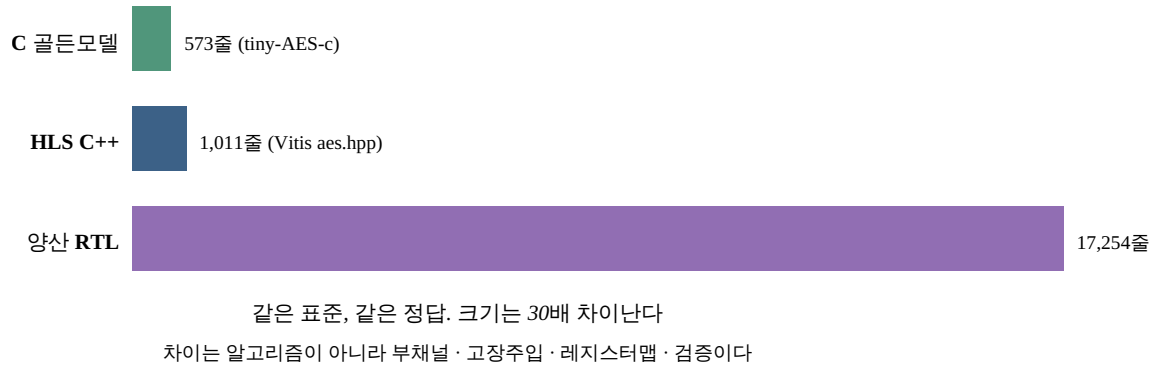


그림 2. 같은 AES 표준(FIPS-197)의 세 구현. 줄 수는 이 교안이 받아 놓은 실제 저장소를 기계로 센 값이다.

줄 수 차이가 30배다. 알고리즘이 어려워서가 아니다. 알고리즘은 세 구현이 똑같다. 차이는 전부 하드웨어에만 있는 문제들이다:

표 2. RTL에만 있고 모델에는 없는 것

RTL 이 추가로 감당하는 것	왜 모델에는 없나	OpenTitan AES 의 예
클럭 · 리셋 · 파이프라인 단	모델은 한 번에 한 블록을 계산하고 끝낸다	aes_cipher_control_fsm.sv
레지스터맵 · 버스 인터페이스	모델은 함수 호출로 받는다	aes_reg_top.sv, TL-UL 어댑터
부채널 대책(마스킹)	모델은 전력을 소비하지 않는다	aes_sbox_dom.sv, aes_prng_masking.sv
고장주입 대책(이중화)	모델은 알파입자를 맞지 않는다	aes_cipher_control_fsm_p/n.sv
키 삭제 · 수명 관리	모델은 메모리를 지우지 않아도 된다	aes_prng_clearing.sv
합성 제약 · 타이밍 클로저	모델은 시간을 소비하지 않는다	SDC · prim_buf 장벽

모델링 팀의 레버리지가 여기 있다. 573줄짜리 C 모델이 17,254줄짜리 RTL 의 정답을 정의한다. 30배 적은 코드로 30배 많은 코드를 붙든다. 대신 모델이 틀리면 전부가 틀린다 — 그래서 모델은 RTL 보다 더 엄격하게 검토된다.

2장. 모델링 팀이 실제로 쓰는 코드

추상적으로 말하지 않는다. RISC-V 의 표준 ISS 인 **Spike** (riscv-software-src/riscv-isa-sim)를 열어 본다. 이것은 실제 업계에서 RTL 검증의 기준으로 쓰이는 참조모델이고, 이 교안에 전문이 들어 있다.

2.1 명령어 하나 = 파일 하나

Spike 의 riscv/insns/ 에는 **1,588**개의 파일이 있고, 파일 하나가 명령어 하나의 의미다. 본문은 대개 한 줄이다.

```
riscv-software-src_riscv-isa-sim/riscv/insns/add.h:1-1  - ADD 명령어의 전부
1  WRITE_RD(sext_xlen(RS1 + RS2));
```

```
riscv-software-src_riscv-isa-sim/riscv/insns/mulh.h:1-5  - MULH -- XLEN 에 따라 갈린다
1  require_either_extension('M', EXT_ZMMUL);
2  if (xlen == 64)
3      WRITE_RD(mulh(RS1, RS2));
4  else
5      WRITE_RD(sext32((sext32(RS1) * sext32(RS2)) >> 32));
```

학부 복습 · WRITE_RD, RS1, sext_xlen 은 매크로다. 명령어 본문이 아키텍처 상태를 어떻게 바꾸는가만 적고, 그것을 어떻게 가져오고 되쓰는지는 매크로가 감춘다. 그래서 본문이 명세 문장과 거의 1:1 로 읽힌다.

석사 수준 · 이 설계가 주는 실무 이득은 확장의 비용이다. 새 확장(extension)이 나오면 insns/ 에 파일을 더하고 디코더 표에 한 줄을 넣으면 된다. 모델 전체를 건드리지 않는다. 당신이 회사에서 만들 모델도 이 구조를 따라야 한다 — 명령어별 파일 · 매크로로 감춘 상태 접근 · 표로 만든 디코더 . 이것을 처음부터 안 하면 확장 50개째에서 무너진다.

2.2 진짜 어려운 부분은 부동소수점이다

Spike 의 riscv/ 가 46,840줄인데 softfloat/ 만 **25,219**줄이다. IEEE 754 를 비트 단위로 정확히 재현하는 것이 명령어 의미를 적는 것보다 무겁다.

석사 수준 · 왜 그런가. IEEE 754 는 반올림 모드 5가지, 예외 플래그 5가지, 비정규수(subnormal), NaN 전파 규칙, 부호 있는 0 을 전부 규정한다. 하드웨어가 이 중 하나라도 다르게 하면 회귀가 빨개진다. 그런데 컴파일러의 double 연산을 그대로 쓰면 호스트 CPU 의 반올림에 끌려가므로 정답이 호스트마다 달라진다. 그래서 Berkeley SoftFloat 같은 비트 단위 정수 구현을 쓴다. 모델링 팀에서 부동소수점을 맡으면 이 점을 가장 먼저 확인하라 — 호스트 부동소수점을 쓰고 있지 않은가.

2.3 모델은 무엇을 흉내내지 않는가

표 3. ISS 가 모델링하는 것과 하지 않는 것

모델링한다	하지 않는다	그래서 DV 가 감당한다
아키텍처 상태(PC · GPR · CSR)	파이프라인 단계 · 스톱	타이밍은 RTL 만의 진실. 비교는 명령어 retire 시점에 한다
메모리 내용	캐시 히트/미스 · 버스 지연	성능은 별도 모델(gem5)이 본다
예외 · 인터럽트 의미	인터럽트가 언제 들어오는지	DV 가 인터럽트 시점을 모델에 주입해 맞춘다
권한 · 가상메모리 변환	TLB 교체 정책	구현 자유 영역

이 표의 오른쪽이 실무에서 가장 자주 사고를 낸다. 모델은 맞는데 비교가 실패한다. 원인은 대개 "모델이 훑내내지 않는 것"을 비교하려 했기 때문이다. 실제 사례가 이 교안 안에 있다 — core-v-verif 의 비교 코드에 이런 주석이 남아 있다:

```
// FIXME: I am removing the static (non-written) register checks, as they fail in  
presence of I and D bus RAM stalls
```

버스 스툴이 있으면 "바뀌지 않았어야 할 레지스터" 검사가 실패한다. 모델이 틀린 것이 아니라 비교 시점이 틀린 것이다.

3장. 참조모델을 쓰는 규율 — 무엇이 '골든'인가

모델링 팀의 코드는 정답으로 쓰인다. 그래서 일반 소프트웨어와 다른 규율이 붙는다. 아래 일곱은 실무에서 사고가 났던 자리들이고, 이 교안이 받아 놓은 실제 코드에서 근거를 댈다.

3.1 규율 일곱

표 4. 참조모델 작성 규율

#	규율	지키지 않으면	근거
1	호스트 산술에 기대지 마라. 폭·반올림·오버플로를 직접 구현한다	같은 모델이 x86 과 ARM 에서 다른 답을 낸다	Spike 가 Berkeley SoftFloat 를 따로 두는 이유
2	상태 접근을 매크로/접근자로 감춰라	명령어 500개를 고칠 일이 생긴다	WRITE_RD · RS1
3	미정의 동작을 '아무거나'로 두지 마라. 명시적으로 표시한다	RTL 이 다른 값을 내는데 누가 맞는지 못 정한다	RISC-V 의 <i>reserved</i> 처리
4	모델은 빨라야 한다. 회귀가 수만 번 돈다	야간 회귀가 아침에 안 끝난다	Spike 는 인터프리터인데도 표 기반 디코더
5	추적(trace)을 낼 수 있어야 한다	DV 가 비교할 대상이 없다	RVFI · Spike 의 commit log
6	결정론적이어야 한다. 같은 입력 → 같은 출력, 항상	재현이 안 되는 버그가 생긴다	난수는 씨앗을 받아 쓴다
7	규격 문서의 절 번호를 코드에 적어라	3년 뒤 왜 그런지 아무도 모른다	OpenTitan RTL 의 주석 관행

석사 수준 · 규율 3 이 가장 자주 싸움을 만든다. 명세가 "이 경우 동작은 정의되지 않는다" 라고 하면, 모델은 무엇을 해야 하는가? 선택지는 셋이다. (가) 임의값을 내고 DV 에게 "여기는 비교하지 마라" 고 알린다. (나) RTL 이 내는 값을 모델도 내도록 맞춘다 — 이러면 모델이 독립성을 잃는다. (다) 모델이 명시적으로 *undefined* 표시를 내보내고 DV 가 그 필드를 마스킹한다. (다)가 정답이다. (나)를 하면 그 순간 그 필드에 대해 검증이 사라지는데, 아무도 모른다.

3.2 검사가 '무는지'부터 확인하라

참조모델 검사에서 가장 흔한 거짓 초록은 검사가 아무것도 안 하는 경우 다. 예를 들어 오류 정정 블록의 복호기를 시험하면서 오류를 0개 넣으면, 복호기가 아무 일도 안 해도 통과한다.

실무 습관. 검사를 짜고 나서 일부러 모델을 망가뜨려 보라. 곱셈에 1을 더하거나, 신드롬을 0으로 박거나, 패리티를 0으로 만들어 보고 검사가 빨개지는지 본다. 빨개지지 않으면 그 검사는 없는 것과 같다. 이 교안 저자의 저장소에 서는 이것을 검사 파일 안에 같이 둔다:

```
def test_망가진_복호기는_잡힌다():
    원래 = c.신드롬
    c.신드롬 = lambda r: [0] * (2 * c.t)      # 조용히 망가뜨린다
    res = 측정(c, 오류수=2, 시행=20)
    assert res["고침"] < 20, "신드롬을 0 으로 박았는데 전부 고쳐졌다 -- 검사가 안 문다"
```

3.3 모델과 RTL 이 다를 때 누가 틀렸는지 가리는 법

표 5. 불일치 삼분법 — 이 순서로 의심한다

순서	의심	확인 방법	실제 빈도
1	비교가 틀렸다 (시점 · 마스크 · 필드 선택)	모델 단독 / RTL 단독으로 같은 입력을 돌려 로그를 눈으로 본다	가장 흔하다
2	모델이 틀렸다	명세 조문을 다시 읽는다. 다른 구현(타사 · 오픈소스)과 대조한다	그 다음
3	RTL 이 틀렸다 — 진짜 버그	최소 재현 케이스를 만들어 설계자에게 넘긴다	가장 드물다

1번을 건너뛰고 3번으로 가면 신뢰를 잃는다. "버그 같습니다" 라고 설계자에게 넘겼는데 비교 코드 문제였던 일이 두 번 반복되면, 세 번째 진짜 버그를 아무도 안 본다. 최소 재현 케이스 없이 버그를 넘기지 마라.

4장. DV 와의 계약 — 실제 비교 코드를 읽는다

모델과 RTL 을 어떻게 맞대는가. 이 교안에는 실제 CPU 검증환경의 비교 코드가 들어 있다 — `core-v-verif` 의 `uvmt_cv32e40p_step_compare.sv` 파일 머리에 출처가 적혀 있다:

```
openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_step_compare.sv:22-26 — 이 파일이 어디서 왔는지
22
23 // Step-and-Compare between the CV32E40PE40P and Imperas OVPsim ISS
24 // Cloned from the Imperas demo at $(IMPERAS_HOME)/RTL_OVPmodel_step_compare/verilog_testbench/
testbench.sv
25
26 /*
```

학부 복습 · **Imperas** 는 상용 ISS(OVPsim) 회사다. 업계가 쓰는 패턴을 오픈 프로젝트가 그대로 따른 것이므로, 여기서 배우는 구조가 곧 실무 구조다.

4.1 step-and-compare: 한 명령어마다 전부 맞댄다

```
openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_step_compare.sv:141-168 — PC 와 GPR 32개를 맞대는
부분
141 // Compare PC
142 //
check_32bit(.compared("PC"), .expected(step_compare_if.ovp_cpu_PCr), .actual(step_compare_if.insn_pc));
143
check_32bit(.compared("PC"), .expected(`CV32E40P_RM_RVVI_STATE.pc), .actual(step_compare_if.insn_pc));
144     step_compare_if.num_pc_checks++;
145
146 // Compare GPR's
147 // Assuming that `CV32E40P_TRACER.insn_regs_write size is never > 1. Check this.
148 // Note that dut_wrap is found 1 level up
149     insn_regs_write_size = `CV32E40P_TRACER.insn_regs_write.size();
150     if (insn_regs_write_size > 1) begin
151         `uvm_error("Step-and-Compare", $sformatf("Assume insn_regs_write size is 0 or 1 but is
%0d", insn_regs_write_size))
152     end
153     else if (insn_regs_write_size == 1) begin // Get `CV32E40P_TRACER.insn_regs_write fields
if size is 1
154         insn_regs_write_addr = `CV32E40P_TRACER.insn_regs_write[0].addr;
155         insn_regs_write_value = `CV32E40P_TRACER.insn_regs_write[0].value;
156         `uvm_info("Step-and-Compare", $sformatf("insn_regs_write queue[0] addr=0x%0x,
value=0x%0x", insn_regs_write_addr, insn_regs_write_value), UVM_DEBUG)
157     end
158
159 // Ignore insn_regs_write_addr=0 just like in riscv_tracer.sv
160     for (idx=0; idx<32; idx++) begin
161         compared_str = $sformatf("GPR[%0d]", idx);
162         if ((idx == insn_regs_write_addr) && (idx != 0) && (insn_regs_write_size == 1)) // Use
register in insn_regs_write queue if it exists
163             check_32bit(.compared(compared_str), .expected(step_compare_if.ovp_cpu_GPR[idx]
[31:0]), .actual(insn_regs_write_value));
164         // FIXME:stichmo:I am removing the static (non-written) register checks, as they fail
in presence of I and D bus RAM stalls
165         // It would be highly desirable to find an alternative for this type of check to ensure
unintended writes to do not
166         else if (!is_stall_sim && !`CV32E40P_TRACER.insn_wb_bypass) // Use actual value from
RTL to compare registers which should have not changed
167             check_32bit(.compared(compared_str), .expected(step_compare_if.ovp_cpu_GPR[idx]
[31:0]), .actual(step_compare_if.riscy_GPR[idx]));
168         step_compare_if.num_gpr_checks++;
```

구조는 단순하다. 명령어가 retire 될 때마다:

1. **PC** 를 맞댄다 — 모델의 state.pc 대 RTL 의 insn_pc
2. **GPR 32개** 전부 를 맞댄다. 이번에 쓰인 레지스터는 쓰인 값으로, 나머지는 바뀌지 않았음을 확인한다
3. **CSR** 을 맞댄다 — mstatus, misa, mie, mtvec, mcountinhibit ...

석사 수준 · 2번의 “나머지는 바뀌지 않았음” 검사가 중요하다. 쓰인 레지스터만 보면, RTL 이 엉뚱한 레지스터를 추가로 덮어쓰는 버그를 못 잡는다. 그런데 바로 이 검사가 버스 스톱이 있을 때 실패해서 실무에서 꺼졌다(같은 파일의 FIXME 주석). 커버리지와 실행 가능성의 맞교환 이고, 이런 결정은 반드시 문서에 남겨야 한다. 남기지 않으면 몇 달 뒤 “그 검사는 왜 없지?” 로 돌아온다.

4.2 RVFI — 비교를 가능하게 하는 표준 포트

모델과 RTL 을 맞대려면 RTL 이 무엇을 언제 했는지 말해 줘야 한다. RISC-V 진영의 표준이 **RVFI**(RISC-V Formal Interface)다. DUT 가 명령어를 retire 할 때마다 다음을 내보낸다.

표 6. RVFI 가 내보내는 것 (핵심 필드)

필드	뜻	비교에서 쓰이는 곳
rvfi_valid	이번 사이클에 명령어가 retire 됐다	비교 시점을 정한다
rvfi_order	프로그램 순서 번호	순서 뒤바뀜 검출
rvfi_insn	retire 된 명령어 인코딩	모델에 같은 것을 먹인다
rvfi_pc_rdata/wdata	명령어 전/후 PC	분기 정확성
rvfi_rs1/rs2_addr, rdata	읽은 레지스터	모델 입력 대조
rvfi_rd_addr, wdata	쓴 레지스터	결과 대조
rvfi_mem_addr, rmask, wmask, rdata, wdata	메모리 접근	메모리 부작용
rvfi_trap	예외 발생	예외 의미 대조

이 교안에는 RVFI UVM 에이전트 구현이 2,174줄 들어 있고 (lib/uvm_agents/uvma_rvfi/), DUT 쪽 구현도 있다 (cv32e40p/bhv/cv32e40p_rvfi.sv).

당신 회사에 **RVFI** 가 없다면 그에 해당하는 것을 찾아라. 이름이 다를 뿐 모든 팀에 있다 — retire 트레이스, commit log, monitor 포트. 없다면 그것을 만드는 것이 첫 과제다. 트레이스 포트가 없으면 비교 자체를 할 수 없다.

4.3 비교 시점 — 실무에서 제일 많이 싸우는 자리

표 7. 무엇을 언제 비교할 것인가

대상	비교 시점	함정
아키텍처 상태(레지스터 · 메모리)	명령어 retire 순간	파이프라인 때문에 RTL 내부에는 여러 시점이 공존한다
CSR	retire 직후. 단, 부작용이 늦게 반영되는 것 주의	카운터(mcycle)는 절대 비교하면 안 된다 — 모델에 사이클 개념이 없다
인터럽트	DV 가 모델에 ‘지금 들어왔다’고 주입한 뒤	주입 시점이 한 사이클 어긋나면 전부 틀어진다
메모리 순서	약한 순서 모델에서는 단일 시점 비교 불가	리터먼트 순서가 아니라 허용 가능한 순서 집합과 비교해야 한다

대상	비교 시점	함정
성능 카운터 · 캐시 상태	비교하지 않는다	모델이 흉내내지 않는 영역

5장. 첫 주 — 무엇을 묻고 무엇을 만드나

5.1 출근 첫날 물어볼 것 열 가지

표 8. 첫 주 질문지 (답을 못 받으면 그것 자체가 정보다)

#	질문	왜 중요한가
1	우리 골든 모델은 무엇인가? 자체 제작인가, 상용(Imperas 등)인가?	자체면 당신이 고칠 수 있고, 상용이면 계약 범위가 한계다
2	모델과 RTL 의 트레이스 인터페이스는 무엇인가?	없으면 그것을 만드는 게 첫 과제다
3	회귀는 하루에 몇 번 도나? 한 번에 몇 시간인가?	모델 성능 요구가 여기서 나온다
4	명세는 무엇인가? 표준 문서인가, 사내 스펙인가, 구두인가?	구두면 모델이 사실상의 명세가 된다 — 책임이 커진다
5	미정의 동작은 어떻게 처리하고 있나?	3장 1.5절의 (가)(나)(다) 중 무엇인가
6	커버리지는 무엇을 재나? 기능 커버리지가 있나?	없으면 '다 돌렸다'를 말할 수 없다
7	모델 자체의 검증은 누가 하나?	대개 공백이다. 여기가 당신의 첫 기여 자리다
8	부동소수점/고정소수점이 있나? 비트 정확한가?	가장 비싼 버그가 여기서 난다
9	모델은 어떤 언어/빌드인가? C++ 표준은? SystemC 를 쓰나?	환경 파악
10	지난 6개월에 놓친 버그(escape)가 있었나? 왜 놓쳤나?	이 질문 하나가 나머지 아홉보다 많은 것을 알려 준다

5.2 도구 — 이 교안에 실물이 들어 있는 것들

표 9. 모델링·검증 도구

도구	무엇	규모	언제 쓰나
Verilator	RTL → C++ 변환·시뮬레이션	887,306줄	모델과 RTL 을 한 프로세스에서 돌린다. 무료. 사이클 정확
SystemC / TLM 2.0	C++ 하드웨어 모델링 표준	397,368줄	가상 플랫폼 · 아키텍처 탐색. 대기업 표준
cocotb	파이썬으로 HDL 공동시물	67,236줄	UVM 없이 빠르게 테스트벤치. 모델이 파이썬일 때
Spike	RISC-V ISS	87,547줄	참조모델의 교과서
Sail	형식 ISA 명세 → C 에뮬레이터 생성	100,524줄	명세와 모델을 한 원천에서
gem5	아키텍처 성능 시뮬레이터	1,719,554 줄	성능 예측 · 아키텍처 탐색. 기능 모델과 다르다
aff3ct	FEC 시뮬레이터 (RS·LDPC·Polar·Turbo)	101,084줄	통신 IP 참조모델
liquid-dsp	통신 DSP C 라이브러리	185,931줄	변복조 · 필터 참조
OpenSSL	암호 참조	946,224줄	암호 IP 의 골든. NIST 벡터 포함
Verible	SystemVerilog 파서·린터	223,396줄	RTL 을 프로그램으로 다뤄야 할 때

5.3 DPI-C — C 모델을 SystemVerilog 에 붙이는 표준 통로

IEEE 1800 의 **DPI-C**(Direct Programming Interface)가 표준 방법이다. SystemVerilog 가 C 함수를 직접 부른다.

```

C 쪽 -- 평범한 C++ 로 컴파일된다
extern "C" void ref_step(const svOpenArrayHandle in, svOpenArrayHandle out);

SystemVerilog 쪽
import "DPI-C" function void ref_step(input int in[], output int out[]);
...
ref_step(stim, expected);
dut_step(stim, actual);
if (actual != expected) $fatal(1, "mismatch at vector %0d", i);

```

흔한 오해를 하나 정정한다. "UVM 이 DPI 로 참조모델을 제공한다" 는 말이 돌지만 사실이 아니다. UVM 의 DPI 계층은 배선용 도구일 뿐이다 — `uvm_hdl_read`, `uvm_hdl_deposit`, `uvm_hdl_force`, 정규식, 도구 이름 질의. 크기가 그것을 말한다: `uvm_component.svh` 가 122,681바이트인데 `uvm_dpi.cc` 는 2,658바이트다 — **46배**. 알고리즘 모델은 당신이 쓴다. 방법론이 주지 않는다.

5.4 첫 달에 만들면 좋은 것

표 10. 기여할 자리 — 대개 비어 있다

만들 것	왜 비어 있나	효과
모델 자체의 회귀(모델 vs 독립 구현)	모델이 정답이라 아무도 모델을 검사하지 않는다	가장 비싼 버그를 막는다
변이 시험(모델을 일부러 망가뜨려 검사가 무는지)	시간이 없어서 안 한다	죽은 검사를 찾아낸다
커버리지 계측(모델이 어떤 경로를 안 밟았나)	RTL 커버리지만 보는 팀이 많다	자극의 빈틈을 본다
불일치 삼분법 자동화(3.3절)	매번 손으로 판단한다	분류 시간을 줄인다
규격 절번호 → 코드 위치 표	문서화가 뒤로 밀린다	새 사람이 올 때마다 값을 한다

수 체계 — 모델과 RTL 이 갈라지는 1번 원인

모델링 팀에서 가장 비싼 버그는 알고리즘 오류가 아니다. 수를 어떻게 자르고 넘치게 하느냐에서 난다. 이 장은 그것만 다룬다.

Q4.12 (부호 1 · 정수 4 · 소수 12 = 16비트)



곱셈: $Q4.12 \times Q4.12 = Q8.24 \rightarrow 32$ 비트. 그대로 두면 폭이 매 연산마다 커진다

그래서 다시 Q4.12 로 줄인다 — 그 순간 두 가지를 정해야 한다:



****이 두 선택이 모델과 RTL 을 갈라놓는 최대 원인이다****

그림 3. 고정소수점 표기와 연산 후 폭 증가. 폭을 줄일 때 반올림과 넘침 방식을 정해야 하고, 그 선택이 명세에 없으면 모델과 RTL 이 달라진다.

1. 고정소수점의 네 가지 결정

표 11. 모든 고정소수점 블록에서 반드시 명세에 적혀야 하는 것

결정	선택지	안 적으면
폭과 소수점 위치	$Qm.n$, 부호 유무	가장 먼저 어긋난다
반올림	절삭(truncate) · 최근접 · 최근접작수 · 0에서 멀리	LSB 가 다르다. 누적되면 크게 벌어진다
넘침	포화(saturate) vs 감싸기(wrap)	최악. 부호가 뒤집힌다
연산 순서	누산 순서 · 결합 순서	포화를 쓰면 순서가 결과를 바꾼다

석사 수준 · 포화와 감싸기는 취향이 아니다. 두 가지를 실측으로 구분해 둔다.

- 2의 보수 감싸기는 환(ring)이다. 덧셈에 대해 $\text{mod } 2^W$ 로 닫혀 있으므로 연산 순서를 바꿔도 결과가 같다. 중간이 넘쳐도 최종값이 범위 안이면 정답이 나온다.
- 포화는 환이 아니다. 한 번 붙잡히면 정보가 사라지므로 순서가 결과를 바꾼다. 같은 식을 다른 순서로 계산한 모델과 RTL 이 갈라진다.

그런데 어느 쪽이 옳은지는 응용이 정한다. 등화기 같은 신호처리 데이터패스는 감싸기가 안전할 때가 많고(중간 넘침이 서로 상쇄됨), LLR 을 다루는 복호기는 반대로 반드시 포화해야 한다 — 감싸면 확률의 부호가 뒤집혀 복호가 무너진다. 저자의 실측: LDPC 복호에서 폭을 줄였을 때 포화는 BLER 0.250, 감싸기는 1.000(완전 실패)이었다.

2. 되먹임 레지스터의 초기값

실측 사고. 되먹임 구조(DFE · IIR · 누산기)에서 모델과 RTL 의 레지스터 초기값이 다르면, 초기 몇 심볼이 다르고 그 차이가 되먹임으로 남는다. 저자의 실측에서 200개 벡터 중 103개가 불일치했고, 초기값을 맞추자 10개로 줄었

다(나머지는 다른 원인). “리셋 후 상태”는 명세 항목이다. 모델이 0으로 시작하는지, RTL 이 X 로 시작하는지 반드시 정하라.

3. 부동소수점 — 호스트를 믿지 마라

표 12. IEEE 754 에서 반드시 맞춰야 하는 것

항목	선택지	모델이 흔히 틀리는 방식
반올림 모드	RNE · RTZ · RUP · RDN · RMM	호스트 기본값(RNE)만 쓰고 모드 전환을 무시
예외 플래그	NV · DZ · OF · UF · NX	플래그를 아예 안 만든다. RTL 은 만든다
비정규수	지원 / flush-to-zero	호스트가 FTZ 로 돌고 있는 줄 모른다
NaN 전파	어느 피연산자의 payload 를 남기나	규격마다 다르다
부호 있는 0	-0 과 +0 구분	비교에서 같다고 판정해 버린다
FMA	중간 반올림 없음	$a*b+c$ 를 따로 계산해 LSB 가 다르다

그래서 **Spike** 는 **Berkeley SoftFloat** 를 쓴다 — 이 교안의 실물로 softfloat/ 만 25,219줄이다. 부동소수점을 맡으면 첫 질문은 “우리 모델이 호스트 부동소수점을 쓰고 있는가” 다.

4. 실무 점검표

표 13. 수 체계 점검표 — 블록을 받으면 이것부터

#	확인	확인 방법
1	모든 신호의 Q 포맷이 문서에 있나	없으면 RTL 에서 추출해 문서화한다
2	반올림 방식이 단 하나로 정해졌나	블록마다 다르면 그 자체가 위험
3	포화인가 감싸기인가	경계값 벡터로 직접 재본다
4	리셋 후 초기값이 모델과 RTL 에서 같나	첫 N 사이클을 따로 비교
5	누산 순서가 정해졌나	포화를 쓴다면 반드시
6	경계값 시험이 있나	최대 · 최소 · 최대+1 · 0 · -1
7	부동소수점이면 SoftFloat 계열인가	호스트 의존이면 즉시 보고

타이밍 · 클럭 · 파이프라인 — 모델에 없는 세계

학부 복습 · 동기 회로는 클럭 상승엣지마다 플립플롭이 입력을 붙잡는다. 붙잡으려면 입력이 엣지 전에 안정되어 있어야 하고(setup), 엣지 후에도 잠시 유지되어야 한다(hold). 두 플립플롭 사이 조합논리 지연이 클럭 주기보다 길면 setup 위반이다.

$$T_{clk} \geq t_{cq} + t_{logic} + t_{setup} + t_{skew}$$

석사 수준 · 모델링 팀이 이것을 알아야 하는 이유. 모델은 시간을 소비하지 않으므로 이 식이 없다. 그런데 아키텍처 결정이 이 식에 걸린다. 예: 되먹임 고리(DFE · 누산기 · 의존성 있는 파이프라인)는 한 사이클 안에 닫혀야 하므로 고리 안의 논리량이 성능 상한을 정한다. 언롤 · 투기실행 · sub-rate 병렬화는 전부 이 제약을 피하려는 기법이고, 모델 쪽에서는 “결과가 같다”로만 보인다. 모델이 같다고 해서 구현 가능한 것이 아니다.

표 14. 모델에는 없고 RTL에만 있는 시간 개념

개념	무엇	모델링 팀이 알아야 할 이유
파이프라인 단(stage)	연산을 쪼개 처리량을 올린다	지연이 늘어난다 — 비교 시점이 밀린다
초기화 구간(II)	새 입력을 몇 사이클마다 받나	처리량의 정의
스틀 / 백프레서	받는 쪽이 못 받으면 멈춘다	비교 실패의 최대 원인. 4.3절
지연(latency)	입력에서 출력까지 사이클	스코어보드 정렬에 필요
클럭 도메인 교차(CDC)	다른 클럭 사이 신호 전달	간헐 오류의 원인
리셋 순서	비동기 assert · 동기 deassert	초기값 불일치

클럭 도메인 교차(CDC)



1비트 제어신호에만 쓴다

다중비트 = 그레이 코드 · 핸드셰이크 · 비동기 FIFO

2단 동기화기를 다중비트에 쓰면 비트마다 다른 사이클에 넘어온다 — 값이 깨진다

그림 4. CDC의 기본. 2단 동기화기는 1비트 제어신호 전용이다. 다중비트에는 그레이 코드 · 핸드셰이크 · 비동기 FIFO를 쓴다.

석사 수준 · 메타스테이빌리티. 플립플롭이 setup/hold를 어긴 입력을 붙잡으면 출력이 0도 1도 아닌 중간 전압에 머물 수 있고, 안정되는 시간이 확률적이다. 평균 고장 간격은 $MTBF = e^{t_r/\tau} / (T_0 \cdot f_{clk} \cdot f_{data})$ 로 모델링되며, 동기화 단을 하나 더 넣으면 t_r 이 한 주기 늘어 MTBF가 지수적으로 좋아진다. 그래서 2단이 표준이고 3단을 쓰기도 한다. 시

물레이션으로는 절대 못 잡는다 — 논리 시뮬레이터에 메타스테이빌리티가 없기 때문이다. CDC 는 정적 검사 도구로 잡는다.

이 교안의 실물에서 CDC 기본 셀을 볼 수 있다 — `pulp-platform/common_cells` (25,535줄) 에 `cdc_2phase`, `cdc_fifo_gray`, `sync` 등이 있다.

검증 이론 — ‘다 돌렸다’를 어떻게 말하나

1. 커버리지 세 종류

표 15. 커버리지의 종류와 한계

종류	재는 것	한계
코드 커버리지(line · branch · toggle · FSM)	RTL 의 어느 줄/분기/천이가 실행됐나	100%여도 기능이 맞다는 뜻이 아니다. 안 쓴 조합은 안 보인다
기능 커버리지(covergroup · cross)	설계자가 ‘보야 한다’고 정의한 상황이 일어났나	정의하지 않은 상황은 영원히 안 보인다
단언 커버리지	속성이 실제로 발동했나	vacuous pass(전제가 거짓이라 자동 통과)에 주의

커버리지 **100%**는 “끝”이 아니라 “내가 세운 목표를 다 채웠다”는 뜻이다. 기능 커버리지는 사람이 적은 것이므로, 적지 않은 위험은 커버리지에 나타나지 않는다. 그래서 escape 분석 (놓친 버그가 왜 안 잡혔나)이 커버리지보다 중요하다.

2. 제약 랜덤(CRV)

손으로 벡터를 쓰면 사람이 생각한 것만 시험한다. 제약 랜덤은 유효 범위를 제약으로 적고 그 안에서 무작위 로 뽑는다. 사람이 생각 못 한 조합이 나온다.

석사 수준 · 실무의 어려움은 ‘랜덤’이 아니라 ‘제약’이다. 제약이 느슨하면 무효 입력만 쏟아지고, 빡빡하면 흥미로운 구석에 못 간다. 그래서 가중치(dist)로 경계값을 자주 뽑게 하고, 커버리지가 안 차는 구멍을 보고 제약을 조인다 — **coverage-driven verification**. 씨앗(seed)을 바꿔 가며 야간 회귀를 수천 번 도는 것이 이 때문이다. 재현성을 위해 씨앗은 반드시 로그에 남겨야 한다.

3. 형식 검증(Formal)

표 16. 형식 검증이 잘 되는 자리와 안 되는 자리

잘 됨	왜	안 됨	왜
제어 FSM · 아비터	상태공간이 작다	데이터패스 전체	상태공간 폭발
프로토콜 준수(AXI 등)	속성이 국소적	부동소수점 유닛	너무 크다
레지스터 접근 규칙	규칙이 명시적	캐시 일관성 전체	조합 폭발
데드락 · 도달불가 상태	도달성 문제	성능 특성	형식 대상이 아니다

모델링 팀과의 접점. 형식 검증은 속성이 필요하다. “무엇이 참이어야 하는가”를 적는 일인데, 그 답을 가장 잘 아는 사람이 모델러다. 명세를 코드로 옮겨 본 사람이기 때문이다. 형식팀에 속성 목록을 제공하는 것은 모델링 팀의 자연스러운 기여다.

4. 회귀와 분류(triage)

표 17. 야간 회귀가 아침에 남기는 것과 대응

결과	뜻	먼저 할 일
새 실패	어제 없던 것	최근 커밋과 대조. 가장 우선

결과	뜻	먼저 할 일
기존 실패	이미 아는 것	티켓 번호가 붙어 있어야 한다
간헐 실패	씨앗에 따라 다름	가장 위험하다. '플레이크'로 치우지 마라 — 대개 진짜 버그다
인프라 실패	라이선스 · 디스크 · 도구	실패 이유가 분류돼야 한다
타임아웃	안 끝남	성능 회귀 또는 라이브락

“플레이크”는 원인이 아니다. 간헐 실패를 재현 못 한다고 닫으면, 같은 버그가 실리콘에서 나온다. 씨앗을 고정해 재현하고, 못 하면 미해결로 남겨 두는 것이 맞다. 닫지 마라.

AES 블록 — 암호 IP 의 표준 사례

표준 FIPS-197 (NIST, 2001) • 블록 128비트 • 키 128/192/256비트 • 라운드 10/12/14 • 이 교안의 실물 C 골든 573줄 • HLS C++ 1,011줄 • 양산 RTL 17,254줄

1. 왜 AES 를 먼저 배우나

AES 는 IP 교육의 표준 예제다. 이유가 네 가지다.

1. 정답이 공개돼 있다. NIST 가 시험 벡터를 낸다. 맞고 틀림에 이견이 없다
2. 알고리즘이 작다. 라운드 함수 네 개가 전부다
3. 그런데 실제 **IP** 는 크다. 30배 차이가 어디서 오는지가 곧 ‘IP 란 무엇인가’에 대한 답이다
4. 거의 모든 칩에 들어간다. 보안 부트 · 저장장치 암호화 · 통신 링크

2. 수학 — GF(2⁸) 위에서 도는 암호

학부 복습 · AES 의 모든 산술은 유한체 **GF(2⁸)** 에서 일어난다. 원소는 바이트 하나(0-255)이고, 각 바이트를 7차 이하 다항식의 계수로 읽는다. 예를 들어 0x57 = 0101 0111 은 $x^6+x^4+x^2+x+1$ 이다.

덧셈은 계수별 XOR 이다. 표수 2 이므로 덧셈과 뺄셈이 같다. 곱셈은 다항식 곱 뒤 기약다항식으로 나눈 나머지를 취한다. AES 가 정한 기약다항식은

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (0x11B)$$

구현에서는 x 를 곱하는 연산(xtime)만 있으면 임의 곱셈을 만들 수 있다. 골든 모델의 실물은 이렇다.

```
kokke_tiny-AES-c/aes.c:294-297 - x 를 곱한다 -- 왼쪽 시프트, 넘치면 0x1B 로 환원
294 static uint8_t xtime(uint8_t x)
295 {
296     return ((x<<1) ^ (((x>>7) & 1) * 0x1b));
297 }
```

석사 수준 · 왜 **0x1b** 인가. 기약다항식 0x11B 에서 최고차항 x^8 (=0x100)을 뺀 나머지가 0x1B 다. 왼쪽 시프트로 8비트를 넘으면 x^8 항이 생기는데, 그것을 x^4+x^3+x+1 로 바꿔 더하는 것이 곧 $\wedge 0x1b$ 다. 조건부를 쓰지 않고 곱셈으로 쓴 것(* 0x1b)에 주목하라 — 분기를 없애 실행 시간이 데이터에 의존하지 않게 한 것이다. 타이밍 부채널 대책의 가장 기초적인 형태다.

2.1 S-box — 표가 아니라 두 연산의 합성이다

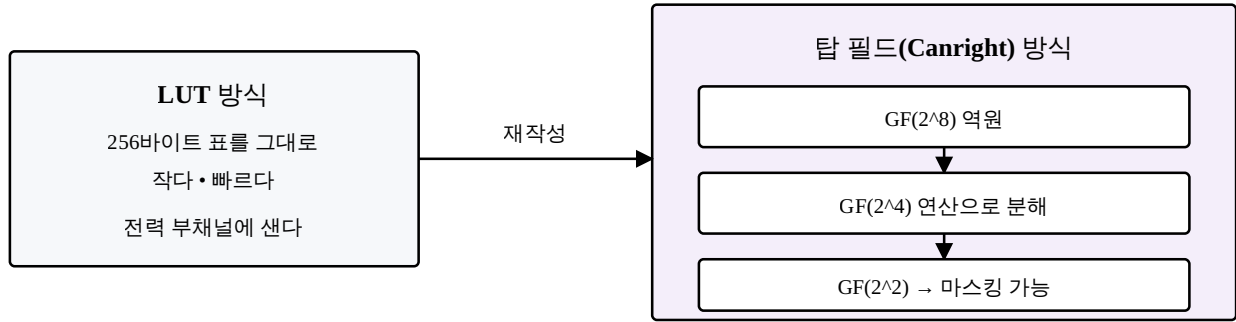
S-box 는 256바이트 표로 주어지지만, 표는 정의가 아니라 계산 결과다. 정의는 이렇다.

$$S(a) = A \cdot a^{-1} \oplus b$$

즉 (i) GF(2⁸) 에서의 곱셈 역원을 구하고(0은 0으로), (ii) 비트에 대한 아핀 변환(고정 행렬 A 와 상수 b=0x63)을 적용한다.

석사 수준 · 이 구분이 하드웨어에서 결정적이다. 표로 두면 256바이트 ROM 하나다 — 작고 빠르다. 그런데 표 참조는 마스킹할 수 없다. 부채널 대책으로 값을 난수로 가리려면(x 대신 $x \oplus m$ 을 들고 다니려면), 연산이 그 마스킹과

호환되어야 한다. 표 참조는 호환되지 않는다 — $S(x \oplus m)$ 은 $S(x)$ 와 아무 관계가 없다. 그래서 역원을 산술로 다시 쓴다.



같은 함수 — 다른 회로. 표는 못 가리고, 산술은 가릴 수 있다

그림 5. S-box 를 표로 두느냐 산술로 분해하느냐. Canright 의 탑 필드(tower field) 분해는 $GF(2^8)$ 역원을 $GF(2^4)$, $GF(2^2)$ 연산으로 내린다. 느리고 크지만 마스킹할 수 있다.

OpenTitan 의 AES 는 S-box 구현을 여섯 벌 가지고 있다. 파라미터로 고른다.

표 18. OpenTitan AES 의 S-box 구현 여섯 가지 (실측 줄 수)

파일	줄	방식	언제 고르나
aes_sbox_lut.sv	120	평범한 256바이트 표	부채널이 위협이 아닐 때. 가장 작고 빠르다
aes_sbox_canright.sv	70	탑 필드 분해, 마스킹 없음	면적을 줄이고 싶을 때(표보다 작을 수 있다)
aes_sbox_canright_masked.sv	486	탑 필드 + 1차 마스킹	부채널 대책이 필요할 때
aes_sbox_canright_masked_noreuse.sv	450	마스킹 재사용을 막은 판	재사용이 누설을 만든다는 분석 이후의 판
aes_sbox_dom.sv	1,077	Domain-Oriented Masking	가장 강한 대책. 가장 크다
aes_sbox.sv	144	위를 고르는 래퍼	—

같은 함수를 아홉 배 크기로 다시 쓰는 것 이 보안 IP 의 현실이다. aes_sbox_lut.sv 120줄 대 aes_sbox_dom.sv 1,077줄. 수학적으로 둘은 완전히 같은 함수다. 모델은 둘을 구분하지 않는다 — 모델은 부채널을 검증할 수 없다. 그것은 별도의 평가(TVLA · 전력 측정)로 간다. 모델링 팀이 이 경계를 알고 있어야 한다: “회귀 초록 = 안전” 이 아니다.

2.2 ShiftRows 와 MixColumns

ShiftRows 는 상태 행렬의 r 번째 행을 왼쪽으로 r 바이트 순환시킨다. 하드웨어에서는 배선만 바뀐다 — 논리 게이트가 0개다. 모델에서는 루프가 도니 비용이 있어 보이지만, 실제로는 공짜다. 모델의 줄 수가 하드웨어 비용을 말해 주지 않는 좋은 예다.

MixColumns 는 각 열을 $GF(2^8)$ 위의 고정 다항식과 곱한다.

$$c(x) = 03 \cdot x^3 + 01 \cdot x^2 + 01 \cdot x + 02 \pmod{x^4 + 1}$$

kokke_tiny-AES-c/aes.c:300-313 - MixColumns 골든

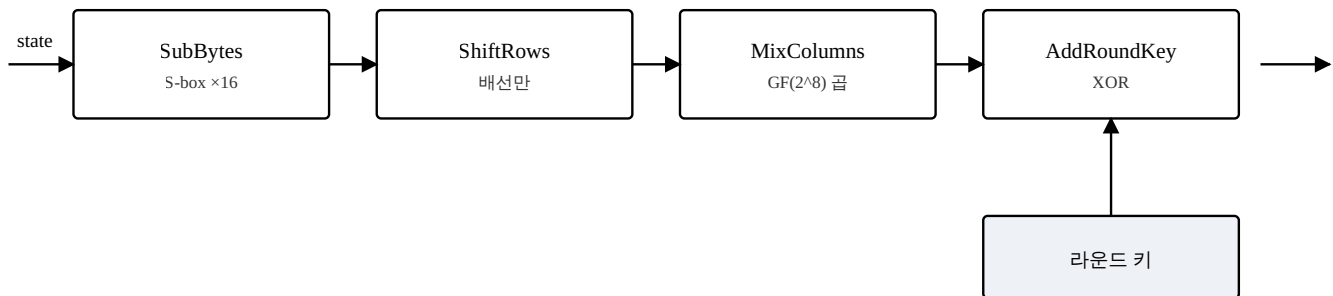
```

300 static void MixColumns(state_t* state)
301 {
302     uint8_t i;
303     uint8_t Tmp, Tm, t;
304     for (i = 0; i < 4; ++i)
305     {
306         t = (*state)[i][0];
307         Tmp = (*state)[i][0] ^ (*state)[i][1] ^ (*state)[i][2] ^ (*state)[i][3] ;
308         Tm = (*state)[i][0] ^ (*state)[i][1] ; Tm = xtime(Tm); (*state)[i][0] ^= Tm ^ Tmp ;
309         Tm = (*state)[i][1] ^ (*state)[i][2] ; Tm = xtime(Tm); (*state)[i][1] ^= Tm ^ Tmp ;
310         Tm = (*state)[i][2] ^ (*state)[i][3] ; Tm = xtime(Tm); (*state)[i][2] ^= Tm ^ Tmp ;
311         Tm = (*state)[i][3] ^ t ; Tm = xtime(Tm); (*state)[i][3] ^= Tm ^ Tmp ;
312     }
313 }

```

석사 수준 · 계수가 **02** 와 **03** 뿐인 것이 설계다. 02 곱은 xtime 한 번, 03 곱은 xtime 한 번 + XOR 한 번이다. 그래서 곱셈기가 필요 없다 — 시프트와 **XOR** 만으로 끝난다. 위 골든 코드가 Tmp(열 전체 XOR)를 한 번 계산해 네 번 재 사용하는 것도 같은 이유다. AES 는 처음부터 하드웨어를 보고 설계된 암호 이고, 이것이 Rijndael 이 표준으로 뽑힌 이유 중 하나다.

면적은 여기서 갈린다: SubBytes 만 조합논리가 아니다



라운드 10회(128비트 키). 마지막 라운드는 MixColumns 없음

그림 6. AES 한 라운드. 네 단계 중 논리 게이트를 쓰는 것은 **SubBytes** 와 **MixColumns** 뿐이다. ShiftRows 는 배선, AddRoundKey 는 XOR 128개.

3. 세 구현을 나란히 읽는다

표 19. 같은 AES, 세 팀, 세 언어

	C 골든모델	HLS C++	양산 RTL
저장소	kokke/tiny-AES-c	AMD Vitis Security Library	lowRISC/opentitan
줄 수	573	1,011	17,254
쓰는 팀	모델링 · DV	FPGA 가속 설계	디자인
S-box	static const uint8_t sbox[256]	#pragma HLS resource ... ROM_nP_LUTRAM	구현 6가지 중 선택
병렬성	없음(순차)	UNROLL 15 · PIPELINE 8	라운드당 1사이클 ~ 완전 언롤까지 파라미터
부채널	고려 안 함	고려 안 함	DOM 마스크 · PRNG

	C 골든모델	HLS C++	양산 RTL
고장주입	고려 안 함	고려 안 함	FSM 이중화 · 상보 논리
버스	함수 인자	hls::stream/AXI	TL-UL + 새도우 레지스터
검증	NIST 벡터	C 시뮬레이션	UVM + 형식 + 부채널 평가

모델링 팀 관점의 요약. 왼쪽 열이 당신의 산출물이다. 573줄이지만 오른쪽 두 열의 정답을 정의한다. 그래서 왼쪽 열에서 실수하면 아무도 못 잡는다 — 오른쪽이 왼쪽에 맞춰지기 때문이다. 이것이 3장에서 말한 모델 자체의 검증이 필요한 이유다. AES 는 다행히 NIST 벡터가 있어 외부 정답이 존재한다. 사내 알고리즘에는 그것이 없다.

4. 물리 — RTL 에만 있는 세계

4.1 부채널: 전력이 비밀을 말한다

학부 복습 · CMOS 회로는 비트가 바뀔 때 전류를 쓴다. 그래서 소비 전력이 내부 값에 의존한다. 공격자가 칩의 전원선에 저항 하나를 달고 수천 번의 암호화를 측정하면, 통계로 키 바이트를 하나씩 복원할 수 있다 — 차분 전력 분석(DPA).

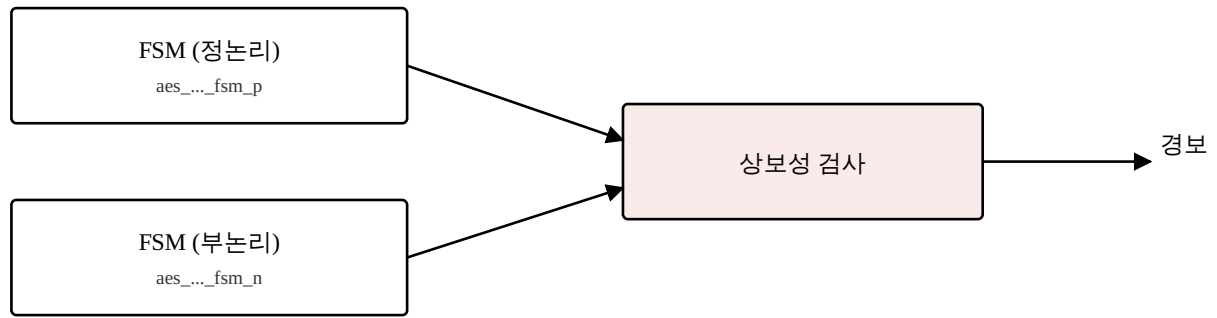
석사 수준 · 마스킹의 원리. 비밀 x 를 직접 들고 다니지 않고, 난수 m 으로 쪼개 ($x \oplus m, m$) 두 몫으로 들고 다닌다. 각 몫은 x 와 통계적으로 독립이므로 1차 누설이 사라진다. 문제는 비선형 연산이다. XOR 은 몫별로 따로 해도 되지만, 곱셈은 몫을 섞어야 하고 그 과정에서 중간값이 잠깐 x 에 의존할 수 있다(글리치). **Domain-Oriented Masking(DOM)** 은 몫을 ‘도메인’으로 나누고 도메인을 넘는 곱에 새 난수를 더한 뒤 레지스터로 끊어 글리치 전파를 막는다. aes_sbox_dom.sv 가 1,077줄인 이유고, aes_prng_masking.sv 가 필요한 이유다 — 마스킹은 난수를 계속 먹는다.

표 20. 부채널 대책의 대가 (OpenTitan AES 실측 줄 수 기준)

항목	LUT S-box	DOM S-box	배수
줄 수	120	1,077	9.0배
난수 소모	없음	S-box 호출마다	—
지연	조합논리 1단	레지스터 여러 단	파이프라인 깊어짐
검증	NIST 벡터로 충분	+ TVLA 전력 평가 필요	—

4.2 고장 주입: 레이저와 전압 글리치

학부 복습 · 공격자가 칩에 레이저를 쏘거나 전원을 순간적으로 흔들어 플립플롭 하나를 뒤집으면, 라운드 카운터가 건너뛰어 암호화가 덜 된 상태로 출력될 수 있다. 그런 출력 한두 개면 키가 복원된다(차분 고장 분석, DFA).



고장이 양쪽을 같은 방향으로 뒤집으면 상보성이 깨진다 — 그때 잡는다
 합성기가 인버터를 FSM 안으로 밀어 넣으므로 두 벌이 같은 회로가 되지 않는다

그림 7. OpenTitan 의 제어 FSM 이중화. 같은 FSM 을 정논리와 부논리로 두 벌 돌리고 상보성을 검사한다.

```

lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv:7-13  - 왜 부논리 판을 따로 두는가 -- RTL 주석 원
문
7 // This module contains the AES cipher core control FSM operating on and producing the negated
8 // values of important control signals. This is achieved by:
9 // - instantiating the regular AES cipher core control FSM operating on and producing the
positive
10 //   values of these signals, and
11 // - inverting these signals between the regular FSM and the prim_buf synthesis barriers.
12 // Synthesis tools will then push the inverters into the actual FSM.
13
  
```

석사 수준 · **prim_buf** 합성 장벽에 주목하라. 인버터를 그냥 넣으면 합성기가 “!!x = x” 라며 없애 버리고 두 FSM 이 같은 회로로 병합된다. 그러면 이중화가 사라지는데 기능 검증은 통과한다. 그래서 합성기가 넘지 못하는 장벽 을 세운다. 이것은 **RTL** 코드가 아니라 합성 제약의 문제이고, 모델에는 대응물이 전혀 없다. 모델링 팀이 이 영역을 ‘내 일이 아니다’라고 알아 두는 것이 중요하다 — 모르면 왜 RTL 이 그렇게 이상하게 생겼는지 설명이 안 된다.

4.3 새도우 레지스터와 키 삭제

표 21. 양산 보안 IP 에만 있는 블록들 (OpenTitan AES)

파일	줄	하는 일	모델에 대응물이
aes_ctrl_reg_shadowed.sv	—	제어 레지스터를 두 벌 두고 두 번 써야 반영. 글리치로 설정이 바 뀌는 것을 막는다	없음
aes_prng_clearing.sv	—	키·상태를 0 이 아니라 난수로 덮는다. 0 으로 덮으면 잔류 전하로 읽힐 수 있다	없음
aes_prng_masking.sv	151	마스킹용 난수 공급	없음
aes_reg_top.sv	—	TL-UL 버스 → 레지스터맵	함수 인자
aes_ghash.sv	—	GCM 모드의 GF(2 ¹²⁸) 곱	모드는 대개 소프 트웨어

5. 검증 — 이 블록을 어떻게 맞다고 말하나

표 22. AES 검증 총위

층위	무엇으로	무엇을 잡나	못 잡는 것
NIST 시험 벡터 (KAT)	FIPS-197 부록 · NIST CAVP	기능 오류. 여기서 틀리면 나머지는 볼 필요 없다	경계 조건 · 상태 전이
제약 랜덤 + 골든 비교	UVM + C 모델	모드 전환 · 키 변경 · 백투백 · 중단	부채널 · 고장
형식 검증	속성(assertion) 증명	FSM 도달 불가 상태 · 데드락 · 레지스터 접근 규칙	데이터패스 전체(상태공간이 너무 크다)
부채널 평가	TVLA · 전력 측정	마스킹 결함 · 글리치 누설	기능 오류
고장 주입 시뮬	게이트 레벨 플립 주입	이중화가 실제로 잡는지	물리적 레이저 특성

모델링 팀이 담당하는 것은 위 두 층이다. NIST 벡터를 모델에 먹여 통과시키고, 그 모델을 DV 가 RTL 과 맞댄다. 아래 세 층은 각각 형식검증팀 · 보안평가팀 · DFT/신뢰성팀이 본다. 그런데 “우리 AES 는 검증됐다”는 말이 어느 층을 뜻하는지 늘 물어야 한다.

6. 이 블록에서 가져갈 일반 원리

표 23. AES 에서 배운 것이 다른 블록에도 적용된다

원리	AES 에서	다른 곳에서
정답은 외부에서 온다	NIST KAT	IEEE 규격 벡터 · 표준 적합성 시험
표는 정의가 아니라 결과다	S-box = 역원 + 아핀	모든 LUT 는 원래 계산이 있다. 그 계산이 최적화의 여지다
같은 함수, 여러 회로	S-box 6가지	곱셈기 · 나눗셈기 · CORDIC · 정렬망
모델이 못 보는 층이 있다	부채널 · 고장	타이밍 · 전력 · EMI · 노화. 기능 회귀 초록 ≠ 출하 가능
파라미터가 곧 제품군이다	키 길이 · S-box 선택 · 언롤	하나의 소스로 여러 PPA 점을 판다. 이것이 IP 비즈니스다

AXI 인터커넥트 — 모든 SoC 의 등뼈

표준 Arm AMBA AXI4 / AXI4-Lite / AXI4-Stream • 이 교안의 실물 `pulp-platform/axi` 37,589줄 • 형식 검증된 구현 `ZipCPU/wb2axip` 71,857줄

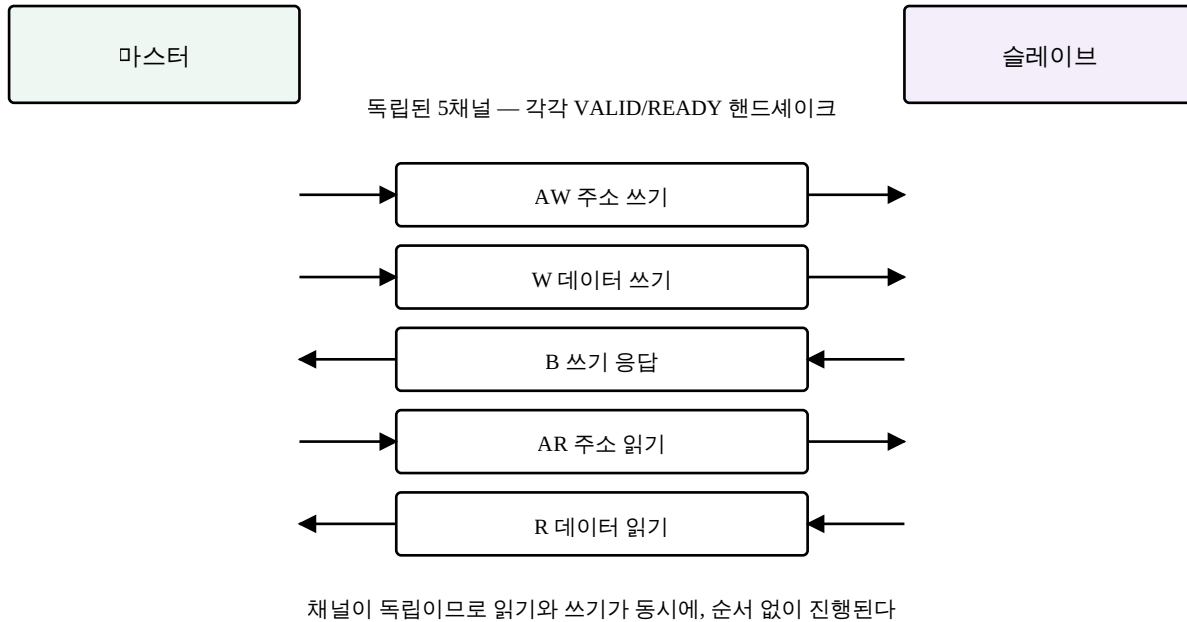


그림 8. AXI4 의 다섯 채널. 각 채널이 독립이고 VALID/READY 로만 흐른다.

학부 복습 · 핸드셰이크 규칙이 전부다. 보내는 쪽이 VALID, 받는 쪽이 READY. 둘 다 1인 클럭 엣지에 한 박자가 넘어 간다. 한 번 올린 **VALID** 는 전송이 끝날 때까지 내리면 안 된다 — 이 규칙 하나를 어기면 프로토콜 위반이고, 검사기가 잡는다.

표 24. AXI 에서 모델링 팀이 꼭 알아야 할 것

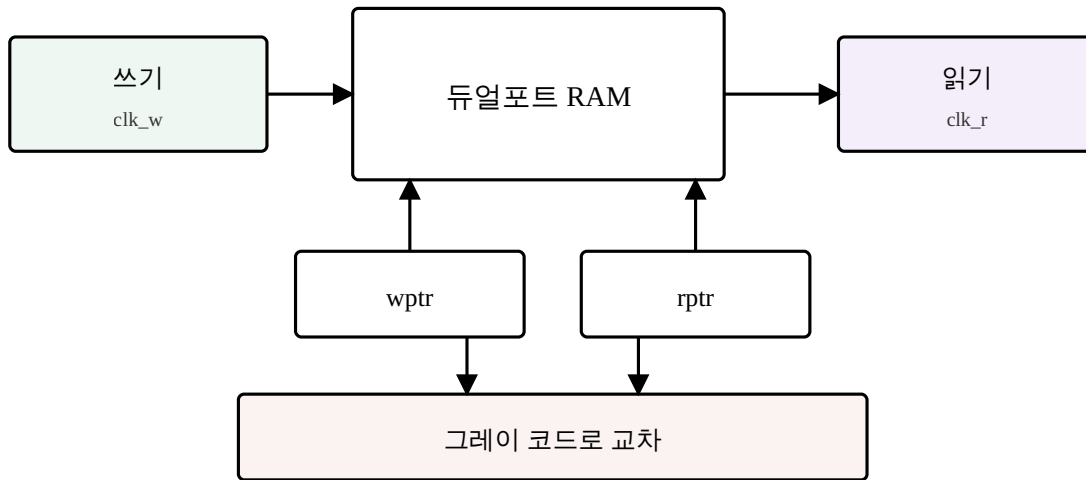
개념	뜻	왜 모델에 영향을 주나
버스트(AWLEN/ARLEN)	한 주소로 여러 박자	메모리 모델이 버스트 경계를 알아야 한다
ID(AWID/ARID)	여러 트랜잭션을 구분	같은 ID 는 순서 보장, 다른 ID 는 순서 없음 — 스코어보드 설계가 갈린다
아웃오브오더	응답이 요청 순서와 다를 수 있다	모델이 '순서대로'를 가정하면 틀린다
4KB 경계	버스트는 4KB 를 넘을 수 없다	자극 생성 제약
WSTRB	바이트 단위 쓰기 마스크	부분 쓰기를 모델이 처리해야 한다
응답 코드	OKAY · EXOKAY · SLVERR · DECERR	오류 경로도 명세의 일부다
배타 접근	load-reserved / store-conditional 용	모델링이 까다롭다. 모니터가 상태를 들고 있어야 한다

석사 수준 · **ID** 와 순서가 검증의 핵심이다. AXI 는 같은 **ID** 안에서만 순서를 보장한다. 그래서 스코어보드는 “큐 하나”가 아니라 **ID**별 큐여야 한다. 이것을 모르고 단일 큐로 만들면, 정상 동작을 오류로 보고하거나(거짓 실패) 진짜 순서 위반을 놓친다(거짓 통과). 모델링 팀이 이 구조를 **DV** 에 알려 주는 것이 흔한 협업 지점이다.

ZipCPU/wb2axip 는 단언이 1,772개 들어 있는 형식 검증된 AXI/Wishbone 브리지다. 프로토콜 속성을 어떻게 적는지 보려면 여기가 가장 좋은 교재다.

FIFO · CDC 기본 셀 — 가장 많이 쓰이고 가장 많이 틀린다

이 교안의 실물 `pulp-platform/common_cells` 25,535줄 — `fifo_v3`, `cdc_2phase`, `cdc_fifo_gray`, `rr_arb_tree`, `stream_*` 등



그레이 코드는 한 번에 한 비트만 바뀐다 — 동기화 중 잘못 읽혀도 ± 1 이다
 ± 1 은 *full/empty* 를 보수적으로 만들 뿐 데이터를 깨지 않는다

그림 9. 비동기 FIFO. 포인터를 그레이 코드로 바꿔 교차시키는 것이 핵심이다.

석사 수준 · 왜 그레이 코드인가. 이진 카운터 7→8 은 0111→1000 으로 네 비트가 동시에 바뀐다. 다른 클럭에서 이 걸 샘플하면 비트마다 도착 시점이 달라 1111 같은 존재한 적 없는 값을 읽을 수 있다. 그레이 코드는 인접값이 1비트만 다르므로, 최악의 경우에도 “이전 값 또는 다음 값”을 읽는다. 그 오차는 *full/empty* 판정을 보수적으로 만들 뿐이다 — 실제보다 짝 찼다고 보거나 비었다고 볼 뿐, 데이터를 깨지 않는다. 이것이 안전한 설계의 전형이다: 틀리더라도 안전한 방향으로 틀리게 만든다.

표 25. FIFO 관련 실무 함정

함정	증상	대책
깊이가 버스트를 못 받는다	생산자가 스톱 → 전체 파이프라인 정지	최악 버스트 + 지연을 계산해 깊이를 정한다
<i>full/empty</i> 를 한 클럭 늦게 본다	오버런/언더런	거의 참(<i>almost_full</i>) 신호를 쓴다
2단 동기화기를 다중비트에	값이 깨진다	그레이 코드 또는 핸드셰이크
리셋이 두 도메인에서 비대칭	한쪽만 초기화돼 포인터가 어긋남	리셋도 CDC 대상이다
깊이 1 FIFO 를 레지스터로 대체	백프레셔 타이밍이 달라짐	<code>stream_register</code> 같은 검증된 셀을 쓴다

모델링 팀 관점. 기능 모델에는 FIFO 깊이가 없다 — 무한 큐로 둔다. 그래서 깊이 부족으로 인한 성능 저하나 데드락은 모델이 절대 못 잡는다. 이것은 성능 모델(SystemC TLM 또는 사이클 근사 모델)의 영역이다. 두 모델을 구분해 두는 것이 중요하다.

RISC-V 코어 — 모델링 팀의 주무대

표 26. 이 교안에 들어 있는 RISC-V 구현들

코어	줄 수	성격	배울 점
openhwgroup/cva6	409,066	64비트 응용 프로세서. 리눅스 부팅. 다수 테이 프아웃	MMU · 캐시 · 분기예측이 있는 진짜 코어
openhwgroup/cv32e40p	48,907	32비트 임베디드. PULP 계열	step-compare 검증의 예제
YosysHQ/picorv32	18,406	초소형. FPGA 에 널리 쓰임	가장 읽기 쉬운 RISC-V RTL
olofk/serv	8,120	비트 직렬 세계 최소 RISC-V	극단적 면적 최적화의 사례
chipsalliance/rocket-chip	58,475	Chisel(Scala) 생성기. SiFive 계보	RTL 을 생성하는 방식

석사 수준 · **Rocket** 이 **Chisel** 인 것이 중요하다. RTL 을 직접 쓰는 대신 프로그램이 **RTL** 을 만든다. 파라미터를 바꾸면 다른 코어가 나온다. IP 비즈니스의 본질(하나의 소스로 여러 PPA 점을 판다)이 언어 차원에 들어간 것이다. 단점은 생성된 Verilog 가 사람이 읽기 어렵다는 것 — 디버그와 검증이 까다로워진다. 실무에서 이 맞교환은 여전히 논쟁 중이다.

코어 모델링에서 반드시 정해야 하는 것

표 27. ISA 모델의 설계 결정

결정	선택지	영향
명령어 표현	파일 하나에 하나(Spike) vs 거대 switch	확장성. Spike 방식이 옳다
디코더	표 기반 vs 조건 트리	속도와 유지보수
메모리	평면 배열 vs 페이지 테이블 vs MMU 완전 모델	권한/가상메모리 검증 여부가 갈린다
예외 우선순위	규격이 정한 순서를 그대로	여기가 자주 틀린다. 동시 발생 시 순서
인터럽트 주입	DV 가 시점을 준다 vs 모델이 정한다	전자라야 RTL 과 맞출 수 있다
카운터(mcycle 등)	모델링하지 않는다	사이클 개념이 없다
미정의 동작	명시적 표시 + DV 마스킹	3장의 (다)

예외 우선순위가 모델링의 최대 함정이다. 한 명령어가 동시에 여러 예외 조건을 만족할 수 있다(정렬 오류 + 페이지 폴트 + 권한 위반). 규격은 우선순위 표를 준다. 모델이 그 순서를 잘못 구현하면 평소에는 안 보이다가 특정 조합에서만 터진다. 첫 주에 규격의 예외 우선순위 표를 찾아 모델과 대조하라.

RS-FEC — 통신 IP 의 필수 블록

표준 IEEE 802.3 Clause 91 (KP4) • 부호 RS(544,514) over GF(2^{10}), $t=15$ 심볼 • 쓰임 100G 이상 이더넷 필수

학부 복습 • 왜 FEC 가 필수가 됐나. 옛날 링크는 BER 10^{-12} 를 물리계층이 직접 맞췄다. 100G/lane 이 되면 그것이 불가능해져, 슬라이서는 10^{-4} 수준으로 내고 FEC 가 10^{-15} 로 내린다는 분업으로 바뀌었다. 그래서 FEC 는 선택이 아니라 규격이 강제하는 블록이다.

표 28. RS 복호기의 네 단계 — 모델과 RTL 이 같은 구조를 쓴다

단계	하는 일	수학	하드웨어 비용
신드롬	$S_i = r(\alpha^{fcrt+i}), i=0..2t-1$	다항식 평가 $2t$ 회	GF 곱셈기 $2t$ 개 (병렬)
Berlekamp-Massey	오류위치다항식 $\sigma(x)$ 를 찾는다	최단 LFSR 합성	반복 $2t$ 회. 되먹임 고리 — 임계경로
Chien 탐색	$\sigma(x)$ 의 근 = 오류 위치	n 개 원소 전수 대입	n /병렬도 사이클
Forney	오류 크기 계산	$\omega(x)/\sigma'(x)$	GF 나눗셈(역원 테이블)

석사 수준 • BM 단계가 아키텍처를 정한다. 신드롬과 Chien 은 전수 병렬화가 쉽다(독립 계산). BM 은 반복마다 이전 결과가 필요한 되먹임 이라 병렬화가 어렵다. 그래서 고속 RS 복호기는 *inversionless* BM(역원 제거), *reformulated* BM(임계경로 단축) 같은 변형을 쓴다. 모델에서는 어느 판을 써도 결과가 같다 — 그래서 모델은 가장 읽기 쉬운 판으로 쓰고, RTL 이 다른 판을 쓰더라도 비교는 성립한다. 이것이 모델과 RTL 이 ‘같은 알고리즘’일 필요가 없다는 좋은 예다. 같은 함수이면 된다.

검사가 반드시 확인해야 하는 성질

표 29. RS 복호기 검사 설계

넣는 오류	기대	이 검사가 없으면
0개	순대지 않는다	이것만 하면 검사가 아니다 — 복호기가 놀아도 통과
1 ~ t 개	전부 정정, 오정정 0	정정 능력 미검증
$t+1$ 개 이상	원본 복원은 불가능	‘전부 검출된다’고 쓰면 과장이다 — 일부는 다른 코드워드로 간다
경계(t 와 $t+1$)	정확히 t 에서 갈린다	off-by-one
정정 후 재검사	신드롬이 0 인지 다시 쟀다	오정정을 성공으로 보고한다

“ $t+1$ 개는 전부 검출된다”는 틀린 문장이다. RS 는 t 개까지 정정하고 그 이상은 검출하거나 오정정한다. 저자의 실측: RS(15,9), $t=3$ 에서 오류 4개를 120회 넣으니 검출 117 • 오정정 3 이었다. IP 사양서에 이 확률을 적는 것이 정직한 문서다.

SoC 하나를 만들려면 어떤 IP가 필요한가

“칩을 만든다”는 말은 실무에서 **IP**를 고르고 붙이고 검증한다는 뜻이다. 자체 설계하는 것은 차별화 블록 한둘이고 나머지는 산다. 아래는 일반적인 SoC의 구성이고, 어느 것이 사는 것이고 어느 것이 만드는 것인지 를 같이 적는다.

표 30. SoC 필수 IP 목록과 조달 방식

계층	IP	보통 어떻게	이 교안의 실물
연산	CPU 코어 (RISC-V · Arm)	라이선스(Arm) 또는 오픈(RISC-V)	cva6 · cv32e40p · rocket-chip
	DSP · NPU · GPU	라이선스 또는 자체	finn_hlslib(QNN)
메모리	DDR/LPDDR 컨트롤러 + PHY	반드시 산다(PHY는 하드매크로)	—
	SRAM 컴파일러	파운드리/메모리벤더	—
	캐시 · 스크래치패드	자체 또는 코어에 포함	cva6 내부
연결	버스/NoC (AXI · TileLink · AHB · APB)	오픈 또는 라이선스	pulp-platform/axi 37,589줄
	기본 셀(FIFO · CDC · 아비터)	자체 라이브러리	common_cells 25,535줄
입출력	PCIe · Ethernet · USB · MIPI · SerDes PHY	PHY 는 반드시 산다 · 컨트롤러는 선택	verilog-pcie · corundum
	SPI · I2C · I3C · UART · GPIO	오픈/자체 · 싸다	OpenTitan에 전부
보안	RoT · AES · SHA · RSA/ECC · TRNG · OTP · 키관리	산업 표준 요구(OCP · FIPS)	opentitan · caliptra-rtl
기반	PLL · 클럭생성 · 리셋 · 전력관리(PMU)	파운드리 IP	—
	디버그(JTAG · 트레이스)	표준 필수	pulp-platform/riscv-dbg
	DFT(스캔 · MBIST · BSD)	EDA 삽입	—
	eFuse · 온도/전압 센서	파운드리	—

석사 수준 · “**PHY**는 산다”가 반복되는 이유. PHY는 아날로그/혼성신호이고 공정에 강하게 묶인다. 7nm PHY를 5nm로 옮기는 것은 재설계에 가깝다. 그래서 파운드리·전문업체가 공정별로 하드 매크로(GDS)로 판다. 디지털 IP(RTL)는 공정 무관이라 이식이 싸다. 이 경계가 **IP**시장의 가격 구조를 만든다 — 하드 매크로가 비싸고, 소프트 IP는 상대적으로 싸다.

칩을 만들 때 꼭 생각해야 하는 것

표 31. 설계 착수 전 점검표 — 놓치면 테이프아웃 뒤에 드러난다

항목	물어야 할 것	놓쳤을 때
클럭 도메인	도메인이 몇 개인가? 모든 교차에 동기화기가 있나?	메타스테이빌리티. 재현 안 되는 간헐 오류
리셋	비동기 assert / 동기 deassert 인가? 리셋 트리 지연은?	리셋 해제 시점이 달라 FSM이 엇갈린다
전력 도메인	꺼지는 블록이 있나? 아이솔레이션 셀 · 리텐션은?	전원이 꺼진 블록의 출력이 X로 퍼진다
DFT	스캔 체인이 모든 FF를 덮나? MBIST가 모든 SRAM을?	양산 테스트 커버리지 부족 → 불량 유출
디버그	칩이 안 뜨면 무엇을 볼 수 있나?	브링업 불가. 가장 비싼 실패

항목	물어야 할 것	놓쳤을 때
레지스터맵	단일 원천에서 RTL·문서·드라이버·UVM을 생성하나?	소프트웨어와 하드웨어가 어긋난다
보안 경계	비밀이 지나는 경로가 어디까지인가? 디버그로 새나?	JTAG로 키가 나온다
타이밍 예외	false path · multicycle을 누가 검토했나?	잘못 선언한 예외 = 검증 안 된 경로
SDC /제약	합성·STA 제약이 RTL 가정과 맞나?	합성은 통과, 실리콘은 실패
IP 통합	산 IP의 가정(클럭비 · 리셋순서 · 초기화)을 지켰나?	벤더는 '문서에 있다'고 답한다
ECO 여유	스페어 셀을 뿌려 뒀나?	금속만 고치는 수정이 불가능해져 풀마스크 재제작
패키지/ IO	핀 수 · SI/PI · ESD가 검토됐나?	동작은 하는데 보드에서 실패

업계 표준 — 무엇을 알고 있어야 하나

IP 를 다룬다는 것은 표준을 다룬다는 뜻이다. 아래는 모델링 팀이 마주치는 순서대로다. 전부 외울 필요는 없고, 무엇이 어디를 규정하는지를 알면 된다.

표 32. 언어 · 방법론 표준

표준	번호	무엇	모델링 팀에서
SystemVerilog	IEEE 1800	설계 + 검증 언어	DPI-C 절을 반드시 읽어라. 모델을 붙이는 통로다
Verilog	IEEE 1364	구형 설계 언어	레거시 IP 에서 만난다
VHDL	IEEE 1076	설계 언어	유럽·항공/방산 IP
SystemC	IEEE 1666	C++ 하드웨어 모델링	가상 플랫폼의 표준. TLM-2.0 포함
UVM	IEEE 1800.2	검증 방법론	DV 가 쓴다. 용어를 알아야 대화가 된다
UPF	IEEE 1801	전력 의도 기술	저전력 설계의 의도를 적는다
IP-XACT	IEEE 1685	IP 메타데이터 · 레지스터맵	레지스터맵 단일 원천의 표준 형식
SDF / SDC	IEEE 1497 / 사실상 표준	지연 · 타이밍 제약	—
PSL / SVA	IEEE 1850 / 1800 내	속성 · 단언	형식 검증의 언어

표 33. 인터페이스 · 프로토콜 표준

영역	표준	핵심 개념
온칩 버스	AMBA AXI4 / AXI4-Lite / AXI4-Stream (Arm)	채널 5개 · 핸드셰이크(VALID/READY) · 버스트 · 아웃오브오더(ID)
	AHB · APB (Arm)	저속 주변장치는 APB
	TileLink (RISC-V 계열)	캐시 일관성까지 규정
	TL-UL (OpenTitan)	TileLink Uncached Lightweight
	CHI (Arm)	일관성 · 다중 코어
칩간	PCIe (PCI-SIG) Gen1~Gen6	PHY · 링크 · 트랜잭션 3계층
	CXL	PCIe 물리계층 위의 메모리 일관성
	UCIe	칩렛 다이-투-다이
네트워크	IEEE 802.3 Ethernet	Clause 별 구성: 49(64b/66b PCS) · 91(RS-FEC) · 161(802.3ck)
메모리	JEDEC DDR4/5 · LPDDR5 · HBM	타이밍 파라미터가 곧 규격
저속	I2C · I3C(MIPI) · SPI · UART	가장 자주 쓰는 주변장치
디버그	IEEE 1149.1 JTAG · RISC-V Debug Spec	브링업의 생명선

표 34. 암호 · 보안 표준

표준	무엇	검증에서
FIPS-197	AES	시험 벡터가 부록에 있다
FIPS-180-4	SHA-1/2	—
FIPS-202	SHA-3 / SHAKE (Keccak)	OpenTitan kmac
FIPS-186	DSA/ECDSA	—
FIPS-203/204/205	양자내성(ML-KEM · ML-DSA · SLH-DSA)	Caliptra 에 ML-DSA 구현이 있다

표준	무엇	검증에서
NIST SP 800-90A/B/C	난수 생성(DRBG · 엔트로피)	OpenTitan csrng · entropy_src
NIST CAVP	암호 알고리즘 검증 프로그램	벡터의 출처
ISO/IEC 17825	부채널 시험	TVLA 방법론
OCP Caliptra	데이터센터 RoT 사양	이 교안에 RTL 이 들어 있다

실무 팁. IEEE 802 계열 규격은 **“Get IEEE 802”** 프로그램으로 무료다. PCI-SIG · JEDEC 는 회원사만 본다(회사가 회원이면 사내 포털에 있다). NIST FIPS 는 전부 무료다. 첫 주에 당신 블록의 규격 원문을 확보하라. 남의 요약으로 모델을 쓰면 그 요약의 오해가 칩에 들어간다.

논문 활용법 — 실무에서 어떻게 쓰나

모델링 팀에서 논문은 읽는 대상이 아니라 도구다. 쓰임새가 분명하다.

표 35. 실무에서 논문이 쓰이는 네 가지 자리

쓰임	무엇을 찾나	어디서	주의
구조 선택	같은 기능의 여러 회로 중 무엇이 왜 나온가	JSSC · ISSCC · VLSI · DAC/ICCAD	논문의 수는 가장 유리한 조건에서 나온다. 조건을 먼저 읽어라
알고리즘 근거	왜 이 수식이 맞는가, 수렴 조건은	IEEE Trans. · arXiv	구현 논문과 이론 논문을 구분하라
공격/취약점	우리 대책이 이미 깨졌나	CHES · CCS · USENIX Security	보안 IP 는 필수
표준의 배경	규격이 왜 그렇게 정해졌나	표준화 기구 기고문(IEEE 802 public 디렉터리 등)	규격 본문보다 기고문이 더 잘 설명한다

논문을 읽는 순서 (30분 안에 판단하기)

- 1. 표/그림부터 본다. 무엇을 했는지가 거기 있다
- 2. 실험 조건을 찾는다. 공정 · 전압 · 온도 · 채널 · 데이터셋. 여기가 비교 가능성을 정한다
- 3. 기준선(baseline)이 무엇인지 본다. 약한 기준선과 비교했으면 수가 커 보인다
- 4. 그제서야 방법을 읽는다
- 5. 마지막에 한계/future work 를 읽는다. 저자가 아는 약점이 거기 있다

실무에서 가장 위험한 인용은 “읽지 않고 쓴 인용”이다. 초록과 검색 조각만 보고 “이 방식이 좋다”고 보고하면, 조건이 우리와 다를 때 몇 달을 버린다. 확인 수준을 표시하는 습관을 권한다 — 전문을 읽었나(전문) · 초록만인가(초록) · 검색 조각인가(조각). 이 교안 저자의 저장소는 문서에 그 표시를 강제한다.

어디서 찾나 (접근 가능성 순)

표 36. 문헌 접근 경로

경로	무엇	비용
arXiv	프리프린트. 통신·ML·아키텍처가 많다	무료
표준화 기구 공개 디렉터리	IEEE 802 기고문 등. 보물창고	무료
Get IEEE 802	802 계열 규격 원문	무료
NIST 간행물	FIPS · SP 800 시리즈	무료
IACR ePrint	암호 논문 전량	무료
IEEE Xplore · ACM DL	JSSC · ISSCC · TC 등	회사/학교 구독
회사 내부 위키 · 이전 프로젝트 문서	가장 관련성 높다	무료
벤더 IP 사양서(PG/PB/데이터시트)	실제 납품물의 형태	공개된 것이 많다

IP 아이디어 — 무엇을 제안할 것인가

“IP 아이디어를 생각해 오라”는 과제에 대한 실무적 접근. 새 알고리즘을 제안하는 자리가 아니다. 디자인 하우스가 파는 것은 알고리즘이 아니라 **PPA** · 인터페이스 · 검증 · 납품 패키지다.

판단 기준 다섯. (1) 표준이 있어 정답이 외부에 있는가 (2) 수요가 구조적인가(규격이 강제하는가) (3) 우리가 가진 공정/도구로 만들 수 있는가 (4) 특허가 데이터패스를 막고 있지 않은가 (5) 기존 공개 구현이 있는가 — 있으면 기준선이 생기는 것이라 오히려 유리하다.

표 37. 아이디어 후보 — 표준이 수요를 강제하는 것들

#	IP	왜 수요가 있나	난이도	정답의 출처
1	RS-FEC KP4 RS(544,514)/GF(2 ¹⁰)	100G 이상 이더넷이 규격으로 강제 . 800G·1.6T 로 계속 간다	중	IEEE 802.3 Clause 91/161
2	연접 FEC (외부 RS-KP4 + 내부 Hamming(128,120))	802.3dj(224G/lane)가 2023~24 채택. 공개 구현이 안 보인다	중상	802.3dj 기고문
3	100G+ PCS (멀티레인 분배 · AM · gearbox)	상용은 'soft PCS' 로 판다. 오픈에는 10G 까지만 있다	중	IEEE 802.3
4	양자내성 암호(ML-KEM/ML-DSA) 가속기	FIPS 203/204 확정. 모든 RoT 가 넣어야 한다	상	FIPS 203/204 + Caliptra 참조
5	TRNG + DRBG (SP 800-90A/B)	보안칩 필수. 인증이 까다로워 진입장벽	중상	NIST SP 800-90 · OpenTitan
6	AXI/CHI 프로토콜 검사기(형식 속성 포함)	모든 SoC 가 필요. 재사용성 최고	중	AMBA 규격
7	CDC /리셋 검증 IP	간헐 버그의 최대 원인	중	사내 규칙
8	레지스터맵 단일원천 생성기 (IP-XACT → RTL/ UVM/문서/드라이버)	모든 팀이 자체 제작해 쓴다. 표준화 여지	중	IEEE 1685
9	ECC 가 붙은 SRAM 래퍼(SECDED · 스크러빙)	안전/보안 요구가 강제(ISO 26262 · FIPS)	하중	Hamming/Hsiao 코드
10	MIPI I3C 컨트롤러	센서 인터페이스 세대교체 중	중	MIPI 규격(회원)

제안서에 반드시 들어가야 하는 것

표 38. IP 제안서 뼈대

절	내용	빠지면
1. 수요	어느 규격/제품이 이것을 강제하는가	'있으면 좋은 것'이 된다
2. 선행	상용 · 오픈 구현을 이름과 규모로. 없다고 쓰지 말 것	이미 있는 것을 만들게 된다
3. 차별	PPA · 파라미터 범위 · 인터페이스 · 검증 패키지 중 무엇으로	'더 좋다'만 남는다
4. 규격 접근	원문을 구할 수 있는가	착수 불가
5. 특허	데이터패스 특허를 조사했는가	팔 수 없는 물건이 된다
6. 검증 계획	정답이 어디서 오는가. 골든 모델은 무엇인가	'돌아간다'에서 멈춘다
7. 규모 추정	기준선과 비교한 줄 수 · 기간 · 인원	일정이 근거 없이 잡힌다
8. 실패 조건	무엇이 참이면 이 제안이 무효인가	무효화 조건이 없는 주장은 광고다

석사 수준 · 8번이 제안서의 질을 가른다. “이 조건에서는 우리가 진다”를 먼저 적은 제안은 신뢰를 얻는다. 예: “MATLAB SerDes Toolbox + HDL Coder 가 데이터패스까지 RTL 을 생성한다면 이 제안의 3절은 무효다” — 이렇게 적고 실제로 확인하는 것이 실무다.

부록 A 본문이 인용한 소스 전문

본문이 가리킨 파일을 전문 그대로 실는다. 줄번호는 파일의 실제 줄번호이므로 본문의 파일:시작-끝 참조를 그대로 따라갈 수 있다.

A.1 kokke_tiny-AES-c/aes.c (572줄, 19,015B)

```

1 /*
2
3 This is an implementation of the AES algorithm, specifically
ECB, CTR and CBC mode.
4 Block size can be chosen in aes.h - available choices are
AES128, AES192, AES256.
5
6 The implementation is verified against the test vectors in:
7 National Institute of Standards and Technology Special
Publication 800-38A 2001 ED
8
9 ECB-AES128
10 -----
11
12 plain-text:
13 6bc1bee22e409f96e93d7e117393172a
14 ae2d8a571e03ac9c9eb76fac45af8e51
15 30c81c46a35ce411e5fbc1191a0a52ef
16 f69f2445df4f9b17ad2b417be66c3710
17
18 key:
19 2b7e15162aed2a6abf7158809cf4f3c
20
21 resulting cipher
22 3ad77bb40d7a3660a89ecaf32466ef97
23 f5d3d58503b9699de785895a96fdbaa
24 43b1cd7f598ece23881b00e3ed030688
25 7b0c785e27e8ad3f8223207104725dd4
26
27
28 NOTE: String length must be evenly divisible by 16byte
(str_len % 16 == 0)
29 You should pad the end of the string with zeros if
this is not the case.
30 For AES192/256 the key size is proportionally larger.
31
32 */
33
34
35 /
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

```

```

66
67
68 /
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549

```

```

114 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb,
0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61,
115 0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69,
0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d };
116 #endif
117
118 // The round constant word array, Rcon[i], contains the values
given by
119 // x to the power (i-1) being powers of x (x is denoted as
{02}) in the field GF(2^8)
120 static const uint8_t Rcon[11] = {
121 0x8d, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b,
0x36 };
122
123 /*
124 * Jordan Goulder points out in PR #12 (https://github.com/
kokke/tiny-AES-C/pull/12),
125 * that you can remove most of the elements in the Rcon array,
because they are unused.
126 *
127 * From Wikipedia's article on the Rijndael key schedule @
https://en.wikipedia.org/wiki/Rijndael_key_schedule#Rcon
128 *
129 * "Only the first some of these constants are actually used -
up to rcon[10] for AES-128 (as 11 round keys are needed),
130 * up to rcon[8] for AES-192, up to rcon[7] for AES-256.
rcon[0] is not used in AES algorithm."
131 */
132
133
134 /
*****
*****/
135 /* Private
functions: */
136 /
*****
*****/
137 /*
138 static uint8_t getSBoxValue(uint8_t num)
139 {
140     return sbox[num];
141 }
142 */
143 #define getSBoxValue(num) (sbox[(num)])
144
145 // This function produces Nb(Nr+1) round keys. The round keys
are used in each round to decrypt the states.
146 static void KeyExpansion(uint8_t* RoundKey, const uint8_t*
Key)
147 {
148     unsigned i, j, k;
149     uint8_t tempa[4]; // Used for the column/row operations
150
151     // The first round key is the key itself.
152     for (i = 0; i < Nk; ++i)
153     {
154         RoundKey[(i * 4) + 0] = Key[(i * 4) + 0];
155         RoundKey[(i * 4) + 1] = Key[(i * 4) + 1];
156         RoundKey[(i * 4) + 2] = Key[(i * 4) + 2];
157         RoundKey[(i * 4) + 3] = Key[(i * 4) + 3];
158     }
159
160     // All other round keys are found from the previous round
keys.
161     for (i = Nk; i < Nb * (Nr + 1); ++i)
162     {
163         {
164             k = (i - 1) * 4;
165             tempa[0] = RoundKey[k + 0];
166             tempa[1] = RoundKey[k + 1];
167             tempa[2] = RoundKey[k + 2];
168             tempa[3] = RoundKey[k + 3];
169         }
170     }
171
172     if (i % Nk == 0)
173     {
174         // This function shifts the 4 bytes in a word to the
left once.
175         // [a0,a1,a2,a3] becomes [a1,a2,a3,a0]
176
177         // Function RotWord()
178         {
179             const uint8_t u8tmp = tempa[0];
180             tempa[0] = tempa[1];
181             tempa[1] = tempa[2];
182             tempa[2] = tempa[3];
183             tempa[3] = u8tmp;
184         }
185
186         // SubWord() is a function that takes a four-byte input
word and
187         // applies the S-box to each of the four bytes to
produce an output word.
188
189         // Function Subword()
190         {

```

```

191     tempa[0] = getSBoxValue(tempa[0]);
192     tempa[1] = getSBoxValue(tempa[1]);
193     tempa[2] = getSBoxValue(tempa[2]);
194     tempa[3] = getSBoxValue(tempa[3]);
195 }
196
197     tempa[0] = tempa[0] ^ Rcon[i/Nk];
198 }
199 #if defined(AES256) && (AES256 == 1)
200 if (i % Nk == 4)
201 {
202     // Function Subword()
203     {
204         tempa[0] = getSBoxValue(tempa[0]);
205         tempa[1] = getSBoxValue(tempa[1]);
206         tempa[2] = getSBoxValue(tempa[2]);
207         tempa[3] = getSBoxValue(tempa[3]);
208     }
209 }
210 #endif
211 j = i * 4; k = (i - Nk) * 4;
212 RoundKey[j + 0] = RoundKey[k + 0] ^ tempa[0];
213 RoundKey[j + 1] = RoundKey[k + 1] ^ tempa[1];
214 RoundKey[j + 2] = RoundKey[k + 2] ^ tempa[2];
215 RoundKey[j + 3] = RoundKey[k + 3] ^ tempa[3];
216 }
217 }
218
219 void AES_init_ctx(struct AES_ctx* ctx, const uint8_t* key)
220 {
221     KeyExpansion(ctx->RoundKey, key);
222 }
223 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
224 void AES_init_ctx_iv(struct AES_ctx* ctx, const uint8_t* key,
const uint8_t* iv)
225 {
226     KeyExpansion(ctx->RoundKey, key);
227     memcpy(ctx->Iv, iv, AES_BLOCKLEN);
228 }
229 void AES_ctx_set_iv(struct AES_ctx* ctx, const uint8_t* iv)
230 {
231     memcpy(ctx->Iv, iv, AES_BLOCKLEN);
232 }
233 #endif
234
235 // This function adds the round key to state.
236 // The round key is added to the state by an XOR function.
237 static void AddRoundKey(uint8_t round, state_t* state, const
uint8_t* RoundKey)
238 {
239     uint8_t i, j;
240     for (i = 0; i < 4; ++i)
241     {
242         for (j = 0; j < 4; ++j)
243         {
244             (*state)[i][j] ^= RoundKey[(round * Nb * 4) + (i * Nb) +
j];
245         }
246     }
247 }
248
249 // The SubBytes Function Substitutes the values in the
state matrix with values in an S-box.
250 static void SubBytes(state_t* state)
251 {
252     {
253         uint8_t i, j;
254         for (i = 0; i < 4; ++i)
255         {
256             for (j = 0; j < 4; ++j)
257             {
258                 (*state)[j][i] = getSBoxValue((*state)[j][i]);
259             }
260         }
261     }
262 }
263 // The ShiftRows() function shifts the rows in the state to
the left.
264 // Each row is shifted with different offset.
265 // Offset = Row number. So the first row is not shifted.
266 static void ShiftRows(state_t* state)
267 {
268     uint8_t temp;
269
270     // Rotate first row 1 columns to left
271     temp = (*state)[0][1];
272     (*state)[0][1] = (*state)[1][1];
273     (*state)[1][1] = (*state)[2][1];
274     (*state)[2][1] = (*state)[3][1];
275     (*state)[3][1] = temp;
276
277     // Rotate second row 2 columns to left
278     temp = (*state)[0][2];
279     (*state)[0][2] = (*state)[2][2];
280     (*state)[2][2] = temp;
281
282     temp = (*state)[1][2];
283     (*state)[1][2] = (*state)[3][2];
284     (*state)[3][2] = temp;
285

```

```

286 // Rotate third row 3 columns to left
287 temp = (*state)[0][3];
288 (*state)[0][3] = (*state)[3][3];
289 (*state)[3][3] = (*state)[2][3];
290 (*state)[2][3] = (*state)[1][3];
291 (*state)[1][3] = temp;
292 }
293
294 static uint8_t xtime(uint8_t x)
295 {
296     return ((x<<1) ^ (((x>>7) & 1) * 0x1b));
297 }
298
299 // MixColumns function mixes the columns of the state matrix
300 static void MixColumns(state_t* state)
301 {
302     uint8_t i;
303     uint8_t Tmp, Tm, t;
304     for (i = 0; i < 4; ++i)
305     {
306         t = (*state)[i][0];
307         Tmp = (*state)[i][0] ^ (*state)[i][1] ^ (*state)[i][2] ^
308         (*state)[i][3];
309         Tm = (*state)[i][0] ^ (*state)[i][1]; Tm = xtime(Tm);
310         (*state)[i][0] ^= Tm ^ Tmp;
311         Tm = (*state)[i][1] ^ (*state)[i][2]; Tm = xtime(Tm);
312         (*state)[i][1] ^= Tm ^ Tmp;
313         Tm = (*state)[i][2] ^ (*state)[i][3]; Tm = xtime(Tm);
314         (*state)[i][2] ^= Tm ^ Tmp;
315         Tm = (*state)[i][3] ^ t; Tm = xtime(Tm);
316         (*state)[i][3] ^= Tm ^ Tmp;
317     }
318
319 // Multiply is used to multiply numbers in the field GF(2^8)
320 // Note: The last call to xtime() is unneeded, but often ends
321 // up generating a smaller binary
322 // The compiler seems to be able to vectorize the
323 // operation better this way.
324 // See https://github.com/kokke/tiny-AES-c/pull/34
325 #if MULTIPLY_AS_A_FUNCTION
326 static uint8_t Multiply(uint8_t x, uint8_t y)
327 {
328     return (((y & 1) * x) ^
329             ((y>>1 & 1) * xtime(x)) ^
330             ((y>>2 & 1) * xtime(xtime(x))) ^
331             ((y>>3 & 1) * xtime(xtime(xtime(x)))) ^
332             ((y>>4 & 1) * xtime(xtime(xtime(xtime(x)))))); /* this
333 last call to xtime() can be omitted */
334 }
335 #else
336 #define Multiply(x, y) \
337     ( ((y & 1) * x) ^ \
338       ((y>>1 & 1) * xtime(x)) ^ \
339       ((y>>2 & 1) * xtime(xtime(x))) ^ \
340       ((y>>3 & 1) * xtime(xtime(xtime(x)))) ^ \
341       ((y>>4 & 1) * xtime(xtime(xtime(xtime(x)))))) \
342     )
343 #endif
344
345 #if (defined(CBC) && CBC == 1) || (defined(ECB) && ECB == 1)
346 /*
347 static uint8_t getSBoxInvert(uint8_t num)
348 {
349     return rsbox[num];
350 }
351 */
352 #define getSBoxInvert(num) (rsbox[(num)])
353
354 // MixColumns function mixes the columns of the state matrix.
355 // The method used to multiply may be difficult to understand
356 // for the inexperienced.
357 // Please use the references to gain more information.
358 static void InvMixColumns(state_t* state)
359 {
360     int i;
361     uint8_t a, b, c, d;
362     for (i = 0; i < 4; ++i)
363     {
364         a = (*state)[i][0];
365         b = (*state)[i][1];
366         c = (*state)[i][2];
367         d = (*state)[i][3];
368
369         (*state)[i][0] = Multiply(a, 0x0e) ^ Multiply(b, 0x0b) ^
370         Multiply(c, 0x0d) ^ Multiply(d, 0x09);
371         (*state)[i][1] = Multiply(a, 0x09) ^ Multiply(b, 0x0e) ^
372         Multiply(c, 0x0b) ^ Multiply(d, 0x0d);
373         (*state)[i][2] = Multiply(a, 0x0d) ^ Multiply(b, 0x09) ^
374         Multiply(c, 0x0e) ^ Multiply(d, 0x0b);
375         (*state)[i][3] = Multiply(a, 0x0b) ^ Multiply(b, 0x0d) ^
376         Multiply(c, 0x09) ^ Multiply(d, 0x0e);
377     }
378 }
379
380 // The SubBytes Function Substitutes the values in the
381 // state matrix with values in an S-box.
382 static void InvSubBytes(state_t* state)
383 {
384     uint8_t i, j;

```

```

374 for (i = 0; i < 4; ++i)
375 {
376     for (j = 0; j < 4; ++j)
377     {
378         (*state)[j][i] = getSBoxInvert((*state)[j][i]);
379     }
380 }
381
382 static void InvShiftRows(state_t* state)
383 {
384     uint8_t temp;
385
386     // Rotate first row 1 columns to right
387     temp = (*state)[3][1];
388     (*state)[3][1] = (*state)[2][1];
389     (*state)[2][1] = (*state)[1][1];
390     (*state)[1][1] = (*state)[0][1];
391     (*state)[0][1] = temp;
392
393     // Rotate second row 2 columns to right
394     temp = (*state)[0][2];
395     (*state)[0][2] = (*state)[2][2];
396     (*state)[2][2] = temp;
397
398     temp = (*state)[1][2];
399     (*state)[1][2] = (*state)[3][2];
400     (*state)[3][2] = temp;
401
402     // Rotate third row 3 columns to right
403     temp = (*state)[0][3];
404     (*state)[0][3] = (*state)[1][3];
405     (*state)[1][3] = (*state)[2][3];
406     (*state)[2][3] = (*state)[3][3];
407     (*state)[3][3] = temp;
408 }
409
410 #endif // #if (defined(CBC) && CBC == 1) || (defined(ECB) &&
411 ECB == 1)
412
413 // Cipher is the main function that encrypts the PlainText.
414 static void Cipher(state_t* state, const uint8_t* RoundKey)
415 {
416     uint8_t round = 0;
417
418     // Add the First round key to the state before starting the
419     rounds.
420     AddRoundKey(0, state, RoundKey);
421
422     // There will be Nr rounds.
423     // The first Nr-1 rounds are identical.
424     // These Nr rounds are executed in the loop below.
425     // Last one without MixColumns()
426     for (round = 1; ; ++round)
427     {
428         SubBytes(state);
429         ShiftRows(state);
430         if (round == Nr) {
431             break;
432         }
433         MixColumns(state);
434         AddRoundKey(round, state, RoundKey);
435     }
436
437     // Add round key to last round
438     AddRoundKey(Nr, state, RoundKey);
439 }
440
441 #if (defined(CBC) && CBC == 1) || (defined(ECB) && ECB == 1)
442 static void InvCipher(state_t* state, const uint8_t* RoundKey)
443 {
444     uint8_t round = 0;
445
446     // Add the First round key to the state before starting the
447     rounds.
448     AddRoundKey(Nr, state, RoundKey);
449
450     // There will be Nr rounds.
451     // The first Nr-1 rounds are identical.
452     // These Nr rounds are executed in the loop below.
453     // Last one without InvMixColumn()
454     for (round = (Nr - 1); ; --round)
455     {
456         InvShiftRows(state);
457         InvSubBytes(state);
458         AddRoundKey(round, state, RoundKey);
459         if (round == 0) {
460             break;
461         }
462         InvMixColumns(state);
463     }
464
465     #endif // #if (defined(CBC) && CBC == 1) || (defined(ECB) &&
466 ECB == 1)
467
468 /
469
470 *****
471
472 /* Public
473 functions:
474 */
475
476 /

```

```

*****
*****/
467 #if defined(ECB) && (ECB == 1)
468
469 void AES_ECB_encrypt(const struct AES_ctx* ctx, uint8_t* buf)
470 {
471     // The next function call encrypts the PlainText with the
    Key using AES algorithm.
472     Cipher((state_t*)buf, ctx->RoundKey);
473 }
474
475 void AES_ECB_decrypt(const struct AES_ctx* ctx, uint8_t* buf)
476 {
477     // The next function call decrypts the PlainText with the
    Key using AES algorithm.
478     InvCipher((state_t*)buf, ctx->RoundKey);
479 }
480
481
482
483 #endif // #if defined(ECB) && (ECB == 1)
484
485
486
487
488
489 #if defined(CBC) && (CBC == 1)
490
491 static void XorWithIv(uint8_t* buf, const uint8_t* Iv)
492 {
493     uint8_t i;
494     for (i = 0; i < AES_BLOCKLEN; ++i) // The block in AES is
always 128bit no matter the key size
495     {
496         buf[i] ^= Iv[i];
497     }
498 }
499
500
501 void AES_CBC_encrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length)
502 {
503     size_t i;
504     uint8_t* Iv = ctx->Iv;
505     for (i = 0; i < length; i += AES_BLOCKLEN)
506     {
507         XorWithIv(buf, Iv);
508         Cipher((state_t*)buf, ctx->RoundKey);
509         Iv = buf;
510         buf += AES_BLOCKLEN;
511     }
512     /* store Iv in ctx for next call */
513     memcpy(ctx->Iv, Iv, AES_BLOCKLEN);
514 }
515
516 void AES_CBC_decrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length)
517 {
518     size_t i;
519     uint8_t storeNextIv[AES_BLOCKLEN];
520     for (i = 0; i < length; i += AES_BLOCKLEN)
521     {
522         memcpy(storeNextIv, buf, AES_BLOCKLEN);
523         InvCipher((state_t*)buf, ctx->RoundKey);
524         XorWithIv(buf, ctx->Iv);
525         memcpy(ctx->Iv, storeNextIv, AES_BLOCKLEN);
526         buf += AES_BLOCKLEN;
527     }
528 }
529
530
531 #endif // #if defined(CBC) && (CBC == 1)
532
533
534
535 #if defined(CTR) && (CTR == 1)
536
537 /* Symmetrical operation: same function for encrypting as for
decrypting. Note any IV/nonce should never be reused with the same
key */
538 void AES_CTR_xcrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length)
539 {
540     uint8_t buffer[AES_BLOCKLEN];
541
542     size_t i;
543     int bi;
544     for (i = 0, bi = AES_BLOCKLEN; i < length; ++i, ++bi)
545     {
546         if (bi == AES_BLOCKLEN) /* we need to regen xor compliment
in buffer */
547         {
548             memcpy(buffer, ctx->Iv, AES_BLOCKLEN);
549             Cipher((state_t*)buffer, ctx->RoundKey);
550
551             /* Increment Iv and handle overflow */
552             for (bi = (AES_BLOCKLEN - 1); bi >= 0; --bi)

```

```

554         {
555             /* inc will overflow */
556             if (ctx->Iv[bi] == 255)
557             {
558                 ctx->Iv[bi] = 0;
559                 continue;
560             }
561             ctx->Iv[bi] += 1;
562             break;
563         }
564         bi = 0;
565     }
566     buf[i] = (buf[i] ^ buffer[bi]);
567 }
568 }
569 }
570
571 #endif // #if defined(CTR) && (CTR == 1)
572

```

A.2 kokke tiny-AES-claes.h (91줄, 2,790B)

```

1 #ifndef _AES_H_
2 #define _AES_H_
3
4 #include <stdint.h>
5 #include <stddef.h>
6
7 // #define the macros below to 1/0 to enable/disable the mode
of operation.
8 //
9 // CBC enables AES encryption in CBC-mode of operation.
10 // CTR enables encryption in counter-mode.
11 // ECB enables the basic ECB 16-byte block algorithm. All can
be enabled simultaneously.
12
13 // The #ifndef-guard allows it to be configured before
#include'ing or at compile time.
14 #ifndef CBC
15 #define CBC 1
16 #endif
17
18 #ifndef ECB
19 #define ECB 1
20 #endif
21
22 #ifndef CTR
23 #define CTR 1
24 #endif
25
26
27 #define AES128 1
28 // #define AES192 1
29 // #define AES256 1
30
31 #define AES_BLOCKLEN 16 // Block length in bytes - AES is 128b
block only
32
33 #if defined(AES256) && (AES256 == 1)
34 #define AES_KEYLEN 32
35 #define AES_keyExpSize 240
36 #elif defined(AES192) && (AES192 == 1)
37 #define AES_KEYLEN 24
38 #define AES_keyExpSize 208
39 #else
40 #define AES_KEYLEN 16 // Key length in bytes
41 #define AES_keyExpSize 176
42 #endif
43
44 struct AES_ctx
45 {
46     uint8_t RoundKey[AES_keyExpSize];
47 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
48     uint8_t Iv[AES_BLOCKLEN];
49 #endif
50 };
51
52 void AES_init_ctx(struct AES_ctx* ctx, const uint8_t* key);
53 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
54 void AES_init_ctx_iv(struct AES_ctx* ctx, const uint8_t* key,
const uint8_t* iv);
55 void AES_ctx_set_iv(struct AES_ctx* ctx, const uint8_t* iv);
56 #endif
57
58 #if defined(ECB) && (ECB == 1)
59 // buffer size is exactly AES_BLOCKLEN bytes;
60 // you need only AES_init_ctx as IV is not used in ECB
61 // NB: ECB is considered insecure for most uses
62 void AES_ECB_encrypt(const struct AES_ctx* ctx, uint8_t* buf);
63 void AES_ECB_decrypt(const struct AES_ctx* ctx, uint8_t* buf);
64
65 #endif // #if defined(ECB) && (ECB == 1)
66
67
68 #if defined(CBC) && (CBC == 1)
69 // buffer size MUST be mutile of AES_BLOCKLEN;
70 // Suggest https://en.wikipedia.org/wiki/

```

```

Padding_(cryptography)#PKCS7 for padding scheme
71 // NOTES: you need to set IV in ctx via AES_init_ctx_iv() or
AES_ctx_set_iv()
72 // no IV should ever be reused with the same key

73 void AES_CBC_encrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length);

74 void AES_CBC_decrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length);

75
76 #endif // #if defined(CBC) && (CBC == 1)
77
78
79 #if defined(CTR) && (CTR == 1)
80
81 // Same function for encrypting as for decrypting.
82 // IV is incremented for every block, and used after
encryption as XOR-compliment for output
83 // Suggesting https://en.wikipedia.org/wiki/
Padding_(cryptography)#PKCS7 for padding scheme
84 // NOTES: you need to set IV in ctx with AES_init_ctx_iv() or
AES_ctx_set_iv()
85 // no IV should ever be reused with the same key
86 void AES_CTR_xcrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
size_t length);
87
88 #endif // #if defined(CTR) && (CTR == 1)
89
90
91 #endif // _AES_H_

```

A.3 B-Con_crypto-algorithms/sha256.c (158줄, 5,263B)

```

1 /
*****
*
2 * Filename:   sha256.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any
guarantees.
6 * Details:   Implementation of the SHA-256 hashing algorithm.
SHA2
7             SHA-256 is one of the three algorithms in the
8             specification. The others, SHA-384 and SHA-512,
are not
9             offered in this implementation.
10            Algorithm specification can be found here:
11            * http://csrc.nist.gov/publications/fips/
fips180-2/fips180-2withchangenotice.pdf
12            This implementation uses little endian byte
order.
13
*****
*/
14
15 /***** HEADER FILES *****/
16 #include <stdlib.h>
17 #include <memory.h>
18 #include "sha256.h"
19
20 /***** MACROS *****/
21 #define ROTLEFT(a,b) (((a) << (b)) | ((a) >> (32-(b))))
22 #define ROTRIGHT(a,b) (((a) >> (b)) | ((a) << (32-(b))))
23
24 #define CH(x,y,z) (((x) & (y)) ^ ((~(x) & (z))))
25 #define MAJ(x,y,z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))
26 #define EP0(x) (ROTRIGHT(x,2) ^ ROTRIGHT(x,13) ^ ROTRIGHT(x,
22))
27 #define EP1(x) (ROTRIGHT(x,6) ^ ROTRIGHT(x,11) ^ ROTRIGHT(x,
25))
28 #define SIG0(x) (ROTRIGHT(x,7) ^ ROTRIGHT(x,18) ^ ((x) >> 3))
29 #define SIG1(x) (ROTRIGHT(x,17) ^ ROTRIGHT(x,19) ^ ((x) >>
10))
30
31 /***** VARIABLES *****/
32 static const WORD k[64] = {
33     0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b,
0x59f111f1, 0x923f82a4, 0xab1c5ed5,
34     0xd807aa98, 0x12835b01, 0x243185be,
0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
35     0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f,
0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
36     0x983e5152, 0xa831c66d,
0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
37     0x27b70a85, 0x2e1b2138, 0x4d2c6dfe,
0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
38     0xa2bfe8a1, 0xa81a664b,
0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
39     0x19a4c116, 0x1e376c08, 0x2748774c,
0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
40     0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa,
0xa4506ceb, 0xbef9a3f7, 0xc67177f2
41 };
42
43 /***** FUNCTION DEFINITIONS *****/

```

```

*****/
44 void sha256_transform(SHA256_CTX *ctx, const BYTE data[])
45 {
46     WORD a, b, c, d, e, f, g, h, i, j, t1, t2, m[64];
47
48     for (i = 0, j = 0; i < 16; ++i, j += 4)
49         m[i] = (data[j] << 24) | (data[j + 1] << 16) |
(data[j + 2] << 8) | (data[j + 3]);
50     for (i = 0; i < 64; ++i)
51         m[i] = SIG1(m[i - 2]) + m[i - 7] + SIG0(m[i -
15]) + m[i - 16];
52
53     a = ctx->state[0];
54     b = ctx->state[1];
55     c = ctx->state[2];
56     d = ctx->state[3];
57     e = ctx->state[4];
58     f = ctx->state[5];
59     g = ctx->state[6];
60     h = ctx->state[7];
61
62     for (i = 0; i < 64; ++i) {
63         t1 = h + EP1(e) + CH(e,f,g) + k[i] + m[i];
64         t2 = EP0(a) + MAJ(a,b,c);
65         h = g;
66         g = f;
67         f = e;
68         e = d + t1;
69         d = c;
70         c = b;
71         b = a;
72         a = t1 + t2;
73     }
74
75     ctx->state[0] += a;
76     ctx->state[1] += b;
77     ctx->state[2] += c;
78     ctx->state[3] += d;
79     ctx->state[4] += e;
80     ctx->state[5] += f;
81     ctx->state[6] += g;
82     ctx->state[7] += h;
83 }
84
85 void sha256_init(SHA256_CTX *ctx)
86 {
87     ctx->datalen = 0;
88     ctx->bitlen = 0;
89     ctx->state[0] = 0x6a09e667;
90     ctx->state[1] = 0xbb67ae85;
91     ctx->state[2] = 0x3c6ef372;
92     ctx->state[3] = 0xa54ff53a;
93     ctx->state[4] = 0x510e527f;
94     ctx->state[5] = 0x9b05688c;
95     ctx->state[6] = 0x1f83d9ab;
96     ctx->state[7] = 0x5be0cd19;
97 }
98
99 void sha256_update(SHA256_CTX *ctx, const BYTE data[], size_t
len)
100 {
101     WORD i;
102
103     for (i = 0; i < len; ++i) {
104         ctx->data[ctx->datalen] = data[i];
105         ctx->datalen++;
106         if (ctx->datalen == 64) {
107             sha256_transform(ctx, ctx->data);
108             ctx->bitlen += 512;
109             ctx->datalen = 0;
110         }
111     }
112 }
113
114 void sha256_final(SHA256_CTX *ctx, BYTE hash[])
115 {
116     WORD i;
117
118     i = ctx->datalen;
119
120     // Pad whatever data is left in the buffer.
121     if (ctx->datalen < 56) {
122         ctx->data[i++] = 0x80;
123         while (i < 56)
124             ctx->data[i++] = 0x00;
125     }
126     else {
127         ctx->data[i++] = 0x80;
128         while (i < 64)
129             ctx->data[i++] = 0x00;
130         sha256_transform(ctx, ctx->data);
131         memset(ctx->data, 0, 56);
132     }
133
134     // Append to the padding the total message's length in
bits and transform.
135     ctx->bitlen += ctx->datalen * 8;
136     ctx->data[63] = ctx->bitlen;
137     ctx->data[62] = ctx->bitlen >> 8;
138     ctx->data[61] = ctx->bitlen >> 16;
139     ctx->data[60] = ctx->bitlen >> 24;

```

```

140     ctx->data[59] = ctx->bitlen >> 32;
141     ctx->data[58] = ctx->bitlen >> 40;
142     ctx->data[57] = ctx->bitlen >> 48;
143     ctx->data[56] = ctx->bitlen >> 56;
144     sha256_transform(ctx, ctx->data);
145
146     // Since this implementation uses little endian byte
ordering and SHA uses big endian,
147     // reverse all the bytes when copying the final state
to the output hash.
148     for (i = 0; i < 4; ++i) {
149         hash[i] = (ctx->state[0] >> (24 - i * 8))
& 0x000000ff;
150         hash[i + 4] = (ctx->state[1] >> (24 - i * 8))
& 0x000000ff;
151         hash[i + 8] = (ctx->state[2] >> (24 - i * 8))
& 0x000000ff;
152         hash[i + 12] = (ctx->state[3] >> (24 - i * 8))
& 0x000000ff;
153         hash[i + 16] = (ctx->state[4] >> (24 - i * 8))
& 0x000000ff;
154         hash[i + 20] = (ctx->state[5] >> (24 - i * 8))
& 0x000000ff;
155         hash[i + 24] = (ctx->state[6] >> (24 - i * 8))
& 0x000000ff;
156         hash[i + 28] = (ctx->state[7] >> (24 - i * 8))
& 0x000000ff;
157     }
158 }

```

A.4 B-Con_crypto-algorithms/aes.c (1,095줄, 42,361B)

```

1 /
*****
2 * Filename:    aes.c
3 * Author:     Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any
guarantees.
6 * Details:    This code is the implementation of the AES
algorithm and
7              the CTR, CBC, and CCM modes of operation it can
be used in.
8
9              AES is, specified by the NIST in in publication
FIPS PUB 197,
10              available at:
* http://csrc.nist.gov/publications/fips/
fips197/fips-197.pdf .
11
12              The CBC and CTR modes of operation are specified by
NIST SP 800-38 A, available at:
13              * http://csrc.nist.gov/publications/nistpubs/
800-38a/sp800-38a.pdf .
14              The CCM mode of operation is specified by NIST
SP800-38 C, available at:
15              * http://csrc.nist.gov/publications/nistpubs/
800-38C/SP800-38C_updated-July20_2007.pdf
16
*****
*/
17
18 /***** HEADER FILES *****/
19 #include <stdlib.h>
20 #include <memory.h>
21 #include "aes.h"
22
23 #include <stdio.h>
24
25 /***** MACROS *****/
26 // The least significant byte of the word is rotated to the
end.
27 #define KE_ROTWORD(x) (((x) << 8) | ((x) >> 24))
28
29 #define TRUE 1
30 #define FALSE 0
31
32 /***** DATA TYPES *****/
33 #define AES_128_ROUNDS 10
34 #define AES_192_ROUNDS 12
35 #define AES_256_ROUNDS 14
36
37 /***** FUNCTION DECLARATIONS *****/
38 void ccm_prepare_first_ctr_blk(BYTE counter[], const BYTE
nonce[], int nonce_len, int payload_len_store_size);
39 void ccm_prepare_first_format_blk(BYTE buf[], int assoc_len,
int payload_len, int payload_len_store_size, int mac_len, const BYTE
nonce[], int nonce_len);
40 void ccm_format_assoc_data(BYTE buf[], int *end_of_buf, const
BYTE assoc[], int assoc_len);
41 void ccm_format_payload_data(BYTE buf[], int *end_of_buf,
const BYTE payload[], int payload_len);
42
43 /***** VARIABLES *****/

```

```

44 // This is the specified AES SBox. To look up a substitution
value, put the first
45 // nibble in the first index (row) and the second nibble in
the second index (column).
46 static const BYTE aes_sbox[16][16] = {
47     {0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F,
0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76},
48     {0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD,
0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0},
49     {0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC,
0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15},
50     {0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A,
0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75},
51     {0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B,
0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84},
52     {0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB,
0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF},
53     {0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D,
0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8},
54     {0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC,
0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2},
55     {0xCD, 0x0C, 0x13, 0xEC, 0x5F,
0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73},
56     {0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE,
0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB},
57     {0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C,
0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79},
58     {0xE7, 0xCB, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C,
0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08},
59     {0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD,
0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A},
60     {0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E,
0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E},
61     {0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E,
0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF},
62     {0x8C, 0xA1, 0x89, 0x0D, 0xBF,
0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16}
63 };
64
65 static const BYTE aes_inv_sbox[16][16] = {
66     {0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF,
0x40, 0xA3, 0x9E, 0x81, 0xF3, 0xD7, 0xFB},
67     {0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E,
0x43, 0x44, 0xC4, 0xDE, 0xE9, 0xCB},
68     {0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C,
0x95, 0x0B, 0x42, 0xFA, 0xC3, 0x4E},
69     {0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B,
0xA2, 0x49, 0x6D, 0x8B, 0xD1, 0x25},
70
71     {0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0x5C, 0xCC, 0x5D,
0x65, 0xB6, 0x92},
72     {0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E,
0x15, 0x46, 0x57, 0xA7, 0x8D, 0x9D, 0x84},
73     {0x90, 0xDB, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A,
0xF7, 0xE4, 0x58, 0x05, 0xB8, 0xB3, 0x45, 0x06},
74     {0xD0, 0x2C, 0x1E, 0x8F, 0xC4, 0x3F, 0x0A, 0x0F, 0x02, 0xC1, 0xAF,
0xBD, 0x03, 0x01, 0x13, 0x8A, 0x6B},
75     {0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA,
0x97, 0xF2, 0xCF, 0xCE, 0xF0, 0xB4, 0xE6, 0x73},
76     {0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD,
0x35, 0x85, 0xE2, 0xF9, 0x37, 0xE8, 0x1C, 0x75, 0xDF, 0x6E},
77     {0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F,
0xB7, 0x62, 0x0E, 0xAA, 0x18, 0xBE, 0x1B},
78     {0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB,
0xC0, 0xFE, 0x78, 0xCD, 0x5A, 0xF4},
79     {0x1F, 0xDD,
0xA8, 0x33, 0xB8, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xEC,
0x5F},
80     {0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D,
0xE5, 0x7A, 0x9F, 0x93, 0xC9, 0x9C, 0xEF},
81     {0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB,
0xBB, 0x3C, 0x83, 0x53, 0x99, 0x61},
82     {0x17, 0x2B, 0x04, 0x7E, 0xBA,
0x77, 0xD6, 0x26, 0xE1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0C, 0x7D}
83 };
84 // This table stores pre-calculated values for all possible
GF(2^8) calculations. This
85 // table is only used by the (Inv)MixColumns steps.
86 // USAGE: The second index (column) is the coefficient of
multiplication. Only 7 different
87 // coefficients are used: 0x01, 0x02, 0x03, 0x09, 0x0B, 0x0D,
0x0E, but multiplication by
88 // 1 is negligible leaving only 6 coefficients. Each column of
the table is devoted to one
89 // of these coefficients, in the ascending order of value,
from values 0x00 to 0xFF.
90 static const BYTE gf_mul[256][6] = {
91     {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, {0x02, 0x03, 0x09, 0x0B,
0x0D, 0x0E},
92     {0x04, 0x06, 0x12, 0x16, 0x1A, 0x1C}, {0x08, 0x0F, 0x1B, 0x1D,
0x17, 0x12},
93     {0x00, 0x0C, 0x24, 0x2C, 0x34, 0x38}, {0x0A, 0x0F, 0x2D,
0x27, 0x39, 0x36},
94     {0x0C, 0x0A, 0x36, 0x3A, 0x2E, 0x24}, {0x0E, 0x09, 0x3F,
0x31, 0x23, 0x2A},
95     {0x10, 0x18, 0x48, 0x58, 0x68, 0x70}, {0x12, 0x1B,
0x41, 0x53, 0x65, 0x7E},
96     {0x14, 0x1E, 0x5A, 0x4E, 0x72, 0x6C}, {0x16, 0x1D,
0x53, 0x45, 0x7F, 0x62},

```

```

97      {0x18, 0x14, 0x6c, 0x74, 0x5c, 0x48}, {0x1a, 0x17, 0x65, 0x7f,
0x51, 0x46},
98      {0x1c, 0x12, 0x7e, 0x62, 0x46, 0x54}, {0x1e,
0x11, 0x77, 0x69, 0x4b, 0x5a},
99      {0x20, 0x30, 0x90, 0xb0, 0xd0, 0xe0}, {0x22, 0x33, 0x99, 0xbb,
0xdd, 0xee},
100     {0x24, 0x36, 0x82, 0xa6, 0xca, 0xfc}, {0x26, 0x35, 0x8b, 0xad,
0xc7, 0xf2},
101     {0x28, 0x3c, 0xb4, 0x9c, 0xe4, 0xd8}, {0x2a, 0x3f, 0xbd,
0x97, 0xe9, 0xd6},
102     {0x2c, 0x3a, 0xa6, 0x8a, 0xfe, 0xc4}, {0x2e, 0x39, 0xaf,
0x81, 0xf3, 0xca},
103     {0x30, 0x28, 0xd8, 0xe8, 0xb8, 0x90}, {0x32, 0x2b,
0xd1, 0xe3, 0xb5, 0x9e},
104     {0x34, 0x2e, 0xca, 0xfe, 0xa2, 0x8c}, {0x36, 0x2d,
0xc3, 0xf5, 0xaf, 0x82},
105     {0x38, 0x24, 0xfc, 0xc4, 0x8c, 0xa8}, {0x3a, 0x27, 0xf5, 0xcf,
0x81, 0xa6},
106     {0x3c, 0x22, 0xee, 0xd2, 0x96, 0xb4}, {0x3e,
0x21, 0xe7, 0xd9, 0x9b, 0xba},
107     {0x40, 0x60, 0x3b, 0x7b, 0xbb, 0xdb},
{0x42, 0x63, 0x32, 0x70, 0xb6, 0xd5},
108     {0x44, 0x66, 0x29, 0x6d, 0xa1, 0xc7},
{0x46, 0x65, 0x20, 0x66, 0xac, 0xc9},
109     {0x48, 0x6c, 0x1f, 0x57, 0x8f, 0xe3}, {0x4a, 0x6f, 0x16, 0x5c,
0x82, 0xed},
110     {0x4c, 0x6a, 0xd0, 0x41, 0x95, 0xff}, {0x4e, 0x69, 0xd0, 0x4a,
0x98, 0xf1},
111     {0x50, 0x78, 0x73, 0x23, 0xd3, 0xab}, {0x52, 0x7b, 0x7a,
0x28, 0xde, 0xa5},
112     {0x54, 0x7e, 0x61, 0x35, 0xc9, 0xb7}, {0x56, 0x7d, 0x68, 0x3e,
0xc4, 0xb9},
113     {0x58, 0x74, 0x57, 0x0f, 0xe7, 0x93}, {0x5a, 0x77, 0x5e,
0x04, 0xea, 0x9d},
114     {0x5c, 0x72, 0x45, 0x19, 0xfd, 0x8f}, {0x5e, 0x71, 0x4c,
0x12, 0xf0, 0x81},
115     {0x60, 0x50, 0xab, 0xcb, 0x6b, 0x3b},
{0x62, 0x53, 0xa2, 0xc0, 0x66, 0x35},
116     {0x64, 0x56, 0xb9, 0xdd, 0x71, 0x27},
{0x66, 0x55, 0xb0, 0xd6, 0x7c, 0x29},
117     {0x68, 0x5c, 0x8f, 0xe7, 0x5f, 0x03}, {0x6a, 0x5f, 0x86, 0xec,
0x52, 0xd0},
118     {0x6c, 0x5a, 0x9d, 0xf1, 0x45, 0x1f}, {0x6e, 0x59, 0x94, 0xfa,
0x48, 0x11},
119     {0x70, 0x48, 0xe3, 0x93, 0x03, 0x4b}, {0x72, 0x4b, 0xea,
0x98, 0x0e, 0x45},
120     {0x74, 0x4e, 0xf1, 0x85, 0x19, 0x57}, {0x76, 0x4d, 0xf8, 0x8e,
0x14, 0x59},
121     {0x78, 0x44, 0xc7, 0xbf, 0x37, 0x73}, {0x7a, 0x47, 0xce,
0xb4, 0x3a, 0x7d},
122     {0x7c, 0x42, 0xd5, 0xa9, 0x2d, 0x6f}, {0x7e, 0x41, 0xdc,
0xa2, 0x20, 0x61},
123     {0x80, 0xc0, 0x76, 0xf6, 0x6d, 0xad}, {0x82, 0xc3, 0x7f, 0xfd,
0x60, 0xa3},
124     {0x84, 0xc6, 0x64, 0xe0, 0x77, 0xb1}, {0x86, 0xc5, 0x6d, 0xeb,
0x7a, 0xbf},
125     {0x88, 0xcc, 0x52, 0xda, 0x59, 0x95}, {0x8a, 0xcf, 0x5b,
0xd1, 0x54, 0x9b},
126     {0x8c, 0xca, 0x40, 0xcc, 0x43, 0x89}, {0x8e,
0xc9, 0x49, 0xc7, 0x4e, 0x87},
127     {0x90, 0xd8, 0x3e, 0xae, 0x05, 0xdd}, {0x92, 0xdb,
0x37, 0xa5, 0x08, 0xd3},
128     {0x94, 0xde, 0x2c, 0xb8, 0x1f, 0xc1}, {0x96, 0xdd,
0x25, 0xb3, 0x12, 0xc7},
129     {0x98, 0xd4, 0x1a, 0x82, 0x31, 0xe5}, {0x9a,
0xd7, 0x13, 0x89, 0x3c, 0xeb},
130     {0x9c, 0xd2, 0x08, 0x94, 0x2b, 0xf9}, {0x9e, 0xd1, 0x01, 0x9f,
0x26, 0xf7},
131     {0xa0, 0xf0, 0xe6, 0x46, 0xbd, 0xd4}, {0xa2, 0xf3, 0xef, 0x4d,
0xb0, 0x43},
132     {0xa4, 0xf6, 0xf4, 0x50, 0xa7, 0x51}, {0xa6, 0xf5, 0xfd, 0x5b,
0xaa, 0x5f},
133     {0xa8, 0xfc, 0xc2, 0x6a, 0x89, 0x75}, {0xaa, 0xff, 0xcb,
0x61, 0x84, 0x7b},
134     {0xac, 0xfa, 0xd0, 0x7c, 0x93, 0x69}, {0xae,
0xf9, 0xd9, 0x77, 0x9e, 0x67},
135     {0xb0, 0xe8, 0xae, 0x1e, 0xd5, 0x3d}, {0xb2, 0xeb,
0xa7, 0x15, 0xd8, 0x33},
136     {0xb4, 0xee, 0xbc, 0x08, 0xcf, 0x21}, {0xb6, 0xed,
0xb5, 0x03, 0xc2, 0x2f},
137     {0xb8, 0xe4, 0x8a, 0x32, 0xe1, 0x05}, {0xba,
0xe7, 0x83, 0x39, 0xec, 0x0b},
138     {0xbc, 0xe2, 0x98, 0x24, 0xfb, 0x19}, {0xbe, 0xe1, 0x91, 0x2f,
0xf6, 0x17},
139     {0xc0, 0xa0, 0x4d, 0x8d, 0xd6, 0x76},
{0xc2, 0xa3, 0x44, 0x86, 0xdb, 0x78},
140     {0xc4, 0xa6, 0x5f, 0x9b, 0xcc, 0x6a},
{0xc6, 0xa5, 0x56, 0x90, 0xc1, 0x64},
141     {0xc8, 0xac, 0x69, 0xa1, 0xe2, 0x4e}, {0xca, 0xaf, 0x60, 0xaa,
0xef, 0x40},
142     {0xcc, 0xaa, 0x7b, 0xb7, 0xf8, 0x52}, {0xce, 0xa9, 0x72, 0xbc,
0xf5, 0x5c},
143     {0xd0, 0xb8, 0x05, 0xd5, 0xbe, 0x06}, {0xd2, 0xbb, 0xc0, 0xde,
0xb3, 0x08},
144     {0xd4, 0xbe, 0x17, 0xc3, 0xa4, 0x1a}, {0xd6, 0xbd, 0x1e,
0xc8, 0xa9, 0x14},
145     {0xd8, 0xb4, 0x21, 0xf9, 0x8a, 0x3e}, {0xda,
0xb7, 0x28, 0xf2, 0x87, 0x30},
146     {0xdc, 0xb2, 0x33, 0xef, 0x90, 0x22}, {0xde, 0xb1, 0x3a,
0xe4, 0x9d, 0x2c},
147     {0xe0, 0x90, 0xdd, 0x3d, 0x06, 0x96},

```

```

{0xe2, 0x93, 0xd4, 0x36, 0x0b, 0x98},
148     {0xe4, 0x96, 0xcf, 0x2b, 0x1c, 0x8a},
{0xe6, 0x95, 0xc6, 0x20, 0x11, 0x84},
149     {0xe8, 0x9c, 0xf9, 0x11, 0x32, 0xae}, {0xea, 0x9f, 0xf0, 0x1a,
0x3f, 0xa0},
150     {0xec, 0x9a, 0xeb, 0x07, 0x28, 0xb2}, {0xee, 0x99, 0xe2, 0xc0,
0x25, 0xbc},
151     {0xf0, 0x88, 0x95, 0x65, 0x6e, 0xe6}, {0xf2, 0x8b, 0x9c, 0x6e,
0xe3, 0xe8},
152     {0xf4, 0x8e, 0x87, 0x73, 0x74, 0xfa}, {0xf6, 0x8d, 0x8e,
0x78, 0x79, 0xf4},
153     {0xf8, 0x84, 0xb1, 0x49, 0x5a, 0xde}, {0xfa,
0x87, 0xb8, 0x42, 0x57, 0xd0},
154     {0xfc, 0x82, 0xa3, 0x5f, 0x40, 0xc2}, {0xfe, 0x81, 0xaa,
0x54, 0x4d, 0xcc},
155     {0x1b, 0x9b, 0xec, 0xf7, 0xda, 0x41}, {0x19, 0x98, 0xe5, 0xfc,
0xd7, 0x4f},
156     {0x1f, 0x9d, 0xfe, 0xe1, 0xc0, 0x5d}, {0x1d, 0x9e, 0xf7, 0xea,
0xcd, 0x53},
157     {0x13, 0x97, 0xc8, 0xdb, 0xee, 0x79},
{0x11, 0x94, 0xc1, 0xd0, 0xe3, 0x77},
158     {0x17, 0x91, 0xda, 0xcd, 0xf4, 0x65},
{0x15, 0x92, 0xd3, 0xc6, 0xf9, 0x6b},
159     {0x0b, 0x83, 0xa4, 0xaf, 0xb2, 0x31}, {0x09, 0x80, 0xad,
0xa4, 0xbf, 0x3f},
160     {0x0f, 0x85, 0xb6, 0xb9, 0xa8, 0x2d}, {0x0d, 0x86, 0xbf,
0xb2, 0xa5, 0x23},
161     {0x03, 0x8f, 0x80, 0x83, 0x86, 0x09}, {0x01, 0x8c,
0x89, 0x88, 0xb8, 0x07},
162     {0x07, 0x89, 0x92, 0x95, 0x9c, 0x15}, {0x05, 0x8a, 0x9b, 0x9e,
0x91, 0x1b},
163     {0x3b, 0xab, 0x7c, 0x47, 0x0a, 0xa1}, {0x39, 0xa8, 0x75, 0x4c,
0x07, 0xaf},
164     {0x3f, 0xad, 0x6e, 0x51, 0x10, 0xbd}, {0x3d, 0xae, 0x67, 0x5a,
0x1d, 0xb3},
165     {0x33, 0xa7, 0x58, 0x6b, 0x3e, 0x99},
{0x31, 0xa4, 0x51, 0x60, 0x33, 0x97},
166     {0x37, 0xa1, 0x4a, 0x7d, 0x24, 0x85},
{0x35, 0xa2, 0x43, 0x76, 0x29, 0x8b},
167     {0x2b, 0xb3, 0x34, 0x1f, 0x62, 0xd1}, {0x29, 0xb0, 0x3d,
0x14, 0x6f, 0xdf},
168     {0x2f, 0xb5, 0x26, 0x09, 0x78, 0xcd}, {0x2d, 0xb6, 0x2f,
0x02, 0x75, 0xc3},
169     {0x23, 0xbf, 0x10, 0x33, 0x56, 0xe9}, {0x21, 0xbc,
0x19, 0x38, 0x5b, 0xe7},
170     {0x27, 0xb9, 0x02, 0x25, 0x4c, 0xf5}, {0x25, 0xba, 0x0b, 0x2e,
0x41, 0xfb},
171     {0x5b, 0xfb, 0xd7, 0x8c, 0x61, 0x9a}, {0x59, 0xf8, 0xde,
0x87, 0xc6, 0x94},
172     {0x5f, 0xfd, 0xc5, 0x9a, 0x7b, 0x86}, {0x5d, 0xfe, 0xcc,
0x91, 0x76, 0x88},
173     {0x53, 0xf7, 0xf3, 0xa0, 0x55, 0xa2}, {0x51, 0xf4, 0xfa, 0xab,
0x58, 0xac},
174     {0x57, 0xf1, 0xe1, 0xb6, 0x4f, 0xbe}, {0x55, 0xf2, 0xe8, 0xbd,
0x42, 0xb0},
175     {0x4b, 0xe3, 0x9f, 0xd4, 0x09, 0xea}, {0x49, 0xe0, 0x96, 0xdf,
0x04, 0xe4},
176     {0x4f, 0xe5, 0x8d, 0xc2, 0x13, 0xf6}, {0x4d,
0xe6, 0x84, 0xc9, 0x1e, 0xf8},
177     {0x43, 0xef, 0xbb, 0xf8, 0x3d, 0xd2}, {0x41, 0xec,
0xb2, 0xf3, 0x30, 0xdc},
178     {0x47, 0xe9, 0xa9, 0xee, 0x27, 0xce}, {0x45, 0xea,
0xa0, 0xe5, 0x2a, 0xc0},
179     {0x7b, 0xcb, 0x47, 0x3c, 0xb1, 0x7a}, {0x79, 0xc8, 0x4e,
0x37, 0xbc, 0x74},
180     {0x7f, 0xcd, 0x55, 0x2a, 0xab, 0x66}, {0x7d, 0xce, 0x5c,
0x21, 0xa6, 0x68},
181     {0x73, 0xc7, 0x63, 0x10, 0x85, 0x42}, {0x71, 0xc4, 0x6a, 0x1b,
0x88, 0x4c},
182     {0x77, 0xc1, 0x71, 0x06, 0x9f, 0x5e}, {0x75, 0xc2, 0x78, 0xd0,
0x92, 0x50},
183     {0x6b, 0xd3, 0xf0, 0x64, 0xd9, 0x0a}, {0x69, 0xd0, 0x06, 0x6f,
0xd4, 0x04},
184     {0x6f, 0xd5, 0x1d, 0x72, 0xc3, 0x16}, {0x6d,
0xd6, 0x14, 0x79, 0xce, 0x18},
185     {0x63, 0xdf, 0x2b, 0x48, 0xed, 0x32}, {0x61, 0xdc,
0x22, 0x43, 0xe0, 0x3c},
186     {0x67, 0xd9, 0x39, 0x5e, 0xf7, 0x2e}, {0x65, 0xda,
0x30, 0x55, 0xfa, 0x20},
187     {0x9b, 0x5b, 0x9a, 0x01, 0xb7, 0xec}, {0x99, 0x58, 0x93, 0x0a,
0xba, 0xe2},
188     {0x9f, 0x5d, 0x88, 0x17, 0xad, 0xf0}, {0x9d, 0x5e, 0x81, 0x1c,
0xa0, 0xfe},
189     {0x93, 0x57, 0xbe, 0x2d, 0x83, 0xd4},
{0x91, 0x54, 0xb7, 0x26, 0x8e, 0xda},
190     {0x97, 0x51, 0xac, 0x3b, 0x99, 0xc8},
{0x95, 0x52, 0xa5, 0x30, 0x94, 0xc6},
191     {0x8b, 0x43, 0xd2, 0x59, 0xdf, 0x9c}, {0x89, 0x40, 0xdb,
0x52, 0xd2, 0x92},
192     {0x8f, 0x45, 0xc0, 0x4f, 0xc5, 0x80}, {0x8d,
0x46, 0xc9, 0x44, 0xc8, 0x8e},
193     {0x83, 0x4f, 0xf6, 0x75, 0xeb, 0xa4}, {0x81, 0x4c, 0xff, 0x7e,
0xe6, 0xaa},
194     {0x87, 0x49, 0xe4, 0x63, 0xf1, 0xb8}, {0x85, 0x4a, 0xed,
0x68, 0xfc, 0xb6},
195     {0xbb, 0x6b, 0x0a, 0xb1, 0x67, 0x0c}, {0xb9, 0x68, 0x03, 0xba,
0x6a, 0x02},
196     {0xbf, 0x6d, 0x18, 0xa7, 0x7d, 0x10}, {0xbd, 0x6e, 0x11, 0xac,
0x70, 0x1e},
197     {0xb3, 0x67, 0x2e, 0x9d, 0x53, 0x34},
{0xb1, 0x64, 0x27, 0x96, 0x5e, 0x3a},

```

```

198     {0xb7, 0x61, 0x3c, 0x8b, 0x49, 0x28},
{0xb5, 0x62, 0x35, 0x80, 0x44, 0x26},
199     {0xab, 0x73, 0x42, 0xe9, 0x0f, 0x7c}, {0xa9, 0x70, 0x4b,
0xe2, 0x02, 0x72},
200     {0xaf, 0x75, 0x50, 0xff, 0x15, 0x60}, {0xad,
0x76, 0x59, 0xf4, 0x18, 0x6e},
201     {0xa3, 0x7f, 0x66, 0xc5, 0x3b, 0x44}, {0xa1, 0x7c, 0x6f, 0xce,
0x36, 0x4a},
202     {0xa7, 0x79, 0x74, 0xd3, 0x21, 0x58}, {0xa5, 0x7a, 0x7d,
0xd8, 0x2c, 0x56},
203     {0xdb, 0x3b, 0xa1, 0x7a, 0x0c, 0x37},
{0xd9, 0x38, 0xa8, 0x71, 0x01, 0x39},
204     {0xdf, 0x3d, 0xb3, 0x6c, 0x16, 0x2b}, {0xdd, 0x3e, 0xba,
0x67, 0x1b, 0x25},
205     {0xd3, 0x37, 0x85, 0x56, 0x38, 0x0f}, {0xd1, 0x34, 0x8c, 0x5d,
0x35, 0x01},
206     {0xd7, 0x31, 0x97, 0x40, 0x22, 0x13}, {0xd5, 0x32, 0x9e, 0x4b,
0x2f, 0x1d},
207     {0xcb, 0x23, 0xe9, 0x22, 0x64, 0x47},
{0xc9, 0x20, 0xe0, 0x29, 0x69, 0x49},
208     {0xcf, 0x25, 0xfb, 0x34, 0x7e, 0x5b}, {0xcd, 0x26, 0xf2, 0x3f,
0x73, 0x55},
209     {0xc3, 0x2f, 0xcd, 0x0e, 0x50, 0x7f}, {0xc1, 0x2c,
0xc4, 0x05, 0x5d, 0x71},
210     {0xc7, 0x29, 0xdf, 0x18, 0x4a, 0x63}, {0xc5, 0x2a,
0xd6, 0x13, 0x47, 0x6d},
211     {0xfb, 0x0b, 0x31, 0xca, 0xdc, 0xd7},
{0xf9, 0x08, 0x38, 0xc1, 0xd1, 0xd9},
212     {0xff, 0x0d, 0x23, 0xdc, 0xc6, 0xcb}, {0xfd, 0x0e, 0x2a,
0xd7, 0xcb, 0xc5},
213     {0xf3, 0x07, 0x15, 0xe6, 0xe8, 0xef}, {0xf1, 0x04, 0x1c, 0xed,
0xe5, 0xe1},
214     {0xf7, 0x01, 0x07, 0xf0, 0xf2, 0xf3}, {0xf5, 0x02, 0x0e, 0xfb,
0xff, 0xfd},
215     {0xeb, 0x13, 0x79, 0x92, 0xb4, 0xa7},
{0xe9, 0x10, 0x70, 0x99, 0xb9, 0xa9},
216     {0xef, 0x15, 0x6b, 0x84, 0xae, 0xbb}, {0xed, 0x16, 0x62, 0x8f,
0xa3, 0xb5},
217     {0xe3, 0x1f, 0x5d, 0xbe, 0x80, 0x9f}, {0xe1, 0x1c,
0x54, 0xb5, 0x8d, 0x91},
218     {0xe7, 0x19, 0x4f, 0xa8, 0x9a, 0x83}, {0xe5, 0x1a,
0x46, 0xa3, 0x97, 0x8d}
219 };
220
221 /***** FUNCTION DEFINITIONS *****/
222 // XORs the in and out buffers, storing the result in out.
Length is in bytes.
223 void xor_buf(const BYTE in[], BYTE out[], size_t len)
224 {
225     size_t idx;
226     for (idx = 0; idx < len; idx++)
227         out[idx] ^= in[idx];
228 }
229
230 /*****
231 * AES - CBC
232 *****/
233
234 int aes_encrypt_cbc(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
235 {
236     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
237     int blocks, idx;
238     if (in_len % AES_BLOCK_SIZE != 0)
239         return(FALSE);
240     blocks = in_len / AES_BLOCK_SIZE;
241     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
242     for (idx = 0; idx < blocks; idx++) {
243         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
244         xor_buf(iv_buf, buf_in, AES_BLOCK_SIZE);
245         aes_encrypt(buf_in, buf_out, key, keysize);
246         memcpy(&out[idx * AES_BLOCK_SIZE], buf_out,
AES_BLOCK_SIZE);
247         memcpy(iv_buf, buf_out, AES_BLOCK_SIZE);
248     }
249     return(TRUE);
250 }
251
252 int aes_encrypt_cbc_mac(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
253 {
254     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
255     int blocks, idx;
256     if (in_len % AES_BLOCK_SIZE != 0)
257         return(FALSE);
258     blocks = in_len / AES_BLOCK_SIZE;
259     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
260     for (idx = 0; idx < blocks; idx++) {

```

```

270         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
271         xor_buf(iv_buf, buf_in, AES_BLOCK_SIZE);
272         aes_encrypt(buf_in, buf_out, key, keysize);
273         memcpy(iv_buf, buf_out, AES_BLOCK_SIZE);
274         // Do not output all encrypted blocks.
275     }
276     memcpy(out, buf_out, AES_BLOCK_SIZE); // Only output
the last block.
277     return(TRUE);
278 }
279
280 int aes_decrypt_cbc(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
281 {
282     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
283     int blocks, idx;
284     if (in_len % AES_BLOCK_SIZE != 0)
285         return(FALSE);
286     blocks = in_len / AES_BLOCK_SIZE;
287     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
288     for (idx = 0; idx < blocks; idx++) {
289         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
290         aes_decrypt(buf_in, buf_out, key, keysize);
291         xor_buf(iv_buf, buf_out, AES_BLOCK_SIZE);
292         memcpy(&out[idx * AES_BLOCK_SIZE], buf_out,
AES_BLOCK_SIZE);
293         memcpy(iv_buf, buf_in, AES_BLOCK_SIZE);
294     }
295     return(TRUE);
296 }
297
298 /*****
299 * AES - CTR
300 *****/
301
302 void increment_iv(BYTE iv[], int counter_size)
303 {
304     int idx;
305     // Use counter_size bytes at the end of the IV as the
big-endian integer to increment.
306     for (idx = AES_BLOCK_SIZE - 1; idx >= AES_BLOCK_SIZE -
counter_size; idx--) {
307         iv[idx]++;
308         if (iv[idx] != 0 || idx == AES_BLOCK_SIZE -
counter_size)
309             break;
310     }
311 }
312
313 // Performs the encryption in-place, the input and output
buffers may be the same.
314 // Input may be an arbitrary length (in bytes).
315 void aes_encrypt_ctr(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
316 {
317     size_t idx = 0, last_block_length;
318     BYTE iv_buf[AES_BLOCK_SIZE], out_buf[AES_BLOCK_SIZE];
319     if (in != out)
320         memcpy(out, in, in_len);
321     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
322     last_block_length = in_len - AES_BLOCK_SIZE;
323     if (in_len > AES_BLOCK_SIZE) {
324         for (idx = 0; idx < last_block_length; idx +=
AES_BLOCK_SIZE) {
325             aes_encrypt(iv_buf, out_buf, key,
keysize);
326             xor_buf(out_buf, &out[idx],
AES_BLOCK_SIZE);
327             increment_iv(iv_buf, AES_BLOCK_SIZE);
328         }
329     }
330     aes_encrypt(iv_buf, out_buf, key, keysize);
331     xor_buf(out_buf, &out[idx], in_len - idx); // Use the
Most Significant bytes.
332 }
333
334 void aes_decrypt_ctr(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
335 {
336     // CTR encryption is its own inverse function.
337     aes_encrypt_ctr(in, in_len, out, key, keysize, iv);
338 }
339
340 /*****
341 * AES - CCM
342 *****/

```

```

354 // out_len = payload_len + assoc_len
355 int aes_encrypt_ccm(const BYTE payload[], WORD payload_len,
const BYTE assoc[], unsigned short assoc_len,
356 const BYTE nonce[], unsigned short
nonce_len, BYTE out[], WORD *out_len,
357 WORD mac_len, const BYTE key_str[], int
keysize)
358 {
359     BYTE temp_iv[AES_BLOCK_SIZE], counter[AES_BLOCK_SIZE],
mac[16], *buf;
360     int end_of_buf, payload_len_store_size;
361     WORD key[60];
362
363     if (mac_len != 4 && mac_len != 6 && mac_len != 8 &&
mac_len != 10 &&
364         mac_len != 12 && mac_len != 14 && mac_len != 16)
365         return(FALSE);
366
367     if (nonce_len < 7 || nonce_len > 13)
368         return(FALSE);
369
370     if (assoc_len > 32768 /* = 2^15 */)
371         return(FALSE);
372
373     buf = (BYTE*)malloc(payload_len + assoc_len + 48 /
*Round both payload and associated data up a block size and add an
extra block."/);
374     if (! buf)
375         return(FALSE);
376
377     // Prepare the key for usage.
378     aes_key_setup(key_str, key, keysize);
379
380     // Format the first block of the formatted data.
381     payload_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
382     ccm_prepare_first_format_blk(buf, assoc_len,
payload_len_store_size, mac_len, nonce, nonce_len);
383     end_of_buf = AES_BLOCK_SIZE;
384
385     // Format the Associated Data, aka, assoc[].
386     ccm_format_assoc_data(buf, &end_of_buf, assoc,
assoc_len);
387
388     // Format the Payload, aka payload[].
389     ccm_format_payload_data(buf, &end_of_buf, payload,
payload_len);
390
391     // Create the first counter block.
392     ccm_prepare_first_ctr_blk(counter, nonce, nonce_len,
payload_len_store_size);
393
394     // Perform the CBC operation with an IV of zeros on the
formatted buffer to calculate the MAC.
395     memset(temp_iv, 0, AES_BLOCK_SIZE);
396
397     aes_encrypt_ccm(buf, end_of_buf, mac, key, keysize,
temp_iv);
398
399     // Copy the Payload and MAC to the output buffer.
400     memcpy(out, payload, payload_len);
401     memcpy(&out[payload_len], mac, mac_len);
402
403     // Encrypt the Payload with CTR mode with a counter
starting at 1.
404     memcpy(temp_iv, counter, AES_BLOCK_SIZE);
405     increment_iv(temp_iv, AES_BLOCK_SIZE - 1 -
mac_len); // Last argument is the byte size of the counting
portion of the counter block. /*BUG?*/
406     aes_encrypt_ctr(out, payload_len, out, key, keysize,
temp_iv);
407
408     // Encrypt the MAC with CTR mode with a counter
starting at 0.
409     aes_encrypt_ctr(&out[payload_len], mac_len,
&out[payload_len], key, keysize, counter);
410
411     free(buf);
412     *out_len = payload_len + mac_len;
413
414     return(TRUE);
415 }
416 // plaintext_len = ciphertext_len - mac_len
417 // Needs a flag for whether the MAC matches.
418 int aes_decrypt_ccm(const BYTE ciphertext[], WORD
ciphertext_len, const BYTE assoc[], unsigned short assoc_len,
419 const BYTE nonce[], unsigned short
nonce_len, BYTE plaintext[], WORD *plaintext_len,
420 WORD mac_len, int *mac_auth, const BYTE
key_str[], int keysize)
421 {
422     BYTE temp_iv[AES_BLOCK_SIZE], counter[AES_BLOCK_SIZE],
mac[16], *buf;
423     int end_of_buf, plaintext_len_store_size;
424     WORD key[60];
425
426     if (ciphertext_len <= mac_len)
427         return(FALSE);
428

```

```

429     buf = (BYTE*)malloc(assoc_len + ciphertext_len /
*ciphertext_len = plaintext_len + mac_len/* + 48);
430     if (! buf)
431         return(FALSE);
432
433     // Prepare the key for usage.
434     aes_key_setup(key_str, key, keysize);
435
436     // Copy the plaintext and MAC to the output buffers.
437     *plaintext_len = ciphertext_len - mac_len;
438     plaintext_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
439     memcpy(plaintext, ciphertext, *plaintext_len);
440     memcpy(mac, &ciphertext[*plaintext_len], mac_len);
441
442     // Prepare the first counter block for use in
decryption.
443     ccm_prepare_first_ctr_blk(counter, nonce, nonce_len,
plaintext_len_store_size);
444
445     // Decrypt the Payload with CTR mode with a counter
starting at 1.
446     memcpy(temp_iv, counter, AES_BLOCK_SIZE);
447     increment_iv(temp_iv, AES_BLOCK_SIZE - 1 -
mac_len); // (AES_BLOCK_SIZE - 1 - mac_len) is the byte size of
the counting portion of the counter block.
448     aes_decrypt_ctr(plaintext, *plaintext_len, plaintext,
key, keysize, temp_iv);
449
450     // Setting mac_auth to NULL disables the authentication
check.
451     if (mac_auth != NULL) {
452         // Decrypt the MAC with CTR mode with a counter
starting at 0.
453         aes_decrypt_ctr(mac, mac_len, mac, key,
keysize, counter);
454
455         // Format the first block of the formatted
data.
456         plaintext_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
457         ccm_prepare_first_format_blk(buf, assoc_len,
*plaintext_len, plaintext_len_store_size, mac_len, nonce,
nonce_len);
458         end_of_buf = AES_BLOCK_SIZE;
459
460         // Format the Associated Data into the
authentication buffer.
461         ccm_format_assoc_data(buf, &end_of_buf, assoc,
assoc_len);
462
463         // Format the Payload into the authentication
buffer.
464         ccm_format_payload_data(buf, &end_of_buf,
plaintext, *plaintext_len);
465
466         // Perform the CBC operation with an IV of
zeros on the formatted buffer to calculate the MAC.
467         memset(temp_iv, 0, AES_BLOCK_SIZE);
468         aes_encrypt_ccm(buf, end_of_buf, mac_buf,
key, keysize, temp_iv);
469
470         // Compare the calculated MAC against the MAC
embedded in the ciphertext to see if they are the same.
471         if (! memcmp(mac, mac_buf, mac_len)) {
472             *mac_auth = TRUE;
473         }
474         else {
475             *mac_auth = FALSE;
476             memset(plaintext, 0, *plaintext_len);
477         }
478     }
479
480     free(buf);
481
482     return(TRUE);
483 }
484
485 // Creates the first counter block. First byte is flags, then
the nonce, then the incremented part.
486 void ccm_prepare_first_ctr_blk(BYTE counter[], const BYTE
nonce[], int nonce_len, int payload_len_store_size)
487 {
488     memset(counter, 0, AES_BLOCK_SIZE);
489     counter[0] = (payload_len_store_size - 1) & 0x07;
490     memcpy(&counter[1], nonce, nonce_len);
491 }
492
493 void ccm_prepare_first_format_blk(BYTE buf[], int assoc_len,
int payload_len, int payload_len_store_size, int mac_len, const BYTE
nonce[], int nonce_len)
494 {
495     // Set the flags for the first byte of the first block.
496     buf[0] = (((mac_len - 2) / 2) & 0x07) << 3 |
((payload_len_store_size - 1) & 0x07);
497     if (assoc_len > 0)
498         buf[0] += 0x40;
499     // Format the rest of the first block, storing the

```

```

nonce and the size of the payload.
500     memcpy(&buf[1], nonce, nonce_len);
501     memset(&buf[1 + nonce_len], 0, AES_BLOCK_SIZE - 1 -
nonce_len);
502     buf[15] = payload_len & 0x000000FF;
503     buf[14] = (payload_len >> 8) & 0x000000FF;
504 }
505
506 void ccm_format_assoc_data(BYTE buf[], int *end_of_buf, const
BYTE assoc[], int assoc_len)
507 {
508     int pad;
509
510     buf[*end_of_buf + 1] = assoc_len & 0x00FF;
511     buf[*end_of_buf] = (assoc_len >> 8) & 0x00FF;
512     *end_of_buf += 2;
513     memcpy(&buf[*end_of_buf], assoc, assoc_len);
514     *end_of_buf += assoc_len;
515     pad = AES_BLOCK_SIZE - (*end_of_buf %
AES_BLOCK_SIZE); /*BUG?*/
516     memset(&buf[*end_of_buf], 0, pad);
517     *end_of_buf += pad;
518 }
519
520 void ccm_format_payload_data(BYTE buf[], int *end_of_buf,
const BYTE payload[], int payload_len)
521 {
522     int pad;
523
524     memcpy(&buf[*end_of_buf], payload, payload_len);
525     *end_of_buf += payload_len;
526     pad = *end_of_buf % AES_BLOCK_SIZE;
527     if (pad != 0)
528         pad = AES_BLOCK_SIZE - pad;
529     memset(&buf[*end_of_buf], 0, pad);
530     *end_of_buf += pad;
531 }
532
533 /******
534 * AES
535 *****/
536 // KEY EXPANSION
537 // Substitutes a word using the AES S-Box.
538 WORD SubWord(WORD word)
539 {
540     unsigned int result;
541
542     result = (int)aes_sbox[(word >> 4) & 0x0000000F][word &
0x0000000F];
543     result += (int)aes_sbox[(word >> 12) & 0x0000000F][
(word >> 8) & 0x0000000F] << 8;
544     result += (int)aes_sbox[(word >> 20) & 0x0000000F][
(word >> 16) & 0x0000000F] << 16;
545     result += (int)aes_sbox[(word >> 28) & 0x0000000F][
(word >> 24) & 0x0000000F] << 24;
546     return(result);
547 }
548
549 // Performs the action of generating the keys that will be
used in every round of
550 // encryption. "key" is the user-supplied input key, "w" is
the output key schedule,
551 // "keysize" is the length in bits of "key", must be 128, 192,
or 256.
552 void aes_key_setup(const BYTE key[], WORD w[], int keysize)
553 {
554     int Nb=4, Nr, Nk, idx;
555     WORD
temp, Rcon[]={0x01000000, 0x02000000, 0x04000000, 0x08000000, 0x10000000,
0x20000000,
556 0x40000000, 0x80000000, 0x1b000000, 0x36000000, 0x6c000000, 0xd8000000,
0xab000000, 0x4d000000, 0x9a000000};
557
558     switch (keysize) {
559         case 128: Nr = 10; Nk = 4; break;
560         case 192: Nr = 12; Nk = 6; break;
561         case 256: Nr = 14; Nk = 8; break;
562         default: return;
563     }
564
565     for (idx=0; idx < Nk; ++idx) {
566         w[idx] = ((key[4 * idx] << 24) | ((key[4 * idx +
1] << 16) |
567         ((key[4 * idx + 2] << 8) |
568         ((key[4 * idx + 3])));
569
570         for (idx = Nk; idx < Nb * (Nr+1); ++idx) {
571             temp = w[idx - 1];
572             if ((idx % Nk) == 0)
573                 temp = SubWord(KE_ROTWORD(temp)) ^
Rcon[(idx-1)/Nk];
574             else if (Nk > 6 && (idx % Nk) == 4)
575                 temp = SubWord(temp);
576         }
577     }

```

```

578         w[idx] = w[idx-Nk] ^ temp;
579     }
580 }
581
582 // ADD ROUND KEY
583 // Performs the AddRoundKey step. Each round has its own pre-
generated 16-byte key in the
584 // form of 4 integers (the "w" array). Each integer is XOR'd
by one column of the state.
585 // Also performs the job of InvAddRoundKey(); since the
function is a simple XOR process,
586 // it is its own inverse.
587 void AddRoundKey(BYTE state[4][4], const WORD w[])
588 {
589     BYTE subkey[4];
590
591     // memcpy(subkey, &w[idx], 4); // Not accurate for big
endian machines
592
593     // Subkey 1
594     subkey[0] = w[0] >> 24;
595     subkey[1] = w[0] >> 16;
596     subkey[2] = w[0] >> 8;
597     subkey[3] = w[0];
598     state[0][0] ^= subkey[0];
599     state[1][0] ^= subkey[1];
600     state[2][0] ^= subkey[2];
601     state[3][0] ^= subkey[3];
602
603     // Subkey 2
604     subkey[0] = w[1] >> 24;
605     subkey[1] = w[1] >> 16;
606     subkey[2] = w[1] >> 8;
607     subkey[3] = w[1];
608     state[0][1] ^= subkey[0];
609     state[1][1] ^= subkey[1];
610     state[2][1] ^= subkey[2];
611     state[3][1] ^= subkey[3];
612
613     // Subkey 3
614     subkey[0] = w[2] >> 24;
615     subkey[1] = w[2] >> 16;
616     subkey[2] = w[2] >> 8;
617     subkey[3] = w[2];
618     state[0][2] ^= subkey[0];
619     state[1][2] ^= subkey[1];
620     state[2][2] ^= subkey[2];
621     state[3][2] ^= subkey[3];
622
623     // Subkey 4
624     subkey[0] = w[3] >> 24;
625     subkey[1] = w[3] >> 16;
626     subkey[2] = w[3] >> 8;
627     subkey[3] = w[3];
628     state[0][3] ^= subkey[0];
629     state[1][3] ^= subkey[1];
630     state[2][3] ^= subkey[2];
631     state[3][3] ^= subkey[3];
632 }
633
634 // (Inv)SubBytes
635 // Performs the SubBytes step. All bytes in the state are
substituted with a
636 // pre-calculated value from a lookup table.
637 void SubBytes(BYTE state[4][4])
638 {
639     state[0][0] = aes_sbox[state[0][0] >> 4][state[0][0] &
0x0F];
640     state[0][1] = aes_sbox[state[0][1] >> 4][state[0][1] &
0x0F];
641     state[0][2] = aes_sbox[state[0][2] >> 4][state[0][2] &
0x0F];
642     state[0][3] = aes_sbox[state[0][3] >> 4][state[0][3] &
0x0F];
643     state[1][0] = aes_sbox[state[1][0] >> 4][state[1][0] &
0x0F];
644     state[1][1] = aes_sbox[state[1][1] >> 4][state[1][1] &
0x0F];
645     state[1][2] = aes_sbox[state[1][2] >> 4][state[1][2] &
0x0F];
646     state[1][3] = aes_sbox[state[1][3] >> 4][state[1][3] &
0x0F];
647     state[2][0] = aes_sbox[state[2][0] >> 4][state[2][0] &
0x0F];
648     state[2][1] = aes_sbox[state[2][1] >> 4][state[2][1] &
0x0F];
649     state[2][2] = aes_sbox[state[2][2] >> 4][state[2][2] &
0x0F];
650     state[2][3] = aes_sbox[state[2][3] >> 4][state[2][3] &
0x0F];
651     state[3][0] = aes_sbox[state[3][0] >> 4][state[3][0] &
0x0F];
652     state[3][1] = aes_sbox[state[3][1] >> 4][state[3][1] &
0x0F];
653     state[3][2] = aes_sbox[state[3][2] >> 4][state[3][2] &
0x0F];
654     state[3][3] = aes_sbox[state[3][3] >> 4][state[3][3] &
0x0F];
655 }

```

```

660
661 void InvSubBytes(BYTE state[][4])
662 {
663     state[0][0] = aes_invbox[state[0][0] >> 4][state[0][0] &
0x0F];
664     state[0][1] = aes_invbox[state[0][1] >> 4][state[0][1] &
0x0F];
665     state[0][2] = aes_invbox[state[0][2] >> 4][state[0][2] &
0x0F];
666     state[0][3] = aes_invbox[state[0][3] >> 4][state[0][3] &
0x0F];
667     state[1][0] = aes_invbox[state[1][0] >> 4][state[1][0] &
0x0F];
668     state[1][1] = aes_invbox[state[1][1] >> 4][state[1][1] &
0x0F];
669     state[1][2] = aes_invbox[state[1][2] >> 4][state[1][2] &
0x0F];
670     state[1][3] = aes_invbox[state[1][3] >> 4][state[1][3] &
0x0F];
671     state[2][0] = aes_invbox[state[2][0] >> 4][state[2][0] &
0x0F];
672     state[2][1] = aes_invbox[state[2][1] >> 4][state[2][1] &
0x0F];
673     state[2][2] = aes_invbox[state[2][2] >> 4][state[2][2] &
0x0F];
674     state[2][3] = aes_invbox[state[2][3] >> 4][state[2][3] &
0x0F];
675     state[3][0] = aes_invbox[state[3][0] >> 4][state[3][0] &
0x0F];
676     state[3][1] = aes_invbox[state[3][1] >> 4][state[3][1] &
0x0F];
677     state[3][2] = aes_invbox[state[3][2] >> 4][state[3][2] &
0x0F];
678     state[3][3] = aes_invbox[state[3][3] >> 4][state[3][3] &
0x0F];
679 }
680 //////////////////////////////////////////////////
681 // (Inv)ShiftRows
682 //////////////////////////////////////////////////
683
684 // Performs the ShiftRows step. All rows are shifted
685 // cylindrically to the left.
686 void ShiftRows(BYTE state[][4])
687 {
688     int t;
689     // Shift left by 1
690     t = state[1][0];
691     state[1][0] = state[1][1];
692     state[1][1] = state[1][2];
693     state[1][2] = state[1][3];
694     state[1][3] = t;
695     // Shift left by 2
696     t = state[2][0];
697     state[2][0] = state[2][2];
698     state[2][2] = t;
699     t = state[2][1];
700     state[2][1] = state[2][3];
701     state[2][3] = t;
702     // Shift left by 3
703     t = state[3][0];
704     state[3][0] = state[3][3];
705     state[3][3] = state[3][2];
706     state[3][2] = state[3][1];
707     state[3][1] = t;
708 }
709
710 // All rows are shifted cylindrically to the right.
711 void InvShiftRows(BYTE state[][4])
712 {
713     int t;
714     // Shift right by 1
715     t = state[1][3];
716     state[1][3] = state[1][2];
717     state[1][2] = state[1][1];
718     state[1][1] = state[1][0];
719     state[1][0] = t;
720     // Shift right by 2
721     t = state[2][3];
722     state[2][3] = state[2][1];
723     state[2][1] = t;
724     t = state[2][2];
725     state[2][2] = state[2][0];
726     state[2][0] = t;

```

```

728     state[2][0] = t;
729     // Shift right by 3
730     t = state[3][3];
731     state[3][3] = state[3][0];
732     state[3][0] = state[3][1];
733     state[3][1] = state[3][2];
734     state[3][2] = t;
735 }
736
737 //////////////////////////////////////////////////
738 // (Inv)MixColumns
739 //////////////////////////////////////////////////
740
741 // Performs the MixColumns step. The state is multiplied by
742 // itself using matrix
743 // multiplication in a Galios Field 2^8. All multiplication is
744 // pre-computed in a table.
745 // Addition is equivalent to XOR. (Must always make a copy of
746 // the column as the original
747 // values will be destroyed.)
748 void MixColumns(BYTE state[][4])
749 {
750     BYTE col[4];
751     // Column 1
752     col[0] = state[0][0];
753     col[1] = state[1][0];
754     col[2] = state[2][0];
755     col[3] = state[3][0];
756     state[0][0] = gf_mul[col[0]][0];
757     state[0][0] ^= gf_mul[col[1]][1];
758     state[0][0] ^= col[2];
759     state[0][0] ^= col[3];
760     state[1][0] = col[0];
761     state[1][0] ^= gf_mul[col[1]][0];
762     state[1][0] ^= gf_mul[col[2]][1];
763     state[1][0] ^= col[3];
764     state[2][0] = col[0];
765     state[2][0] ^= gf_mul[col[1]][0];
766     state[2][0] ^= gf_mul[col[2]][1];
767     state[2][0] ^= col[3];
768     state[3][0] = col[0];
769     state[3][0] ^= gf_mul[col[1]][0];
770     state[3][0] ^= gf_mul[col[2]][1];
771     state[3][0] ^= col[3];
772     // Column 2
773     col[0] = state[0][1];
774     col[1] = state[1][1];
775     col[2] = state[2][1];
776     col[3] = state[3][1];
777     state[0][1] = gf_mul[col[0]][0];
778     state[0][1] ^= gf_mul[col[1]][1];
779     state[0][1] ^= col[2];
780     state[0][1] ^= col[3];
781     state[1][1] = col[0];
782     state[1][1] ^= gf_mul[col[1]][0];
783     state[1][1] ^= gf_mul[col[2]][1];
784     state[1][1] ^= col[3];
785     state[2][1] = col[0];
786     state[2][1] ^= gf_mul[col[1]][0];
787     state[2][1] ^= gf_mul[col[2]][1];
788     state[2][1] ^= col[3];
789     state[3][1] = col[0];
790     state[3][1] ^= gf_mul[col[1]][0];
791     state[3][1] ^= gf_mul[col[2]][1];
792     state[3][1] ^= col[3];
793     // Column 3
794     col[0] = state[0][2];
795     col[1] = state[1][2];
796     col[2] = state[2][2];
797     col[3] = state[3][2];
798     state[0][2] = gf_mul[col[0]][0];
799     state[0][2] ^= gf_mul[col[1]][1];
800     state[0][2] ^= col[2];
801     state[0][2] ^= col[3];
802     state[1][2] = col[0];
803     state[1][2] ^= gf_mul[col[1]][0];
804     state[1][2] ^= gf_mul[col[2]][1];
805     state[1][2] ^= col[3];
806     state[2][2] = col[0];
807     state[2][2] ^= gf_mul[col[1]][0];
808     state[2][2] ^= gf_mul[col[2]][1];
809     state[2][2] ^= col[3];
810     state[3][2] = col[0];
811     state[3][2] ^= gf_mul[col[1]][0];
812     state[3][2] ^= gf_mul[col[2]][1];
813     state[3][2] ^= col[3];
814     // Column 4
815     col[0] = state[0][3];
816     col[1] = state[1][3];
817     col[2] = state[2][3];
818     col[3] = state[3][3];
819     state[0][3] = gf_mul[col[0]][0];
820     state[0][3] ^= gf_mul[col[1]][1];
821     state[0][3] ^= col[2];
822     state[0][3] ^= col[3];
823     state[1][3] = col[0];
824     state[1][3] ^= gf_mul[col[1]][0];
825     state[1][3] ^= gf_mul[col[2]][1];
826     state[1][3] ^= col[3];

```

```

825     state[2][3] = col[0];
826     state[2][3] ^= col[1];
827     state[2][3] ^= gf_mul[col[2]][0];
828     state[2][3] ^= gf_mul[col[3]][1];
829     state[3][3] = gf_mul[col[0]][1];
830     state[3][3] ^= col[1];
831     state[3][3] ^= col[2];
832     state[3][3] ^= gf_mul[col[3]][0];
833 }
834
835 void InvMixColumns(BYTE state[][4])
836 {
837     BYTE col[4];
838
839     // Column 1
840     col[0] = state[0][0];
841     col[1] = state[1][0];
842     col[2] = state[2][0];
843     col[3] = state[3][0];
844     state[0][0] = gf_mul[col[0]][5];
845     state[0][0] ^= gf_mul[col[1]][3];
846     state[0][0] ^= gf_mul[col[2]][4];
847     state[0][0] ^= gf_mul[col[3]][2];
848     state[1][0] = gf_mul[col[0]][2];
849     state[1][0] ^= gf_mul[col[1]][5];
850     state[1][0] ^= gf_mul[col[2]][3];
851     state[1][0] ^= gf_mul[col[3]][4];
852     state[2][0] = gf_mul[col[0]][4];
853     state[2][0] ^= gf_mul[col[1]][2];
854     state[2][0] ^= gf_mul[col[2]][5];
855     state[2][0] ^= gf_mul[col[3]][3];
856     state[3][0] = gf_mul[col[0]][3];
857     state[3][0] ^= gf_mul[col[1]][4];
858     state[3][0] ^= gf_mul[col[2]][2];
859     state[3][0] ^= gf_mul[col[3]][5];
860
861     // Column 2
862     col[0] = state[0][1];
863     col[1] = state[1][1];
864     col[2] = state[2][1];
865     col[3] = state[3][1];
866     state[0][1] = gf_mul[col[0]][5];
867     state[0][1] ^= gf_mul[col[1]][3];
868     state[0][1] ^= gf_mul[col[2]][4];
869     state[0][1] ^= gf_mul[col[3]][2];
870     state[1][1] = gf_mul[col[0]][2];
871     state[1][1] ^= gf_mul[col[1]][5];
872     state[1][1] ^= gf_mul[col[2]][3];
873     state[1][1] ^= gf_mul[col[3]][4];
874     state[2][1] = gf_mul[col[0]][4];
875     state[2][1] ^= gf_mul[col[1]][2];
876     state[2][1] ^= gf_mul[col[2]][5];
877     state[2][1] ^= gf_mul[col[3]][3];
878     state[3][1] = gf_mul[col[0]][3];
879     state[3][1] ^= gf_mul[col[1]][4];
880     state[3][1] ^= gf_mul[col[2]][2];
881     state[3][1] ^= gf_mul[col[3]][5];
882
883     // Column 3
884     col[0] = state[0][2];
885     col[1] = state[1][2];
886     col[2] = state[2][2];
887     col[3] = state[3][2];
888     state[0][2] = gf_mul[col[0]][5];
889     state[0][2] ^= gf_mul[col[1]][3];
890     state[0][2] ^= gf_mul[col[2]][4];
891     state[0][2] ^= gf_mul[col[3]][2];
892     state[1][2] = gf_mul[col[0]][2];
893     state[1][2] ^= gf_mul[col[1]][5];
894     state[1][2] ^= gf_mul[col[2]][3];
895     state[1][2] ^= gf_mul[col[3]][4];
896     state[2][2] = gf_mul[col[0]][4];
897     state[2][2] ^= gf_mul[col[1]][2];
898     state[2][2] ^= gf_mul[col[2]][5];
899     state[2][2] ^= gf_mul[col[3]][3];
900     state[3][2] = gf_mul[col[0]][3];
901     state[3][2] ^= gf_mul[col[1]][4];
902     state[3][2] ^= gf_mul[col[2]][2];
903     state[3][2] ^= gf_mul[col[3]][5];
904
905     // Column 4
906     col[0] = state[0][3];
907     col[1] = state[1][3];
908     col[2] = state[2][3];
909     col[3] = state[3][3];
910     state[0][3] = gf_mul[col[0]][5];
911     state[0][3] ^= gf_mul[col[1]][3];
912     state[0][3] ^= gf_mul[col[2]][4];
913     state[0][3] ^= gf_mul[col[3]][2];
914     state[1][3] = gf_mul[col[0]][2];
915     state[1][3] ^= gf_mul[col[1]][5];
916     state[1][3] ^= gf_mul[col[2]][3];
917     state[1][3] ^= gf_mul[col[3]][4];
918     state[2][3] = gf_mul[col[0]][4];
919     state[2][3] ^= gf_mul[col[1]][2];
920     state[2][3] ^= gf_mul[col[2]][5];
921     state[2][3] ^= gf_mul[col[3]][3];
922     state[3][3] = gf_mul[col[0]][3];
923     state[3][3] ^= gf_mul[col[1]][4];
924     state[3][3] ^= gf_mul[col[2]][2];
925     state[3][3] ^= gf_mul[col[3]][5];
926 }
927
928 // (En/De)Crypt
929 void aes_encrypt(const BYTE in[], BYTE out[], const WORD
key[], int keysize)
930 {
931     BYTE state[4][4];
932
933     // Copy input array (should be 16 bytes long) to a
matrix (sequential bytes are ordered
934     // by row, not col) called "state" for processing.
935     // *** Implementation note: The official AES
documentation references the state by
936     // column, then row. Accessing an element in C requires
row then column. Thus, all state
937     // references in AES must have the column and row
indexes reversed for C implementation.
938     state[0][0] = in[0];
939     state[1][0] = in[1];
940     state[2][0] = in[2];
941     state[3][0] = in[3];
942     state[0][1] = in[4];
943     state[1][1] = in[5];
944     state[2][1] = in[6];
945     state[3][1] = in[7];
946     state[0][2] = in[8];
947     state[1][2] = in[9];
948     state[2][2] = in[10];
949     state[3][2] = in[11];
950     state[0][3] = in[12];
951     state[1][3] = in[13];
952     state[2][3] = in[14];
953     state[3][3] = in[15];
954
955     // Perform the necessary number of rounds. The round
key is added first.
956     // The last round does not perform the MixColumns step.
957     AddRoundKey(state, &key[0]);
958     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[4]);
959     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[8]);
960     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[12]);
961     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[16]);
962     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[20]);
963     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[24]);
964     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[28]);
965     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[32]);
966     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[36]);
967     if (keysize != 128) {
968         SubBytes(state); ShiftRows(state);
MixColumns(state); AddRoundKey(state, &key[40]);
969         SubBytes(state); ShiftRows(state);
MixColumns(state); AddRoundKey(state, &key[44]);
970         if (keysize != 192) {
971             SubBytes(state); ShiftRows(state);
MixColumns(state); AddRoundKey(state, &key[48]);
972             SubBytes(state); ShiftRows(state);
MixColumns(state); AddRoundKey(state, &key[52]);
973             SubBytes(state); ShiftRows(state);
AddRoundKey(state, &key[56]);
974         }
975         else {
976             SubBytes(state); ShiftRows(state);
AddRoundKey(state, &key[48]);
977         }
978     }
979     else {
980         SubBytes(state); ShiftRows(state);
AddRoundKey(state, &key[40]);
981     }
982
983     // Copy the state to the output array.
984     out[0] = state[0][0];
985     out[1] = state[1][0];
986     out[2] = state[2][0];
987     out[3] = state[3][0];
988     out[4] = state[0][1];
989     out[5] = state[1][1];
990     out[6] = state[2][1];
991     out[7] = state[3][1];
992     out[8] = state[0][2];
993     out[9] = state[1][2];
994     out[10] = state[2][2];
995     out[11] = state[3][2];
996     out[12] = state[0][3];
997     out[13] = state[1][3];
998     out[14] = state[2][3];
999     out[15] = state[3][3];
1000 }
1001
1002 void aes_decrypt(const BYTE in[], BYTE out[], const WORD
key[], int keysize)

```

```

1003 {
1004     BYTE state[4][4];
1005
1006     // Copy the input to the state.
1007     state[0][0] = in[0];
1008     state[1][0] = in[1];
1009     state[2][0] = in[2];
1010     state[3][0] = in[3];
1011     state[0][1] = in[4];
1012     state[1][1] = in[5];
1013     state[2][1] = in[6];
1014     state[3][1] = in[7];
1015     state[0][2] = in[8];
1016     state[1][2] = in[9];
1017     state[2][2] = in[10];
1018     state[3][2] = in[11];
1019     state[0][3] = in[12];
1020     state[1][3] = in[13];
1021     state[2][3] = in[14];
1022     state[3][3] = in[15];
1023
1024     // Perform the necessary number of rounds. The round
key is added first.
1025     // The last round does not perform the MixColumns step.
1026     if (keysize > 128) {
1027         if (keysize > 192) {
1028             AddRoundKey(state, &key[56]);
1029
1030             InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[52]);
1031             InvMixColumns(state);
1032         }
1033         else {
1034             AddRoundKey(state, &key[48]);
1035         }
1036
1037         InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[44]);
1038         InvMixColumns(state);
1039     }
1040     else {
1041         AddRoundKey(state, &key[40]);
1042
1043         InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[36]);
1044         InvMixColumns(state);
1045     }
1046     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[32]);
1047     InvMixColumns(state);
1048     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[28]);
1049     InvMixColumns(state);
1050     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[24]);
1051     InvMixColumns(state);
1052     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[20]);
1053     InvMixColumns(state);
1054     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[16]);
1055     InvMixColumns(state);
1056     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[12]);
1057     InvMixColumns(state);
1058     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[8]);
1059     InvMixColumns(state);
1060     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[4]);
1061     InvMixColumns(state);
1062     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[0]);
1063
1064     // Copy the state to the output array.
1065     out[0] = state[0][0];
1066     out[1] = state[1][0];
1067     out[2] = state[2][0];
1068     out[3] = state[3][0];
1069     out[4] = state[0][1];
1070     out[5] = state[1][1];
1071     out[6] = state[2][1];
1072     out[7] = state[3][1];
1073     out[8] = state[0][2];
1074     out[9] = state[1][2];
1075     out[10] = state[2][2];
1076     out[11] = state[3][2];
1077     out[12] = state[0][3];
1078     out[13] = state[1][3];
1079     out[14] = state[2][3];
1080     out[15] = state[3][3];
1081 }
1082
1083 // *****
1084 ** AES DEBUGGING FUNCTIONS
1085 *****
1086 //
1087 // This prints the "state" grid as a linear hex string.

```

```

1076 void print_state(BYTE state[][4])
1077 {
1078     int idx, idx2;
1079
1080     for (idx=0; idx < 4; idx++)
1081         for (idx2=0; idx2 < 4; idx2++)
1082             printf("%02x", state[idx2][idx]);
1083     printf("\n");
1084 }
1085
1086 // This prints the key (4 consecutive ints) used for a given
round as a linear hex string.
1087 void print_rnd_key(WORD key[])
1088 {
1089     int idx;
1090
1091     for (idx=0; idx < 4; idx++)
1092         printf("%08x", key[idx]);
1093     printf("\n");
1094 }
1095 */

```

A.5 riscv-software-src_riscv-isa-sim/riscv/insns/add.h (1줄, 32B)

```
1 WRITE_RD(sext_xlen(RS1 + RS2));
```

A.6 riscv-software-src_riscv-isa-sim/riscv/insns/mulh.h (5줄, 146B)

```

1 require_either_extension('M', EXT_ZMMUL);
2 if (xlen == 64)
3     WRITE_RD(mulh(RS1, RS2));
4 else
5     WRITE_RD(sext32((sext32(RS1) * sext32(RS2)) >> 32));

```

A.7 riscv-software-src_riscv-isa-sim/riscv/insns/fadd_d.h (5줄, 140B)

```

1 require_either_extension('D', EXT_ZDINX);
2 require_fp;
3 softfloat_roundingMode = RM;
4 WRITE_FRD_D(f64_add(FRS1_D, FRS2_D));
5 set_fp_exceptions;

```

A.8 riscv-software-src_riscv-isa-sim/riscv/decode.h (257줄, 7,919B)

```

1 // See LICENSE for license details.
2
3 #ifndef _RISCV_DECODE_H
4 #define _RISCV_DECODE_H
5
6 #if (-1 != ~0) || ((-1 >> 1) != -1)
7 # error spike requires a two's-complement c++ implementation
8 #endif
9
10 #include "../softfloat/softfloat_types.h"
11 #include <algorithm>
12 #include <array>
13 #include <stdint>
14 #include <string>
15 #include <strings.h>
16 #include <inttypes>
17 #include <type_traits>
18
19 typedef int64_t sreg_t;
20 typedef uint64_t reg_t;
21 typedef float128_t freg_t;
22
23 const int NXPR = 32;
24 const int NFPR = 32;
25 const int NVPR = 32;
26 const int NCSR = 4096;
27
28 #define X_RA 1
29 #define X_SP 2
30 #define X_TO 5
31 #define X_S0 8
32 #define X_A0 10
33 #define X_A1 11
34 #define X_Sn 16
35
36 #define VCSR_VXRM_SHIFT 1
37 #define VCSR_VXRM (0x3 << VCSR_VXRM_SHIFT)
38
39 #define VCSR_VXSAT_SHIFT 0
40 #define VCSR_VXSAT (0x1 << VCSR_VXSAT_SHIFT)
41
42 #define FP_RD_NE 0
43 #define FP_RD_0 1
44 #define FP_RD_DN 2
45 #define FP_RD_UP 3
46 #define FP_RD_NMM 4
47
48 #define FSR_RD_SHIFT 5
49 #define FSR_RD (0x7 << FSR_RD_SHIFT)

```

```

50
51 #define FPEXC_NX 0x01
52 #define FPEXC_UF 0x02
53 #define FPEXC_OF 0x04
54 #define FPEXC_DZ 0x08
55 #define FPEXC_NV 0x10
56
57 #define FSR_AEXC_SHIFT 0
58 #define FSR_NVA (FPEXC_NV << FSR_AEXC_SHIFT)
59 #define FSR_OFA (FPEXC_OF << FSR_AEXC_SHIFT)
60 #define FSR_UFA (FPEXC_UF << FSR_AEXC_SHIFT)
61 #define FSR_DZA (FPEXC_DZ << FSR_AEXC_SHIFT)
62 #define FSR_NXA (FPEXC_NX << FSR_AEXC_SHIFT)
63 #define FSR_AEXC (FSR_NVA | FSR_OFA | FSR_UFA | FSR_DZA |
FSR_NXA)
64
65 #define insn_length(x) \
66 ((x) & 0x03) < 0x03 ? 2 : \
67 ((x) & 0x1f) < 0x1f ? 4 : \
68 ((x) & 0x3f) < 0x3f ? 6 : \
69 8)
70 #define MAX_INSN_LENGTH 8
71 #define PC_ALIGN 2
72
73 #define Sn(n) ((n) < 2 ? X_S0 + (n) : X_Sn + (n))
74
75 typedef uint64_t insn_bits_t;
76 class insn_t
77 {
78 public:
79     insn_t() = default;
80     insn_t(insn_bits_t bits) : b(bits) {}
81     insn_bits_t bits() { return b; }
82     int length() { return insn_length(b); }
83     [[maybe_unused]] int64_t opcode() { return x(0, 7); }
84     [[maybe_unused]] int64_t funct7() { return x(25, 7); }
85     [[maybe_unused]] int64_t funct3() { return x(12, 3); }
86     [[maybe_unused]] int64_t funct2() { return x(25, 2); }
87     int64_t i_imm() { return xs(20, 12); }
88     int64_t shamt() { return x(20, 6); }
89     int64_t s_imm() { return x(7, 5) + (xs(25, 7) << 5); }
90
91     int64_t sb_imm() { return (x(8, 4) << 1) + (x(25, 6) << 5) +
(x(7, 1) << 11) + (imm_sign() << 12); }
92     int64_t u_imm() { return xs(12, 20) << 12; }
93     int64_t uj_imm() { return (x(21, 10) << 1) + (x(20, 1) <<
11) + (x(12, 8) << 12) + (imm_sign() << 20); }
94     uint64_t rd() { return x(7, 5); }
95     uint64_t rs1() { return x(15, 5); }
96     uint64_t rs2() { return x(20, 5); }
97     uint64_t rs3() { return x(27, 5); }
98     uint64_t rm() { return x(12, 3); }
99     uint64_t csr() { return x(20, 12); }
100    uint64_t iorw() { return x(20, 8); }
101
102    uint64_t bs() { return x(30, 2); } // Crypto ISE - SM4/AES32
byte select.
103    uint64_t rcon() { return x(20, 4); } // Crypto ISE - AES64
round const.
104    uint64_t rd_p() { return x(8, 4) * 2; }
105    uint64_t rs1_p() { return x(16, 4) * 2; }
106    uint64_t rs2_p() { return x(21, 4) * 2; }
107
108    [[maybe_unused]] int64_t rvc_opcode() { return x(0, 2); }
109    int64_t rvc_imm() { return x(2, 5) + (xs(12, 1) << 5); }
110    int64_t rvc_zimm() { return x(2, 5) + (x(12, 1) << 5); }
111    int64_t rvc_addi4spn_imm() { return x(6, 1) << 2) + (x(5,
1) << 3) + (x(11, 2) << 4) + (x(7, 4) << 6); }
112    int64_t rvc_addi16sp_imm() { return (x(6, 1) << 4) + (x(2,
1) << 5) + (x(5, 1) << 6) + (x(3, 2) << 7) + (xs(12, 1) << 9); }
113    int64_t rvc_lwsp_imm() { return (x(4, 3) << 2) + (x(12, 1)
<< 5) + (x(2, 2) << 6); }
114    int64_t rvc_ldsp_imm() { return (x(5, 2) << 3) + (x(12, 1)
<< 5) + (x(2, 3) << 6); }
115
116    int64_t rvc_swsb_imm() { return (x(9, 4) << 2) + (x(7, 2) <<
6); }
117    int64_t rvc_sdsp_imm() { return (x(10, 3) << 3) + (x(7, 3)
<< 6); }
118    int64_t rvc_lw_imm() { return (x(6, 1) << 2) + (x(10, 3) <<
3) + (x(5, 1) << 6); }
119    int64_t rvc_ld_imm() { return (x(10, 3) << 3) + (x(5, 2) <<
6); }
120    int64_t rvc_j_imm() { return (x(3, 3) << 1) + (x(11, 1) <<
4) + (x(2, 1) << 5) + (x(7, 1) << 6) + (x(6, 1) << 7) + (x(9, 2) <<
8) + (x(8, 1) << 10) + (xs(12, 1) << 11); }
121    int64_t rvc_b_imm() { return (x(3, 2) << 1) + (x(10, 2) <<
3) + (x(2, 1) << 5) + (x(5, 2) << 6) + (xs(12, 1) << 8); }
122    int64_t rvc_simm3() { return x(10, 3); }
123    uint64_t rvc_rd() { return rd(); }
124    uint64_t rvc_rs1() { return rs1(); }
125    uint64_t rvc_rs2() { return rs2(); }
126    uint64_t rvc_rs3() { return rs3(); }
127    uint64_t rvc_rs1s() { return 8 + x(7, 3); }
128    uint64_t rvc_rs2s() { return 8 + x(2, 3); }
129
130    uint64_t rvc_lbimm() { return (x(5, 1) << 1) + x(6, 1); }
131    uint64_t rvc_lhimm() { return (x(5, 1) << 1); }
132
133    uint64_t rvc_r1sc() { return x(7, 3); }
134    uint64_t rvc_r2sc() { return x(2, 3); }
135    uint64_t rvc_rlist() { return x(4, 4); }
136
137    uint64_t rvc_spimm() { return x(2, 2) << 4; }
138
139    uint64_t rvc_index() { return x(2, 8); }
140
141    uint64_t v_vm() { return x(25, 1); }
142    uint64_t v_wd() { return x(26, 1); }
143    uint64_t v_nf() { return x(29, 3); }
144    uint64_t v_simm5() { return xs(15, 5); }
145    uint64_t v_zimm5() { return x(15, 5); }
146    uint64_t v_zimm10() { return x(20, 10); }
147    uint64_t v_zimm11() { return x(20, 11); }
148    uint64_t v_lmul() { return x(20, 2); }
149    uint64_t v_frac_lmul() { return x(22, 1); }
150    uint64_t v_sew() { return 1 << (x(23, 3) + 3); }
151    uint64_t v_width() { return x(12, 3); }
152    uint64_t v_mop() { return x(26, 2); }
153    uint64_t v_lumop() { return x(20, 5); }
154    uint64_t v_sumop() { return x(20, 5); }
155    uint64_t v_vta() { return x(26, 1); }
156    uint64_t v_vma() { return x(27, 1); }
157    uint64_t v_mew() { return x(28, 1); }
158    uint64_t v_zimm6() { return x(15, 5) + (x(26, 1) << 5); }
159
160    uint64_t p_imm8() { return x(16, 8); }
161    uint64_t p_imm10csl() { return x(16, 9) + (x(15, 1) << 9); }
162    uint64_t p_imm10csr() { return (x(24, 1) << 6) + (x(15, 9)
<< 7); }
163
164    uint64_t p_imm10csr() { return (x(24, 1) << 22) + (x(15, 9)
<< 23); }
165
166    uint64_t shamtd() { return x(20, 6); }
167    uint64_t shamtw() { return x(20, 5); }
168    uint64_t shamth() { return x(20, 4); }
169    uint64_t shamtb() { return x(20, 3); }
170
171    uint64_t b_imm5() { return (x(20, 5) == 0) ? -1ul : x(20,
5); }
172
173    uint64_t zcmp_regmask() {
174        unsigned mask = 0;
175        uint64_t rlist = rvc_rlist();
176
177        if (rlist >= 4)
178            mask |= 1U << X_RA;
179
180        for (reg_t i = 5; i <= rlist; i++)
181            mask |= 1U << Sn(i - 5);
182
183        if (rlist == 15)
184            mask |= 1U << Sn(11);
185
186        return mask;
187    }
188
189    uint64_t zcmp_stack_adjustment(int xlen) {
190        reg_t stack_adj_base = 0;
191        switch (rvc_rlist()) {
192        case 15:
193            stack_adj_base += 16;
194            [[fallthrough]];
195        case 14:
196            if (xlen == 64)
197                stack_adj_base += 16;
198            [[fallthrough]];
199        case 13:
200            stack_adj_base += 16;
201            [[fallthrough]];
202        case 12:
203            stack_adj_base += 16;
204            [[fallthrough]];
205        case 11:
206            stack_adj_base += 16;
207            [[fallthrough]];
208        case 10:
209            if (xlen == 64)
210                stack_adj_base += 16;
211            [[fallthrough]];
212        case 9:
213            stack_adj_base += 16;
214            [[fallthrough]];
215        case 8:
216            stack_adj_base += 16;
217            [[fallthrough]];
218        case 7:
219            stack_adj_base += 16;
220            [[fallthrough]];
221        case 6:
222            if (xlen == 64)
223                stack_adj_base += 16;
224            [[fallthrough]];
225        case 5:
226            stack_adj_base += 16;
227            break;
228        case 4:
229            stack_adj_base += 16;
230            break;
231        }
232        return stack_adj_base + rvc_spimm();
233    }
234
235 private:
236     insn_bits_t b;
237     uint64_t x(int lo, int len) { return (b >> lo) &
((insn_bits_t(1) << len) - 1); }
238
239     uint64_t xs(int lo, int len) { return int64_t(b) << (64 - lo -
len) >> (64 - len); }
240     uint64_t imm_sign() { return xs(31, 1); }
241 };

```

```

226 template <class T, size_t N, bool zero_reg>
227 class regfile_t
228 {
229 public:
230     void write(size_t i, T value)
231     {
232         if (!zero_reg || i != 0)
233             data[i] = value;
234     }
235     const T& operator [] (size_t i) const
236     {
237         return data[i];
238     }
239     regfile_t()
240     {
241         reset();
242     }
243     void reset()
244     {
245         memset(data, 0, sizeof(data));
246     }
247 private:
248     T data[N];
249 };
250
251 #define get_field(reg, mask) \
252     (((reg) & (std::remove_cv<decltype(reg)>::type)(mask)) / \
253     ((mask) & ~((mask) << 1)))
254
255 #define set_field(reg, mask, val) \
256     (((reg) & ~(std::remove_cv<decltype(reg)>::type)(mask)) | \
257     ((std::remove_cv<decltype(reg)>::type)(val) * ((mask) & ~((mask) << \
258     1))) & (std::remove_cv<decltype(reg)>::type)(mask)))
259
260 #endif

```

A.9 riscv-software-src_riscv-isa-sim/riscv/processor.h (438줄, 12,234B)

```

1 // See LICENSE for license details.
2 #ifndef _RISC_V_PROCESSOR_H
3 #define _RISC_V_PROCESSOR_H
4
5 #include "decode.h"
6 #include "trap.h"
7 #include "abstract_device.h"
8 #include <string>
9 #include <vector>
10 #include <unordered_map>
11 #include <map>
12 #include <cassert>
13 #include "debug_rom_defines.h"
14 #include "entropy_source.h"
15 #include "csrs.h"
16 #include "isa_parser.h"
17 #include "triggers.h"
18 #include "../fesvr/memif.h"
19 #include "vector_unit.h"
20 #include "imsic.h"
21
22 #define FIRST_HPM_COUNTER 3
23 #define N_HPM_COUNTERS 29
24
25 class processor_t;
26 class mmu_t;
27 typedef reg_t (*insn_func_t)(processor_t*, insn_t, reg_t);
28 class simif_t;
29 class trap_t;
30 class extension_t;
31 class disassembler_t;
32
33 reg_t illegal_instruction(processor_t* p, insn_t insn, reg_t
pc);
34
35 struct insn_desc_t
36 {
37     insn_bits_t match;
38     insn_bits_t mask;
39     insn_func_t fast_rv32i;
40     insn_func_t fast_rv64i;
41     insn_func_t fast_rv32e;
42     insn_func_t fast_rv64e;
43     insn_func_t logged_rv32i;
44     insn_func_t logged_rv64i;
45     insn_func_t logged_rv32e;
46     insn_func_t logged_rv64e;
47
48     insn_func_t func(int xlen, bool rve, bool logged) const
49     {
50         if (logged)
51             if (rve)
52                 return xlen == 64 ? logged_rv64e : logged_rv32e;
53             else
54                 return xlen == 64 ? logged_rv64i : logged_rv32i;
55         else
56             if (rve)
57                 return xlen == 64 ? fast_rv64e : fast_rv32e;
58             else
59                 return xlen == 64 ? fast_rv64i : fast_rv32i;
60     }

```

```

61
62     static const insn_desc_t illegal_instruction;
63 };
64
65 struct opcode_map_entry_t
66 {
67     insn_bits_t match;
68     insn_bits_t mask;
69     insn_func_t func;
70 };
71
72 // regnum, data
73 typedef std::map<reg_t, freg_t> commit_log_reg_t;
74
75 // addr, value, size
76 typedef std::vector<std::tuple<reg_t, uint64_t, uint8_t>>
commit_log_mem_t;
77
78 // architectural state of a RISC-V hart
79 struct state_t
80 {
81     void add_ireg_proxy(sscsrind_reg_csr_t_p ired,
aia_ireg_proxy_csr_t_p aia_proxy);
82     void reset(processor_t* const proc, reg_t max_isa);
83     void add_csr(reg_t addr, const csr_t_p& csr);
84
85     reg_t pc;
86     regfile_t<reg_t, NXPR, true> XPR;
87     regfile_t<freg_t, NFPR, false> FPR;
88
89     // control and status registers
90     std::unordered_map<reg_t, csr_t_p> csrmap;
91     reg_t prv; // TODO: Can this be an enum instead?
92     reg_t prev_prv;
93     bool prv_changed;
94     bool v_changed;
95     bool v;
96     bool prev_v;
97     bool in_wfi;
98     misa_csr_t_p misa;
99     mstatus_csr_t_p mstatus;
100     csr_t_p mstatus;
101     csr_t_p mepc;
102     csr_t_p mtval;
103     csr_t_p mtvec;
104     csr_t_p mcause;
105     smcntxpmf_csr_t_p minstretcfg;
106     smcntxpmf_csr_t_p mcyclecf;
107     wide_counter_csr_t_p minstret;
108     wide_counter_csr_t_p mcycle;
109     mie_csr_t_p mie;
110     mip_csr_t_p mip;
111     csr_t_p nonvirtual_sip;
112     csr_t_p nonvirtual_sie;
113     csr_t_p medeleg;
114     csr_t_p mideleg;
115     csr_t_p mcounteren;
116     csr_t_p mcountinhibit;
117     csr_t_p mevent[N_HPM_COUNTERS];
118     csr_t_p mnstatus;
119     csr_t_p mnepc;
120     csr_t_p scounteren;
121     csr_t_p sepc;
122     csr_t_p stval;
123     csr_t_p stvec;
124     virtualized_csr_t_p satp;
125     csr_t_p scause;
126     csr_t_p scountinhibit;
127
128     // When taking a trap into HS-mode, we must access the
nonvirtualized HS-mode CSRs directly:
129     csr_t_p nonvirtual_stvec;
130     csr_t_p nonvirtual_scause;
131     csr_t_p nonvirtual_stval;
132     csr_t_p nonvirtual_sstval;
133     sstatus_proxy_csr_t_p nonvirtual_sstatus;
134
135     csr_t_p mtval2;
136     csr_t_p mtinst;
137     csr_t_p hstatus;
138     csr_t_p hdeleg;
139     csr_t_p hedeleg;
140     csr_t_p hcounteren;
141     csr_t_p htval;
142     csr_t_p htinst;
143     csr_t_p hgatp;
144     csr_t_p hgeie;
145     csr_t_p hgeip;
146     csr_t_p hip;
147     csr_t_p hie;
148     hvip_csr_t_p hvip;
149     sstatus_csr_t_p sstatus;
150     vsstatus_csr_t_p vsstatus;
151     csr_t_p vstvec;
152     csr_t_p vsepc;
153     csr_t_p vscause;
154     csr_t_p vstval;
155     csr_t_p vsatp;
156
157     csr_t_p dpc;
158     dcsr_csr_t_p dcsr;

```

```

159 csr_t_p tselect;
160 csr_t_p tdata2;
161 csr_t_p tcontrol;
162 csr_t_p scontext;
163 csr_t_p mcontext;
164
165 csr_t_p jvt;
166
167 bool debug_mode;
168
169 mseccfg_csr_t_p mseccfg;
170 csr_t_p mseccfgh;
171
172 static const int max_pmp = 64;
173 csr_t_p mpmdeleg;
174 base_pmpaddr_csr_t_p pmpaddr[max_pmp];
175 csr_t_p spmpen;
176
177 float_csr_t_p fflags;
178 float_csr_t_p frm;
179
180 csr_t_p vxsat;
181
182 csr_t_p menvcfg;
183 csr_t_p senvcfg;
184 csr_t_p henvcfg;
185
186 csr_t_p mstateen[4];
187 csr_t_p sstateen[4];
188 csr_t_p hstateen[4];
189
190 csr_t_p htimedelta;
191 time_counter_csr_t_p time;
192 csr_t_p time_proxy;
193
194 csr_t_p stimecmp;
195 csr_t_p vstimecmp;
196
197 csr_t_p ssp;
198
199 sscsrind_reg_csr_t_p mireg[6];
200 sscsrind_reg_csr_t_p vsireg[6];
201 sscsrind_reg_csr_t_p nonvirtual_sireg[6];
202
203 csr_t_p mvien;
204 mvip_csr_t_p mvip;
205 csr_t_p hvictl;
206 csr_t_p vstopi;
207
208 bool serialized; // whether timer CSRs are in a well-defined
state
209
210 // When true, execute a single instruction and then enter
debug mode. This
211 // can only be set by executing dret.
212 enum {
213     STEP_NONE,
214     STEP_STEPPING,
215     STEP_STEPPED
216 } single_step;
217
218 commit_log_reg_t log_reg_write;
219 commit_log_mem_t log_mem_read;
220 commit_log_mem_t log_mem_write;
221 reg_t last_inst_priv;
222 int last_inst_xlen;
223 int last_inst_flen;
224
225 elp_t elp;
226
227 bool critical_error;
228
229 private:
230 void csr_init(processor_t* const proc, reg_t max_isa);
231 };
232
233 // this class represents one processor in a RISC-V machine.
234 class processor_t : public abstract_device_t
235 {
236 public:
237     processor_t(const char* isa_str, const char* priv_str,
238                 const cfg_t* cfg,
239                 simif_t* sim, uint32_t id, bool halt_on_reset,
240                 FILE *log_file, std::ostream& sout_); // because
of command line option --log and -s we need both
241     ~processor_t() override;
242
243     const isa_parser_t &get_isa() const & { return isa; }
244     const cfg_t &get_cfg() const & { return *cfg; }
245
246     void set_debug(bool value);
247     void set_histogram(bool value);
248     void enable_log_commits();
249     bool get_log_commits_enabled() const { return
log_commits_enabled; }
250     void reset();
251     void step(size_t n); // run for n cycles
252     void put_csr(int which, reg_t val);
253     uint32_t get_id() const { return id; }

```

```

254 reg_t get_csr(int which, insn_t insn, bool write, bool peek
= 0);
255 reg_t get_csr(int which) { return get_csr(which, insn_t(0),
false, true); }
256 mmu_t* get_mmu() { return mmu; }
257 state_t* get_state() { return &state; }
258 unsigned get_xlen() const { return xlen; }
259 unsigned paddr_bits() { return isa.get_max_xlen() == 64 ?
56 : 34; }
260 unsigned get_const_xlen() const {
261     // Any code that assumes a const xlen should use this
method to
262     // document that assumption. If Spike ever changes to
allow
263     // variable xlen, this method should be removed.
264     return xlen;
265 }
266 unsigned get_flen() const {
267     return extension_enabled('Q') ? 128 :
268     extension_enabled('D') ? 64 :
269     extension_enabled('F') ? 32 : 0;
270 }
271 extension_t* get_extension();
272 extension_t* get_extension(const char* name);
273 bool any_custom_extensions() const {
274     return !custom_extensions.empty();
275 }
276 bool any_vector_extensions() const {
277     return VU.VLEN > 0;
278 }
279 bool extension_enabled(unsigned char ext) const {
280     return extension_enabled(isa_extension_t(ext));
281 }
282 bool extension_enabled(isa_extension_t ext) const {
283     if (ext >= 'A' && ext <= 'Z')
284         return state.misa->extension_enabled(ext);
285     else
286         return extension_enable_table[ext];
287 }
288 // Is this extension enabled? and abort if this extension
can
289 // possibly be disabled dynamically. Useful for documenting
290 // assumptions about writable misa bits.
291 bool extension_enabled_const(unsigned char ext) const {
292     return extension_enabled_const(isa_extension_t(ext));
293 }
294 bool extension_enabled_const(isa_extension_t ext) const {
295     if (ext >= 'A' && ext <= 'Z') {
296         return state.misa->extension_enabled_const(ext);
297     } else {
298         assert(!extension_dynamic[ext]);
299         extension_assumed_const[ext] = true;
300         return extension_enabled(ext);
301     }
302 }
303 void set_extension_enable(unsigned char ext, bool enable) {
304     assert(!extension_assumed_const[ext]);
305     extension_dynamic[ext] = true;
306     extension_enable_table[ext] = enable &&
isa.extension_enabled(ext);
307 }
308 void set_impl(uint8_t impl, bool val) { impl_table[impl] =
val; }
309 bool has_mmu() const { return max_vaddr_bits != 0; }
310 unsigned get_max_vaddr_bits() const { return
max_vaddr_bits; }
311 void set_max_vaddr_bits(unsigned);
312 bool supports_impl(uint8_t impl) const {
313     return impl_table[impl];
314 }
315 reg_t pc_alignment_mask() {
316     const int ialign = extension_enabled(EXT_ZCA) ? 16 : 32;
317     return ~(reg_t)(ialign == 16 ? 0 : 2);
318 }
319 void set_vxsat() {
320     state.vxsat->write(1);
321 }
322 reg_t throw_instruction_address_misaligned(reg_t pc);
323 reg_t legalize_privilege(reg_t);
324 void set_privilege(reg_t, bool);
325 const char* get_privilege_string() const;
326 void update_histogram(reg_t pc);
327 const disassembler_t* get_disassembler() { return
disassembler; }
328
329 FILE *get_log_file() { return log_file; }
330
331 void register_base_insn(insn_desc_t insn) {
332     register_insn(insn, instructions);
333 }
334 void register_custom_insn(insn_desc_t insn) {
335     register_insn(insn, custom_instructions);
336 }
337 void register_extension(extension_t*);
338 void build_opcode_map();
339
340 // MMIO slave interface
341 bool load(reg_t addr, size_t len, uint8_t* bytes) override;
342 bool store(reg_t addr, size_t len, const uint8_t* bytes)
override;
343 reg_t size() override;

```

```

344
345 // When true, display disassembly of each instruction that's
executed.
346 bool debug;
347 // When true, take the slow simulation path.
348 bool slow_path() const;
349 bool halted() const { return state.debug_mode; }
350 enum {
351     HR_NONE, /* Halt request is inactive. */
352     HR_REGULAR, /* Regular halt request/debug interrupt. */
353     HR_GROUP, /* Halt requested due to halt group. */
354 } halt_request;
355
356 void trigger_updated(const std::vector<triggers::trigger_t
*> &triggers);
357
358 void set_pmp_num(reg_t pmp_num);
359 void set_pmp_granularity(reg_t pmp_granularity);
360 void set_smp_addr_entry();
361
362 const char* get_symbol(uint64_t addr);
363
364 void clear_waiting_for_interrupt() { state.in_wfi =
false; };
365 bool is_waiting_for_interrupt() const { return
state.in_wfi; };
366
367 void check_if_lpad_required();
368 reg_t set_lpad_expected(reg_t pc);
369
370 reg_t select_an_interrupt_with_default_priority(reg_t
enabled_interrupts) const;
371
372 private:
373     const isa_parser_t isa;
374     const cfg_t * const cfg;
375
376     simif_t* sim;
377     mmu_t* mmu; // main memory is always accessed via the mmu
378     std::unordered_map<std::string, extension_t*>
custom_extensions;
379     disassembler_t* disassembler;
380     state_t state;
381     uint32_t id;
382     unsigned xlen;
383     unsigned max_vaddr_bits;
384     bool histogram_enabled;
385     bool log_commits_enabled;
386     FILE *log_file;
387     std::ostream sout; // needed for socket command interface -
s, also
used for -d and -l, but not for --log
388     bool halt_on_reset;
389     bool check_triggers_icount;
390     std::vector<bool> impl_table;
391
392     // Note: does not include single-letter extensions in misa
393     std::bitset<NUM_ISA_EXTENSIONS> extension_enable_table;
394     std::bitset<NUM_ISA_EXTENSIONS> extension_dynamic;
395     mutable std::bitset<NUM_ISA_EXTENSIONS>
extension_assumed_const;
396
397     std::vector<opcode_map_entry_t> opcode_map[128];
398     std::vector<insn_desc_t> instructions;
399     std::vector<insn_desc_t> custom_instructions;
400     std::unordered_map<reg_t, uint64_t> pc_histogram;
401
402     void take_pending_interrupt() { take_interrupt(state.mip-
>read() & state.mie->read()); }
403     void take_interrupt(reg_t mask); // take first enabled
interrupt in mask
404     void take_trap(trap_t & t, reg_t epc); // take an exception
405     void take_trigger_action(triggers::action_t action, reg_t
breakpoint_tval, reg_t epc, bool virt);
406     void disasm(insn_t insn); // disassemble and print an
instruction
407     void register_insn(insn_desc_t, std::vector<insn_desc_t>&
pool);
408
409     void enter_debug_mode(uint8_t cause, uint8_t ext_cause);
410
411     void debug_output_log(std::stringstream *s); // either
output
to interactive user or write to log file
412
413     friend class mmu_t;
414     friend class clint_t;
415     friend class plic_t;
416     friend class extension_t;
417
418     void parse_priv_string(const char*);
419     void register_base_instructions();
420     insn_func_t decode_insn(insn_t insn);
421
422     // Track repeated executions for processor_t::disasm()
423     uint64_t last_pc, last_bits, executions;
424 public:
425     entropy_source es; // Crypto ISE Entropy source.
426
427     reg_t n_pmp;
428     reg_t lg_pmp_granularity;
429     reg_t pmp_tor_mask() { return ~(reg_t(1) <<

```

```

(lg_pmp_granularity - PMP_SHIFT)); }
430
431     vectorUnit_t VU;
432     triggers::module_t TM;
433
434     unsigned geilen;
435     imsic_t_p imsic;
436 };
437
438 #endif

```

A.10 riscv-software-src_riscv-isa-sim/riscv/execute.cc (361줄, 10,315B)

```

1 // See LICENSE for license details.
2
3 #include "config.h"
4 #include "processor.h"
5 #include "mmu.h"
6 #include "disasm.h"
7 #include "decode_macros.h"
8 #include <cassert>
9
10 static void commit_log_reset(processor_t* p)
11 {
12     p->get_state()->log_reg_write.clear();
13     p->get_state()->log_mem_read.clear();
14     p->get_state()->log_mem_write.clear();
15 }
16
17 static void commit_log_stash_privilege(processor_t* p)
18 {
19     state_t* state = p->get_state();
20     state->last_inst_priv = state->prv;
21     state->last_inst_xlen = p->get_xlen();
22     state->last_inst_flen = p->get_flen();
23 }
24
25 static void commit_log_print_value(FILE *log_file, int width,
const void *data)
26 {
27     assert(log_file);
28
29     switch (width) {
30     case 8:
31         fprintf(log_file, "0x%02" PRIx8, *(const uint8_t
*)data);
32         break;
33     case 16:
34         fprintf(log_file, "0x%04" PRIx16, *(const uint16_t
*)data);
35         break;
36     case 32:
37         fprintf(log_file, "0x%08" PRIx32, *(const uint32_t
*)data);
38         break;
39     case 64:
40         fprintf(log_file, "0x%016" PRIx64, *(const uint64_t
*)data);
41         break;
42     default:
43         if (width % 8 == 0) {
44             const uint8_t *arr = (const uint8_t *)data;
45
46             fprintf(log_file, "0x");
47             for (int idx = width / 8 - 1; idx >= 0; --idx) {
48                 fprintf(log_file, "%02" PRIx8, arr[idx]);
49             }
50         } else {
51             abort();
52         }
53         break;
54     }
55 }
56
57 static void commit_log_print_value(FILE *log_file, int width,
uint64_t val)
58 {
59     commit_log_print_value(log_file, width, &val);
60 }
61
62 static void commit_log_print_insn(processor_t *p, reg_t pc,
insn_t insn)
63 {
64     FILE *log_file = p->get_log_file();
65
66     auto& reg = p->get_state()->log_reg_write;
67     auto& load = p->get_state()->log_mem_read;
68     auto& store = p->get_state()->log_mem_write;
69     int priv = p->get_state()->last_inst_priv;
70     int xlen = p->get_state()->last_inst_xlen;
71     int flen = p->get_state()->last_inst_flen;
72
73     // print core id on all lines so it is easy to grep
74     fprintf(log_file, "core%4" PRIid32 ": ", p->get_id());
75
76     fprintf(log_file, "%1d ", priv);
77     commit_log_print_value(log_file, xlen, pc);
78     fprintf(log_file, " (");
79     commit_log_print_value(log_file, insn.length() * 8,

```

```

insn.bits());
80 fprintf(log_file, "));
81 bool show_vec = false;
82
83 for (auto item : reg) {
84     if (item.first == 0)
85         continue;
86
87     char prefix = ' ';
88     int size;
89     int rd = item.first >> 4;
90     bool is_vec = false;
91     bool is_vreg = false;
92     switch (item.first & 0xf) {
93     case 0:
94         size = xlen;
95         prefix = 'x';
96         break;
97     case 1:
98         size = flen;
99         prefix = 'f';
100        break;
101     case 2:
102         size = p->VU.VLEN;
103         prefix = 'v';
104         is_vreg = true;
105         break;
106     case 3:
107         is_vec = true;
108         break;
109     case 4:
110         size = xlen;
111         prefix = 'c';
112         break;
113     default:
114         assert("can't been here" && 0);
115         break;
116     }
117
118     if (!show_vec && (is_vreg || is_vec)) {
119         fprintf(log_file, " e%ld %s%ld %ld",
120             (long)p->VU.vsew,
121             p->VU.vflmul < 1 ? "mf" : "m",
122             p->VU.vflmul < 1 ? (long)(1 / p->VU.vflmul) :
123             (long)p->VU.vl->read());
124         show_vec = true;
125     }
126
127     if (!is_vec) {
128         if (prefix == 'c')
129             fprintf(log_file, " c%d%s ", rd, csr_name(rd));
130         else
131             fprintf(log_file, " %c%-2d ", prefix, rd);
132         if (is_vreg)
133             commit_log_print_value(log_file, size, &p-
134             >VU.elt<uint8_t>(rd, 0));
135         else
136             commit_log_print_value(log_file, size, item.second.v);
137     }
138
139     for (auto item : load) {
140         fprintf(log_file, " mem ");
141         commit_log_print_value(log_file, xlen, std::get<0>(item));
142     }
143
144     for (auto item : store) {
145         fprintf(log_file, " mem ");
146         commit_log_print_value(log_file, xlen, std::get<0>(item));
147         fprintf(log_file, " ");
148         commit_log_print_value(log_file, std::get<2>(item) << 3,
149         std::get<1>(item));
150     }
151     fprintf(log_file, "\n");
152 }
153 inline void processor_t::update_histogram(reg_t pc)
154 {
155     if (histogram_enabled)
156         pc_histogram[pc]++;
157 }
158
159 // These two functions are expected to be inlined by the
160 compiler separately in
161
162 // the processor_t::step() loop. The logged variant is used in
163 the slow path
164 static inline reg_t execute_insn_fast(processor_t* p, reg_t
165 pc, insn_fetch_t fetch) {
166     return fetch.func(p, fetch.insn, pc);
167 }
168 static inline reg_t execute_insn_logged(processor_t* p, reg_t
169 pc, insn_fetch_t fetch)
170 {
171     if (p->get_log_commits_enabled()) {
172         commit_log_reset(p);
173         commit_log_stash_privilege(p);
174     }
175     reg_t npc;

```

```

172
173 try {
174     npc = fetch.func(p, fetch.insn, pc);
175     if (npc != PC_SERIALIZE_BEFORE) {
176         if (p->get_log_commits_enabled()) {
177             commit_log_print_insn(p, pc, fetch.insn);
178         }
179     }
180 } catch(mem_trap_t& t) {
181     //handle segfault in middle of vector load/store
182     if (p->get_log_commits_enabled()) {
183         for (auto item : p->get_state()->log_reg_write) {
184             if ((item.first & 3) == 3) {
185                 commit_log_print_insn(p, pc, fetch.insn);
186                 break;
187             }
188         }
189     }
190     throw;
191 } catch(...) {
192     throw;
193 }
194 p->update_histogram(pc);
195
196 return npc;
197 }
198
199 bool processor_t::slow_path() const
200 {
201     return debug || state.single_step != state.STEP_NONE ||
202     state.debug_mode ||
203     log_commits_enabled || histogram_enabled ||
204     is_waiting_for_interrupt() || check_triggers_icount;
205 }
206
207 // fetch/decode/execute loop
208 void processor_t::step(size_t n)
209 {
210     mmu_t* _mmu = mmu;
211
212     if (!state.debug_mode) {
213         if (halt_request == HR_REGULAR) {
214             enter_debug_mode(DCSR_CAUSE_DEBUGINT, 0);
215         } else if (halt_request == HR_GROUP) {
216             enter_debug_mode(DCSR_CAUSE_GROUP, 0);
217         } else if (halt_on_reset) {
218             halt_on_reset = false;
219             enter_debug_mode(DCSR_CAUSE_HALT, 0);
220         }
221     }
222     while (n > 0) {
223         size_t instret = 0;
224         state.prv_changed = false;
225         state.v_changed = false;
226         reg_t mcountinhibit = state.mcountinhibit->read();
227         _mmu->reset_triggers();
228
229         #define retire_one() { \
230             ++instret; \
231             _mmu->check_triggers_after(); \
232         }
233
234         #define advance_pc(npc) { \
235             if (unlikely(invalid_pc(npc))) { \
236                 switch (npc) { \
237                     case PC_SERIALIZE_BEFORE: state.serialized = true;
238                     goto serialize; \
239                     case PC_SERIALIZE_AFTER: retire_one(); goto
240                     serialize; \
241                     default: abort(); \
242                 } \
243             } else { \
244                 state.pc = npc; \
245                 retire_one(); \
246             }
247         }
248
249         try {
250             take_pending_interrupt();
251             check_if_lpad_required();
252             if (unlikely(slow_path()))
253             {
254                 // Main simulation loop, slow path.
255                 while (instret < n)
256                 {
257                     if (unlikely(!state.serialized && state.single_step
258                     == state.STEP_STEPPED)) {
259                         state.single_step = state.STEP_NONE;
260                         if (!state.debug_mode) {
261                             enter_debug_mode(DCSR_CAUSE_STEP, 0);
262                             goto serialize;
263                         }
264                     }
265                     if (unlikely(state.single_step ==
266                     state.STEP_STEPPING)) {
267                         state.single_step = state.STEP_STEPPED;

```

```

267
268     if (!state.serialized && check_triggers_icount) {
269         auto match = TM.detect_icount_match();
270         if (match.has_value()) {
271             assert(match->timing ==
triggers::TIMING_BEFORE);
272             throw
triggers::matched_t((triggers::operation_t)0, 0, match->action,
state.v);
273         }
274     }
275
276     if (unlikely(is_waiting_for_interrupt()))
277         return;
278
279     insn_fetch_t fetch = mmu->load_insn(state.pc);
280     if (debug && !state.serialized)
281         disasm(fetch.insn);
282     auto new_pc = execute_insn_logged(this, state.pc,
fetch);
283     advance_pc(new_pc);
284
285     // Resume from debug mode in critical error
286     if (state.critical_error && !state.debug_mode) {
287         if (state.dcsr->read() & DCSR_CETRIG) {
288             enter_debug_mode(DCSR_CAUSE_EXTCAUSE,
DCSR_EXTCAUSE_CRITERR);
289         } else {
290             // Handling of critical error is implementation
defined
291             // For now just enter debug mode
292             enter_debug_mode(DCSR_CAUSE_HALT, 0);
293         }
294     }
295 }
296 }
297 else while (instret < n)
298 {
299     // Main simulation loop, fast path.
300     for (auto ic_entry = _mmu->access_icache(state.pc);
instret < n; instret++) {
301         auto fetch = ic_entry->data;
302         ic_entry = ic_entry->next;
303         auto new_pc = execute_insn_fast(this, state.pc,
fetch);
304         if (unlikely(ic_entry->tag != new_pc)) {
305             ic_entry = &_mmu->icache[_mmu-
>icache_index(new_pc)];
306             _mmu->icache[_mmu->icache_index(state.pc)].next =
ic_entry;
307             if (ic_entry->tag != new_pc) {
308                 advance_pc(new_pc);
309                 break;
310             }
311         }
312         state.pc = ic_entry->tag;
313     }
314 }
315 }
316 catch(trap_t& t)
317 {
318     take_trap(t, state.pc);
319     n = instret;
320 }
321 // If critical error then enter debug mode critical
error trigger enabled
322 if (state.critical_error) {
323     if (state.dcsr->read() & DCSR_CETRIG) {
324         enter_debug_mode(DCSR_CAUSE_EXTCAUSE,
DCSR_EXTCAUSE_CRITERR);
325     } else {
326         // Handling of critical error is implementation
defined
327         // For now just enter debug mode
328         enter_debug_mode(DCSR_CAUSE_HALT, 0);
329     }
330 }
331 // Trigger action takes priority over single step
332 auto match = TM.detect_trap_match(t);
333 if (match.has_value())
334     take_trigger_action(match->action, 0, state.pc, 0);
335 else if (unlikely(state.single_step ==
state.STEP_STEPPED)) {
336     state.single_step = state.STEP_NONE;
337     enter_debug_mode(DCSR_CAUSE_STEP, 0);
338 }
339 }
340 catch (triggers::matched_t& t)
341 {
342     take_trigger_action(t.action, t.address, state.pc,
t.gva);
343     // End this step at the trigger boundary. In
particular, a timing-before
344     // trigger retires no instruction, so continuing would
immediately execute
345     // from the debug ROM or trap vector to consume the
remaining step count.
346     n = instret;
347 }
348 catch(trap_debug_mode&)
349 {

```

```

350     enter_debug_mode(DCSR_CAUSE_SWBP, 0);
351 }
352
353 serialize:
354     state.minstret->bump((mcountinhibit & MCOUNTINHIBIT_IR) ?
0 : instret);
355
356     // Model a hart whose CPI is 1.
357     state.mcycle->bump((mcountinhibit & MCOUNTINHIBIT_CY) ?
0 : instret);
358
359     n -= instret;
360 }
361 }

```

A.11 riscv-software-src_riscv-isa-sim/riscv/mmu.h (651줄, 21,513B)

```

1 // See LICENSE for license details.
2
3 #ifndef _RISC_V_MMU_H
4 #define _RISC_V_MMU_H
5
6 #include "bloom_filter.h"
7 #include "decode.h"
8 #include "trap.h"
9 #include "common.h"
10 #include "simif.h"
11 #include "processor.h"
12 #include "memtracer.h"
13 #include "../fesvr/byteorder.h"
14 #include "triggers.h"
15 #include "cfg.h"
16
17 #include <algorithm>
18 #include <cassert>
19 #include <stdint.h>
20 #include <stdlib.h>
21
22 // virtual memory configuration
23 #define PGSHIFT 12
24 const reg_t PGSIZE = 1 << PGSHIFT;
25
26 // observability hooks for load, store and fetch
27 // intentionally empty not to cause runtime overhead
28 // can be redefined if needed
29 #ifndef MMU_OBSERVE_FETCH
30 #define MMU_OBSERVE_FETCH(addr, insn, length)
31 #endif
32
33 #ifndef MMU_OBSERVE_LOAD
34 #define MMU_OBSERVE_LOAD(addr, data, length)
35 #endif
36
37 #ifndef MMU_OBSERVE_STORE
38 #define MMU_OBSERVE_STORE(addr, data, length)
39 #endif
40
41 struct insn_fetch_t
42 {
43     insn_func_t func;
44     insn_t insn;
45 };
46
47 struct icache_entry_t {
48     reg_t tag;
49     icache_entry_t* next;
50     insn_fetch_t data;
51 };
52
53 struct tlb_entry_t {
54     uintptr_t host_addr;
55     reg_t target_addr;
56 };
57
58 struct dtlb_entry_t {
59     tlb_entry_t data;
60     reg_t tag;
61 };
62
63 struct pte_cache_entry_t {
64     reg_t paddr;
65     reg_t pte;
66 };
67
68 struct xlate_flags_t {
69     const bool forced_virt : 1 {false};
70     const bool hlvx : 1 {false};
71     const bool lr : 1 {false};
72     const bool ss_access : 1 {false};
73     const bool clean_inval : 1 {false};
74     const bool access_fault : 1 {false};
75     const bool require_alignment : 1 {false};
76
77     bool is_special_access() const {
78         return forced_virt || hlvx || lr || ss_access ||
clean_inval || access_fault || require_alignment;
79     }
80 };
81

```

```

82 struct mem_access_info_t {
83     const reg_t vaddr;
84     const reg_t transformed_vaddr;
85     const reg_t effective_priv;
86     const bool effective_virt;
87     const xlate_flags_t flags;
88     const access_type type;
89 };
90
91 [[noreturn]] void throw_access_exception(bool virt, reg_t
addr, access_type type);
92 [[noreturn]] void throw_page_fault_exception(bool virt, reg_t
addr, access_type type);
93
94 // this class implements a processor's port into the virtual
memory system.
95 // an MMU and instruction cache are maintained for simulator
performance.
96 class mmu_t
97 {
98 private:
99     reg_t get_pmlen(bool effective_virt, reg_t effective_priv,
xlate_flags_t flags) const;
100     mem_access_info_t generate_access_info(reg_t addr,
access_type type, xlate_flags_t xlate_flags);
101 public:
102     mmu_t(simif_t* sim, endianness_t endianness, processor_t*
proc, reg_t cache_blocksz);
103     ~mmu_t();
104
105     template<typename T>
106
107     T ALWAYS_INLINE load(reg_t addr, xlate_flags_t xlate_flags =
{}) {
108         target_endian<T> res;
109         bool aligned = (addr & (sizeof(T) - 1)) == 0;
110         auto [tlb_hit, host_addr, _] = access_tlb(tlb_load, addr);
111
112         if (likely(!xlate_flags.is_special_access()) && aligned &&
tlb_hit) {
113             res = *(target_endian<T>*)host_addr;
114         } else {
115             load_slow_path(addr, sizeof(T), (uint8_t*)&res,
xlate_flags);
116         }
117
118         MMU_OBSERVE_LOAD(addr, from_target(res), sizeof(T));
119
120         return from_target(res);
121     }
122
123     template<typename T>
124     T load_reserved(reg_t addr) {
125         return load<T>(addr, {.lr=true, .require_alignment=true});
126     }
127
128     template<typename T>
129     T guest_load(reg_t addr) {
130         return load<T>(addr, {.forced_virt = true});
131     }
132
133     template<typename T>
134     T guest_load_x(reg_t addr) {
135         return load<T>(addr, {.forced_virt=true, .hlvx=true});
136     }
137
138     // shadow stack load
139     template<typename T>
140     T ss_load(reg_t addr) {
141         bool misaligned = addr % sizeof(T);
142         return load<T>(addr,
{.ss_access=true, .access_fault=misaligned});
143     }
144
145     template<typename T>
146     T load_acquire(reg_t addr) {
147         return load<T>(addr,
{.require_alignment=enforce_amo_alignment(addr, sizeof(T))});
148     }
149
150     template<typename T>
151     void ALWAYS_INLINE store(reg_t addr, T val, xlate_flags_t
xlate_flags = {}) {
152         MMU_OBSERVE_STORE(addr, val, sizeof(T));
153         bool aligned = (addr & (sizeof(T) - 1)) == 0;
154         auto [tlb_hit, host_addr, _] = access_tlb(tlb_store,
addr);
155
156         if (!xlate_flags.is_special_access() && likely(aligned &&
tlb_hit)) {
157             *(target_endian<T>*)host_addr = to_target(val);
158         } else {
159             target_endian<T> target_val = to_target(val);
160             store_slow_path(addr, sizeof(T), (const
uint8_t*)&target_val, xlate_flags, true);
161         }
162     }
163
164     template<typename T>
165     void guest_store(reg_t addr, T val) {

```

```

166         store(addr, val, {.forced_virt=true});
167     }
168
169     // shadow stack store
170     template<typename T>
171     void ss_store(reg_t addr, T val) {
172         bool misaligned = addr % sizeof(T);
173         store<T>(addr, val,
{.ss_access=true, .access_fault=misaligned});
174     }
175
176     template<typename T>
177     void store_release(reg_t addr, T val) {
178         store<T>(addr, val,
{.require_alignment=enforce_amo_alignment(addr, sizeof(T))});
179     }
180
181     // AMO/Zicbom faults should be reported as store faults
182     #define convert_load_traps_to_store_traps(BODY) \
183     try { \
184         BODY \
185     } catch (trap_load_address_misaligned& t) { \
186         throw trap_store_address_misaligned(t.has_gva(),
t.get_tval(), t.get_tval2(), t.get_tinst()); \
187     } catch (trap_load_page_fault& t) { \
188         throw trap_store_page_fault(t.has_gva(), t.get_tval(),
t.get_tval2(), t.get_tinst()); \
189     } catch (trap_load_access_fault& t) { \
190         throw trap_store_access_fault(t.has_gva(), t.get_tval(),
t.get_tval2(), t.get_tinst()); \
191     } catch (trap_load_guest_page_fault& t) { \
192         throw trap_store_guest_page_fault(t.get_tval(),
t.get_tval2(), t.get_tinst()); \
193     }
194
195     inline bool enforce_amo_alignment(reg_t addr, size_t size)
196     {
197         if (proc->extension_enabled(EXT_ZAMA16B))
198             return (addr / 16) != ((addr + size - 1) / 16);
199
200         return addr & (size - 1);
201     }
202
203     // template for functions that perform an atomic memory
operation
204     template<typename T, typename op>
205     T amo(reg_t addr, op f) {
206         convert_load_traps_to_store_traps({
207             store_slow_path(addr, sizeof(T), nullptr,
{.ss_access=true, .access_fault=misaligned}, false);
208             auto lhs = load<T>(addr);
209             store<T>(addr, f(lhs));
210             return lhs;
211         })
212     }
213
214     // for shadow stack amoswap
215     template<typename T>
216     T ssamoswap(reg_t addr, reg_t value) {
217         convert_load_traps_to_store_traps({
218             bool misaligned = addr % sizeof(T);
219             store_slow_path(addr, sizeof(T), nullptr,
{.ss_access=true, .access_fault=misaligned}, false);
220             auto data = load<T>(addr, {.ss_access=true});
221             store<T>(addr, value, {.ss_access=true});
222             return data;
223         })
224     }
225
226     template<typename T>
227     T amo_compare_and_swap(reg_t addr, T comp, T swap) {
228         convert_load_traps_to_store_traps({
229             store_slow_path(addr, sizeof(T), nullptr,
{.require_alignment=enforce_amo_alignment(addr, sizeof(T))}, false);
230             auto lhs = load<T>(addr);
231             if (lhs == comp)
232                 store<T>(addr, swap);
233             return lhs;
234         })
235     }
236
237     void cbo_zero(reg_t addr) {
238         auto access_info = generate_access_info(addr, STORE, {});
239         reg_t transformed_addr = access_info.transformed_vaddr;
240
241         check_triggers(triggers::OPERATION_STORE,
transformed_addr, false, blocksz);
242
243         reg_t paddr = translate(access_info, 1) -
(transformed_addr & (blocksz - 1));
244         if (auto host_addr = sim->addr_to_mem(paddr)) {
245             if (tracer.interested_in_range(paddr, paddr + blocksz,
STORE))
246                 tracer.trace(paddr, blocksz, STORE);
247             memset(host_addr, 0, blocksz);
248         } else {
249             throw trap_store_access_fault((proc) ? proc->state.v :
false, transformed_addr, 0, 0);
250         }
251     }

```

```

252
253 void clean_inval(reg_t addr, bool clean, bool inval) {
254     auto access_info = generate_access_info(addr, LOAD,
{.clean_inval = true});
255     reg_t transformed_addr = access_info.transformed_vaddr;
256
257     check_triggers(triggers::OPERATION_STORE,
transformed_addr, false, blocksz);
258     convert_load_traps_to_store_traps({
259         const reg_t paddr = translate(access_info, 1) -
(transformed_addr & (blocksz - 1));
260         if (sim->reservable(paddr)) {
261             if (tracer.interested_in_range(paddr, paddr + blocksz,
LOAD))
262                 tracer.clean_invalidate(paddr, blocksz, clean,
inval);
263         } else {
264             throw trap_store_access_fault((proc) ? proc->state.v :
false, transformed_addr, 0, 0);
265         }
266     })
267 }
268
269 inline void yield_load_reservation()
270 {
271     load_reservation_address = (reg_t)-1;
272 }
273
274 inline bool check_load_reservation(reg_t vaddr, size_t size)
275 {
276     if (vaddr & (size-1)) {
277         // Raise either access fault or misaligned exception
278         store_slow_path(vaddr, size, nullptr,
{.require_alignment=true}, false);
279     }
280
281     auto [tlb_hit, _, paddr] = access_tlb(tlb_store, vaddr);
282     if (!tlb_hit)
283         paddr = translate(generate_access_info(vaddr, STORE,
{}), 1);
284
285     if (sim->reservable(paddr))
286         return load_reservation_address == paddr;
287
288     // SC to non-reservable region: report the PMM-masked
effective vaddr.
289     auto access_info = generate_access_info(vaddr, STORE, {});
290     throw trap_store_access_fault(access_info.effective_virt,
access_info.transformed_vaddr, 0, 0);
291 }
292
293 template<typename T>
294 bool store_conditional(reg_t addr, T val)
295 {
296     bool have_reservation = check_load_reservation(addr,
sizeof(T));
297
298     if (have_reservation)
299         store(addr, val);
300
301     yield_load_reservation();
302
303     return have_reservation;
304 }
305
306 static const reg_t ICACHE_ENTRIES = 4096;
307
308 inline size_t icache_index(reg_t addr)
309 {
310     return (addr / PC_ALIGN) % ICACHE_ENTRIES;
311 }
312
313 template<typename T>
314 T ALWAYS_INLINE fetch_jump_table(reg_t addr) {
315     T res = 0;
316     for (size_t i = 0; i < sizeof(T) / sizeof(insn_parcel_t);
i++)
317         res |= (T)fetch_slow_path(addr + i *
sizeof(insn_parcel_t)) << (i * sizeof(insn_parcel_t) * 8);
318
319     // table accesses use data endianness, not instruction
(little) endianness
320     return target_big_endian ? to_be(res) : res;
321 }
322
323 inline icache_entry_t* refill_icache(reg_t addr,
icache_entry_t* entry)
324 {
325     auto [insn, length] = fetch_insn(addr);
326
327     insn_fetch_t fetch = {proc->decode_insn(insn), insn};
328     entry->tag = addr;
329     entry->next = &icache[icache_index(addr + length)];
330     entry->data = fetch;
331
332     auto [check_tracer, _, paddr] = access_tlb(tlb_insn, addr,
TLB_FLAGS, TLB_CHECK_TRACER);
333     if (unlikely(check_tracer)) {

```

```

334     if (tracer.interested_in_range(paddr, paddr + 1, FETCH)) {
335         entry->tag = -1;
336         tracer.trace(paddr, paddr + length, FETCH);
337     }
338 }
339 MMU_OBSERVE_FETCH(addr, insn, length);
340 return entry;
341 }
342
343 inline icache_entry_t* access_icache(reg_t addr)
344 {
345     icache_entry_t* entry = &icache[icache_index(addr)];
346     if (likely(entry->tag == addr)){
347         MMU_OBSERVE_FETCH(addr, entry->data.insn,
insn_length(entry->data.insn.bits()));
348         return entry;
349     }
350     return refill_icache(addr, entry);
351 }
352
353 inline insn_fetch_t load_insn(reg_t addr)
354 {
355     return refill_icache(addr, &icache[icache_index(addr)])-
>data;
356 }
357
358 std::tuple<bool, uintptr_t, reg_t> ALWAYS_INLINE
access_tlb(const dtlb_entry_t* tlb, reg_t vaddr, reg_t allowed_flags
= 0, reg_t required_flags = 0)
359 {
360     auto vpn = vaddr / PGSIZE, pgoff = vaddr % PGSIZE;
361     auto& entry = tlb[vpn % TLB_ENTRIES];
362     auto hit = likely((entry.tag & (~allowed_flags |
required_flags)) == (vpn | required_flags));
363     bool mmio = allowed_flags & TLB_MMIO & entry.tag;
364     auto host_addr = mmio ? 0 : entry.data.host_addr + pgoff;
365     auto paddr = entry.data.target_addr + pgoff;
366     return std::make_tuple(hit, host_addr, paddr);
367 }
368
369 void flush_tlb();
370 void flush_icache();
371
372 void register_memtracer(memtracer_t*);
373
374 int is_misaligned_enabled()
375 {
376     return proc && proc->extension_enabled(EXT_ZICCLSM);
377 }
378
379 bool is_target_big_endian()
380 {
381     return target_big_endian;
382 }
383
384 template<typename T> inline T from_target(target_endian<T>
n) const
385 {
386     return target_big_endian? n.from_be() : n.from_le();
387 }
388
389 template<typename T> inline target_endian<T> to_target(T n)
const
390 {
391     return target_big_endian? target_endian<T>::to_be(n) :
target_endian<T>::to_le(n);
392 }
393
394 private:
395     simif_t* sim;
396     processor_t* proc;
397     memtracer_list_t tracer;
398     reg_t load_reservation_address;
399     reg_t blocksz;
400
401     // implement an instruction cache for simulator performance
402     icache_entry_t icache[ICACHE_ENTRIES];
403
404     // implement a TLB for simulator performance
405     static const reg_t TLB_ENTRIES = 256;
406     // If a TLB tag has TLB_CHECK_TRIGGERS set, then the MMU
must check for a
407     // trigger match before completing an access.
408     static const reg_t TLB_CHECK_TRIGGERS = reg_t(1) << 63;
409     static const reg_t TLB_CHECK_TRACER = reg_t(1) << 62;
410     static const reg_t TLB_MMIO = reg_t(1) << 61;
411     static const reg_t TLB_FLAGS = TLB_CHECK_TRIGGERS |
TLB_CHECK_TRACER | TLB_MMIO;
412     dtlb_entry_t tlb_load[TLB_ENTRIES];
413     dtlb_entry_t tlb_store[TLB_ENTRIES];
414     dtlb_entry_t tlb_insn[TLB_ENTRIES];
415
416     static const reg_t PTE_CACHE_ENTRIES = 251;
417     pte_cache_entry_t pte_cache[PT_CACHE_ENTRIES];
418
419     typedef bloom_filter_t<reg_t, simple_hash1, simple_hash2,
TLB_ENTRIES * 16, 3> reverse_tags_t;
420     reverse_tags_t tlb_store_reverse_tags;
421     reverse_tags_t tlb_insn_reverse_tags;
422

```

```

423 bool flush_tlb_ppn(reg_t ppn, dtlb_entry_t* tlb,
reverse_tags_t& filter);
424 void flush_itlb_ppn(reg_t ppn);
425 void flush_stlb_ppn(reg_t ppn);
426
427 // finish translation on a TLB miss and update the TLB
428 tlb_entry_t refill_tlb(reg_t vaddr, reg_t paddr, char*
host_addr, access_type type);
429 const char* fill_from_mmio(reg_t vaddr, reg_t paddr);
430
431 // perform a stage2 translation for a given guest address
432 reg_t s2xlate(reg_t gva, reg_t gpa, access_type type,
access_type trap_type, bool virt, bool hlvx, bool
is_for_vs_pt_addr);
433
434 // perform a page table walk for a given VA; set referenced/
dirty bits
435 reg_t walk(mem_access_info_t access_info);
436
437 // handle uncommon cases: TLB misses, page faults, MMIO
438 typedef uint16_t insn_parcel_t;
439
440 insn_parcel_t fetch_slow_path(reg_t addr);

441 insn_parcel_t perform_intrapage_fetch(reg_t vaddr, uintptr_t
host_addr, reg_t paddr);
442
443 void load_slow_path(reg_t original_addr, std::size_t len,
std::uint8_t* bytes, xlate_flags_t xlate_flags);
444 void load_slow_path_intrapage(reg_t len, uint8_t* bytes,
mem_access_info_t access_info);
445 void perform_intrapage_load(reg_t vaddr, uintptr_t
host_addr, reg_t paddr, reg_t len, uint8_t* bytes, xlate_flags_t
xlate_flags);
446 void store_slow_path(reg_t original_addr, std::size_t len,
const std::uint8_t* bytes,
xlate_flags_t xlate_flags, bool actually_store);
447 void store_slow_path_intrapage(reg_t len, const uint8_t*
bytes, mem_access_info_t access_info, bool actually_store);
448 void perform_intrapage_store(reg_t vaddr, uintptr_t
host_addr, reg_t paddr, reg_t len, const uint8_t* bytes,
xlate_flags_t xlate_flags);
449 bool mmio_fetch(reg_t paddr, size_t len, uint8_t* bytes);
450 bool mmio_load(reg_t paddr, size_t len, uint8_t* bytes);
451 bool mmio_store(reg_t paddr, size_t len, const uint8_t*
bytes);
452 bool mmio(reg_t paddr, size_t len, uint8_t* bytes,
access_type type);
453 void check_triggers(triggers::operation_t operation,
reg_t addr, bool virt, std::size_t data_size, const
std::uint8_t* bytes);
454 void check_triggers(triggers::operation_t operation,
reg_t addr, bool virt, std::size_t access_len);
455 void check_triggers(triggers::operation_t operation, reg_t
address,
bool virt, std::size_t size, std::optional<reg_t> data);
456 bool svukte_qualified(mem_access_info_t access_info);
457 bool svukte_fault(reg_t addr, mem_access_info_t
access_info);
458 reg_t translate(mem_access_info_t access_info, reg_t len);
459 reg_t pte_load(reg_t pte_paddr, reg_t addr, bool virt,
access_type trap_type, size_t ptesize) {
460 if (ptesize == 4)
461 return pte_load<uint32_t>(pte_paddr, addr, virt,
trap_type);
462 else
463 return pte_load<uint64_t>(pte_paddr, addr, virt,
trap_type);
464 }
465 void pte_store(reg_t pte_paddr, reg_t new_pte, reg_t addr,
bool virt, access_type trap_type, size_t ptesize) {
466 if (ptesize == 4)
467 return pte_store<uint32_t>(pte_paddr, new_pte, addr,
virt, trap_type);
468 else
469 return pte_store<uint64_t>(pte_paddr, new_pte, addr,
virt, trap_type);
470 }
471 template<typename T> inline reg_t pte_load(reg_t pte_paddr,
reg_t addr, bool virt, access_type trap_type)
472 {
473 if (auto [hit, pte] = pte_cache_access(pte_paddr); hit)
474 return pte;
475
476 const size_t ptesize = sizeof(T);
477 if (!pmp_ok(pte_paddr, ptesize, LOAD, PRV_S, false))
478 throw_access_exception(virt, addr, trap_type);
479
480 void* host_pte_addr = sim->addr_to_mem(pte_paddr);
481 target_endian_t> target_pte;
482 if (host_pte_addr) {
483 memcpy(&target_pte, host_pte_addr, ptesize);
484 } else if (!mmio_load(pte_paddr, ptesize,

```

```

(uint8_t*)&target_pte)) {
498 throw_access_exception(virt, addr, trap_type);
499 }
500
501 auto res = from_target(target_pte);
502 pte_cache_insert(pte_paddr, res);
503 return res;
504 }
505
506 template<typename T> inline void pte_store(reg_t pte_paddr,
reg_t new_pte, reg_t addr, bool virt, access_type trap_type)
507 {
508 const size_t ptesize = sizeof(T);
509
510 if (!pmp_ok(pte_paddr, ptesize, STORE, PRV_S, false))
511 throw_access_exception(virt, addr, trap_type);
512
513 void* host_pte_addr = sim->addr_to_mem(pte_paddr);
514 target_endian_t> target_pte = to_target((T)new_pte);
515 if (host_pte_addr) {
516 memcpy(host_pte_addr, &target_pte, ptesize);
517 } else if (!mmio_store(pte_paddr, ptesize,
(uint8_t*)&target_pte)) {
518 throw_access_exception(virt, addr, trap_type);
519 }
520
521 pte_cache_insert(pte_paddr, new_pte);
522 }
523
524 std::tuple<bool, reg_t> pte_cache_access(reg_t key)
525 {
526 auto e = pte_cache[key % PTE_CACHE_ENTRIES];
527 return std::make_tuple(e.paddr == key, e.pte);
528 }
529
530 void pte_cache_insert(reg_t key, reg_t value)
531 {
532 if (value & PTE_V)
533 pte_cache[key % PTE_CACHE_ENTRIES] = {key, value};
534 }
535
536 std::tuple<insn_bits_t, size_t> ALWAYS_INLINE
fetch_insn(reg_t addr)
537 {
538 bool intrapage = addr % PGSIZE + sizeof(insn_bits_t) <=
PGSIZE;
539 if (auto [tlb_hit, host_addr, paddr] =
access_tlb(tlb_insn, addr); tlb_hit && intrapage) {
540 insn_bits_t insn = from_le(*(insn_parcel_t*)host_addr);
541 size_t length = insn_length(insn);
542
543 for (size_t pos = sizeof(insn_parcel_t); pos < length;
pos += sizeof(insn_parcel_t)) {
544 insn |= insn_bits_t(from_le(*(insn_parcel_t*)
(host_addr + pos))) << (8 * pos);
545 length = insn_length(insn);
546 }
547
548 return std::make_tuple(insn, length);
549 }
550
551 insn_bits_t insn = fetch_slow_path(addr);
552 size_t length = insn_length(insn);
553
554 for (size_t pos = sizeof(insn_parcel_t); pos < length; pos
+= sizeof(insn_parcel_t)) {
555 insn |= insn_bits_t(fetch_slow_path(addr + pos)) << (8 *
pos);
556 length = insn_length(insn);
557 }
558
559 if (check_triggers_fetch) {
560 auto access_info = generate_access_info(addr, FETCH,
{});
561 check_triggers(triggers::OPERATION_EXECUTE, addr,
access_info.effective_virt, length, insn);
562 }
563
564 return std::make_tuple(insn, length);
565 }
566
567 void reset_triggers()
568 {
569 matched_trigger.reset();
570 }
571
572 void check_triggers_after()
573 {
574 if (matched_trigger)
575 throw matched_trigger.value();
576 }
577
578 inline bool in_mprv() const
579 {
580 return proc != nullptr
581 && !(proc->state.mnstatus && !get_field(proc-
>state.mnstatus->read(), MNSTATUS_NMIE))
582 && (!proc->state.debug_mode || get_field(proc-
>state.dcsr->read(), DCSR_MPRVEN))

```

```

584      && get_field(proc->state.mstatus->read(),
MSTATUS_MPRV);
585  }
586
587  reg_t pmp_homogeneous(reg_t addr, reg_t len);
588  std::optional<base_pmpaddr_csr_t> pmp_lookup(reg_t addr,
reg_t len, size_t start, size_t pmp_num);
589  bool pmp_ok(reg_t addr, reg_t len, access_type type, reg_t
mode, bool hlvx);
590  bool spmp_ok(reg_t addr, reg_t len, access_type type, reg_t
mode);
591
592  #ifdef RISC_V_ENABLE_DUAL_ENDIAN
593  bool target_big_endian;
594  #else
595  static const bool target_big_endian = false;
596  #endif
597  bool check_triggers_fetch;
598  bool check_triggers_load;
599  bool check_triggers_store;
600  std::optional<triggers::matched_t> matched_trigger;
601
602  friend class processor_t;
603 };
604
605 struct vm_info {
606  int levels;
607  int idxbits;
608  int widenbits;
609  int ptesize;
610  reg_t ptbase;
611 };
612
613 inline vm_info decode_vm_info(int xlen, bool stage2, reg_t
prv, reg_t satp)
614 {
615   if (prv == PRV_M) {
616     return {0, 0, 0, 0, 0};
617   } else if (!stage2 && prv <= PRV_S && xlen == 32) {
618     switch (get_field(satp, SATP32_MODE)) {
619       case SATP_MODE_OFF: return {0, 0, 0, 0, 0};
620       case SATP_MODE_SV32: return {2, 10, 0, 4, (satp &
SATP32_PPN) << PGSHIFT};
621       default: abort();
622     }
623   } else if (!stage2 && prv <= PRV_S && xlen == 64) {
624     switch (get_field(satp, SATP64_MODE)) {
625       case SATP_MODE_OFF: return {0, 0, 0, 0, 0};
626       case SATP_MODE_SV39: return {3, 9, 0, 8, (satp &
SATP64_PPN) << PGSHIFT};
627       case SATP_MODE_SV48: return {4, 9, 0, 8, (satp &
SATP64_PPN) << PGSHIFT};
628       case SATP_MODE_SV57: return {5, 9, 0, 8, (satp &
SATP64_PPN) << PGSHIFT};
629       case SATP_MODE_SV64: return {6, 9, 0, 8, (satp &
SATP64_PPN) << PGSHIFT};
630       default: abort();
631     }
632   } else if (stage2 && xlen == 32) {
633     switch (get_field(satp, HGATP32_MODE)) {
634       case HGATP_MODE_OFF: return {0, 0, 0, 0, 0};
635       case HGATP_MODE_SV32X4: return {2, 10, 2, 4, (satp &
HGATP32_PPN) << PGSHIFT};
636       default: abort();
637     }
638   } else if (stage2 && xlen == 64) {
639     switch (get_field(satp, HGATP64_MODE)) {
640       case HGATP_MODE_OFF: return {0, 0, 0, 0, 0};
641       case HGATP_MODE_SV39X4: return {3, 9, 2, 8, (satp &
HGATP64_PPN) << PGSHIFT};
642       case HGATP_MODE_SV48X4: return {4, 9, 2, 8, (satp &
HGATP64_PPN) << PGSHIFT};
643       case HGATP_MODE_SV57X4: return {5, 9, 2, 8, (satp &
HGATP64_PPN) << PGSHIFT};
644       default: abort();
645     }
646   } else {
647     abort();
648   }
649 }
650
651 #endif

```

A.12 lowRISC_opentitan/hw/ip/aes/rtl/aes_sbox_lut.sv (120줄, 4,503B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES LUT-based SBox
6
7 module aes_sbox_lut (
8   input  aes_pkg::ciphe_op_e op_i,
9   input  logic [7:0] data_i,
10  output logic [7:0] data_o
11 );
12

```

```

13 import aes_pkg::*;
14
15 // Define the LUTs
16 localparam logic [7:0] SBOX_FWD [256] = '{
17   8'h63, 8'h7C, 8'h77, 8'h7B, 8'hF2, 8'h6B, 8'h6F, 8'hC5,
18   8'h30, 8'h01, 8'h67, 8'h2B, 8'hFE, 8'hD7, 8'hAB, 8'h76,
19
20   8'hCA, 8'h82, 8'hC9, 8'h7D, 8'hFA, 8'h59, 8'h47, 8'hF0,
21   8'hAD, 8'hD4, 8'hA2, 8'hAF, 8'h9C, 8'hA4, 8'h72, 8'hC0,
22
23   8'hB7, 8'hFD, 8'h93, 8'h26, 8'h36, 8'h3F, 8'hF7, 8'hCC,
24   8'h34, 8'hA5, 8'hE5, 8'hF1, 8'h71, 8'hD8, 8'h31, 8'h15,
25
26   8'h04, 8'hC7, 8'h23, 8'hC3, 8'h18, 8'h96, 8'h05, 8'h9A,
27   8'h07, 8'h12, 8'h80, 8'hE2, 8'hEB, 8'h27, 8'hB2, 8'h75,
28
29   8'h09, 8'h83, 8'h2C, 8'h1A, 8'h1B, 8'h6E, 8'h5A, 8'hA0,
30   8'h52, 8'h3B, 8'hD6, 8'hB3, 8'h29, 8'hE3, 8'h2F, 8'h84,
31
32   8'h53, 8'hD1, 8'h00, 8'hED, 8'h20, 8'hFC, 8'hB1, 8'h5B,
33   8'h6A, 8'hCB, 8'hBE, 8'h39, 8'h4A, 8'h4C, 8'h58, 8'hCF,
34
35   8'hD0, 8'hEF, 8'hAA, 8'hFB, 8'h43, 8'h4D, 8'h33, 8'h85,
36   8'h45, 8'hF9, 8'h02, 8'h7F, 8'h50, 8'h3C, 8'h9F, 8'hA8,
37
38   8'h51, 8'hA3, 8'h40, 8'h8F, 8'h92, 8'h9D, 8'h38, 8'hF5,
39   8'hBC, 8'hB6, 8'hDA, 8'h21, 8'h10, 8'hFF, 8'hF3, 8'hD2,
40
41   8'hCD, 8'h0C, 8'h13, 8'hEC, 8'h5F, 8'h97, 8'h44, 8'h17,
42   8'hC4, 8'hA7, 8'h7E, 8'h3D, 8'h64, 8'h5D, 8'h19, 8'h73,
43
44   8'h60, 8'h81, 8'h4F, 8'hDC, 8'h22, 8'h2A, 8'h90, 8'h88,
45   8'h46, 8'hEE, 8'hB8, 8'h14, 8'hDE, 8'h5E, 8'h0B, 8'hDB,
46
47   8'hE0, 8'h32, 8'h3A, 8'h0A, 8'h49, 8'h06, 8'h24, 8'h5C,
48   8'hC2, 8'hD3, 8'hAC, 8'h62, 8'h91, 8'h95, 8'hE4, 8'h79,
49
50   8'hE7, 8'hC8, 8'h37, 8'h6D, 8'h8D, 8'hD5, 8'h4E, 8'hA9,
51   8'h6C, 8'h56, 8'hF4, 8'hEA, 8'h65, 8'h7A, 8'hAE, 8'h08,
52
53   8'hBA, 8'h78, 8'h25, 8'h2E, 8'h1C, 8'hA6, 8'hB4, 8'hC6,
54   8'hE8, 8'hDD, 8'h74, 8'h1F, 8'h4B, 8'hBD, 8'h8B, 8'h8A,
55
56   8'h70, 8'h3E, 8'hB5, 8'h66, 8'h48, 8'h03, 8'hF6, 8'h0E,
57   8'h61, 8'h35, 8'h57, 8'h89, 8'h86, 8'hC1, 8'h1D, 8'h9E,
58
59   8'hE1, 8'hF8, 8'h98, 8'h11, 8'h69, 8'hD9, 8'h8E, 8'h94,
60   8'h9B, 8'h1E, 8'h87, 8'hE9, 8'hCE, 8'h55, 8'h28, 8'hDF,
61
62   8'h8C, 8'hA1, 8'h89, 8'h0D, 8'hBF, 8'hE6, 8'h42, 8'h68,
63   8'h41, 8'h99, 8'h2D, 8'hOF, 8'hB0, 8'h54, 8'hBB, 8'h16
64 };
65
66 localparam logic [7:0] SBOX_INV [256] = '{
67   8'h52, 8'h09, 8'h6A, 8'hD5, 8'h30, 8'h36, 8'ha5, 8'h38,
68   8'hbf, 8'h40, 8'ha3, 8'h9e, 8'h81, 8'h33, 8'hdf, 8'hfb,
69
70   8'h7c, 8'he3, 8'h39, 8'h82, 8'h9b, 8'h2f, 8'hff, 8'h87,
71   8'h34, 8'h8e, 8'h43, 8'h44, 8'hc4, 8'hde, 8'he9, 8'hcb,
72
73   8'h54, 8'h7b, 8'h94, 8'h32, 8'ha6, 8'hc2, 8'h23, 8'h3d,
74   8'hee, 8'h4c, 8'h95, 8'h0b, 8'h42, 8'hfa, 8'hc3, 8'h4e,
75
76   8'h08, 8'h2e, 8'ha1, 8'h66, 8'h28, 8'hd9, 8'h24, 8'hb2,
77   8'h76, 8'h5b, 8'ha2, 8'h49, 8'h6d, 8'h8b, 8'hdl, 8'h25,
78
79   8'h72, 8'hf8, 8'hf6, 8'h64, 8'h86, 8'h68, 8'h98, 8'h16,
80   8'hd4, 8'ha4, 8'h5c, 8'hcc, 8'h5d, 8'h65, 8'hb6, 8'h92,
81
82   8'h6c, 8'h70, 8'h48, 8'h50, 8'hfd, 8'hed, 8'hb9, 8'hda,
83   8'h5e, 8'h15, 8'h46, 8'h57, 8'ha7, 8'h8d, 8'h9d, 8'h84,
84
85   8'h90, 8'hd8, 8'hab, 8'h00, 8'h8c, 8'hbc, 8'h3d, 8'h0a,
86   8'hf7, 8'he4, 8'h58, 8'h05, 8'hb8, 8'hb3, 8'h45, 8'h06,
87
88   8'hd0, 8'h2c, 8'h1e, 8'h8f, 8'hca, 8'h3f, 8'h0f, 8'h02,
89   8'hcl, 8'haf, 8'hbd, 8'h03, 8'h01, 8'h13, 8'h8a, 8'h6b,
90
91   8'h3a, 8'h91, 8'h11, 8'h41, 8'h4f, 8'h67, 8'hdc, 8'hea,
92   8'h97, 8'hf2, 8'hcf, 8'hce, 8'hf0, 8'hb4, 8'he6, 8'h73,
93
94   8'h96, 8'hac, 8'h74, 8'h22, 8'he7, 8'had, 8'h35, 8'h85,
95   8'he2, 8'hf9, 8'h37, 8'he8, 8'h1c, 8'h75, 8'hdf, 8'h6e,
96
97   8'h47, 8'hf1, 8'h1a, 8'h71, 8'h1d, 8'h29, 8'hc5, 8'h89,
98   8'h6f, 8'hb7, 8'h62, 8'h0e, 8'haa, 8'h18, 8'hbe, 8'h1b,
99
100  8'hfc, 8'h56, 8'h3e, 8'h4b, 8'hc6, 8'hd2, 8'h79, 8'h20,
101  8'h9a, 8'hdb, 8'hc0, 8'hfe, 8'h78, 8'hcd, 8'h5a, 8'hf4,
102
103  8'h1f, 8'hdd, 8'ha8, 8'h33, 8'h88, 8'h07, 8'hc7, 8'h31,
104  8'hbl, 8'h12, 8'h10, 8'h59, 8'h27, 8'h80, 8'hcc, 8'h5f,
105
106  8'h60, 8'h51, 8'h7f, 8'ha9, 8'h19, 8'hb5, 8'h4a, 8'h0d,
107  8'h2d, 8'he5, 8'h7a, 8'h9f, 8'h93, 8'hc9, 8'h9c, 8'hcf,
108
109  8'ha0, 8'he0, 8'h3b, 8'h4d, 8'hae, 8'h2a, 8'hf5, 8'hb0,
110  8'hc8, 8'hcb, 8'hbb, 8'h3c, 8'h83, 8'h53, 8'h99, 8'h61,
111
112  8'h17, 8'h2b, 8'h04, 8'h7e, 8'hba, 8'h77, 8'hd6, 8'h26,
113  8'he1, 8'h69, 8'h14, 8'h63, 8'h55, 8'h21, 8'h0c, 8'h7d

```

```

114 };
115
116 // Drive output
117 assign data_o = (op_i == CIPH_FWD) ? SBOX_FWD[data_i] :
118                (op_i == CIPH_INV) ? SBOX_INV[data_i] :
SBOX_FWD[data_i];
119
120 endmodule

```

A.13 lowRISC_opentitan/hw/ip/aes/rtl/aes_sbox_canright.sv

(70줄, 2,421B)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES Canright SBox #4
6 //
7 // For details, see the technical report: Canright, "A very
  compact Rijndael S-box"
8 // available at https://hdl.handle.net/10945/25608
9
10 module aes_sbox_canright (
11     input aes_pkg::ciph_op_e op_i,
12     input logic [7:0] data_i,
13     output logic [7:0] data_o
14 );
15
16 import aes_pkg::*;
17 import aes_sbox_canright_pkg::*;
18
19 // Functions //
20 //
21 // Inverse in GF(2^4), using normal basis [alpha^8, alpha^2]
22 // (see Figure 12 in the technical report)
23
24 function automatic logic [3:0] aes_inverse_gf2p4(logic [3:0]
  gamma);
25     logic [3:0] delta;
26     logic [1:0] a, b, c, d;
27     a = gamma[3:2] ^ gamma[1:0];
28     b = aes_mul_gf2p2(gamma[3:2], gamma[1:0]);
29     c = aes_scale_omega2_gf2p2(aes_square_gf2p2(a));
30     d = aes_square_gf2p2(c ^ b);
31     delta[3:2] = aes_mul_gf2p2(d, gamma[1:0]);
32     delta[1:0] = aes_mul_gf2p2(d, gamma[3:2]);
33     return delta;
34 endfunction
35
36 // Inverse in GF(2^8), using normal basis [d^16, d]
37 // (see Figure 11 in the technical report)
38
39 function automatic logic [7:0] aes_inverse_gf2p8(logic [7:0]
  gamma);
40     logic [7:0] delta;
41     logic [3:0] a, b, c, d;
42     a = gamma[7:4] ^ gamma[3:0];
43     b = aes_mul_gf2p4(gamma[7:4], gamma[3:0]);
44     c = aes_square_scale_gf2p4_gf2p2(a);
45     d = aes_inverse_gf2p4(c ^ b);
46     delta[7:4] = aes_mul_gf2p4(d, gamma[3:0]);
47     delta[3:0] = aes_mul_gf2p4(d, gamma[7:4]);
48     return delta;
49 endfunction
50
51 // Canright SBox //
52 //
53 //
54
55 logic [7:0] data_basis_x, data_inverse;
56
57 // Convert to normal basis X.
58 assign data_basis_x = (op_i == CIPH_FWD) ? aes_mvm(data_i,
  A2X) :
59                        (op_i == CIPH_INV) ? aes_mvm(data_i ^
  8'h63, S2X) :
60                        aes_mvm(data_i,
  A2X);
61
62 // Do the inversion in normal basis X.
63 assign data_inverse = aes_inverse_gf2p8(data_basis_x);
64
65 // Convert to basis S or A.
66 assign data_o = (op_i == CIPH_FWD) ?
  aes_mvm(data_inverse, X2S) ^ 8'h63 :
67           (op_i == CIPH_INV) ?
  aes_mvm(data_inverse, X2A) :
68           aes_mvm(data_inverse, X2S) ^ 8'h63;
69
70 endmodule

```

A.14 lowRISC_opentitan/hw/ip/aes/rtl/aes_sbox_dom.sv

(1,077줄, 41,942B)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES S-Box with First-Order Domain-Oriented Masking
6 //
7 // This is the unpipelined version using DOM-dep multipliers.
  It has a latency of 5 clock cycles
8 // and requires 28 bits of pseudo-random data per evaluation.
  Pipelining would only be beneficial
9 // when using
10 // - either a cipher core architecture with a data path
  smaller than 128 bit, i.e., where the
11 // individual S-Boxes are evaluated more than once per
  round, or
12 // - a fully unrolled cipher core architecture for maximum
  throughput.
13 //
14 // Note: The DOM AES S-Box is built on top of the Canright
  masked S-Box without mask re-use.
15 //
16 // For details, see the following papers and reports:
17 // [1] Gross, "Domain-Oriented Masking: Compact Masked
  Hardware Implementations with Arbitrary
18 // Protection Order" available at https://eprint.iacr.org/
  2016/486.pdf
19 // [2] Canright, "A very compact 'perfectly masked' S-box for
  AES (corrected)" available at
20 // https://eprint.iacr.org/2009/011.pdf
21 // [3] Canright, "A very compact Rijndael S-box" available at
  https://hdl.handle.net/10945/25608
22 //
23 // Using the Coco-Alma tool in transient mode, this
  implementation has been formally verified to be
24 // secure against first-order side-channel analysis (SCA). For
  more information on the tool,
25 // refer to the following papers:
26 // [4] Gigerl, "COCO: Co-design and co-verification of masked
  software implementations on CPUs"
27 // available at https://eprint.iacr.org/2020/1294.pdf
28 // [5] Bloem, "Formal verification of masked hardware
  implementations in the presence of glitches"
29 // available at https://eprint.iacr.org/2017/897.pdf
30
31 // IMPORTANT
32 //
33 // DO NOT USE THIS FOR SYNTHESIS
  BLINDLY!
34 //
35 // This implementation relies on primitive cells like
  prim_buf/flop_en containing tool-specific //
36 // synthesis attributes to prevent the synthesis tool from
  optimizing away/re-ordering registers //
37 // and to enforce the correct ordering of operations. Without
  the proper primitives, synthesis //
38 // tools might heavily optimize the design. The result is
  likely insecure. Use with care. //
39 //
40 // `include "prim_assert.sv"
41
42
43 // Packed struct for pseudo-random data (PRD) input. Stages 1, 3
  and 4 require 8 bits each. Stage 2
44 // requires just 4 bits.
45 typedef struct packed {
46     logic [7:0] prd_1;
47     logic [3:0] prd_2;
48     logic [7:0] prd_3;
49     logic [7:0] prd_4;
50 } aes_sbox_dom_prd_in_t;
51
52 // Packed struct for pseudo-random data (PRD) output. Stages 2
  and 3 produce 8 bits each. Stage 1
53 // produces just 4 bits.
54 typedef struct packed {
55     logic [3:0] prd_1;
56     logic [7:0] prd_2;
57     logic [7:0] prd_3;
58 } aes_sbox_dom_prd_out_t;
59
60 // DOM-indep GF(2^N) multiplier, first-order masked.
61 // Computes (a_q ^ b_q) = (a_x ^ b_x) * (a_y ^ b_y), i.e. q =
  x * y using first-order
62 // domain-oriented masking. The sharings of x and y are
  required to be uniformly random and

```

```

63 // independent from each other.
64 // See Fig. 2 in [1].
65 module aes_dom_indep_mul_gf2pn #(
66     parameter int unsigned NPower = 4,
67     parameter bit Pipeline = 1'b0
68 ) (
69     input logic clk_i,
70     input logic rst_ni,
71     input logic we_i,
72     input logic [NPower-1:0] a_x, // Share a of x
73     input logic [NPower-1:0] a_y, // Share a of y
74     input logic [NPower-1:0] b_x, // Share b of x
75     input logic [NPower-1:0] b_y, // Share b of y
76     input logic [NPower-1:0] z_0, // Randomness for
resharing
77     output logic [NPower-1:0] a_q, // Share a of q
78     output logic [NPower-1:0] b_q // Share b of q
79 );
80
81 import aes_sbox_canright_pkg::*;
82
83 // Calculation //
84 // Inner-domain terms
85 // Calculation //
86 logic [NPower-1:0] mul_ax_ay_d, mul_bx_by_d;
87 if (NPower == 4) begin : gen_inner_mul_gf2p4
88     assign mul_ax_ay_d = aes_mul_gf2p4(a_x, a_y);
89     assign mul_bx_by_d = aes_mul_gf2p4(b_x, b_y);
90 end else begin : gen_inner_mul_gf2p2
91     assign mul_ax_ay_d = aes_mul_gf2p2(a_x, a_y);
92     assign mul_bx_by_d = aes_mul_gf2p2(b_x, b_y);
93 end
94
95 // Cross-domain terms
96 logic [NPower-1:0] mul_ax_by, mul_ay_bx;
97 if (NPower == 4) begin : gen_cross_mul_gf2p4
98     assign mul_ax_by = aes_mul_gf2p4(a_x, b_y);
99     assign mul_ay_bx = aes_mul_gf2p4(a_y, b_x);
100 end else begin : gen_cross_mul_gf2p2
101     assign mul_ax_by = aes_mul_gf2p2(a_x, b_y);
102     assign mul_ay_bx = aes_mul_gf2p2(a_y, b_x);
103 end
104
105 // Resharing //
106 // Resharing of cross-domain terms
107 logic [NPower-1:0] aq_z0_d, bq_z0_d;
108 logic [NPower-1:0] aq_z0_q, bq_z0_q;
109 assign aq_z0_d = z_0 ^ mul_ax_by;
110 assign bq_z0_d = z_0 ^ mul_ay_bx;
111
112 // Registers
113 prim_flop_en #(
114     .Width ( 2*NPower ),
115     .ResetValue ( '0 )
116 ) u_prim_flop_abq_z0 (
117     .clk_i ( clk_i ),
118     .rst_ni ( rst_ni ),
119     .en_i ( we_i ),
120     .d_i ( {aq_z0_d, bq_z0_d} ),
121     .q_o ( {aq_z0_q, bq_z0_q} )
122 );
123
124 // Optional Pipelining //
125 logic [NPower-1:0] mul_ax_ay, mul_bx_by;
126
127 if (Pipeline == 1'b1) begin : gen_pipeline
128     // Add pipeline registers on inner-domain terms prior to
129     // integration. This allows accepting new
130     // input data every clock cycle and prevents SCA leakage
131     // occurring due to the integration of
132     // reshared cross-domain terms with inner-domain terms
133     // derived from different input data.
134     logic [NPower-1:0] mul_ax_ay_q, mul_bx_by_q;
135     prim_flop_en #(
136         .Width ( 2*NPower ),
137         .ResetValue ( '0 )
138     ) u_prim_flop_mul_abx_aby (
139         .clk_i ( clk_i ),
140         .rst_ni ( rst_ni ),
141         .en_i ( we_i ),
142         .d_i ( {mul_ax_ay_d, mul_bx_by_d} ),
143         .q_o ( {mul_ax_ay_q, mul_bx_by_q} )
144     );
145     assign mul_ax_ay = mul_ax_ay_q;
146     assign mul_bx_by = mul_bx_by_q;
147 end else begin : gen_no_pipeline
148     // Do not add the optional pipeline registers on the
149     // inner-domain terms. This allows to save
150     // some area in case the multiplier does not need to
151     // accept new data in every cycle. However,

```

```

157 // this can cause SCA leakage as during the clock cycle in
158 // which new data arrives, the new
159 // reshared cross-domain terms are integrated with the previous,
160 // reshared cross-domain terms.
161 // Avoid aggressive synthesis optimizations.
162 logic [NPower-1:0] mul_ax_ay_buf, mul_bx_by_buf;
163 prim_buf #(
164     .Width ( 2*NPower )
165 ) u_prim_buf_mul_abx_aby (
166     .in_i ( {mul_ax_ay_d, mul_bx_by_d} ),
167     .out_o ( {mul_ax_ay_buf, mul_bx_by_buf} )
168 );
169 assign mul_ax_ay = mul_ax_ay_buf;
170 assign mul_bx_by = mul_bx_by_buf;
171 end
172
173 // Integration //
174 // Integration //
175 // Integration //
176 assign a_q = mul_ax_ay ^ aq_z0_q;
177 assign b_q = mul_bx_by ^ bq_z0_q;
178
179 // Only GF(2^4) and GF(2^2) is supported.
180 `ASSERT_INIT(AesDomIndepMulPower, NPower == 4 || NPower ==
2)
181 endmodule
182
183 // DOM-dep GF(2^N) multiplier, first-order masked.
184 // Computes (a_q ^ b_q) = (a_x ^ b_x) * (a_y ^ b_y), i.e. q =
185 // x * y using first-order
186 // domain-oriented masking. The sharings of x and y are NOT
187 // required to be independent from each
188 // other. This is the un-optimized version consuming 3 times N
189 // bits of randomness for blinding and
190 // resharing. It is not used in the design but we keep it for
191 // reference.
192 // See Fig. 4 and Formulas 8 - 11 in [1].
193 module aes_dom_dep_mul_gf2pn_unopt #(
194     parameter int unsigned NPower = 4,
195     parameter bit Pipeline = 1'b0
196 ) (
197     input logic clk_i,
198     input logic rst_ni,
199     input logic we_i,
200     input logic [NPower-1:0] a_x, // Share a of x
201     input logic [NPower-1:0] a_y, // Share a of y
202     input logic [NPower-1:0] b_x, // Share b of x
203     input logic [NPower-1:0] b_y, // Share b of y
204     input logic [NPower-1:0] a_z, // Randomness for blinding
205     input logic [NPower-1:0] b_z, // Randomness for blinding
206     input logic [NPower-1:0] z_0, // Randomness for
resharing
207     output logic [NPower-1:0] a_q, // Share a of q
208     output logic [NPower-1:0] b_q // Share b of q
209 );
210
211 import aes_sbox_canright_pkg::*;
212
213 // Blinding //
214 // Blinding of y by z.
215 logic [NPower-1:0] a_yz_d, b_yz_d;
216 logic [NPower-1:0] a_yz_q, b_yz_q;
217 assign a_yz_d = a_y ^ a_z;
218 assign b_yz_d = b_y ^ b_z;
219
220 // Registers
221 prim_flop_en #(
222     .Width ( 2*NPower ),
223     .ResetValue ( '0 )
224 ) u_prim_flop_ab_yz (
225     .clk_i ( clk_i ),
226     .rst_ni ( rst_ni ),
227     .en_i ( we_i ),
228     .d_i ( {a_yz_d, b_yz_d} ),
229     .q_o ( {a_yz_q, b_yz_q} )
230 );
231
232 // Correction //
233 // Correction //
234 logic [NPower-1:0] a_mul_x_z, b_mul_x_z;
235 aes_dom_indep_mul_gf2pn #(
236     .NPower ( NPower ),
237     .Pipeline ( Pipeline )
238 ) u_aes_dom_indep_mul_gf2pn (
239     .clk_i ( clk_i ),
240     .rst_ni ( rst_ni ),
241     .we_i ( we_i ),
242     .a_x ( a_x ), // Share a of x
243     .a_y ( a_z ), // Share a of z
244     .b_x ( b_x ), // Share b of x
245     .b_y ( b_z ), // Share b of z
246     .z_0 ( z_0 ), // Randomness for resharing
247     .a_q ( a_mul_x_z ), // Share a of x * z
248     .b_q ( b_mul_x_z ) // Share b of x * z

```

```

249 );
250
251 //////////////////////////////////////////////////
252 // Optional Pipelining //
253 //////////////////////////////////////////////////
254 logic [NPower-1:0] a_x_calc, b_x_calc;
255
256 if (Pipeline == 1'b1) begin : gen_pipeline
257 // Add pipeline registers for input x. This allows
accepting new input data every clock cycle
258 // and prevents SCA leakage occurring due to the
multiplication of input x with b belonging to
259 // different clock cycles.
260
261 logic [NPower-1:0] a_x_q, b_x_q;
262 prim_flop_en # (
263     .Width ( 2*NPower ),
264     .ResetValue ( '0 )
265 ) u_prim_flop_ab_x (
266     .clk_i ( clk_i ),
267     .rst_ni ( rst_ni ),
268     .en_i ( we_i ),
269     .d_i ( {a_x, b_x} ),
270     .q_o ( {a_x_q, b_x_q} )
271 );
272
273 assign a_x_calc = a_x_q;
274 assign b_x_calc = b_x_q;
275
276 end else begin : gen_no_pipeline
277
278 // Do not add the optional pipeline registers for input x.
This allows to save some area in
279
280 // case the multiplier does not need to accept new data in
every cycle. However, this can cause
281
282 // SCA leakage as during the clock cycle in which new data
arrives, the new x input is
283 // multiplied with the previous b.
284
285 assign a_x_calc = a_x;
286 assign b_x_calc = b_x;
287 end
288
289 //////////////////////////////////////////////////
290 // Calculation //
291 //////////////////////////////////////////////////
292 // Combine shares of blinded y to obtain b.
293 logic [NPower-1:0] b;
294 assign b = a_yz_q ^ b_yz_q;
295
296 logic [NPower-1:0] a_mul_ax_b, b_mul_bx_b;
297 if (NPower == 4) begin : gen_mul_gf2p4
298 assign a_mul_ax_b = aes_mul_gf2p4(a_x_calc, b);
299 assign b_mul_bx_b = aes_mul_gf2p4(b_x_calc, b);
300 end else begin : gen_mul_gf2p2
301 assign a_mul_ax_b = aes_mul_gf2p2(a_x_calc, b);
302 assign b_mul_bx_b = aes_mul_gf2p2(b_x_calc, b);
303 end
304
305 //////////////////////////////////////////////////
306 // Integration //
307 //////////////////////////////////////////////////
308 assign a_q = a_mul_ax_z ^ a_mul_ax_b;
309 assign b_q = b_mul_x_z ^ b_mul_bx_b;
310
311 // Only GF(2^4) and GF(2^2) is supported.
312 `ASSERT_INIT(AesDomDepMulUnoptPower, NPower == 4 || NPower
== 2)
313
314 endmodule
315
316 // DOM-dep GF(2^N) multiplier, first-order masked.
317 // Computes (a_q ^ b_q) = (a_x ^ b_x) * (a_y ^ b_y), i.e. q =
x * y using first-order
318 // domain-oriented masking. The sharings of x and y are NOT
required to be independent from each
319
320 // other. This is the optimized version consuming 2 instead of 3
times N bits of randomness for
321 // blinding and resharing.
322 // See Formula 12 in [1].
323 module aes_dom_dep_mul_gf2pn # (
324     parameter int unsigned NPower = 4,
325     parameter bit Pipeline = 1'b0,
326     parameter bit PreDomIndep = 1'b0 // 1'b0: Not
followed by an un-pipelined DOM-indep
327 multiplier, this enables additional area
328 //
329 optimizations
330 //
331 followed by an un-pipelined
332 // 1'b1: Directly
333
334 // DOM-indep
335 multiplier, this is the version
336
337 // discussed
338 in [1].

```

```

329 ) (
330     input logic clk_i,
331     input logic rst_ni,
332     input logic we_i,
333     input logic [NPower-1:0] a_x, // Share a of x
334     input logic [NPower-1:0] a_y, // Share a of y
335     input logic [NPower-1:0] b_x, // Share b of x
336     input logic [NPower-1:0] b_y, // Share b of y
337     input logic [NPower-1:0] a_x_q, // Share a of x,
pipelined (for Pipeline=1 or PreDomIndep=1)
338     input logic [NPower-1:0] a_y_q, // Share a of y,
pipelined (for Pipeline=1)
339     input logic [NPower-1:0] b_x_q, // Share b of x,
pipelined (for Pipeline=1 or PreDomIndep=1)
340     input logic [NPower-1:0] b_y_q, // Share b of y,
pipelined (for Pipeline=1)
341     input logic [NPower-1:0] z_0, // Randomness for
blinding
342     input logic [NPower-1:0] z_1, // Randomness for
resharing
343     output logic [NPower-1:0] a_q, // Share a of q
344     output logic [NPower-1:0] b_q, // Share b of q
345     output logic [2*NPower-1:0] prd_o // Randomness for use in
another S-Box instance
346 );
347
348 import aes_sbox_canright_pkg::*;
349
350 //////////////////////////////////////////////////
351 // Blinding //
352 //////////////////////////////////////////////////
353 // Blinding of y by z_0.
354 logic [NPower-1:0] a_yz0_d, b_yz0_d;
355 logic [NPower-1:0] a_yz0_q, b_yz0_q;
356 assign a_yz0_d = a_y ^ z_0;
357 assign b_yz0_d = b_y ^ z_0;
358
359 // Registers
360 prim_flop_en # (
361     .Width ( 2*NPower ),
362     .ResetValue ( '0 )
363 ) u_prim_flop_ab_yz0 (
364     .clk_i ( clk_i ),
365     .rst_ni ( rst_ni ),
366     .en_i ( we_i ),
367     .d_i ( {a_yz0_d, b_yz0_d} ),
368     .q_o ( {a_yz0_q, b_yz0_q} )
369 );
370
371 //////////////////////////////////////////////////
372 // Correction //
373 //////////////////////////////////////////////////
374 // Basically, this a DOM-indep multiplier with:
375 // - a_x = a_x, b_x = b_x, and
376 // - a_y = z_0, b_y = 0 (constant),
377 // which allows for further optimizations.
378
379 // Calculation
380 logic [NPower-1:0] mul_ax_z0, mul_bx_z0;
381 if (NPower == 4) begin : gen_corr_mul_gf2p4
382 assign mul_ax_z0 = aes_mul_gf2p4(a_x, z_0);
383 assign mul_bx_z0 = aes_mul_gf2p4(b_x, z_0);
384 end else begin : gen_corr_mul_gf2p2
385 assign mul_ax_z0 = aes_mul_gf2p2(a_x, z_0);
386 assign mul_bx_z0 = aes_mul_gf2p2(b_x, z_0);
387 end
388
389 // Avoid aggressive synthesis optimizations.
390 logic [NPower-1:0] mul_ax_z0_buf, mul_bx_z0_buf;
391 prim_buf # (
392     .Width ( 2*NPower )
393 ) u_prim_buf_mul_abx_z0 (
394     .in_i ( {mul_ax_z0, mul_bx_z0} ),
395     .out_o ( {mul_ax_z0_buf, mul_bx_z0_buf} )
396 );
397
398 // Resharing
399 logic [NPower-1:0] axz0_z1_d, bxz0_z1_d;
400 logic [NPower-1:0] axz0_z1_q, bxz0_z1_q;
401 assign axz0_z1_d = mul_ax_z0_buf ^ z_1;
402 assign bxz0_z1_d = mul_bx_z0_buf ^ z_1;
403
404 // Registers
405 prim_flop_en # (
406     .Width ( 2*NPower ),
407     .ResetValue ( '0 )
408 ) u_prim_flop_abxz0_z1 (
409     .clk_i ( clk_i ),
410     .rst_ni ( rst_ni ),
411     .en_i ( we_i ),
412     .d_i ( {axz0_z1_d, bxz0_z1_d} ),
413     .q_o ( {axz0_z1_q, bxz0_z1_q} )
414 );
415
416 // Use intermediate results for generating PRD for another
S-Box instance.
417
418 // Use one share only. Directly use output of flops updating
with we_i.

```

```

419 // These intermediate results are obtained by remasking b_y
and mul_bx_z0 with z_0 and z_1,
420 // respectively. Since z_0/1 are uniformly distributed and
independent of b_y and mul_bx_z0,
421 // the intermediate results are also uniformly distributed
and independent of b_y and mul_bx_z0.
422 // For details, see Lemma 1 in [2].
423 assign prd_o = {b_yz0_q, bxz0_z1_q};
424
425 // Optional Pipelining //
426 // Optional Pipelining //
427 // Optional Pipelining //
428 logic [NPower-1:0] a_x_calc, b_x_calc, a_y_calc, b_y_calc;
429
430 if (Pipeline == 1'b1 && PreDomIndep != 1'b1) begin :
gen_pipeline_use
431 // Use pipelined inputs x and y. This allows accepting new
input data every clock cycle and
432 // prevents SCA leakage occurring due to the
multiplication of inputs x and y with d_b
433 // belonging to different clock cycles.
434 //
435 // The PreDomIndep variant uses the pipelined inputs
directly.
436 assign a_x_calc = a_x_q;
437 assign b_x_calc = b_x_q;
438 assign a_y_calc = a_y_q;
439 assign b_y_calc = b_y_q;
440
441 end else begin : gen_no_pipeline_use
442 // Do not use pipelined inputs x and y. This allows to
save some area in case the multiplier
443 // does not need to accept new data in every cycle.
However, this can cause SCA leakage as
444 // during the clock cycle in which new data arrives, the
new x and y inputs are multiplied
445 // with the previous d_b.
446 assign a_x_calc = a_x_q;
447 assign b_x_calc = b_x_q;
448 assign a_y_calc = a_y_q;
449 assign b_y_calc = b_y_q;
450
451 // Tie off unused signals.
452 if (PreDomIndep != 1'b1) begin : gen_ab_x_q
453 logic [NPower-1:0] unused_a_x_q, unused_b_x_q;
454 assign unused_a_x_q = a_x_q;
455 assign unused_b_x_q = b_x_q;
456 end
457 logic [NPower-1:0] unused_a_y_q, unused_b_y_q;
458 assign unused_a_y_q = a_y_q;
459 assign unused_b_y_q = b_y_q;
460 end
461
462 // Calculation & Integration //
463 // Calculation & Integration //
464 // Compute b. Note that unlike for the unoptimized
implementation, we don't combine the blinded
465 // shares of y to obtain a single b value. Instead, every
domain d gets its own version of b:
466 //
467 // d_b = d_y ^ _D_y_z0
468 //
469 // where _D_y_z0 corresponds to the sum of all domains of y
except for domain d, each
470 // individually blinded by z0 (needs to happen before the
register bank). This optimization
471 // is only suitable for first-order masking.
472 // See Formula 12 in [1].
473 if (PreDomIndep == 1'b1) begin : gen_pre_dom_indep
474 // This DOM-dep multiplier is directly followed by an un-
pipelined DOM-indep multiplier. To
475 // prevent SCA leakage in the un-pipelined DOM-indep
multiplier, the d_y and _D_y_z0 parts of
476 // d_b need to be individually multiplied with input x and
then the results need to be
477 // integrated (summed up) on a per-domain basis.
478 //
479 // d_y part: Inner-domain terms of x * y
480 logic [NPower-1:0] mul_ax_ay_d, mul_bx_by_d;
481 logic [NPower-1:0] mul_ax_ay_q, mul_bx_by_q;
482 if (NPower == 4) begin : gen_inner_mul_gf2p4
483 assign mul_ax_ay_d = aes_mul_gf2p4(a_x_calc, a_y_calc);
484 assign mul_bx_by_d = aes_mul_gf2p4(b_x_calc, b_y_calc);
485 end else begin : gen_inner_mul_gf2p2
486 assign mul_ax_ay_d = aes_mul_gf2p2(a_x_calc, a_y_calc);
487 assign mul_bx_by_d = aes_mul_gf2p2(b_x_calc, b_y_calc);
488 end
489 // Registers
490 prim_flop_en #(
491 .Width ( 2*NPower ),
492 .ResetValue ( '0 )
493 ) u_prim_flop_mul_abx_aby (
494 .clk_i ( clk_i

```

```

501 .rst_ni ( rst_ni
502 .en_i ( we_i
503 .d_i ( {mul_ax_ay_d, mul_bx_by_d} ),
504 .q_o ( {mul_ax_ay_q, mul_bx_by_q} )
505 );
506
507 // _D_y_z0 part: Cross-domain terms: d_x * _D_y_z0
508 // Need to use registered version of input x.
509 logic [NPower-1:0] mul_ax_byz0, mul_bx_ayz0;
510 if (NPower == 4) begin : gen_cross_mul_gf2p4
511 assign mul_ax_byz0 = aes_mul_gf2p4(a_x_q, b_yz0_q);
512 assign mul_bx_ayz0 = aes_mul_gf2p4(b_x_q, a_yz0_q);
513 end else begin : gen_cross_mul_gf2p2
514 assign mul_ax_byz0 = aes_mul_gf2p2(a_x_q, b_yz0_q);
515 assign mul_bx_ayz0 = aes_mul_gf2p2(b_x_q, a_yz0_q);
516 end
517
518 // Avoid aggressive synthesis optimizations.
519 logic [NPower-1:0] mul_ax_byz0_buf, mul_bx_ayz0_buf;
520 prim_buf #(
521 .Width ( 2*NPower )
522 ) u_prim_buf_mul_abx_bayz0 (
523 .in_i ( {mul_ax_byz0, mul_bx_ayz0} ),
524 .out_o ( {mul_ax_byz0_buf, mul_bx_ayz0_buf} )
525 );
526
527 // Integration
528 assign a_q = axz0_z1_q ^ mul_ax_ay_q ^ mul_ax_byz0_buf;
529 assign b_q = bxz0_z1_q ^ mul_bx_by_q ^ mul_bx_ayz0_buf;
530
531 end else begin : gen_not_pre_dom_indep
532 // This DOM-dep multiplier is not directly followed by an
un-pipelined DOM-indep multiplier. As
533 // a result, the d_y and _D_y_z0 parts of d_b can be
summed up prior to the multiplication
534 // with input x which allows saving 2 GF multipliers.
535
536 // Sum up d_y and _D_y_z0.
537 logic [NPower-1:0] a_b, b_b;
538 assign a_b = a_y_calc ^ b_yz0_q;
539 assign b_b = b_y_calc ^ a_yz0_q;
540
541 // Avoid aggressive synthesis optimizations.
542 logic [NPower-1:0] a_b_buf, b_b_buf;
543 prim_buf #(
544 .Width ( 2*NPower )
545 ) u_prim_buf_ab_b (
546 .in_i ( {a_b, b_b} ),
547 .out_o ( {a_b_buf, b_b_buf} )
548 );
549
550 // GF multiplications
551 logic [NPower-1:0] a_mul_ax_b, b_mul_bx_b;
552 if (NPower == 4) begin : gen_mul_gf2p4
553 assign a_mul_ax_b = aes_mul_gf2p4(a_x_calc, a_b_buf);
554 assign b_mul_bx_b = aes_mul_gf2p4(b_x_calc, b_b_buf);
555 end else begin : gen_mul_gf2p2
556 assign a_mul_ax_b = aes_mul_gf2p2(a_x_calc, a_b_buf);
557 assign b_mul_bx_b = aes_mul_gf2p2(b_x_calc, b_b_buf);
558 end
559
560 // Avoid aggressive synthesis optimizations.
561 logic [NPower-1:0] a_mul_ax_b_buf, b_mul_bx_b_buf;
562 prim_buf #(
563 .Width ( 2*NPower )
564 ) u_prim_buf_ab_mul_abx_b (
565 .in_i ( {a_mul_ax_b, b_mul_bx_b} ),
566 .out_o ( {a_mul_ax_b_buf, b_mul_bx_b_buf} )
567 );
568
569 // Integration
570 assign a_q = axz0_z1_q ^ a_mul_ax_b_buf;
571 assign b_q = bxz0_z1_q ^ b_mul_bx_b_buf;
572 end
573
574 // Only GF(2^4) and GF(2^2) is supported.
575 `ASSERT_INIT(AesDomDepMulPower, NPower == 4 || NPower == 2)
576
577 endmodule
578
579 // Inverse in GF(2^4) using first-order domain-oriented
masking and normal basis [z^4, z].
580
581 // See Fig. 6 in [2] (grey block, Stages 2 and 3) and Formulas
6, 13, 14, 15, 16, 17 in [2].
582 module aes_dom_inverse_gf2p4 #(
583 parameter bit PipelineMul = 1'b1
584 ) (
585 input logic clk_i,
586 input logic rst_ni,
587 input logic [1:0] we_i,
588 input logic [3:0] a_gamma,
589 input logic [3:0] b_gamma,
590 input logic [3:0] prd_2_i,
591 input logic [7:0] prd_3_i,
592 output logic [3:0] a_gamma_inv,
593 output logic [3:0] b_gamma_inv,
594 output logic [7:0] prd_2_o,
595 output logic [7:0] prd_3_o
596 );

```

```

597
598 import aes_sbox_canright_pkg::*;
599
600 ////////////////
601 // Stage 2 //
602 ////////////////
603 // Formula 13 in [2].
604
605 logic [1:0] a_gamma1, a_gamma0, b_gamma1, b_gamma0,
a_gamma1_gamma0, b_gamma1_gamma0;
606 assign a_gamma1 = a_gamma[3:2];
607 assign a_gamma0 = a_gamma[1:0];
608 assign b_gamma1 = b_gamma[3:2];
609 assign b_gamma0 = b_gamma[1:0];
610
611 logic [1:0] a_gamma_ss_d, b_gamma_ss_d;
612 logic [1:0] a_gamma_ss_q, b_gamma_ss_q;
613 assign a_gamma_ss_d =
aes_scale_omega2_gf2p2(aes_square_gf2p2(a_gamma1 ^ a_gamma0));
614 assign b_gamma_ss_d =
aes_scale_omega2_gf2p2(aes_square_gf2p2(b_gamma1 ^ b_gamma0));
615 prim_flop_en #(
616     .Width ( 4 ),
617     .ResetValue ( '0 )
618 ) u_prim_flop_ab_gamma_ss (
619     .clk_i ( clk_i ),
620     .rst_ni ( rst_ni ),
621     .en_i ( we_i[0] ),
622     .d_i ( {a_gamma_ss_d, b_gamma_ss_d} ),
623     .q_o ( {a_gamma_ss_q, b_gamma_ss_q} )
624 );
625
626 logic [1:0] a_gamma1_q, a_gamma0_q, b_gamma1_q, b_gamma0_q;
627 prim_flop_en #(
628     .Width ( 8 ),
629     .ResetValue ( '0 )
630 ) u_prim_flop_ab_gamma10 (
631     .clk_i (
632         .rst_ni ( rst_ni ),
633         .en_i ( we_i[0] ),
634         .d_i ( {a_gamma1, a_gamma0, b_gamma1,
b_gamma0} ),
635         .q_o ( {a_gamma1_q, a_gamma0_q, b_gamma1_q,
b_gamma0_q} )
636     );
637
638 logic [3:0] b_gamma10_prd2;
639 aes_dom_dep_mul_gf2pn #(
640     .NPower ( 2 ),
641     .Pipeline ( PipelineMul ),
642     .PreDomIndep ( 1'b0 )
643 ) u_aes_dom_mul_gamma1_gamma0 (
644     .clk_i ( clk_i ),
645     .rst_ni ( rst_ni ),
646     .we_i ( we_i[0] ),
647     .a_x ( a_gamma1 ), // Share a of x
648     .a_y ( a_gamma0 ), // Share a of y
649     .b_x ( b_gamma1 ), // Share b of x
650     .b_y ( b_gamma0 ), // Share b of y
651     .a_x_q ( a_gamma1_q ), // Share a of x, pipelined
652     .a_y_q ( a_gamma0_q ), // Share a of y, pipelined
653     .b_x_q ( b_gamma1_q ), // Share b of x, pipelined
654     .b_y_q ( b_gamma0_q ), // Share b of y, pipelined
655     .z_0 ( prd_2_i[1:0] ), // Randomness for blinding
656     .z_1 ( prd_2_i[3:2] ), // Randomness for resharing
657     .a_q ( a_gamma1_gamma0 ), // Share a of q
658     .b_q ( b_gamma1_gamma0 ), // Share b of q
659     .prd_o ( b_gamma10_prd2 ) // Randomness for use in
another S-Box instance
660 );
661
662 // Use intermediate results for generating PRD for Stage 3
of another S-Box instance.
663 // Use one share only. Directly use output of flops updating
with we_i[0].
664 // b_gamma10_prd2 is based on b_gamma1_q, b_gamma0_q but
XORed with prd_2_i, thus uniformly
665 // distributed and independent of b_gamma1/0_q (See Lemma 1
in [2]).
666 //
667 // In Stage 3 of another S-Box instance, the MSBs and LSBs
of the term below are used:
668 // 1. as randomness in the DOM-dep multipliers
u_aes_dom_mul_omega_gamma1/0, and
669 // 2. to generate randomness for the DOM-indep multipliers
u_aes_dom_mul_theta_y1/0 in Stage 4 of
670 // yet another S-Box instance, respectively.
671 // Without interleaving b_gamma1/0_q as well as the upper
and lower halves of b_gamma10_prd2 here,
672 // a glitch on the write-enable signal on the input pipeline
register of these DOM-indep
673 // multipliers may result in undesirable SCA leakage.

```

```

674 assign prd_2_o = {b_gamma1_q, b_gamma10_prd2[3:2],
b_gamma0_q, b_gamma10_prd2[1:0]};
675
676 ////////////////
677 // Stage 3 //
678 ////////////////
679
680 // Formulas 14 and 15 in [2].
681 logic [1:0] a_omega, b_omega;
682 assign a_omega = aes_square_gf2p2(a_gamma1_gamma0 ^
a_gamma_ss_q);
683 assign b_omega = aes_square_gf2p2(b_gamma1_gamma0 ^
b_gamma_ss_q);
684
685 // Avoid aggressive synthesis optimizations.
686 logic [1:0] a_omega_buf, b_omega_buf;
687 prim_buf #(
688     .Width ( 4 )
689 ) u_prim_buf_ab_omega (
690     .in_i ( {a_omega, b_omega} ),
691     .out_o ( {a_omega_buf, b_omega_buf} )
692 );
693
694 // Pipeline registers
695 logic [1:0] a_gamma1_qq, a_gamma0_qq, b_gamma1_qq,
b_gamma0_qq, a_omega_buf_q, b_omega_buf_q;
696 if (PipelineMul == 1'b1) begin: gen_prim_flop_omega_gamma10
697     // We instantiate the input pipeline registers for the
DOM-dep multiplier outside of the
698     // multiplier to enable sharing of pipeline registers
where applicable.
699
700     prim_flop_en #(
701         .Width ( 8 ),
702         .ResetValue ( '0 )
703     ) u_prim_flop_ab_gamma10_q (
704         .clk_i (
705             .rst_ni ( rst_ni ),
706             .en_i ( we_i[1] ),
707             .d_i ( {a_gamma1_q, a_gamma0_q, b_gamma1_q,
b_gamma0_q} ),
708             .q_o ( {a_gamma1_qq, a_gamma0_qq, b_gamma1_qq,
b_gamma0_qq} )
709         );
710
711     // These inputs are used by both DOM-dep multipliers
below.
712     prim_flop_en #(
713         .Width ( 4 ),
714         .ResetValue ( '0 )
715     ) u_prim_flop_ab_omega_buf (
716         .clk_i ( clk_i ),
717         .rst_ni ( rst_ni ),
718         .en_i ( we_i[1] ),
719         .d_i ( {a_omega_buf, b_omega_buf} ),
720         .q_o ( {a_omega_buf_q, b_omega_buf_q} )
721     );
722
723     end else begin : gen_no_prim_flop_ab_y10
724         // When using un-pipelined multipliers, there is no need
to insert additional registers.
725
726         // We drive the corresponding inputs to 0 to make sure the
functionality isn't correct in case
727         // the pipelined inputs are erroneously used.
728
729         assign a_gamma1_qq = '0;
730         assign a_gamma0_qq = '0;
731         assign b_gamma1_qq = '0;
732         assign b_gamma0_qq = '0;
733         assign a_omega_buf_q = '0;
734         assign b_omega_buf_q = '0;
735     end
736
737 // Formulas 16 and 17 in [2].
738 logic [3:0] b_gamma10_omega_prd3;
739 aes_dom_dep_mul_gf2pn #(
740     .NPower ( 2 ),
741     .Pipeline ( PipelineMul ),
742     .PreDomIndep ( 1'b0 )
743 ) u_aes_dom_mul_omega_gamma1 (
744     .clk_i ( clk_i ),
745     .rst_ni ( rst_ni ),
746     .we_i ( we_i[1] ),
747     .a_x ( a_gamma1_q ), // Share a of x
748     .a_y ( a_omega_buf ), // Share a of y
749     .b_x ( b_gamma1_q ), // Share b of x
750     .b_y ( b_omega_buf ), // Share b of y
751     .a_x_q ( a_gamma1_qq ), // Share a of x,
pipelined (for Pipeline=1 or PreDomIndep=1)
752     .a_y_q ( a_omega_buf_q ), // Share a of y,
pipelined (for Pipeline=1)
753     .b_x_q ( b_gamma1_qq ), // Share b of x,
pipelined (for Pipeline=1 or PreDomIndep=1)
754     .b_y_q ( b_omega_buf_q ), // Share b of y,
pipelined (for Pipeline=1)
755     .z_0 ( prd_3_i[5:4] ), // Randomness for
blinding

```

```

755 .z_1 ( prd_3_i[7:6] ), // Randomness for
resharing
756 .a_q ( a_gamma_inv[1:0] ), // Share a of q
757 .b_q ( b_gamma_inv[1:0] ), // Share b of q
758 .prd_o ( b_gamma1_omega_prd3 ) // Randomness for use in
another S-Box instance
759 );
760
761 logic [3:0] b_gamma0_omega_prd3;
762 aes_dom_dep_mul_gf2pn #(
763 .NPower ( 2 ),
764 .Pipeline ( PipelineMul ),
765 .PreDomIndep ( 1'b0 )
766 ) u_aes_dom_mul_omega_gamma0 (
767 .clk_i ( clk_i ),
768 .rst_ni ( rst_ni ),
769 .we_i ( we_i[1] ),
770 .a_x ( a_omega_buf ), // Share a of x
771 .a_y ( a_gamma0_q ), // Share a of y
772 .b_x ( b_omega_buf ), // Share b of x
773 .b_y ( b_gamma0_q ), // Share b of y
774 .a_x_q ( a_omega_buf_q ), // Share a of x,
pipelined (for Pipeline=1 or PreDomIndep=1)
775 .a_y_q ( a_gamma0_qq ), // Share a of y,
pipelined (for Pipeline=1)
776 .b_x_q ( b_omega_buf_q ), // Share b of x,
pipelined (for Pipeline=1 or PreDomIndep=1)
777 .b_y_q ( b_gamma0_qq ), // Share b of y,
pipelined (for Pipeline=1)
778 .z_0 ( prd_3_i[1:0] ), // Randomness for
blinding
779 .z_1 ( prd_3_i[3:2] ), // Randomness for
resharing
780 .a_q ( a_gamma_inv[3:2] ), // Share a of q
781 .b_q ( b_gamma_inv[3:2] ), // Share b of q
782 .prd_o ( b_gamma0_omega_prd3 ) // Randomness for use in
another S-Box instance
783 );
784
785 // Use intermediate results for generating PRD for Stage 4
of another S-Box instance.

786 // Use one share only. Directly use output of flops updating
with we_i[1].

787 // b_gamma1/0_omega_prd3 are both based on b_omega but X0Red
with different parts of prd_3_i,
788 // thus uniformly distributed and independent of b_omega
(see Lemma 1 in [2]).
789 assign prd_3_o = {b_gamma1_omega_prd3, b_gamma0_omega_prd3};
790
791 endmodule
792
793 // Inverse in GF(2^8) using first-order domain-oriented
masking and normal basis [y^16, y].
794 // See Fig. 6 in [1] and Formulas 3, 12, 18 and 19 in [2].
795 module aes_dom_inverse_gf2p8 #(
796 parameter bit PipelineMul = 1'b1
797 ) (
798 input logic clk_i,
799 input logic rst_ni,
800 input logic [3:0] we_i,
801 input logic [7:0] a_y, // input data masked
by b_y
802 input logic [7:0] b_y, // input mask
803 input aes_sbox_dom_prd_in_t prd_i, // pseudo-random
data, e.g. for intermediate masks
804 output logic [7:0] a_y_inv, // output data masked
by b_y_inv
805 output logic [7:0] b_y_inv, // output mask
806 output aes_sbox_dom_prd_out_t prd_o // pseudo-random
data, e.g. for use in another S-Box
807 );
808 // instance
809
810 import aes_sbox_canright_pkg::*;
811
812 // Stage 1 //
813 // Formula 12 in [2].
814
815 logic [3:0] a_y1, a_y0, b_y1, b_y0, a_y1_y0, b_y1_y0;
816 assign a_y1 = a_y[7:4];
817 assign a_y0 = a_y[3:0];
818 assign b_y1 = b_y[7:4];
819 assign b_y0 = b_y[3:0];
820
821 logic [3:0] a_y_ss_d, b_y_ss_d;
822 logic [3:0] a_y_ss_q, b_y_ss_q;
823 assign a_y_ss_d = aes_square_scale_gf2p4_gf2p2(a_y1 ^ a_y0);
824 assign b_y_ss_d = aes_square_scale_gf2p4_gf2p2(b_y1 ^ b_y0);
825 prim_flop_en #(
826 .Width ( 8 ),
827 .ResetValue ( '0 )
828 ) u_prim_flop_ab_y_ss (
829 .clk_i ( clk_i ),
830 .rst_ni ( rst_ni ),
831 .en_i ( we_i[0] ),
832 .d_i ( {a_y_ss_d, b_y_ss_d} ),
833 .q_o ( {a_y_ss_q, b_y_ss_q} )

```

```

835 );
836
837 logic [3:0] a_y1_q, a_y0_q, b_y1_q, b_y0_q;
838 if (PipelineMul == 1'b1) begin: gen_prim_flop_ab_y10
839 // We instantiate the input pipeline registers for the
DOM-dep multiplier outside of the
840 // multiplier to enable sharing of pipeline registers
where applicable.
841
842 prim_flop_en #(
843 .Width ( 16 ),
844 .ResetValue ( '0 )
845 ) u_prim_flop_ab_y10 (
846 .clk_i ( clk_i ),
847 .rst_ni ( rst_ni ),
848 .en_i ( we_i[0] ),
849 .d_i ( {a_y1, a_y0, b_y1, b_y0} ),
850 .q_o ( {a_y1_q, a_y0_q, b_y1_q, b_y0_q} )
851 );
852
853 end else begin : gen_no_prim_flop_ab_y10
854 // When using un-pipelined multipliers, there is no need
to insert additional registers.
855 // We drive the corresponding inputs to 0 to make sure the
functionality isn't correct in case
856 // the pipelined inputs are erroneously used.
857
858 assign a_y1_q = '0;
859 assign a_y0_q = '0;
860 assign b_y1_q = '0;
861 assign b_y0_q = '0;
862 end
863
864 logic [7:0] b_y10_prd1;
865 aes_dom_dep_mul_gf2pn #(
866 .NPower ( 4 ),
867 .Pipeline ( PipelineMul ),
868 .PreDomIndep ( 1'b0 )
869 ) u_aes_dom_mul_y1_y0 (
870 .clk_i ( clk_i ),
871 .rst_ni ( rst_ni ),
872 .we_i ( we_i[0] ),
873 .a_x ( a_y1 ), // Share a of x
874 .a_y ( a_y0 ), // Share a of y
875 .b_x ( b_y1 ), // Share b of x
876 .b_y ( b_y0 ), // Share b of y
877 .a_x_q ( a_y1_q ), // Share a of x, pipelined
(for Pipeline=1 or PreDomIndep=1)
878 .a_y_q ( a_y0_q ), // Share a of y, pipelined
(for Pipeline=1)
879 .b_x_q ( b_y1_q ), // Share b of x, pipelined
(for Pipeline=1 or PreDomIndep=1)
880 .b_y_q ( b_y0_q ), // Share b of y, pipelined
(for Pipeline=1)
881 .z_0 ( prd_i.prd_1[3:0] ), // Randomness for blinding
882 .z_1 ( prd_i.prd_1[7:4] ), // Randomness for resharing
883 .a_q ( a_y1_y0 ), // Share a of q
884 .b_q ( b_y1_y0 ), // Share b of q
885 .prd_o ( b_y10_prd1 ) // Randomness for use in
another S-Box instance
886 );
887
888 logic [3:0] a_gamma, b_gamma;
889 assign a_gamma = a_y_ss_q ^ a_y1_y0;
890 assign b_gamma = b_y_ss_q ^ b_y1_y0;
891
892 // Avoid aggressive synthesis optimizations.
893 logic [3:0] a_gamma_buf, b_gamma_buf;
894 prim_buf #(
895 .Width ( 8 )
896 ) u_prim_buf_ab_gamma (
897 .in_i ( {a_gamma, b_gamma} ),
898 .out_o ( {a_gamma_buf, b_gamma_buf} )
899 );
900
901 // Use intermediate results for generating PRD for Stage 2
of another S-Box instance.
902 // Use one share only. Directly use output of flops updating
with we_i[0].
903 // b_y10_prd1 is based on b_y and X0Red with prd_1. We just
use the lower part involving a
904 // non-linear element.
905 assign prd_o.prd_1 = b_y10_prd1[3:0];
906 logic [3:0] unused_prd;
907 assign unused_prd = b_y10_prd1[7:4];
908
909 // Stages 2 and 3 //
910
911 logic [3:0] a_theta, b_theta;
912
913 // a_gamma is masked by b_gamma, a_gamma_inv is masked by
b_gamma_inv.
914 aes_dom_inverse_gf2p4 #(
915 .PipelineMul ( PipelineMul )
916 ) u_aes_dom_inverse_gf2p4 (
917 .clk_i ( clk_i ),
918 .rst_ni ( rst_ni ),

```

```

921 .we_i      ( we_i[2:1] ),
922 .a_gamma   ( a_gamma_buf ),
923 .b_gamma   ( b_gamma_buf ),
924 .prd_2_i   ( prd_i.prd_2 ),
925 .prd_3_i   ( prd_i.prd_3 ),
926 .a_gamma_inv ( a_theta ),
927 .b_gamma_inv ( b_theta ),
928 .prd_2_o   ( prd_o.prd_2 ),
929 .prd_3_o   ( prd_o.prd_3 )
930 );
931
932 //////////////////////////////////////////////////
933 // Stage 4 //
934 //////////////////////////////////////////////////
935 // Formulas 18 and 19 in [2].
936
937 logic [3:0] a_y1_qqq, a_y0_qqq, b_y1_qqq, b_y0_qqq;
938 prim_flop_en #(
939   .Width      ( 16 ),
940   .ResetValue ( '0 )
941 ) u_prim_flop_ab_y10_qqq (
942   .clk_i ( clk_i ),
943   .rst_ni ( rst_ni ),
944   .en_i ( we_i[2] ),
945   .d_i ( {a_y1, a_y0, b_y1, b_y0} ),
946   .q_o ( {a_y1_qqq, a_y0_qqq, b_y1_qqq, b_y0_qqq} )
947 );
948
949 aes_dom_indep_mul_gf2pn #(
950   .NPower      ( 4 ),
951   .Pipeline     ( PipelineMul )
952 ) u_aes_dom_mul_theta_y1 (
953   .clk_i ( clk_i ),
954   .rst_ni ( rst_ni ),
955   .we_i ( we_i[3] ),
956   .a_x ( a_y1_qqq ), // Share a of x
957   .a_y ( a_theta ), // Share a of y
958   .b_x ( b_y1_qqq ), // Share b of x
959   .b_y ( b_theta ), // Share b of y
960   .z_0 ( prd_i.prd_4[7:4] ), // Randomness for resharing
961   .a_q ( a_y_inv[3:0] ), // Share a of q
962   .b_q ( b_y_inv[3:0] ) // Share b of q
963 );
964
965 aes_dom_indep_mul_gf2pn #(
966   .NPower      ( 4 ),
967   .Pipeline     ( PipelineMul )
968 ) u_aes_dom_mul_theta_y0 (
969   .clk_i ( clk_i ),
970   .rst_ni ( rst_ni ),
971   .we_i ( we_i[3] ),
972   .a_x ( a_theta ), // Share a of x
973   .a_y ( a_y0_qqq ), // Share a of y
974   .b_x ( b_theta ), // Share b of x
975   .b_y ( b_y0_qqq ), // Share b of y
976   .z_0 ( prd_i.prd_4[3:0] ), // Randomness for resharing
977   .a_q ( a_y_inv[7:4] ), // Share a of q
978   .b_q ( b_y_inv[7:4] ) // Share b of q
979 );
980
981 endmodule
982
983 // SEC_CM: KEY.MASKING
984 module aes_sbox_dom
985 #(
986   parameter bit PipelineMul = 1'b1
987 ) (
988   input logic clk_i,
989   input logic rst_ni,
990   input logic en_i,
991   output logic out_req_o,
992   input logic out_ack_i,
993   input aes_pkg::ciph_op_e op_i,
994   input logic [7:0] data_i, // masked, the actual
995   input logic [7:0] mask_i, // input mask
996   input logic [27:0] prd_i, // pseudo-random data for
997   // remasking, in total we need 28 bits
998   // of PRD per evaluation,
999   // but at most 8 bits per cycle
1000  output logic [7:0] data_o, // masked, the actual
1001  output logic [7:0] mask_o, // output mask
1002  output logic [7:0] prd_o, // PRD for usage in Stages
1003  // 2 - 4 of other S-Box instances
1004 );
1005
1006 import aes_pkg::*;
1007 import aes_sbox_canright_pkg::*;
1008
1009 logic [7:0] in_data_basis_x, out_data_basis_x;
1010 logic [7:0] in_mask_basis_x, out_mask_basis_x;
1011 logic [3:0] we;
1012
1013 aes_sbox_dom_prd_in_t in_prd;
1014 aes_sbox_dom_prd_out_t out_prd;
1015
1016 // Convert data to normal basis X.
1017 assign in_data_basis_x = (op_i == CIPH_FWD) ?
aes_mvm(data_i, A2X) :

```

```

1014 (op_i == CIPH_INV) ? aes_mvm(data_i
^ 8'h63, S2X) :
1015
aes_mvm(data_i, A2X);
1016
1017 // Convert mask to normal basis X.
1018 // The addition of constant 8'h63 prior to the affine
transformation is skipped.
1019 assign in_mask_basis_x = (op_i == CIPH_FWD) ?
aes_mvm(mask_i, A2X) :
1020 (op_i == CIPH_INV) ?
aes_mvm(mask_i, S2X) :
1021
aes_mvm(mask_i, A2X);
1022
1023 // Do the inversion in normal basis X.
1024 aes_dom_inverse_gf2p8 #(
1025   .PipelineMul ( PipelineMul )
1026 ) u_aes_dom_inverse_gf2p8 (
1027   .clk_i ( clk_i ),
1028   .rst_ni ( rst_ni ),
1029   .we_i ( we ),
1030   .a_y ( in_data_basis_x ), // input
1031   .b_y ( in_mask_basis_x ), // input
1032   .prd_i ( in_prd ), // input
1033   .a_y_inv ( out_data_basis_x ), // output
1034   .b_y_inv ( out_mask_basis_x ), // output
1035   .prd_o ( out_prd ) // output
1036 );
1037
1038 // Convert data to basis S or A.
1039 assign data_o = (op_i == CIPH_FWD) ?
(aes_mvm(out_data_basis_x, X2S) ^ 8'h63) :
1040 (op_i == CIPH_INV) ?
(aes_mvm(out_data_basis_x, X2A)) :
1041
(aes_mvm(out_data_basis_x, X2S) ^ 8'h63);
1042
1043 // Convert mask to basis S or A.
1044 // The addition of constant 8'h63 following the affine
transformation is skipped.
1045 assign mask_o = (op_i == CIPH_FWD) ?
aes_mvm(out_mask_basis_x, X2S) :
1046 (op_i == CIPH_INV) ?
aes_mvm(out_mask_basis_x, X2A) :
1047
aes_mvm(out_mask_basis_x, X2S);
1048
1049 // Counter register
1050 logic [2:0] count_d, count_q;
1051 assign count_d = (out_req_o && out_ack_i) ? '0 :
1052 out_req_o ? count_q :
1053 en_i ? count_q + 3'd1 :
count_q;
1054 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_count
1055   if (!rst_ni) begin
1056     count_q <= '0;
1057   end else begin
1058     count_q <= count_d;
1059   end
1060 end
1061 assign out_req_o = en_i & count_q == 3'd4;
1062
1063 // Write enable signals for internal registers
1064 assign we[0] = en_i & count_q == 3'd0;
1065 assign we[1] = en_i & count_q == 3'd1;
1066 assign we[2] = en_i & count_q == 3'd2;
1067 assign we[3] = en_i & count_q == 3'd3;
1068
1069 // PRD forwarding for the individual stages. We get 8 bits
from the PRNG for usage in Stage 1.
1070 // Stages 2, 3 and 4 are driven by other S-Box instances.
1071 assign in_prd = {prd_1: prd_i[7:0],
1072 prd_2: prd_i[11:8],
1073 prd_3: prd_i[19:12],
1074 prd_4: prd_i[27:20]};
1075 assign prd_o = {out_prd.prd_3, out_prd.prd_2,
out_prd.prd_1};
1076
1077 endmodule

```

A.15 lowRISC_opentitan/hw/ip/aes/rtl/aes_sbox.sv (144줄, 4,441B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES SBox
6
7 `include "prim_assert.sv"
8
9 module aes_sbox import aes_pkg::*;
10 #(
11   parameter sbox_impl_e SecSBoxImpl = SBoxImplLut
12 ) (

```

```

13 input logic clk_i,
14 input logic rst_ni,
15 input logic en_i,
16 output logic out_req_o,
17 input logic out_ack_i,
18 input ciph_op_e op_i,
19 input logic [7:0] data_i,
20 input logic [7:0] mask_i,
21 input logic [WidthPRDSBox+19:0] prd_i,
22 output logic [7:0] data_o,
23 output logic [7:0] mask_o,
24 output logic [19:0] prd_o
25 );
26
27 // Create a lint error to reduce the risk of accidentally
using a less secure SBox
28 // implementation.
29 `ASSERT_STATIC_LINT_ERROR(AesSBoxSecSBoxImplNonDefault,
SecSBoxImpl == SBoxImplDom)
30
31 localparam bit SBoxMasked = (SecSBoxImpl ==
SBoxImplCanrightMasked ||
32 SecSBoxImpl ==
SBoxImplCanrightMaskedNoreuse ||
33 SecSBoxImpl == SBoxImplDom) ?
1'b1 : 1'b0;
34
35 localparam bit SBoxSingleCycle = (SecSBoxImpl ==
SBoxImplDom) ? 1'b0 : 1'b1;
36
37 if (!SBoxMasked) begin : gen_sbox_unmasked
38 // Tie off unused inputs.
39 logic unused_clk;
40 logic unused_rst;
41 logic [7:0] unused_mask;
42 logic [WidthPRDSBox+19:0] unused_prd;
43 assign unused_clk = clk_i;
44 assign unused_rst = rst_ni;
45 assign unused_mask = mask_i;
46 assign unused_prd = prd_i;
47
48 if (SecSBoxImpl == SBoxImplCanright) begin :
gen_sbox_canright
49 aes_sbox_canright u_aes_sbox (
50 .op_i ( op_i ),
51 .data_i ( data_i ),
52 .data_o ( data_o )
53 );
54
55 end else begin : gen_sbox_lut // SecSBoxImpl ==
SBoxImplLut
56 aes_sbox_lut u_aes_sbox (
57 .op_i ( op_i ),
58 .data_i ( data_i ),
59 .data_o ( data_o )
60 );
61 end
62
63 assign mask_o = '0;
64 assign prd_o = '0;
65
66 end else begin : gen_sbox_masked
67
68 // SEC_CM: KEY.MASKING
69 if (SecSBoxImpl == SBoxImplDom) begin : gen_sbox_dom
70
71 aes_sbox_dom u_aes_sbox (
72 .clk_i ( clk_i ),
73 .rst_ni ( rst_ni ),
74 .en_i ( en_i ),
75 .out_req_o ( out_req_o ),
76 .out_ack_i ( out_ack_i ),
77 .op_i ( op_i ),
78 .data_i ( data_i ),
79 .mask_i ( mask_i ),
80 .prd_i ( prd_i[27:0] ),
81 .data_o ( data_o ),
82 .mask_o ( mask_o ),
83 .prd_o ( prd_o )
84 );
85
86 `ASSERT_INIT(AesWidthPRDSBox, WidthPRDSBox == 8)
87
88 end else if (SecSBoxImpl == SBoxImplCanrightMaskedNoreuse)
begin :
89 gen_sbox_canright_masked_noreuse
90 // Tie off unused inputs.
91 logic unused_clk;
92 logic unused_rst;
93 logic [19:0] unused_prd;
94 assign unused_clk = clk_i;
95 assign unused_rst = rst_ni;
96 assign unused_prd = prd_i[WidthPRDSBox+19:WidthPRDSBox];
97
98 aes_sbox_canright_masked_noreuse u_aes_sbox (
99 .op_i ( op_i ),
100 .data_i ( data_i ),
101 .mask_i ( mask_i ),
102 .prd_i ( prd_i[17:0] ),
103 .data_o ( data_o ),

```

```

104 .mask_o ( mask_o )
105 );
106
107 assign prd_o = '0;
108
109 `ASSERT_INIT(AesWidthPRDSBox, WidthPRDSBox == 18)
110
111 end else begin : gen_sbox_canright_masked // SecSBoxImpl
== SBoxImplCanrightMasked
112 // Tie off unused inputs.
113 logic unused_clk;
114 logic unused_rst;
115 logic [19:0] unused_prd;
116 assign unused_clk = clk_i;
117 assign unused_rst = rst_ni;
118 assign unused_prd = prd_i[WidthPRDSBox+19:WidthPRDSBox];
119
120 aes_sbox_canright_masked u_aes_sbox (
121 .op_i ( op_i ),
122 .data_i ( data_i ),
123 .mask_i ( mask_i ),
124 .prd_i ( prd_i[7:0] ),
125 .data_o ( data_o ),
126 .mask_o ( mask_o )
127 );
128
129 assign prd_o = '0;
130
131 `ASSERT_INIT(AesWidthPRDSBox, WidthPRDSBox == 8)
132 end
133
134 if (SBoxSingleCycle) begin : gen_req_singlecycle
135 // Tie off unused inputs.
136 logic unused_out_ack;
137 assign unused_out_ack = out_ack_i;
138
139 // Signal that we have valid output right away.
140 assign out_req_o = en_i;
141 end
142
143 endmodule
144

```

A.16 lowRISC_opentitan/hw/ip/aes/rtl/aes_mix_columns.sv (32줄, 863B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES MixColumns
6
7 module aes_mix_columns (
8 input aes_pkg::ciph_op_e op_i,
9 input logic [3:0][3:0][7:0] data_i_transposed,
10 output logic [3:0][3:0][7:0] data_o
11 );
12
13 import aes_pkg::*;
14
15 // Transpose to operate on columns
16 logic [3:0][3:0][7:0] data_i_transposed;
17 logic [3:0][3:0][7:0] data_o_transposed;
18
19 assign data_i_transposed = aes_transpose(data_i);
20
21 // Individually mix columns
22 for (genvar i = 0; i < 4; i++) begin : gen_mix_column
23 aes_mix_single_column u_aes_mix_column_i (
24 .op_i ( op_i ),
25 .data_i ( data_i_transposed[i] ),
26 .data_o ( data_o_transposed[i] )
27 );
28 end
29
30 assign data_o = aes_transpose(data_o_transposed);
31
32 endmodule

```

A.17 lowRISC_opentitan/hw/ip/aes/rtl/aes_mix_single_column.sv (70줄, 1,923B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES MixColumns for one single column of the state matrix
6 //
7 // For details, see Equations 4-7 of:
8 // Satoh et al., "A Compact Rijndael Hardware Architecture
with S-Box Optimization"
9
10 module aes_mix_single_column (
11 input aes_pkg::ciph_op_e op_i,
12 input logic [3:0][7:0] data_i,

```

```

13 output logic [3:0][7:0] data_o
14 );
15
16 import aes_pkg::*;
17
18 logic [3:0][7:0] x;
19 logic [1:0][7:0] y;
20 logic [1:0][7:0] z;
21
22 logic [3:0][7:0] x_mul2;
23 logic [1:0][7:0] y_pre_mul4;
24 logic [7:0] y2, y2_pre_mul2;
25
26 logic [1:0][7:0] z_muxed;
27
28 // Drive x
29 assign x[0] = data_i[0] ^ data_i[3];
30 assign x[1] = data_i[3] ^ data_i[2];
31 assign x[2] = data_i[2] ^ data_i[1];
32 assign x[3] = data_i[1] ^ data_i[0];
33
34 // Mul2(x)
35 for (genvar i = 0; i < 4; i++) begin : gen_x_mul2
36     assign x_mul2[i] = aes_mul2(x[i]);
37 end
38
39 // Drive y_pre_mul4
40 assign y_pre_mul4[0] = data_i[3] ^ data_i[1];
41 assign y_pre_mul4[1] = data_i[2] ^ data_i[0];
42
43 // Mul4(y_pre_mul4)
44 for (genvar i = 0; i < 2; i++) begin : gen_mul4
45     assign y[i] = aes_mul4(y_pre_mul4[i]);
46 end
47
48 // Drive y2_pre_mul2
49 assign y2_pre_mul2 = y[0] ^ y[1];
50
51 // Mul2(y)
52 assign y2 = aes_mul2(y2_pre_mul2);
53
54 // Drive z
55 assign z[0] = y2 ^ y[0];
56 assign z[1] = y2 ^ y[1];
57
58 // Mux z
59 assign z_muxed[0] = (op_i == CIPH_FWD) ? 8'b0 :
60                     (op_i == CIPH_INV) ? z[0] : 8'b0;
61 assign z_muxed[1] = (op_i == CIPH_FWD) ? 8'b0 :
62                     (op_i == CIPH_INV) ? z[1] : 8'b0;
63
64 // Drive outputs
65 assign data_o[0] = data_i[1] ^ x_mul2[3] ^ x[1] ^
66 z_muxed[1];
67 assign data_o[1] = data_i[0] ^ x_mul2[2] ^ x[1] ^
68 z_muxed[0];
69 assign data_o[2] = data_i[3] ^ x_mul2[1] ^ x[3] ^
70 z_muxed[1];
71 assign data_o[3] = data_i[2] ^ x_mul2[0] ^ x[3] ^
72 z_muxed[0];
73
74 endmodule

```

A.18 lowRISC_opentitan/hw/ip/aes/rtl/aes_key_expand.sv (503줄, 16,955B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // AES KeyExpand
7
8 `include "prim_assert.sv"
9
10 module aes_key_expand import aes_pkg::*;
11 #()
12     parameter bit AES192Enable = 1,
13     parameter bit SecMasking = 0,
14     parameter sbbox_impl_e SecSBBoxImpl = SBBoxImplLut,
15     localparam int NumShares = SecMasking ? 2 : 1 //
16     derived parameter
17     (
18         input logic clk_i,
19         input logic rst_ni,
20         input logic cfg_valid_i,
21         input logic op_i,
22         input logic sp2v_e,
23         input logic en_i,
24         input logic prd_we_i,
25         input logic sp2v_e,
26         input logic out_req_o,
27         input logic sp2v_e,
28         input logic out_ack_i,
29         input logic clear_i,
30         input logic [3:0] round_i,
31         input logic key_len_i,
32         input logic [7:0][31:0] key_i [NumShares],
33         input logic [7:0][31:0] key_o [NumShares],
34         input logic [WidthPRDKey-1:0] prd_i,
35         output logic err_o

```

```

32 );
33
34 sp2v_e en;
35 logic en_err;
36 sp2v_e out_ack;
37 logic out_ack_err;
38
39 logic [7:0] rcon_d, rcon_q;
40 logic rcon_we;
41 logic use_rcon;
42
43 logic [3:0] rnd;
44 logic [3:0] rnd_type;
45
46 logic [31:0] spec_in_128 [NumShares];
47 logic [31:0] spec_in_192 [NumShares];
48 logic [31:0] rot_word_in [NumShares];
49 logic [31:0] rot_word_out [NumShares];
50 logic use_rot_word;
51 logic prd_we, prd_we_force, prd_we_inhibit;
52 logic [31:0] sub_word_in, sub_word_out;
53 logic [3:0] sub_word_out_req;
54 logic [31:0] sw_in_mask, sw_out_mask;
55 logic [7:0] rcon_add_in, rcon_add_out;
56 logic [31:0] rcon_added;
57
58 logic [31:0] irregular [NumShares];
59 logic [7:0][31:0] regular [NumShares];
60
61 // cfg_valid_i is used for gating assertions only.
62 logic unused_cfg_valid;
63 assign unused_cfg_valid = cfg_valid_i;
64
65 // Get a shorter reference.
66 assign rnd = round_i;
67
68 // For AES-192, there are four different types of rounds.
69 always_comb begin : get_rnd_type
70     if (AES192Enable) begin
71         rnd_type[0] = (rnd == 0);
72         rnd_type[1] = (rnd == 1 || rnd == 4 || rnd == 7 || rnd
73 == 10);
74         rnd_type[2] = (rnd == 2 || rnd == 5 || rnd == 8 || rnd
75 == 11);
76         rnd_type[3] = (rnd == 3 || rnd == 6 || rnd == 9 || rnd
77 == 12);
78     end else begin
79         rnd_type = '0;
80     end
81 end
82
83 // Irregular part involving Rcon, RotWord & SubWord //
84 // Depending on key length and round, RotWord may not be
85 used.
86 assign use_rot_word = (key_len_i == AES_256 && rnd[0] ==
87 1'b0) ? 1'b0 : 1'b1;
88
89 // Depending on operation, key length and round, Rcon may
90 not be used thus must not be updated.
91 always_comb begin : rcon_usage
92     use_rcon = 1'b1;
93
94     if (AES192Enable) begin
95         if (key_len_i == AES_192 &&
96             ((op_i == CIPH_FWD && rnd_type[1]) ||
97              (op_i == CIPH_INV && (rnd_type[0] ||
98 rnd_type[3])))) begin
99             use_rcon = 1'b0;
100         end
101     end
102
103     if (key_len_i == AES_256 && rnd[0] == 1'b0) begin
104         use_rcon = 1'b0;
105     end
106 end
107
108 // Generate Rcon
109 always_comb begin : rcon_update
110     rcon_d = rcon_q;
111
112     if (clear_i) begin
113         rcon_d = (op_i == CIPH_FWD)
114 ?
115 8'h01 :
116 ((op_i == CIPH_INV) && (key_len_i == AES_128)) ?
117 8'h36 :
118 ((op_i == CIPH_INV) && (key_len_i == AES_192)) ?
119 8'h80 :
120 ((op_i == CIPH_INV) && (key_len_i == AES_256)) ?
121 8'h40 : 8'h01;
122     end else begin
123         rcon_d = (op_i == CIPH_FWD) ? aes_mul2(rcon_q) :
124                 (op_i == CIPH_INV) ? aes_div2(rcon_q) : 8'h01;
125     end
126 end

```

```

118
119 // Advance.
120 assign rcon_we = clear_i | use_rcon &
121 (en == SP2V_HIGH) & (out_req_o == SP2V_HIGH) & (out_ack
== SP2V_HIGH);
122
123 // Rcon register
124 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_rcon
125   if (!rst_ni) begin
126     rcon_q <= '0;
127   end else if (rcon_we) begin
128     rcon_q <= rcon_d;
129   end
130 end
131
132 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_rot_word_out
133   // Special input, equivalent to key_o[3] in the used cases
134   assign spec_in_128[s] = key_i[s][3] ^ key_i[s][2];
135   assign spec_in_192[s] = AES192Enable ? key_i[s][5] ^
key_i[s][1] ^ key_i[s][0] : '0;
136
137   // Select input
138   always_comb begin : rot_word_in_mux
139     unique case (key_len_i)
140
141       ////////////
142       // AES-128 //
143       ////////////
144       AES_128: begin
145         unique case (op_i)
146           CIPH_FWD: rot_word_in[s] = key_i[s][3];
147           CIPH_INV: rot_word_in[s] = spec_in_128[s];
148           default: rot_word_in[s] = key_i[s][3];
149         endcase
150       end
151
152       ////////////
153       // AES-192 //
154       ////////////
155       AES_192: begin
156         if (AES192Enable) begin
157           unique case (op_i)
158             CIPH_FWD: begin
159               rot_word_in[s] = rnd_type[0] ? key_i[s]
[5] :
160               rnd_type[2] ? key_i[s]
[5] :
161               rnd_type[3] ?
spec_in_192[s] : key_i[s][3];
162             end
163             CIPH_INV: begin
164               rot_word_in[s] = rnd_type[1] ? key_i[s][3] :
rnd_type[2] ? key_i[s][1] :
165               key_i[s][3];
166             end
167             default: rot_word_in[s] = key_i[s][3];
168           endcase
169         end else begin
170           rot_word_in[s] = key_i[s][3];
171         end
172       end
173
174       ////////////
175       // AES-256 //
176       ////////////
177       AES_256: begin
178         unique case (op_i)
179           CIPH_FWD: rot_word_in[s] = key_i[s][7];
180           CIPH_INV: rot_word_in[s] = key_i[s][3];
181           default: rot_word_in[s] = key_i[s][7];
182         endcase
183       end
184
185       default: rot_word_in[s] = key_i[s][3];
186     endcase
187   end
188
189   // RotWord: cyclic byte shift
190   assign rot_word_out[s] =
aes_circ_byte_shift(rot_word_in[s], 2'h3);
191   end
192
193   // Mux input for SubWord
194   assign sub_word_in = use_rot_word ? rot_word_out[0] :
rot_word_in[0];
195
196   // Masking
197   if (!SecMasking) begin : gen_no_sw_in_mask
198     // The mask share is ignored anyway, it can be 0.
199     assign sw_in_mask = '0;
200
201     // Tie-off unused signals.
202     logic [31:0] unused_sw_out_mask;
203     assign unused_sw_out_mask = sw_out_mask;
204
205   end else begin : gen_sw_in_mask
206     // The input mask is the mask share of rot_word_in/out.
207     assign sw_in_mask = use_rot_word ? rot_word_out[1] :
rot_word_in[1];

```

```

208   end
209
210   // Make sure that whenever the data/mask inputs of the S-
Boxes update, the buffered PRD is
211   // updated in sync. There are two special cases we need to
handle here:
212   // - For AES-256, the initial round is short (no round key
computation). But the data/mask inputs
213   // are updated either way. Thus, we need to force a PRD
update as well.
214   // - For AES-192 in FWD mode, the data/mask inputs aren't
updated in Round 1, 4, 7 and 10. Thus,
215   // we need to inhibit PRD updates triggered at the end of
Round 0, 3, 6 and 9.
216   assign prd_we_force = (key_len_i == AES_256) & (rnd == 0);
217   assign prd_we_inhibit = (key_len_i == AES_192) & (op_i ==
CIPH_FWD) &
218   (rnd == 0 || rnd == 3 || rnd == 6 || rnd == 9);
219   assign prd_we = (prd_we_i & ~prd_we_inhibit) | prd_we_force;
220
221   // PRD buffering
222   logic [WidthPRDKey-1:0] prd_q;
223
224   if (!SecMasking) begin : gen_no_prd_buffer
225     // The masks are ignored anyway.
226     assign prd_q = prd_i;
227
228     // Tie-off unused signals.
229     logic unused_prd_we;
230     assign unused_prd_we = prd_we;
231
232   end else begin : gen_prd_buffer
233     // PRD buffer stage to:
234     // 1. Make sure the S-Boxes get always presented new data/
mask inputs together with fresh PRD
235     // for remasking.
236
237     // 2. Prevent glitches originating from inside the masking
PRNG from propagating into the
238     // masked S-Boxes.
239     always_ff @(posedge clk_i or negedge rst_ni) begin :
prd_reg
240       if (!rst_ni) begin
241         prd_q <= '0;
242       end else if (prd_we) begin
243         prd_q <= prd_i;
244       end
245     end
246
247     // SubWord - individually substitute bytes.
248     // Every DOM S-Box instance consumes 28 bits of randomness
but itself produces 20 bits for use in
249     // another S-Box instance. For other S-Box implementations,
only the bits corresponding to prd_q
250     // are used. Other bits are ignored and tied to 0.
251     logic [3:0][WidthPRDSBox+19:0] in_prd;
252     logic [3:0] [19:0] out_prd;
253
254     for (genvar i = 0; i < 4; i++) begin : gen_sbox
255
256       // Rotate the randomness produced by the S-Boxes. The LSBs
are taken from the masking PRNG
257       // (prd_q) whereas the MSBs are produced by the other S-
Box instances.
258       assign in_prd[i] = {out_prd[aes_rot_int(i,4)],
prd_q[WidthPRDSBox*i +: WidthPRDSBox]};
259
260       aes_sbox #(
261         .SecSBoxImpl ( SecSBoxImpl )
262       ) u_aes_sbox_i (
263         .clk_i      ( clk_i ),
264         .rst_ni     ( rst_ni ),
265         .en_i       ( en == SP2V_HIGH ),
266         .out_req_o   ( sub_word_out_req[i] ),
267         .out_ack_i   ( out_ack == SP2V_HIGH ),
268         .op_i        ( CIPH_FWD ),
269         .data_i      ( sub_word_in[8*i +: 8] ),
270         .mask_i      ( sw_in_mask[8*i +: 8] ),
271         .prd_i       ( in_prd[i] ),
272         .data_o      ( sub_word_out[8*i +: 8] ),
273         .mask_o      ( sw_out_mask[8*i +: 8] ),
274         .prd_o       ( out_prd[i] )
275       );
276     end
277
278     // Add Rcon
279     assign rcon_add_in = sub_word_out[7:0];
280     assign rcon_add_out = rcon_add_in ^ rcon_q;
281     assign rcon_added = {sub_word_out[31:8], rcon_add_out};
282
283     // Mux output coming from Rcon & SubWord
284     for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_irregular
285       if (s == 0) begin : gen_irregular_rcon
286         // The (masked) key share
287         assign irregular[s] = use_rcon ? rcon_added :
sub_word_out;
288       end else begin : gen_irregular_no_rcon
289         // The mask share
290         assign irregular[s] = sw_out_mask;

```

```

290     end
291 end
292
293 //////////////////////////////////////////////////
294 // The more regular part //
295 //////////////////////////////////////////////////
296
297 // To reduce muxing resources, we re-use existing
298 // connections for unused words and default cases.
299 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_regular
300     always_comb begin : drive_regular
301         unique case (key_len_i)
302
303             ///////////////
304             // AES-128 //
305             ///////////////
306             AES_128: begin
307                 // key_o[7:4] not used
308                 regular[s][7:4] = key_i[s][3:0];
309
310                 regular[s][0] = irregular[s] ^ key_i[s][0];
311                 unique case (op_i)
312                     CIPH_FWD: begin
313                         for (int i = 1; i < 4; i++) begin
314                             regular[s][i] = regular[s][i-1] ^ key_i[s][i];
315                         end
316                     end
317                     CIPH_INV: begin
318                         for (int i = 1; i < 4; i++) begin
319                             regular[s][i] = key_i[s][i-1] ^ key_i[s][i];
320                         end
321                     end
322                 end
323
324                 default: regular[s] = {key_i[s][3:0], key_i[s]
[7:4]};
325             endcase
326         end
327
328         ///////////////
329         // AES-192 //
330         ///////////////
331         AES_192: begin
332             // key_o[7:6] not used
333             regular[s][7:6] = key_i[s][3:2];
334
335             if (AES192Enable) begin
336                 unique case (op_i)
337                     CIPH_FWD: begin
338                         if (rnd_type[0]) begin
339                             // Shift down four upper most words
340                             regular[s][3:0] = key_i[s][5:2];
341                             // Generate Words 6 and 7
342                             regular[s][4] = irregular[s] ^ key_i[s]
[0];
343                             regular[s][5] = regular[s][4] ^ key_i[s]
[1];
344                         end else begin
345                             // Shift down two upper most words
346                             regular[s][1:0] = key_i[s][5:4];
347                             // Generate new upper four words
348                             for (int i = 0; i < 4; i++) begin
349                                 if ((i == 0 && rnd_type[2]) ||
350                                     (i == 2 && rnd_type[3])) begin
351                                     regular[s][i+2] = irregular[s] ^
key_i[s][i];
352                                 end else begin
353                                     regular[s][i+2] = regular[s][i+1] ^
key_i[s][i];
354                                 end
355                             end
356                         end // rnd_type[0]
357                     end
358
359                     CIPH_INV: begin
360                         if (rnd_type[0]) begin
361                             // Shift up four lowest words
362                             regular[s][5:2] = key_i[s][3:0];
363                             // Generate Word 44 and 45
364                             for (int i = 0; i < 2; i++) begin
365                                 regular[s][i] = key_i[s][3+i] ^ key_i[s]
[3+i+1];
366                             end
367                         end else begin
368                             // Shift up two lowest words
369                             regular[s][5:4] = key_i[s][1:0];
370                             // Generate new lower four words
371                             for (int i = 0; i < 4; i++) begin
372                                 if ((i == 0 && rnd_type[1]) ||
373                                     (i == 0 && rnd_type[2])) begin
374                                     regular[s][i] = irregular[s] ^ key_i[s]
[i+2];
375                                 end else begin
376                                     regular[s][i] = key_i[s][i+1] ^ key_i[s]
[i+2];
377                                 end
378                             end
379                         end // rnd_type[0]
380                     end
381                 end

```

```

382         default: regular[s] = {key_i[s][3:0], key_i[s]
[7:4]};
383     endcase
384
385     end else begin
386         regular[s] = {key_i[s][3:0], key_i[s][7:4]};
387     end // AES192Enable
388 end
389
390 ///////////////
391 // AES-256 //
392 ///////////////
393 AES_256: begin
394     unique case (op_i)
395         CIPH_FWD: begin
396             if (rnd == 0) begin
397                 // Round 0: Nothing to be done
398                 // The Full Key registers are not updated
399                 regular[s] = {key_i[s][3:0], key_i[s][7:4]};
400             end else begin
401                 // Shift down old upper half
402                 regular[s][3:0] = key_i[s][7:4];
403                 // Generate new upper half
404                 regular[s][4] = irregular[s] ^ key_i[s][0];
405                 for (int i = 1; i < 4; i++) begin
406                     regular[s][i+4] = regular[s][i+4-1] ^
key_i[s][i];
407                 end
408                 end // rnd == 0
409             end
410
411             CIPH_INV: begin
412                 if (rnd == 0) begin
413                     // Round 0: Nothing to be done
414                     // The Full Key registers are not updated
415                     regular[s] = {key_i[s][3:0], key_i[s][7:4]};
416                 end else begin
417                     // Shift up old lower half
418                     regular[s][7:4] = key_i[s][3:0];
419                     // Generate new lower half
420                     regular[s][0] = irregular[s] ^ key_i[s][4];
421                     for (int i = 0; i < 3; i++) begin
422                         regular[s][i+1] = key_i[s][4+i] ^ key_i[s]
[4+i+1];
423                     end
424                     end // rnd == 0
425                 end
426
427                 default: regular[s] = {key_i[s][3:0], key_i[s]
[7:4]};
428             endcase
429         end
430
431         default: regular[s] = {key_i[s][3:0], key_i[s][7:4]};
432     endcase // key_len_i
433 end // drive_regular
434 end // gen_shares_regular
435
436 // Drive output
437 assign key_o = regular;
438 assign out_req_o = &sub_word_out_req ? SP2V_HIGH : SP2V_LOW;
439
440 //////////////////////////////////////////////////
441 // Sparsely Encoded Signals //
442 //////////////////////////////////////////////////
443
444 logic [Sp2VWidth-1:0] en_raw;
445 aes_sel_buf_chk #(
446     .Num ( Sp2VNum ),
447     .Width ( Sp2VWidth ),
448     .EnSecBuf ( 1'b1 )
449 ) u_aes_key_expand_en_buf_chk (
450     .clk_i ( clk_i ),
451     .rst_ni ( rst_ni ),
452     .sel_i ( en_i ),
453     .sel_o ( en_raw ),
454     .err_o ( en_err )
455 );
456 assign en = sp2v_e'(en_raw);
457
458 logic [Sp2VWidth-1:0] out_ack_raw;
459 aes_sel_buf_chk #(
460     .Num ( Sp2VNum ),
461     .Width ( Sp2VWidth ),
462     .EnSecBuf ( 1'b1 )
463 ) u_aes_key_expand_out_ack_buf_chk (
464     .clk_i ( clk_i ),
465     .rst_ni ( rst_ni ),
466     .sel_i ( out_ack_i ),
467     .sel_o ( out_ack_raw ),
468     .err_o ( out_ack_err )
469 );
470 assign out_ack = sp2v_e'(out_ack_raw);
471
472 // Collect encoding errors.
473 assign err_o = en_err | out_ack_err;
474
475 //////////////////////////////////////////////////
476 // Assertions //
477 //////////////////////////////////////////////////
478

```

```

479 // Create a lint error to reduce the risk of accidentally
disabling the masking.
480 `ASSERT_STATIC_LINT_ERROR(AesKeyExpandSecMaskingNonDefault,
SecMasking == 1)
481
482 // Cipher core masking requires a masked SBox and vice
versa.
483 `ASSERT_INIT(AesMaskedCoreAndSBox,
484 (SecMasking &&
485 (SecSBoxImpl == SBoxImplCanrightMasked ||
486 SecSBoxImpl == SBoxImplCanrightMaskedNoreuse ||
487 SecSBoxImpl == SBoxImplDom)) ||
488 (!SecMasking &&
489 (SecSBoxImpl == SBoxImplLut ||
490 SecSBoxImpl == SBoxImplCanright)))
491
492 // Selectors must be known/valid
493 `ASSERT(AesCiphOpValid, cfg_valid_i |-> op_i inside {
494 CIPH_FWD,
495 CIPH_INV
496 })
497 `ASSERT(AesKeyLenValid, cfg_valid_i |-> key_len_i inside {
498 AES_128,
499 AES_192,
500 AES_256
501 })
502
503 endmodule

```

A.19 lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv (932줄, 39,413B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core implementation
6 //
7 // This module contains the AES cipher core including, state
register, full key and decryption key
8 // registers as well as key expand module and control unit.
9 //
10 //
11 // Masking
12 // -----
13 //
14 // If the parameter "Masking" is set to one, first-order
masking is applied to the entire
15 // cipher core including key expand module. For details, see
Rivain et al., "Provably secure
16 // higher-order masking of AES" available at https://
eprint.iacr.org/2010/441.pdf .
17 //
18 //
19 // Details on the data formats
20 // -----
21 //
22 // This implementation uses 4-dimensional SystemVerilog arrays to
represent the AES state:
23 //
24 // logic [3:0][3:0][7:0] state_q [NumShares];
25 //
26 // The fourth dimension (unpacked) corresponds to the
different shares. The first element holds the
27 // (masked) data share whereas the other elements hold the
masks (masked implementation only).
28 // The three packed dimensions correspond to the 128-bit state
matrix per share. This
29 // implementation uses the same encoding as the Advanced
Encryption Standard (AES) FIPS Publication
30 // 197 available at https://www.nist.gov/publications/
advanced-encryption-standard-aes (see Section
31 // 3.4). An input sequence of 16 bytes (128-bit, left most
byte is the first one)
32 //
33 // b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
34 //
35 // is mapped to the state matrix as
36 //
37 // [ b0 b4 b8 b12 ]
38 // [ b1 b5 b9 b13 ]
39 // [ b2 b6 b10 b14 ]
40 // [ b3 b7 b11 b15 ] .
41 //
42 // This is mapped to three packed dimensions of SystemVerilog
array as follows:
43 // - The first dimension corresponds to the rows. Thus,
state_q[0] gives
44 // - The first row of the state matrix [ b0 b4 b8
b12 ], or
45 // - A 32-bit packed SystemVerilog array 32h' { b12, b8, b4,
b0 }.
46 //
47 // - The second dimension corresponds to the columns. To
access complete columns, the state matrix
48 // must be transposed first. Thus state_transposed =

```

```

aes_pkg::aes_transpose(state_q) and then
49 // state_transposed[1] gives
50 // - The second column of the state matrix [ b4 b5 b6
b7 ], or
51 // - A 32-bit packed SystemVerilog array 32h' { b7, b6, b5,
b4 }.
52 //
53 // - The third dimension corresponds to the bytes.
54 //
55 // Note that the CSRs are little-endian. The input sequence
above is provided to 32-bit DATA_IN_0 -
56 // DATA_IN_3 registers as
57 // MSB LSB
58 // - DATA_IN_0 32h' { b3 , b2 , b1 , b0 }
59 // - DATA_IN_1 32h' { b7 , b6 , b4 , b4 }
60 // - DATA_IN_2 32h' { b11, b10, b9 , b8 }
61 // - DATA_IN_3 32h' { b15, b14, b13, b12 } .
62 //
63 // The input state can thus be obtained by transposing the
content of the DATA_IN_0 - DATA_IN_3
64 // registers.
65 //
66 // Similarly, the implementation uses a 3-dimensional array to
represent the AES keys:
67 //
68 // logic [7:0][31:0] key_full_q [NumShares]
69 //
70 // The third dimension (unpacked) corresponds to the different
shares. The first element holds the
71 // (masked) key share whereas the other elements hold the
masks (masked implementation only).
72 // The two packed dimensions correspond to the 256-bit key per
share. This implementation uses
73 // the same encoding as the Advanced Encryption Standard (AES)
FIPS Publication
74 // 197 available at https://www.nist.gov/publications/
advanced-encryption-standard-aes .
75 //
76 // The first packed dimension corresponds to the 8 key words.
The second packed dimension
77 // corresponds to the 32 bits per key word. A key sequence of
32 bytes (256-bit, left most byte is
78 // the first one)
79 //
80 // b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14
b15 ... b28 b29 b30 b31
81 //
82 // is mapped to the key words and registers (little-endian) as
83 // MSB LSB
84 // - KEY_SHARE0_0 32h' { b3 , b2 , b1 , b0 }
85 // - KEY_SHARE0_1 32h' { b7 , b6 , b4 , b4 }
86 // - KEY_SHARE0_2 32h' { b11, b10, b9 , b8 }
87 // - KEY_SHARE0_3 32h' { b15, b14, b13, b12 }
88 // - KEY_SHARE0_4 32h' { . . . . }
89 // - KEY_SHARE0_5 32h' { . . . . }
90 // - KEY_SHARE0_6 32h' { . . . . }
91 // - KEY_SHARE0_7 32h' { b31, b30, b29, b28 } .
92
93 `include "prim_assert.sv"
94
95 module aes_cipher_core import aes_pkg::*;
96 #()
97 parameter bit AES192Enable = 1,
98 parameter bit CiphOpFwdOnly = 0,
99 parameter bit SecMasking = 1,
100 parameter bit sbbox_impl_e = SBoxImplDom,
101 parameter bit SecAllowForcingMasks = 0,
102 parameter bit SecSkipPRNGReseeding = 0,
103 parameter int unsigned EntropyWidth =
edn_pkg::ENDPOINT_BUS_WIDTH,
104
105 localparam int NumShares = SecMasking ?
2 : 1, // derived parameter
106
107 parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed =
RndCnstMaskingLfsrSeedDefault,
108 parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm =
RndCnstMaskingLfsrPermDefault
109 ) (
110 input logic clk_i,
111 input logic rst_ni,
112
113 // Input handshake signals
114 input sp2v_e in_valid_i,
115 output sp2v_e in_ready_o,
116
117 // Output handshake signals
118 output sp2v_e out_valid_o,
119 input sp2v_e out_ready_i,
120
121 // Control and sync signals
122 input logic cfg_valid_i, // Used for
gating assertions only.
123 input ciph_op_e op_i,
124 input key_len_e key_len_i,
125 input sp2v_e crypt_i,

```

```

126 output sp2v_e crypt_o,
127 input sp2v_e dec_key_gen_i,
128 output sp2v_e dec_key_gen_o,
129 input logic prng_reseed_i,
130 output logic prng_reseed_o,
131 input logic key_clear_i,
132 output logic key_clear_o,
133 input logic data_out_clear_i, // Re-
use the cipher core muxes.
134 output logic data_out_clear_o,
135 input logic alert_fatal_i,
136 output logic alert_o,
137
138 // Pseudo-random data for register clearing
139 input logic [3:0][3:0][7:0] prd_clearing_state_i
[NumShares],
140 input logic [7:0][31:0] prd_clearing_key_i
[NumShares],
141
142 // Masking PRNG
143 input logic force_masks_i, // Useful
for SCA only.
144 output logic [3:0][3:0][7:0] data_in_mask_o,
145 output logic entropy_req_o,
146 input logic entropy_ack_i,
147 input logic [EntropyWidth-1:0] entropy_i,
148
149 // I/O data & initial key
150 input logic [3:0][3:0][7:0] state_init_i
[NumShares],
151 input logic [7:0][31:0] key_init_i [NumShares],
152 output logic [3:0][3:0][7:0] state_o [NumShares]
153 );
154
155 // Signals
156 logic [3:0][3:0][7:0] state_d [NumShares];
157 logic [3:0][3:0][7:0] state_q [NumShares];
158 sp2v_e state_we_ctrl;
159 sp2v_e state_we;
160 logic [StateSelWidth-1:0] state_sel_raw;
161 state_sel_e state_sel_ctrl;
162 state_sel_e state_sel;
163 logic state_sel_err;
164
165 sp2v_e sub_bytes_en;
166 sp2v_e sub_bytes_out_req;
167 sp2v_e sub_bytes_out_ack;
168 logic sub_bytes_err;
169 logic [3:0][3:0][7:0] sub_bytes_out;
170 logic [3:0][3:0][7:0] sb_in_mask;
171 logic [3:0][3:0][7:0] sb_out_mask;
172 logic [3:0][3:0][7:0] shift_rows_in
[NumShares];
173 logic [3:0][3:0][7:0] shift_rows_out
[NumShares];
174 logic [3:0][3:0][7:0] mix_columns_out
[NumShares];
175 logic [3:0][3:0][7:0] add_round_key_in
[NumShares];
176 logic [3:0][3:0][7:0] add_round_key_out
[NumShares];
177 logic [AddRKSelWidth-1:0] add_rk_sel_raw;
178 add_rk_sel_e add_rk_sel_ctrl;
179 add_rk_sel_e add_rk_sel;
180 logic add_rk_sel_err;
181
182 logic [7:0][31:0] key_full_d [NumShares];
183 logic [7:0][31:0] key_full_q [NumShares];
184 sp2v_e key_full_we_ctrl;
185 sp2v_e key_full_we;
186 logic [KeyFullSelWidth-1:0] key_full_sel_raw;
187 key_full_sel_e key_full_sel_ctrl;
188 key_full_sel_e key_full_sel;
189 logic key_full_sel_err;
190 logic [7:0][31:0] key_dec_d [NumShares];
191 logic [7:0][31:0] key_dec_q [NumShares];
192 sp2v_e key_dec_we_ctrl;
193 sp2v_e key_dec_we;
194 logic [KeyDecSelWidth-1:0] key_dec_sel_raw;
195 key_dec_sel_e key_dec_sel_ctrl;
196 key_dec_sel_e key_dec_sel;
197 logic key_dec_sel_err;
198 logic [7:0][31:0] key_expand_out
[NumShares];
199 ciph_op_e key_expand_op;
200 sp2v_e key_expand_en;
201 logic key_expand_prd_we;
202 sp2v_e key_expand_out_req;
203 sp2v_e key_expand_out_ack;
204 logic key_expand_err;
205 logic key_expand_clear;
206 logic [3:0] key_expand_round;
207 logic [KeyWordsSelWidth-1:0] key_words_sel_raw;
208 key_words_sel_e key_words_sel_ctrl;
209 key_words_sel_e key_words_sel;
210 logic key_words_sel_err;
211 logic [3:0][31:0] key_words [NumShares];
212 logic [3:0][3:0][7:0] key_bytes [NumShares];
213 logic [3:0][3:0][7:0] key_mix_columns_out
[NumShares];

```

```

214 logic [3:0][3:0][7:0] round_key [NumShares];
215 logic [RoundKeySelWidth-1:0] round_key_sel_raw;
216 round_key_sel_e round_key_sel_ctrl;
217 round_key_sel_e round_key_sel;
218 logic round_key_sel_err;
219
220 logic cfg_valid;
221 logic mux_sel_err;
222 logic sp_enc_err_d,
sp_enc_err_q;
223 logic op_err;
224
225 // Pseudo-random data for masking purposes
226 logic [WidthPRDMasking-1:0] prd_masking;
227 logic [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_d;
228 logic [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_q;
229 logic [WidthPRDKey-1:0] prd_key_expand;
230 logic prd_masking_upd;
231 logic prd_masking_rsd_req;
232 logic prd_masking_rsd_ack;
233
234 logic [3:0][3:0][7:0] data_in_mask;
235
236 // op_i is one-hot encoded. Check the provided value and
trigger an alert upon detecting invalid
237 // encodings.
238 assign op_err = ~(op_i == CIPH_FWD || op_i == CIPH_INV);
239 assign cfg_valid = cfg_valid_i & ~op_err;
240
241 // Data
242 // Data
243 // Data
244
245 // SEC_CM: DATA_REG.SEC_WIPE
246 // State registers
247 always_comb begin : state_mux
248     unique case (state_sel)
249         STATE_INIT: state_d = state_init_i;
250         STATE_ROUND: state_d = add_round_key_out;
251         STATE_CLEAR: state_d = prd_clearing_state_i;
252         default: state_d = prd_clearing_state_i;
253     endcase
254 end
255
256 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_reg
257     if (!rst_ni) begin
258         state_q <= '{default: '0};
259     end else if (state_we == SP2V_HIGH) begin
260         state_q <= state_d;
261     end
262 end
263
264 // Masking
265 if (!SecMasking) begin : gen_no_masks
266     // The masks are ignored anyway, they can be 0.
267     assign sb_in_mask = '0;
268     assign prd_masking = '0;
269
270 // Tie-off unused signals.
271 logic unused_entropy_ack;
272 logic [EntropyWidth-1:0] unused_entropy;
273 assign unused_entropy_ack = entropy_ack_i;
274 assign unused_entropy = entropy_i;
275 assign entropy_req_o = '1'b0;
276
277 logic unused_force_masks;
278 logic unused_prd_masking_upd;
279 logic unused_prd_masking_rsd_req;
280 assign unused_force_masks = force_masks_i;
281 assign unused_prd_masking_upd = prd_masking_upd;
282 assign unused_prd_masking_rsd_req = prd_masking_rsd_req;
283 assign prd_masking_rsd_ack = '1'b0;
284
285 logic [3:0][3:0][7:0] unused_sb_out_mask;
286 assign unused_sb_out_mask = sb_out_mask;
287
288 end else begin : gen_masks
289     // The input mask is the mask share of the state.
290     assign sb_in_mask = state_q[1];
291
292 // The masking PRNG generates:
293 // - the pseudo-random data (PRD) required by SubBytes,
294 // - the PRD required by the key expand module (has 4 S-
Boxes internally).
295 aes_prng_masking #(
296     .Width ( WidthPRDMasking ),
297     .EntropyWidth ( EntropyWidth ),
298     .SecAllowForcingMasks ( SecAllowForcingMasks ),
299     .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
300     .RndCnstLfSrSeed ( RndCnstMaskingLfSrSeed ),
301     .RndCnstLfSrPerm ( RndCnstMaskingLfSrPerm )
302 ) u_aes_prng_masking (
303     .clk_i ( clk_i ),
304     .rst_ni ( rst_ni ),
305     .force_masks_i ( force_masks_i ),
306     .data_update_i ( prd_masking_upd ),
307     .data_o ( prd_masking ),
308     .reseed_req_i ( prd_masking_rsd_req ),
309     .reseed_ack_o ( prd_masking_rsd_ack ),
310     .entropy_req_o ( entropy_req_o ),

```

```

311     .entropy_ack_i ( entropy_ack_i      ),
312     .entropy_i     ( entropy_i         ),
313 );
314 end
315
316 // Extract randomness for key expand module and SubBytes.
317 //
318 // The masking PRNG output has the following shape:
319 // prd_masking = { prd_key_expand, prd_sub_bytes_d }
320 assign prd_key_expand = prd_masking[WidthPRDMasking-1 -:
WidthPRDKey];
321 assign prd_sub_bytes_d = prd_masking[WidthPRDData-1 -:
WidthPRDData];
322
323 // PRD buffering
324 if (!SecMasking) begin : gen_no_prd_buffer
325 // The masks are ignored anyway.
326 assign prd_sub_bytes_q = prd_sub_bytes_d;
327
328 end else begin : gen_prd_buffer
329 // PRD buffer stage to:
330 // 1. Make sure the S-Boxes get always presented new data/
mask inputs together with fresh PRD
331 // for remasking.
332
333 // 2. Prevent glitches originating from inside the masking
PRNG from propagating into the
334 // masked S-Boxes.
335 always_ff @(posedge clk_i or negedge rst_ni) begin :
prd_sub_bytes_reg
336 if (!rst_ni) begin
337 prd_sub_bytes_q <= '0;
338 end else if (state_we == SP2V_HIGH) begin
339 prd_sub_bytes_q <= prd_sub_bytes_d;
340 end
341 end
342
343 // Convert the 3-dimensional prd_sub_bytes_q array to a 1-
dimensional packed array for the
344 // aes_prd_get_lsbs() function used below.
345 logic [WidthPRDData-1:0] prd_sub_bytes;
346 assign prd_sub_bytes = prd_sub_bytes_q;
347
348 // Extract randomness for masking the input data.
349 //
350 // The masking PRNG is used for generating both the PRD for
the S-Boxes/SubBytes operation as
351 // well as for the input data masks. When using any of the
masked Canright S-Box implementations,
352 // it is important that the SubBytes input masks (generated
by the PRNG in Round X-1) and the
353 // SubBytes output masks (generated by the PRNG in Round X)
are independent. This can be achieved
354 // by using e.g. an unrolled Bivium stream cipher primitive
inside the PRNG. Since the input data
355 // masks become the SubBytes input masks in the first round,
we select the same 8 bit lanes for
356 // the input data masks which are also used to form the
SubBytes output mask for the masked
357 // Canright S-Box implementations, i.e., the 8 LSBs of the
per S-Box PRD. In particular, we have:
358 //
359 // prd_masking = { prd_key_expand, ... , sb_prd[4],
sb_out_mask[4], sb_prd[0], sb_out_mask[0] }
360 //
361 // Where sb_out_mask[x] contains the SubBytes output mask
for byte x (when using a masked
362 // Canright S-Box implementation) and sb_prd[x] contains
additional PRD consumed by SubBytes for
363 // byte x.
364 //
365 // When using a masked S-Box implementation other than
Canright, we still select the 8 LSBs of
366 // the per-S-Box PRD to form the input data mask of the
corresponding byte. We do this to
367 // distribute the input data masks over all output bits the
masking PRNG. We do the extraction on
368 // a row basis.
369 localparam int unsigned WidthPRDRow = 4*WidthPRDSBox;
370 for (genvar i = 0; i < 4; i++) begin : gen_in_mask
371 assign data_in_mask[i] = aes_prd_get_lsbs(prd_sub_bytes[i
* WidthPRDRow -: WidthPRDRow]);
372 end
373
374 // Rotate the data input masks by 64 bits to ensure the data
input masks are independent
375 // from the PRD fed to the S-Boxes/SubBytes operation.
376 assign data_in_mask_o = {data_in_mask[1], data_in_mask[0],
data_in_mask[3], data_in_mask[2]};
377
378 // Cipher data path
379 aes_sub_bytes #(
380     .SecSBoxImpl ( SecSBoxImpl )
381 ) u_aes_sub_bytes (
382     .clk_i      ( clk_i      ),
383     .rst_ni     ( rst_ni     ),
384     .en_i       ( sub_bytes_en ),
385     .out_req_o  ( sub_bytes_out_req ),

```

```

386     .out_ack_i  ( sub_bytes_out_ack ),
387     .op_i       ( op_i       ),
388     .data_i     ( state_q[0] ),
389     .mask_i     ( sb_in_mask ),
390     .prd_i      ( prd_sub_bytes_q ),
391     .data_o     ( sub_bytes_out ),
392     .mask_o     ( sb_out_mask ),
393     .err_o      ( sub_bytes_err ),
394 );
395
396 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_shift_mux
397 if (s == 0) begin : gen_shift_in_data
398 // The (masked) data share
399 assign shift_rows_in[s] = sub_bytes_out;
400 end else begin : gen_shift_in_mask
401 // The mask share
402 assign shift_rows_in[s] = sb_out_mask;
403 end
404
405 aes_shift_rows u_aes_shift_rows (
406     .op_i      ( op_i      ),
407     .data_i    ( shift_rows_in[s] ),
408     .data_o    ( shift_rows_out[s] ),
409 );
410
411 aes_mix_columns u_aes_mix_columns (
412     .op_i      ( op_i      ),
413     .data_i    ( shift_rows_out[s] ),
414     .data_o    ( mix_columns_out[s] ),
415 );
416 end
417
418 always_comb begin : add_round_key_in_mux
419 unique case (add_rk_sel)
420     ADD_RK_INIT: add_round_key_in = state_q;
421     ADD_RK_ROUND: add_round_key_in = mix_columns_out;
422     ADD_RK_FINAL: add_round_key_in = shift_rows_out;
423     default: add_round_key_in = state_q;
424 endcase
425 end
426
427 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_add_round_key
428 assign add_round_key_out[s] = add_round_key_in[s] ^
round_key[s];
429 end
430
431 // Key
432 // Key
433 // Key
434
435 // SEC_CM: KEY.SEC_WIPE
436 // Full Key registers
437 always_comb begin : key_full_mux
438 unique case (key_full_sel)
439     KEY_FULL_ENC_INIT: key_full_d = key_init_i;
440     KEY_FULL_DEC_INIT: key_full_d = !CiphOpFwdOnly ?
key_dec_q : prd_clearing_key_i;
441     KEY_FULL_ROUND: key_full_d = key_expand_out;
442     KEY_FULL_CLEAR: key_full_d = prd_clearing_key_i;
443     default: key_full_d = prd_clearing_key_i;
444 endcase
445 end
446
447 always_ff @(posedge clk_i or negedge rst_ni) begin :
key_full_reg
448 if (!rst_ni) begin
449 key_full_q <= '{default: '0};
450 end else if (key_full_we == SP2V_HIGH) begin
451 key_full_q <= key_full_d;
452 end
453 end
454
455 if (!CiphOpFwdOnly) begin : gen_key_dec
456 // SEC_CM: KEY.SEC_WIPE
457 // Decryption Key registers
458 always_comb begin : key_dec_mux
459 unique case (key_dec_sel)
460     KEY_DEC_EXPAND: key_dec_d = key_expand_out;
461     KEY_DEC_CLEAR: key_dec_d = prd_clearing_key_i;
462     default: key_dec_d = prd_clearing_key_i;
463 endcase
464 end
465
466 always_ff @(posedge clk_i or negedge rst_ni) begin :
key_dec_reg
467 if (!rst_ni) begin
468 key_dec_q <= '{default: '0};
469 end else if (key_dec_we == SP2V_HIGH) begin
470 key_dec_q <= key_dec_d;
471 end
472 end
473 end else begin : gen_no_key_dec
474 // No Decryption Key registers
475 assign key_dec_q = '{default: '0};
476 assign key_dec_d = key_dec_q;
477
478 // Tie-off unused signals.
479 logic unused_key_dec;
480 always_comb begin

```

```

481     unused_key_dec = ^{key_dec_sel, key_dec_we};
482     for (int s = 0; s < NumShares; s++) begin
483         unused_key_dec ^= ^{key_dec_d[s]};
484     end
485 end
486 end
487
488 // Make sure that whenever the data/mask inputs of the S-
Boxes update, the internally buffered
489 // PRD is updated in sync.
490 assign key_expand_prd_we = (key_full_we == SP2V_HIGH) ?
1'b1 : 1'b0;
491
492 // Key expand data path
493 aes_key_expand #(
494     .AES192Enable ( AES192Enable ),
495     .SecMasking ( SecMasking ),
496     .SecSBoxImpl ( SecSBoxImpl )
497 ) u_aes_key_expand (
498     .clk_i ( clk_i ),
499     .rst_ni ( rst_ni ),
500     .cfg_valid_i ( cfg_valid ),
501     .op_i ( key_expand_op ),
502     .en_i ( key_expand_en ),
503     .prd_we_i ( key_expand_prd_we ),
504     .out_req_o ( key_expand_out_req ),
505     .out_ack_i ( key_expand_out_ack ),
506     .clear_i ( key_expand_clear ),
507     .round_i ( key_expand_round ),
508     .key_len_i ( key_len_i ),
509     .key_i ( key_full_q ),
510     .key_o ( key_expand_out ),
511     .prd_i ( prd_key_expand ),
512     .err_o ( key_expand_err )
513 );
514
515 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_round_key
516     always_comb begin : key_words_mux
517         unique case (key_words_sel)
518             KEY_WORDS_0123: key_words[s] = key_full_q[s][3:0];
519             KEY_WORDS_2345: key_words[s] = AES192Enable ?
key_full_q[s][5:2] : '0;
520             KEY_WORDS_4567: key_words[s] = key_full_q[s][7:4];
521             KEY_WORDS_ZERO: key_words[s] = '0;
522             default: key_words[s] = '0;
523         endcase
524     end
525
526 // Convert words to bytes (every key word contains one
column).
527 assign key_bytes[s] = aes_transpose(key_words[s]);
528
529 aes_mix_columns u_aes_key_mix_columns (
530     .op_i ( CIPH_INV ),
531     .data_i ( key_bytes[s] ),
532     .data_o ( key_mix_columns_out[s] )
533 );
534 end
535
536 always_comb begin : round_key_mux
537     unique case (round_key_sel)
538         ROUND_KEY_DIRECT: round_key = key_bytes;
539         ROUND_KEY_MIXED: round_key = !CiphOpFwdOnly ?
key_mix_columns_out : key_bytes;
540         default: round_key = key_bytes;
541     endcase
542 end
543
544 if (CiphOpFwdOnly) begin : gen_unused_key_mix_columns_out
545     // Tie-off unused signals.
546     logic unused_key_mix_columns_out;
547     always_comb begin
548         unused_key_mix_columns_out = 1'b0;
549         for (int s = 0; s < NumShares; s++) begin
550             unused_key_mix_columns_out ^=
^key_mix_columns_out[s];
551         end
552     end
553 end
554
555 // Control //
556 // Control //
557 // Control
558
559 aes_cipher_control #(
560     .CiphOpFwdOnly ( CiphOpFwdOnly ),
561     .SecMasking ( SecMasking ),
562     .SecSBoxImpl ( SecSBoxImpl )
563 ) u_aes_cipher_control (
564     .clk_i ( clk_i ),
565     .rst_ni ( rst_ni ),
566     .in_valid_i ( in_valid_i ),
567     .in_ready_o ( in_ready_o ),
568     .out_valid_o ( out_valid_o ),
569     .out_ready_i ( out_ready_i ),
570     .cfg_valid_i ( cfg_valid ),

```

```

575     .op_i ( op_i ),
576     .key_len_i ( key_len_i ),
577     .crypt_i ( crypt_i ),
578     .crypt_o ( crypt_o ),
579     .dec_key_gen_i ( dec_key_gen_i ),
580     .dec_key_gen_o ( dec_key_gen_o ),
581     .prng_reseed_i ( prng_reseed_i ),
582     .prng_reseed_o ( prng_reseed_o ),
583     .key_clear_i ( key_clear_i ),
584     .key_clear_o ( key_clear_o ),
585     .data_out_clear_i ( data_out_clear_i ),
586     .data_out_clear_o ( data_out_clear_o ),
587     .mux_sel_err_i ( mux_sel_err ),
588     .sp_enc_err_i ( sp_enc_err_q ),
589     .op_err_i ( op_err ),
590     .alert_fatal_i ( alert_fatal_i ),
591     .alert_o ( alert_o ),
592
593     .prng_update_o ( prd_masking_upd ),
594     .prng_reseed_req_o ( prd_masking_rsd_req ),
595     .prng_reseed_ack_i ( prd_masking_rsd_ack ),
596
597     .state_sel_o ( state_sel_ctrl ),
598     .state_we_o ( state_we_ctrl ),
599     .sub_bytes_en_o ( sub_bytes_en ),
600     .sub_bytes_out_req_i ( sub_bytes_out_req ),
601     .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
602     .add_rk_sel_o ( add_rk_sel_ctrl ),
603
604     .key_expand_op_o ( key_expand_op ),
605     .key_full_sel_o ( key_full_sel_ctrl ),
606     .key_full_we_o ( key_full_we_ctrl ),
607     .key_dec_sel_o ( key_dec_sel_ctrl ),
608     .key_dec_we_o ( key_dec_we_ctrl ),
609     .key_expand_en_o ( key_expand_en ),
610     .key_expand_out_req_i ( key_expand_out_req ),
611     .key_expand_out_ack_o ( key_expand_out_ack ),
612     .key_expand_clear_o ( key_expand_clear ),
613     .key_expand_round_o ( key_expand_round ),
614     .key_words_sel_o ( key_words_sel_ctrl ),
615     .round_key_sel_o ( round_key_sel_ctrl )
616 );
617
618 // Selectors //
619 // Selectors //
620 // Selectors //
621
622 // We use sparse encodings for these mux selector signals
and must ensure that:
623 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
624 // 2. The selector signal is always valid. More precisely,
an alert or SVA is triggered if a
625 // selector signal takes on an invalid value.
626 // 3. The alert signal remains asserted until reset even if
the selector signal becomes valid
627 // again. This is achieved by driving the control FSM
into the terminal error state whenever
628 // any mux selector signal becomes invalid.
629 //
630 // If any mux selector signal becomes invalid, the cipher
core further immediately de-asserts
631 // the out_valid_o signal to prevent any data from being
released.
632
633 aes_sel_buf_chk #(
634     .Num ( StateSelNum ),
635     .Width ( StateSelWidth ),
636     .EnSecBuf ( 1'b1 )
637 ) u_aes_state_sel_buf_chk (
638     .clk_i ( clk_i ),
639     .rst_ni ( rst_ni ),
640     .sel_i ( state_sel_ctrl ),
641     .sel_o ( state_sel_raw ),
642     .err_o ( state_sel_err )
643 );
644 assign state_sel = state_sel_e'(state_sel_raw);
645
646 aes_sel_buf_chk #(
647     .Num ( AddRKSelNum ),
648     .Width ( AddRKSelWidth ),
649     .EnSecBuf ( 1'b1 )
650 ) u_aes_add_rk_sel_buf_chk (
651     .clk_i ( clk_i ),
652     .rst_ni ( rst_ni ),
653     .sel_i ( add_rk_sel_ctrl ),
654     .sel_o ( add_rk_sel_raw ),
655     .err_o ( add_rk_sel_err )
656 );
657 assign add_rk_sel = add_rk_sel_e'(add_rk_sel_raw);
658
659 aes_sel_buf_chk #(
660     .Num ( KeyFullSelNum ),
661     .Width ( KeyFullSelWidth ),
662     .EnSecBuf ( 1'b1 )
663 ) u_aes_key_full_sel_buf_chk (
664     .clk_i ( clk_i ),
665     .rst_ni ( rst_ni ),
666     .sel_i ( key_full_sel_ctrl ),
667     .sel_o ( key_full_sel_raw ),
668     .err_o ( key_full_sel_err )

```

```

669 );
670 assign key_full_sel = key_full_sel_e'(key_full_sel_raw);
671
672 aes_sel_buf_chk #(
673   .Num      ( KeyDecSelNum      ),
674   .Width    ( KeyDecSelWidth    ),
675   .EnSecBuf ( 1'b1              )
676 ) u_aes_key_dec_sel_buf_chk (
677   .clk_i    ( clk_i              ),
678   .rst_ni   ( rst_ni             ),
679   .sel_i    ( key_dec_sel_ctrl   ),
680   .sel_o    ( key_dec_sel_raw    ),
681   .err_o    ( key_dec_sel_err    )
682 );
683 assign key_dec_sel = key_dec_sel_e'(key_dec_sel_raw);
684
685 aes_sel_buf_chk #(
686   .Num      ( KeywordsSelNum     ),
687   .Width    ( KeywordsSelWidth  ),
688   .EnSecBuf ( 1'b1              )
689 ) u_aes_key_words_sel_buf_chk (
690   .clk_i    ( clk_i              ),
691   .rst_ni   ( rst_ni             ),
692   .sel_i    ( key_words_sel_ctrl ),
693   .sel_o    ( key_words_sel_raw  ),
694   .err_o    ( key_words_sel_err  )
695 );
696 assign key_words_sel = key_words_sel_e'(key_words_sel_raw);
697
698 aes_sel_buf_chk #(
699   .Num      ( RoundKeySelNum     ),
700   .Width    ( RoundKeySelWidth  ),
701   .EnSecBuf ( 1'b1              )
702 ) u_aes_round_key_sel_buf_chk (
703   .clk_i    ( clk_i              ),
704   .rst_ni   ( rst_ni             ),
705   .sel_i    ( round_key_sel_ctrl ),
706   .sel_o    ( round_key_sel_raw  ),
707   .err_o    ( round_key_sel_err  )
708 );
709 assign round_key_sel = round_key_sel_e'(round_key_sel_raw);
710
711 // Signal invalid mux selector signals to control FSM which
will lock up and trigger an alert.
712 assign mux_sel_err = state_sel_err | add_rk_sel_err |
key_full_sel_err |
713   key_dec_sel_err | key_words_sel_err | round_key_sel_err;
714
715 ////////////////////////////////////////////////////
716 // Sparsely Encoded Signals //
717 ////////////////////////////////////////////////////
718
719 // We use sparse encodings for various critical signals and
must ensure that:
720 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
721 // 2. The sparsely encoded signal is always valid. More
precisely, an alert or SVA is triggered
722 // if a sparse signal takes on an invalid value.
723 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
724 // This is achieved by driving the control FSM into the
terminal error state whenever any
725 // sparsely encoded signal becomes invalid.
726 //
727 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
728 // the out_valid_o signal to prevent any data from being
released.
729
730 // We use vectors of sparsely encoded signals to reduce code
duplication.
731 localparam int unsigned NumSp2VSig = 3;
732 sp2v_e [NumSp2VSig-1:0] sp2v_sig;
733 sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
734 logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
735 logic [NumSp2VSig-1:0] sp2v_sig_err;
736
737 assign sp2v_sig[0] = state_we_ctrl;
738 assign sp2v_sig[1] = key_full_we_ctrl;
739 assign sp2v_sig[2] = key_dec_we_ctrl;
740
741 // All signals inside sp2v_sig are eventually converted to
single-rail signals.
742 localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf =
{NumSp2VSig{1'b1}};
743
744 // Individually check sparsely encoded signals.
745 for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
746   aes_sel_buf_chk #(
747     .Num      ( Sp2VNum          ),
748     .Width    ( Sp2VWidth        ),
749     .EnSecBuf ( Sp2VEnSecBuf[i] )
750   ) u_aes_sp2v_sig_buf_chk_i (
751     .clk_i    ( clk_i            ),
752     .rst_ni   ( rst_ni           ),
753     .sel_i    ( sp2v_sig[i]      ),
754     .sel_o    ( sp2v_sig_chk_raw[i] ),
755     .err_o    ( sp2v_sig_err[i]  )

```

```

756   );
757   assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
758   end
759
760   assign state_we      = sp2v_sig_chk[0];
761   assign key_full_we   = sp2v_sig_chk[1];
762   assign key_dec_we    = sp2v_sig_chk[2];
763
764   // Collect encoding errors.
765   // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
766   // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
767   assign sp_enc_err_d = |sp2v_sig_err | sub_bytes_err |
key_expand_err;
768
769   // We need to register the collected error signal to avoid
circular loops in the cipher core
770
771   // controller related to out_valid_o and detecting errors in
state_we_o and sub_bytes_out_ack.
772   always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_sp_enc_err
773     if (!rst_ni) begin
774       sp_enc_err_q <= 1'b0;
775     end else if (sp_enc_err_d) begin
776       sp_enc_err_q <= 1'b1;
777     end
778   end
779   ////////////////////////////////////////////////////
780   // Outputs //
781   ////////////////////////////////////////////////////
782
783   // The output of the last round is not stored into the state
register but forwarded directly.
784   assign state_o = add_round_key_out;
785
786   ////////////////////////////////////////////////////
787   // Assertions //
788   ////////////////////////////////////////////////////
789
790   // Typically assertions already contain this macro, which
ensures that assertions are only compiled
791   // in simulation and FPV. However, we wrap the entire
assertion section with INC_ASSERT so that the
792   // helper logic below is not synthesized either, since that
could cause issues in DC.
793   `ifdef INC_ASSERT
794     //VCS coverage off
795     // pragma coverage off
796
797     // Create a lint error to reduce the risk of accidentally
disabling the masking.
798     `ASSERT_STATIC_LINT_ERROR(AesSecMaskingNonDefault,
SecMasking == 1)
799
800     // Cipher core masking requires a masked SBox and vice
versa.
801     `ASSERT_INIT(AesMaskedCoreAndSBox,
802       (SecMasking &&
803         (SecSBoxImpl == SBoxImplCanrightMasked ||
804           SecSBoxImpl == SBoxImplCanrightMaskedNoreuse ||
805             SecSBoxImpl == SBoxImplDom)) ||
806       (!SecMasking &&
807         (SecSBoxImpl == SBoxImplLut ||
808           SecSBoxImpl == SBoxImplCanright)))
809
810     // Signals used for assertions only.
811     logic prd_clearing_equals_output,
unused_prd_clearing_equals_output;
812
813     assign prd_clearing_equals_output = (prd_clearing_state_i ==
add_round_key_out);
814     assign unused_prd_clearing_equals_output =
prd_clearing_equals_output;
815
816     // Ensure that the state register gets cleared with pseudo-
random data at the end of the last
817     // round. The following two scenarios are unlikely but not
illegal:
818     // 1. The newly loaded initial state matches the previous
output (the round counter is only
819     // cleared upon loading the new initial state).
820     // 2. The previous pseudo-random data is equal to the
previous output.
821     // Otherwise, we must see an alert e.g. because the state
multiplexer got glitched.
822     `ASSERT(AesSecCmDataRegKeySca, (state_we == SP2V_HIGH) &&
823       ((key_len_i == AES_128 && u_aes_cipher_control.rnd_ctr
== 4'd10) ||
824         (key_len_i == AES_192 && u_aes_cipher_control.rnd_ctr
== 4'd12) ||
825         (key_len_i == AES_256 && u_aes_cipher_control.rnd_ctr
== 4'd14)) ==>
826       (state_q != $past(add_round_key_out)) ||
827       (state_q == $past(state_init_i)) ||
828       $past(prd_clearing_equals_output) || alert_o)
829     if (SecMasking) begin : gen_sec_cm_key_masking_svcs

```

```

830 // The number of clock cycles a regular AES round takes
- only used for assertions.

831 localparam int unsigned NumCyclesPerRound = (SecSBoxImpl
== SBoxImplDom) ? 5 : 1;
832 logic unused_param;
833 assign unused_param = (NumCyclesPerRound == 1);
834 // Ensure that SubBytes gets fresh PRD input for every
evaluation unless mask forcing is
835 // enabled. We effectively check that the PRNG has been
updated at least once within the
836 // last NumCyclesPerRound cycles. This also holds for
the very first round, as the PRNG

837 // is always updated in the last cycle of the IDLE state
and/or the first cycle of the
838 // INIT state.
839 `ASSERT(AesSecCmKeyMaskingPrdSubBytes,

840 sub_bytes_en == SP2V_HIGH && ($past(sub_bytes_en) ==
SP2V_LOW ||
841 ($past(sub_bytes_out_req) == SP2V_HIGH &&
842 $past(sub_bytes_out_ack) == SP2V_HIGH)) |>=
843 $past(prd_sub_bytes_q) != $past(prd_sub_bytes_q,
NumCyclesPerRound + 1) ||
844 SecAllowForcingMasks && force_masks_i)
845

846 // Ensure that the PRNG has been updated between masking
the input and starting the first
847 // SubBytes evaluation/KeyExpand operation unless mask
forcing is enabled. For AES-256,

848 // we just spend 1 cycle in the INIT state and KeyExpand
isn't evaluating its S-Boxes,
849 // i.e., no fresh randomness is required. For the other
key lengths, KeyExpand evaluates
850 // its S-Boxes which takes NumCyclesPerRound cycles.
When computing the start key for
851 // decryption, the input isn't loaded and the PRNG is
thus not advanced.
852 `ASSERT(AesSecCmKeyMaskingInitialPrngUpdateSubBytes,
853 sub_bytes_en == SP2V_HIGH && $past(sub_bytes_en) ==
SP2V_LOW |>=
854 (key_len_i == AES_256 &&
855 $past(prd_masking) != $past(prd_masking, 3)) ||
856 ((key_len_i == AES_128 || key_len_i == AES_192) &&
857 $past(prd_masking) != $past(prd_masking,
NumCyclesPerRound + 2)) ||
858 (SecAllowForcingMasks && force_masks_i))
859 `ASSERT(AesSecCmKeyMaskingInitialPrngUpdateKeyExpand,
860 key_expand_en == SP2V_HIGH && $past(key_expand_en)
== SP2V_LOW |>=
861 (key_len_i == AES_256 &&
862 $past(prd_masking) != $past(prd_masking, 3)) ||
863 ((key_len_i == AES_128 || key_len_i == AES_192) &&
864 $past(prd_masking) != $past(prd_masking, 2)) ||
865 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
866
867 // Ensure none of the state shares keeps being constant
during encryption/decryption
868 // unless mask forcing is enabled. Even though unlikely
it's not impossible that one

869 // share remains constant throughout one round. The SVAs
thus only fire if a share
870 // remains constant across two rounds.
871 for (genvar s = 0; s < NumShares; s++) begin :
gen_sec_cm_key_masking_share_svsa
872 `ASSERT(AesSecCmKeyMaskingStateShare, state_we ==
SP2V_HIGH &&
873 (crypt_i == SP2V_HIGH || crypt_o == SP2V_HIGH) |>=
874 state_q[s] != $past(state_q[s], NumCyclesPerRound)
||
875 $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
876 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
877 `ASSERT(AesSecCmKeyMaskingOutputShare,

878 (out_valid_o == SP2V_HIGH && $past(out_valid_o) ==
SP2V_LOW) &&
879 (crypt_o == SP2V_HIGH) |>=
880 $past(state_o[s]) != $past(state_q[s],
NumCyclesPerRound) ||
881 $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
882 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
883 end
884 end
885
886 // Make sure the output of the masking PRNG is properly
extracted without creating overlaps
887 // in the data input masks, or between the PRD fed to the
key expand module and SubBytes.
888 if (WidthPRDSBox > 8) begin : gen_prd_extract_assert
889 // For one row of the state matrix, extract the
WidthPRDSBox-8 MSBs of the per-S-Box PRD from

```

```

890 // the PRNG output.
891 function automatic logic [3:0][WidthPRDSBox-1:0]
aes_prd_get_msbs(
892 logic [(4*WidthPRDSBox)-1:0] in
893 );
894 logic [3:0][WidthPRDSBox-1:0] prd_msbs;
895 for (int i = 0; i < 4; i++) begin
896 prd_msbs[i] = in[(i*WidthPRDSBox) + 8 +:
(WidthPRDSBox-8)];
897 end
898 return prd_msbs;
899 endfunction
900

901 // For one row of the state matrix, undo the extraction of
LSBs and MSBs of the per-S-Box PRD

902 // from the PRNG output. This can be used to verify proper
extraction (no overlap of output
903 // masks and PRD for masked Canright S-Box
implementations, no unused PRNG output).
904 function automatic logic [4*WidthPRDSBox-1:0]
aes_prd_concat_bits(
905 logic [3:0] [7:0] prd_lsb,
906 logic [3:0][WidthPRDSBox-1:0] prd_msbs
907 );
908 logic [(4*WidthPRDSBox)-1:0] prd;
909 for (int i = 0; i < 4; i++) begin
910 prd[(i*WidthPRDSBox) +: WidthPRDSBox] = {prd_msbs[i],
prd_lsb[i]};
911 end
912 return prd;
913 endfunction
914
915 // Check for correct extraction of masking PRNG output
without overlaps.
916 logic [WidthPRDMasking-1:0] unused_prd_masking;
917 logic [3:0][3:0][WidthPRDSBox-1:0] unused_prd_msbs;
918 for (genvar i = 0; i < 4; i++) begin : gen_unused_prd_msbs
919 assign unused_prd_msbs[i] =
aes_prd_get_msbs(prd_masking[i * WidthPRDRow +: WidthPRDRow]);
920 end
921 for (genvar i = 0; i < 4; i++) begin :
gen_unused_prd_masking
922 assign unused_prd_masking[i * WidthPRDRow +:
WidthPRDRow] =
923 aes_prd_concat_bits(data_in_mask[i],
unused_prd_msbs[i]);
924 end
925 assign unused_prd_masking[WidthPRDMasking-1 -:
WidthPRDKey] = prd_key_expand;
926 `ASSERT(AesMskgPrdExtraction, prd_masking ==
unused_prd_masking)
927 end
928 //VCS coverage on
929 // pragma coverage on
930 `endif
931
932 endmodule

```

A.20 lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv (485줄, 22,389B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control
6 //
7 // This module controls the AES cipher core including the key
expand module.
8
9 `include "prim_assert.sv"
10
11 module aes_cipher_control import aes_pkg::*;
12 #(
13 parameter bit CiphOpFwdOnly = 0,
14 parameter bit SecMasking = 0,
15 parameter sbbox_impl_e SecSBoxImpl = SBoxImplDom
16 ) (
17 input logic clk_i,
18 input logic rst_ni,
19
20 // Input handshake signals
21 input sp2v_e in_valid_i,
22 output sp2v_e in_ready_o,
23
24 // Output handshake signals
25 output sp2v_e out_valid_o,
26 input sp2v_e out_ready_i,
27
28 // Control and sync signals
29 input logic cfg_valid_i,
30 input ciph_op_e op_i,
31 input key_len_e key_len_i,
32 input sp2v_e crypt_i,
33 output sp2v_e crypt_o,
34 input sp2v_e dec_key_gen_i,

```

```

35 output sp2v_e dec_key_gen_o,
36 input logic prng_reseed_i,
37 output logic prng_reseed_o,
38 input logic key_clear_i,
39 output logic key_clear_o,
40 input logic data_out_clear_i,
41 output logic data_out_clear_o,
42 input logic mux_sel_err_i,
43 input logic sp_enc_err_i,
44 input logic op_err_i,
45 input logic alert_fatal_i,
46 output logic alert_o,
47
48 // Control signals for masking PRNG
49 output logic prng_update_o,
50 output logic prng_reseed_req_o,
51 input logic prng_reseed_ack_i,
52
53 // Control and sync signals for cipher data path
54 output state_sel_e state_sel_o,
55 output sp2v_e state_we_o,
56 output sp2v_e sub_bytes_en_o,
57 input sp2v_e sub_bytes_out_req_i,
58 output sp2v_e sub_bytes_out_ack_o,
59 output add_rk_sel_e add_rk_sel_o,
60
61 // Control and sync signals for key expand data path
62 output ciph_op_e key_expand_op_o,
63 output key_full_sel_e key_full_sel_o,
64 output sp2v_e key_full_we_o,
65 output key_dec_sel_e key_dec_sel_o,
66 output sp2v_e key_dec_we_o,
67 output sp2v_e key_expand_en_o,
68 input sp2v_e key_expand_out_req_i,
69 output sp2v_e key_expand_out_ack_o,
70 output logic key_expand_clear_o,
71 output logic [3:0] key_expand_round_o,
72 output key_words_sel_e key_words_sel_o,
73 output round_key_sel_e round_key_sel_o,
74 );
75
76 // Signals
77 logic [3:0] rnd_ctr;
78 sp2v_e crypt_d, crypt_q;
79 sp2v_e dec_key_gen_d,
dec_key_gen_q;
80 logic prng_reseed_d,
prng_reseed_q;
81 logic key_clear_q;
82 logic data_out_clear_q;
83 sp2v_e sub_bytes_out_req;
84 sp2v_e key_expand_out_req;
85 sp2v_e in_valid;
86 sp2v_e out_ready;
87 sp2v_e crypt;
88 sp2v_e dec_key_gen;
89 logic mux_sel_err;
90 logic mr_err;
91 logic sp_enc_err;
92 logic rnd_ctr_err;
93
94 // Sparsified FSM signals. These are needed for connecting
the individual bits of the Sp2V
95 // signals to the single-rail FSMs.
96 logic [Sp2VWidth-1:0] sp_in_valid;
97 logic [Sp2VWidth-1:0] sp_in_ready;
98 logic [Sp2VWidth-1:0] sp_out_valid;
99 logic [Sp2VWidth-1:0] sp_out_ready;
100 logic [Sp2VWidth-1:0] sp_crypt;
101 logic [Sp2VWidth-1:0] sp_dec_key_gen;
102 logic [Sp2VWidth-1:0] sp_state_we;
103 logic [Sp2VWidth-1:0] sp_sub_bytes_en;
104 logic [Sp2VWidth-1:0] sp_sub_bytes_out_req;
105 logic [Sp2VWidth-1:0] sp_sub_bytes_out_ack;
106 logic [Sp2VWidth-1:0] sp_key_full_we;
107 logic [Sp2VWidth-1:0] sp_key_dec_we;
108 logic [Sp2VWidth-1:0] sp_key_expand_en;
109 logic [Sp2VWidth-1:0] sp_key_expand_out_req;
110 logic [Sp2VWidth-1:0] sp_key_expand_out_ack;
111 logic [Sp2VWidth-1:0] sp_crypt_d;
112 logic [Sp2VWidth-1:0] sp_crypt_q;
113 logic [Sp2VWidth-1:0] sp_dec_key_gen_d;
114 logic [Sp2VWidth-1:0] sp_dec_key_gen_q;
115
116 // Multi-rail signals. These are outputs of the single-rail
FSMs and need combining.
117 logic [Sp2VWidth-1:0] mr_alert;
118 logic [Sp2VWidth-1:0] mr_prng_update;
119 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
120 logic [Sp2VWidth-1:0] mr_key_expand_clear;
121 logic [Sp2VWidth-1:0] mr_prng_reseed_d;
122 logic [Sp2VWidth-1:0] mr_key_clear_d;
123 logic [Sp2VWidth-1:0] mr_data_out_clear_d;
124
125 state_sel_e [Sp2VWidth-1:0] mr_state_sel;
126 add_rk_sel_e [Sp2VWidth-1:0] mr_add_rk_sel;
127 key_full_sel_e [Sp2VWidth-1:0] mr_key_full_sel;
128 key_dec_sel_e [Sp2VWidth-1:0] mr_key_dec_sel;
129 key_words_sel_e [Sp2VWidth-1:0] mr_key_words_sel;

```

```

130 round_key_sel_e [Sp2VWidth-1:0] mr_round_key_sel;
131
132 logic [Sp2VWidth-1:0][3:0] mr_rnd_ctr;
133
134 // FSM
135 // FSM
136 // FSM
137
138 // Convert sp2v_e signals to sparsified inputs.
139 assign sp_in_valid = {in_valid};
140 assign sp_out_ready = {out_ready};
141 assign sp_crypt = {crypt};
142 assign sp_dec_key_gen = {dec_key_gen};
143 assign sp_sub_bytes_out_req = {sub_bytes_out_req};
144 assign sp_key_expand_out_req = {key_expand_out_req};
145 assign sp_crypt_q = {crypt_q};
146 assign sp_dec_key_gen_q = {dec_key_gen_q};
147
148 // SEC_CM: CIPHER.FSM.REDUN
149 // SEC_CM: CIPHER.CTR.REDUN
150 // For every bit in the Sp2V signals, one separate rail is
instantiated. The inputs and outputs
151 // of every rail are buffered to prevent aggressive
synthesis optimizations.
152 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
153   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p
154     aes_cipher_control_fsm_p #(
155       .SecMasking ( SecMasking ),
156       .SecSBoxImpl ( SecSBoxImpl )
157     ) u_aes_cipher_control_fsm_i (
158       .clk_i ( clk_i ),
159       .rst_i ( rst_i ),
160       .in_valid_i ( sp_in_valid[i] ), // Sparsified
161       .in_ready_o ( sp_in_ready[i] ), // Sparsified
162       .out_valid_o ( sp_out_valid[i] ), // Sparsified
163       .out_ready_i ( sp_out_ready[i] ), // Sparsified
164       .cfg_valid_i ( cfg_valid_i ),
165       .op_i ( op_i ),
166       .key_len_i ( key_len_i ),
167       .crypt_i ( sp_crypt[i] ), // Sparsified
168       .dec_key_gen_i ( sp_dec_key_gen[i] ), // Sparsified
169       .prng_reseed_i ( prng_reseed_i ),
170       .key_clear_i ( key_clear_i ),
171       .data_out_clear_i ( data_out_clear_i ),
172       .mux_sel_err_i ( mux_sel_err ),
173       .sp_enc_err_i ( sp_enc_err ),
174       .rnd_ctr_err_i ( rnd_ctr_err ),
175       .op_err_i ( op_err_i ),
176       .alert_fatal_i ( alert_fatal_i ),
177       .alert_o ( mr_alert[i] ), // OR-combine
178       .prng_update_o ( mr_prng_update[i] ), // OR-combine
179       .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
180       .prng_reseed_ack_i ( prng_reseed_ack_i ),
181       .state_sel_o ( mr_state_sel[i] ), // OR-combine
182       .state_we_o ( sp_state_we[i] ), // Sparsified
183       .sub_bytes_en_o ( sp_sub_bytes_en[i] ), // Sparsified
184       .sub_bytes_out_req_i ( sp_sub_bytes_out_req[i] ), // Sparsified
185       .sub_bytes_out_ack_o ( sp_sub_bytes_out_ack[i] ), // Sparsified
186       .add_rk_sel_o ( mr_add_rk_sel[i] ), // OR-combine
187       .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
188       .key_full_we_o ( sp_key_full_we[i] ), // Sparsified
189       .key_dec_sel_o ( mr_key_dec_sel[i] ), // OR-combine
190       .key_dec_we_o ( sp_key_dec_we[i] ), // Sparsified
191       .key_expand_en_o ( sp_key_expand_en[i] ), // Sparsified
192       .key_expand_out_req_i ( sp_key_expand_out_req[i] ), // Sparsified
193       .key_expand_out_ack_o ( sp_key_expand_out_ack[i] ), // Sparsified
194       .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
195       .rnd_ctr_o ( mr_rnd_ctr[i] ), // OR-combine
196       .key_words_sel_o ( mr_key_words_sel[i] ), // OR-combine
197       .round_key_sel_o ( mr_round_key_sel[i] )
198     );
199   end
200 end

```

```

( mr_round_key_sel[i] ), // OR-combine
204
205 .crypt_q_i
( sp_crypt_q[i] ), // Sparsified
206 .crypt_d_o
( sp_crypt_d[i] ), // Sparsified
207 .dec_key_gen_q_i
( sp_dec_key_gen_q[i] ), // Sparsified
208 .dec_key_gen_d_o
( sp_dec_key_gen_d[i] ), // Sparsified
209 .prng_reseed_q_i ( prng_reseed_q ),
210 .prng_reseed_d_o
( mr_prng_reseed_d[i] ), // AND-combine
211 .key_clear_q_i ( key_clear_q ),
212 .key_clear_d_o
( mr_key_clear_d[i] ), // AND-combine
213 .data_out_clear_q_i ( data_out_clear_q ),
214 .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
215 );
216 end else begin : gen_fsm_n
217   aes_cipher_control_fsm_n #(
218     .SecMasking ( SecMasking ),
219     .SecSBoxImpl ( SecSBoxImpl )
220   ) u_aes_cipher_control_fsm_i (
221     .clk_i ( clk_i ),
222     .rst_ni ( rst_ni ),
223
224     .in_valid_ni
( sp_in_valid[i] ), // Sparsified
225 .in_ready_no
( sp_in_ready[i] ), // Sparsified
226
227 .out_valid_no
( sp_out_valid[i] ), // Sparsified
228 .out_ready_ni
( sp_out_ready[i] ), // Sparsified
229
230 .cfg_valid_i ( cfg_valid_i ),
231 .op_i ( op_i ),
232 .key_len_i ( key_len_i ),
233 .crypt_ni
( sp_crypt[i] ), // Sparsified
234 .dec_key_gen_ni
( sp_dec_key_gen[i] ), // Sparsified
235 .prng_reseed_i ( prng_reseed_i ),
236 .key_clear_i ( key_clear_i ),
237 .data_out_clear_i ( data_out_clear_i ),
238 .mux_sel_err_i ( mux_sel_err ),
239 .sp_enc_err_i ( sp_enc_err ),
240 .rnd_ctr_err_i ( rnd_ctr_err ),
241 .op_err_i ( op_err_i ),
242 .alert_fatal_i ( alert_fatal_i ),
243 .alert_o
( mr_alert[i] ), // OR-combine
244
245 .prng_update_o
( mr_prng_update[i] ), // OR-combine
246 .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
247 .prng_reseed_ack_i ( prng_reseed_ack_i ),
248
249 .state_sel_o
( mr_state_sel[i] ), // OR-combine
250 .state_we_no
( sp_state_we[i] ), // Sparsified
251 .sub_bytes_en_no
( sp_sub_bytes_en[i] ), // Sparsified
252 .sub_bytes_out_req_ni
( sp_sub_bytes_out_req[i] ), // Sparsified
253 .sub_bytes_out_ack_no
( sp_sub_bytes_out_ack[i] ), // Sparsified
254 .add_rk_sel_o
( mr_add_rk_sel[i] ), // OR-combine
255
256 .key_full_sel_o
( mr_key_full_sel[i] ), // OR-combine
257 .key_full_we_no
( sp_key_full_we[i] ), // Sparsified
258 .key_dec_sel_o
( mr_key_dec_sel[i] ), // OR-combine
259 .key_dec_we_no
( sp_key_dec_we[i] ), // Sparsified
260 .key_expand_en_no
( sp_key_expand_en[i] ), // Sparsified
261 .key_expand_out_req_ni
( sp_key_expand_out_req[i] ), // Sparsified
262 .key_expand_out_ack_no
( sp_key_expand_out_ack[i] ), // Sparsified
263 .key_expand_clear_o
( mr_key_expand_clear[i] ), // OR-combine
264 .rnd_ctr_o
( mr_rnd_ctr[i] ), // OR-combine
265 .key_words_sel_o
( mr_key_words_sel[i] ), // OR-combine
266 .round_key_sel_o
( mr_round_key_sel[i] ), // OR-combine
267
268 .crypt_q_ni
( sp_crypt_q[i] ), // Sparsified
269 .crypt_d_no

```

```

( sp_crypt_d[i] ), // Sparsified
270 .dec_key_gen_q_ni
( sp_dec_key_gen_q[i] ), // Sparsified
271 .dec_key_gen_d_no
( sp_dec_key_gen_d[i] ), // Sparsified
272 .prng_reseed_q_i ( prng_reseed_q ),
273 .prng_reseed_d_o
( mr_prng_reseed_d[i] ), // AND-combine
274 .key_clear_q_i ( key_clear_q ),
275 .key_clear_d_o
( mr_key_clear_d[i] ), // AND-combine
276 .data_out_clear_q_i ( data_out_clear_q ),
277 .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
278 );
279 end
280 end
281
282 // Convert sparsified outputs to sp2v_e type.
283 assign in_ready_o = sp2v_e'(sp_in_ready);
284 assign out_valid_o = sp2v_e'(sp_out_valid);
285 assign state_we_o = sp2v_e'(sp_state_we);
286 assign sub_bytes_en_o = sp2v_e'(sp_sub_bytes_en);
287 assign sub_bytes_out_ack_o = sp2v_e'(sp_sub_bytes_out_ack);
288 assign key_full_we_o = sp2v_e'(sp_key_full_we);
289 assign key_dec_we_o = sp2v_e'(sp_key_dec_we);
290 assign key_expand_en_o = sp2v_e'(sp_key_expand_en);
291 assign key_expand_out_ack_o =
sp2v_e'(sp_key_expand_out_ack);
292 assign crypt_d = sp2v_e'(sp_crypt_d);
293 assign dec_key_gen_d = sp2v_e'(sp_dec_key_gen_d);
294
295 // Combine single-bit FSM outputs.
296 // OR: One bit is sufficient to drive the corresponding
output bit high.
297 assign alert_o = |mr_alert;
298 assign prng_update_o = |mr_prng_update;
299 assign prng_reseed_req_o = |mr_prng_reseed_req;
300 assign key_expand_clear_o = |mr_key_expand_clear;
301 // AND: Only if all bits are high, the corresponding status
is signaled which will lead to
302 // the clearing of these trigger bits.
303 assign prng_reseed_d = &mr_prng_reseed_d;
304 assign key_clear_d = &mr_key_clear_d;
305 assign data_out_clear_d = &mr_data_out_clear_d;
306
307 // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
308 // the same outputs, two cases are possible:
309 // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
310 // - A valid encoding results: The outputs are compared
below to cover this case, see mr_err;
311 always_comb begin : combine_sparse_signals
312   state_sel_o = state_sel_e'({StateSelWidth{1'b0}});
313   add_rk_sel_o = add_rk_sel_e'({AddRKSelWidth{1'b0}});
314   key_full_sel_o =
key_full_sel_e'({KeyFullSelWidth{1'b0}});
315   key_dec_sel_o = key_dec_sel_e'({KeyDecSelWidth{1'b0}});
316   key_words_sel_o =
key_words_sel_e'({KeyWordsSelWidth{1'b0}});
317   round_key_sel_o =
round_key_sel_e'({RoundKeySelWidth{1'b0}});
318   mr_err = 1'b0;
319
320   for (int i = 0; i < Sp2VWidth; i++) begin
321     state_sel_o = state_sel_e'({state_sel_o} |
{mr_state_sel[i]});
322     add_rk_sel_o = add_rk_sel_e'({add_rk_sel_o} |
{mr_add_rk_sel[i]});
323     key_full_sel_o = key_full_sel_e'({key_full_sel_o} |
{mr_key_full_sel[i]});
324     key_dec_sel_o = key_dec_sel_e'({key_dec_sel_o} |
{mr_key_dec_sel[i]});
325     key_words_sel_o = key_words_sel_e'({key_words_sel_o} |
{mr_key_words_sel[i]});
326     round_key_sel_o = round_key_sel_e'({round_key_sel_o} |
{mr_round_key_sel[i]});
327   end
328
329   for (int i = 0; i < Sp2VWidth; i++) begin
330     if (state_sel_o != mr_state_sel[i] ||
add_rk_sel_o != mr_add_rk_sel[i] ||
key_full_sel_o != mr_key_full_sel[i] ||
key_dec_sel_o != mr_key_dec_sel[i] ||
key_words_sel_o != mr_key_words_sel[i] ||
round_key_sel_o != mr_round_key_sel[i]) begin
331       mr_err = 1'b1;
332     end
333   end
334 end
335
336 // Collect errors in mux selector signals.
337 assign mux_sel_err = mux_sel_err_i | mr_err;
338
339 // Combine counter signals. We simply OR them together. If
the FSMs don't provide the same
340 // outputs, rnd_ctr_err will be set.
341 always_comb begin : combine_counter_signals
342   rnd_ctr = '0;
343   rnd_ctr_err = 1'b0;

```

```

349   for (int i = 0; i < Sp2VWidth; i++) begin
350     rnd_ctr |= mr_rnd_ctr[i];
351   end
352
353   for (int i = 0; i < Sp2VWidth; i++) begin
354     if (rnd_ctr != mr_rnd_ctr[i]) begin
355       rnd_ctr_err = 1'b1;
356     end
357   end
358 end
359
360 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
361   if (!rst_ni) begin
362     prng_reseed_q <= 1'b0;
363     key_clear_q <= 1'b0;
364     data_out_clear_q <= 1'b0;
365   end else begin
366     prng_reseed_q <= prng_reseed_d;
367     key_clear_q <= key_clear_d;
368     data_out_clear_q <= data_out_clear_d;
369   end
370 end
371
372 // Use separate signal for key expand operation, forward
round.
373   assign key_expand_op_o = (dec_key_gen_d == SP2V_HIGH ||
374     dec_key_gen_q == SP2V_HIGH) ||
CiphOpFwdOnly ? CIPH_FWD : op_i;
375   assign key_expand_round_o = rnd_ctr;
376
377 // Let the main controller know what we are doing.
378   assign crypt_o = crypt_q;
379   assign dec_key_gen_o = dec_key_gen_q;
380   assign prng_reseed_o = prng_reseed_q;
381   assign key_clear_o = key_clear_q;
382   assign data_out_clear_o = data_out_clear_q;
383
384
385 ///////////////////////////////////////////////////
386 // Sparsely Encoded Signals //
387 ///////////////////////////////////////////////////
388
389 // SEC_CM: CTRL.SPARSE
390 // We use sparse encodings for various critical signals and
must ensure that:
391 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
392 // 2. The sparsely encoded signal is always valid. More
precisely, an alert or SVA is triggered
393 //   if a sparse signal takes on an invalid value.
394 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
395 //   This is achieved by driving the control FSM into the
terminal error state whenever any
396 //   sparsely encoded signal becomes invalid.
397 //
398 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
399 // the out_valid_o signal to prevent any data from being
released.
400
401 // The following primitives are used to place a size-only
constraint on the
402 // flops in order to prevent optimizations on these status
signals.
403   logic [Sp2VWidth-1:0] crypt_q_raw;
404   prim_flop #(
405     .Width ( Sp2VWidth ),
406     .ResetValue ( Sp2VWidth'(SP2V_LOW) )
407   ) u_crypt_regs (
408     .clk_i ( clk_i ),
409     .rst_ni ( rst_ni ),
410     .d_i ( crypt_d ),
411     .q_o ( crypt_q_raw )
412   );
413
414   logic [Sp2VWidth-1:0] dec_key_gen_q_raw;
415   prim_flop #(
416     .Width ( Sp2VWidth ),
417     .ResetValue ( Sp2VWidth'(SP2V_LOW) )
418   ) u_dec_key_gen_regs (
419     .clk_i ( clk_i ),
420     .rst_ni ( rst_ni ),
421     .d_i ( dec_key_gen_d ),
422     .q_o ( dec_key_gen_q_raw )
423   );
424
425 // We use vectors of sparsely encoded signals to reduce code
duplication.
426   localparam int unsigned NumSp2VSig = 8;
427   sp2v_e [NumSp2VSig-1:0] sp2v_sig;
428   sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
429   logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
430   logic [NumSp2VSig-1:0] sp2v_sig_err;
431
432   assign sp2v_sig[0] = in_valid_i;
433   assign sp2v_sig[1] = out_ready_i;
434   assign sp2v_sig[2] = crypt_i;
435   assign sp2v_sig[3] = dec_key_gen_i;
436   assign sp2v_sig[4] = sp2v_e'(crypt_q_raw);

```

```

437   assign sp2v_sig[5] = sp2v_e'(dec_key_gen_q_raw);
438   assign sp2v_sig[6] = sub_bytes_out_req_i;
439   assign sp2v_sig[7] = key_expand_out_req_i;
440
441 // All signals inside sp2v_sig except for sub_bytes/
key_expand_out_req_i are driven and consumed
442 // by multi-rail FSMs.
443   localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf = 8'b1100_0000;
444
445 // Individually check sparsely encoded signals.
446   for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
447     aes_sel_buf_chk #(
448       .Num ( Sp2VNum ),
449       .Width ( Sp2VWidth ),
450       .EnSecBuf ( Sp2VEnSecBuf[i] )
451     ) u_aes_sp2v_sig_buf_chk_i (
452       .clk_i ( clk_i ),
453       .rst_ni ( rst_ni ),
454       .sel_i ( sp2v_sig[i] ),
455       .sel_o ( sp2v_sig_chk_raw[i] ),
456       .err_o ( sp2v_sig_err[i] )
457     );
458     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
459   end
460
461   assign in_valid = sp2v_sig_chk[0];
462   assign out_ready = sp2v_sig_chk[1];
463   assign crypt = sp2v_sig_chk[2];
464   assign dec_key_gen = sp2v_sig_chk[3];
465   assign crypt_q = sp2v_sig_chk[4];
466   assign dec_key_gen_q = sp2v_sig_chk[5];
467   assign sub_bytes_out_req = sp2v_sig_chk[6];
468   assign key_expand_out_req = sp2v_sig_chk[7];
469
470 // Collect encoding errors.
471 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
472 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
473   assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
474
475 ///////////////////////////////////////////////////
476 // Assertions //
477 ///////////////////////////////////////////////////
478
479 // Selectors must be known/valid
480   `ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
481     CIPH_FWD,
482     CIPH_INV
483   })
484
485 endmodule

```

A.21 lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm.sv (515줄, 20,842B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM.
8
9 `include "prim_assert.sv"
10
11 module aes_cipher_control_fsm import aes_pkg::*;
12 #(
13   parameter bit SecMasking = 0,
14   parameter sbbox_impl_e SecSBoxImpl = SBoxImplDom
15 ) (
16   input logic clk_i,
17   input logic rst_ni,
18
19   // Input handshake signals
20   input logic in_valid_i, // Sparsify
using multi-rail.
21   output logic in_ready_o, // Sparsify
using multi-rail.
22
23   // Output handshake signals
24   output logic out_valid_o, // Sparsify
using multi-rail.
25   input logic out_ready_i, // Sparsify
using multi-rail.
26
27   // Control and sync signals
28   input logic cfg_valid_i, // Used for
SVAs only.
29   input ciph_op_e op_i,
30   input key_len_e key_len_i,
31   input logic crypt_i, // Sparsify
using multi-rail.
32   input logic dec_key_gen_i, // Sparsify
using multi-rail.
33   input logic prng_reseed_i,
34   input logic key_clear_i,

```

```

35 input logic data_out_clear_i,
36 input logic mux_sel_err_i,
37 input logic sp_enc_err_i,
38 input logic rnd_ctr_err_i,
39 input logic op_err_i,
40 input logic alert_fatal_i,
41 output logic alert_o,
42
43 // Control signals for masking PRNG
44 output logic prng_update_o,
45 output logic prng_reseed_req_o,
46 input logic prng_reseed_ack_i,
47
48 // Control and sync signals for cipher data path
49 output state_sel_e state_sel_o,
50 output logic state_we_o, // Sparsify
using multi-rail.
51 output logic sub_bytes_en_o, // Sparsify
using multi-rail.
52 input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
53 output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
54 output add_rk_sel_e add_rk_sel_o,
55
56 // Control and sync signals for key expand data path
57 output key_full_sel_e key_full_sel_o,
58 output logic key_full_we_o, // Sparsify
using multi-rail.
59 output key_dec_sel_e key_dec_sel_o,
60 output logic key_dec_we_o, // Sparsify
using multi-rail.
61 output logic key_expand_en_o, // Sparsify
using multi-rail.
62 input logic key_expand_out_req_i, // Sparsify
using multi-rail.
63 output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
64 output logic key_expand_clear_o,
65 output logic [3:0] rnd_ctr_o,
66 output key_words_sel_e key_words_sel_o,
67 output round_key_sel_e round_key_sel_o,
68
69 // Register signals
70 input logic crypt_q_i, // Sparsify
using multi-rail.
71 output logic crypt_d_o, // Sparsify
using multi-rail.
72 input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
73 output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
74 input logic prng_reseed_q_i,
75 output logic prng_reseed_d_o,
76 input logic key_clear_q_i,
77 output logic key_clear_d_o,
78 input logic data_out_clear_q_i,
79 output logic data_out_clear_d_o
80 );
81
82 // cfg_valid_i is used for SVAs only.
83 logic unused_cfg_valid;
84 assign unused_cfg_valid = cfg_valid_i;
85
86 // Tie off unused inputs.
87 if (!SecMasking) begin : gen_unused_prng_reseed
88 logic unused_prng_reseed;
89 assign unused_prng_reseed = prng_reseed_i;
90 end
91
92 // Signals
93 aes_cipher_ctrl_e aes_cipher_ctrl_ns, aes_cipher_ctrl_cs;
94 logic advance;
95 logic [2:0] cyc_ctr_d, cyc_ctr_q;
96 logic cyc_ctr_expr;
97 logic prng_reseed_done_d, prng_reseed_done_q;
98 logic [3:0] rnd_ctr_d, rnd_ctr_q;
99 logic [3:0] num_rounds_d, num_rounds_q;
100 logic [3:0] num_rounds_regular;
101
102 // Use separate signal for number of regular rounds.
103 assign num_rounds_regular = num_rounds_q - 4'd1;
104
105 // FSM
106 always_comb begin : aes_cipher_ctrl_fsm
107
108 // Handshake signals
109 in_ready_o = 1'b0;
110 out_valid_o = 1'b0;
111
112 // Masking PRNG signals
113 prng_update_o = 1'b0;
114 prng_reseed_req_o = 1'b0;
115
116 // Cipher data path
117 state_sel_o = STATE_ROUND;
118 state_we_o = 1'b0;
119 add_rk_sel_o = ADD_RK_ROUND;
120 sub_bytes_en_o = 1'b0;
121 sub_bytes_out_ack_o = 1'b0;
122

```

```

123 // Key expand data path
124 key_full_sel_o = KEY_FULL_ROUND;
125 key_full_we_o = 1'b0;
126 key_dec_sel_o = KEY_DEC_EXPAND;
127 key_dec_we_o = 1'b0;
128 key_expand_en_o = 1'b0;
129 key_expand_out_ack_o = 1'b0;
130 key_expand_clear_o = 1'b0;
131 key_words_sel_o = KEY_WORDS_ZERO;
132 round_key_sel_o = ROUND_KEY_DIRECT;
133
134 // FSM
135 aes_cipher_ctrl_ns = aes_cipher_ctrl_cs;
136 num_rounds_d = num_rounds_q;
137 rnd_ctr_d = rnd_ctr_q;
138 crypt_d_o = crypt_q_i;
139 dec_key_gen_d_o = dec_key_gen_q_i;
140 prng_reseed_d_o = prng_reseed_q_i;
141 key_clear_d_o = key_clear_q_i;
142 data_out_clear_d_o = data_out_clear_q_i;
143 prng_reseed_done_d = prng_reseed_done_q |
prng_reseed_ack_i;
144 advance = 1'b0;
145 cyc_ctr_d = (SecSBoxImpl == SBoxImplDom) ?
cyc_ctr_q + 3'd1 : 3'd0;
146
147 // Alert
148 alert_o = 1'b0;
149
150 unique case (aes_cipher_ctrl_cs)
151
152 CIPHER_CTRL_IDLE: begin
153 cyc_ctr_d = 3'd0;
154
155 // Signal that we are ready, wait for handshake.
156 in_ready_o = 1'b1;
157 if (in_valid_i) begin
158 if (SecMasking && prng_reseed_i && !dec_key_gen_i
&& !crypt_i) begin
159 // Reseed the masking PRNG without starting
encryption/decryption or generation of the
160 // start key for decryption.
161 prng_reseed_d_o = 1'b1;
162 prng_reseed_done_d = 1'b0;
163 aes_cipher_ctrl_ns = CIPHER_CTRL_PRNG_RESEED;
164
165 end else if (key_clear_i || data_out_clear_i) begin
166 // Clear internal key registers. The cipher core
muxes are used to clear the data
167 // output registers.
168 key_clear_d_o = key_clear_i;
169 data_out_clear_d_o = data_out_clear_i;
170
171 // To clear the data output registers, we must
first clear the state.
172 aes_cipher_ctrl_ns = data_out_clear_i ?
CIPHER_CTRL_CLEAR_S : CIPHER_CTRL_CLEAR_KD;
173
174 end else if (dec_key_gen_i || crypt_i) begin
175 // Start encryption/decryption or generation of
start key for decryption.
176 crypt_d_o = ~dec_key_gen_i & crypt_i;
177 dec_key_gen_d_o = dec_key_gen_i;
178
179 // Latch whether we shall reseed the masking PRNG.
180 prng_reseed_d_o = SecMasking & prng_reseed_i;
181
182 // Load input data to state
183 state_sel_o = dec_key_gen_i ? STATE_CLEAR :
STATE_INIT;
184
185 state_we_o = 1'b1;
186
187 // Make the masking PRNG advance. The current
pseudo-random data is used to mask the
188 // input data.
189 prng_update_o = SecMasking;
190
191 // Init key expand
192 key_expand_clear_o = 1'b1;
193
194 // Load full key
195 key_full_sel_o = dec_key_gen_i ?
KEY_FULL_ENC_INIT :
196 (op_i == CIPH_FWD) ?
KEY_FULL_ENC_INIT :
197 (op_i == CIPH_INV) ?
KEY_FULL_DEC_INIT :
198 key_full_we_o = 1'b1;
199
200 // Load num_rounds, initialize round counter.
201 num_rounds_d = (key_len_i == AES_128) ? 4'd10 :
202 (key_len_i == AES_192) ? 4'd12 :
203 4'd14;
204
205 rnd_ctr_d = '0;
206 aes_cipher_ctrl_ns = CIPHER_CTRL_INIT;
207
208 end else begin
209 // Handshake without a valid command. We should
never get here. If we do (e.g. via a

```

```

209         // malicious glitch), error out immediately.
210         aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
211     end
212 end
213 end
214
215 CIPHER_CTRL_INIT: begin
216     // Initial round: just add key to state
217     add_rk_sel_o = ADD_RK_INIT;
218
219     // Select key words for initial add_round_key
220     key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
221     (key_len_i == AES_128) ?
KEY_WORDS_0123 :
222     (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
223     (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_2345 :
224     (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
225     (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_4567 : KEY_WORDS_ZERO;
226
227     // Clear masking PRNG reseed status.
228     prng_reseed_done_d = 1'b0;
229
230     // AES-256 has two round keys available right from
beginning. Pseudo-random data is
231     // required by KeyExpand only.
232     if (key_len_i != AES_256) begin
233         // Advance in sync with KeyExpand. Based on the S-
Box implementation, it can take
234         // multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
235         // only in the first clock cycle and that PRD is
taken from the buffer stage updated
236         // based on key_full_we_o. The PRNG itself is
updated in every clock cycle to increase
237         // the noise.
238         advance = key_expand_out_req_i &
cyc_ctr_expr;
239         prng_update_o = SecMasking;
240         key_expand_en_o = 1'b1;
241         if (advance) begin
242             key_expand_out_ack_o = 1'b1;
243             state_we_o = ~dec_key_gen_q_i;
244             key_full_we_o = 1'b1;
245             rnd_ctr_d = rnd_ctr_q + 4'b0001;
246             cyc_ctr_d = 3'd0;
247             aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
248         end
249     end else begin
250         prng_update_o = SecMasking;
251         state_we_o = ~dec_key_gen_q_i;
252         rnd_ctr_d = rnd_ctr_q + 4'b0001;
253         cyc_ctr_d = 3'd0;
254         aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
255     end
256 end
257
258 CIPHER_CTRL_ROUND: begin
259     // Normal rounds
260
261     // Select key words for add_round_key
262     key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
263     (key_len_i == AES_128) ?
KEY_WORDS_0123 :
264     (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
265     (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
266     (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
267     (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
268
269     // Keep requesting PRNG reseeding until it is
acknowledged.
270     prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
271
272     // Select round key: direct or mixed (equivalent
inverse cipher)
273     round_key_sel_o = (op_i == CIPH_FWD) ?
ROUND_KEY_DIRECT :
274     (op_i == CIPH_INV) ?
ROUND_KEY_MIXED : ROUND_KEY_DIRECT;
275
276     // Advance in sync with SubBytes and KeyExpand. Based
on the S-Box implementation, both can
277     // take multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
278     // only in the first clock cycle and that PRD is taken
from the buffer stages updated
279     // with state_we_o / based on key_full_we_o. The PRNG
itself is updated in every clock
280     // cycle to increase the noise.

```

```

281     advance = key_expand_out_req_i & cyc_ctr_expr &
(dec_key_gen_q_i | sub_bytes_out_req_i);
282     prng_update_o = SecMasking;
283     sub_bytes_en_o = ~dec_key_gen_q_i;
284     key_expand_en_o = 1'b1;
285
286     if (advance) begin
287         sub_bytes_out_ack_o = ~dec_key_gen_q_i;
288         key_expand_out_ack_o = 1'b1;
289
290         state_we_o = ~dec_key_gen_q_i;
291         key_full_we_o = 1'b1;
292
293         // Update round
294         rnd_ctr_d = rnd_ctr_q + 4'b0001;
295         cyc_ctr_d = 3'd0;
296
297         // Are we doing the last regular round?
298         if (rnd_ctr_q >= num_rounds_regular) begin
299             aes_cipher_ctrl_ns = CIPHER_CTRL_FINISH;
300
301             if (dec_key_gen_q_i) begin
302                 // Write decryption key.
303                 key_dec_we_o = 1'b1;
304
305                 // Indicate that we are done, try to perform the
handshake. But we don't wait here.
306                 // If we don't get the handshake now, we will
wait in the finish state. When using
307                 // masking, we only finish if the masking PRNG
has been reseeded.
308                 out_valid_o = SecMasking ? (prng_reseed_q_i ?
prng_reseed_done_q : 1'b1) : 1'b1;
309                 if (out_valid_o && out_ready_i) begin
310                     // Go to idle state directly.
311                     dec_key_gen_d_o = 1'b0;
312                     prng_reseed_d_o = 1'b0;
313                     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
314                 end
315             end
316             end // rnd_ctr_q
317             end // SubBytes/KeyExpand REQ/ACK
318             end
319
320             CIPHER_CTRL_FINISH: begin
321                 // Final round
322
323                 // Select key words for add_round_key
324                 key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
325                 (key_len_i == AES_128) ?
KEY_WORDS_0123 :
326                 (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
327                 (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
328                 (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
329                 (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
330
331                 // Skip mix_columns
332                 add_rk_sel_o = ADD_RK_FINAL;
333
334                 // Keep requesting PRNG reseeding until it is
acknowledged.
335                 prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
336
337                 // SEC_CM: DATA_REG.KEY.SCA
338                 // Once we're done, we won't need the state anymore.
We actually clear it when progressing
339                 // to the next state.
340                 state_sel_o = STATE_CLEAR;
341
342                 // SEC_CM: DATA_REG.LOCAL_ESC
343                 // Advance in sync with SubBytes. Based on the S-Box
implementation, it can take multiple
344                 // cycles to finish. Only indicate that we are done
if:
345                 // - we have valid output (SubBytes finished),
346                 // - the masking PRNG has been reseeded (if masking is
used),
347                 // - all mux selector signals are valid (don't release
data in case of errors), and
348                 // - all sparsely encoded signals are valid (don't
release data in case of errors).
349                 // Perform both handshakes simultaneously.
350                 advance = (sub_bytes_out_req_i & cyc_ctr_expr)
| dec_key_gen_q_i;
351                 sub_bytes_en_o = ~dec_key_gen_q_i;
352                 out_valid_o = {mux_sel_err_i || sp_enc_err_i ||
op_err_i} ? 1'b0 :
353                 SecMasking ? (prng_reseed_q_i ? prng_reseed_done_q &
advance : advance) : advance;
354
355                 // Stop updating the cycle counter once we have valid

```

```

output.
356     cyc_ctr_d =
357         (SecSBoxImpl == SBoxImplDom) ? (!advance ?
cyc_ctr_q + 3'd1 : cyc_ctr_q) : 3'd0;
358
359     // The DOM S-Boxes consume fresh PRD only in the first
clock cycle and that PRD is taken
360     // from the buffer stages updated with state_we_o /
based on key_full_we_o. The PRNG itself
361     // is updated in every but the last processing clock
cycle to increase the noise. Once the
362     // processing is all done (e.g. if we're just waiting
for the PRNG reseeding to finish or
363     // if we're waiting for out_ready_i), the PRNG is no
longer updated to save power. In the
364     // very last clock cycle, we update the PRNG again to
get ready for the next block.
365     prng_update_o =
366         ((SecSBoxImpl == SBoxImplDom) ? !advance : 1'b0) |
(out_valid_o & out_ready_i);
367
368     if (out_valid_o && out_ready_i) begin
369         sub_bytes_out_ack_o = ~dec_key_gen_q_i;
370
371         // Clear the state.
372         state_we_o         = 1'b1;
373         crypt_d_o          = 1'b0;
374         cyc_ctr_d          = 3'd0;
375         // If we were generating the decryption key and
didn't get the handshake in the last
376         // regular round, we should clear dec_key_gen now.
377         dec_key_gen_d_o    = 1'b0;
378         prng_reseed_d_o    = 1'b0;
379         aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
380     end
381 end
382
383 CIPHER_CTRL_PRNG_RESEED: begin
384     // Keep requesting PRNG reseeding until it is
acknowledged.
385     prng_reseed_req_o = prng_reseed_q_i &
~prng_reseed_done_q;
386
387     // Don't update the cycle counter as we don't need it.
388     cyc_ctr_d = 3'd0;
389
390     // Once we're done, wait for handshake.
391     out_valid_o = prng_reseed_done_q;
392     if (out_valid_o && out_ready_i) begin
393         prng_reseed_d_o = 1'b0;
394         aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
395     end
396 end
397
398 CIPHER_CTRL_CLEAR_S: begin
399     // Clear the state with pseudo-random data.
400     state_we_o         = 1'b1;
401     state_sel_o         = STATE_CLEAR;
402     aes_cipher_ctrl_ns = CIPHER_CTRL_CLEAR_KD;
403 end
404
405 CIPHER_CTRL_CLEAR_KD: begin
406     // Clear internal key registers and/or external data
output registers.
407     if (key_clear_q_i) begin
408         key_full_sel_o = KEY_FULL_CLEAR;
409         key_full_we_o  = 1'b1;
410         key_dec_sel_o  = KEY_DEC_CLEAR;
411         key_dec_we_o   = 1'b1;
412     end
413     if (data_out_clear_q_i) begin
414         // Forward the state (previously cleared with
pseudo-random data).
415         // SEC_CM: DATA_REG.SEC_WIPE
416         add_rk_sel_o  = ADD_RK_INIT;
417         key_words_sel_o = KEY_WORDS_ZERO;
418         round_key_sel_o = ROUND_KEY_DIRECT;
419     end
420     // Indicate that we are done, wait for handshake.
421     out_valid_o = 1'b1;
422     if (out_ready_i) begin
423         key_clear_d_o = 1'b0;
424         data_out_clear_d_o = 1'b0;
425         aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
426     end
427 end
428
429 CIPHER_CTRL_ERROR: begin
430     // SEC_CM: CIPHER.FSM.LOCAL_ESC
431     // Terminal error state
432     alert_o = 1'b1;
433 end
434
435     // We should never get here. If we do (e.g. via a
malicious glitch), error out immediately.
436     default: begin
437         aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
438         alert_o = 1'b1;
439     end

```

```

440     endcase
441
442     // Unconditionally jump into the terminal error state in
case a mux selector or a sparsely
443     // encoded signal becomes invalid, in case we have
detected a fault in the round counter,
444     // or if a fatal alert has been triggered.
445     if (mux_sel_err_i || sp_enc_err_i || rnd_ctr_err_i ||
op_err_i || alert_fatal_i) begin
446         aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
447     end
448 end
449
450 // SEC_CM: CIPHER.FSM.SPARSE
451 `PRIM_FLOP_SPARSE_FSM(u_state_regs, aes_cipher_ctrl_ns,
452     aes_cipher_ctrl_cs, aes_cipher_ctrl_e, CIPHER_CTRL_IDLE)
453
454 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
455     if (!rst_ni) begin
456         prng_reseed_done_q <= 1'b0;
457         rnd_ctr_q          <= '0;
458         num_rounds_q       <= '0;
459     end else begin
460         prng_reseed_done_q <= prng_reseed_done_d;
461         rnd_ctr_q          <= rnd_ctr_d;
462         num_rounds_q       <= num_rounds_d;
463     end
464 end
465
466 assign rnd_ctr_o = rnd_ctr_q;
467
468 if (SecSBoxImpl == SBoxImplDom) begin : gen_cyc_ctr
469     always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_cyc_ctr
470         if (!rst_ni) begin
471             cyc_ctr_q <= 3'd0;
472         end else begin
473             cyc_ctr_q <= cyc_ctr_d;
474         end
475     end
476     assign cyc_ctr_expr = cyc_ctr_q >= 3'd4;
477 end else begin : gen_no_cyc_ctr
478     logic [2:0] unused_cyc_ctr;
479     assign cyc_ctr_q = cyc_ctr_d;
480     assign unused_cyc_ctr = cyc_ctr_q;
481     assign cyc_ctr_expr = 1'b1;
482 end
483
484 ///////////////////////////////////////////////////////////////////
485 // Assertions //
486 ///////////////////////////////////////////////////////////////////
487
488 // Create a lint error to reduce the risk of accidentally
disabling the masking.
489 `ASSERT_STATIC_LINT_ERROR(AesCipherControlFsmSecMaskingNonDefault,
SecMasking == 1)
490
491 // Create a lint error to reduce the risk of accidentally
using a less secure SBox
492 // implementation.
493 `ASSERT_STATIC_LINT_ERROR(AesCipherControlFsmSecSBoxImplNonDefault,
SecSBoxImpl == SBoxImplDom)
494
495 // Selectors must be known/valid
496 `ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
497     CIPH_FWD,
498     CIPH_INV
499 })
500 `ASSERT(AesKeyLenValid, cfg_valid_i |> key_len_i inside {
501     AES_128,
502     AES_192,
503     AES_256
504 })
505 `ASSERT(AesCipherControlStateValid, !alert_o |>
aes_cipher_ctrl_cs inside {
506     CIPHER_CTRL_IDLE,
507     CIPHER_CTRL_INIT,
508     CIPHER_CTRL_ROUND,
509     CIPHER_CTRL_FINISH,
510     CIPHER_CTRL_PRNG_RESEED,
511     CIPHER_CTRL_CLEAR_S,
512     CIPHER_CTRL_CLEAR_KD
513 })
514
515 endmodule

```

A.22 lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm_n.sv (400줄, 13,595B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM

```

```

operating on and producing the negated
8 // values of important control signals. This is achieved by:
9 // - instantiating the regular AES cipher core control FSM
operating on and producing the positive
10 // values of these signals, and
11 // - inverting these signals between the regular FSM and the
prim_buf synthesis barriers.
12 // Synthesis tools will then push the inverters into the
actual FSM.
13
14 module aes_cipher_control_fsm_n import aes_pkg::*;
15 #(
16     parameter bit          SecMasking = 0,
17     parameter sbbox_impl_e SecSBBoxImpl = SBBoxImplDom
18 )
19 input logic                clk_i,
20 input logic                rst_ni,
21
22 // Input handshake signals
23 input logic                in_valid_ni,          // Sparsify
using multi-rail.
24 output logic               in_ready_no,          // Sparsify
using multi-rail.
25
26 // Output handshake signals
27 output logic              out_valid_no,          // Sparsify
using multi-rail.
28 input logic               out_ready_ni,          // Sparsify
using multi-rail.
29
30 // Control and sync signals
31 input logic               cfg_valid_i,          // Used for
SVAs only.
32 input  ciph_op_e          op_i,
33 input  key_len_e          key_len_i,
34 input logic               crypt_ni,            // Sparsify
using multi-rail.
35 input logic               dec_key_gen_ni,        // Sparsify
using multi-rail.
36 input logic               prng_reseed_i,
37 input logic               key_clear_i,
38 input logic               data_out_clear_i,
39 input logic               mux_sel_err_i,
40 input logic               sp_enc_err_i,
41 input logic               rnd_ctr_err_i,
42 input logic               op_err_i,
43 input logic               alert_fatal_i,
44 output logic              alert_o,
45
46 // Control signals for masking PRNG
47 output logic               prng_update_o,
48 output logic               prng_reseed_req_o,
49 input logic               prng_reseed_ack_i,
50
51 // Control and sync signals for cipher data path
52 output state_sel_e         state_sel_o,
53 output logic               state_we_no,          // Sparsify
using multi-rail.
54 output logic               sub_bytes_en_no,        // Sparsify
using multi-rail.
55 input logic               sub_bytes_out_req_ni,   // Sparsify
using multi-rail.
56 output logic               sub_bytes_out_ack_no,  // Sparsify
using multi-rail.
57 output add_rk_sel_e        add_rk_sel_o,
58
59 // Control and sync signals for key expand data path
60 output key_full_sel_e       key_full_sel_o,
61 output logic               key_full_we_no,        // Sparsify
using multi-rail.
62 output key_dec_sel_e        key_dec_sel_o,
63 output logic               key_dec_we_no,          // Sparsify
using multi-rail.
64 output logic               key_expand_en_no,        // Sparsify
using multi-rail.
65 input logic               key_expand_out_req_ni,   // Sparsify
using multi-rail.
66 output logic               key_expand_out_ack_no,  // Sparsify
using multi-rail.
67 output logic               key_expand_clear_o,
68 output logic [3:0]         rnd_ctr_o,
69 output key_words_sel_e      key_words_sel_o,
70 output round_key_sel_e      round_key_sel_o,
71
72 // Register signals
73 input logic               crypt_q_ni,            // Sparsify
using multi-rail.
74 output logic               crypt_d_no,            // Sparsify
using multi-rail.
75 input logic               dec_key_gen_q_ni,        // Sparsify
using multi-rail.
76 output logic               dec_key_gen_d_no,       // Sparsify
using multi-rail.
77 input logic               prng_reseed_q_i,
78 output logic               prng_reseed_d_o,
79 input logic               key_clear_q_i,
80 output logic               key_clear_d_o,
81 input logic               data_out_clear_q_i,
82 output logic               data_out_clear_d_o
83 );
84

```

```

85 ///////////////////////////////////////////////////
86 // Input Buffering //
87 ///////////////////////////////////////////////////
88
89 localparam int NumInBufBits = $bits({
90     in_valid_ni,
91     out_ready_ni,
92     cfg_valid_i,
93     op_i,
94     key_len_i,
95     crypt_ni,
96     dec_key_gen_ni,
97     prng_reseed_i,
98     key_clear_i,
99     data_out_clear_i,
100    mux_sel_err_i,
101    sp_enc_err_i,
102    rnd_ctr_err_i,
103    op_err_i,
104    alert_fatal_i,
105    prng_reseed_ack_i,
106    sub_bytes_out_req_ni,
107    key_expand_out_req_ni,
108    crypt_q_ni,
109    dec_key_gen_q_ni,
110    prng_reseed_q_i,
111    key_clear_q_i,
112    data_out_clear_q_i
113 });
114
115 logic [NumInBufBits-1:0] in, in_buf;
116
117 assign in = {
118     in_valid_ni,
119     out_ready_ni,
120     cfg_valid_i,
121     op_i,
122     key_len_i,
123     crypt_ni,
124     dec_key_gen_ni,
125     prng_reseed_i,
126     key_clear_i,
127     data_out_clear_i,
128     mux_sel_err_i,
129     sp_enc_err_i,
130     rnd_ctr_err_i,
131     op_err_i,
132     alert_fatal_i,
133     prng_reseed_ack_i,
134     sub_bytes_out_req_ni,
135     key_expand_out_req_ni,
136     crypt_q_ni,
137     dec_key_gen_q_ni,
138     prng_reseed_q_i,
139     key_clear_q_i,
140     data_out_clear_q_i
141 };
142
143 // This primitive is used to place a size-only constraint on
the
144 // buffers to act as a synthesis optimization barrier.
145 prim_buf #(
146     .Width(NumInBufBits)
147 ) u_prim_buf_in (
148     .in_i(in),
149     .out_o(in_buf)
150 );
151
152 logic                in_valid_n;
153 logic                out_ready_n;
154 logic                cfg_valid;
155 ciph_op_e            op;
156 logic [$bits(op)-1:0] op_raw;
157 key_len_e            key_len;
158 logic                crypt_n;
159 logic                dec_key_gen_n;
160 logic                prng_reseed;
161 logic                key_clear;
162 logic                data_out_clear;
163 logic                mux_sel_err;
164 logic                sp_enc_err;
165 logic                rnd_ctr_err;
166 logic                op_err;
167 logic                alert_fatal;
168 logic                prng_reseed_ack;
169 logic                sub_bytes_out_req_n;
170 logic                key_expand_out_req_n;
171 logic                crypt_q_n;
172 logic                dec_key_gen_q_n;
173 logic                prng_reseed_q;
174 logic                key_clear_q;
175 logic                data_out_clear_q;
176
177 assign {in_valid_n,
178     out_ready_n,
179     cfg_valid,
180     op_raw,
181     key_len,
182     crypt_n,
183     dec_key_gen_n,

```

```

184     prng_reseed,
185     key_clear,
186     data_out_clear,
187     mux_sel_err,
188     sp_enc_err,
189     rnd_ctr_err,
190     op_err,
191     alert_fatal,
192     prng_reseed_ack,
193     sub_bytes_out_req_n,
194     key_expand_out_req_n,
195     crypt_q_n,
196     dec_key_gen_q_n,
197     prng_reseed_q,
198     key_clear_q,
199     data_out_clear_q} = in_buf;
200
201 assign op = ciph_op_e'(op_raw);
202
203 // Intermediate output signals
204 logic      in_ready;
205 logic      out_valid;
206 logic      alert;
207 logic      prng_update;
208 logic      prng_reseed_req;
209 state_sel_e state_sel;
210 logic      state_we;
211 logic      sub_bytes_en;
212 logic      sub_bytes_out_ack;
213 add_rk_sel_e add_rk_sel;
214 key_full_sel_e key_full_sel;
215 logic      key_full_we;
216 key_dec_sel_e key_dec_sel;
217 logic      key_dec_we;
218 logic      key_expand_en;
219 logic      key_expand_out_ack;
220 logic      key_expand_clear;
221 key_words_sel_e key_words_sel;
222 round_key_sel_e round_key_sel;
223 logic [3:0] rnd_ctr;
224 logic      crypt_d;
225 logic      dec_key_gen_d;
226 logic      prng_reseed_d;
227 logic      key_clear_d;
228 logic      data_out_clear_d;
229
230 //////////////////////////////////////////////////
231 // Regular FSM //
232 //////////////////////////////////////////////////
233
234 // The regular FSM operates on and produces the positive
values of important control signals.
235 // Invert *_n input signals here to get the positive values
for the regular FSM. To obtain the
236 // negated outputs, important output signals are inverted
further below. Thanks to the prim_buf
237 // synthesis optimization barriers, tools will push the
inverters into the regular FSM.
238 aes_cipher_control_fsm #(
239     .SecMasking ( SecMasking ),
240     .SecSBoxImpl ( SecSBoxImpl )
241 ) u_aes_cipher_control_fsm (
242     .clk_i      ( clk_i          ),
243     .rst_ni     ( rst_ni         ),
244
245     .in_valid_i ( ~in_valid_n    ), //
Invert for regular FSM.
246     .in_ready_o ( in_ready       ), //
Invert below for negated output.
247
248     .out_valid_o ( out_valid     ), //
Invert below for negated output.
249     .out_ready_i ( ~out_ready_n  ), //
Invert for regular FSM.
250
251     .cfg_valid_i ( cfg_valid     ),
252     .op_i        ( op            ),
253     .key_len_i   ( key_len       ),
254     .crypt_i     ( ~crypt_n      ), //
Invert for regular FSM.
255     .dec_key_gen_i ( ~dec_key_gen_n ), //
Invert for regular FSM.
256     .prng_reseed_i ( prng_reseed ),
257     .key_clear_i   ( key_clear    ),
258     .data_out_clear_i ( data_out_clear ),
259     .mux_sel_err_i ( mux_sel_err  ),
260     .sp_enc_err_i  ( sp_enc_err   ),
261     .rnd_ctr_err_i ( rnd_ctr_err  ),
262     .op_err_i      ( op_err       ),
263     .alert_fatal_i ( alert_fatal  ),
264     .alert_o       ( alert        ),
265
266     .prng_update_o ( prng_update  ),
267     .prng_reseed_req_o ( prng_reseed_req ),
268     .prng_reseed_ack_i ( prng_reseed_ack ),
269
270     .state_sel_o ( state_sel      ),
271     .state_we_o  ( state_we      ), //
Invert below for negated output.
272     .sub_bytes_en_o ( sub_bytes_en ), //
Invert below for negated output.

```

```

273     .sub_bytes_out_req_i ( ~sub_bytes_out_req_n ), //
Invert for regular FSM.
274     .sub_bytes_out_ack_o ( sub_bytes_out_ack     ), //
Invert below for negated output.
275     .add_rk_sel_o        ( add_rk_sel           ),
276
277     .key_full_sel_o      ( key_full_sel         ),
278     .key_full_we_o       ( key_full_we          ), //
Invert below for negated output.
279     .key_dec_sel_o       ( key_dec_sel          ),
280     .key_dec_we_o        ( key_dec_we           ), //
Invert below for negated output.
281     .key_expand_en_o     ( key_expand_en        ), //
Invert below for negated output.
282     .key_expand_out_req_i ( ~key_expand_out_req_n ), //
Invert for regular FSM.
283     .key_expand_out_ack_o ( key_expand_out_ack   ), //
Invert below for negated output.
284     .key_expand_clear_o   ( key_expand_clear     ),
285     .rnd_ctr_o            ( rnd_ctr              ),
286     .key_words_sel_o      ( key_words_sel        ),
287     .round_key_sel_o      ( round_key_sel        ),
288
289     .crypt_q_i           ( ~crypt_q_n           ), //
Invert for regular FSM.
290     .crypt_d_o           ( crypt_d              ), //
Invert below for negated output.
291     .dec_key_gen_q_i     ( ~dec_key_gen_q_n     ), //
Invert for regular FSM.
292     .dec_key_gen_d_o     ( dec_key_gen_d        ), //
Invert below for negated output.
293     .key_clear_q_i       ( key_clear_q          ),
294     .key_clear_d_o       ( key_clear_d          ),
295     .prng_reseed_q_i     ( prng_reseed_q       ),
296     .prng_reseed_d_o     ( prng_reseed_d       ),
297     .data_out_clear_q_i  ( data_out_clear_q     ),
298     .data_out_clear_d_o  ( data_out_clear_d     )
299 );
300
301 //////////////////////////////////////////////////
302 // Output Buffering //
303 //////////////////////////////////////////////////
304
305 localparam int NumOutBufBits = $bits({
306     in_ready_no,
307     out_valid_no,
308     alert_o,
309     prng_update_o,
310     prng_reseed_req_o,
311     state_sel_o,
312     state_we_no,
313     sub_bytes_en_no,
314     sub_bytes_out_ack_no,
315     add_rk_sel_o,
316     key_full_sel_o,
317     key_full_we_no,
318     key_dec_sel_o,
319     key_dec_we_no,
320     key_expand_en_no,
321     key_expand_out_ack_no,
322     key_expand_clear_o,
323     rnd_ctr_o,
324     key_words_sel_o,
325     round_key_sel_o,
326     crypt_d_no,
327     dec_key_gen_d_no,
328     key_clear_d_o,
329     prng_reseed_d_o,
330     data_out_clear_d_o
331 });
332
333 logic [NumOutBufBits-1:0] out, out_buf;
334
335 // Important output control signals need to be inverted
here. Synthesis tools will push the
336 // inverters back into the regular FSM.
337 assign out = {
338     ~in_ready,
339     ~out_valid,
340     alert,
341     prng_update,
342     prng_reseed_req,
343     state_sel,
344     ~state_we,
345     ~sub_bytes_en,
346     ~sub_bytes_out_ack,
347     add_rk_sel,
348     key_full_sel,
349     ~key_full_we,
350     key_dec_sel,
351     ~key_dec_we,
352     ~key_expand_en,
353     ~key_expand_out_ack,
354     key_expand_clear,
355     rnd_ctr,
356     key_words_sel,
357     round_key_sel,
358     ~crypt_d,
359     ~dec_key_gen_d,
360     key_clear_d,
361     prng_reseed_d,

```

```

362     data_out_clear_d
363 };
364
365 // This primitive is used to place a size-only constraint on
the
366 // buffers to act as a synthesis optimization barrier.
367 prim_buf #(
368     .Width(NumOutBufBits)
369 ) u_prim_buf_out (
370     .in_i(out),
371     .out_o(out_buf)
372 );
373
374 assign {in_ready_no,
375         out_valid_no,
376         alert_o,
377         prng_update_o,
378         prng_reseed_req_o,
379         state_sel_o,
380         state_we_o,
381         sub_bytes_en_o,
382         sub_bytes_out_ack_no,
383         add_rk_sel_o,
384         key_full_sel_o,
385         key_full_we_o,
386         key_dec_sel_o,
387         key_dec_we_o,
388         key_expand_en_o,
389         key_expand_out_ack_no,
390         key_expand_clear_o,
391         rnd_ctr_o,
392         key_words_sel_o,
393         round_key_sel_o,
394         crypt_d_no,
395         dec_key_gen_d_no,
396         key_clear_d_o,
397         prng_reseed_d_o,
398         data_out_clear_d_o} = out_buf;
399
400 endmodule

```

A.23 lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm_p.sv (390줄, 12,123B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM
operating on
8 // and producing the positive values of important control
signals.
9
10 module aes_cipher_control_fsm_p import aes_pkg::*;
11 #(
12     parameter bit SecMasking = 0,
13     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
14 ) (
15     input logic clk_i,
16     input logic rst_ni,
17
18     // Input handshake signals
19     input logic in_valid_i, // Sparsify
using multi-rail.
20     output logic in_ready_o, // Sparsify
using multi-rail.
21
22     // Output handshake signals
23     output logic out_valid_o, // Sparsify
using multi-rail.
24     input logic out_ready_i, // Sparsify
using multi-rail.
25
26     // Control and sync signals
27     input logic cfg_valid_i, // Used for
SVAs only.
28     input ciph_op_e op_i,
29     input key_len_e key_len_i,
30     input logic crypt_i, // Sparsify
using multi-rail.
31     input logic dec_key_gen_i, // Sparsify
using multi-rail.
32     input logic prng_reseed_i,
33     input logic key_clear_i,
34     input logic data_out_clear_i,
35     input logic mux_sel_err_i,
36     input logic sp_enc_err_i,
37     input logic rnd_ctr_err_i,
38     input logic op_err_i,
39     input logic alert_fatal_i,
40     output logic alert_o,
41
42     // Control signals for masking PRNG
43     output logic prng_update_o,
44     output logic prng_reseed_req_o,

```

```

45     input logic prng_reseed_ack_i,
46
47     // Control and sync signals for cipher data path
48     output state_sel_e state_sel_o,
49     output logic state_we_o, // Sparsify
using multi-rail.
50     output logic sub_bytes_en_o, // Sparsify
using multi-rail.
51     input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
52     output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
53     output add_rk_sel_e add_rk_sel_o,
54
55     // Control and sync signals for key expand data path
56     output key_full_sel_e key_full_sel_o,
57     output logic key_full_we_o, // Sparsify
using multi-rail.
58     output key_dec_sel_e key_dec_sel_o,
59     output logic key_dec_we_o, // Sparsify
using multi-rail.
60     output logic key_expand_en_o, // Sparsify
using multi-rail.
61     input logic key_expand_out_req_i, // Sparsify
using multi-rail.
62     output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
63     output logic key_expand_clear_o,
64     output logic [3:0] rnd_ctr_o,
65     output key_words_sel_e key_words_sel_o,
66     output round_key_sel_e round_key_sel_o,
67
68     // Register signals
69     input logic crypt_q_i, // Sparsify
using multi-rail.
70     output logic crypt_d_o, // Sparsify
using multi-rail.
71     input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
72     output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
73     input logic prng_reseed_q_i,
74     output logic prng_reseed_d_o,
75     input logic key_clear_q_i,
76     output logic key_clear_d_o,
77     input logic data_out_clear_q_i,
78     output logic data_out_clear_d_o
79 );
80
81 ///////////////////////////////////////////////////
82 // Input Buffering //
83 ///////////////////////////////////////////////////
84
85 localparam int NumInBufBits = $bits({
86     in_valid_i,
87     out_ready_i,
88     cfg_valid_i,
89     op_i,
90     key_len_i,
91     crypt_i,
92     dec_key_gen_i,
93     prng_reseed_i,
94     key_clear_i,
95     data_out_clear_i,
96     mux_sel_err_i,
97     sp_enc_err_i,
98     rnd_ctr_err_i,
99     op_err_i,
100    alert_fatal_i,
101    prng_reseed_ack_i,
102    sub_bytes_out_req_i,
103    key_expand_out_req_i,
104    crypt_q_i,
105    dec_key_gen_q_i,
106    prng_reseed_q_i,
107    key_clear_q_i,
108    data_out_clear_q_i
109 });
110
111 logic [NumInBufBits-1:0] in, in_buf;
112
113 assign in = {
114     in_valid_i,
115     out_ready_i,
116     cfg_valid_i,
117     op_i,
118     key_len_i,
119     crypt_i,
120     dec_key_gen_i,
121     prng_reseed_i,
122     key_clear_i,
123     data_out_clear_i,
124     mux_sel_err_i,
125     sp_enc_err_i,
126     rnd_ctr_err_i,
127     op_err_i,
128     alert_fatal_i,
129     prng_reseed_ack_i,
130     sub_bytes_out_req_i,
131     key_expand_out_req_i,
132     crypt_q_i,

```

```

133     dec_key_gen_q_i,
134     prng_reseed_q_i,
135     key_clear_q_i,
136     data_out_clear_q_i
137   };
138
139 // This primitive is used to place a size-only constraint on
the
140 // buffers to act as a synthesis optimization barrier.
141 prim_buf #(
142   .Width(NumInBufBits)
143 ) u_prim_buf_in (
144   .in_i(in),
145   .out_o(in_buf)
146 );
147
148 logic          in_valid;
149 logic          out_ready;
150 logic          cfg_valid;
151 ciph_op_e      op;
152 logic [$bits(op)-1:0] op_raw;
153 key_len_e      key_len;
154 logic          crypt;
155 logic          dec_key_gen;
156 logic          prng_reseed;
157 logic          key_clear;
158 logic          data_out_clear;
159 logic          mux_sel_err;
160 logic          sp_enc_err;
161 logic          rnd_ctr_err;
162 logic          op_err;
163 logic          alert_fatal;
164 logic          prng_reseed_ack;
165 logic          sub_bytes_out_req;
166 logic          key_expand_out_req;
167 logic          crypt_q;
168 logic          dec_key_gen_q;
169 logic          prng_reseed_q;
170 logic          key_clear_q;
171 logic          data_out_clear_q;
172
173 assign {in_valid,
174        out_ready,
175        cfg_valid,
176        op_raw,
177        key_len,
178        crypt,
179        dec_key_gen,
180        prng_reseed,
181        key_clear,
182        data_out_clear,
183        mux_sel_err,
184        sp_enc_err,
185        rnd_ctr_err,
186        op_err,
187        alert_fatal,
188        prng_reseed_ack,
189        sub_bytes_out_req,
190        key_expand_out_req,
191        crypt_q,
192        dec_key_gen_q,
193        prng_reseed_q,
194        key_clear_q,
195        data_out_clear_q} = in_buf;
196
197 assign op = ciph_op_e'(op_raw);
198
199 // Intermediate output signals
200 logic          in_ready;
201 logic          out_valid;
202 logic          alert;
203 logic          prng_update;
204 logic          prng_reseed_req;
205 state_sel_e    state_sel;
206 logic          state_we;
207 logic          sub_bytes_en;
208 logic          sub_bytes_out_ack;
209 add_rk_sel_e   add_rk_sel;
210 key_full_sel_e key_full_sel;
211 logic          key_full_we;
212 key_dec_sel_e  key_dec_sel;
213 logic          key_dec_we;
214 logic          key_expand_en;
215 logic          key_expand_out_ack;
216 logic          key_expand_clear;
217 logic [3:0]    rnd_ctr;
218 key_words_sel_e key_words_sel;
219 round_key_sel_e round_key_sel;
220 logic          crypt_d;
221 logic          dec_key_gen_d;
222 logic          prng_reseed_d;
223 logic          key_clear_d;
224 logic          data_out_clear_d;
225
226 ///////////////////////////////////////////////////
227 // Regular FSM //
228 ///////////////////////////////////////////////////
229
230 aes_cipher_control_fsm #(
231   .SecMasking ( SecMasking ),

```

```

232   .SecSBoxImpl ( SecSBoxImpl )
233 ) u_aes_cipher_control_fsm (
234   .clk_i          ( clk_i          ),
235   .rst_ni         ( rst_ni         ),
236
237   .in_valid_i     ( in_valid       ),
238   .in_ready_o     ( in_ready       ),
239
240   .out_valid_o    ( out_valid      ),
241   .out_ready_i    ( out_ready      ),
242
243   .cfg_valid_i    ( cfg_valid      ),
244   .op_i           ( op             ),
245   .key_len_i      ( key_len        ),
246   .crypt_i        ( crypt          ),
247   .dec_key_gen_i  ( dec_key_gen    ),
248   .prng_reseed_i  ( prng_reseed    ),
249   .key_clear_i    ( key_clear      ),
250   .data_out_clear_i ( data_out_clear ),
251   .mux_sel_err_i  ( mux_sel_err    ),
252   .sp_enc_err_i   ( sp_enc_err     ),
253   .rnd_ctr_err_i  ( rnd_ctr_err    ),
254   .op_err_i       ( op_err         ),
255   .alert_fatal_i  ( alert_fatal    ),
256   .alert_o        ( alert          ),
257
258   .prng_update_o  ( prng_update    ),
259   .prng_reseed_req_o ( prng_reseed_req ),
260   .prng_reseed_ack_i ( prng_reseed_ack ),
261
262   .state_sel_o    ( state_sel      ),
263   .state_we_o     ( state_we       ),
264   .sub_bytes_en_o ( sub_bytes_en   ),
265   .sub_bytes_out_req_i ( sub_bytes_out_req ),
266   .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
267   .add_rk_sel_o   ( add_rk_sel     ),
268
269   .key_full_sel_o ( key_full_sel    ),
270   .key_full_we_o  ( key_full_we    ),
271   .key_dec_sel_o  ( key_dec_sel     ),
272   .key_dec_we_o   ( key_dec_we     ),
273   .key_expand_en_o ( key_expand_en  ),
274   .key_expand_out_req_i ( key_expand_out_req ),
275   .key_expand_out_ack_o ( key_expand_out_ack ),
276   .key_expand_clear_o ( key_expand_clear ),
277   .rnd_ctr_o      ( rnd_ctr        ),
278   .key_words_sel_o ( key_words_sel  ),
279   .round_key_sel_o ( round_key_sel  ),
280
281   .crypt_q_i      ( crypt_q        ),
282   .crypt_d_o      ( crypt_d        ),
283   .dec_key_gen_q_i ( dec_key_gen_q  ),
284   .dec_key_gen_d_o ( dec_key_gen_d  ),
285   .key_clear_q_i  ( key_clear_q    ),
286   .key_clear_d_o  ( key_clear_d    ),
287   .prng_reseed_q_i ( prng_reseed_q ),
288   .prng_reseed_d_o ( prng_reseed_d ),
289   .data_out_clear_q_i ( data_out_clear_q ),
290   .data_out_clear_d_o ( data_out_clear_d ),
291 );
292
293 ///////////////////////////////////////////////////
294 // Output Buffering //
295 ///////////////////////////////////////////////////
296
297 localparam int NumOutBufBits = $bits({
298   in_ready_o,
299   out_valid_o,
300   alert_o,
301   prng_update_o,
302   prng_reseed_req_o,
303   state_sel_o,
304   state_we_o,
305   sub_bytes_en_o,
306   sub_bytes_out_ack_o,
307   add_rk_sel_o,
308   key_full_sel_o,
309   key_full_we_o,
310   key_dec_sel_o,
311   key_dec_we_o,
312   key_expand_en_o,
313   key_expand_out_ack_o,
314   key_expand_clear_o,
315   rnd_ctr_o,
316   key_words_sel_o,
317   round_key_sel_o,
318   crypt_d_o,
319   dec_key_gen_d_o,
320   key_clear_d_o,
321   prng_reseed_d_o,
322   data_out_clear_d_o
323 });
324
325 logic [NumOutBufBits-1:0] out, out_buf;
326
327 assign out = {
328   in_ready,
329   out_valid,
330   alert,
331   prng_update,
332   prng_reseed_req,

```

```

333 state_sel,
334 state_we,
335 sub_bytes_en,
336 sub_bytes_out_ack,
337 add_rk_sel,
338 key_full_sel,
339 key_full_we,
340 key_dec_sel,
341 key_dec_we,
342 key_expand_en,
343 key_expand_out_ack,
344 key_expand_clear,
345 rnd_ctr,
346 key_words_sel,
347 round_key_sel,
348 crypt_d,
349 dec_key_gen_d,
350 key_clear_d,
351 prng_reseed_d,
352 data_out_clear_d
353 };
354
355 // This primitive is used to place a size-only constraint on
the
356 // buffers to act as a synthesis optimization barrier.
357 prim_buf #(
358   .Width(NumOutBufBits)
359 ) u_prim_buf_out (
360   .in_i(out),
361   .out_o(out_buf)
362 );
363
364 assign {in_ready_o,
365        out_valid_o,
366        alert_o,
367        prng_update_o,
368        prng_reseed_req_o,
369        state_sel_o,
370        state_we_o,
371        sub_bytes_en_o,
372        sub_bytes_out_ack_o,
373        add_rk_sel_o,
374        key_full_sel_o,
375        key_full_we_o,
376        key_dec_sel_o,
377        key_dec_we_o,
378        key_expand_en_o,
379        key_expand_out_ack_o,
380        key_expand_clear_o,
381        rnd_ctr_o,
382        key_words_sel_o,
383        round_key_sel_o,
384        crypt_d_o,
385        dec_key_gen_d_o,
386        key_clear_d_o,
387        prng_reseed_d_o,
388        data_out_clear_d_o} = out_buf;
389
390 endmodule

```

A.24 lowRISC_opentitan/hw/ip/aes/rtl/aes_prng_masking.sv (151줄, 6,433B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES high-bandwidth pseudo-random number generator for
masking
6 //
7 // This module uses a stream cipher primitive followed by a
linear
permutation layer to generate
8 // pseudo-random data for masking the AES cipher core. The
stream
cipher primitive can be reseeded
9 // using an external interface.
10
11 `include "prim_assert.sv"
12
13 module aes_prng_masking import aes_pkg::*;
14 #(
15   parameter int unsigned Width      = WidthPRDMasking,
16   parameter int unsigned EntropyWidth =
edn_pkg::ENDPOINT_BUS_WIDTH,
17   parameter bit SecAllowForcingMasks = 0, // Allow
forcing
masks to constant values using
18                                     //
force_masks_i. Useful for SCA only.
19   parameter bit SecSkipPRNGReseeding = 0, // The
current
SCA setup doesn't provide
20                                     //
sufficient resources to implement the
21                                     //
infrastructure required for PRNG reseeding.
22                                     // To
23 enable SCA resistance evaluations, we
24                                     // need
to skip reseeding requests.

```

```

24
25   parameter masking_lfsr_seed_t RndCnstLfsrSeed =
RndCnstMaskingLfsrSeedDefault,
26   parameter masking_lfsr_perm_t RndCnstLfsrPerm =
RndCnstMaskingLfsrPermDefault
27 ) (
28   input logic          clk_i,
29   input logic          rst_ni,
30
31   input logic          force_masks_i,
32
33   // Connections to AES internals, PRNG consumers
34   input logic          data_update_i,
35   output logic [Width-1:0] data_o,
36   input logic          reseed_req_i,
37   output logic         reseed_ack_o,
38
39   // Connections to outer world, LFSR reseeding
40   output logic         entropy_req_o,
41   input logic          entropy_ack_i,
42   input logic [EntropyWidth-1:0] entropy_i
43 );
44
45   logic          prng_seed_en;
46   logic          prng_seed_done;
47   logic [Width-1:0] prng_key;
48   logic          prng_err;
49
50   ////////////
51   // Control //
52   ////////////
53
54   // The data requests are fed from a stream cipher primitive
that is
reseeded in chunks of
55   // EntropyWidth bits. Internally, the primitive generates
the
output based on the current state.
56   // If there is an outstanding reseed request, the PRNG can
keep
updating and providing
57   // pseudo-random data (using the current state of the
primitive). When reseeding a chunk without
58   // requesting a PRNG update, the current output might change or
not,
depending on the Width and
59   // current chunk index.
60
61   // In the current SCA setup, we don't have sufficient
resources
to implement the infrastructure
62   // required for PRNG reseeding (CSRNG, EDN, etc.).
Therefore, we skip any reseeding requests if
63   // the SecSkipPRNGReseeding parameter is set. Performing the
reseeding
without proper entropy
64   // provided from CSRNG would result in quickly repeating,
fully
deterministic PRNG output,
65   // which prevents meaningful SCA resistance evaluations.
66
67   // Create a lint error to reduce the risk of accidentally
enabling
this feature.
68   `ASSERT_STATIC_LINT_ERROR(AesSecAllowForcingMasksNonDefault,
SecAllowForcingMasks == 0)
69
70   if (SecAllowForcingMasks == 0) begin :
gen_unused_force_masks
71     logic unused_force_masks;
72     assign unused_force_masks = force_masks_i;
73   end
74
75   // Create a lint error to reduce the risk of accidentally
enabling
this feature.
76   `ASSERT_STATIC_LINT_ERROR(AesSecSkipPRNGReseedingNonDefault,
SecSkipPRNGReseeding == 0)
77
78   if (SecSkipPRNGReseeding == 1) begin :
gen_unused_prng_seed_done
79     logic unused_prng_seed_done;
80     assign unused_prng_seed_done = prng_seed_done;
81   end
82
83   // Reseed interface handling
84   assign prng_seed_en = SecSkipPRNGReseeding ? 1'b0 :
reseed_req_i;
85   assign reseed_ack_o = SecSkipPRNGReseeding ? reseed_req_i :
prng_seed_done;
86
87   ////////////
88   // Bivium Stream Cipher Primitive //
89   ////////////
90   prim_trivium #(
91     .BiviumVariant      (1),
92     .OutputWidth         (Width),
93     .StrictLockupProtection(!SecAllowForcingMasks),
94     .SeedType            (prim_trivium_pkg::SeedTypeStatePartial),
95     .PartialSeedWidth    (EntropyWidth),
96     .RndCnstTriviumLfsrSeed(RndCnstLfsrSeed)
97   ) u_prim_bivium (
98     .clk_i (clk_i),
99     .rst_ni (rst_ni),
100

```

```

101 .en_i (data_update_i),
102 .allow_lockup_i (SecAllowForcingMasks &
force_masks_i),
103 .seed_en_i (prng_seed_en),
104 .seed_done_o (prng_seed_done),
105 .seed_req_o (entropy_req_o),
106 .seed_ack_i (entropy_ack_i),
107 .seed_key_i ('0), // Not used.
108 .seed_iv_i ('0), // Not used.
109 .seed_state_full_i ('0), // Not used.
110 .seed_state_partial_i(entropy_i),
111
112 .key_o(prng_key),
113 .state_o(), // Not used.
114 .err_o(prng_err)
115 );
116
117 // Add a permutation layer to obfuscate the output of stream
cipher primitive.
118 for (genvar b = 0; b < Width; b++) begin : gen_perm
119     assign data_o[b] = prng_key[RndCnstLfSrPerm[b]];
120 end
121
122 // At the moment, the PRNG error output is ignored. This
signal is asserted only if the stream
123 // cipher primitive enters an all-zero state. Depending on
the value of the SecAllowForcingMasks
124 // parameter, the primitive behaves differently:

125 // - If SecAllowForcingMasks == 0 or if SecAllowForcingMasks
== 1 and force_masks_i == 0, the
126 // primitive is automatically reset to the default value
defined by a secret netlist constant.
127 // - If SecAllowForcingMasks == 1 and force_masks_i == 1,
the primitive keeps the all-zero value.
128 // This may be required to switch off the masking for
evaluating the SCA resistance.
129 logic unused_prng_err;
130 assign unused_prng_err = prng_err;
131
132 ///////////////////////////////////////////////////////////////////
133 // Assertions //
134 ///////////////////////////////////////////////////////////////////
135
136 // the code below is not meant to be synthesized,
137 // but it is intended to be used in simulation and FV
`ifndef SYNTHESIS
138 // Check that the supplied permutation is valid.
139 logic [Width-1:0] perm_test;
140 initial begin : p_perm_check
141     perm_test = '0;
142     for (int k = 0; k < Width; k++) begin
143         perm_test[RndCnstLfSrPerm[k]] = 1'b1;
144     end
145     // All bit positions must be marked with 1.
146     `ASSERT_I(PermutationCheck_A, &perm_test)
147 end
148 `endif
149 `endif
150
151 endmodule

```

A.25 lowRISC_opentitan/hw/ip/aes/rtl/aes_prng_clearing.sv (165줄, 7,075B)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES low-bandwidth pseudo-random number generator for
register clearing
6 //
7 // This module uses an LFSR followed by an aligned
permutation, a non-linear layer (PRINCE S-Boxes)
8 // and another permutation to generate pseudo-random data for
the AES module for clearing
9 // registers (secure wipe). The LFSR can be reseeded using an
external interface.
10
11 `include "prim_assert.sv"
12
13 module aes_prng_clearing import aes_pkg::*;
14 #(
15     parameter int unsigned Width = 64, // At the
moment we just support a width of 64.
16     parameter int unsigned EntropyWidth =
edn_pkg::ENDPOINT_BUS_WIDTH,
17     parameter bit SecSkipPRNGReseeding = 0, // The
current SCA setup doesn't provide
18 //
sufficient resources to implement the
19 //
infrastructure required for PRNG reseeding.
20 // To
enable SCA resistance evaluations, we
21 // need to
skip reseeding requests.

```

```

22 parameter clearing_lfsr_seed_t RndCnstLfSrSeed =
RndCnstClearingLfSrSeedDefault,
23 parameter clearing_lfsr_perm_t RndCnstLfSrPerm =
RndCnstClearingLfSrPermDefault,
24 parameter clearing_lfsr_perm_t RndCnstSharePerm =
RndCnstClearingSharePermDefault
25 ) (
26     input logic clk_i,
27     input logic rst_ni,
28
29     // Connections to AES internals, PRNG consumers
30     input logic data_update_i,
31     output logic [Width-1:0] data_o [NumSharesKey],
32     input logic reseed_req_i,
33     output logic reseed_ack_o,
34
35     // Connections to outer world, LFSR re-seed
36     output logic entropy_req_o,
37     input logic entropy_ack_i,
38     input logic [EntropyWidth-1:0] entropy_i
39 );
40
41 logic seed_valid;
42 logic seed_en;
43 logic [Width-1:0] seed;
44 logic lfsr_en;
45 logic [Width-1:0] lfsr_state;
46
47 // The data requests are fed from an LFSR. Reseed requests
take precedence internally to the
48 // LFSR. If there is an outstanding reseed request, the PRNG
can keep updating and providing
49 // pseudo-random data (using the old seed). If the reseeding is
taking place, the LFSR will
50 // provide fresh pseudo-random data (the new seed) in the
next cycle anyway. This means the PRNG
51 // is always ready to provide new pseudo-random data.
52
53 // In the current SCA setup, we don't have sufficient
resources to implement the infrastructure
54 // required for PRNG reseeding (CSRNG, EDN, etc.).
Therefore, we skip any reseeding requests if
55 // the SecSkipPRNGReseeding parameter is set. Performing the
reseeding without proper entropy
56 // provided from CSRNG would result in quickly repeating,
fully deterministic PRNG output,
57 // which prevents meaningful SCA resistance evaluations.
58
59 // Create a lint error to reduce the risk of accidentally
enabling this feature.
60 `ASSERT_STATIC_LINT_ERROR(AesSecSkipPRNGReseedingNonDefault,
SecSkipPRNGReseeding == 0)
61
62 // LFSR control
63 assign lfsr_en = data_update_i;
64 assign seed_en = SecSkipPRNGReseeding ? 1'b0 : seed_valid;
65
66 // Width adaption for reseeding interface. We get
EntropyWidth bits at a time.
67 if (Width/2 == EntropyWidth) begin : gen_buffer
68     // We buffer the first EntropyWidth bits.
69     logic [EntropyWidth-1:0] buffer_d, buffer_q;
70     logic buffer_valid_d, buffer_valid_q;
71
72     // Stop requesting entropy once we have reseeded the LFSR.
73     assign entropy_req_o = SecSkipPRNGReseeding ?
1'b0 : reseed_req_i;
74     assign reseed_ack_o = SecSkipPRNGReseeding ?
reseed_req_i : seed_valid;
75
76     // Buffer
77     assign buffer_valid_d = entropy_req_o && entropy_ack_i ?
~buffer_valid_q : buffer_valid_q;
78
79     // Only update the buffer upon receiving the first
EntropyWidth bits.
80     assign buffer_d = entropy_req_o && entropy_ack_i && !
buffer_valid_q ? entropy_i : buffer_q;
81
82     always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_buffer
83         if (!rst_ni) begin
84             buffer_q <= '0;
85             buffer_valid_q <= 1'b0;
86         end else begin
87             buffer_q <= buffer_d;
88             buffer_valid_q <= buffer_valid_d;
89         end
90     end
91
92     assign seed = {buffer_q, entropy_i};
93     assign seed_valid = buffer_valid_q && entropy_req_o &
entropy_ack_i;
94
95 end else begin : gen_packer
96     // Upsizing of entropy input to correct width for LFSR
reseeding.

```

```

97
98 // Stop requesting entropy once the desired amount is
available.
99 assign entropy_req_o = SecSkipPRNGReseeding ?
1'b0 : reseed_req_i & ~seed_valid;
100 assign reseed_ack_o = SecSkipPRNGReseeding ?
reseed_req_i : seed_valid;
101
102 prim_packer_fifo #(
103     .InW      ( EntropyWidth ),
104     .OutW      ( Width ),
105     .ClearOnRead ( 1'b0 )
106 ) u_prim_packer_fifo (
107     .clk_i      ( clk_i ),
108     .rst_ni     ( rst_ni ),
109     .clr_i      ( 1'b0 ), // Not needed.
110     .wvalid_i   ( entropy_ack_i ),
111     .wdata_i    ( entropy_i ),
112     .wready_o   ( ), // Not needed, we're always
ready to sink data at this point.
113     .rvalid_o   ( seed_valid ),
114     .rdata_o    ( seed ),
115     .rready_i   ( 1'b1 ), // We're always ready to
receive the packed output word.
116     .depth_o    ( ) // Not needed.
117 );
118 end
119
120 // LFSR instance
121 prim_lfsr #(
122     .LfsrType    ( "GAL_XOR" ),
123     .LfsrDw      ( Width ),
124     .StateOutDw   ( Width ),
125     .DefaultSeed  ( RndCnstLfsrSeed ),
126     .StatePermEn  ( 1'b1 ),
127     .StatePerm    ( RndCnstLfsrPerm ),
128     .NonLinearOut ( 1'b1 )
129 ) u_lfsr (
130     .clk_i      ( clk_i ),
131     .rst_ni     ( rst_ni ),
132     .seed_en_i  ( seed_en ),
133     .seed_i     ( seed ),
134     .lfsr_en_i  ( lfsr_en ),
135     .entropy_i   ( '0 ),
136     .state_o    ( lfsr_state )
137 );
138 assign data_o[0] = lfsr_state;
139
140 // A separate permutation is applied to obtain the pseudo-
random data for clearing the second
141 // share of registers (e.g. key registers or state registers
in case masking is enabled).
142 for (genvar i = 0; i < Width; i++) begin : gen_share_perm
143     assign data_o[1][i] = lfsr_state[RndCnstSharePerm[i]];
144 end
145
146 // Width must be 64.
147 `ASSERT_INIT(AesPrgWidth, Width == 64)
148
149 // the code below is not meant to be synthesized,
150 // but it is intended to be used in simulation and FV
151 `ifndef SYNTHESIS
152 // Check that the supplied permutation is valid.
153 logic [Width-1:0] share_perm_test, unused_share_perm_test;
154 initial begin : p_share_perm_check
155     share_perm_test = '0;
156     for (int k = 0; k < Width; k++) begin
157         share_perm_test[RndCnstSharePerm[k]] = 1'b1;
158     end
159     unused_share_perm_test = share_perm_test;
160     // All bit positions must be marked with 1.
161     `ASSERT_I(SharePermutationCheck_A, &share_perm_test)
162 end
163 `endif
164
165 endmodule

```

A.26 lowRISC_opentitan/hw/ip/aes/rtl/aes_ctr.sv (217줄, 7,338B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES counter for CTR mode
6 //
7 // This module uses one counter with a width of SliceSizeCtr
to iteratively increment the 128-bit
8 // counter value.
9
10 module aes_ctr import aes_pkg::*;
11 (
12     input logic clk_i,
13     input logic rst_ni,
14
15     input sp2v_e inc32_i,

```

```

16     input sp2v_e incr_i,
17     output sp2v_e ready_o,
18     output logic alert_o,
19
20     input logic [NumSlicesCtr-1:0][SliceSizeCtr-1:0] ctr_i,
21     output logic [NumSlicesCtr-1:0][SliceSizeCtr-1:0] ctr_o,
22     output sp2v_e [NumSlicesCtr-1:0] ctr_we_o
23 );
24
25 // Reverse byte order - unrelated to NumSlicesCtr and
SliceSizeCtr
26 function automatic logic [15:0][7:0]
aes_rev_order_byte(logic [15:0][7:0] in);
27     logic [15:0][7:0] out;
28     for (int i = 0; i < 16; i++) begin
29         out[i] = in[15-i];
30     end
31     return out;
32 endfunction
33
34 // Reverse sp2v order
35 function automatic sp2v_e [NumSlicesCtr-1:0]
aes_rev_order_sp2v(sp2v_e [NumSlicesCtr-1:0] in);
36     sp2v_e [NumSlicesCtr-1:0] out;
37     for (int i = 0; i < NumSlicesCtr; i++) begin
38         out[i] = in[NumSlicesCtr - 1 - i];
39     end
40     return out;
41 endfunction
42
43 // Signals
44 logic [SliceIdxWidth-1:0] ctr_slice_idx;
45
46 logic [NumSlicesCtr-1:0][SliceSizeCtr-1:0] ctr_i_rev; // 8
times 2 bytes
47 logic [NumSlicesCtr-1:0][SliceSizeCtr-1:0] ctr_o_rev; // 8
times 2 bytes
48 sp2v_e [NumSlicesCtr-1:0] ctr_we_o_rev;
49 sp2v_e ctr_we;
50
51 logic [SliceSizeCtr-1:0] ctr_i_slice;
52 logic [SliceSizeCtr-1:0] ctr_o_slice;
53
54 sp2v_e inc32;
55 logic inc32_err;
56 sp2v_e incr;
57 logic incr_err;
58 logic sp_enc_err;
59 logic mr_err;
60
61 // Sparsified FSM signals. These are needed for connecting
the individual bits of the Sp2V
62 // signals to the single-rail FSMs.
63 logic [Sp2VWidth-1:0] sp_inc32;
64 logic [Sp2VWidth-1:0] sp_incr;
65 logic [Sp2VWidth-1:0] sp_ready;
66 logic [Sp2VWidth-1:0] sp_ctr_we;
67
68 // Multi-rail signals. These are outputs of the single-rail
FSMs and need combining.
69 logic [Sp2VWidth-1:0] mr_alert;
70 logic [Sp2VWidth-1:0][SliceIdxWidth-1:0]
mr_ctr_slice_idx;
71 logic [Sp2VWidth-1:0] [SliceSizeCtr-1:0] mr_ctr_o_slice;
72
73 // Inputs //
74 //
75 // Reverse byte order
76 assign ctr_i_rev = aes_rev_order_byte(ctr_i);
77
78 // SEC_CM: CTRL.SPARSE
79 // Check sparsely encoded inc32 and incr signals.
80 logic [Sp2VWidth-1:0] inc32_raw;
81 aes_sel_buf_chk #(
82     .Num      ( Sp2VNum ),
83     .Width     ( Sp2VWidth ),
84     .EnSecBuf  ( 1'b0 )
85 ) u_aes_inc32_buf_chk (
86     .clk_i      ( clk_i ),
87     .rst_ni     ( rst_ni ),
88     .sel_i      ( inc32_i ),
89     .sel_o      ( inc32_raw ),
90     .err_o      ( inc32_err )
91 );
92 assign inc32 = sp2v_e'(inc32_raw);
93
94 logic [Sp2VWidth-1:0] incr_raw;
95 aes_sel_buf_chk #(
96     .Num      ( Sp2VNum ),
97     .Width     ( Sp2VWidth ),
98     .EnSecBuf  ( 1'b0 )
99 ) u_aes_incr_buf_chk (
100     .clk_i      ( clk_i ),
101     .rst_ni     ( rst_ni ),
102     .sel_i      ( incr_i ),
103     .sel_o      ( incr_raw ),
104     .err_o      ( incr_err )
105 );
106 assign incr = sp2v_e'(incr_raw);
107
108

```

```

109
110 // Collect encoding errors.
111 assign sp_enc_err = inc32_err | incr_err;
112
113 // Counter //
114 // Counter //
115 // Counter //
116
117 // We do SliceSizeCtr bits at a time.
118 assign ctr_i_slice = ctr_i_rev[ctr_slice_idx];
119
120 // FSM //
121 // FSM //
122 // FSM //
123
124 // Convert sp2v_e signals to sparsified inputs.
125 assign sp_inc32 = {inc32};
126 assign sp_incr = {incr};
127
128 // SEC_CM: CTR.FSM.REDUN
129 // For every bit in the Sp2V signals, one separate rail is
instantiated. The inputs and outputs
130 // of every rail are buffered to prevent aggressive
synthesis optimizations.
131 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
132   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p
133     aes_ctr_fsm_p u_aes_ctr_fsm_i (
134       .clk_i      ( clk_i      ),
135       .rst_ni     ( rst_ni     ),
136
137       .inc32_i    ( sp_inc32[i] ), //
138       .incr_i     ( sp_incr[i]  ), //
139       .ready_o    ( sp_ready[i] ), //
140       .sp_enc_err_i ( sp_enc_err ),
141       .mr_err_i   ( mr_err      ),
142       .alert_o    ( mr_alert[i] ), // OR-
143
144       .ctr_slice_idx_o ( mr_ctr_slice_idx[i] ), // OR-
145       .ctr_slice_i    ( ctr_i_slice ),
146       .ctr_slice_o    ( mr_ctr_o_slice[i] ), // OR-
147       .ctr_we_o       ( sp_ctr_we[i] ) //
148     );
149   end else begin : gen_fsm_n
150     aes_ctr_fsm_n u_aes_ctr_fsm_i (
151       .clk_i      ( clk_i      ),
152       .rst_ni     ( rst_ni     ),
153
154       .inc32_ni    ( sp_inc32[i] ), //
155       .incr_ni     ( sp_incr[i]  ), //
156       .ready_no    ( sp_ready[i] ), //
157       .sp_enc_err_i ( sp_enc_err ),
158       .mr_err_i   ( mr_err      ),
159       .alert_o    ( mr_alert[i] ), // OR-
160
161       .ctr_slice_idx_o ( mr_ctr_slice_idx[i] ), // OR-
162       .ctr_slice_i    ( ctr_i_slice ),
163       .ctr_slice_o    ( mr_ctr_o_slice[i] ), // OR-
164       .ctr_we_no     ( sp_ctr_we[i] ) //
165     );
166   end
167 end
168
169 // Convert sparsified outputs to sp2v_e type.
170 assign ready_o = sp2v_e'(sp_ready);
171 assign ctr_we = sp2v_e'(sp_ctr_we);
172
173 // Combine single-bit FSM outputs.
174 // OR: One bit is sufficient to drive the corresponding
output bit high.
175 assign alert_o = |mr_alert;
176
177 // Combine multi-bit FSM outputs. We simply OR them together
and compare the values
178 // to detect errors.
179 always_comb begin : combine_sparse_signals
180   ctr_slice_idx = '0;
181   ctr_o_slice   = '0;
182   mr_err        = 1'b0;
183
184   for (int i = 0; i < Sp2VWidth; i++) begin
185     ctr_slice_idx |= mr_ctr_slice_idx[i];
186     ctr_o_slice   |= mr_ctr_o_slice[i];
187   end
188
189   for (int i = 0; i < Sp2VWidth; i++) begin
190     if (ctr_slice_idx != mr_ctr_slice_idx[i] ||

```

```

191     ctr_o_slice   != mr_ctr_o_slice[i]) begin
192       mr_err = 1'b1;
193     end
194   end
195 end
196
197 // Outputs //
198 // Outputs //
199 // Outputs //
200
201 // Combine input and counter output.
202 always_comb begin
203   ctr_o_rev = ctr_i_rev;
204   ctr_o_rev[ctr_slice_idx] = ctr_o_slice;
205 end
206
207 // Generate the sliced write enable.
208 always_comb begin
209   ctr_we_o_rev = {NumSlicesCtr{SP2V_LOW}};
210   ctr_we_o_rev[ctr_slice_idx] = ctr_we;
211 end
212
213 // Reverse byte and bit order.
214 assign ctr_o = aes_rev_order_byte(ctr_o_rev);
215 assign ctr_we_o = aes_rev_order_sp2v(ctr_we_o_rev);
216
217 endmodule

```

A.27 lowRISC_opentitan/hw/ip/aes/rtl/aes_core.sv (1,145줄, 48,933B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES core implementation
6
7 `include "prim_assert.sv"
8
9 module aes_core
10   import aes_pkg::*;
11   import aes_reg_pkg::*;
12   #(
13     parameter bit AES192Enable = 1,
14     parameter bit AESGCMEEnable = 1,
15     parameter bit SecMasking = 1,
16     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom,
17     parameter int unsigned SecStartTriggerDelay = 0,
18     parameter bit SecAllowForcingMasks = 0,
19     parameter bit SecSkipPRNGReseeding = 0,
20     parameter int unsigned EntropyWidth =
edn_pkg::ENDPOINT_BUS_WIDTH,
21
22     localparam int NumShares = SecMasking ?
2 : 1, // derived parameter
23
24     parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeed =
RndCnstClearingLfsrSeedDefault,
25     parameter clearing_lfsr_perm_t RndCnstClearingLfsrPerm =
RndCnstClearingLfsrPermDefault,
26     parameter clearing_lfsr_perm_t RndCnstClearingSharePerm =
RndCnstClearingSharePermDefault,
27     parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed =
RndCnstMaskingLfsrSeedDefault,
28     parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm =
RndCnstMaskingLfsrPermDefault
29   ) (
30     input logic clk_i,
31     input logic rst_ni,
32     input logic rst_shadowed_ni,
33
34     // Entropy request interfaces for clearing and masking PRNGs
35     output logic entropy_clearing_req_o,
36     input logic entropy_clearing_ack_i,
37     input logic [EntropyWidth-1:0] entropy_clearing_i,
38     output logic entropy_masking_req_o,
39     input logic entropy_masking_ack_i,
40     input logic [EntropyWidth-1:0] entropy_masking_i,
41
42     // Key manager (keymgr) key sideload interface
43     input keymgr_pkg::hw_key_req_t keymgr_key_i,
44
45     // Life cycle
46     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
47
48     // Alerts
49     input logic shadowed_storage_err_i,
50     input logic shadowed_update_err_i,
51     input logic intg_err_alert_i,
52     output logic alert_recov_o,
53     output logic alert_fatal_o,
54
55     // Bus Interface
56     input aes_reg2hw_t reg2hw,
57     output aes_hw2reg_t hw2reg
58   );
59
60 // Signals

```

61	logic		ctrl_qe;	128	sp2v_e	[NumSlicesCtr-1:0]		iv_we_ctrl;
62	logic		ctrl_we;	129	sp2v_e	[NumSlicesCtr-1:0]		iv_we;
63	logic		ctrl_phase;	130	logic		[IVSelWidth-1:0]	iv_sel_raw;
64	aes_op_e		aes_op_q;	131	iv_sel_e			iv_sel_ctrl;
65	aes_mode_e		aes_mode_q;	132	iv_sel_e			iv_sel;
66	ciph_op_e		cipher_op;	133	logic			iv_sel_err;
67	ciph_op_e		cipher_op_buf;	134				
68	key_len_e		key_len_q;	135	logic	[NumSlicesCtr-1:0][SliceSizeCtr-1:0]		ctr;
69	logic		sideload_q;	136	sp2v_e	[NumSlicesCtr-1:0]		ctr_we;
70	prs_rate_e			137	sp2v_e			ctr_inc32;
prng_reseed_rate_q;				138	sp2v_e			ctr_incr;
71	logic			139	sp2v_e			ctr_ready;
manual_operation_q;				140	logic			ctr_alert;
72	logic			141				
ctrl_reg_err_update;				142	logic	[NumRegsData-1:0][31:0]		data_in_prev_d;
73	logic			143	logic	[NumRegsData-1:0][31:0]		data_in_prev_q;
ctrl_reg_err_storage;				144	sp2v_e			
74	logic		ctrl_gcm_qe;	145	sp2v_e			data_in_prev_we;
75	logic		ctrl_gcm_we;	146	logic		[DIPSelWidth-1:0]	
76	logic		ctrl_gcm_phase;	147	dip_sel_e			
77	logic		gcm_init_done;	148	dip_sel_e			
78	gcm_phase_e		gcm_phase_q;	149	logic			
79	logic	[4:0]		150	data_in_prev_sel;			
num_valid_bytes_q;				151	data_in_prev_sel_err;			
80	logic			152	logic	[NumRegsData-1:0][31:0]		data_in;
ctrl_gcm_reg_err_update;			ctrl_err_update;	153	logic	[NumRegsData-1:0]		data_in_qe;
81	logic			154	logic	[NumRegsData-1:0]		data_in_qe_buf;
ctrl_gcm_reg_err_storage;				155				data_in_we;
82	logic			156	logic		[3:0][3:0][7:0]	add_state_out;
83	logic			157	logic		[AddSOSelWidth-1:0]	
ctrl_err_storage;			ctrl_alert;	158	add_state_out_sel_raw;			
84	logic			159	add_so_sel_e			
ctrl_err_storage_d;				160	add_state_out_sel_ctrl;			
85	logic			161	add_so_sel_e			
ctrl_err_storage_q;				162	add_state_out_sel;			
86	logic		force_masks;	163	logic			
87	logic		mux_sel_err;	164	logic			
key_touch_forces_reseed;			sp_enc_err_d,	165	sp2v_e			
88	logic			166	data_out_we_ctrl;			
89	logic			167	sp2v_e			data_out_we;
90	logic			168	logic	[NumRegsData-1:0]		data_out_re;
sp_enc_err_q;			clear_on_fatal;	169	logic	[NumRegsData-1:0]		data_out_re_buf;
91	logic			170	data_out_sel_raw;		[DataOutSelWidth-1:0]	
92				171	data_out_sel_e			
93	logic	[3:0][3:0][7:0]	state_in;	172	data_out_sel_err;			data_out_sel;
94	logic	[SISelWidth-1:0]		173				
state_in_sel_raw;				174	sp2v_e			cipher_in_valid;
95	si_sel_e			175	sp2v_e			cipher_in_ready;
state_in_sel_ctrl;			state_in_sel;	176	sp2v_e			
96	si_sel_e			177	cipher_out_valid;			
97	logic			178	sp2v_e			cipher_crypt;
state_in_sel_err;				179	sp2v_e			
98	logic	[3:0][3:0][7:0]	add_state_in;	180	cipher_crypt_busy;			
99	logic	[AddSISelWidth-1:0]		181	sp2v_e			
add_state_in_sel_raw;				182	cipher_dec_key_gen;			
100	add_si_sel_e			183	sp2v_e			
add_state_in_sel_ctrl;				184	cipher_dec_key_gen_busy;			
101	add_si_sel_e			185	logic			
add_state_in_sel;				186	cipher_prng_reseed;			
102	logic			187	logic			
add_state_in_sel_err;				188	cipher_prng_reseed_busy;			
103				189	logic			
104	logic	[3:0][3:0][7:0]	state_mask;	190	cipher_key_clear;			
105	logic	[3:0][3:0][7:0]	state_init	191	logic			
[NumShares];				192	cipher_key_clear_busy;			
106	logic	[3:0][3:0][7:0]	state_done	193	logic			
[NumShares];				194	cipher_data_out_clear;			
107	logic	[3:0][3:0][7:0]	state_out;	195	logic			

```

201 logic
prd_clearing_rsd_req;
202 logic
prd_clearing_rsd_ack;

203 logic [127:0] prd_clearing_128
[NumShares];

204 logic [255:0] prd_clearing_256
[NumShares];
205 logic [3:0][31:0]
prd_clearing_data;
206 logic [255:0]
prd_clearing_key_init [NumSharesKey];
207 logic [3:0][3:0][7:0]
prd_clearing_state [NumShares];

208 logic [7:0][31:0] prd_clearing_key
[NumShares];
209
210 // Unused signals
211 logic [NumRegsData-1:0][31:0]
unused_data_out_q;
212
213 // The clearing PRNG provides pseudo-random data for
register clearing purposes.
214 aes_prng_clearing #(
215     .Width ( WidthPRDClearing ),
216     .EntropyWidth ( EntropyWidth ),
217     .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
218     .RndCnstLfsrcSeed ( RndCnstClearingLfsrcSeed ),
219     .RndCnstLfsrcPerm ( RndCnstClearingLfsrcPerm ),
220     .RndCnstSharePerm ( RndCnstClearingSharePerm )
221 ) u_aes_prng_clearing (
222     .clk_i ( clk_i ),
223     .rst_ni ( rst_ni ),
224
225     .data_update_i ( prd_clearing_update ),
226     .data_o ( prd_clearing ),
227     .reseed_req_i ( prd_clearing_rsd_req ),
228     .reseed_ack_o ( prd_clearing_rsd_ack ),
229
230     .entropy_req_o ( entropy_clearing_req_o ),
231     .entropy_ack_i ( entropy_clearing_ack_i ),
232     .entropy_i ( entropy_clearing_i )
233 );
234
235 // Generate clearing signals of appropriate widths.
236 // Different shares need to be cleared with different
pseudo-random data.
237 for (genvar s = 0; s < NumShares; s++) begin :
gen_prd_clearing_shares
238     for (genvar c = 0; c < NumChunksPRDClearing128; c++)
begin : gen_prd_clearing_128
239         assign prd_clearing_128[s][c * WidthPRDClearing +:
WidthPRDClearing] = prd_clearing[s];
240     end
241     for (genvar c = 0; c < NumChunksPRDClearing256; c++)
begin : gen_prd_clearing_256
242         assign prd_clearing_256[s][c * WidthPRDClearing +:
WidthPRDClearing] = prd_clearing[s];
243     end
244 end
245 // The data registers are always unmasked.
246 assign prd_clearing_data = prd_clearing_128[0];

247 // The initial key is always provided in two shares. The two
shares of the initial key register
248 // need to be cleared with different pseudo-random data.
249 for (genvar s = 0; s < NumSharesKey; s++) begin :
gen_prd_clearing_key_init_shares
250     for (genvar c = 0; c < NumChunksPRDClearing256; c++)
begin : gen_prd_clearing_key_init
251         assign prd_clearing_key_init[s][c * WidthPRDClearing +:
WidthPRDClearing] = prd_clearing[s];
252     end
253 end
254 // The cipher core uses multiple packed dimensions
internally but the number of bits remain the
255 // same. Since some tools fail to perform the 'conversion'
on input ports, we do it here.
256 assign prd_clearing_state = prd_clearing_128;
257 assign prd_clearing_key = prd_clearing_256;
258
259 // Inputs //
260 // Inputs //
261 // Inputs //
262
263 always_comb begin : key_init_get
264     for (int i = 0; i < NumRegsKey; i++) begin
265         key_init[0][i] = reg2hw.key_share0[i].q;
266         key_init_qe[0][i] = reg2hw.key_share0[i].qe;
267         key_init[1][i] = reg2hw.key_share1[i].q;
268         key_init_qe[1][i] = reg2hw.key_share1[i].qe;
269     end
270 end
271
272 prim_sec_anchor_buf #(
273     .Width ( NumSharesKey * NumRegsKey )
274 ) u_prim_buf_key_init_qe (
275     .in_i ( {key_init_qe[1], key_init_qe[0]} ),

```

```

276     .out_o ( {key_init_qe_buf[1], key_init_qe_buf[0]} )
277 );
278
279 always_comb begin : key_sideload_get
280     for (int s = 0; s < NumSharesKey; s++) begin
281         for (int i = 0; i < NumRegsKey; i++) begin
282             key_sideload[s][i] = keymgr_key_i.key[s][i * 32 +:
32];
283         end
284     end
285 end
286
287 always_comb begin : iv_get
288     for (int i = 0; i < NumRegsIV; i++) begin
289         iv[i] = reg2hw.iv[i].q;
290         iv_qe[i] = reg2hw.iv[i].qe;
291     end
292 end
293
294 prim_sec_anchor_buf #(
295     .Width ( NumRegsIV )
296 ) u_prim_buf_iv_qe (
297     .in_i ( iv_qe ),
298     .out_o ( iv_qe_buf )
299 );
300
301 always_comb begin : data_in_get
302     for (int i = 0; i < NumRegsData; i++) begin
303         data_in[i] = reg2hw.data_in[i].q;
304         data_in_qe[i] = reg2hw.data_in[i].qe;
305     end
306 end
307
308 prim_sec_anchor_buf #(
309     .Width ( NumRegsData )
310 ) u_prim_buf_data_in_qe (
311     .in_i ( data_in_qe ),
312     .out_o ( data_in_qe_buf )
313 );
314
315 always_comb begin : data_out_get
316     for (int i = 0; i < NumRegsData; i++) begin
317         // data_out is actually hwo, but we need hrw for hwre
318         unused_data_out_q[i] = reg2hw.data_out[i].q;
319         data_out_re[i] = reg2hw.data_out[i].re;
320     end
321 end
322
323 prim_sec_anchor_buf #(
324     .Width ( NumRegsData )
325 ) u_prim_buf_data_out_re (
326     .in_i ( data_out_re ),
327     .out_o ( data_out_re_buf )
328 );
329
330 // Key, IV and Data //
331 // Key, IV and Data //
332 // Key, IV and Data //
333
334 // SEC_CM: KEY.SEC_WIPE
335 // SEC_CM: KEY.SIDELOAD
336 // Initial Key registers
337 always_comb begin : key_init_mux
338     unique case (key_init_sel)
339         KEY_INIT_INPUT: key_init_d = key_init;
340         KEY_INIT_KEYMGR: key_init_d = key_sideload;
341         KEY_INIT_CLEAR: key_init_d = prd_clearing_key_init;
342         default: key_init_d = prd_clearing_key_init;
343     endcase
344 end
345
346 always_ff @(posedge clk_i or negedge rst_ni) begin :
key_init_reg
347     if (!rst_ni) begin
348         key_init_q <= '{default: '0};
349     end else begin
350         for (int s = 0; s < NumSharesKey; s++) begin
351             for (int i = 0; i < NumRegsKey; i++) begin
352                 if (key_init_we[s][i] == SP2V_HIGH) begin
353                     key_init_q[s][i] <= key_init_d[s][i];
354                 end
355             end
356         end
357     end
358 end
359
360 // SEC_CM: IV.CONFIG.SEC_WIPE
361 // IV registers
362 always_comb begin : iv_mux
363     unique case (iv_sel)
364         IV_INPUT: iv_d = iv;
365         IV_DATA_OUT: iv_d = data_out_d;
366         IV_DATA_OUT_RAW: iv_d = aes_transpose(state_out);
367         IV_DATA_IN_PREV: iv_d = data_in_prev.q;
368         IV_CTR: iv_d = ctr;
369         IV_CLEAR: iv_d = prd_clearing_data;
370         default: iv_d = prd_clearing_data;
371     endcase
372 end
373
374 always_ff @(posedge clk_i or negedge rst_ni) begin : iv_reg

```

```

375   if (!rst_ni) begin
376     iv_q <= '0';
377   end else begin
378     for (int i = 0; i < NumSlicesCtr; i++) begin
379       if (iv_we[i] == SP2V_HIGH) begin
380         iv_q[i] <= iv_d[i];
381       end
382     end
383   end
384 end
385
386 // SEC_CM: DATA_REG.SEC_WIPE
387 // Previous input data register
388 always_comb begin : data_in_prev_mux
389   unique case (data_in_prev_sel)
390     DIP_DATA_IN: data_in_prev_d = data_in;
391     DIP_CLEAR:  data_in_prev_d = prd_clearing_data;
392     default:    data_in_prev_d = prd_clearing_data;
393   endcase
394 end
395
396 always_ff @(posedge clk_i or negedge rst_ni) begin :
data_in_prev_reg
397   if (!rst_ni) begin
398     data_in_prev_q <= '0';
399   end else if (data_in_prev_we == SP2V_HIGH) begin
400     data_in_prev_q <= data_in_prev_d;
401   end
402 end
403
404 // Counter
405 // Counter
406 // Counter
407
408 aes_ctr_u_aes_ctr (
409   .clk_i   ( clk_i   ),
410   .rst_ni  ( rst_ni  ),
411
412   .inc32_i ( ctr_inc32 ),
413   .incr_i  ( ctr_incr  ),
414   .ready_o ( ctr_ready ),
415   .alert_o ( ctr_alert ),
416
417   .ctr_i   ( iv_q     ),
418   .ctr_o   ( ctr      ),
419   .ctr_we_o ( ctr_we   )
420 );
421
422 // Cipher Core
423 // Cipher Core
424 // Cipher Core
425
426 // Cipher core operation
427 assign cipher_op = (aes_mode_q == AES_ECB && aes_op_q ==
AES_ENC) ? CIPH_FWD :
428   (aes_mode_q == AES_ECB && aes_op_q ==
AES_DEC) ? CIPH_INV :
429   (aes_mode_q == AES_CBC && aes_op_q ==
AES_ENC) ? CIPH_FWD :
430   (aes_mode_q == AES_CBC && aes_op_q ==
AES_DEC) ? CIPH_INV :
431   (aes_mode_q ==
? CIPH_FWD :
432   (aes_mode_q ==
? CIPH_FWD :
433   (aes_mode_q ==
? CIPH_FWD :
434   (aes_mode_q ==
? CIPH_FWD : CIPH_FWD;
435
436 // This primitive is used to place a size-only constraint on
the
437 // buffers to act as a synthesis optimization barrier.
438 logic [$bits(ciph_op_e)-1:0] cipher_op_raw;
439 prim_buf #(
440   .Width($bits(ciph_op_e))
441 ) u_prim_buf_op (
442   .in_i(cipher_op),
443   .out_o(cipher_op_raw)
444 );
445 assign cipher_op_buf = ciph_op_e'(cipher_op_raw);
446
447 // Convert input data/IV to state format (every word
corresponds to one state column).
448 // Mux for state input
449 always_comb begin : state_in_mux
450   unique case (state_in_sel)
451     SI_ZERO: state_in = '0';
452     SI_DATA: state_in = aes_transpose(data_in);
453     default: state_in = '0';
454   endcase
455 end
456
457 // Mux for addition to state input
458 always_comb begin : add_state_in_mux
459   unique case (add_state_in_sel)
460     ADD_SI_ZERO: add_state_in = '0';
461     ADD_SI_IV:  add_state_in = aes_transpose(iv_q);
462     default:    add_state_in = '0';
463   endcase

```

```

464   end
465
466   if (!SecMasking) begin : gen_state_init_unmasked
467     assign state_init[0] = state_in ^ add_state_in;
468
469     logic [3:0][3:0][7:0] unused_state_mask;
470     assign unused_state_mask = state_mask;
471
472   end else begin : gen_state_init_masked
473     assign state_init[0] = (state_in ^ add_state_in) ^
state_mask; // Masked data share
474     assign state_init[1] =
state_mask; // Mask share
475   end
476
477   if (!SecMasking) begin : gen_key_init_unmasked
478     // Combine the two key shares for the unmasked cipher
core. This causes SCA leakage of the key
479     // and thus should be avoided.
480     assign key_init_cipher[0] = key_init_q[0] ^ key_init_q[1];
481
482   end else begin : gen_key_init_masked
483     // Forward the masked key share and the mask share to the
masked cipher core.
484     assign key_init_cipher = key_init_q;
485   end
486
487   // SEC_CM: KEY.MASKING
488   // Cipher core
489   aes_cipher_core #(
490     .AES192Enable ( AES192Enable ),
491     .SecMasking   ( SecMasking   ),
492     .SecSBoxImpl  ( SecSBoxImpl  ),
493     .SecAllowForcingMasks ( SecAllowForcingMasks ),
494     .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
495     .RndCnstMaskingLfSrSeed ( RndCnstMaskingLfSrSeed ),
496     .RndCnstMaskingLfSrPerm ( RndCnstMaskingLfSrPerm )
497   ) u_aes_cipher_core (
498     .clk_i   ( clk_i   ),
499     .rst_ni  ( rst_ni  ),
500
501     .in_valid_i ( cipher_in_valid ),
502     .in_ready_o ( cipher_in_ready ),
503
504     .out_valid_o ( cipher_out_valid ),
505     .out_ready_i ( cipher_out_ready ),
506
507     .cfg_valid_i ( ~ctrl_err_storage ), //
Used for gating assertions only.
508     .op_i        ( cipher_op_buf ),
509     .key_len_i   ( key_len_q ),
510     .crypt_i     ( cipher_crypt ),
511     .crypt_o     ( cipher_crypt_busy ),
512     .dec_key_gen_i ( cipher_dec_key_gen ),
513     .dec_key_gen_o ( cipher_dec_key_gen_busy ),
514     .prng_reseed_i ( cipher_prng_reseed ),
515     .prng_reseed_o ( cipher_prng_reseed_busy ),
516     .key_clear_i  ( cipher_key_clear ),
517     .key_clear_o  ( cipher_key_clear_busy ),
518     .data_out_clear_i ( cipher_data_out_clear ),
519     .data_out_clear_o ( cipher_data_out_clear_busy ),
520     .alert_fatal_i ( alert_fatal_o ),
521     .alert_o      ( cipher_alert ),
522
523     .prd_clearing_state_i ( prd_clearing_state ),
524     .prd_clearing_key_i   ( prd_clearing_key ),
525
526     .force_masks_i ( force_masks ),
527     .data_in_mask_o ( state_mask ),
528     .entropy_req_o  ( entropy_masking_req_o ),
529     .entropy_ack_i  ( entropy_masking_ack_i ),
530     .entropy_i      ( entropy_masking_i ),
531
532     .state_init_i   ( state_init ),
533     .key_init_i     ( key_init_cipher ),
534     .state_o        ( state_done )
535   );
536
537   if (!SecMasking) begin : gen_state_out_unmasked
538     assign state_out = state_done[0];
539   end else begin : gen_state_out_masked
540     // Unmask the cipher core output. This might get reworked
in the future when masking the
541     // counter and feedback path through the IV regs.
542
543     // Only unmask the final cipher core output. Unmasking
intermediate output data causes
544     // additional SCA leakage and thus has to be avoided.
Forward PRD instead of a deterministic
545     // value to avoid leaking the cipher core output when it
becomes valid. Don't unmask the hash
546     // subkey in case of GCM.
547     logic [3:0][3:0][7:0] state_done_muxed [NumShares];
548     for (genvar s = 0; s < NumShares; s++) begin :
gen_state_done_muxed
549       assign state_done_muxed[s] = ((cipher_out_valid ==
SP2V_HIGH) &&
550         !(aes_mode_q == AES_GCM &&
gcm_phase_q == GCM_INIT)) ? state_done[s] :
prd_clearing_state[s];
551     end
552

```

```

553
554 // Avoid aggressive synthesis optimizations.
555 logic [3:0][3:0][7:0] state_done_buf [NumShares];
556 prim_buf #(
557   .Width ( 128 * NumShares )
558 ) u_prim_state_done_muxed (
559   .in_i ( {state_done_muxed[1], state_done_muxed[0]} ),
560   .out_o ( {state_done_buf[1], state_done_buf[0]} )
561 );
562
563 // Unmask the cipher core output.
564 assign state_out = state_done_buf[0] ^ state_done_buf[1];
565 end
566
567 // Mux for addition to state output
568 always_comb begin : add_state_out_mux
569   unique case (add_state_out_sel)
570     ADD_SO_ZERO: add_state_out = '0;
571     ADD_SO_IV:  add_state_out = aes_transpose(iv_q);
572     ADD_SO_DIP: add_state_out =
aes_transpose(data_in_prev_q);
573     default:    add_state_out = '0;
574   endcase
575 end
576
577 // Convert output state to output data format (every column
corresponds to one output word).
578 assign data_out = aes_transpose(state_out ^ add_state_out);
579
580 //////////////////////////////////////////////////
581 // GHASH //
582 //////////////////////////////////////////////////
583
584 if (AESGCMEnable) begin : gen_ghash
585   logic [3:0][3:0][7:0] ghash_state_out;
586
587   logic ghash_clear;
588   assign ghash_clear = cipher_key_clear |
cipher_key_clear_busy;
589
590 // The number of cycles must be a power of two and ideally
matches the minimum latency of the
591 // cipher core which is 56 clock cycles (masked) or 12
clock cycles (unmasked) for AES-128.
592 localparam int unsigned GhashGFMultCycles = (SecSBoxImpl
== SBoxImplDom) ? 32 : 8;
593
594 // SEC_CM: KEY.MASKING
595 // The actual GHASH module.
596 aes_ghash #(
597   .SecMasking ( SecMasking ),
598   .GFMultCycles ( GhashGFMultCycles )
599 ) u_aes_ghash (
600   .clk_i ( clk_i ),
601   .rst_ni ( rst_ni ),
602
603   .in_valid_i ( ghash_in_valid ),
604   .in_ready_o ( ghash_in_ready ),
605
606   .out_valid_o ( ghash_out_valid ),
607   .out_ready_i ( ghash_out_ready ),
608
609   .op_i ( aes_op_q ),
610   .gcm_phase_i ( gcm_phase_q ),
611   .num_valid_bytes_i ( num_valid_bytes_q ),
612   .load_hash_subkey_i ( ghash_load_hash_subkey ),
613   .clear_i ( ghash_clear ),
614   .first_block_o ( ghash_first_block ),
615   .alert_fatal_i ( alert_fatal_o ),
616   .alert_o ( ghash_alert ),
617
618   .data_in_prev_i ( data_in_prev_q ),
619   .data_out_i ( data_out ),
620   .cipher_state_done_i ( state_done ),
621   .ghash_state_done_o ( ghash_state_out )
622 );
623
624 // Mux for output data registers
625 always_comb begin : data_out_mux
626   unique case (data_out_sel)
627     DATA_OUT_CIPHER: data_out_d = data_out;
628     DATA_OUT_GHASH: data_out_d = ghash_state_out;
629     default:          data_out_d = ghash_state_out;
630   endcase
631 end
632
633 end else begin : gen_no_ghash
634 // GHASH isn't there and thus generates no back pressure /
alerts / output to be muxed.
635 assign ghash_in_ready = SP2V_HIGH;
636 assign ghash_out_valid = SP2V_HIGH;
637 assign ghash_first_block = 1'b0;
638 assign ghash_alert = 1'b0;
639 assign data_out_d = data_out;
640
641 // Tie-off unused signals.
642 sp2v_e unused_ghash_in_valid;
643 sp2v_e unused_ghash_out_ready;
644 sp2v_e unused_ghash_load_hash_subkey;

```

```

645 logic [4:0] unused_num_valid_bytes;
646 data_out_sel_e unused_data_out_sel;
647 assign unused_ghash_in_valid = ghash_in_valid;
648 assign unused_ghash_out_ready = ghash_out_ready;
649 assign unused_ghash_load_hash_subkey =
ghash_load_hash_subkey;
650 assign unused_num_valid_bytes = num_valid_bytes_q;
651 assign unused_data_out_sel = data_out_sel;
652 end
653
654 //////////////////////////////////////////////////
655 // Control Registers //
656 //////////////////////////////////////////////////
657
658 // Shadowed register primitive
659 aes_ctrl_reg_shadowed #(
660   .AES192Enable ( AES192Enable ),
661   .AESGCMEnable ( AESGCMEnable )
662 ) u_ctrl_reg_shadowed (
663   .clk_i ( clk_i ),
664   .rst_ni ( rst_ni ),
665   .rst_shadowed_ni ( rst_shadowed_ni ),
666   .qe_o ( ctrl_qe ),
667   .we_i ( ctrl_we ),
668   .phase_o ( ctrl_phase ),
669   .operation_o ( aes_op_q ),
670   .mode_o ( aes_mode_q ),
671   .key_len_o ( key_len_q ),
672   .sideload_o ( sideload_q ),
673   .prng_reseed_rate_o ( prng_reseed_rate_q ),
674   .manual_operation_o ( manual_operation_q ),
675   .err_update_o ( ctrl_reg_err_update ),
676   .err_storage_o ( ctrl_reg_err_storage ),
677   .reg2hw_ctrl_i ( reg2hw_ctrl_shadowed ),
678   .hw2reg_ctrl_o ( hw2reg_ctrl_shadowed )
679 );
680
681 // Auxiliary control register signals
682 assign key_touch_forces_reseed =
reg2hw_ctrl_aux_shadowed.key_touch_forces_reseed.q;
683 assign force_masks =
reg2hw_ctrl_aux_shadowed.force_masks.q;
684
685 // GCM control register
686 aes_ctrl_gcm_reg_shadowed #(
687   .AESGCMEnable ( AESGCMEnable )
688 ) u_ctrl_gcm_reg_shadowed (
689   .clk_i ( clk_i ),
690   .rst_ni ( rst_ni ),
691   .rst_shadowed_ni ( rst_shadowed_ni ),
692   .qe_o ( ctrl_gcm_qe ),
693   .we_i ( ctrl_gcm_we ),
694   .phase_o ( ctrl_gcm_phase ),
695   .init_done_i ( gcm_init_done ),
696   .first_block_i ( ghash_first_block ),
697   .gcm_phase_o ( gcm_phase_q ),
698   .num_valid_bytes_o ( num_valid_bytes_q ),
699   .err_update_o ( ctrl_gcm_reg_err_update ),
700   .err_storage_o ( ctrl_gcm_reg_err_storage ),
701   .reg2hw_ctrl_gcm_i ( reg2hw_ctrl_gcm_shadowed ),
702   .hw2reg_ctrl_gcm_o ( hw2reg_ctrl_gcm_shadowed )
703 );
704
705 //////////////////////////////////////////////////
706 // Control //
707 //////////////////////////////////////////////////
708
709 // Control
710 aes_control #(
711   .AESGCMEnable ( AESGCMEnable ),
712   .SecMasking ( SecMasking ),
713   .SecStartTriggerDelay ( SecStartTriggerDelay )
714 ) u_aes_control (
715   .clk_i ( clk_i ),
716   .rst_ni ( rst_ni ),
717
718   .ctrl_qe_i ( ctrl_qe ),
719   .ctrl_we_o ( ctrl_we ),
720   .ctrl_phase_i ( ctrl_phase ),
721   .ctrl_err_storage_i ( ctrl_err_storage ),
722   .op_i ( aes_op_q ),
723   .mode_i ( aes_mode_q ),
724   .cipher_op_i ( cipher_op_buf ),
725   .sideload_i ( sideload_q ),
726   .prng_reseed_rate_i ( prng_reseed_rate_q ),
727   .manual_operation_i ( manual_operation_q ),
728   .key_touch_forces_reseed_i ( key_touch_forces_reseed ),
729   .ctrl_gcm_qe_i ( ctrl_gcm_qe_i )

```

```

( ctrl_gcm_qe ),
730 .ctrl_gcm_we_o ),
( ctrl_gcm_we ),
731 .ctrl_gcm_phase_i ),
( ctrl_gcm_phase ),
732 .gcm_init_done_o ),
( gcm_init_done ),
733 .gcm_phase_i ),
( gcm_phase_q ),
734 .start_i ),
( reg2hw.trigger.start.q ),
735 .key_iv_data_in_clear_i ),
( reg2hw.trigger.key_iv_data_in_clear.q ),
736 .data_out_clear_i ),
( reg2hw.trigger.data_out_clear.q ),
737 .prng_reseed_i ),
( reg2hw.trigger.prng_reseed.q ),
738 .mux_sel_err_i ),
( mux_sel_err ),
739 .sp_enc_err_i ),
( sp_enc_err_q ),
740 .lc_escalate_en_i ),
( lc_escalate_en_i ),
741 .alert_fatal_i ),
( alert_fatal_o ),
742 .alert_o ),
( ctrl_alert ),
743 .key_sideload_valid_i ),
( keymgr.key_i.valid ),
745 .key_init_qe_i ),
( key_init_qe_buf ),
746 .iv_qe_i ),
( iv_qe_buf ),
747 .data_in_qe_i ),
( data_in_qe_buf ),
748 .data_out_re_i ),
( data_out_re_buf ),
749 .data_in_we_o ),
( data_in_we ),
750 .data_out_sel_o ),
( data_out_sel_ctrl ),
751 .data_out_we_o ),
( data_out_we_ctrl ),
752 .data_in_prev_sel_o ),
( data_in_prev_sel_ctrl ),
754 .data_in_prev_we_o ),
( data_in_prev_we_ctrl ),
755 .state_in_sel_o ),
( state_in_sel_ctrl ),
757 .add_state_in_sel_o ),
( add_state_in_sel_ctrl ),
758 .add_state_out_sel_o ),
( add_state_out_sel_ctrl ),
759 .ctr_inc32_o ),
( ctr_inc32 ),
761 .ctr_incr_o ),
( ctr_incr ),
762 .ctr_ready_i ),
( ctr_ready ),
763 .ctr_we_i ),
( ctr_we ),
764 .cipher_in_valid_o ),
( cipher_in_valid ),
766 .cipher_in_ready_i ),
( cipher_in_ready ),
767 .cipher_out_valid_i ),
( cipher_out_valid ),
768 .cipher_out_ready_o ),
( cipher_out_ready ),
769 .cipher_crypt_o ),
( cipher_crypt ),
770 .cipher_crypt_i ),
( cipher_crypt_busy ),
771 .cipher_dec_key_gen_o ),
( cipher_dec_key_gen ),
772 .cipher_dec_key_gen_i ),
( cipher_dec_key_gen_busy ),
773 .cipher_prng_reseed_o ),
( cipher_prng_reseed ),
774 .cipher_prng_reseed_i ),
( cipher_prng_reseed_busy ),
775 .cipher_key_clear_o ),
( cipher_key_clear ),
776 .cipher_key_clear_i ),
( cipher_key_clear_busy ),
777 .cipher_data_out_clear_o ),
( cipher_data_out_clear ),
778 .cipher_data_out_clear_i ),
( cipher_data_out_clear_busy ),
779 .ghash_in_valid_o ),
( ghash_in_valid ),
781 .ghash_in_ready_i ),
( ghash_in_ready ),
782 .ghash_out_valid_i ),
( ghash_out_valid ),

```

```

783 .ghash_out_ready_o ),
( ghash_out_ready ),
784 .ghash_load_hash_subkey_o ),
( ghash_load_hash_subkey ),
785 .key_init_sel_o ),
( key_init_sel_ctrl ),
787 .key_init_we_o ),
( key_init_we_ctrl ),
788 .iv_sel_o ),
( iv_sel_ctrl ),
789 .iv_we_o ),
( iv_we_ctrl ),
790 .prng_update_o ),
( prd_clearing_update ),
792 .prng_reseed_req_o ),
( prd_clearing_rsd_req ),
793 .prng_reseed_ack_i ),
( prd_clearing_rsd_ack ),
794 .start_o ),
( hw2reg.trigger.start.d ),
796 .start_we_o ),
( hw2reg.trigger.start.de ),
797 .key_iv_data_in_clear_o ),
( hw2reg.trigger.key_iv_data_in_clear.d ),
798 .key_iv_data_in_clear_we_o ),
( hw2reg.trigger.key_iv_data_in_clear.de ),
799 .data_out_clear_o ),
( hw2reg.trigger.data_out_clear.d ),
800 .data_out_clear_we_o ),
( hw2reg.trigger.data_out_clear.de ),
801 .prng_reseed_o ),
( hw2reg.trigger.prng_reseed.d ),
802 .prng_reseed_we_o ),
( hw2reg.trigger.prng_reseed.de ),
803 .idle_o ),
( hw2reg.status.idle.d ),
805 .idle_we_o ),
( hw2reg.status.idle.de ),
806 .stall_o ),
( hw2reg.status.stall.d ),
807 .stall_we_o ),
( hw2reg.status.stall.de ),
808 .output_lost_i ),
( reg2hw.status.output_lost.q ),
809 .output_lost_o ),
( hw2reg.status.output_lost.d ),
810 .output_lost_we_o ),
( hw2reg.status.output_lost.de ),
811 .output_valid_o ),
( hw2reg.status.output_valid.d ),
812 .output_valid_we_o ),
( hw2reg.status.output_valid.de ),
813 .input_ready_o ),
( hw2reg.status.input_ready.d ),
814 .input_ready_we_o ),
( hw2reg.status.input_ready.de ),
815 );
816 // SEC_CM: DATA_REG.SEC_WIPE
817 // Input data register clear
818 always_comb begin : data_in_reg_clear
819   for (int i = 0; i < NumRegsData; i++) begin
820     hw2reg.data_in[i].d = prd_clearing_data[i];
821     hw2reg.data_in[i].de = data_in_we;
822   end
823 end
824 // Selectors //
825 // We use sparse encodings for these mux selector signals
826 // and must ensure that:
827 // 1. The synthesis tool doesn't optimize away the sparse
828 // encoding.
829 // 2. The selector signal is always valid. More precisely,
830 // an alert or SVA is triggered if a
831 // selector signal takes on an invalid value.
832 // 3. The alert signal remains asserted until reset even if
833 // the selector signal becomes valid
834 // again. This is achieved by driving the control FSM
835 // into the terminal error state whenever
836 // any mux selector signal becomes invalid.
837 // If any mux selector signal becomes invalid, the control
838 // FSM further prevents any data from
839 // being released from the cipher core by de-asserting the
840 // write enable of the output data
841 // registers.
842 aes_sel_buf_chk #(
843   .Num ( DIPSelNum ),
844   .Width ( DIPSelWidth ),
845   .EnSecBuf ( 1'b1 )
846 ) u_aes_data_in_prev_sel_buf_chk (
847   .clk_i ( clk_i ),
848   .rst_ni ( rst_ni ),

```

```

849 .sel_i ( data_in_prev_sel_ctrl ),
850 .sel_o ( data_in_prev_sel_raw ),
851 .err_o ( data_in_prev_sel_err );
852 );
853 assign data_in_prev_sel = dip_sel_e'(data_in_prev_sel_raw);
854
855 aes_sel_buf_chk #(
856 .Num ( SISelNum ),
857 .Width ( SISelWidth ),
858 .EnSecBuf ( 1'b1 )
859 ) u_aes_state_in_sel_buf_chk (
860 .clk_i ( clk_i ),
861 .rst_ni ( rst_ni ),
862 .sel_i ( state_in_sel_ctrl ),
863 .sel_o ( state_in_sel_raw ),
864 .err_o ( state_in_sel_err );
865 );
866 assign state_in_sel = si_sel_e'(state_in_sel_raw);
867
868 aes_sel_buf_chk #(
869 .Num ( AddSISelNum ),
870 .Width ( AddSISelWidth ),
871 .EnSecBuf ( 1'b1 )
872 ) u_aes_add_state_in_sel_buf_chk (
873 .clk_i ( clk_i ),
874 .rst_ni ( rst_ni ),
875 .sel_i ( add_state_in_sel_ctrl ),
876 .sel_o ( add_state_in_sel_raw ),
877 .err_o ( add_state_in_sel_err );
878 );
879 assign add_state_in_sel =
add_si_sel_e'(add_state_in_sel_raw);
880
881 aes_sel_buf_chk #(
882 .Num ( AddSOSelNum ),
883 .Width ( AddSOSelWidth ),
884 .EnSecBuf ( 1'b1 )
885 ) u_aes_add_state_out_sel_buf_chk (
886 .clk_i ( clk_i ),
887 .rst_ni ( rst_ni ),
888 .sel_i ( add_state_out_sel_ctrl ),
889 .sel_o ( add_state_out_sel_raw ),
890 .err_o ( add_state_out_sel_err );
891 );
892 assign add_state_out_sel =
add_so_sel_e'(add_state_out_sel_raw);
893
894 aes_sel_buf_chk #(
895 .Num ( KeyInitSelNum ),
896 .Width ( KeyInitSelWidth ),
897 .EnSecBuf ( 1'b1 )
898 ) u_aes_key_init_sel_buf_chk (
899 .clk_i ( clk_i ),
900 .rst_ni ( rst_ni ),
901 .sel_i ( key_init_sel_ctrl ),
902 .sel_o ( key_init_sel_raw ),
903 .err_o ( key_init_sel_err );
904 );
905 assign key_init_sel = key_init_sel_e'(key_init_sel_raw);
906
907 aes_sel_buf_chk #(
908 .Num ( IVSelNum ),
909 .Width ( IVSelWidth ),
910 .EnSecBuf ( 1'b1 )
911 ) u_aes_iv_sel_buf_chk (
912 .clk_i ( clk_i ),
913 .rst_ni ( rst_ni ),
914 .sel_i ( iv_sel_ctrl ),
915 .sel_o ( iv_sel_raw ),
916 .err_o ( iv_sel_err );
917 );
918 assign iv_sel = iv_sel_e'(iv_sel_raw);
919
920 aes_sel_buf_chk #(
921 .Num ( DataOutSelNum ),
922 .Width ( DataOutSelWidth ),
923 .EnSecBuf ( 1'b1 )
924 ) u_aes_data_out_sel_buf_chk (
925 .clk_i ( clk_i ),
926 .rst_ni ( rst_ni ),
927 .sel_i ( data_out_sel_ctrl ),
928 .sel_o ( data_out_sel_raw ),
929 .err_o ( data_out_sel_err );
930 );
931 assign data_out_sel = data_out_sel_e'(data_out_sel_raw);
932
933 // Signal invalid mux selector signals to control FSM which
will lock up and trigger an alert.
934 assign mux_sel_err = data_in_prev_sel_err | state_in_sel_err |
add_state_in_sel_err |
935 add_state_out_sel_err | key_init_sel_err | iv_sel_err |
data_out_sel_err;
936
937 //////////////////////////////////////////////////
938 // Sparsely Encoded Signals //
939 //////////////////////////////////////////////////
940
941 // We use sparse encodings for various critical signals and
must ensure that:
942 // 1. The synthesis tool doesn't optimize away the sparse

```

```

encoding.
943 // 2. The sparsely encoded signal is always valid. More
precisely, an alert or SVA is triggered
944 // if a sparse signal takes on an invalid value.
945 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
946 // This is achieved by driving the control FSM into the
terminal error state whenever any
947 // sparsely encoded signal becomes invalid.
948 //
949 // If any sparsely encoded signal becomes invalid, the core
controller further immediately
950 // de-asserts the data_out_we_o signal to prevent any data
from being released.
951
952 // We use vectors of sparsely encoded signals to reduce code
duplication.
953 localparam int unsigned NumSp2Vsig = NumSharesKey *
NumRegsKey + NumSlicesCtr + 2;
954 sp2v_e [NumSp2Vsig-1:0] sp2v_sig;
955 sp2v_e [NumSp2Vsig-1:0] sp2v_sig_chk;
956 logic [NumSp2Vsig-1:0][Sp2Vwidth-1:0] sp2v_sig_chk_raw;
957 logic [NumSp2Vsig-1:0] sp2v_sig_err;
958
959 for (genvar s = 0; s < NumSharesKey; s++) begin :
gen_use_key_init_we_ctrl_shares
960 for (genvar i = 0; i < NumRegsKey; i++) begin :
gen_use_key_init_we_ctrl
961 assign sp2v_sig[s * NumRegsKey + i] =
key_init_we_ctrl[s][i];
962 end
963 end
964 for (genvar i = 0; i < NumSlicesCtr; i++) begin :
gen_use_iv_we_ctrl
965 assign sp2v_sig[NumSharesKey * NumRegsKey + i] =
iv_we_ctrl[i];
966 end
967 assign sp2v_sig[NumSharesKey * NumRegsKey + NumSlicesCtr +
0] = data_in_prev_we_ctrl;
968 assign sp2v_sig[NumSharesKey * NumRegsKey + NumSlicesCtr +
1] = data_out_we_ctrl;
969
970 // All signals inside sp2v_sig are eventually converted to
single-rail signals.
971 localparam bit [NumSp2Vsig-1:0] Sp2VEnSecBuf =
{NumSp2Vsig{1'b1}};
972
973 // Individually check sparsely encoded signals.
974 for (genvar i = 0; i < NumSp2Vsig; i++) begin :
gen_sel_buf_chk
975 aes_sel_buf_chk #(
976 .Num ( Sp2VNum ),
977 .Width ( Sp2Vwidth ),
978 .EnSecBuf ( Sp2VEnSecBuf[i] )
979 ) u_aes_sp2v_sig_buf_chk_i (
980 .clk_i ( clk_i ),
981 .rst_ni ( rst_ni ),
982 .sel_i ( sp2v_sig[i] ),
983 .sel_o ( sp2v_sig_chk_raw[i] ),
984 .err_o ( sp2v_sig_err[i] );
985 );
986 assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
987 end
988
989 for (genvar s = 0; s < NumSharesKey; s++) begin :
gen_key_init_we_shares
990 for (genvar i = 0; i < NumRegsKey; i++) begin :
gen_key_init_we
991 assign key_init_we[s][i] = sp2v_sig_chk[s * NumRegsKey +
i];
992 end
993 end
994 for (genvar i = 0; i < NumSlicesCtr; i++) begin : gen_iv_we
995 assign iv_we[i] = sp2v_sig_chk[NumSharesKey *
NumRegsKey + i];
996 end
997 assign data_in_prev_we = sp2v_sig_chk[NumSharesKey *
NumRegsKey + NumSlicesCtr + 0];
998 assign data_out_we = sp2v_sig_chk[NumSharesKey *
NumRegsKey + NumSlicesCtr + 1];
999
1000 // Collect encoding errors.
1001 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
1002 // are used. Here, we collect also encoding errors detected
in other places of the core.
1003 assign sp_enc_err_d = |sp2v_sig_err;
1004
1005 // We need to register the collected error signal to avoid
circular loops in the core controller
1006 // related to iv_we and data_out_we.
1007 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_sp_enc_err
1008 if (!rst_ni) begin
1009 sp_enc_err_q <= 1'b0;
1010 end else if (sp_enc_err_d) begin
1011 sp_enc_err_q <= 1'b1;
1012 end
1013 end

```

```

1014
1015 ////////////////
1016 // Outputs //
1017 ////////////////
1018
1019 always_ff @(posedge clk_i or negedge rst_ni) begin :
data_out_reg
1020 if (!rst_ni) begin
1021 data_out_q <= '0;
1022 end else if (data_out_we == SP2V_HIGH) begin
1023 data_out_q <= data_out_d;
1024 end
1025 end
1026
1027 always_comb begin : key_reg_put
1028 for (int i = 0; i < NumRegsKey; i++) begin
1029 hw2reg.key_share0[i].d = key_init_q[0][i];
1030 hw2reg.key_share1[i].d = key_init_q[1][i];
1031 end
1032 end
1033
1034 always_comb begin : iv_reg_put
1035 for (int i = 0; i < NumRegsIv; i++) begin
1036 // Software updates IV in chunks of 32 bits. Internally,
the counter updates SliceSizeCtr
1037 // bits at a time.
1038 hw2reg.iv[i].d = {iv_q[2 * i + 1], iv_q[2 * i]};
1039 end
1040 end
1041
1042 always_comb begin : data_out_put
1043 for (int i = 0; i < NumRegsData; i++) begin
1044 hw2reg.data_out[i].d = data_out_q[i];
1045 end
1046 end
1047
1048 ////////////////
1049 // Alerts //
1050 ////////////////
1051
1052 // Should fatal alerts clear the status register?
1053 assign clear_on_fatal = ClearStatusOnFatalAlert ?
alert_fatal_o : 1'b0;
1054
1055 // Recoverable alert conditions are signaled as a single
alert event.
1056 assign ctrl_err_update = ctrl_reg_err_update |
shadowed_update_err_i | ctrl_gcm_reg_err_update;
1057 assign alert_recov_o = ctrl_err_update;
1058
1059 // The recoverable alert is observable via status register
until the AES operation is restarted
1060 // by re-writing the Control Register. Fatal alerts clear
all other bits in the status register.
1061 assign hw2reg.status.alert_recov_ctrl_update_err.d =
ctrl_err_update & ~clear_on_fatal;
1062 assign hw2reg.status.alert_recov_ctrl_update_err.de =
ctrl_err_update | ctrl_we | clear_on_fatal;
1063
1064 // Fatal alert conditions need to remain asserted until
reset.
1065 assign ctrl_err_storage_d =
ctrl_reg_err_storage | shadowed_storage_err_i |
ctrl_gcm_reg_err_storage;
1066 always_ff @(posedge clk_i or negedge rst_ni) begin :
ctrl_err_storage_reg
1067 if (!rst_ni) begin
1068 ctrl_err_storage_q <= 1'b0;
1069 end else if (ctrl_err_storage_d) begin
1070 ctrl_err_storage_q <= 1'b1;
1071 end
1072 end
1073
1074 assign ctrl_err_storage = ctrl_err_storage_d |
ctrl_err_storage_q;
1075
1076 // Collect fatal alert signals.
1077 assign alert_fatal_o = ctrl_err_storage |
1078 ctr_alert |
1079 cipher_alert |
1080 ghash_alert |
1081 ctrl_alert |
1082 intg_err_alert_i;
1083
1084 // Make the fatal alert observable via status register.
1085 assign hw2reg.status.alert_fatal_fault.d = alert_fatal_o;
1086 assign hw2reg.status.alert_fatal_fault.de = alert_fatal_o;
1087
1088 // Unused alert signals
1089 logic unused_alert_signals;
1090 assign unused_alert_signals = ^reg2hw.alert_test;
1091
1092 // Unused inputs
1093 logic unused_status;
1094 assign unused_status =
1095 reg2hw.status.idle.q ^ reg2hw.status.output_valid.q ^
reg2hw.status.input_ready.q;
1096
1097 ////////////////
1098 // Assertions //
1099 ////////////////

```

```

1100
1101 // Create a lint error to reduce the risk of accidentally
disabling the masking.
1102 `ASSERT_STATIC_LINT_ERROR(AesCoreSecMaskingNonDefault,
SecMasking == 1)
1103
1104 // Selectors must be known/valid
1105 `ASSERT(AesModeValid, !ctrl_err_storage |> aes_mode_q
inside {
1106 AES_ECB,
1107 AES_CBC,
1108 AES_CFB,
1109 AES_OFB,
1110 AES_CTR,
1111 AES_GCM,
1112 AES_NONE
1113 })
1114 `ASSERT(AesOpValid, !ctrl_err_storage |> aes_op_q inside {
1115 AES_ENC,
1116 AES_DEC
1117 })
1118
1119 // Check parameters
1120 `ASSERT_INIT(AesNumSlicesCtr, NumSlicesCtr == 8)
1121
1122 // Signals used for assertions only.
1123 logic [3:0][31:0] state_done_transposed,
unused_state_done_transposed;
1124 if (!SecMasking) begin : gen_state_done_transposed_unmasked
1125 assign state_done_transposed =
aes_transpose(state_done[0]);
1126 end else begin : gen_state_done_transposed_masked
1127 assign state_done_transposed = aes_transpose(state_done[0]
^ state_done[1]);
1128 end
1129 assign unused_state_done_transposed = state_done_transposed;
1130
1131 // Ensure that upon local escalation of any of the FSMs, no
intermediate state is released from
1132 // the cipher core into the software readable output data or
IV registers.
1133 `ASSERT(AesSecCmDataRegLocalEscDataOut, $changed(data_out_q)
&& alert_fatal_o &&
1134 ($past(cipher_crypt, 2) == SP2V_HIGH ||
$past(cipher_crypt_busy, 2) == SP2V_HIGH) |>
1135 ($past(data_out_q) != $past(state_done_transposed, 2))
&&
1136 ($past(data_out_q) != $past(state_done_transposed, 2) ^
$past(iv_q, 2)) &&
1137 ($past(data_out_q) != $past(state_done_transposed, 2) ^
$past(data_in_prev_q, 2)))
1138
1139 `ASSERT(AesSecCmDataRegLocalEscIv, $changed(iv_q) &&
alert_fatal_o &&
1140 ($past(cipher_crypt, 2) == SP2V_HIGH ||
$past(cipher_crypt_busy, 2) == SP2V_HIGH) |>
1141 ($past(iv_q) != $past(state_done_transposed, 2)) &&
1142 ($past(iv_q) != $past(state_done_transposed, 2) ^
$past(iv_q, 2)) &&
1143 ($past(iv_q) != $past(state_done_transposed, 2) ^
$past(data_in_prev_q, 2)))
1144
1145 endmodule

```

A.28 lowRISC_opentitan/hw/ip/aes/rtl/aes.sv (328줄, 13,989B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES top-level wrapper
6
7 `include "prim_assert.sv"
8
9 module aes
10 import aes_pkg::*;
11 import aes_reg_pkg::*;
12 #(
13 parameter bit AES192Enable = 1, // Can be
0 (disable), or 1 (enable).
14 parameter bit AESGCMEEnable = 1, // Can be
0 (disable), or 1 (enable).
15 parameter bit SecMasking = 1, // Can be
0 (no masking), or
16 // 1
17 (first-order masking) of the cipher // core.
18 // masked
19 parameter sbbox_impl_e SecSBoxImpl =
S-Box, see SecSBoxImpl parameter.
20 parameter int unsigned SecStartTriggerDelay = 0, // Manual
start trigger delay, useful for
21 // SCA

```

```

measurements. A value of e.g. 40
22                                     // allows
the processor to go into sleep
23                                     // before
AES starts operation.
24 parameter bit SecAllowForcingMasks = 0, // Allow
forcing masks to constant values using
25                                     //
FORCE_MASKS bit in Auxiliary Control
26                                     //
Register. Useful for SCA only.
27 parameter bit SecSkipPRNGReseeding = 0, // The
current SCA setup doesn't provide enough
28                                     //
resources to implement the infrastructure
29                                     //
required for PRNG reseeding (CSRNG, EDN).
30                                     // To
enable SCA resistance evaluations, we
31                                     // need to
skip reseeding requests.
32                                     // Useful
for SCA only.
33 parameter logic [NumAlerts-1:0] AlertAsyncOn =
{NumAlerts{1'b1}},
34 // Number of cycles a differential skew is tolerated on the
alert signal
35 parameter int unsigned AlertSkewCycles = 1,
36 parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeed =
RndCnstClearingLfsrSeedDefault,
37 parameter clearing_lfsr_perm_t RndCnstClearingLfsrPerm =
RndCnstClearingLfsrPermDefault,
38 parameter clearing_lfsr_perm_t RndCnstClearingSharePerm =
RndCnstClearingSharePermDefault,
39 parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed =
RndCnstMaskingLfsrSeedDefault,
40 parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm =
RndCnstMaskingLfsrPermDefault
41 ) (
42 input logic clk_i,
43 input logic rst_ni,
44 input logic
rst_shadowed_ni,
45
46 // Idle indicator for clock manager
47 output prim_mubi_pkg::mubi4_t idle_o,
48
49 // Status indicators for DMA integrations
50 output logic
output_valid_o,
51 output logic
input_ready_o,
52
53 // Life cycle
54 input lc_ctrl_pkg::lc_tx_t
lc_escalate_en_i,
55
56 // Entropy distribution network (EDN) interface
57 input logic clk_edn_i,
58 input logic
rst_edn_ni,
59 output edn_pkg::edn_req_t edn_o,
60 input edn_pkg::edn_rsp_t edn_i,
61
62 // Key manager (keymgr) key sideload interface
63 input keymgr_pkg::hw_key_req_t
keymgr_key_i,
64
65 // Bus interface
66 input tlul_pkg::tl_h2d_t tl_i,
67 output tlul_pkg::tl_d2h_t tl_o,
68
69 // Alerts
70 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
71 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o
72 );
73
74 localparam int unsigned EntropyWidth =
edn_pkg::ENDPOINT_BUS_WIDTH;
75
76 // Signals
77 aes_reg2hw_t reg2hw;
78 aes_hw2reg_t hw2reg;
79
80 logic [NumAlerts-1:0] alert;
81 lc_ctrl_pkg::lc_tx_t lc_escalate_en;
82
83 logic edn_req_int;
84 logic edn_req_hold_d, edn_req_hold_q;
85 logic edn_req;
86 logic edn_ack;
87 logic [EntropyWidth-1:0] edn_data;
88 logic unused_edn_fips;
89 logic entropy_clearing_req;
entropy_masking_req;
90 logic entropy_clearing_ack;
entropy_masking_ack;
91
92 ///////////////

```

```

93 // Inputs //
94 ///////////////
95
96 // SEC_CM: BUS INTEGRITY
97 // SEC_CM: AUX.CONFIG.SHADOW
98 // SEC_CM: AUX.CONFIG.REGWEN
99 // SEC_CM: KEY.SW_UNREADABLE
100 // SEC_CM: DATA.REG.SW_UNREADABLE
101 // Register interface
102 logic intg_err_alert;
103 logic shadowed_storage_err, shadowed_update_err;
104 aes_reg_top u_reg (
105     .clk_i,
106     .rst_ni,
107     .rst_shadowed_ni,
108     .tl_i,
109     .tl_o,
110     .reg2hw,
111     .hw2reg,
112     .shadowed_storage_err_o(shadowed_storage_err),
113     .shadowed_update_err_o(shadowed_update_err),
114     .intg_err_o(intg_err_alert)
115 );
116
117 // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI
118 // Synchronize life cycle input
119 prim_lc_sync #(
120     .NumCopies (1)
121 ) u_prim_lc_sync (
122     .clk_i,
123     .rst_ni,
124     .lc_en_i ( lc_escalate_en_i ),
125     .lc_en_o ( {lc_escalate_en} )
126 );
127
128 // EDN Interface //
129 ///////////////
130
131 // Internally, we have up to two PRNGs that share the EDN
132 // interface for reseeding. Here, we just
133 // arbitrate the requests. Upsizing of the entropy to the
134 // correct width is performed inside the
135 // PRNGs.
136 // Reseed operations for the clearing PRNG are initiated by
137 // software. Reseed operations for the
138 // masking PRNG can also be automatically initiated.
139 assign edn_req_int = entropy_clearing_req |
entropy_masking_req;
140 // Only forward ACK to PRNG currently requesting entropy.
141 // Give higher priority to clearing PRNG.
142 assign entropy_clearing_ack = entropy_clearing_req &
edn_ack;
143 assign entropy_masking_ack = ~entropy_clearing_req &
entropy_masking_req & edn_ack;
144
145 // Upon escalation or detection of a fatal alert, an EDN
146 // request signal can be dropped before
147
148 // getting acknowledged. This is okay with respect to AES as
149 // the module will need to be reset
150 // anyway. However, to not leave EDN in a strange state, we
151 // hold the request until it's actually
152 // acknowledged.
153 assign edn_req = edn_req_int | edn_req_hold_q;
154 assign edn_req_hold_d = (edn_req_hold_q | edn_req) &
~edn_ack;
155 always_ff @(posedge clk_i or negedge rst_ni) begin :
edn_req_reg
156     if (!rst_ni) begin
157         edn_req_hold_q <= '0;
158     end else begin
159         edn_req_hold_q <= edn_req_hold_d;
160     end
161 end
162
163 // Synchronize EDN interface
164 prim_sync_reqack_data #(
165     .Width(EntropyWidth),
166     .DataSrc2Dst(1'b0),
167     .DataReg(1'b0)
168 ) u_prim_sync_reqack_data (
169     .clk_src_i ( clk_i ),
170     .rst_src_ni ( rst_ni ),
171     .clk_dst_i ( clk_edn_i ),
172     .rst_dst_ni ( rst_edn_ni ),
173     .req_chk_i ( 1'b1 ),
174     .src_req_i ( edn_req ),
175     .src_ack_o ( edn_ack ),
176     .dst_req_o ( edn_o.edn_req ),
177     .dst_ack_i ( edn_i.edn_ack ),
178     .data_i ( edn_i.edn_data ),
179     .data_o ( edn_data )
180 );
181 // We don't track whether the entropy is pre-FIPS or not
182 // inside AES.
183 assign unused_edn_fips = edn_i.edn_fips;
184
185 ///////////////
186 // Core //
187 ///////////////

```

```

180
181 // AES core
182 aes_core #(
183   .AES192Enable      ( AES192Enable      ),
184   .AESGCMEnable      ( AESGCMEnable      ),
185   .SecMasking        ( SecMasking        ),
186   .SecSBoxImpl       ( SecSBoxImpl       ),
187   .SecStartTriggerDelay ( SecStartTriggerDelay ),
188   .SecAllowForcingMasks ( SecAllowForcingMasks ),
189   .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
190   .EntropyWidth      ( EntropyWidth      ),
191   .RndCnstClearingLfsrSeed ( RndCnstClearingLfsrSeed ),
192   .RndCnstClearingLfsrPerm ( RndCnstClearingLfsrPerm ),
193   .RndCnstClearingSharePerm ( RndCnstClearingSharePerm ),
194   .RndCnstMaskingLfsrSeed ( RndCnstMaskingLfsrSeed ),
195   .RndCnstMaskingLfsrPerm ( RndCnstMaskingLfsrPerm )
196 ) u_aes_core (
197   .clk_i
198   .rst_ni
199   .rst_shadowed_ni
200   .entropy_clearing_req_o
201   .entropy_clearing_ack_i
202   .entropy_clearing_i
203   .entropy_masking_req_o
204   .entropy_masking_ack_i
205   .entropy_masking_i
206   .edn_data
207   .keymgr_key_i
208   .keymgr_key_i
209   .lc_escalate_en_i
210   .lc_escalate_en
211   .shadowed_storage_err_i
212   .shadowed_storage_err
213   .shadowed_update_err_i
214   .shadowed_update_err
215   .intg_err_alert_i
216   .intg_err_alert
217   .alert_recov_o
218   .alert_fatal_o
219   .alert_fatal_idx
220   .reg2hw
221   .reg2hw
222   .hw2reg
223   .hw2reg
224   );
225 assign idle_o =
226   pram_mubi_pkg::mubi4_bool_to_mubi(reg2hw.status.idle.q);
227 assign output_valid_o = reg2hw.status.output_valid.q;
228 assign input_ready_o = reg2hw.status.input_ready.q;
229
230 // Alerts //
231 // Alerts //
232 // Alerts //
233
234 logic [NumAlerts-1:0] alert_test;
235 assign alert_test[AlertRecovCtrlUpdateErrIdx] =
236   reg2hw.alert_test.recov_ctrl_update_err.q &
237   reg2hw.alert_test.recov_ctrl_update_err.qe;
238 assign alert_test[AlertFatalFaultIdx] =
239   reg2hw.alert_test.fatal_fault.q &
240   reg2hw.alert_test.fatal_fault.qe;
241
242 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
243   prim_alert_sender #(
244     .AsyncOn(AlertAsyncOn[i]),
245     .SkewCycles(AlertSkewCycles),
246     .IsFatal(i == AlertFatalFaultIdx)
247   ) u_prim_alert_sender (
248     .clk_i,
249     .rst_ni,
250     .alert_test_i ( alert_test[i] ),
251     .alert_req_i ( alert[i] ),
252     .alert_ack_o ( ),
253     .alert_state_o ( ),
254     .alert_rx_i ( alert_rx_i[i] ),
255     .alert_tx_o ( alert_tx_o[i] )
256   );
257 end
258
259 // All outputs should have a known value after reset
260 `ASSERT_KNOWN(TLODValidKnown, tl_o.d_valid)

```

```

258 `ASSERT_KNOWN(TLOAReadyKnown, tl_o.a_ready)
259 `ASSERT_KNOWN(IdleKnown, idle_o)
260 `ASSERT_KNOWN(OutputValidKnown, output_valid_o)
261 `ASSERT_KNOWN(InputReadyKnown, input_ready_o)
262 `ASSERT_KNOWN(EdnReqKnown, edn_o)
263 `ASSERT_KNOWN(AlertTxKnown, alert_tx_o)
264
265 // Alert assertions for sparse FSMs.
266 for (genvar i = 0; i < Sp2VWidth; i++) begin :
267   gen_control_fsm_svas
268     if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
269       gen_control_fsm_svas_p
270         `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesControlFsmCheck_A,
271           u_aes_core.u_aes_control.gen_fsm[i].gen_fsm_p.
272           u_aes_control_fsm_i.u_aes_control_fsm.u_state_regs,
273           alert_tx_o[1])
274         end else begin : gen_control_fsm_svas_n
275           `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesControlFsmCheck_A,
276             u_aes_core.u_aes_control.gen_fsm[i].gen_fsm_n.
277             u_aes_control_fsm_i.u_aes_control_fsm.u_state_regs,
278             alert_tx_o[1])
279           end
280         end
281       for (genvar i = 0; i < Sp2VWidth; i++) begin :
282         gen_ctr_fsm_svas
283           if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_ctr_fsm_svas_p
284             `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
285               u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm_p.
286               u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
287               alert_tx_o[1])
288             end else begin : gen_ctr_fsm_svas_n
289               `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
290                 u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm_n.
291                 u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
292                 alert_tx_o[1])
293             end
294           end
295         for (genvar i = 0; i < Sp2VWidth; i++) begin :
296           gen_cipher_control_fsm_svas
297             if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
298               gen_cipher_control_fsm_svas_p
299                 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
300                   u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm_p.
301                   u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
302                   alert_tx_o[1])
303                 end else begin : gen_cipher_control_fsm_svas_n
304                   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
305                     u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm_n.
306                     u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
307                     alert_tx_o[1])
308                   end
309                 if (AESGCMEnable) begin : gen_ghash_fsm_sva
310                   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
311                     u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
312                     alert_tx_o[1])
313                   end
314                 if (AESGCMEnable && SecMasking) begin : gen_ghash_onehot_sva
315                   for (genvar s = 0; s < 2; s++) begin :
316                     gen_ghash_onehot_add_in_sva
317                       `ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
318                         u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s].
319                         u_prim_onehot_check_add_in_sel,
320                         alert_tx_o[1])
321                       end
322                     `ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
323                       u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_prim_onehot_che
324                       ck_gf_mult1_in_sel,
325                       alert_tx_o[1])
326                     end
327                   // Alert assertions for reg_we onehot check
328                   `ASSERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
329                     u_reg, alert_tx_o[1])
330                 endmodule

```

A.29 lowRISC_opentitan/hw/ip/aes/rtl/aes_pkg.sv (746줄, 23,269B)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES package
6
7 package aes_pkg;
8
9 // If this parameter is set, fatal alerts clear all status and
  trigger bits to zero. By
10 // default, it's not set, i.e., no clearing is happening, in
  order to simplify debugging.
11 parameter bit ClearStatusOnFatalAlert = 1'b0;
12
13 // The initial key is always provided in two shares,
  independently whether the cipher core is
14 // masked or not.
15 parameter int unsigned NumSharesKey = 2;
16
17 // Software updates IV in chunks of 32 bits, the counter
  updates 16 bits at a time.
18 parameter int unsigned SliceSizeCtr = 16;
19
20 parameter int unsigned NumSlicesCtr = aes_reg_pkg::NumRegsIv *
  32 / SliceSizeCtr;
21 parameter int unsigned SliceIdxWidth =
  prim_util_pkg::vbits(NumSlicesCtr);
22 // In GCM, the counter performs inc32() instead of inc128(),
  i.e., the counter wraps at 32 bits.
23 parameter int unsigned SliceIdxMaxInc32 = 32 / SliceSizeCtr -
  1;
24
25 // Widths of signals carrying pseudo-random data for clearing
26 parameter int unsigned WidthPRDClearing = 64;
27 parameter int unsigned NumChunksPRDClearing128 = 128 /
  WidthPRDClearing;
28 parameter int unsigned NumChunksPRDClearing256 = 256 /
  WidthPRDClearing;
29
30 // Widths of signals carrying pseudo-random data for masking
31 parameter int unsigned WidthPRDSBox = 8; // Number PRD
  bits per S-Box. This includes the
32 // 8 bits for
33 // masked
  Canright S-Box implementations.
34 parameter int unsigned WidthPRDDData = 16 * WidthPRDSBox; //
  16 S-Boxes for the data path
35 parameter int unsigned WidthPRDKey = 4 * WidthPRDSBox; //
  4 S-Boxes for the key expand
36 parameter int unsigned WidthPRDMasking = WidthPRDDData +
  WidthPRDKey;
37
38 // Clearing PRNG default LFSR seed and permutation
39 // These LFSR parameters have been generated with
40 // $ util/design/gen-lfsr-seed.py --width 64 --seed 31468618
  --prefix "Clearing"
41 parameter int ClearingLfsrWidth = 64;
42 typedef logic [ClearingLfsrWidth-1:0] clearing_lfsr_seed_t;
43 typedef logic [ClearingLfsrWidth-1:0]
  [$clog2(ClearingLfsrWidth)-1:0] clearing_lfsr_perm_t;
44 parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeedDefault
  = 64'h32d580f74f1713a;
45 parameter clearing_lfsr_perm_t RndCnstClearingLfsrPermDefault
  = {
46     128'hb33dfdc81deb6292c21f8a3102585067,
47 };
256'h9c2f4be1bbe937b4b7c9d7f4e57568d99c8ae291a899143e0d8459d31b14322
  3
48 };
49 // A second permutation is needed for the second share.
50
51 parameter clearing_lfsr_perm_t RndCnstClearingSharePermDefault =
  {
52     128'hf66fd61b27847edc2286706fb3a2e900,
53 };
256'h9736b95ac3f3b5205caf8dc536aad73605d393c8dd94476e830e97891d4828
  d0
54 };
55 // Masking PRNG default state seed and output permutation
56 // The output width is 160 bits (WidthPRDMasking =
  WidthPRDSBox * (16 + 4)).
57 // These LFSR parameters have been generated with
58 // $ util/design/gen-lfsr-seed.py --width 160 --seed 31468618
  --prefix "Masking"
59 parameter int MaskingLfsrWidth = 160; // = WidthPRDMasking =
  WidthPRDSBox * (16 + 4)
60 typedef logic [MaskingLfsrWidth-1:0]
  [$clog2(MaskingLfsrWidth)-1:0] masking_lfsr_perm_t;
61 parameter masking_lfsr_perm_t RndCnstMaskingLfsrPermDefault =
  {

```

```

62
256'h17261943423e4c5c03872194050c7e5f8497081d9666d406f4b60647330346
  9,
63
256'h8e7c721c8832471f59919e0b128f067b25622768462e554d8970815d490d7f4
  4,
64
256'h048c867d907a239b20220f6c79071a852d76485452189f14091b1e744e39673
  7,
65
256'h4f785b772b352f6550613c58130a8b104a3f28019c9a380233956b00563a512
  c,
66
256'h808d419d63982a16995e0e3b57826a36718a9329452492533d83115a75316e1
  5
67 };
68 // The state width is 177 bits (Bivium) but the primitive
  expects a 288-bit seed (Trivium).
69 // These LFSR parameters have been generated with
70 // $ util/design/gen-lfsr-seed.py --width 288 --seed 31468618
  --prefix "Masking"
71 parameter int MaskingPrngStateWidth = 288;
72 typedef logic [MaskingPrngStateWidth-1:0] masking_lfsr_seed_t;
73 parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeedDefault =
  {
74     32'h758a4420,
75
256'h31e1c461_6ea343ec_153282a3_0c132b57_23c5a4cf_4743b3c7_c32d580f_
  74f1713a
76 };
77
78 typedef enum integer {
79     SBoxImplLut, // Unmasked LUT-based S-Box
80     SBoxImplCanright, // Unmasked Canright S-Box,
  see aes_sbox_canright.sv
81     SBoxImplCanrightMasked, // First-order masked
  Canright S-Box
82 } // see
  aes_sbox_canright_masked.sv
83     SBoxImplCanrightMaskedNoreuse, // First-order masked
  Canright S-Box without mask reuse,
84 } // see
  aes_sbox_canright_masked_noreuse.sv
85     SBoxImplDom // First-order masked S-Box
  using domain-oriented masking,
86 } // see
  aes_sbox_canright_dom.sv
87 } sbox_impl_e;
88
89 // GF(2^128) irreducible, field-generating polynomial for AES-
  GCM
90 // See Section "6.3 Multiplication Operation on Blocks" of
  https://nvlpubs.nist.gov/nistpubs/Legacy/SP/
  nistspecialpublication800-38d.pdf on Page 11:
  "Let R be the bit string 11100001 || 0^120."
91 // And further on Page 12:
  "The reduction modulus is the polynomial of degree 128
  that corresponds to R || 1"
92 // Or in other words: x^128 + x^7 + x^2 + x + 1
93 // The MSB gets clipped off below.
94 parameter int unsigned GCMDegree = 128;
95
96 parameter bit [GCMDegree-1:0] GCMIPoly = GCMDegree'(1'b1) << 7 |
  GCMDegree'(1'b1) << 2 |
  GCMDegree'(1'b1) << 1 |
  GCMDegree'(1'b1) <<
  0;
97
98 // Parameters used for controlgroups in the coverage
99 parameter int AES_OP_WIDTH = 2;
100 parameter int AES_MODE_WIDTH = 6;
101 parameter int AES_KEYLEN_WIDTH = 3;
102 parameter int AES_PRNGRESEEDRATE_WIDTH = 3;
103 parameter int AES_GCMPhase_WIDTH = 6;
104
105 // SEC_CM: MAIN.CONFIG.SPARSE
106 typedef enum logic [AES_OP_WIDTH-1:0] {
107     AES_ENC = 2'b01,
108     AES_DEC = 2'b10
109 } aes_op_e;
110
111 // SEC_CM: MAIN.CONFIG.SPARSE
112 typedef enum logic [AES_MODE_WIDTH-1:0] {
113     AES_ECB = 6'b00_0001,
114     AES_CBC = 6'b00_0010,
115     AES_CFB = 6'b00_0100,
116     AES_OFB = 6'b00_1000,
117     AES_CTR = 6'b01_0000,
118     AES_GCM = 6'b10_0000,
119     AES_NONE = 6'b11_1111
120 } aes_mode_e;
121
122 // SEC_CM: MAIN.CONFIG.SPARSE
123 typedef enum logic [AES_OP_WIDTH-1:0] {
124     CIPH_FWD = 2'b01,
125     CIPH_INV = 2'b10
126 } ciph_op_e;
127
128 // SEC_CM: MAIN.CONFIG.SPARSE

```

```

134 typedef enum logic [AES_KEYLEN_WIDTH-1:0] {
135     AES_128 = 3'b001,
136     AES_192 = 3'b010,
137     AES_256 = 3'b100
138 } key_len_e;
139
140 // SEC_CM: MAIN.CONFIG.SPARSE
141 typedef enum logic [AES_PRNGRESEEDRATE_WIDTH-1:0] {
142     PER_1 = 3'b001,
143     PER_64 = 3'b010,
144     PER_8K = 3'b100
145 } prs_rate_e;
146 parameter int unsigned BlockCtrWidth = 13;
147
148 // SEC_CM: GCM.CONFIG.SPARSE
149 typedef enum logic [AES_GCM_PHASE_WIDTH-1:0] {
150     GCM_INIT = 6'b00_0001,
151     GCM_RESTORE = 6'b00_0010,
152     GCM_AAD = 6'b00_0100,
153     GCM_TEXT = 6'b00_1000,
154     GCM_SAVE = 6'b01_0000,
155     GCM_TAG = 6'b10_0000
156 } gcm_phase_e;
157
158 typedef struct packed {
159     logic [31:7] unused;
160     logic alert_fatal_fault;
161     logic alert_recov_ctrl_update_err;
162     logic input_ready;
163     logic output_valid;
164     logic output_lost;
165     logic stall;
166     logic idle;
167 } status_t;
168
169 typedef struct packed {
170     logic recov_ctrl_update_err;
171     logic fatal_fault;
172 } alert_test_t;
173
174 // Sparse state encodings
175
176 // Encoding generated with:
177 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 8 -n 6 \
178 // -s 31468618 --language=sv
179 //
180 // Hamming distance histogram:
181 //
182 // 0: --
183 // 1: --
184 // 2: --
185 // 3: ||||| (57.14%)
186 // 4: ||||| (42.86%)
187 // 5: --
188 // 6: --
189 //
190 // Minimum Hamming distance: 3
191 // Maximum Hamming distance: 4
192 // Minimum Hamming weight: 1
193 // Maximum Hamming weight: 5
194 //
195 localparam int CipherCtrlStateWidth = 6;
196 typedef enum logic [CipherCtrlStateWidth-1:0] {
197     CIPHER_CTRL_IDLE = 6'b001001,
198     CIPHER_CTRL_INIT = 6'b100011,
199     CIPHER_CTRL_ROUND = 6'b111101,
200     CIPHER_CTRL_FINISH = 6'b010000,
201     CIPHER_CTRL_PRNG_RESEED = 6'b100100,
202     CIPHER_CTRL_CLEAR_S = 6'b111010,
203     CIPHER_CTRL_CLEAR_KD = 6'b001110,
204     CIPHER_CTRL_ERROR = 6'b010111
205 } aes_cipher_ctrl_e;
206
207 // $ ./sparse-fsm-encode.py -d 3 -m 3 -n 5 \
208 // -s 31468618 --language=sv
209 //
210 // Hamming distance histogram:
211 //
212 // 0: --
213 // 1: --
214 // 2: --
215 // 3: ||||| (66.67%)
216 // 4: ||||| (33.33%)
217 // 5: --
218 //
219 // Minimum Hamming distance: 3
220 // Maximum Hamming distance: 4
221 //
222 localparam int CtrStateWidth = 5;
223 typedef enum logic [CtrStateWidth-1:0] {
224     CTR_IDLE = 5'b01110,
225     CTR_INCR = 5'b11000,
226     CTR_ERROR = 5'b00001
227 } aes_ctr_e;
228
229 // Encoding generated with:
230 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 8 -n 6 \
231 // -s 31468618 --language=sv
232 //
233 // Hamming distance histogram:
234 //
235 // 0: --
236 // 1: --
237 // 2: --
238 // 3: ||||| (57.14%)
239 // 4: ||||| (42.86%)
240 // 5: --
241 // 6: --
242 //
243 // Minimum Hamming distance: 3
244 // Maximum Hamming distance: 4
245 // Minimum Hamming weight: 1
246 // Maximum Hamming weight: 5
247 //
248 localparam int CtrlStateWidth = 6;
249 typedef enum logic [CtrlStateWidth-1:0] {
250     CTRL_IDLE = 6'b001001,
251     CTRL_LOAD = 6'b100011,
252     CTRL_GHASH_READY = 6'b111101,
253     CTRL_PRNG_RESEED = 6'b010000,
254     CTRL_FINISH = 6'b100100,
255     CTRL_CLEAR_I = 6'b111010,
256     CTRL_CLEAR_CO = 6'b001110,
257     CTRL_ERROR = 6'b010111
258 } aes_ctrl_e;
259
260 // Encoding generated with:
261 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 9 -n 7 \
262 // -s 31468618 --language=sv
263 //
264 // Hamming distance histogram:
265 //
266 // 0: --
267 // 1: --
268 // 2: --
269 // 3: ||||| (38.89%)
270 // 4: ||||| (41.67%)
271 // 5: ||||| (16.67%)
272 // 6: | (2.78%)
273 // 7: --
274 //
275 // Minimum Hamming distance: 3
276 // Maximum Hamming distance: 6
277 // Minimum Hamming weight: 1
278 // Maximum Hamming weight: 5
279 //
280 localparam int GhashStateWidth = 7;
281 typedef enum logic [GhashStateWidth-1:0] {
282     GHASH_IDLE = 7'b1100001,
283     GHASH_MULT = 7'b0010001,
284     GHASH_ADD_S = 7'b0000110,
285     GHASH_OUT = 7'b0110111,
286     GHASH_ERROR = 7'b0111010,
287     GHASH_MASKED_INIT = 7'b1111100,
288     GHASH_MASKED_ADD_STATE_SHARES = 7'b0101101,
289     GHASH_MASKED_ADD_CORR = 7'b0001000,
290     GHASH_MASKED_SETTLE = 7'b1001111
291 } aes_ghash_e;
292
293 // Generic, sparse mux selector encodings
294
295 // Encoding generated with:
296 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 2 -n 3 \
297 // -s 31468618 --language=sv
298 //
299 // Hamming distance histogram:
300 //
301 // 0: --
302 // 1: --
303 // 2: --
304 // 3: ||||| (100.00%)
305 //
306 // Minimum Hamming distance: 3
307 // Maximum Hamming distance: 3
308 // Minimum Hamming weight: 1
309 // Maximum Hamming weight: 2
310 //
311 parameter int Mux2SelWidth = 3;
312 typedef enum logic [Mux2SelWidth-1:0] {
313     MUX2_SEL_0 = 3'b011,
314     MUX2_SEL_1 = 3'b100
315 } mux2_sel_e;
316
317 // Encoding generated with:
318 // $ ./sparse-fsm-encode.py -d 3 -m 3 -n 5 \
319 // -s 31468618 --language=sv
320 //
321 // Hamming distance histogram:
322 //
323 // 0: --
324 // 1: --
325 // 2: --
326 // 3: ||||| (66.67%)
327 // 4: ||||| (33.33%)
328 // 5: --
329 //
330 // Minimum Hamming distance: 3
331 // Maximum Hamming distance: 4
332 //
333 parameter int Mux3SelWidth = 5;
334 typedef enum logic [Mux3SelWidth-1:0] {
335     MUX3_SEL_0 = 5'b01110,

```

```

336 MUX3_SEL_1 = 5'b11000,
337 MUX3_SEL_2 = 5'b00001
338 } mux3_sel_e;
339
340 // Encoding generated with:
341 // $ ./sparse-fsm-encode.py -d 3 -m 4 -n 5 \
342 // -s 31468618 --language=sv
343 //
344 // Hamming distance histogram:
345 //
346 // 0: --
347 // 1: --
348 // 2: --
349 // 3: ||| (66.67%)
350 // 4: ||| (33.33%)
351 // 5: --
352 //
353 // Minimum Hamming distance: 3
354 // Maximum Hamming distance: 4
355 //
356 parameter int Mux4SelWidth = 5;
357 typedef enum logic [Mux4SelWidth-1:0] {
358 MUX4_SEL_0 = 5'b01110,
359 MUX4_SEL_1 = 5'b11000,
360 MUX4_SEL_2 = 5'b00001,
361 MUX4_SEL_3 = 5'b10111
362 } mux4_sel_e;
363
364 // Encoding generated with:
365 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 5 -n 6 \
366 // -s 31468618 --language=sv
367 //
368 // Hamming distance histogram:
369 //
370 // 0: --
371 // 1: --
372 // 2: --
373 // 3: ||| (50.00%)
374 // 4: ||| (40.00%)
375 // 5: ||| (10.00%)
376 // 6: --
377 //
378 // Minimum Hamming distance: 3
379 // Maximum Hamming distance: 5
380 // Minimum Hamming weight: 1
381 // Maximum Hamming weight: 5
382 //
383 localparam int Mux5SelWidth = 6;
384 typedef enum logic [Mux5SelWidth-1:0] {
385 MUX5_SEL_0 = 6'b110000,
386 MUX5_SEL_1 = 6'b001000,
387 MUX5_SEL_2 = 6'b000011,
388 MUX5_SEL_3 = 6'b011101,
389 MUX5_SEL_4 = 6'b111110
390 } mux5_sel_e;
391
392
393 // $ ./sparse-fsm-encode.py -d 3 -m 6 -n 6 \
394 // -s 31468618 --language=sv
395 //
396 // Hamming distance histogram:
397 //
398 // 0: --
399 // 1: --
400 // 2: --
401 // 3: ||| (53.33%)
402 // 4: ||| (40.00%)
403 // 5: || (6.67%)
404 // 6: --
405 //
406 // Minimum Hamming distance: 3
407 // Maximum Hamming distance: 5
408 //
409 parameter int Mux6SelWidth = 6;
410 typedef enum logic [Mux6SelWidth-1:0] {
411 MUX6_SEL_0 = 6'b011101,
412 MUX6_SEL_1 = 6'b110000,
413 MUX6_SEL_2 = 6'b001000,
414 MUX6_SEL_3 = 6'b000011,
415 MUX6_SEL_4 = 6'b111110,
416 MUX6_SEL_5 = 6'b100101
417 } mux6_sel_e;
418
419 // Mux selector signal types. These use the generic types
420 // defined above.
421 parameter int DIPSelNum = 2;
422 parameter int DIPSelWidth = Mux2SelWidth;
423 typedef enum logic [DIPSelWidth-1:0] {
424 DIP_DATA_IN = MUX2_SEL_0,
425 DIP_CLEAR = MUX2_SEL_1
426 } dip_sel_e;
427
428 parameter int SISElNum = 2;
429 parameter int SISElWidth = Mux2SelWidth;
430 typedef enum logic [SISElWidth-1:0] {
431 SI_ZERO = MUX2_SEL_0,
432 SI_DATA = MUX2_SEL_1
433 } si_sel_e;
434
435 parameter int AddSISElNum = 2;
436 parameter int AddSISElWidth = Mux2SelWidth;
437 typedef enum logic [AddSISElWidth-1:0] {
438 ADD_SI_ZERO = MUX2_SEL_0,
439 ADD_SI_IV = MUX2_SEL_1
440 } add_si_sel_e;
441
442 parameter int StateSelNum = 3;
443 parameter int StateSelWidth = Mux3SelWidth;
444 typedef enum logic [StateSelWidth-1:0] {
445 STATE_INIT = MUX3_SEL_0,
446 STATE_ROUND = MUX3_SEL_1,
447 STATE_CLEAR = MUX3_SEL_2
448 } state_sel_e;
449
450 parameter int AddRKSelNum = 3;
451 parameter int AddRKSelWidth = Mux3SelWidth;
452 typedef enum logic [AddRKSelWidth-1:0] {
453 ADD_RK_INIT = MUX3_SEL_0,
454 ADD_RK_ROUND = MUX3_SEL_1,
455 ADD_RK_FINAL = MUX3_SEL_2
456 } add_rk_sel_e;
457
458 parameter int KeyInitSelNum = 3;
459 parameter int KeyInitSelWidth = Mux3SelWidth;
460 typedef enum logic [KeyInitSelWidth-1:0] {
461 KEY_INIT_INPUT = MUX3_SEL_0,
462 KEY_INIT_KEYMGR = MUX3_SEL_1,
463 KEY_INIT_CLEAR = MUX3_SEL_2
464 } key_init_sel_e;
465
466 parameter int IVSelNum = 6;
467 parameter int IVSelWidth = Mux6SelWidth;
468 typedef enum logic [IVSelWidth-1:0] {
469 IV_INPUT = MUX6_SEL_0,
470 IV_DATA_OUT = MUX6_SEL_1,
471 IV_DATA_OUT_RAW = MUX6_SEL_2,
472 IV_DATA_IN_PREV = MUX6_SEL_3,
473 IV_CTR = MUX6_SEL_4,
474 IV_CLEAR = MUX6_SEL_5
475 } iv_sel_e;
476
477 parameter int KeyFullSelNum = 4;
478 parameter int KeyFullSelWidth = Mux4SelWidth;
479 typedef enum logic [KeyFullSelWidth-1:0] {
480 KEY_FULL_ENC_INIT = MUX4_SEL_0,
481 KEY_FULL_DEC_INIT = MUX4_SEL_1,
482 KEY_FULL_ROUND = MUX4_SEL_2,
483 KEY_FULL_CLEAR = MUX4_SEL_3
484 } key_full_sel_e;
485
486 parameter int KeyDecSelNum = 2;
487 parameter int KeyDecSelWidth = Mux2SelWidth;
488 typedef enum logic [KeyDecSelWidth-1:0] {
489 KEY_DEC_EXPAND = MUX2_SEL_0,
490 KEY_DEC_CLEAR = MUX2_SEL_1
491 } key_dec_sel_e;
492
493 parameter int KeyWordsSelNum = 4;
494 parameter int KeyWordsSelWidth = Mux4SelWidth;
495 typedef enum logic [KeyWordsSelWidth-1:0] {
496 KEY_WORDS_0123 = MUX4_SEL_0,
497 KEY_WORDS_2345 = MUX4_SEL_1,
498 KEY_WORDS_4567 = MUX4_SEL_2,
499 KEY_WORDS_ZERO = MUX4_SEL_3
500 } key_words_sel_e;
501
502 parameter int RoundKeySelNum = 2;
503 parameter int RoundKeySelWidth = Mux2SelWidth;
504 typedef enum logic [RoundKeySelWidth-1:0] {
505 ROUND_KEY_DIRECT = MUX2_SEL_0,
506 ROUND_KEY_MIXED = MUX2_SEL_1
507 } round_key_sel_e;
508
509 parameter int AddS0SelNum = 3;
510 parameter int AddS0SelWidth = Mux3SelWidth;
511 typedef enum logic [AddS0SelWidth-1:0] {
512 ADD_S0_ZERO = MUX3_SEL_0,
513 ADD_S0_IV = MUX3_SEL_1,
514 ADD_S0_DIP = MUX3_SEL_2
515 } add_so_sel_e;
516
517 parameter int GHashInSelNum = 2;
518 parameter int GHashInSelWidth = Mux2SelWidth;
519 typedef enum logic [GHashInSelWidth-1:0] {
520 GHASH_IN_DATA_IN_PREV = MUX2_SEL_0,
521 GHASH_IN_DATA_OUT = MUX2_SEL_1
522 } ghash_in_sel_e;
523
524 parameter int GHashAddInSelWidth = 3;
525 typedef enum logic [GHashAddInSelWidth-1:0] {
526 ADD_IN_GHASH_IN = 3'b001,
527 ADD_IN_CORR_A = 3'b010,
528 ADD_IN_CORR_B = 3'b100,
529 ADD_IN_ZERO = 3'b000
530 } ghash_add_in_sel_e;
531
532 parameter int GHashStateSelNum = 5;
533 parameter int GHashStateSelWidth = Mux5SelWidth;
534 typedef enum logic [GHashStateSelWidth-1:0] {
535 GHASH_STATE_RESTORE = MUX5_SEL_0,
536 GHASH_STATE_INIT = MUX5_SEL_1,

```

```

537 GHASH_STATE_ADD = MUX5_SEL_2,
538 GHASH_STATE_ADD_S = MUX5_SEL_3,
539 GHASH_STATE_MULT = MUX5_SEL_4
540 } ghash_state_sel_e;
541
542 parameter int GFMultInSelWidth = 3;
543 typedef enum logic [GFMultInSelWidth-1:0] {
544     MULT_IN_STATE0 = 3'b001,
545     MULT_IN_STATE1 = 3'b010,
546     MULT_IN_S1 = 3'b100,
547     MULT_IN_ZERO = 3'b000
548 } gf_mult_in_sel_e;
549
550 parameter int DataOutSelNum = 2;
551 parameter int DataOutSelWidth = Mux2SelWidth;
552 typedef enum logic [DataOutSelWidth-1:0] {
553     DATA_OUT_CIPHER = MUX2_SEL_0,
554     DATA_OUT_GHASH = MUX2_SEL_1
555 } data_out_sel_e;
556
557 // Sparse two-value signal type sp2v_e
558 parameter int Sp2VNum = 2;
559 parameter int Sp2VWidth = Mux2SelWidth;
560 typedef enum logic [Sp2VWidth-1:0] {
561     SP2V_HIGH = MUX2_SEL_0,
562     SP2V_LOW = MUX2_SEL_1
563 } sp2v_e;
564
565 typedef logic [Sp2VWidth-1:0] sp2v_logic_t;
566 parameter sp2v_logic_t SP2V_LOGIC_HIGH = {SP2V_HIGH};
567
568 // Control register type
569 typedef struct packed {
570     logic manual_operation;
571     prs_rate_e prng_reseed_rate;
572     logic sideload;
573     key_len_e key_len;
574     aes_mode_e mode;
575     aes_op_e operation;
576 } ctrl_reg_t;
577
578 parameter ctrl_reg_t CTRL_RESET = '{
579     manual_operation:
aes_reg_pkg::AES_CTRL_SHADOWED_MANUAL_OPERATION_RESVAL,
580     prng_reseed_rate:
prs_rate_e'(aes_reg_pkg::AES_CTRL_SHADOWED_PRNG_RESEED_RATE_RESVAL),
581     sideload:
aes_reg_pkg::AES_CTRL_SHADOWED_SIDELOAD_RESVAL,
582     key_len:
key_len_e'(aes_reg_pkg::AES_CTRL_SHADOWED_KEY_LEN_RESVAL),
583     mode:
aes_mode_e'(aes_reg_pkg::AES_CTRL_SHADOWED_MODE_RESVAL),
584     operation:
aes_op_e'(aes_reg_pkg::AES_CTRL_SHADOWED_OPERATION_RESVAL)
585 };
586
587 // GCM control register type
588 typedef struct packed {
589     logic [4:0] num_valid_bytes;
590     gcm_phase_e phase;
591 } ctrl_gcm_reg_t;
592
593 // Multiplication by {02} (i.e. x) on GF(2^8)
594 // with field generating polynomial {01}{1b} (9'h11b)
595 // Sometimes also denoted by xtime().
596 function automatic logic [7:0] aes_mul2(logic [7:0] in);
597     logic [7:0] out;
598     out[7] = in[6];
599     out[6] = in[5];
600     out[5] = in[4];
601     out[4] = in[3] ^ in[7];
602     out[3] = in[2] ^ in[7];
603     out[2] = in[1];
604     out[1] = in[0] ^ in[7];
605     out[0] = in[7];
606     return out;
607 endfunction
608
609 // Multiplication by {04} (i.e. x^2) on GF(2^8)
610 // with field generating polynomial {01}{1b} (9'h11b)
611 function automatic logic [7:0] aes_mul4(logic [7:0] in);
612     return aes_mul2(aes_mul2(in));
613 endfunction
614
615 // Division by {02} (i.e. x) on GF(2^8)
616 // with field generating polynomial {01}{1b} (9'h11b)
617 // This is the inverse of aes_mul2() or xtime().
618 function automatic logic [7:0] aes_div2(logic [7:0] in);
619     logic [7:0] out;
620     out[7] = in[0];
621     out[6] = in[7];
622     out[5] = in[6];
623     out[4] = in[5];
624     out[3] = in[4] ^ in[0];
625     out[2] = in[3] ^ in[0];
626     out[1] = in[2];
627     out[0] = in[1] ^ in[0];
628     return out;
629 endfunction
630
631 // Circular byte shift to the left

```

```

632 function automatic logic [31:0] aes_circ_byte_shift(logic
[31:0] in, logic [1:0] shift);
633     logic [31:0] out;
634     logic [31:0] s;
635     s = {30'b0, shift};
636     out = {in[8*((7-s)%4) +: 8], in[8*((6-s)%4) +: 8],
637         in[8*((5-s)%4) +: 8], in[8*((4-s)%4) +: 8]};
638     return out;
639 endfunction
640
641 // Transpose state matrix
642 function automatic logic [3:0][3:0][7:0] aes_transpose(logic
[3:0][3:0][7:0] in);
643     logic [3:0][3:0][7:0] transpose;
644     transpose = '0;
645     for (int j = 0; j < 4; j++) begin
646         for (int i = 0; i < 4; i++) begin
647             transpose[i][j] = in[j][i];
648         end
649     end
650     return transpose;
651 endfunction
652
653 // Convert AES byte state matrix to internal GHASH bit vector
representation.
654 function automatic logic [127:0] aes_state_to_ghash_vec(logic
[3:0][3:0][7:0] in);
655     logic [127:0] out;
656     logic [15:0][7:0] byte_vec;
657     for (int i = 0; i < 4; i++) begin
658         for (int j = 0; j < 4; j++) begin
659             byte_vec[15 - 4*i - j] = in[j][i];
660         end
661     end
662     out = byte_vec;
663     return out;
664 endfunction
665
666 // Convert internal GHASH bit vector representation to simple
bit vector.
667 function automatic logic [127:0]
aes_ghash_reverse_bit_order(logic [127:0] in);
668     logic [127:0] out;
669     for (int i = 0; i < 128; i++) begin
670         out[i] = in[127-i];
671     end
672     return out;
673 endfunction
674
675 // Extract single column from state matrix
676 function automatic logic [3:0][7:0] aes_col_get(logic [3:0]
[3:0][7:0] in, logic [1:0] idx);
677     logic [3:0][7:0] out;
678     for (int i = 0; i < 4; i++) begin
679         out[i] = in[i][idx];
680     end
681     return out;
682 endfunction
683
684 // Matrix-vector multiplication in GF(2^8): c = A * b
685 function automatic logic [7:0] aes_mvnm(
686     logic [7:0] vec_b,
687     logic [7:0] mat_a [8]
688 );
689     logic [7:0] vec_c;
690     vec_c = '0;
691     for (int i = 0; i < 8; i++) begin
692         for (int j = 0; j < 8; j++) begin
693             vec_c[i] = vec_c[i] ^ (mat_a[j][i] & vec_b[7-j]);
694         end
695     end
696     return vec_c;
697 endfunction
698
699 // Rotate integer indices
700 function automatic integer aes_rot_int(integer in, integer
num);
701     integer out;
702     if (in == 0) begin
703         out = num - 1;
704     end else begin
705         out = in - 1;
706     end
707     return out;
708 endfunction
709
710 // Function for extracting LSBs of the per-S-Box pseudo-random
data (PRD) from the output of the
711 // masking PRNG.
712 //
713 // The masking PRNG is used for generating both the PRD for
the S-Boxes/SubBytes operation as
714 // well as for the input data masks. When using any of the
masked Canright S-Box implementations,
715 // it is important that the SubBytes input masks (generated by
the PRNG in Round X-1) and the
716 // SubBytes output masks (generated by the PRNG in Round X)
are independent. This can be achieved
717 // by using e.g. an unrolled Bivium stream cipher primitive

```

```

inside the PRNG. Since the input data
718 // masks become the SubBytes input masks in the first round,
we select the same 8 bit lanes for the
719 // input data masks which are also used to form the SubBytes
output mask for the masked Canright
720 // S-Box implementations, i.e., the 8 LSBs of the per S-Box
PRD. In particular, we have:
721 //
722 // prng_output = { prd_key_expand, ... , sb_prd[4],
sb_out_mask[4], sb_prd[0], sb_out_mask[0] }
723 //
724 // Where sb_out_mask[x] contains the SubBytes output mask for
byte x (when using a masked
725 // Canright S-Box implementation) and sb_prd[x] contains
additional PRD consumed by SubBytes for
726 // byte x.
727 //
728 // When using a masked S-Box implementation other than
Canright, we still select the 8 LSBs of
729 // the per-S-Box PRD to form the input data mask of the
corresponding byte. We do this to
730 // distribute the input data masks over all output bits the
masking PRNG.
731 //
732 // For one row of the state matrix, extract the 8 LSBs of the
per-S-Box PRD from the PRNG output.
733 // These bits are used as:
734 // - input data masks, and
735 // - SubBytes output mask when using a masked Canright S-Box
implementation.
736 function automatic logic [3:0][7:0] aes_prd_get_lsbs(
737     logic [(4*WidthPRDSBox)-1:0] in
738 );
739     logic [3:0][7:0] prd_lsbs;
740     for (int i = 0; i < 4; i++) begin
741         prd_lsbs[i] = in[i*WidthPRDSBox +: 8];
742     end
743     return prd_lsbs;
744 endfunction
745
746 endpackage

```

A.30 secworks_aes/src/rtl/aes_core.v (342줄, 10,557B)

```

1 //
=====
2 //
3 // aes_core.v
4 // -----
5 // The AES core. This core supports key size of 128, and 256
bits.
6 // Most of the functionality is within the submodules.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, 2014, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the
following
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following
disclaimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following
disclaimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
OTHERWISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //

```

```

=====
39
40 `default_nettype none
41
42 module aes_core(
43     input wire          clk,
44     input wire          reset_n,
45
46     input wire          encdec,
47     input wire          init,
48     input wire          next,
49     output wire         ready,
50
51     input wire [255 : 0] key,
52     input wire          keylen,
53
54     input wire [127 : 0] block,
55     output wire [127 : 0] result,
56     output wire         result_valid
57 );
58
59
60
61
62 //-----
63 // Internal constant and parameter definitions.
64 //-----
65 localparam CTRL_IDLE = 2'h0;
66 localparam CTRL_INIT = 2'h1;
67 localparam CTRL_NEXT = 2'h2;
68
69
70 //-----
71 // Registers including update variables and write enable.
72 //-----
73 reg [1 : 0] aes_core_ctrl_reg;
74 reg [1 : 0] aes_core_ctrl_new;
75 reg        aes_core_ctrl_we;
76
77 reg        result_valid_reg;
78 reg        result_valid_new;
79 reg        result_valid_we;
80
81 reg        ready_reg;
82 reg        ready_new;
83 reg        ready_we;
84
85
86 //-----
87 // Wires.
88 //-----
89 reg        init_state;
90
91 wire [127 : 0] round_key;
92 wire          key_ready;
93
94 reg          enc_next;
95 wire [3 : 0] enc_round_nr;
96 wire [127 : 0] enc_new_block;
97 wire          enc_ready;
98 wire [31 : 0] enc_sboxw;
99
100 reg          dec_next;
101 wire [3 : 0] dec_round_nr;
102 wire [127 : 0] dec_new_block;
103 wire          dec_ready;
104
105 reg [127 : 0] muxed_new_block;
106 reg [3 : 0] muxed_round_nr;
107 reg          muxed_ready;
108
109 wire [31 : 0] keymem_sboxw;
110
111 /* verilator lint_off UNOPTFLAT */
112 reg [31 : 0] muxed_sboxw;
113 wire [31 : 0] new_sboxw;
114 /* verilator lint_on UNOPTFLAT */
115
116
117 //-----
118 // Instantiations.
119 //-----
120
121 aes_encipher_block enc_block(
122     .clk(clk),
123     .reset_n(reset_n),
124     .next(enc_next),
125     .keylen(keylen),
126     .round(enc_round_nr),
127     .round_key(round_key),
128
129

```

```

130         .sboxw(enc_sboxw),
131         .new_sboxw(new_sboxw),
132
133         .block(block),
134         .new_block(enc_new_block),
135         .ready(enc_ready)
136     );
137
138
139     aes_decipher_block dec_block(
140         .clk(clk),
141         .reset_n(reset_n),
142
143         .next(dec_next),
144
145         .keylen(keylen),
146         .round(dec_round_nr),
147         .round_key(round_key),
148
149         .block(block),
150         .new_block(dec_new_block),
151         .ready(dec_ready)
152     );
153
154
155     aes_key_mem keymem(
156         .clk(clk),
157         .reset_n(reset_n),
158
159         .key(key),
160         .keylen(keylen),
161         .init(init),
162
163         .round(muxed_round_nr),
164         .round_key(round_key),
165         .ready(key_ready),
166
167         .sboxw(keymem_sboxw),
168         .new_sboxw(new_sboxw)
169     );
170
171
172     aes_sbox
173     sbox_inst(.sboxw(muxed_sboxw), .new_sboxw(new_sboxw));
174
175     //-----
176     // Concurrent connectivity for ports etc.
177     //-----
178
179     assign ready      = ready_reg;
180     assign result     = muxed_new_block;
181     assign result_valid = result_valid_reg;
182
183     //-----
184
185     // reg_update
186     //
187     // Update functionality for all registers in the core.
188     // All registers are positive edge triggered with
189     // asynchronous
190     // active low reset. All registers have write enable.
191     //-----
192
193     always @ (posedge clk or negedge reset_n)
194     begin: reg_update
195         if (!reset_n)
196         begin
197             result_valid_reg <= 1'b0;
198             ready_reg <= 1'b1;
199             aes_core_ctrl_reg <= CTRL_IDLE;
200         end
201         else
202         begin
203             if (result_valid_we)
204                 result_valid_reg <= result_valid_new;
205
206             if (ready_we)
207                 ready_reg <= ready_new;
208
209             if (aes_core_ctrl_we)
210                 aes_core_ctrl_reg <= aes_core_ctrl_new;
211         end
212     end // reg_update
213
214     //-----
215
216     // sbox_mux
217     //
218     // Controls which of the encipher datapath or the key memory
219     // that gets access to the sbox.
220     //-----
221
222     always @*
223     begin : sbox_mux
224         if (init_state)
225         begin
226             muxed_sboxw = keymem_sboxw;
227
228             end
229         else
230         begin
231             muxed_sboxw = enc_sboxw;
232         end
233     end // sbox_mux
234
235     //-----
236
237     // encdec_mux
238     //
239     // Controls which of the datapaths that get the next signal,
240     // have
241     // access to the memory as well as the block processing
242     //-----
243
244     always @*
245     begin : encdec_mux
246         enc_next = 1'b0;
247         dec_next = 1'b0;
248
249         if (encdec)
250         begin
251             // Encipher operations
252             enc_next = next;
253             muxed_round_nr = enc_round_nr;
254             muxed_new_block = enc_new_block;
255             muxed_ready = enc_ready;
256         end
257         else
258         begin
259             // Decipher operations
260             dec_next = next;
261             muxed_round_nr = dec_round_nr;
262             muxed_new_block = dec_new_block;
263             muxed_ready = dec_ready;
264         end
265     end // encdec_mux
266
267     //-----
268
269     // aes_core_ctrl
270     //
271     // Control FSM for aes core. Basically tracks if we are in
272     // key init, encipher or decipher modes and connects the
273     // different submodules to shared resources and interface
274     //-----
275
276     ports.
277     //-----
278
279     always @*
280     begin : aes_core_ctrl
281         init_state = 1'b0;
282         ready_new = 1'b0;
283         ready_we = 1'b0;
284         result_valid_new = 1'b0;
285         result_valid_we = 1'b0;
286         aes_core_ctrl_new = CTRL_IDLE;
287         aes_core_ctrl_we = 1'b0;
288
289         case (aes_core_ctrl_reg)
290             CTRL_IDLE:
291             begin
292                 if (init)
293                 begin
294                     init_state = 1'b1;
295                     ready_new = 1'b0;
296                     ready_we = 1'b1;
297                     result_valid_new = 1'b0;
298                     result_valid_we = 1'b1;
299                     aes_core_ctrl_new = CTRL_INIT;
300                     aes_core_ctrl_we = 1'b1;
301                 end
302                 else if (next)
303                 begin
304                     init_state = 1'b0;
305                     ready_new = 1'b0;
306                     ready_we = 1'b1;
307                     result_valid_new = 1'b0;
308                     result_valid_we = 1'b1;
309                     aes_core_ctrl_new = CTRL_NEXT;
310                     aes_core_ctrl_we = 1'b1;
311                 end
312             end
313
314             CTRL_INIT:
315             begin
316                 init_state = 1'b1;
317
318                 if (key_ready)
319                 begin
320                     ready_new = 1'b1;
321                     ready_we = 1'b1;
322                     aes_core_ctrl_new = CTRL_IDLE;
323                     aes_core_ctrl_we = 1'b1;
324                 end
325             end
326         end
327     end
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

316     CTRL_NEXT:
317     begin
318         init_state = 1'b0;
319
320         if (muxed_ready)
321         begin
322             ready_new      = 1'b1;
323             ready_we       = 1'b1;
324             result_valid_new = 1'b1;
325             result_valid_we = 1'b1;
326             aes_core_ctrl_new = CTRL_IDLE;
327             aes_core_ctrl_we = 1'b1;
328         end
329     end
330
331     default:
332     begin
333
334     end
335     endcase // case (aes_core_ctrl_reg)
336
337     end // aes_core_ctrl
338 endmodule // aes_core
339
340 //
=====
341 // EOF aes_core.v
342 //
=====

```

A.31 secworks_aes/src/rtl/aes_encipher_block.v (487줄, 14,416B)

```

1 //
=====
2 //
3 // aes_encipher_block.v
4 // -----
5 // The AES encipher round. A pure combinational module that
implements
6 // the initial round, main round and final round logic for
7 // encipher operations.
8 //
9 //
10 // Author: Joachim Strombergson
11 // Copyright (c) 2013, 2014, Secworks Sweden AB
12 // All rights reserved.
13 //
14 // Redistribution and use in source and binary forms, with or
15 // without modification, are permitted provided that the
following
16 // conditions are met:
17 //
18 // 1. Redistributions of source code must retain the above
copyright
19 // notice, this list of conditions and the following
disclaimer.
20 //
21 // 2. Redistributions in binary form must reproduce the above
copyright
22 // notice, this list of conditions and the following
disclaimer in
23 // the documentation and/or other materials provided with
the
24 // distribution.
25 //
26 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS
27 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
28 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
29 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
30 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
31 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
32 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES;
33 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
34 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
35 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
OTHERWISE)
36 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
37 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
38 //
39 //
=====
40
41 `default_nettype none
42
43 module aes_encipher_block(
44     input wire      clk,
45     input wire      reset_n,

```

```

46
47     input wire      next,
48
49     input wire      keylen,
50     output wire [3 : 0] round,
51     input wire [127 : 0] round_key,
52
53     output wire [31 : 0] sboxw,
54     input wire [31 : 0] new_sboxw,
55
56     input wire [127 : 0] block,
57     output wire [127 : 0] new_block,
58     output wire      ready
59 );
60
61
62 //-----
63 // Internal constant and parameter definitions.
64 //-----
65
66 localparam AES_128_BIT_KEY = 1'h0;
67 localparam AES_256_BIT_KEY = 1'h1;
68
69 localparam AES128_ROUNDS = 4'ha;
70 localparam AES256_ROUNDS = 4'he;
71
72 localparam NO_UPDATE      = 3'h0;
73 localparam INIT_UPDATE    = 3'h1;
74 localparam SBOX_UPDATE    = 3'h2;
75 localparam MAIN_UPDATE    = 3'h3;
76 localparam FINAL_UPDATE   = 3'h4;
77
78 localparam CTRL_IDLE      = 2'h0;
79 localparam CTRL_INIT      = 2'h1;
80 localparam CTRL_SBOX      = 2'h2;
81 localparam CTRL_MAIN      = 2'h3;
82
83 //-----
84 // Round functions with sub functions.
85 //-----
86
87 function automatic [7 : 0] gm2(input [7 : 0] op);
88 begin
89     gm2 = {op[6 : 0], 1'b0} ^ (8'h1b & {8{op[7]}});
90 end
91 endfunction // gm2
92
93 function automatic [7 : 0] gm3(input [7 : 0] op);
94 begin
95     gm3 = gm2(op) ^ op;
96 end
97 endfunction // gm3
98
99 function automatic [31 : 0] mixw(input [31 : 0] w);
100 reg [7 : 0] b0, b1, b2, b3;
101 reg [7 : 0] mb0, mb1, mb2, mb3;
102 begin
103     b0 = w[31 : 24];
104     b1 = w[23 : 16];
105     b2 = w[15 : 08];
106     b3 = w[07 : 00];
107
108     mb0 = gm2(b0) ^ gm3(b1) ^ b2 ^ b3;
109     mb1 = b0 ^ gm2(b1) ^ gm3(b2) ^ b3;
110     mb2 = b0 ^ b1 ^ gm2(b2) ^ gm3(b3);
111     mb3 = gm3(b0) ^ b1 ^ b2 ^ gm2(b3);
112
113     mixw = {mb0, mb1, mb2, mb3};
114 end
115 endfunction // mixw
116
117 function automatic [127 : 0] mixcolumns(input [127 : 0]
data);
118 reg [31 : 0] w0, w1, w2, w3;
119 reg [31 : 0] ws0, ws1, ws2, ws3;
120 begin
121     w0 = data[127 : 096];
122     w1 = data[095 : 064];
123     w2 = data[063 : 032];
124     w3 = data[031 : 000];
125
126     ws0 = mixw(w0);
127     ws1 = mixw(w1);
128     ws2 = mixw(w2);
129     ws3 = mixw(w3);
130
131     mixcolumns = {ws0, ws1, ws2, ws3};
132 end
133 endfunction // mixcolumns
134
135 function automatic [127 : 0] shiftrows(input [127 : 0]
data);
136 reg [31 : 0] w0, w1, w2, w3;
137 reg [31 : 0] ws0, ws1, ws2, ws3;
138 begin
139     w0 = data[127 : 096];
140     w1 = data[095 : 064];
141     w2 = data[063 : 032];

```

```

141     w3 = data[031 : 000];
142
143     ws0 = {w0[31 : 24], w1[23 : 16], w2[15 : 08], w3[07 :
00]};
144     ws1 = {w1[31 : 24], w2[23 : 16], w3[15 : 08], w0[07 :
00]};
145     ws2 = {w2[31 : 24], w3[23 : 16], w0[15 : 08], w1[07 :
00]};
146     ws3 = {w3[31 : 24], w0[23 : 16], w1[15 : 08], w2[07 :
00]};
147
148     shiftrows = {ws0, ws1, ws2, ws3};
149     end
150 endfunction // shiftrows
151
152 function automatic [127 : 0] addroundkey(input [127 : 0]
data, input [127 : 0] rkey);
153     begin
154         addroundkey = data ^ rkey;
155     end
156 endfunction // addroundkey
157
158
159 //-----
160 // Registers including update variables and write enable.
161 //-----
162 reg [1 : 0] sword_ctr_reg;
163 reg [1 : 0] sword_ctr_new;
164 reg sword_ctr_we;
165 reg sword_ctr_inc;
166 reg sword_ctr_rst;
167
168 reg [3 : 0] round_ctr_reg;
169 reg [3 : 0] round_ctr_new;
170 reg round_ctr_we;
171 reg round_ctr_rst;
172 reg round_ctr_inc;
173
174 reg [127 : 0] block_new;
175 reg [31 : 0] block_w0_reg;
176 reg [31 : 0] block_w1_reg;
177 reg [31 : 0] block_w2_reg;
178 reg [31 : 0] block_w3_reg;
179 reg block_w0_we;
180 reg block_w1_we;
181 reg block_w2_we;
182 reg block_w3_we;
183
184 reg ready_reg;
185 reg ready_new;
186 reg ready_we;
187
188 reg [1 : 0] enc_ctrl_reg;
189 reg [1 : 0] enc_ctrl_new;
190 reg enc_ctrl_we;
191
192 //-----
193 // Wires.
194 //-----
195
196 reg [2 : 0] update_type;
197 reg [31 : 0] muxed_sboxw;
198
199 //-----
200
201 // Concurrent connectivity for ports etc.
202 //-----
203 assign round = round_ctr_reg;
204 assign sboxw = muxed_sboxw;
205 assign new_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
206 assign ready = ready_reg;
207
208
209 //-----
210 // reg_update
211 //
212 // Update functionality for all registers in the core.
213 // All registers are positive edge triggered with
asynchronous
214 // active low reset. All registers have write enable.
215 //-----
216 always @ (posedge clk or negedge reset_n)
217 begin: reg_update
218     if (!reset_n)
219     begin
220         block_w0_reg <= 32'h0;
221         block_w1_reg <= 32'h0;
222         block_w2_reg <= 32'h0;
223         block_w3_reg <= 32'h0;
224         sword_ctr_reg <= 2'h0;
225         round_ctr_reg <= 4'h0;
226         ready_reg <= 1'b1;

```

```

227     enc_ctrl_reg <= CTRL_IDLE;
228 end
229 else
230 begin
231     if (block_w0_we)
232         block_w0_reg <= block_new[127 : 096];
233
234     if (block_w1_we)
235         block_w1_reg <= block_new[095 : 064];
236
237     if (block_w2_we)
238         block_w2_reg <= block_new[063 : 032];
239
240     if (block_w3_we)
241         block_w3_reg <= block_new[031 : 000];
242
243     if (sword_ctr_we)
244         sword_ctr_reg <= sword_ctr_new;
245
246     if (round_ctr_we)
247         round_ctr_reg <= round_ctr_new;
248
249     if (ready_we)
250         ready_reg <= ready_new;
251
252     if (enc_ctrl_we)
253         enc_ctrl_reg <= enc_ctrl_new;
254     end
255 end // reg_update
256
257 //-----
258 // round_logic
259 //
260 // The logic needed to implement init, main and final
rounds.
261 //-----
262
263 always @*
264 begin : round_logic
265     reg [127 : 0] old_block, shiftrows_block,
mixcolumns_block;
266     reg [127 : 0] addkey_init_block, addkey_main_block,
addkey_final_block;
267
268     block_new = 128'h0;
269     muxed_sboxw = 32'h0;
270     block_w0_we = 1'b0;
271     block_w1_we = 1'b0;
272     block_w2_we = 1'b0;
273     block_w3_we = 1'b0;
274
275     old_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
276     shiftrows_block = shiftrows(old_block);
277     mixcolumns_block = mixcolumns(shiftrows_block);
278     addkey_init_block = addroundkey(block, round_key);
279     addkey_main_block = addroundkey(mixcolumns_block,
round_key);
280     addkey_final_block = addroundkey(shiftrows_block,
round_key);
281
282     case (update_type)
283     INIT_UPDATE:
284     begin
285         block_new = addkey_init_block;
286         block_w0_we = 1'b1;
287         block_w1_we = 1'b1;
288         block_w2_we = 1'b1;
289         block_w3_we = 1'b1;
290     end
291
292     SBOX_UPDATE:
293     begin
294         block_new = {new_sboxw, new_sboxw, new_sboxw,
new_sboxw};
295
296         case (sword_ctr_reg)
297         2'h0:
298         begin
299             muxed_sboxw = block_w0_reg;
300             block_w0_we = 1'b1;
301         end
302
303         2'h1:
304         begin
305             muxed_sboxw = block_w1_reg;
306             block_w1_we = 1'b1;
307         end
308
309         2'h2:
310         begin
311             muxed_sboxw = block_w2_reg;
312             block_w2_we = 1'b1;
313         end
314
315         2'h3:
316         begin
317             muxed_sboxw = block_w3_reg;
318             block_w3_we = 1'b1;

```

```

319         end
320     endcase // case (sbox_mux_ctrl_reg)
321 end
322
323 MAIN_UPDATE:
324 begin
325     block_new = addkey_main_block;
326     block_w0_we = 1'b1;
327     block_w1_we = 1'b1;
328     block_w2_we = 1'b1;
329     block_w3_we = 1'b1;
330 end
331
332 FINAL_UPDATE:
333 begin
334     block_new = addkey_final_block;
335     block_w0_we = 1'b1;
336     block_w1_we = 1'b1;
337     block_w2_we = 1'b1;
338     block_w3_we = 1'b1;
339 end
340
341 default:
342 begin
343 end
344 endcase // case (update_type)
345 end // round_logic
346
347 //-----
348 // sword_ctr
349 //
350 // The subbytes word counter with reset and increase logic.
351 //-----
352
353 always @*
354 begin : sword_ctr
355     sword_ctr_new = 2'h0;
356     sword_ctr_we = 1'b0;
357
358     if (sword_ctr_rst)
359     begin
360         sword_ctr_new = 2'h0;
361         sword_ctr_we = 1'b1;
362     end
363     else if (sword_ctr_inc)
364     begin
365         sword_ctr_new = sword_ctr_reg + 1'b1;
366         sword_ctr_we = 1'b1;
367     end
368 end // sword_ctr
369
370 //-----
371 // round_ctr
372 //
373 // The round counter with reset and increase logic.
374 //-----
375
376 always @*
377 begin : round_ctr
378     round_ctr_new = 4'h0;
379     round_ctr_we = 1'b0;
380
381     if (round_ctr_rst)
382     begin
383         round_ctr_new = 4'h0;
384         round_ctr_we = 1'b1;
385     end
386     else if (round_ctr_inc)
387     begin
388         round_ctr_new = round_ctr_reg + 1'b1;
389         round_ctr_we = 1'b1;
390     end
391 end // round_ctr
392
393 //-----
394 // encipher_ctrl
395 //
396 // The FSM that controls the encipher operations.
397 //-----
398
399 always @*
400 begin: encipher_ctrl
401     reg [3 : 0] num_rounds;
402
403     // Default assignments.
404     sword_ctr_inc = 1'b0;
405     sword_ctr_rst = 1'b0;
406     round_ctr_inc = 1'b0;
407     round_ctr_rst = 1'b0;
408     ready_new = 1'b0;
409     ready_we = 1'b0;
410     update_type = NO_UPDATE;
411     enc_ctrl_new = CTRL_IDLE;
412     enc_ctrl_we = 1'b0;
413

```

```

414 if (keylen == AES_256_BIT_KEY)
415 begin
416     num_rounds = AES256_ROUNDS;
417 end
418 else
419 begin
420     num_rounds = AES128_ROUNDS;
421 end
422
423 case(enc_ctrl_reg)
424 CTRL_IDLE:
425 begin
426     if (next)
427     begin
428         round_ctr_rst = 1'b1;
429         ready_new = 1'b0;
430         ready_we = 1'b1;
431         enc_ctrl_new = CTRL_INIT;
432         enc_ctrl_we = 1'b1;
433     end
434 end
435
436 CTRL_INIT:
437 begin
438     round_ctr_inc = 1'b1;
439     sword_ctr_rst = 1'b1;
440     update_type = INIT_UPDATE;
441     enc_ctrl_new = CTRL_SBOX;
442     enc_ctrl_we = 1'b1;
443 end
444
445 CTRL_SBOX:
446 begin
447     sword_ctr_inc = 1'b1;
448     update_type = SBOX_UPDATE;
449     if (sword_ctr_reg == 2'h3)
450     begin
451         enc_ctrl_new = CTRL_MAIN;
452         enc_ctrl_we = 1'b1;
453     end
454 end
455
456 CTRL_MAIN:
457 begin
458     sword_ctr_rst = 1'b1;
459     round_ctr_inc = 1'b1;
460     if (round_ctr_reg < num_rounds)
461     begin
462         update_type = MAIN_UPDATE;
463         enc_ctrl_new = CTRL_SBOX;
464         enc_ctrl_we = 1'b1;
465     end
466     else
467     begin
468         update_type = FINAL_UPDATE;
469         ready_new = 1'b1;
470         ready_we = 1'b1;
471         enc_ctrl_new = CTRL_IDLE;
472         enc_ctrl_we = 1'b1;
473     end
474 end
475
476 default:
477 begin
478     // Empty. Just here to make the synthesis tool
479     happy.
480 end
481 endcase // case (enc_ctrl_reg)
482 end // encipher_ctrl
483 endmodule // aes_encipher_block
484
485 //

```

```

=====
486 // EOF aes_encipher_block.v
487 //
=====

```

A.32 secworks_aes/src/rtl/aes_sbox.v (327줄, 10,489B)

```

1 //
=====
2 //
3 // aes_sbox.v
4 // -----
5 // The AES S-box. Basically a 256 Byte ROM. This
implementation
6 // contains four parallel S-boxes to handle a 32 bit word.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2014, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the
following

```

```

15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following
disclaimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following
disclaimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
OTHERWISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
==
39
40 `default_nettype none
41
42 module aes_sbox(
43     input wire [31 : 0] sboxw,
44     output wire [31 : 0] new_sboxw
45 );
46
47
48 //-----
49 // The sbox array.
50 //-----
51
52 wire [7 : 0] sbox [0 : 255];
53
54 //-----
55
56 // Four parallel muxes.
57 //-----
58
59 assign new_sboxw[31 : 24] = sbox[sboxw[31 : 24]];
60 assign new_sboxw[23 : 16] = sbox[sboxw[23 : 16]];
61 assign new_sboxw[15 : 08] = sbox[sboxw[15 : 08]];
62 assign new_sboxw[07 : 00] = sbox[sboxw[07 : 00]];
63
64 //-----
65
66 // Creating the sbox array contents.
67 //-----
68
69 assign sbox[8'h00] = 8'h63;
70 assign sbox[8'h01] = 8'h7c;
71 assign sbox[8'h02] = 8'h77;
72 assign sbox[8'h03] = 8'h7b;
73 assign sbox[8'h04] = 8'hf2;
74 assign sbox[8'h05] = 8'h6b;
75 assign sbox[8'h06] = 8'h6f;
76 assign sbox[8'h07] = 8'hc5;
77 assign sbox[8'h08] = 8'h30;
78 assign sbox[8'h09] = 8'h01;
79 assign sbox[8'h0a] = 8'h67;
80 assign sbox[8'h0b] = 8'h2b;
81 assign sbox[8'h0c] = 8'hfe;
82 assign sbox[8'h0d] = 8'hd7;
83 assign sbox[8'h0e] = 8'hab;
84 assign sbox[8'h0f] = 8'h76;
85 assign sbox[8'h10] = 8'hca;
86 assign sbox[8'h11] = 8'h82;
87 assign sbox[8'h12] = 8'hc9;
88 assign sbox[8'h13] = 8'h13;
89 assign sbox[8'h14] = 8'hfa;
90 assign sbox[8'h15] = 8'h59;
91 assign sbox[8'h16] = 8'h47;
92 assign sbox[8'h17] = 8'h0f;
93 assign sbox[8'h18] = 8'had;
94 assign sbox[8'h19] = 8'hd4;
95 assign sbox[8'h1a] = 8'ha2;
96 assign sbox[8'h1b] = 8'haf;

```

```

94 assign sbox[8'h1c] = 8'h9c;
95 assign sbox[8'h1d] = 8'ha4;
96 assign sbox[8'h1e] = 8'h72;
97 assign sbox[8'h1f] = 8'h00;
98 assign sbox[8'h20] = 8'hb7;
99 assign sbox[8'h21] = 8'hfd;
100 assign sbox[8'h22] = 8'h93;
101 assign sbox[8'h23] = 8'h26;
102 assign sbox[8'h24] = 8'h36;
103 assign sbox[8'h25] = 8'h3f;
104 assign sbox[8'h26] = 8'hf7;
105 assign sbox[8'h27] = 8'hcc;
106 assign sbox[8'h28] = 8'h34;
107 assign sbox[8'h29] = 8'ha5;
108 assign sbox[8'h2a] = 8'he5;
109 assign sbox[8'h2b] = 8'hf1;
110 assign sbox[8'h2c] = 8'h71;
111 assign sbox[8'h2d] = 8'hd8;
112 assign sbox[8'h2e] = 8'h31;
113 assign sbox[8'h2f] = 8'h15;
114 assign sbox[8'h30] = 8'h04;
115 assign sbox[8'h31] = 8'hc7;
116 assign sbox[8'h32] = 8'h23;
117 assign sbox[8'h33] = 8'hc3;
118 assign sbox[8'h34] = 8'h18;
119 assign sbox[8'h35] = 8'h96;
120 assign sbox[8'h36] = 8'h05;
121 assign sbox[8'h37] = 8'h9a;
122 assign sbox[8'h38] = 8'h07;
123 assign sbox[8'h39] = 8'h12;
124 assign sbox[8'h3a] = 8'h80;
125 assign sbox[8'h3b] = 8'he2;
126 assign sbox[8'h3c] = 8'heb;
127 assign sbox[8'h3d] = 8'h27;
128 assign sbox[8'h3e] = 8'hb2;
129 assign sbox[8'h3f] = 8'h75;
130 assign sbox[8'h40] = 8'h09;
131 assign sbox[8'h41] = 8'h83;
132 assign sbox[8'h42] = 8'h2c;
133 assign sbox[8'h43] = 8'h1a;
134 assign sbox[8'h44] = 8'h1b;
135 assign sbox[8'h45] = 8'h6e;
136 assign sbox[8'h46] = 8'h5a;
137 assign sbox[8'h47] = 8'ha0;
138 assign sbox[8'h48] = 8'h52;
139 assign sbox[8'h49] = 8'h3b;
140 assign sbox[8'h4a] = 8'hd6;
141 assign sbox[8'h4b] = 8'hb3;
142 assign sbox[8'h4c] = 8'h29;
143 assign sbox[8'h4d] = 8'he3;
144 assign sbox[8'h4e] = 8'h2f;
145 assign sbox[8'h4f] = 8'h84;
146 assign sbox[8'h50] = 8'h53;
147 assign sbox[8'h51] = 8'hd1;
148 assign sbox[8'h52] = 8'h00;
149 assign sbox[8'h53] = 8'hed;
150 assign sbox[8'h54] = 8'h20;
151 assign sbox[8'h55] = 8'hfc;
152 assign sbox[8'h56] = 8'hb1;
153 assign sbox[8'h57] = 8'h5b;
154 assign sbox[8'h58] = 8'h6a;
155 assign sbox[8'h59] = 8'hcb;
156 assign sbox[8'h5a] = 8'hbe;
157 assign sbox[8'h5b] = 8'h39;
158 assign sbox[8'h5c] = 8'h4a;
159 assign sbox[8'h5d] = 8'h4c;
160 assign sbox[8'h5e] = 8'h58;
161 assign sbox[8'h5f] = 8'hcf;
162 assign sbox[8'h60] = 8'hd0;
163 assign sbox[8'h61] = 8'hcf;
164 assign sbox[8'h62] = 8'haa;
165 assign sbox[8'h63] = 8'hfb;
166 assign sbox[8'h64] = 8'h43;
167 assign sbox[8'h65] = 8'h4d;
168 assign sbox[8'h66] = 8'h33;
169 assign sbox[8'h67] = 8'h85;
170 assign sbox[8'h68] = 8'h45;
171 assign sbox[8'h69] = 8'hf9;
172 assign sbox[8'h6a] = 8'h02;
173 assign sbox[8'h6b] = 8'h7f;
174 assign sbox[8'h6c] = 8'h50;
175 assign sbox[8'h6d] = 8'h3c;
176 assign sbox[8'h6e] = 8'h9f;
177 assign sbox[8'h6f] = 8'ha8;
178 assign sbox[8'h70] = 8'h51;
179 assign sbox[8'h71] = 8'ha3;
180 assign sbox[8'h72] = 8'h40;
181 assign sbox[8'h73] = 8'h8f;
182 assign sbox[8'h74] = 8'h92;
183 assign sbox[8'h75] = 8'h9d;
184 assign sbox[8'h76] = 8'h38;
185 assign sbox[8'h77] = 8'hf5;
186 assign sbox[8'h78] = 8'hbc;
187 assign sbox[8'h79] = 8'hb6;
188 assign sbox[8'h7a] = 8'hda;
189 assign sbox[8'h7b] = 8'h21;
190 assign sbox[8'h7c] = 8'h10;
191 assign sbox[8'h7d] = 8'hff;
192 assign sbox[8'h7e] = 8'hf3;
193 assign sbox[8'h7f] = 8'hd2;
194 assign sbox[8'h80] = 8'hcd;

```

```

195 assign sbox[8'h81] = 8'h0c;
196 assign sbox[8'h82] = 8'h13;
197 assign sbox[8'h83] = 8'hec;
198 assign sbox[8'h84] = 8'h5f;
199 assign sbox[8'h85] = 8'h97;
200 assign sbox[8'h86] = 8'h44;
201 assign sbox[8'h87] = 8'h17;
202 assign sbox[8'h88] = 8'hc4;
203 assign sbox[8'h89] = 8'ha7;
204 assign sbox[8'h8a] = 8'h7e;
205 assign sbox[8'h8b] = 8'h3d;
206 assign sbox[8'h8c] = 8'h64;
207 assign sbox[8'h8d] = 8'h5d;
208 assign sbox[8'h8e] = 8'h19;
209 assign sbox[8'h8f] = 8'h73;
210 assign sbox[8'h90] = 8'h60;
211 assign sbox[8'h91] = 8'h81;
212 assign sbox[8'h92] = 8'h4f;
213 assign sbox[8'h93] = 8'hdc;
214 assign sbox[8'h94] = 8'h22;
215 assign sbox[8'h95] = 8'h2a;
216 assign sbox[8'h96] = 8'h90;
217 assign sbox[8'h97] = 8'h88;
218 assign sbox[8'h98] = 8'h46;
219 assign sbox[8'h99] = 8'hee;
220 assign sbox[8'h9a] = 8'hb8;
221 assign sbox[8'h9b] = 8'h14;
222 assign sbox[8'h9c] = 8'hde;
223 assign sbox[8'h9d] = 8'h5e;
224 assign sbox[8'h9e] = 8'h0b;
225 assign sbox[8'h9f] = 8'hdb;
226 assign sbox[8'ha0] = 8'he0;
227 assign sbox[8'ha1] = 8'h32;
228 assign sbox[8'ha2] = 8'h3a;
229 assign sbox[8'ha3] = 8'h0a;
230 assign sbox[8'ha4] = 8'h49;
231 assign sbox[8'ha5] = 8'h06;
232 assign sbox[8'ha6] = 8'h24;
233 assign sbox[8'ha7] = 8'h5c;
234 assign sbox[8'ha8] = 8'hc2;
235 assign sbox[8'ha9] = 8'hd3;
236 assign sbox[8'haa] = 8'hac;
237 assign sbox[8'hab] = 8'h62;
238 assign sbox[8'hac] = 8'h91;
239 assign sbox[8'had] = 8'h95;
240 assign sbox[8'hae] = 8'he4;
241 assign sbox[8'haf] = 8'h79;
242 assign sbox[8'hb0] = 8'he7;
243 assign sbox[8'hb1] = 8'hc8;
244 assign sbox[8'hb2] = 8'h37;
245 assign sbox[8'hb3] = 8'h6d;
246 assign sbox[8'hb4] = 8'hd8;
247 assign sbox[8'hb5] = 8'hd5;
248 assign sbox[8'hb6] = 8'h4e;
249 assign sbox[8'hb7] = 8'ha9;
250 assign sbox[8'hb8] = 8'h6c;
251 assign sbox[8'hb9] = 8'h56;
252 assign sbox[8'hba] = 8'hf4;
253 assign sbox[8'hbb] = 8'hea;
254 assign sbox[8'hbc] = 8'h65;
255 assign sbox[8'hbd] = 8'h7a;
256 assign sbox[8'hbe] = 8'hae;
257 assign sbox[8'hbf] = 8'h08;
258 assign sbox[8'hc0] = 8'hba;
259 assign sbox[8'hc1] = 8'h78;
260 assign sbox[8'hc2] = 8'h25;
261 assign sbox[8'hc3] = 8'h2e;
262 assign sbox[8'hc4] = 8'h1c;
263 assign sbox[8'hc5] = 8'ha6;
264 assign sbox[8'hc6] = 8'hb4;
265 assign sbox[8'hc7] = 8'hc6;
266 assign sbox[8'hc8] = 8'he8;
267 assign sbox[8'hc9] = 8'hdd;
268 assign sbox[8'hca] = 8'h74;
269 assign sbox[8'hcb] = 8'h1f;
270 assign sbox[8'hcc] = 8'h4b;
271 assign sbox[8'hcd] = 8'hbd;
272 assign sbox[8'hce] = 8'h8b;
273 assign sbox[8'hcf] = 8'h8a;
274 assign sbox[8'hd0] = 8'h70;
275 assign sbox[8'hd1] = 8'h3e;
276 assign sbox[8'hd2] = 8'hb5;
277 assign sbox[8'hd3] = 8'h66;
278 assign sbox[8'hd4] = 8'h48;
279 assign sbox[8'hd5] = 8'h03;
280 assign sbox[8'hd6] = 8'hf6;
281 assign sbox[8'hd7] = 8'h0e;
282 assign sbox[8'hd8] = 8'h61;
283 assign sbox[8'hd9] = 8'h35;
284 assign sbox[8'hda] = 8'h57;
285 assign sbox[8'hdb] = 8'hb9;
286 assign sbox[8'hdc] = 8'h86;
287 assign sbox[8'hdd] = 8'hc1;
288 assign sbox[8'hde] = 8'h1d;
289 assign sbox[8'hdf] = 8'h9e;
290 assign sbox[8'he0] = 8'h1;
291 assign sbox[8'he1] = 8'hf8;
292 assign sbox[8'he2] = 8'h98;
293 assign sbox[8'he3] = 8'h11;
294 assign sbox[8'he4] = 8'h69;
295 assign sbox[8'he5] = 8'hd9;

296 assign sbox[8'he6] = 8'h8e;
297 assign sbox[8'he7] = 8'h94;
298 assign sbox[8'he8] = 8'h9b;
299 assign sbox[8'he9] = 8'h1e;
300 assign sbox[8'hea] = 8'h87;
301 assign sbox[8'heb] = 8'he9;
302 assign sbox[8'hec] = 8'hce;
303 assign sbox[8'hed] = 8'h55;
304 assign sbox[8'hee] = 8'h28;
305 assign sbox[8'hef] = 8'hdf;
306 assign sbox[8'hf0] = 8'h8c;
307 assign sbox[8'hf1] = 8'ha1;
308 assign sbox[8'hf2] = 8'h89;
309 assign sbox[8'hf3] = 8'h0d;
310 assign sbox[8'hf4] = 8'hbf;
311 assign sbox[8'hf5] = 8'he6;
312 assign sbox[8'hf6] = 8'h42;
313 assign sbox[8'hf7] = 8'h68;
314 assign sbox[8'hf8] = 8'h41;
315 assign sbox[8'hf9] = 8'h99;
316 assign sbox[8'hfa] = 8'h2d;
317 assign sbox[8'hfb] = 8'h0f;
318 assign sbox[8'hfc] = 8'hb0;
319 assign sbox[8'hfd] = 8'h54;
320 assign sbox[8'hfe] = 8'hbb;
321 assign sbox[8'hff] = 8'h16;
322
323 endmodule // aes_sbox
324
325 //
=====
326 // EOF aes_sbox.v
327 //
=====

A.33 secworks_sha256/src/rtl/sha256_core.v (554줄, 15,492B)

1 //
=====
2 //
3 // sha256_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-256 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the
following
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following
disclaimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following
disclaimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
OTHERWISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256_core(

```

```

43         input wire          clk,
44         input wire          reset_n,
45
46         input wire          init,
47         input wire          next,
48         input wire          mode,
49
50         input wire [511 : 0] block,
51
52         output wire         ready,
53         output wire [255 : 0] digest,
54         output wire         digest_valid
55     );
56
57 //-----
58
59 // Internal constant and parameter definitions.
60 //-----
61 localparam SHA224_H0_0 = 32'hc1059ed8;
62 localparam SHA224_H0_1 = 32'h367cd507;
63 localparam SHA224_H0_2 = 32'h3070dd17;
64 localparam SHA224_H0_3 = 32'hf70e5939;
65 localparam SHA224_H0_4 = 32'hffc00b31;
66 localparam SHA224_H0_5 = 32'h68581511;
67 localparam SHA224_H0_6 = 32'h64f98fa7;
68 localparam SHA224_H0_7 = 32'hbfa4fa4;
69
70 localparam SHA256_H0_0 = 32'h6a09e667;
71 localparam SHA256_H0_1 = 32'hbb67ae85;
72 localparam SHA256_H0_2 = 32'h3c6ef372;
73 localparam SHA256_H0_3 = 32'ha54ff53a;
74 localparam SHA256_H0_4 = 32'h510e527f;
75 localparam SHA256_H0_5 = 32'h9b05688c;
76 localparam SHA256_H0_6 = 32'h1f83d9ab;
77 localparam SHA256_H0_7 = 32'h5be0cd19;
78
79 localparam SHA256_ROUNDS = 63;
80
81 localparam CTRL_IDLE = 0;
82 localparam CTRL_ROUNDS = 1;
83 localparam CTRL_DONE = 2;
84
85 //-----
86
87 // Registers including update variables and write enable.
88 //-----
89 reg [31 : 0] a_reg;
90 reg [31 : 0] a_new;
91 reg [31 : 0] b_reg;
92 reg [31 : 0] b_new;
93 reg [31 : 0] c_reg;
94 reg [31 : 0] c_new;
95 reg [31 : 0] d_reg;
96 reg [31 : 0] d_new;
97 reg [31 : 0] e_reg;
98 reg [31 : 0] e_new;
99 reg [31 : 0] f_reg;
100 reg [31 : 0] f_new;
101 reg [31 : 0] g_reg;
102 reg [31 : 0] g_new;
103 reg [31 : 0] h_reg;
104 reg [31 : 0] h_new;
105 reg a_h_we;
106
107 reg [31 : 0] H0_reg;
108 reg [31 : 0] H0_new;
109 reg [31 : 0] H1_reg;
110 reg [31 : 0] H1_new;
111 reg [31 : 0] H2_reg;
112 reg [31 : 0] H2_new;
113 reg [31 : 0] H3_reg;
114 reg [31 : 0] H3_new;
115 reg [31 : 0] H4_reg;
116 reg [31 : 0] H4_new;
117 reg [31 : 0] H5_reg;
118 reg [31 : 0] H5_new;
119 reg [31 : 0] H6_reg;
120 reg [31 : 0] H6_new;
121 reg [31 : 0] H7_reg;
122 reg [31 : 0] H7_new;
123 reg H_we;
124
125 reg [5 : 0] t_ctr_reg;
126 reg [5 : 0] t_ctr_new;
127 reg t_ctr_we;
128 reg t_ctr_inc;
129 reg t_ctr_rst;
130
131 reg digest_valid_reg;
132 reg digest_valid_new;
133 reg digest_valid_we;
134
135 reg [1 : 0] sha256_ctrl_reg;
136 reg [1 : 0] sha256_ctrl_new;
137 reg sha256_ctrl_we;
138
139
140 //-----
141 // Wires.
142 //-----
143 reg digest_init;
144 reg digest_update;
145
146 reg state_init;
147 reg state_update;
148
149 reg first_block;
150
151 reg ready_flag;
152
153 reg [31 : 0] t1;
154 reg [31 : 0] t2;
155
156 wire [31 : 0] k_data;
157
158 reg w_init;
159 reg w_next;
160 reg [5 : 0] w_round;
161 wire [31 : 0] w_data;
162
163
164 //-----
165 // Module instantiations.
166 //-----
167 sha256_k_constants k_constants_inst(
168     .round(t_ctr_reg),
169     .K(k_data)
170 );
171
172 sha256_w_mem w_mem_inst(
173     .clk(clk),
174     .reset_n(reset_n),
175     .block(block),
176     .round(t_ctr_reg),
177     .init(w_init),
178     .next(w_next),
179     .w(w_data)
180 );
181
182 //-----
183 // Concurrent connectivity for ports etc.
184 //-----
185
186 assign ready = ready_flag;
187
188 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
189     H4_reg, H5_reg, H6_reg, H7_reg};
190
191 assign digest_valid = digest_valid_reg;
192
193 //-----
194 // reg_update
195 // Update functionality for all registers in the core.
196 // All registers are positive edge triggered with
197 // asynchronous
198 // active low reset. All registers have write enable.
199 //-----
200
201 always @ (posedge clk or negedge reset_n)
202     begin : reg_update
203         if (!reset_n)
204             begin
205                 a_reg <= 32'h0;
206                 b_reg <= 32'h0;
207                 c_reg <= 32'h0;
208                 d_reg <= 32'h0;
209                 e_reg <= 32'h0;
210                 f_reg <= 32'h0;
211                 g_reg <= 32'h0;
212                 h_reg <= 32'h0;
213                 H0_reg <= 32'h0;
214                 H1_reg <= 32'h0;
215                 H2_reg <= 32'h0;
216                 H3_reg <= 32'h0;
217                 H4_reg <= 32'h0;
218                 H5_reg <= 32'h0;
219                 H6_reg <= 32'h0;
220                 H7_reg <= 32'h0;
221                 digest_valid_reg <= 0;
222                 t_ctr_reg <= 6'h0;
223                 sha256_ctrl_reg <= CTRL_IDLE;
224             end
225         else
226             begin
227                 if (a_h_we)
228                     begin
229                         a_reg <= a_new;
230                         b_reg <= b_new;
231                         c_reg <= c_new;
232                         d_reg <= d_new;
233                         e_reg <= e_new;
234                         f_reg <= f_new;
235                         g_reg <= g_new;
236                         h_reg <= h_new;
237                         H0_reg <= H0_new;
238                         H1_reg <= H1_new;
239                         H2_reg <= H2_new;
240                         H3_reg <= H3_new;
241                         H4_reg <= H4_new;
242                         H5_reg <= H5_new;
243                         H6_reg <= H6_new;
244                         H7_reg <= H7_new;
245                         digest_valid_reg <= digest_valid_new;
246                         t_ctr_reg <= t_ctr_new;
247                         sha256_ctrl_reg <= sha256_ctrl_new;
248                     end
249                 else
250                     begin
251                         a_h_we <= a_h_we;
252                         t_ctr_we <= t_ctr_we;
253                         t_ctr_inc <= t_ctr_inc;
254                         t_ctr_rst <= t_ctr_rst;
255                         digest_valid_we <= digest_valid_we;
256                         sha256_ctrl_we <= sha256_ctrl_we;
257                     end
258             end
259         end
260     end

```

```

232     a_reg <= a_new;
233     b_reg <= b_new;
234     c_reg <= c_new;
235     d_reg <= d_new;
236     e_reg <= e_new;
237     f_reg <= f_new;
238     g_reg <= g_new;
239     h_reg <= h_new;
240 end
241
242 if (H_we)
243 begin
244     H0_reg <= H0_new;
245     H1_reg <= H1_new;
246     H2_reg <= H2_new;
247     H3_reg <= H3_new;
248     H4_reg <= H4_new;
249     H5_reg <= H5_new;
250     H6_reg <= H6_new;
251     H7_reg <= H7_new;
252 end
253
254 if (t_ctr_we)
255     t_ctr_reg <= t_ctr_new;
256
257 if (digest_valid_we)
258     digest_valid_reg <= digest_valid_new;
259
260 if (sha256_ctrl_we)
261     sha256_ctrl_reg <= sha256_ctrl_new;
262 end
263 end // reg_update
264
265 //-----
266 // digest_logic
267 //
268 // The logic needed to init as well as update the digest.
269 //-----
270
271 always @*
272 begin : digest_logic
273     H0_new = 32'h0;
274     H1_new = 32'h0;
275     H2_new = 32'h0;
276     H3_new = 32'h0;
277     H4_new = 32'h0;
278     H5_new = 32'h0;
279     H6_new = 32'h0;
280     H7_new = 32'h0;
281     H_we = 0;
282
283 if (digest_init)
284 begin
285     H_we = 1;
286     if (mode)
287     begin
288         H0_new = SHA256_H0_0;
289         H1_new = SHA256_H0_1;
290         H2_new = SHA256_H0_2;
291         H3_new = SHA256_H0_3;
292         H4_new = SHA256_H0_4;
293         H5_new = SHA256_H0_5;
294         H6_new = SHA256_H0_6;
295         H7_new = SHA256_H0_7;
296     end
297 else
298 begin
299     H0_new = SHA224_H0_0;
300     H1_new = SHA224_H0_1;
301     H2_new = SHA224_H0_2;
302     H3_new = SHA224_H0_3;
303     H4_new = SHA224_H0_4;
304     H5_new = SHA224_H0_5;
305     H6_new = SHA224_H0_6;
306     H7_new = SHA224_H0_7;
307 end
308 end
309
310 if (digest_update)
311 begin
312     H0_new = H0_reg + a_reg;
313     H1_new = H1_reg + b_reg;
314     H2_new = H2_reg + c_reg;
315     H3_new = H3_reg + d_reg;
316     H4_new = H4_reg + e_reg;
317     H5_new = H5_reg + f_reg;
318     H6_new = H6_reg + g_reg;
319     H7_new = H7_reg + h_reg;
320     H_we = 1;
321 end
322 end // digest_logic
323
324 //-----
325 // t1_logic
326 //
327 // The logic for the T1 function.
328 //-----

```

```

330 always @*
331 begin : t1_logic
332     reg [31 : 0] sum1;
333     reg [31 : 0] ch;
334
335     sum1 = {e_reg[5 : 0], e_reg[31 : 6]} ^
336           {e_reg[10 : 0], e_reg[31 : 11]} ^
337           {e_reg[24 : 0], e_reg[31 : 25]};
338
339     ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
340
341     t1 = h_reg + sum1 + ch + w_data + k_data;
342 end // t1_logic
343
344 //-----
345 // t2_logic
346 //
347 // The logic for the T2 function
348 //-----
349
350 always @*
351 begin : t2_logic
352     reg [31 : 0] sum0;
353     reg [31 : 0] maj;
354
355     sum0 = {a_reg[1 : 0], a_reg[31 : 2]} ^
356           {a_reg[12 : 0], a_reg[31 : 13]} ^
357           {a_reg[21 : 0], a_reg[31 : 22]};
358
359     maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &
360 c_reg);
361
362     t2 = sum0 + maj;
363 end // t2_logic
364
365 //-----
366 // state_logic
367 //
368 // The logic needed to init as well as update the state
369 during
370 // round processing.
371 //-----
372
373 always @*
374 begin : state_logic
375     a_new = 32'h0;
376     b_new = 32'h0;
377     c_new = 32'h0;
378     d_new = 32'h0;
379     e_new = 32'h0;
380     f_new = 32'h0;
381     g_new = 32'h0;
382     h_new = 32'h0;
383     a_h_we = 0;
384
385 if (state_init)
386 begin
387     a_h_we = 1;
388     if (first_block)
389     begin
390         if (mode)
391         begin
392             a_new = SHA256_H0_0;
393             b_new = SHA256_H0_1;
394             c_new = SHA256_H0_2;
395             d_new = SHA256_H0_3;
396             e_new = SHA256_H0_4;
397             f_new = SHA256_H0_5;
398             g_new = SHA256_H0_6;
399             h_new = SHA256_H0_7;
400         end
401     end
402 else
403 begin
404     a_new = SHA224_H0_0;
405     b_new = SHA224_H0_1;
406     c_new = SHA224_H0_2;
407     d_new = SHA224_H0_3;
408     e_new = SHA224_H0_4;
409     f_new = SHA224_H0_5;
410     g_new = SHA224_H0_6;
411     h_new = SHA224_H0_7;
412 end
413 end
414
415 if (mode)
416 begin
417     a_new = H0_reg;
418     b_new = H1_reg;
419     c_new = H2_reg;
420     d_new = H3_reg;
421     e_new = H4_reg;
422     f_new = H5_reg;
423     g_new = H6_reg;
424     h_new = H7_reg;
425 end
426 end

```

```

424     if (state_update)
425     begin
426         a_new = t1 + t2;
427         b_new = a_reg;
428         c_new = b_reg;
429         d_new = c_reg;
430         e_new = d_reg + t1;
431         f_new = e_reg;
432         g_new = f_reg;
433         h_new = g_reg;
434         a_h_we = 1;
435     end
436 end // state_logic
437
438 //-----
439
440 // t_ctr
441 //
442 // Update logic for the round counter, a monotonically
443 // increasing counter with reset.
444 //-----
445
446 always @*
447 begin : t_ctr
448     t_ctr_new = 0;
449     t_ctr_we = 0;
450
451     if (t_ctr_rst)
452     begin
453         t_ctr_new = 0;
454         t_ctr_we = 1;
455     end
456
457     if (t_ctr_inc)
458     begin
459         t_ctr_new = t_ctr_reg + 1'b1;
460         t_ctr_we = 1;
461     end
462 end // t_ctr
463
464 //-----
465
466 // sha256_ctrl_fsm
467 //
468 // Logic for the state machine controlling the core
behaviour.
469 //-----
470
471 always @*
472 begin : sha256_ctrl_fsm
473     digest_init = 0;
474     digest_update = 0;
475
476     state_init = 0;
477     state_update = 0;
478
479     first_block = 0;
480     ready_flag = 0;
481
482     w_init = 0;
483     w_next = 0;
484
485     t_ctr_inc = 0;
486     t_ctr_rst = 0;
487
488     digest_valid_new = 0;
489     digest_valid_we = 0;
490
491     sha256_ctrl_new = CTRL_IDLE;
492     sha256_ctrl_we = 0;
493
494     case (sha256_ctrl_reg)
495     CTRL_IDLE:
496     begin
497         ready_flag = 1;
498
499         if (init)
500         begin
501             digest_init = 1;
502             w_init = 1;
503             state_init = 1;
504             first_block = 1;
505             t_ctr_rst = 1;
506             digest_valid_new = 0;
507             digest_valid_we = 1;
508             sha256_ctrl_new = CTRL_ROUNDS;
509             sha256_ctrl_we = 1;
510         end
511
512         if (next)
513         begin
514             t_ctr_rst = 1;
515             w_init = 1;
516             state_init = 1;
517             digest_valid_new = 0;
518             digest_valid_we = 1;
519             sha256_ctrl_new = CTRL_ROUNDS;
520             sha256_ctrl_we = 1;

```

```

520     end
521 end
522
523 CTRL_ROUNDS:
524 begin
525     w_next = 1;
526     state_update = 1;
527     t_ctr_inc = 1;
528
529     if (t_ctr_reg == SHA256_ROUNDS)
530     begin
531         sha256_ctrl_new = CTRL_DONE;
532         sha256_ctrl_we = 1;
533     end
534 end
535
536 CTRL_DONE:
537 begin
538     digest_update = 1;
539     digest_valid_new = 1;
540     digest_valid_we = 1;
541
542     sha256_ctrl_new = CTRL_IDLE;
543     sha256_ctrl_we = 1;
544 end
545 endcase // case (sha256_ctrl_reg)
546 end // sha256_ctrl_fsm
547
548 endmodule // sha256_core
549
550 //
551 //
552 //
=====
553 // EOF sha256_core.v
554 //
=====

```

A.34 secworks_sha256/src/rtl/sha256_w_mem.v (241줄, 7,815B)

```

1 //
=====
2 //
3 // sha256_w_mem_regs.v
4 //
5 // The W memory. This version uses 16 32-bit registers as a
sliding
6 // window to generate the 64 words.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the
following
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following
disclaimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following
disclaimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
OTHERWISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====

```

```

==
39
40 `default_nettype none
41
42 module sha256_w_mem(
43     input wire          clk,
44     input wire          reset_n,
45
46     input wire [511 : 0] block,
47     input wire [5 : 0]   round,
48
49     input wire          init,
50     input wire          next,
51     output wire [31 : 0] w
52 );
53
54
55 //-----
56 // Registers including update variables and write enable.
57 //-----
58 reg [31 : 0] w_mem [0 : 15];
59 reg [31 : 0] w_mem00_new;
60 reg [31 : 0] w_mem01_new;
61 reg [31 : 0] w_mem02_new;
62 reg [31 : 0] w_mem03_new;
63 reg [31 : 0] w_mem04_new;
64 reg [31 : 0] w_mem05_new;
65 reg [31 : 0] w_mem06_new;
66 reg [31 : 0] w_mem07_new;
67 reg [31 : 0] w_mem08_new;
68 reg [31 : 0] w_mem09_new;
69 reg [31 : 0] w_mem10_new;
70 reg [31 : 0] w_mem11_new;
71 reg [31 : 0] w_mem12_new;
72 reg [31 : 0] w_mem13_new;
73 reg [31 : 0] w_mem14_new;
74 reg [31 : 0] w_mem15_new;
75 reg          w_mem_we;
76
77
78 //-----
79 // Wires.
80 //-----
81 reg [31 : 0] w_tmp;
82 reg [31 : 0] w_new;
83
84
85 //-----
86 // Concurrent connectivity for ports etc.
87 //-----
88 assign w = w_tmp;
89
90
91 //-----
92 // reg_update
93 // Update functionality for all registers in the core.
94 // All registers are positive edge triggered with
synchronous
95 // active low reset. All registers have write enable.
96 //-----
97 always @ (posedge clk or negedge reset_n)
98     begin : reg_update
99         integer i;
100
101         if (!reset_n)
102             begin
103                 for (i = 0 ; i < 16 ; i = i + 1) begin
104                     w_mem[i] <= 32'h0;
105                 end
106             end
107         else
108             begin
109                 if (w_mem_we)
110                     begin
111                         w_mem[00] <= w_mem00_new;
112                         w_mem[01] <= w_mem01_new;
113                         w_mem[02] <= w_mem02_new;
114                         w_mem[03] <= w_mem03_new;
115                         w_mem[04] <= w_mem04_new;
116                         w_mem[05] <= w_mem05_new;
117                         w_mem[06] <= w_mem06_new;
118                         w_mem[07] <= w_mem07_new;
119                         w_mem[08] <= w_mem08_new;
120                         w_mem[09] <= w_mem09_new;
121                         w_mem[10] <= w_mem10_new;
122                         w_mem[11] <= w_mem11_new;
123                         w_mem[12] <= w_mem12_new;
124                         w_mem[13] <= w_mem13_new;
125                         w_mem[14] <= w_mem14_new;
126                         w_mem[15] <= w_mem15_new;
127                     end
128                 end
129             end // reg_update
130
131
132 //-----
133 // select_w
134 //
135 // Mux for the external read operation. This is where we
extract
136 // the W variable.
137 //-----
138 always @*
139     begin : select_w
140         if (round < 16)
141             w_tmp = w_mem[round[3 : 0]];
142         else
143             w_tmp = w_new;
144         end // select_w
145
146
147 //-----
148 // w_new_logic
149 //
150 // Logic that calculates the next value to be inserted into
151 // the sliding window of the memory.
152 //-----
153 always @*
154     begin : w_mem_update_logic
155         reg [31 : 0] w_0;
156         reg [31 : 0] w_1;
157         reg [31 : 0] w_9;
158         reg [31 : 0] w_14;
159         reg [31 : 0] d0;
160         reg [31 : 0] d1;
161
162         w_mem00_new = 32'h0;
163         w_mem01_new = 32'h0;
164         w_mem02_new = 32'h0;
165         w_mem03_new = 32'h0;
166         w_mem04_new = 32'h0;
167         w_mem05_new = 32'h0;
168         w_mem06_new = 32'h0;
169         w_mem07_new = 32'h0;
170         w_mem08_new = 32'h0;
171         w_mem09_new = 32'h0;
172         w_mem10_new = 32'h0;
173         w_mem11_new = 32'h0;
174         w_mem12_new = 32'h0;
175         w_mem13_new = 32'h0;
176         w_mem14_new = 32'h0;
177         w_mem15_new = 32'h0;
178         w_mem_we     = 0;
179
180         w_0 = w_mem[0];
181         w_1 = w_mem[1];
182         w_9 = w_mem[9];
183         w_14 = w_mem[14];
184
185         d0 = {w_1[6 : 0], w_1[31 : 7]} ^
186             {w_1[17 : 0], w_1[31 : 18]} ^
187             {3'b000, w_1[31 : 3]};
188
189         d1 = {w_14[16 : 0], w_14[31 : 17]} ^
190             {w_14[18 : 0], w_14[31 : 19]} ^
191             {10'b0000000000, w_14[31 : 10]};
192
193         w_new = d1 + w_9 + d0 + w_0;
194
195         if (init)
196             begin
197                 w_mem00_new = block[511 : 480];
198                 w_mem01_new = block[479 : 448];
199                 w_mem02_new = block[447 : 416];
200                 w_mem03_new = block[415 : 384];
201                 w_mem04_new = block[383 : 352];
202                 w_mem05_new = block[351 : 320];
203                 w_mem06_new = block[319 : 288];
204                 w_mem07_new = block[287 : 256];
205                 w_mem08_new = block[255 : 224];
206                 w_mem09_new = block[223 : 192];
207                 w_mem10_new = block[191 : 160];
208                 w_mem11_new = block[159 : 128];
209                 w_mem12_new = block[127 : 96];
210                 w_mem13_new = block[95 : 64];
211                 w_mem14_new = block[63 : 32];
212                 w_mem15_new = block[31 : 0];
213                 w_mem_we     = 1;
214             end
215
216         if (next && (round > 15))
217             begin
218                 w_mem00_new = w_mem[01];
219                 w_mem01_new = w_mem[02];
220                 w_mem02_new = w_mem[03];
221                 w_mem03_new = w_mem[04];
222                 w_mem04_new = w_mem[05];
223                 w_mem05_new = w_mem[06];
224                 w_mem06_new = w_mem[07];
225                 w_mem07_new = w_mem[08];

```

```

226     w_mem08_new = w_mem[09];
227     w_mem09_new = w_mem[10];
228     w_mem10_new = w_mem[11];
229     w_mem11_new = w_mem[12];
230     w_mem12_new = w_mem[13];
231     w_mem13_new = w_mem[14];
232     w_mem14_new = w_mem[15];
233     w_mem15_new = w_new;
234     w_mem_we    = 1;
235     end
236   end // w_mem_update_logic
237 endmodule // sha256_w_mem
238
239 //
=====
240 // sha256_w_mem.v
241 //
=====

```

A.35 pulp-platform_axi/src/axi_pkg.sv (543줄, 26,563B)

```

1 // Copyright (c) 2014-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the
Solderpad Hardware

4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations
under the license.
11 //
12 // Authors:
13 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
14 // - Florian Zaruba <zarubaf@iis.ee.ethz.ch>
15 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
16 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
17 // - Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
18 // - Cyril Koenig <cykoenig@iis.ee.ethz.ch>
19 // - Matheus Cavalcante <matheusd@iis.ee.ethz.ch>
20
21 /// AXI Package
22 /// Contains all necessary type definitions, constants, and
generally useful functions.
23 package axi_pkg;
24   /// AXI Transaction Burst Width.
25   parameter int unsigned BurstWidth = 32'd2;
26   /// AXI Transaction Response Width.
27   parameter int unsigned RespWidth = 32'd2;
28   /// AXI Transaction Cacheability Width.
29   parameter int unsigned CacheWidth = 32'd4;
30   /// AXI Transaction Protection Width.
31   parameter int unsigned ProtWidth = 32'd3;
32   /// AXI Transaction Quality of Service Width.
33   parameter int unsigned QosWidth = 32'd4;
34   /// AXI Transaction Region Width.
35   parameter int unsigned RegionWidth = 32'd4;
36   /// AXI Transaction Length Width.
37   parameter int unsigned LenWidth = 32'd8;
38   /// AXI Transaction Size Width.
39   parameter int unsigned SizeWidth = 32'd3;
40   /// AXI Lock Width.
41   parameter int unsigned LockWidth = 32'd1;
42   /// AXI5 Atomic Operation Width.
43   parameter int unsigned AtopWidth = 32'd6;
44   /// AXI5 Non-Secure Address Identifier.
45   parameter int unsigned NsaidWidth = 32'd4;
46
47   /// AXI Transaction Burst Width.
48   typedef logic [1:0] burst_t;
49   /// AXI Transaction Response Type.
50   typedef logic [1:0] resp_t;
51   /// AXI Transaction Cacheability Type.
52   typedef logic [3:0] cache_t;
53   /// AXI Transaction Protection Type.
54   typedef logic [2:0] prot_t;
55   /// AXI Transaction Quality of Service Type.
56   typedef logic [3:0] qos_t;
57   /// AXI Transaction Region Type.
58   typedef logic [3:0] region_t;
59   /// AXI Transaction Length Type.
60   typedef logic [7:0] len_t;
61   /// AXI Transaction Size Type.
62   typedef logic [2:0] size_t;
63   /// AXI5 Atomic Operation Type.
64   typedef logic [5:0] atop_t; // atomic operations
65   /// AXI5 Non-Secure Address Identifier.
66   typedef logic [3:0] nsaid_t;
67

```

```

68   /// In a fixed burst:
69   /// - The address is the same for every transfer in the
burst.
70   /// - The byte lanes that are valid are constant for all
beats in the burst. However, within
71   /// those byte lanes, the actual bytes that have `wstrb`
asserted can differ for each beat in
72   /// the burst.
73   /// This burst type is used for repeated accesses to the
same location such as when loading or
74   /// emptying a FIFO.
75   localparam BURST_FIXED = 2'b00;
76   /// In an incrementing burst, the address for each transfer
in the burst is an increment of the
77   /// address for the previous transfer. The increment value
depends on the size of the transfer.
78   /// For example, the address for each transfer in a burst
with a size of 4 bytes is the previous
79   /// address plus four.
80   /// This burst type is used for accesses to normal
sequential memory.
81   localparam BURST_INCR = 2'b01;
82   /// A wrapping burst is similar to an incrementing burst,
except that the address wraps around to
83   /// a lower address if an upper address limit is reached.
84   /// The following restrictions apply to wrapping bursts:
85   /// - The start address must be aligned to the size of each
transfer.
86   /// - The length of the burst must be 2, 4, 8, or 16
transfers.
87   localparam BURST_WRAP = 2'b10;
88
89   /// Normal access success. Indicates that a normal access
has been successful. Can also indicate
90   /// that an exclusive access has failed.
91   localparam RESP_OKAY = 2'b00;
92   /// Exclusive access okay. Indicates that either the read
or write portion of an exclusive access
93   /// has been successful.
94   localparam RESP_EXOKAY = 2'b01;
95
96   /// Slave error. Used when the access has reached the slave
successfully, but the slave wishes to
97   /// return an error condition to the originating master.
98   localparam RESP_SLVERR = 2'b10;
99   /// Decode error. Generated, typically by an interconnect
component, to indicate that there is no
100  /// slave at the transaction address.
101  localparam RESP_DECERR = 2'b11;
102
103  /// When this bit is asserted, the interconnect, or any
component, can delay the transaction
104  /// reaching its final destination for any number of cycles.
105  localparam CACHE_BUFFERABLE = 4'b0001;
106
107  /// When HIGH, Modifiable indicates that the characteristics
of the transaction can be modified.
108  /// When Modifiable is LOW, the transaction is Non-
modifiable.
109  localparam CACHE_MODIFIABLE = 4'b0010;
110  /// When this bit is asserted, read allocation of the
transaction is recommended but is not
111  /// mandatory.
112  localparam CACHE_RD_ALLOC = 4'b0100;
113  /// When this bit is asserted, write allocation of the
transaction is recommended but is not
114  /// mandatory.
115  localparam CACHE_WR_ALLOC = 4'b1000;
116
117  /// Maximum number of bytes per burst, as specified by
'size' (see Table A3-2).
118  function automatic shortint unsigned num_bytes(size_t size);
119  return shortint'(1 << size);
120  endfunction
121
122  /// An overly long address type.
123  /// It lets us define functions that work generically for
shorter addresses. We rely on the
124  /// synthesizer to optimize the unused bits away.
125  typedef logic [127:0] largest_addr_t;
126
127  /// Aligned address of burst (see A3-51).
128  function automatic largest_addr_t
aligned_addr(largest_addr_t addr, size_t size);
129  return (addr >> size) << size;
130  endfunction
131
132  /// Warp boundary of a `BURST_WRAP` transfer (see A3-51).
133  /// This is the lowest address accessed within a wrapping
burst.
134  /// This address is aligned to the size and length of the
burst.
135  /// The length of a `BURST_WRAP` has to be 2, 4, 8, or 16
transfers.
136  function automatic largest_addr_t wrap_boundary
(largest_addr_t addr, size_t size, len_t len);
137  largest_addr_t wrap_addr;
138
139  // pragma translate_off
140  `ifndef VERILATOR
141    assume (len == len_t'(4'b1) || len == len_t'(4'b11) ||

```

```

len == len_t('4'b111) ||
140     len == len_t('4'b1111)) else
141     $error("AXI BURST_WRAP with not allowed len of: %0h",
len);
142     `endif
143     // pragma translate_on
144
145     // In A3-51 the wrap boundary is defined as:
146     // `Wrap_Boundary = (INT(Start_Address / (Number_Bytes *
Burst_Length))) *
147     // (Number_Bytes * Burst_Length)`
148     // Whereas the aligned address is defined as:
149     // `Aligned_Address = (INT(Start_Address / Number_Bytes))
* Number_Bytes`
150     // This leads to the wrap boundary using the same
calculation as the aligned address, difference
151     // being the additional dependency on the burst length.
The addition in the case statement
152     // is equal to the multiplication with `Burst_Length` as a
shift (used by `aligned_addr`) is
153     // equivalent with multiplication and division by a power
of two, which conveniently are the
154     // only allowed values for `len` of a `BURST_WRAP`.
155     unique case (len)
156     len_t('4'b1 ) : wrap_addr = (addr >> (unsigned'(size)
+ 1)) << (unsigned'(size) + 1); // multiply `Number_Bytes` by `2`
157     len_t('4'b11 ) : wrap_addr = (addr >> (unsigned'(size)
+ 2)) << (unsigned'(size) + 2); // multiply `Number_Bytes` by `4`
158     len_t('4'b111 ) : wrap_addr = (addr >> (unsigned'(size)
+ 3)) << (unsigned'(size) + 3); // multiply `Number_Bytes` by `8`
159     len_t('4'b1111 ) : wrap_addr = (addr >> (unsigned'(size)
+ 4)) << (unsigned'(size) + 4); // multiply `Number_Bytes` by `16`
160     default : wrap_addr = '0;
161     endcase
162     return wrap_addr;
163 endfunction
164
165     /// Address of beat (see A3-51).
166     function automatic largest_addr_t
167     beat_addr(largest_addr_t addr, size_t size, len_t len,
burst_t burst, shortint unsigned i_beat);
168     largest_addr_t ret_addr = addr;
169     largest_addr_t wrp_bond = '0;
170     if (burst == BURST_WRAP) begin
171     // do not trigger the function if there is no wrapping
burst, to prevent assumptions firing
172     wrp_bond = wrap_boundary(addr, size, len);
173     end
174     if (i_beat != 0 && burst != BURST_FIXED) begin
175     // From A3-51:
176
177     // For an INCR burst, and for a WRAP burst for which the
address has not wrapped, this
178
179     // equation determines the address of any transfer after
the first transfer in a burst:
180     // `Address_N = Aligned_Address + (N - 1) *
Number_Bytes` (N counts from 1 to len!)
181     ret_addr = aligned_addr(addr, size) + i_beat *
num_bytes(size);
182     // From A3-51:
183     // For a WRAP burst, if Address_N = Wrap_Boundary +
(Number_Bytes * Burst_Length), then:
184     // * Use this equation for the current transfer:
185     // `Address_N = Wrap_Boundary`
186     // * Use this equation for any subsequent transfers:
187     // `Address_N = Start_Address + ((N - 1) *
Number_Bytes) - (Number_Bytes * Burst_Length)`
188     // This means that the address calculation of a
`BURST_WRAP` fundamentally works the same
189     // as for a `BURST_INCR`, the difference is when the
calculated address increments
190     // over the wrap threshold, the address wraps around by
subtracting the accessed address
191
192     // space from the normal `BURST_INCR` address. The lower
wrap boundary is equivalent to
193     // The wrap trigger condition minus the container size
(`num_bytes(size) * (len + 1)`).
194     if (burst == BURST_WRAP && ret_addr >= wrp_bond +
(num_bytes(size) * (largest_addr_t'(len) + 1))) begin
195     ret_addr = ret_addr - (num_bytes(size) *
(largest_addr_t'(len) + 1));
196     end
197     end
198     return ret_addr;
199 endfunction
200
201     /// Index of lowest byte in beat (see A3-51).
202     function automatic shortint unsigned
beat_lower_byte(largest_addr_t addr, size_t size, len_t len,
burst_t burst,
203     shortint unsigned strobe_width, shortint unsigned
i_beat);
204     largest_addr_t addr = beat_addr(addr, size, len, burst,
i_beat);
205     return shortint'(($bits(_addr) +
$bits(strobe_width))'(_addr) - (_addr /
largest_addr_t'(strobe_width)) * strobe_width);

```

```

204     endfunction
205
206     /// Index of highest byte in beat (see A3-51).
207     function automatic shortint unsigned
beat_upper_byte(largest_addr_t addr, size_t size, len_t len,
burst_t burst,
208     shortint unsigned strobe_width, shortint unsigned
i_beat);
209     typedef shortint unsigned SU;
210     if (i_beat == 0) begin
211     return SU'(aligned_addr(addr, size) +
(largest_addr_t'(num_bytes(size)) - 1) - (addr /
largest_addr_t'(strobe_width)) * strobe_width);
212     end else begin
213     return beat_lower_byte(addr, size, len, burst,
strobe_width, i_beat) + num_bytes(size) - 1;
214     end
215     endfunction
216
217     /// Is the bufferable bit set?
218     function automatic logic bufferable(cache_t cache);
219     return |(cache & CACHE_BUFFERABLE);
220 endfunction
221
222     /// Is the modifiable bit set?
223     function automatic logic modifiable(cache_t cache);
224     return |(cache & CACHE_MODIFIABLE);
225 endfunction
226
227     /// Memory Type.
228     typedef enum logic [3:0] {
229     DEVICE_NONBUFFERABLE,
230     DEVICE_BUFFERABLE,
231     NORMAL_NONCACHEABLE_NONBUFFERABLE,
232     NORMAL_NONCACHEABLE_BUFFERABLE,
233     WTHRU_NOALLOCATE,
234     WTHRU_RALLOCATE,
235     WTHRU_WALLOCATE,
236     WTHRU_RWALLOCATE,
237     WBACK_NOALLOCATE,
238     WBACK_RALLOCATE,
239     WBACK_WALLOCATE,
240     WBACK_RWALLOCATE
241 } mem_type_t;
242
243     /// Create an `AR_CACHE` field from a `mem_type_t` type.
244     function automatic logic [3:0] get_arscache(mem_type_t
mtype);
245     unique case (mtype)
246     DEVICE_NONBUFFERABLE : return 4'b0000;
247     DEVICE_BUFFERABLE : return 4'b0001;
248     NORMAL_NONCACHEABLE_NONBUFFERABLE : return 4'b0010;
249     NORMAL_NONCACHEABLE_BUFFERABLE : return 4'b0011;
250     WTHRU_NOALLOCATE : return 4'b0100;
251     WTHRU_RALLOCATE : return 4'b0101;
252     WTHRU_WALLOCATE : return 4'b0110;
253     WTHRU_RWALLOCATE : return 4'b0111;
254     WBACK_NOALLOCATE : return 4'b1000;
255     WBACK_RALLOCATE : return 4'b1001;
256     WBACK_WALLOCATE : return 4'b1010;
257     WBACK_RWALLOCATE : return 4'b1011;
258     default : return 4'bxxxx;
259     endcase // mtype
260 endfunction
261
262     /// Create an `AW_CACHE` field from a `mem_type_t` type.
263     function automatic logic [3:0] get_awsache(mem_type_t
mtype);
264     unique case (mtype)
265     DEVICE_NONBUFFERABLE : return 4'b0000;
266     DEVICE_BUFFERABLE : return 4'b0001;
267     NORMAL_NONCACHEABLE_NONBUFFERABLE : return 4'b0010;
268     NORMAL_NONCACHEABLE_BUFFERABLE : return 4'b0011;
269     WTHRU_NOALLOCATE : return 4'b0100;
270     WTHRU_RALLOCATE : return 4'b0101;
271     WTHRU_WALLOCATE : return 4'b0110;
272     WTHRU_RWALLOCATE : return 4'b0111;
273     WBACK_NOALLOCATE : return 4'b1000;
274     WBACK_RALLOCATE : return 4'b1001;
275     WBACK_WALLOCATE : return 4'b1010;
276     WBACK_RWALLOCATE : return 4'b1011;
277     default : return 4'bxxxx;
278     endcase // mtype
279 endfunction
280
281     /// RESP precedence: DECERR > SLVERR > OKAY > EXOKAY. This
is not defined in the AXI standard but
282     /// depends on the implementation. We consistently use the
precedence above. Rationale:
283     /// - EXOKAY means an exclusive access was successful,
whereas OKAY means it was not. Thus, if
284     /// OKAY and EXOKAY are to be merged, OKAY precedes
because the exclusive access was not fully
285     /// successful.
286     /// - Both DECERR and SLVERR mean (part of) a transaction
were unsuccessful, whereas OKAY means an
287     /// entire transaction was successful. Thus both DECERR
and SLVERR precede OKAY.
288     /// - DECERR means (part of) a transactions could not be
routed to a slave component, whereas

```

```

290 /// SLVERR means the transaction reached a slave component
but lead to an error condition there.
291 /// Thus DECERR precedes SLVERR because DECERR happens
earlier in the handling of a transaction.
292 function automatic resp_t resp_precedence(resp_t resp_a,
resp_t resp_b);
293     unique case (resp_a)
294     RESP_OKAY: begin
295         // Any response except EXOKAY precedes OKAY.
296         if (resp_b == RESP_EXOKAY) begin
297             return resp_a;
298         end else begin
299             return resp_b;
300         end
301     end
302     RESP_EXOKAY: begin
303         // Any response precedes EXOKAY.
304         return resp_b;
305     end
306     RESP_SLVERR: begin
307         // Only DECERR precedes SLVERR.
308         if (resp_b == RESP_DECERR) begin
309             return resp_b;
310         end else begin
311             return resp_a;
312         end
313     end
314     RESP_DECERR: begin
315         // No response precedes DECERR.
316         return resp_a;
317     end
318 endcase
319 endfunction
320
321 /// AW Width: Returns the width of the AW channel payload
322 function automatic int unsigned aw_width(int unsigned
addr_width, int unsigned id_width,
323                                     int unsigned
user_width );
324 // Sum the individual bit widths of the signals
325 return (id_width + addr_width + LenWidth + SizeWidth +
BurstWidth + LockWidth + CacheWidth +
326         ProtWidth + QosWidth + RegionWidth + AtopWidth +
user_width );
327 endfunction
328
329 /// W Width: Returns the width of the W channel payload
330 function automatic int unsigned w_width(int unsigned
data_width, int unsigned user_width );
331 // Sum the individual bit widths of the signals
332 return (data_width + data_width / 32'd8 + 32'd1 +
user_width);
333 //
334 //
335 //
336 /// B Width: Returns the width of the B channel payload
337 function automatic int unsigned b_width(int unsigned
id_width, int unsigned user_width );
338 // Sum the individual bit widths of the signals
339 return (id_width + RespWidth + user_width);
340 endfunction
341
342 /// AR Width: Returns the width of the AR channel payload
343 function automatic int unsigned ar_width(int unsigned
addr_width, int unsigned id_width,
344                                     int unsigned
user_width );
345 // Sum the individual bit widths of the signals
346 return (id_width + addr_width + LenWidth + SizeWidth +
BurstWidth + LockWidth + CacheWidth +
347         ProtWidth + QosWidth + RegionWidth + user_width );
348 endfunction
349
350 /// R Width: Returns the width of the R channel payload
351 function automatic int unsigned r_width(int unsigned
data_width, int unsigned id_width,
352                                     int unsigned
user_width );
353 // Sum the individual bit widths of the signals
354 return (id_width + data_width + RespWidth + 32'd1 +
user_width);
355 //
356 //
357 //
358 /// Request Width: Returns the width of the request channel
359 function automatic int unsigned req_width(int unsigned
addr_width, int unsigned data_width,
360                                     int unsigned
id_width, int unsigned aw_user_width,
361                                     int unsigned
ar_user_width, int unsigned w_user_width );
362 // Sum the individual bit widths of the signals and their
handshakes
363 //
364 //
365 //
366 //
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //
748 //
749 //
750 //
751 //
752 //
753 //
754 //
755 //
756 //
757 //
758 //
759 //
760 //
761 //
762 //
763 //
764 //
765 //
766 //
767 //
768 //
769 //
770 //
771 //
772 //
773 //
774 //
775 //
776 //
777 //
778 //
779 //
780 //
781 //
782 //
783 //
784 //
785 //
786 //
787 //
788 //
789 //
790 //
791 //
792 //
793 //
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802 //
803 //
804 //
805 //
806 //
807 //
808 //
809 //
810 //
811 //
812 //
813 //
814 //
815 //
816 //
817 //
818 //
819 //
820 //
821 //
822 //
823 //
824 //
825 //
826 //
827 //
828 //
829 //
830 //
831 //
832 //
833 //
834 //
835 //
836 //
837 //
838 //
839 //
840 //
841 //
842 //
843 //
844 //
845 //
846 //
847 //
848 //
849 //
850 //
851 //
852 //
853 //
854 //
855 //
856 //
857 //
858 //
859 //
860 //
861 //
862 //
863 //
864 //
865 //
866 //
867 //
868 //
869 //
870 //
871 //
872 //
873 //
874 //
875 //
876 //
877 //
878 //
879 //
880 //
881 //
882 //
883 //
884 //
885 //
886 //
887 //
888 //
889 //
890 //
891 //
892 //
893 //
894 //
895 //
896 //
897 //
898 //
899 //
900 //
901 //
902 //
903 //
904 //
905 //
906 //
907 //
908 //
909 //
910 //
911 //
912 //
913 //
914 //
915 //
916 //
917 //
918 //
919 //
920 //
921 //
922 //
923 //
924 //
925 //
926 //
927 //
928 //
929 //
930 //
931 //
932 //
933 //
934 //
935 //
936 //
937 //
938 //
939 //
940 //
941 //
942 //
943 //
944 //
945 //
946 //
947 //
948 //
949 //
950 //
951 //
952 //
953 //
954 //
955 //
956 //
957 //
958 //
959 //
960 //
961 //
962 //
963 //
964 //
965 //
966 //
967 //
968 //
969 //
970 //
971 //
972 //
973 //
974 //
975 //
976 //
977 //
978 //
979 //
980 //
981 //
982 //
983 //
984 //
985 //
986 //
987 //
988 //
989 //
990 //
991 //
992 //
993 //
994 //
995 //
996 //
997 //
998 //
999 //
1000 //

```

```

366         ar_width(addr_width, id_width, ar_user_width) +
32'd1 + 32'd1 + 32'd1 );
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //
748 //
749 //
750 //
751 //
752 //
753 //
754 //
755 //
756 //
757 //
758 //
759 //
760 //
761 //
762 //
763 //
764 //
765 //
766 //
767 //
768 //
769 //
770 //
771 //
772 //
773 //
774 //
775 //
776 //
777 //
778 //
779 //
780 //
781 //
782 //
783 //
784 //
785 //
786 //
787 //
788 //
789 //
790 //
791 //
792 //
793 //
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802 //
803 //
804 //
805 //
806 //
807 //
808 //
809 //
810 //
811 //
812 //
813 //
814 //
815 //
816 //
817 //
818 //
819 //
820 //
821 //
822 //
823 //
824 //
825 //
826 //
827 //
828 //
829 //
830 //
831 //
832 //
833 //
834 //
835 //
836 //
837 //
838 //
839 //
840 //
841 //
842 //
843 //
844 //
845 //
846 //
847 //
848 //
849 //
850 //
851 //
852 //
853 //
854 //
855 //
856 //
857 //
858 //
859 //
860 //
861 //
862 //
863 //
864 //
865 //
866 //
867 //
868 //
869 //
870 //
871 //
872 //
873 //
874 //
875 //
876 //
877 //
878 //
879 //
880 //
881 //
882 //
883 //
884 //
885 //
886 //
887 //
888 //
889 //
890 //
891 //
892 //
893 //
894 //
895 //
896 //
897 //
898 //
899 //
900 //
901 //
902 //
903 //
904 //
905 //
906 //
907 //
908 //
909 //
910 //
911 //
912 //
913 //
914 //
915 //
916 //
917 //
918 //
919 //
920 //
921 //
922 //
923 //
924 //
925 //
926 //
927 //
928 //
929 //
930 //
931 //
932 //
933 //
934 //
935 //
936 //
937 //
938 //
939 //
940 //
941 //
942 //
943 //
944 //
945 //
946 //
947 //
948 //
949 //
950 //
951 //
952 //
953 //
954 //
955 //
956 //
957 //
958 //
959 //
960 //
961 //
962 //
963 //
964 //
965 //
966 //
967 //
968 //
969 //
970 //
971 //
972 //
973 //
974 //
975 //
976 //
977 //
978 //
979 //
980 //
981 //
982 //
983 //
984 //
985 //
986 //
987 //
988 //
989 //
990 //
991 //
992 //
993 //
994 //
995 //
996 //
997 //
998 //
999 //
1000 //

```

```

431  /// Every set bit in the sent data sets the corresponding
432  bit of the data in memory.
433  localparam ATOP_SET = 3'b011;
434  /// The value stored in memory is the maximum of the
435  existing value and sent data. This operation
436  /// assumes signed data.
437  localparam ATOP_SMAX = 3'b100;
438  /// The value stored in memory is the minimum of the
439  existing value and sent data. This operation
440  /// assumes signed data.
441  localparam ATOP_SMIN = 3'b101;
442  /// The value stored in memory is the maximum of the
443  existing value and sent data. This operation
444  /// assumes unsigned data.
445  localparam ATOP_UMAX = 3'b110;
446  /// The value stored in memory is the minimum of the
447  existing value and sent data. This operation
448  /// assumes unsigned data.
449  localparam ATOP_UMIN = 3'b111;
450  /// ATOP[5] == 1'b1 indicated that an atomic transaction has
451  a read response
452  // Usage eg: if (req_i.aw.atop[axi_pkg::ATOP_R_RESP]) begin
453  localparam ATOP_R_RESP = 32'd5;
454
455  // `xbar_latency_e` and `xbar_cfg_t` are documented in `doc/
456  axi_xbar.md`.
457  /// Slice on Demux AW channel.
458  localparam bit [9:0] DemuxAw = (1 << 9);
459  /// Slice on Demux W channel.
460  localparam bit [9:0] DemuxW = (1 << 8);
461  /// Slice on Demux B channel.
462  localparam bit [9:0] DemuxB = (1 << 7);
463  /// Slice on Demux AR channel.
464  localparam bit [9:0] DemuxAr = (1 << 6);
465  /// Slice on Demux R channel.
466  localparam bit [9:0] DemuxR = (1 << 5);
467  /// Slice on Mux AW channel.
468  localparam bit [9:0] MuxAw = (1 << 4);
469  /// Slice on Mux W channel.
470  localparam bit [9:0] MuxW = (1 << 3);
471  /// Slice on Mux B channel.
472  localparam bit [9:0] MuxB = (1 << 2);
473  /// Slice on Mux AR channel.
474  localparam bit [9:0] MuxAr = (1 << 1);
475  /// Slice on Mux R channel.
476  localparam bit [9:0] MuxR = (1 << 0);
477  /// Latency configuration for `axi_xbar`.
478  typedef enum bit [9:0] {
479    NO_LATENCY = 10'b000_00_000_00,
480    CUT_SLV_AX = DemuxAw | DemuxAr,
481    CUT_MST_AX = MuxAw | MuxAr,
482    CUT_ALL_AX = DemuxAw | DemuxAr | MuxAw | MuxAr,
483    CUT_SLV_PORTS = DemuxAw | DemuxW | DemuxB | DemuxAr |
484    DemuxR,
485    CUT_MST_PORTS = MuxAw | MuxW | MuxB | MuxAr | MuxR,
486    CUT_ALL_PORTS = 10'b111_11_111_11
487  } xbar_latency_e;
488
489  /// Configuration for `axi_xbar`.
490  typedef struct packed {
491    /// Number of slave ports of the crossbar.
492    /// This many master modules are connected to it.
493    int unsigned NoSlvPorts;
494    /// Number of master ports of the crossbar.
495    /// This many slave modules are connected to it.
496    int unsigned NoMstPorts;
497    /// Maximum number of open transactions each master
498    connected to the crossbar can have in
499    /// flight at the same time.
500    int unsigned MaxMstTrans;
501    /// Maximum number of open transactions each slave
502    connected to the crossbar can have in
503    /// flight at the same time.
504    int unsigned MaxSlvTrans;
505    /// Determine if the internal FIFOs of the crossbar are
506    instantiated in fallthrough mode.
507    /// 0: No fallthrough
508    /// 1: Fallthrough
509    bit FallThrough;
510    /// The Latency mode of the xbar. This determines if the
511    channels on the ports have
512    /// a spill register instantiated.
513    /// Example configurations are provided with the enum
514    `xbar_latency_e`.
515    bit [9:0] LatencyMode;
516    /// This is the number of `axi_multicut` stages
517    instantiated in the line cross of the channels.
518    /// Having multiple stages can potentially add a large
519    number of FFs!
520    int unsigned PipelineStages;
521    /// AXI ID width of the slave ports. The ID width of the
522    master ports is determined
523    /// automatically. See `axi_mux` for details.
524    int unsigned AxiIdWidthSlvPorts;
525    /// The used ID portion to determine if a different slave
526    is used for the same ID.
527    /// See `axi_demux` for details.
528    int unsigned AxiIdUsedSlvPorts;
529    /// Are IDs unique?
530    bit UniqueIds;
531    /// AXI4+ATOP address field width.

```

```

515  int unsigned AxiAddrWidth;
516  /// AXI4+ATOP data field width.
517  int unsigned AxiDataWidth;
518
519  /// The number of address rules defined for routing of the
520  transactions.
521  /// Each master port can have multiple rules, should have
522  however at least one.
523  /// If a transaction can not be routed the xbar will
524  answer with an `axi_pkg::RESP_DECERR`.
525  int unsigned NoAddrRules;
526  } xbar_cfg_t;
527
528  /// Commonly used rule types for `axi_xbar` (64-bit
529  addresses).
530  typedef struct packed {
531    int unsigned idx;
532    logic [63:0] start_addr;
533    logic [63:0] end_addr;
534  } xbar_rule_64_t;
535
536  /// Commonly used rule types for `axi_xbar` (32-bit
537  addresses).
538  typedef struct packed {
539    int unsigned idx;
540    logic [31:0] start_addr;
541    logic [31:0] end_addr;
542  } xbar_rule_32_t;
543
544  /// Return either the argument minus 1 or 0 if 0; useful for
545  IO vector width declaration
546  function automatic integer unsigned iomsb (input integer
547  unsigned width);
548    return (width != 32'd0) ? unsigned'(width-1) : 32'd0;
549  endfunction
550
551  endpackage

```

A.36 pulp-platform_axi/src/axi_demux.sv (305줄, 11,244B)

```

1  // Copyright (c) 2019 ETH Zurich and University of Bologna.
2  // Copyright and related rights are licensed under the
3  Solderpad Hardware
4
5  // License, Version 0.51 (the "License"); you may not use this
6  file except in
7  // compliance with the License. You may obtain a copy of the
8  License at
9  // http://solderpad.org/licenses/SHL-0.51. Unless required by
10 applicable law
11 // or agreed to in writing, software, hardware and materials
12 distributed under
13 // this License is distributed on an "AS IS" BASIS, WITHOUT
14 WARRANTIES OR
15 // CONDITIONS OF ANY KIND, either express or implied. See the
16 License for the
17 // specific language governing permissions and limitations
18 under the License.
19
20 //
21 // Authors:
22 // - Michael Rogenmoser <michaelr@iis.ee.ethz.ch>
23 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
24 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
25 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
26
27 `include "common_cells/assertions.svh"
28 `include "common_cells/registers.svh"
29
30 `ifdef QUESTA
31  // Derive `TARGET_VSIM`, which is used for tool-specific
32  workarounds in this file, from `QUESTA`,
33  // which is automatically set in Questa.
34  `define TARGET_VSIM
35  `endif
36
37 26 /// Demultiplex one AXI4+ATOP slave port to multiple AXI4+ATOP
38  master ports.
39
40 27 ///
41
42 28 /// The AW and AR slave channels each have a `select` input to
43  determine to which master port the
44 29 /// current request is sent. The `select` can, for example,
45  be driven by an address decoding module
46 30 /// to map address ranges to different AXI slaves.
47 31 ///
48 32 /// ## Design overview
49 33 ///
50 34 /// ![Block diagram](module.axi_demux.png "Block diagram")
51 35 ///
52 36 /// Beats on the W channel are routed by demultiplexer
53 according to the selection for the
54 37 /// corresponding AW beat. This relies on the AXI property
55  that W bursts must be sent in the same
56 38 /// order as AW beats and beats from different W bursts may
57  not be interleaved.
58 39 ///
59 40 /// Beats on the B and R channel are multiplexed from the
60  master ports to the slave port with
61 41 /// a round-robin arbitration tree.

```

```

42 module axi_demux #(
43     parameter int unsigned AxiIdWidth    = 32'd0,
44     parameter bit   AtopSupport          = 1'b1,
45     parameter type   aw_chan_t           = logic,
46     parameter type   w_chan_t            = logic,
47     parameter type   b_chan_t            = logic,
48     parameter type   ar_chan_t           = logic,
49     parameter type   r_chan_t            = logic,
50     parameter type   axi_req_t           = logic,
51     parameter type   axi_resp_t          = logic,
52     parameter int unsigned NoMstPorts     = 32'd0,
53     parameter int unsigned MaxTrans       = 32'd8,
54     parameter int unsigned AxiLookBits    = 32'd3,
55     parameter bit   UniqueIds            = 1'b0,
56     parameter bit   SpillAw              = 1'b1,
57     parameter bit   SpillW               = 1'b0,
58     parameter bit   SpillB               = 1'b0,
59     parameter bit   SpillAr              = 1'b1,
60     parameter bit   SpillR               = 1'b0,
61     // Dependent parameters, DO NOT OVERRIDE!
62     parameter int unsigned SelectWidth    = (NoMstPorts >
32'd1) ? $clog2(NoMstPorts) : 32'd1,
63     parameter type   select_t            = logic
[SelectWidth-1:0]
64 ) (
65     input logic      clk_i,
66     input logic      rst_ni,
67     // Slave Port
68     input axi_req_t  slv_req_i,
69     input select_t   slv_aw_select_i,
70     input select_t   slv_ar_select_i,
71     output axi_resp_t slv_resp_o,
72     // Master Ports
73     output axi_req_t [NoMstPorts-1:0] mst_reqs_o,
74     input axi_resp_t [NoMstPorts-1:0] mst_resps_i
75 );
76
77 axi_req_t slv_req_cut;
78 axi_resp_t slv_resp_cut;
79
80 logic slv_aw_ready_chan, slv_aw_ready_sel;
81 logic slv_aw_valid_chan, slv_aw_valid_sel;
82
83 logic slv_ar_ready_chan, slv_ar_ready_sel;
84 logic slv_ar_valid_chan, slv_ar_valid_sel;
85
86 select_t slv_aw_select, slv_ar_select;
87
88 cc_spill_register #(
89     .data_t ( aw_chan_t ),
90     .Bypass ( ~SpillAw )
91 ) i_aw_spill_reg (
92     .clk_i,
93     .rst_ni,
94     .clr_i ( 1'b0 ),
95     .valid_i ( slv_req_i.aw_valid ),
96     .ready_o ( slv_aw_ready_chan ),
97     .data_i ( slv_req_i.aw ),
98     .valid_o ( slv_aw_valid_chan ),
99     .ready_i ( slv_resp_cut.aw_ready ),
100    .data_o ( slv_req_cut.aw )
101 );
102 cc_spill_register #(
103     .data_t ( select_t ),
104     .Bypass ( ~SpillAw )
105 ) i_aw_select_spill_reg (
106     .clk_i,
107     .rst_ni,
108     .clr_i ( 1'b0 ),
109     .valid_i ( slv_req_i.aw_valid ),
110     .ready_o ( slv_aw_ready_sel ),
111     .data_i ( slv_aw_select_i ),
112     .valid_o ( slv_aw_valid_sel ),
113     .ready_i ( slv_resp_cut.aw_ready ),
114     .data_o ( slv_aw_select )
115 );
116
117 assign slv_resp_o.aw_ready = slv_aw_ready_chan &
slv_aw_ready_sel;
118 assign slv_req_cut.aw_valid = slv_aw_valid_chan &
slv_aw_valid_sel;
119
120 cc_spill_register #(
121     .data_t ( w_chan_t ),
122     .Bypass ( ~SpillW )
123 ) i_w_spill_reg (
124     .clk_i,
125     .rst_ni,
126     .clr_i ( 1'b0 ),
127     .valid_i ( slv_req_i.w_valid ),
128     .ready_o ( slv_resp_o.w_ready ),
129     .data_i ( slv_req_i.w ),
130     .valid_o ( slv_req_cut.w_valid ),
131     .ready_i ( slv_resp_cut.w_ready ),
132     .data_o ( slv_req_cut.w )
133 );
134 cc_spill_register #(
135     .data_t ( ar_chan_t ),
136     .Bypass ( ~SpillAr )
137 ) i_ar_spill_reg (
138     .clk_i,

```

```

139     .rst_ni,
140     .clr_i ( 1'b0 ),
141     .valid_i ( slv_req_i.ar_valid ),
142     .ready_o ( slv_ar_ready_chan ),
143     .data_i ( slv_req_i.ar ),
144     .valid_o ( slv_ar_valid_chan ),
145     .ready_i ( slv_resp_cut.ar_ready ),
146     .data_o ( slv_req_cut.ar )
147 );
148 cc_spill_register #(
149     .data_t ( select_t ),
150     .Bypass ( ~SpillAr )
151 ) i_ar_sel_spill_reg (
152     .clk_i,
153     .rst_ni,
154     .clr_i ( 1'b0 ),
155     .valid_i ( slv_req_i.ar_valid ),
156     .ready_o ( slv_ar_ready_sel ),
157     .data_i ( slv_ar_select_i ),
158     .valid_o ( slv_ar_valid_sel ),
159     .ready_i ( slv_resp_cut.ar_ready ),
160     .data_o ( slv_ar_select )
161 );
162
163 assign slv_resp_o.ar_ready = slv_ar_ready_chan &
slv_ar_ready_sel;
164 assign slv_req_cut.ar_valid = slv_ar_valid_chan &
slv_ar_valid_sel;
165
166 cc_spill_register #(
167     .data_t ( b_chan_t ),
168     .Bypass ( ~SpillB )
169 ) i_b_spill_reg (
170     .clk_i,
171     .rst_ni,
172     .clr_i ( 1'b0 ),
173     .valid_i ( slv_resp_cut.b_valid ),
174     .ready_o ( slv_req_cut.b_ready ),
175     .data_i ( slv_resp_cut.b ),
176     .valid_o ( slv_resp_o.b_valid ),
177     .ready_i ( slv_req_i.b_ready ),
178     .data_o ( slv_resp_o.b )
179 );
180 cc_spill_register #(
181     .data_t ( r_chan_t ),
182     .Bypass ( ~SpillR )
183 ) i_r_spill_reg (
184     .clk_i,
185     .rst_ni,
186     .clr_i ( 1'b0 ),
187     .valid_i ( slv_resp_cut.r_valid ),
188     .ready_o ( slv_req_cut.r_ready ),
189     .data_i ( slv_resp_cut.r ),
190     .valid_o ( slv_resp_o.r_valid ),
191     .ready_i ( slv_req_i.r_ready ),
192     .data_o ( slv_resp_o.r )
193 );
194
195 axi_demux_simple #(
196     .AxiIdWidth ( AxiIdWidth ),
197     .AtopSupport ( AtopSupport ),
198     .axi_req_t ( axi_req_t ),
199     .axi_resp_t ( axi_resp_t ),
200     .NoMstPorts ( NoMstPorts ),
201     .MaxTrans ( MaxTrans ),
202     .AxiLookBits ( AxiLookBits ),
203     .UniqueIds ( UniqueIds )
204 ) i_demux_simple (
205     .clk_i,
206     .rst_ni,
207
208     .slv_req_i ( slv_req_cut ),
209     .slv_aw_select_i ( slv_aw_select ),
210     .slv_ar_select_i ( slv_ar_select ),
211     .slv_resp_o ( slv_resp_cut ),
212     .mst_reqs_o ( mst_reqs_o ),
213     .mst_resps_i ( mst_resps_i )
214 );
215
216 endmodule
217
218 // interface wrapper
219 `include "axi/assign.svh"
220 `include "axi/typedef.svh"
221 module axi_demux_intf #(
222     parameter int unsigned AXI_ID_WIDTH    = 32'd0, // Synopsys
DC requires default value for params
223     parameter bit   ATOP_SUPPORT          = 1'b1,
224     parameter int unsigned AXI_ADDR_WIDTH  = 32'd0,
225     parameter int unsigned AXI_DATA_WIDTH  = 32'd0,
226     parameter int unsigned AXI_USER_WIDTH  = 32'd0,
227     parameter int unsigned NO_MST_PORTS    = 32'd3,
228     parameter int unsigned MAX_TRANS       = 32'd8,
229     parameter int unsigned AXI_LOOK_BITS   = 32'd3,
230     parameter bit   UNIQUE_IDS            = 1'b0,
231     parameter bit   SPILL_AW              = 1'b1,
232     parameter bit   SPILL_W               = 1'b0,
233     parameter bit   SPILL_B               = 1'b0,
234     parameter bit   SPILL_AR              = 1'b1,
235     parameter bit   SPILL_R               = 1'b0,

```

```

236 // Dependent parameters, DO NOT OVERRIDE!
237 parameter int unsigned SELECT_WIDTH = (NO_MST_PORTS >
32'd1) ? $clog2(NO_MST_PORTS) : 32'd1,
238 parameter type select_t = logic
[SELECT_WIDTH-1:0] // MST port select type
239 ) (
240 input logic clk_i, // Clock
241 input logic rst_ni, // Asynchronous reset
active low
242 input select_t slv_aw_select_i, // has to be stable,
when aw_valid
243 input select_t slv_ar_select_i, // has to be stable,
when ar_valid
244 AXI_BUS.Slave slv, // slave port
245 AXI_BUS.Master mst [NO_MST_PORTS-1:0] // master ports
246 );
247
248 typedef logic [AXI_ID_WIDTH-1:0] id_t;
249 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
250 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
251 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
252 typedef logic [AXI_USER_WIDTH-1:0] user_t;
253 `AXI_TPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
254 `AXI_TPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
255 `AXI_TPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
256 `AXI_TPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
257 `AXI_TPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
258 `AXI_TPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
259 `AXI_TPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
260
261 axi_req_t slv_req;
262 axi_resp_t slv_resp;
263 axi_req_t [NO_MST_PORTS-1:0] mst_req;
264 axi_resp_t [NO_MST_PORTS-1:0] mst_resp;
265
266 `AXI_ASSIGN_TO_REQ(slv_req, slv)
267 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
268
269 for (genvar i = 0; i < NO_MST_PORTS; i++) begin :
gen_assign_mst_ports
270 `AXI_ASSIGN_FROM_REQ(mst[i], mst_req[i])
271 `AXI_ASSIGN_TO_RESP(mst_resp[i], mst[i])
272 end
273
274 axi_demux #(
275 .AxiIdWidth ( AXI_ID_WIDTH ), // ID Width
276 .AtopSupport ( ATOP_SUPPORT ),
277 .aw_chan_t ( aw_chan_t ), // AW Channel Type
278 .w_chan_t ( w_chan_t ), // W Channel Type
279 .b_chan_t ( b_chan_t ), // B Channel Type
280 .ar_chan_t ( ar_chan_t ), // AR Channel Type
281 .r_chan_t ( r_chan_t ), // R Channel Type
282 .axi_req_t ( axi_req_t ),
283 .axi_resp_t ( axi_resp_t ),
284 .NoMstPorts ( NO_MST_PORTS ),
285 .MaxTrans ( MAX_TRANS ),
286 .AxiLookBits ( AXI_LOOK_BITS ),
287 .UniqueIds ( UNIQUE_IDS ),
288 .SpillAw ( SPILL_AW ),
289 .SpillW ( SPILL_W ),
290 .SpillB ( SPILL_B ),
291 .SpillAr ( SPILL_AR ),
292 .SpillR ( SPILL_R )
293 ) i_axi_demux (
294 .clk_i, // Clock
295 .rst_ni, // Asynchronous reset active low
296 // slave port
297 .slv_req_i ( slv_req ),
298 .slv_aw_select_i ( slv_aw_select_i ),
299 .slv_ar_select_i ( slv_ar_select_i ),
300 .slv_resp_o ( slv_resp ),
301 // master port
302 .mst_reqs_o ( mst_req ),
303 .mst_resps_i ( mst_resp )
304 );
305 endmodule

```

A.37 pulp-platform_axi/src/axi_mux.sv (605줄, 25,317B)

```

1 // Copyright (c) 2019 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the
Solderpad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations
under the License.

```

```

11 //
12 // Authors:
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 // AXI Multiplexer: This module multiplexes the AXI4 slave
ports down to one master port.
17 // The AXI IDs from the slave ports get extended with the
respective slave port index.
18 // The extension width can be calculated with
`$clog2(NoSlvPorts)`. This means the AXI
19 // ID for the master port has to be this `$clog2(NoSlvPorts)`
wider than the ID for the
20 // slave ports.
21 // Responses are switched based on these bits. For example,
with 4 slave ports
22 // a response with ID `6'b100110` will be forwarded to slave
port 2 (`2'b10`).
23
24 // register macros
25 `include "common_cells/assertions.svh"
26 `include "common_cells/registers.svh"
27
28 module axi_mux #(
29 // AXI parameter and channel types
30 parameter int unsigned SlvAxiIdWidth = 32'd0, // AXI ID
width, slave ports
31 parameter type slv_aw_chan_t = logic, // AW Channel
Type, slave ports
32 parameter type mst_aw_chan_t = logic, // AW Channel
Type, master port
33 parameter type w_chan_t = logic, // W Channel
Type, all ports
34 parameter type slv_b_chan_t = logic, // B Channel
Type, slave ports
35 parameter type mst_b_chan_t = logic, // B Channel
Type, master port
36 parameter type slv_ar_chan_t = logic, // AR Channel
Type, slave ports
37 parameter type mst_ar_chan_t = logic, // AR Channel
Type, master port
38 parameter type slv_r_chan_t = logic, // R Channel
Type, slave ports
39 parameter type mst_r_chan_t = logic, // R Channel
Type, master port
40 parameter type slv_req_t = logic, // Slave port
request type
41 parameter type slv_resp_t = logic, // Slave port
response type
42 parameter type mst_req_t = logic, // Master
ports request type
43 parameter type mst_resp_t = logic, // Master
ports response type
44 parameter int unsigned NoSlvPorts = 32'd0, // Number of
slave ports
45 // Maximum number of outstanding transactions per write
parameter int unsigned MaxWTrans = 32'd8,
46 // If enabled, this multiplexer is purely combinatorial
parameter bit FallThrough = 1'b0,
47 // add spill register on write master ports, adds a cycle
latency on write channels
48 parameter bit SpillAw = 1'b1,
49 parameter bit SpillW = 1'b0,
50 parameter bit SpillB = 1'b0,
51 parameter bit SpillAr = 1'b1,
52 parameter bit SpillR = 1'b0
53 // add spill register on read master ports, adds a cycle
latency on read channels
54 parameter bit SpillAr = 1'b1,
55 parameter bit SpillR = 1'b0
56 ) (
57 input logic clk_i, // Clock
58 input logic rst_ni, // Asynchronous
reset active low
59 // slave ports (AXI inputs), connect master modules here
60 input slv_req_t [NoSlvPorts-1:0] slv_reqs_i,
61 output slv_resp_t [NoSlvPorts-1:0] slv_resps_o,
62 // master port (AXI outputs), connect slave modules here
63 output mst_req_t mst_req_o,
64 input mst_resp_t mst_resp_i
65 );
66
67 localparam int unsigned MstIdxBits = $clog2(NoSlvPorts);
68 localparam int unsigned MstAxiIdWidth = SlvAxiIdWidth +
MstIdxBits;
69
70 // pass through if only one slave port
71 if (NoSlvPorts == 32'h1) begin : gen_no_mux
72 cc_spill_register #(
73 .data_t ( mst_aw_chan_t ),
74 .Bypass ( ~SpillAw )
75 ) i_aw_spill_reg (
76 .clk_i ( clk_i ),
77 .rst_ni ( rst_ni ),
78 .clr_i ( 1'b0 ),
79 .valid_i ( slv_reqs_i[0].aw_valid ),
80 .ready_o ( slv_resps_o[0].aw_ready ),
81 .data_i ( slv_reqs_i[0].aw ),
82 .valid_o ( mst_req_o.aw_valid ),
83 .ready_i ( mst_resp_i.aw_ready ),
84 .data_o ( mst_req_o.aw )
85 );

```

```

86 cc_spill_register #(
87   .data_t ( w_chan_t ),
88   .Bypass ( ~SpillW )
89 ) i_w_spill_reg (
90   .clk_i ( clk_i ),
91   .rst_ni ( rst_ni ),
92   .clr_i ( 1'b0 ),
93   .valid_i ( slv_reqs_i[0].w_valid ),
94   .ready_o ( slv_resps_o[0].w_ready ),
95   .data_i ( slv_reqs_i[0].w ),
96   .valid_o ( mst_req_o.w_valid ),
97   .ready_i ( mst_resp_i.w_ready ),
98   .data_o ( mst_req_o.w )
99 );
100 cc_spill_register #(
101   .data_t ( mst_b_chan_t ),
102   .Bypass ( ~SpillB )
103 ) i_b_spill_reg (
104   .clk_i ( clk_i ),
105   .rst_ni ( rst_ni ),
106   .clr_i ( 1'b0 ),
107   .valid_i ( mst_resp_i.b_valid ),
108   .ready_o ( mst_req_o.b_ready ),
109   .data_i ( mst_resp_i.b ),
110   .valid_o ( slv_resps_o[0].b_valid ),
111   .ready_i ( slv_reqs_i[0].b_ready ),
112   .data_o ( slv_resps_o[0].b )
113 );
114 cc_spill_register #(
115   .data_t ( mst_ar_chan_t ),
116   .Bypass ( ~SpillAr )
117 ) i_ar_spill_reg (
118   .clk_i ( clk_i ),
119   .rst_ni ( rst_ni ),
120   .clr_i ( 1'b0 ),
121   .valid_i ( slv_reqs_i[0].ar_valid ),
122   .ready_o ( slv_resps_o[0].ar_ready ),
123   .data_i ( slv_reqs_i[0].ar ),
124   .valid_o ( mst_req_o.ar_valid ),
125   .ready_i ( mst_resp_i.ar_ready ),
126   .data_o ( mst_req_o.ar )
127 );
128 cc_spill_register #(
129   .data_t ( mst_r_chan_t ),
130   .Bypass ( ~SpillR )
131 ) i_r_spill_reg (
132   .clk_i ( clk_i ),
133   .rst_ni ( rst_ni ),
134   .clr_i ( 1'b0 ),
135   .valid_i ( mst_resp_i.r_valid ),
136   .ready_o ( mst_req_o.r_ready ),
137   .data_i ( mst_resp_i.r ),
138   .valid_o ( slv_resps_o[0].r_valid ),
139   .ready_i ( slv_reqs_i[0].r_ready ),
140   .data_o ( slv_resps_o[0].r )
141 );
142 // Validate parameters.
143 // pragma translate_off
144 `ASSERT_INIT(CorrectIdWidthSrvAw,
145 $bits(slv_reqs_i[0].aw.id) == SrvAxIDWidth)
146 `ASSERT_INIT(CorrectIdWidthSrvB,
147 $bits(slv_resps_o[0].b.id) == SrvAxIDWidth)
148 `ASSERT_INIT(CorrectIdWidthSrvAr,
149 $bits(slv_reqs_i[0].ar.id) == SrvAxIDWidth)
150 `ASSERT_INIT(CorrectIdWidthSrvR,
151 $bits(slv_resps_o[0].r.id) == SrvAxIDWidth)
152 `ASSERT_INIT(CorrectIdWidthMstAw, $bits(mst_req_o.aw.id)
153 == SrvAxIDWidth)
154 `ASSERT_INIT(CorrectIdWidthMstB, $bits(mst_resp_i.b.id) ==
155 SrvAxIDWidth)
156 `ASSERT_INIT(CorrectIdWidthMstAr, $bits(mst_req_o.ar.id)
157 == SrvAxIDWidth)
158 `ASSERT_INIT(CorrectIdWidthMstR, $bits(mst_resp_i.r.id) ==
159 SrvAxIDWidth)
160 // pragma translate_on
161 // other non degenerate cases
162 end else begin : gen_mux
163   typedef logic [MstIdxBits-1:0] switch_id_t;
164   // AXI channels between the ID prepend unit and the rest
165   of the multiplexer
166   mst_aw_chan_t [NoSrvPorts-1:0] slv_aw_chans;
167   logic [NoSrvPorts-1:0] slv_aw_valids,
168   slv_w_readies;
169   w_chan_t [NoSrvPorts-1:0] slv_w_chans;
170   logic [NoSrvPorts-1:0] slv_w_valids,
171   slv_b_readies;
172   mst_b_chan_t [NoSrvPorts-1:0] slv_b_chans;
173   logic [NoSrvPorts-1:0] slv_b_valids,
174   slv_ar_readies;
175   mst_ar_chan_t [NoSrvPorts-1:0] slv_ar_chans;
176   logic [NoSrvPorts-1:0] slv_ar_valids,
177   slv_r_readies;
178   mst_r_chan_t [NoSrvPorts-1:0] slv_r_chans;
179   logic [NoSrvPorts-1:0] slv_r_valids,
180   slv_r_readies;
181 end

```

```

171 // These signals are all ID prepended
172 // AW channel
173 mst_aw_chan_t mst_aw_chan;
174 logic mst_aw_valid, mst_aw_ready;
175
176 // AW master handshake internal, so that we are able to
177 stall, if w_fifo is full
178 logic aw_valid, aw_ready;
179 // FF to lock the AW valid signal, when a new arbitration
180 decision is made the decision
181
182 // gets pushed into the W FIFO, when it now stalls prevent
183 subsequent pushing
184 // This FF removes AW to W dependency
185 logic lock_aw_valid_d, lock_aw_valid_q;
186 logic load_aw_lock;
187
188 // signals for the FIFO that holds the last switching
189 decision of the AW channel
190 logic w_fifo_full, w_fifo_empty;
191 logic w_fifo_push, w_fifo_pop;
192 switch_id_t w_fifo_data;
193
194 // W channel spill reg
195 w_chan_t mst_w_chan;
196 logic mst_w_valid, mst_w_ready;
197
198 // master ID in the b_id
199 switch_id_t switch_b_id;
200
201 // B channel spill reg
202 mst_b_chan_t mst_b_chan;
203 logic mst_b_valid;
204
205 // AR channel for when spill is enabled
206 mst_ar_chan_t mst_ar_chan;
207 logic ar_valid, ar_ready;
208
209 // master ID in the r_id
210 switch_id_t switch_r_id;
211
212 // R channel spill reg
213 mst_r_chan_t mst_r_chan;
214 logic mst_r_valid;
215
216 //-----
217 // ID prepend for all slave ports
218 //-----
219 for (genvar i = 0; i < NoSrvPorts; i++) begin :
220 gen_id_prepend
221   axi_id_prepend #(
222     .NoBus ( 32'd1 ), // one AXI
223     bus_per_slave_port
224     .AxIDWidthSrvPort( SrvAxIDWidth ),
225     .AxIDWidthMstPort( MstAxIDWidth ),
226     .slv_aw_chan_t ( slv_aw_chan_t ),
227     .slv_w_chan_t ( w_chan_t ),
228     .slv_b_chan_t ( slv_b_chan_t ),
229     .slv_ar_chan_t ( slv_ar_chan_t ),
230     .slv_r_chan_t ( slv_r_chan_t ),
231     .mst_aw_chan_t ( mst_aw_chan_t ),
232     .mst_w_chan_t ( w_chan_t ),
233     .mst_b_chan_t ( mst_b_chan_t ),
234     .mst_ar_chan_t ( mst_ar_chan_t ),
235     .mst_r_chan_t ( mst_r_chan_t )
236   ) i_id_prepend (
237     .pre_id_i ( switch_id_t'(i) ),
238     .slv_aw_chans_i ( slv_reqs_i[i].aw ),
239     .slv_aw_valids_i ( slv_reqs_i[i].aw_valid ),
240     .slv_w_readies_o ( slv_resps_o[i].w_ready ),
241     .slv_w_chans_i ( slv_reqs_i[i].w ),
242     .slv_w_valids_i ( slv_reqs_i[i].w_valid ),
243     .slv_w_readies_o ( slv_resps_o[i].w_ready ),
244     .slv_b_chans_o ( slv_resps_o[i].b ),
245     .slv_b_valids_o ( slv_resps_o[i].b_valid ),
246     .slv_b_readies_i ( slv_reqs_i[i].b_ready ),
247     .slv_ar_chans_i ( slv_reqs_i[i].ar ),
248     .slv_ar_valids_i ( slv_reqs_i[i].ar_valid ),
249     .slv_ar_readies_o ( slv_resps_o[i].ar_ready ),
250     .slv_r_chans_o ( slv_resps_o[i].r ),
251     .slv_r_valids_o ( slv_resps_o[i].r_valid ),
252     .slv_r_readies_i ( slv_reqs_i[i].r_ready ),
253     .mst_aw_chans_o ( slv_aw_chans[i] ),
254     .mst_w_valids_o ( slv_w_valids[i] ),
255     .mst_w_readies_i ( slv_w_readies[i] ),
256     .mst_b_chans_i ( slv_b_chans[i] ),
257     .mst_b_valids_i ( slv_b_valids[i] ),
258     .mst_b_readies_o ( slv_b_readies[i] ),
259     .mst_ar_chans_o ( slv_ar_chans[i] ),
260     .mst_ar_valids_o ( slv_ar_valids[i] ),
261     .mst_ar_readies_i ( slv_ar_readies[i] ),
262     .mst_r_chans_i ( slv_r_chans[i] ),
263     .slv_r_valids_i ( slv_r_valids[i] ),
264     .mst_r_readies_o ( slv_r_readies[i] )
265   );
266 end

```

```

265 //-----
266 // AW Channel
267 //-----
268 cc_rr_arb_tree #(
269     .NumIn      ( NoSlvPorts  ),
270     .data_t      ( mst_aw_chan_t ),
271     .AxiVldRdy( 1'b1 ),
272     .LockIn      ( 1'b1 )
273 ) i_aw_arbiter (
274     .clk_i      ( clk_i ),
275     .rst_ni     ( rst_ni ),
276     .clr_i      ( 1'b0 ),
277     .rr_i       ( '0 ),
278     .req_i      ( slv_aw_valids ),
279     .gnt_o      ( slv_aw_readies ),
280     .data_i      ( slv_aw_chans ),
281     .gnt_i      ( aw_ready ),
282     .req_o      ( aw_valid ),
283     .data_o      ( mst_aw_chan ),
284     .idx_o      ( )
285 );
286
287 // control of the AW channel
288 always_comb begin
289     // default assignments
290     lock_aw_valid_d = lock_aw_valid_q;
291     load_aw_lock    = 1'b0;
292     w_fifo_push     = 1'b0;
293     mst_aw_valid    = 1'b0;
294     aw_ready        = 1'b0;
295     // had a downstream stall, be valid and send the AW
296     if (lock_aw_valid_q) begin
297         mst_aw_valid = 1'b1;
298         // transaction
299         if (mst_aw_ready) begin
300             aw_ready    = 1'b1;
301             lock_aw_valid_d = 1'b0;
302             load_aw_lock  = 1'b1;
303         end
304     end else begin
305         if (!w_fifo_full && aw_valid) begin
306             mst_aw_valid = 1'b1;
307             w_fifo_push  = 1'b1;
308             if (mst_aw_ready) begin
309                 aw_ready = 1'b1;
310             end else begin
311                 // go to lock if transaction not in this cycle
312                 lock_aw_valid_d = 1'b1;
313                 load_aw_lock    = 1'b1;
314             end
315         end
316     end
317 end
318
319 `FFL(lock_aw_valid_q, lock_aw_valid_d, load_aw_lock, '0,
clk_i, rst_ni)
320
321 cc_fifo #(
322     .FallThrough ( FallThrough ),
323     .Depth        ( MaxWTrans ),
324     .data_t       ( switch_id_t )
325 ) i_w_fifo (
326     .clk_i
327     ( clk_i ),
328     .rst_ni
329     ( rst_ni ),
330     .clr_i
331     ( 1'b0 ),
332     .flush_i
333     ( 1'b0 ),
334     .full_o
335     ( w_fifo_full ),
336     .empty_o
337     ( w_fifo_empty ),
338     .usage_o
339     ( ),
340     .data_i
341     ( mst_aw_chan.id[SlvAxiIDWidth+:MstIdxBits] ),
342     .push_i
343     ( w_fifo_push ),
344     .data_o
345     ( w_fifo_data ),
346     .pop_i
347     ( w_fifo_pop ),
348     );
349
350 cc_spill_register #(
351     .data_t      ( mst_aw_chan_t ),
352     .Bypass      ( ~SpillAw ) // Param indicated that we
353 want a spill reg
354 ) i_aw_spill_reg (
355     .clk_i      ( clk_i ),
356     .rst_ni     ( rst_ni ),
357     .clr_i      ( 1'b0 ),
358     .valid_i    ( mst_aw_valid ),
359     .ready_o    ( mst_aw_ready ),
360     .data_i     ( mst_aw_chan ),
361     .valid_o    ( mst_req_o.aw_valid ),
362     .ready_i    ( mst_resp_i.aw_ready ),
363     .data_o     ( mst_req_o.aw ),
364 );

```

```

353 //-----
354 // W Channel
355 //-----
356 // multiplexer
357 assign mst_w_chan = slv_w_chans[w_fifo_data];
358 always_comb begin
359     // default assignments
360     mst_w_valid = 1'b0;
361     slv_w_readies = '0;
362     w_fifo_pop   = 1'b0;
363     // control
364     if (!w_fifo_empty) begin
365         // connect the handshake
366         mst_w_valid =
367         slv_w_valids[w_fifo_data];
368         slv_w_readies[w_fifo_data] = mst_w_ready;
369         // pop FIFO on a last transaction
370         w_fifo_pop = slv_w_valids[w_fifo_data] & mst_w_ready &
371         mst_w_chan.last;
372     end
373 end
374
375 cc_spill_register #(
376     .data_t      ( w_chan_t ),
377     .Bypass      ( ~SpillW )
378 ) i_w_spill_reg (
379     .clk_i      ( clk_i ),
380     .rst_ni     ( rst_ni ),
381     .clr_i      ( 1'b0 ),
382     .valid_i    ( mst_w_valid ),
383     .ready_o    ( mst_w_ready ),
384     .data_i     ( mst_w_chan ),
385     .valid_o    ( mst_req_o.w_valid ),
386     .ready_i    ( mst_resp_i.w_ready ),
387     .data_o     ( mst_req_o.w ),
388 );
389
390 //-----
391 // B Channel
392 //-----
393 // replicate B channels
394 assign slv_b_chans = {NoSlvPorts{mst_b_chan}};
395 // control B channel handshake
396 assign switch_b_id =
397 mst_b_chan.id[SlvAxiIDWidth+:MstIdxBits];
398
399 assign slv_b_valids = (mst_b_valid) ? (1 << switch_b_id) :
400 '0;
401
402 cc_spill_register #(
403     .data_t      ( mst_b_chan_t ),
404     .Bypass      ( ~SpillB )
405 ) i_b_spill_reg (
406     .clk_i      ( clk_i ),
407     .rst_ni     ( rst_ni ),
408     .clr_i      ( 1'b0 ),
409     .valid_i    ( mst_resp_i.b_valid ),
410     .ready_o    ( mst_req_o.b_ready ),
411     .data_i     ( mst_resp_i.b ),
412     .valid_o    ( mst_b_valid ),
413     .ready_i    ( slv_b_readies[switch_b_id] ),
414     .data_o     ( mst_b_chan ),
415 );
416
417 //-----
418 // AR Channel
419 //-----
420 cc_rr_arb_tree #(
421     .NumIn      ( NoSlvPorts  ),
422     .data_t      ( mst_ar_chan_t ),
423     .AxiVldRdy( 1'b1 ),
424     .LockIn      ( 1'b1 )
425 ) i_ar_arbiter (
426     .clk_i      ( clk_i ),
427     .rst_ni     ( rst_ni ),
428     .clr_i      ( 1'b0 ),
429     .rr_i       ( '0 ),
430     .req_i      ( slv_ar_valids ),
431     .gnt_o      ( slv_ar_readies ),
432     .data_i      ( slv_ar_chans ),
433     .gnt_i      ( ar_ready ),
434     .req_o      ( ar_valid ),
435     .data_o      ( mst_ar_chan ),
436     .idx_o      ( )
437 );
438
439 cc_spill_register #(
440     .data_t      ( mst_ar_chan_t ),
441     .Bypass      ( ~SpillAr )
442 ) i_ar_spill_reg (
443     .clk_i      ( clk_i ),
444     .rst_ni     ( rst_ni ),
445     .clr_i      ( 1'b0 ),
446     .valid_i    ( ar_valid ),
447     .ready_o    ( ar_ready ),
448     .data_i     ( mst_ar_chan ),
449     .valid_o    ( mst_req_o.ar_valid ),
450     .ready_i    ( mst_resp_i.ar_ready ),
451     .data_o     ( mst_req_o.ar ),
452 );

```

```

448 );
449 //-----
450 // R Channel
451 //-----
452 // replicate R channels
453 assign slv_r_chans = {NoSlvPorts{mst_r_chan}};
454 // R channel handshake control
455 assign switch_r_id =
mst_r_chan.id[SlvAxiIDWidth+:MstIdxBits];
457 assign slv_r_valids = (mst_r_valid) ? (1 << switch_r_id) :
'0;
458
459 cc_spill_register #(
460   .data_t ( mst_r_chan_t ),
461   .Bypass ( ~SpillR )
462 ) i_r_spill_reg (
463   .clk_i ( clk_i ),
464   .rst_ni ( rst_ni ),
465   .clr_i ( 1'b0 ),
466   .valid_i ( mst_resp_i.r_valid ),
467   .ready_o ( mst_req_o.r_ready ),
468   .data_i ( mst_resp_i.r ),
469   .valid_o ( mst_r_valid ),
470   .ready_i ( slv_r_readies[switch_r_id] ),
471   .data_o ( mst_r_chan )
472 );
473 end
474
475 // pragma translate_off
476 `ifndef VERILATOR
477   initial begin
478     assert (SlvAxiIDWidth > 0) else $fatal(1, "AXI ID width of
slave ports must be non-zero!");
479     assert (NoSlvPorts > 0) else $fatal(1, "Number of slave
ports must be non-zero!");
480     assert (MaxWTrans > 0)
481     else $fatal(1, "Maximum number of outstanding writes
must be non-zero!");
482     assert (MstAxiIDWidth >= SlvAxiIDWidth +
$log2(NoSlvPorts))
483     else $fatal(1, "AXI ID width of master ports must be
wide enough to identify slave ports!");
484     // Assert ID widths (one slave is sufficient since they
all have the same type).
485     assert ($unsigned($bits(slv_reqs_i[0].aw.id)) ==
SlvAxiIDWidth)
486     else $fatal(1, "ID width of AW channel of slave ports
does not match parameter!");
487     assert ($unsigned($bits(slv_reqs_i[0].ar.id)) ==
SlvAxiIDWidth)
488     else $fatal(1, "ID width of AR channel of slave ports
does not match parameter!");
489     assert ($unsigned($bits(slv_resps_o[0].b.id)) ==
SlvAxiIDWidth)
490     else $fatal(1, "ID width of B channel of slave ports
does not match parameter!");
491     assert ($unsigned($bits(slv_resps_o[0].r.id)) ==
SlvAxiIDWidth)
492     else $fatal(1, "ID width of R channel of slave ports
does not match parameter!");
493     assert ($unsigned($bits(mst_req_o.aw.id)) ==
MstAxiIDWidth)
494     else $fatal(1, "ID width of AW channel of master port is
wrong!");
495     assert ($unsigned($bits(mst_req_o.ar.id)) ==
MstAxiIDWidth)
496     else $fatal(1, "ID width of AR channel of master port is
wrong!");
497     assert ($unsigned($bits(mst_resp_i.b.id)) ==
MstAxiIDWidth)
498     else $fatal(1, "ID width of B channel of master port is
wrong!");
499     assert ($unsigned($bits(mst_resp_i.r.id)) ==
MstAxiIDWidth)
500     else $fatal(1, "ID width of R channel of master port is
wrong!");
501   end
502 `endif
503 // pragma translate_on
504 endmodule
505
506 // interface wrap
507 `include "axi/assign.svh"
508 `include "axi/typedef.svh"
509 module axi_mux_intf #(
510   parameter int unsigned SLV_AXI_ID_WIDTH = 32'd0, // Synopsys
DC requires default value for params
511   parameter int unsigned MST_AXI_ID_WIDTH = 32'd0,
512   parameter int unsigned AXI_ADDR_WIDTH = 32'd0,
513   parameter int unsigned AXI_DATA_WIDTH = 32'd0,
514   parameter int unsigned AXI_USER_WIDTH = 32'd0,
515   parameter int unsigned NO_SLV_PORTS = 32'd0, // Number
of slave ports
516   // Maximum number of outstanding transactions per write
517   parameter int unsigned MAX_W_TRANS = 32'd8,

```

```

518 // if enabled, this multiplexer is purely combinatorial
519 parameter bit FALL_THROUGH = 1'b0,
520 // add spill register on write master ports, adds a cycle
latency on write channels
521 parameter bit SPILL_AW = 1'b1,
522 parameter bit SPILL_W = 1'b0,
523 parameter bit SPILL_B = 1'b0,
524 // add spill register on read master ports, adds a cycle
latency on read channels
525 parameter bit SPILL_AR = 1'b1,
526 parameter bit SPILL_R = 1'b0,
527 ) (
528   input logic clk_i, // Clock
529   input logic rst_ni, // Asynchronous reset
active low
530   AXI_BUS.Slave slv [NO_SLV_PORTS-1:0], // slave ports
531   AXI_BUS.Master mst // master port
532 );
533
534 typedef logic [SLV_AXI_ID_WIDTH-1:0] slv_id_t;
535 typedef logic [MST_AXI_ID_WIDTH-1:0] mst_id_t;
536 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
537 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
538 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
539 typedef logic [AXI_USER_WIDTH-1:0] user_t;
540 // channels typedef
541 `AXI_TYPEDEF_AW_CHAN_T(slv_aw_chan_t, addr_t, slv_id_t,
user_t)
542 `AXI_TYPEDEF_AW_CHAN_T(mst_aw_chan_t, addr_t, mst_id_t,
user_t)
543
544 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
545
546 `AXI_TYPEDEF_B_CHAN_T(slv_b_chan_t, slv_id_t, user_t)
547 `AXI_TYPEDEF_B_CHAN_T(mst_b_chan_t, mst_id_t, user_t)
548
549 `AXI_TYPEDEF_AR_CHAN_T(slv_ar_chan_t, addr_t, slv_id_t,
user_t)
550 `AXI_TYPEDEF_AR_CHAN_T(mst_ar_chan_t, addr_t, mst_id_t,
user_t)
551
552 `AXI_TYPEDEF_R_CHAN_T(slv_r_chan_t, data_t, slv_id_t,
user_t)
553 `AXI_TYPEDEF_R_CHAN_T(mst_r_chan_t, data_t, mst_id_t,
user_t)
554
555 `AXI_TYPEDEF_REQ_T(slv_req_t, slv_aw_chan_t, w_chan_t,
slv_ar_chan_t)
556 `AXI_TYPEDEF_RESP_T(slv_resp_t, slv_b_chan_t, slv_r_chan_t)
557
558 `AXI_TYPEDEF_REQ_T(mst_req_t, mst_aw_chan_t, w_chan_t,
mst_ar_chan_t)
559 `AXI_TYPEDEF_RESP_T(mst_resp_t, mst_b_chan_t, mst_r_chan_t)
560
561 slv_req_t [NO_SLV_PORTS-1:0] slv_reqs;
562 slv_resp_t [NO_SLV_PORTS-1:0] slv_resps;
563 mst_req_t mst_req;
564 mst_resp_t mst_resp;
565
566 for (genvar i = 0; i < NO_SLV_PORTS; i++) begin :
gen_assign_slv_ports
567   `AXI_ASSIGN_TO_REQ(slv_reqs[i], slv[i])
568   `AXI_ASSIGN_FROM_RESP(slv[i], slv_resps[i])
569 end
570
571 `AXI_ASSIGN_FROM_REQ(mst, mst_req)
572 `AXI_ASSIGN_TO_RESP(mst_resp, mst)
573
574 axi_mux #(
575   .SlvAxiIDWidth ( SLV_AXI_ID_WIDTH ),
576   .slv_aw_chan_t ( slv_aw_chan_t ), // AW Channel Type,
slave ports
577   .mst_aw_chan_t ( mst_aw_chan_t ), // AW Channel Type,
master port
578   .w_chan_t ( w_chan_t ), // W Channel Type,
all ports
579   .slv_b_chan_t ( slv_b_chan_t ), // B Channel Type,
slave ports
580   .mst_b_chan_t ( mst_b_chan_t ), // B Channel Type,
master port
581   .slv_ar_chan_t ( slv_ar_chan_t ), // AR Channel Type,
slave ports
582   .mst_ar_chan_t ( mst_ar_chan_t ), // AR Channel Type,
master port
583   .slv_r_chan_t ( slv_r_chan_t ), // R Channel Type,
slave ports
584   .mst_r_chan_t ( mst_r_chan_t ), // R Channel Type,
master port
585   .slv_req_t ( slv_req_t ),
586   .slv_resp_t ( slv_resp_t ),
587   .mst_req_t ( mst_req_t ),
588   .mst_resp_t ( mst_resp_t ),
589   .NoSlvPorts ( NO_SLV_PORTS ), // Number of slave
ports
590   .MaxWTrans ( MAX_W_TRANS ),
591   .FallThrough ( FALL_THROUGH ),
592   .SpillAw ( SPILL_AW ),
593   .SpillW ( SPILL_W ),
594   .SpillB ( SPILL_B ),
595   .SpillAr ( SPILL_AR ),

```

```

596 .SpillR      ( SPILL_R      )
597 ) i_axi_mux (
598   .clk_i      ( clk_i      ), // Clock
599   .rst_ni     ( rst_ni     ), // Asynchronous reset active
600   .slv_reqs_i ( slv_reqs_i ),
601   .slv_resps_o ( slv_resps_o ),
602   .mst_req_o  ( mst_req_o  ),
603   .mst_resp_i ( mst_resp_i ),
604 );
605 endmodule

```

A.38 pulp-platform_axi/src/axi_xbar.sv (246줄, 11,764B)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the
Solderpad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations
under the License.
10 //
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
13 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
14 // - Florian Zaruba <zarubaf@iis.ee.ethz.ch>
15
16 /// axi_xbar: Fully-connected AXI4+ATOP crossbar with an
arbitrary number of slave and master ports.
17
18 /// See `doc/axi_xbar.md` for the documentation, including the
definition of parameters and ports.
19 module axi_xbar
20 import cc_pkg::idx_width;
21 #(
22   /// Configuration struct for the crossbar see `axi_pkg` for
fields and definitions.
23   parameter axi_pkg::xbar_cfg_t = '0,
24   /// Enable atomic operations support.
25   parameter bit = 1'b1,
26   ATOPs = 1'b1,
27   /// Connectivity matrix
28   parameter bit [Cfg.NoSlvPorts-1:0][Cfg.NoMstPorts-1:0]
Connectivity = '1,
29   /// AXI4+ATOP AW channel struct type for the slave ports.
30   parameter type
31   slv_aw_chan_t = logic,
32   /// AXI4+ATOP AW channel struct type for the master ports.
33   parameter type
34   mst_aw_chan_t = logic,
35   /// AXI4+ATOP W channel struct type for all ports.
36   parameter type
37   w_chan_t = logic,
38   /// AXI4+ATOP B channel struct type for the slave ports.
39   parameter type
40   slv_b_chan_t = logic,
41   /// AXI4+ATOP B channel struct type for the master ports.
42   parameter type
43   mst_b_chan_t = logic,
44   /// AXI4+ATOP AR channel struct type for the slave ports.
45   parameter type
46   slv_ar_chan_t = logic,
47   /// AXI4+ATOP AR channel struct type for the master ports.
48   parameter type
49   mst_ar_chan_t = logic,
50   /// AXI4+ATOP R channel struct type for the slave ports.
51   parameter type
52   slv_r_chan_t = logic,
53   /// AXI4+ATOP R channel struct type for the master ports.
54   parameter type
55   mst_r_chan_t = logic,
56   /// AXI4+ATOP request struct type for the slave ports.
57   parameter type
58   slv_req_t = logic,
59   /// AXI4+ATOP response struct type for the slave ports.
60   parameter type
61   slv_resp_t = logic,
62   /// AXI4+ATOP request struct type for the master ports.
63   parameter type
64   mst_req_t = logic,
65   /// AXI4+ATOP response struct type for the master ports.
66   parameter type
67   mst_resp_t = logic,
68   /// Address rule type for the address decoders from
`common_cells:addr_decode`.
69   /// Example types are provided in `axi_pkg`.
70   /// Required struct fields:
71   ///

```

```

57 /// typedef struct packed {
58 ///   int unsigned idx;
59 ///   axi_addr_t start_addr;
60 ///   axi_addr_t end_addr;
61 /// } rule_t;
62 /// ...
63 parameter type
rule_t =
axi_pkg::xbar_rule_64_t,
64 localparam int unsigned MstPortsIdxWidth =
65   (Cfg.NoMstPorts == 32'd1) ? 32'd1 :
unsigned'($clog2(Cfg.NoMstPorts))
66 ) (
67   /// Clock, positive edge triggered.
68   input
69   clk_i,
70   /// Asynchronous reset, active low.
71   input
72   rst_ni,
73   /// AXI4+ATOP requests to the slave ports.
74   input slv_req_t
75   [Cfg.NoSlvPorts-1:0] slv_ports_req_i,
76   /// AXI4+ATOP responses of the slave ports.
77   output slv_resp_t
78   [Cfg.NoSlvPorts-1:0] slv_ports_resp_o,
79   /// AXI4+ATOP requests to the master ports.
80   output mst_req_t
81   [Cfg.NoMstPorts-1:0] mst_ports_req_o,
82   /// AXI4+ATOP responses to the master ports.
83   input mst_resp_t
84   [Cfg.NoMstPorts-1:0] mst_ports_resp_i,
85   /// Address map array input for the crossbar. This map is
global for the whole module.
86   /// It is used for routing the transactions to the
respective master ports.
87   /// Each master port can have multiple different rules.
88   input rule_t
89   [Cfg.NoAddrRules-1:0] addr_map_i,
90   /// Enable default master port.
91   input logic
92   [Cfg.NoSlvPorts-1:0] en_default_mst_port_i,
93   /// Enables a default master port for each slave port. When
this is enabled unmapped
94   /// transactions get issued at the master port given by
`default_mst_port_i`.
95   /// When not used, tie to `0`.
96   input logic [Cfg.NoSlvPorts-1:0]
97   [MstPortsIdxWidth-1:0] default_mst_port_i
98 );
99
100 // signals into the axi_muxes, are of type slave as the
multiplexer extends the ID
101 slv_req_t [Cfg.NoMstPorts-1:0][Cfg.NoSlvPorts-1:0]
102 mst_reqs;
103 slv_resp_t [Cfg.NoMstPorts-1:0][Cfg.NoSlvPorts-1:0]
104 mst_resps;
105
106 axi_xbar_unmuxed #(
107   .Cfg (Cfg),
108   .ATOPs (ATOPs),
109   .Connectivity (Connectivity),
110   .aw_chan_t (slv_aw_chan_t),
111   .w_chan_t (w_chan_t),
112   .b_chan_t (slv_b_chan_t),
113   .ar_chan_t (slv_ar_chan_t),
114   .r_chan_t (slv_r_chan_t),
115   .req_t (slv_req_t),
116   .resp_t (slv_resp_t),
117   .rule_t (rule_t)
118 ) i_xbar_unmuxed (
119   .clk_i,
120   .rst_ni,
121   .slv_ports_req_i,
122   .slv_ports_resp_o,
123   .mst_ports_req_o (mst_reqs),
124   .mst_ports_resp_i (mst_resps),
125   .addr_map_i,
126   .en_default_mst_port_i,
127   .default_mst_port_i
128 );
129
130 for (genvar i = 0; i < Cfg.NoMstPorts; i++) begin :
gen_mst_port_mux
131   axi_mux #(
132     .SlvAxIDWidth ( Cfg.AxiIDWidthSlvPorts ), // ID width
of the slave ports
133     .slv_aw_chan_t ( slv_aw_chan_t ), // AW Channel
Type, slave ports
134     .mst_aw_chan_t ( mst_aw_chan_t ), // AW Channel
Type, master port
135     .w_chan_t ( w_chan_t ), // W Channel
Type, all ports

```

```

125     .slv_b_chan_t ( slv_b_chan_t          ), // B Channel
Type, slave ports

126     .mst_b_chan_t ( mst_b_chan_t          ), // B Channel
Type, master port

127     .slv_ar_chan_t ( slv_ar_chan_t        ), // AR Channel
Type, slave ports

128     .mst_ar_chan_t ( mst_ar_chan_t        ), // AR Channel
Type, master port

129     .slv_r_chan_t ( slv_r_chan_t          ), // R Channel
Type, slave ports

130     .mst_r_chan_t ( mst_r_chan_t          ), // R Channel
Type, master port
131     .slv_req_t     ( slv_req_t             ),
132     .slv_resp_t    ( slv_resp_t            ),
133     .mst_req_t     ( mst_req_t             ),
134     .mst_resp_t    ( mst_resp_t            ),
135     .NoSlvPorts    ( Cfg.NoSlvPorts        ), // Number of
Masters for the module
136     .MaxWTrans     ( Cfg.MaxSlvTrans       ),
137     .FallThrough   ( Cfg.FallThrough       ),
138     .SpillAw        ( Cfg.LatencyMode[4]    ),
139     .SpillW         ( Cfg.LatencyMode[3]    ),
140     .SpillB         ( Cfg.LatencyMode[2]    ),
141     .SpillAr        ( Cfg.LatencyMode[1]    ),
142     .SpillR         ( Cfg.LatencyMode[0]    )
143 ) i_axi_mux (
144     .clk_i, // Clock
145     .rst_ni, // Asynchronous reset active low
146     .slv_reqs_i ( mst_reqs[i] ),
147     .slv_resps_o ( mst_resps[i] ),
148     .mst_req_o ( mst_ports_req_o[i] ),
149     .mst_resp_i ( mst_ports_resp_i[i] )
150 );
151 end
152
153 endmodule
154
155 `include "axi/assign.svh"
156 `include "axi/typedef.svh"
157
158 module axi_xbar_intf
159 import cc_pkg::idx_width;
160 #(
161     parameter int unsigned AXI_USER_WIDTH = 0,
162     parameter axi_pkg::xbar_cfg_t Cfg = '0,
163     parameter bit ATOPS = 1'b1,
164     parameter bit [Cfg.NoSlvPorts-1:0] [Cfg.NoMstPorts-1:0]
CONNECTIVITY = '1,
165     parameter type rule_t =
axi_pkg::xbar_rule_64_t,
166     localparam int unsigned MstPortsIdxWidth =
167         (Cfg.NoMstPorts == 32'd1) ? 32'd1 :
unsigned'($clog2(Cfg.NoMstPorts))
168 ) (
169     input
170     logic                                     clk_i,
171     logic                                     rst_ni,
172     AXI_BUS.Slave
slv_ports [Cfg.NoSlvPorts-1:0],
173     AXI_BUS.Master
mst_ports [Cfg.NoMstPorts-1:0],
174     input rule_t
[Cfg.NoAddrRules-1:0]
175     input logic
[Cfg.NoSlvPorts-1:0]
176     en_default_mst_port_i,
177     input logic
[Cfg.NoSlvPorts-1:0]
178     [MstPortsIdxWidth-1:0]
default_mst_port_i
179 );
180 localparam int unsigned AxiIdWidthMstPorts =
Cfg.AxiIdWidthSlvPorts + $clog2(Cfg.NoSlvPorts);
181
182 typedef logic [AxiIdWidthMstPorts-1:0] id_mst_t;
183 typedef logic [Cfg.AxiIdWidthSlvPorts-1:0] id_slv_t;
184 typedef logic [Cfg.AxiAddrWidth-1:0] addr_t;
185 typedef logic [Cfg.AxiDataWidth-1:0] data_t;
186 typedef logic [Cfg.AxiDataWidth/8-1:0] strb_t;
187 typedef logic [AXI_USER_WIDTH-1:0] user_t;
188
189 `AXI_TYPEDEF_AW_CHAN_T(mst_aw_chan_t, addr_t, id_mst_t,
user_t)
190 `AXI_TYPEDEF_AW_CHAN_T(slv_aw_chan_t, addr_t, id_slv_t,
user_t)
191 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
192 `AXI_TYPEDEF_B_CHAN_T(mst_b_chan_t, id_mst_t, user_t)
193 `AXI_TYPEDEF_B_CHAN_T(slv_b_chan_t, id_slv_t, user_t)
194 `AXI_TYPEDEF_AR_CHAN_T(mst_ar_chan_t, addr_t, id_mst_t,
user_t)
195 `AXI_TYPEDEF_AR_CHAN_T(slv_ar_chan_t, addr_t, id_slv_t,
user_t)
196 `AXI_TYPEDEF_R_CHAN_T(mst_r_chan_t, data_t, id_mst_t,
user_t)
197 `AXI_TYPEDEF_R_CHAN_T(slv_r_chan_t, data_t, id_slv_t,
user_t)

```

```

195 `AXI_TYPEDEF_R_CHAN_T(slv_r_chan_t, data_t, id_slv_t,
user_t)
196 `AXI_TYPEDEF_REQ_T(mst_req_t, mst_aw_chan_t, w_chan_t,
mst_ar_chan_t)
197 `AXI_TYPEDEF_REQ_T(slv_req_t, slv_aw_chan_t, w_chan_t,
slv_ar_chan_t)
198 `AXI_TYPEDEF_RESP_T(mst_resp_t, mst_b_chan_t, mst_r_chan_t)
199 `AXI_TYPEDEF_RESP_T(slv_resp_t, slv_b_chan_t, slv_r_chan_t)
200
201 mst_req_t [Cfg.NoMstPorts-1:0] mst_reqs;
202 mst_resp_t [Cfg.NoMstPorts-1:0] mst_resps;
203 slv_req_t [Cfg.NoSlvPorts-1:0] slv_reqs;
204 slv_resp_t [Cfg.NoSlvPorts-1:0] slv_resps;
205
206 for (genvar i = 0; i < Cfg.NoMstPorts; i++) begin :
gen_assign_mst
207     `AXI_ASSIGN_FROM_REQ(mst_ports[i], mst_reqs[i])
208     `AXI_ASSIGN_TO_RESP(mst_resps[i], mst_ports[i])
209 end
210
211 for (genvar i = 0; i < Cfg.NoSlvPorts; i++) begin :
gen_assign_slv
212     `AXI_ASSIGN_TO_REQ(slv_reqs[i], slv_ports[i])
213     `AXI_ASSIGN_FROM_RESP(slv_ports[i], slv_resps[i])
214 end
215
216 axi_xbar #(
217     .Cfg (Cfg),
218     .ATOPS ( ATOPS ),
219     .Connectivity ( CONNECTIVITY ),
220     .slv_aw_chan_t ( slv_aw_chan_t ),
221     .mst_aw_chan_t ( mst_aw_chan_t ),
222     .w_chan_t ( w_chan_t ),
223     .slv_b_chan_t ( slv_b_chan_t ),
224     .mst_b_chan_t ( mst_b_chan_t ),
225     .slv_ar_chan_t ( slv_ar_chan_t ),
226     .mst_ar_chan_t ( mst_ar_chan_t ),
227     .slv_r_chan_t ( slv_r_chan_t ),
228     .mst_r_chan_t ( mst_r_chan_t ),
229     .slv_req_t ( slv_req_t ),
230     .slv_resp_t ( slv_resp_t ),
231     .mst_req_t ( mst_req_t ),
232     .mst_resp_t ( mst_resp_t ),
233     .rule_t ( rule_t )
234 ) i_xbar (
235     .clk_i,
236     .rst_ni,
237     .slv_ports_req_i (slv_reqs),
238     .slv_ports_resp_o (slv_resps),
239     .mst_ports_req_o (mst_reqs),
240     .mst_ports_resp_i (mst_resps),
241     .addr_map_i,
242     .en_default_mst_port_i,
243     .default_mst_port_i
244 );
245
246 endmodule

```

A.39 pulp-platform_axi/src/axi_cut.sv (295줄, 9,826B)

```

1 // Copyright (c) 2014-2018 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the
Solderpad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations
under the License.
11 //
12 // Authors:
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
15 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
16
17 /// An AXI4 cut.
18 ///
19 /// Breaks all combinatorial paths between its input and
output.
20 module axi_cut #(
21     // bypass enable
22     parameter bit Bypass = 1'b0,
23     parameter bit BypassAw = Bypass,
24     parameter bit BypassW = Bypass,
25     parameter bit BypassB = Bypass,
26     parameter bit BypassAr = Bypass,
27     parameter bit BypassR = Bypass,
28     // AXI channel structs
29     parameter type aw_chan_t = logic,
30     parameter type w_chan_t = logic,

```

```

31 parameter type    b_chan_t = logic,
32 parameter type    ar_chan_t = logic,
33 parameter type    r_chan_t = logic,
34 // AXI request & response structs
35 parameter type    axi_req_t = logic,
36 parameter type    axi_resp_t = logic
37 ) (
38   input logic      clk_i,
39   input logic      rst_ni,
40   // slave port
41   input axi_req_t  slv_req_i,
42   output axi_resp_t slv_resp_o,
43   // master port
44   output axi_req_t mst_req_o,
45   input axi_resp_t mst_resp_i
46 );
47
48 // a spill register for each channel
49 cc_spill_register #(
50   .data_t ( aw_chan_t ),
51   .Bypass ( BypassAw )
52 ) i_reg_aw (
53   .clk_i ( clk_i ),
54   .rst_ni ( rst_ni ),
55   .clr_i ( 1'b0 ),
56   .valid_i ( slv_req_i.aw_valid ),
57   .ready_o ( slv_resp_o.aw_ready ),
58   .data_i ( slv_req_i.aw ),
59   .valid_o ( mst_req_o.aw_valid ),
60   .ready_i ( mst_resp_i.aw_ready ),
61   .data_o ( mst_req_o.aw )
62 );
63
64 cc_spill_register #(
65   .data_t ( w_chan_t ),
66   .Bypass ( BypassW )
67 ) i_reg_w (
68   .clk_i ( clk_i ),
69   .rst_ni ( rst_ni ),
70   .clr_i ( 1'b0 ),
71   .valid_i ( slv_req_i.w_valid ),
72   .ready_o ( slv_resp_o.w_ready ),
73   .data_i ( slv_req_i.w ),
74   .valid_o ( mst_req_o.w_valid ),
75   .ready_i ( mst_resp_i.w_ready ),
76   .data_o ( mst_req_o.w )
77 );
78
79 cc_spill_register #(
80   .data_t ( b_chan_t ),
81   .Bypass ( BypassB )
82 ) i_reg_b (
83   .clk_i ( clk_i ),
84   .rst_ni ( rst_ni ),
85   .clr_i ( 1'b0 ),
86   .valid_i ( mst_resp_i.b_valid ),
87   .ready_o ( mst_req_o.b_ready ),
88   .data_i ( mst_resp_i.b ),
89   .valid_o ( slv_resp_o.b_valid ),
90   .ready_i ( slv_req_i.b_ready ),
91   .data_o ( slv_resp_o.b )
92 );
93
94 cc_spill_register #(
95   .data_t ( ar_chan_t ),
96   .Bypass ( BypassAr )
97 ) i_reg_ar (
98   .clk_i ( clk_i ),
99   .rst_ni ( rst_ni ),
100   .clr_i ( 1'b0 ),
101   .valid_i ( slv_req_i.ar_valid ),
102   .ready_o ( slv_resp_o.ar_ready ),
103   .data_i ( slv_req_i.ar ),
104   .valid_o ( mst_req_o.ar_valid ),
105   .ready_i ( mst_resp_i.ar_ready ),
106   .data_o ( mst_req_o.ar )
107 );
108
109 cc_spill_register #(
110   .data_t ( r_chan_t ),
111   .Bypass ( BypassR )
112 ) i_reg_r (
113   .clk_i ( clk_i ),
114   .rst_ni ( rst_ni ),
115   .clr_i ( 1'b0 ),
116   .valid_i ( mst_resp_i.r_valid ),
117   .ready_o ( mst_req_o.r_ready ),
118   .data_i ( mst_resp_i.r ),
119   .valid_o ( slv_resp_o.r_valid ),
120   .ready_i ( slv_req_i.r_ready ),
121   .data_o ( slv_resp_o.r )
122 );
123 endmodule
124
125 `include "axi/assign.svh"
126 `include "axi/typedef.svh"
127
128 // interface wrapper
129 module axi_cut_intf #(
130   // Bypass enable
131   parameter bit    BYPASS    = 1'b0,

```

```

132   parameter bit    BYPASS_AW = BYPASS,
133   parameter bit    BYPASS_W  = BYPASS,
134   parameter bit    BYPASS_B  = BYPASS,
135   parameter bit    BYPASS_AR = BYPASS,
136   parameter bit    BYPASS_R  = BYPASS,
137   // The address width.
138   parameter int unsigned ADDR_WIDTH = 0,
139   // The data width.
140   parameter int unsigned DATA_WIDTH = 0,
141   // The ID width.
142   parameter int unsigned ID_WIDTH  = 0,
143   // The user data width.
144   parameter int unsigned USER_WIDTH = 0
145 ) (
146   input logic      clk_i ,
147   input logic      rst_ni ,
148   AXI_BUS.Slave   in_ ,
149   AXI_BUS.Master  out_ ,
150 );
151
152 typedef logic [ID_WIDTH-1:0]    id_t;
153 typedef logic [ADDR_WIDTH-1:0]  addr_t;
154 typedef logic [DATA_WIDTH-1:0]  data_t;
155 typedef logic [DATA_WIDTH/8-1:0] strb_t;
156 typedef logic [USER_WIDTH-1:0]  user_t;
157
158 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
159 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
160 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
161 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
162 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
163 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
164 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
165
166 axi_req_t slv_req, mst_req;
167 axi_resp_t slv_resp, mst_resp;
168
169 `AXI_ASSIGN_TO_REQ(slv_req, in)
170 `AXI_ASSIGN_FROM_RESP(in, slv_resp)
171
172 `AXI_ASSIGN_FROM_REQ(out, mst_req)
173 `AXI_ASSIGN_TO_RESP(mst_resp, out)
174
175 axi_cut #(
176   .Bypass ( BYPASS ),
177   .BypassAw ( BYPASS_AW ),
178   .BypassW ( BYPASS_W ),
179   .BypassB ( BYPASS_B ),
180   .BypassAr ( BYPASS_AR ),
181   .BypassR ( BYPASS_R ),
182   .aw_chan_t ( aw_chan_t ),
183   .w_chan_t ( w_chan_t ),
184   .b_chan_t ( b_chan_t ),
185   .ar_chan_t ( ar_chan_t ),
186   .r_chan_t ( r_chan_t ),
187   .axi_req_t ( axi_req_t ),
188   .axi_resp_t ( axi_resp_t )
189 ) i_axi_cut (
190   .clk_i,
191   .rst_ni,
192   .slv_req_i ( slv_req ),
193   .slv_resp_o ( slv_resp ),
194   .mst_req_o ( mst_req ),
195   .mst_resp_i ( mst_resp )
196 );
197
198 // Check the invariants.
199 // pragma translate_off
200 `ifndef VERILATOR
201   initial begin
202     assert (ADDR_WIDTH > 0) else $fatal(1, "Wrong addr width
parameter");
203     assert (DATA_WIDTH > 0) else $fatal(1, "Wrong data width
parameter");
204     assert (ID_WIDTH > 0) else $fatal(1, "Wrong id width
parameter");
205     assert (USER_WIDTH > 0) else $fatal(1, "Wrong user width
parameter");
206     assert (in.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
207     assert (in.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
208     assert (in.AXI_ID_WIDTH == ID_WIDTH) else $fatal(1,
"Wrong interface definition");
209     assert (in.AXI_USER_WIDTH == USER_WIDTH) else $fatal(1,
"Wrong interface definition");
210     assert (out.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
211     assert (out.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
212     assert (out.AXI_ID_WIDTH == ID_WIDTH) else $fatal(1,
"Wrong interface definition");
213     assert (out.AXI_USER_WIDTH == USER_WIDTH) else $fatal(1,
"Wrong interface definition");
214   end
215 `endif
216 // pragma translate_on
217 endmodule
218
219 module axi_lite_cut_intf #(

```

```

220 // bypass enable
221 parameter bit          BYPASS    = 1'b0,
222 parameter bit          BYPASS_AW = BYPASS,
223 parameter bit          BYPASS_W  = BYPASS,
224 parameter bit          BYPASS_B  = BYPASS,
225 parameter bit          BYPASS_AR = BYPASS,
226 parameter bit          BYPASS_R  = BYPASS,
227 /// The address width.
228 parameter int unsigned ADDR_WIDTH = 0,
229 /// The data width.
230 parameter int unsigned DATA_WIDTH = 0
231 ) (
232   input logic   clk_i ,
233   input logic   rst_ni ,
234   AXI_LITE.Slave in ,
235   AXI_LITE.Master out
236 );
237
238 typedef logic [ADDR_WIDTH-1:0]  addr_t;
239 typedef logic [DATA_WIDTH-1:0]  data_t;
240 typedef logic [DATA_WIDTH/8-1:0] strb_t;
241
242 `AXI_LITE_TYPEDEF_AW_CHAN_T(addr_t)
243 `AXI_LITE_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
244 `AXI_LITE_TYPEDEF_B_CHAN_T(b_chan_t)
245 `AXI_LITE_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
246 `AXI_LITE_TYPEDEF_R_CHAN_T(r_chan_t, data_t)
247 `AXI_LITE_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
248 `AXI_LITE_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
249
250 axi_req_t  slv_req, mst_req;
251 axi_resp_t slv_resp, mst_resp;
252
253 `AXI_LITE_ASSIGN_TO_REQ(slv_req, in)
254 `AXI_LITE_ASSIGN_FROM_RESP(in, slv_resp)
255
256 `AXI_LITE_ASSIGN_FROM_REQ(out, mst_req)
257 `AXI_LITE_ASSIGN_TO_RESP(mst_resp, out)
258
259 axi_cut #(
260   .Bypass    ( BYPASS ),
261   .BypassAw  ( BYPASS_AW ),
262   .BypassW   ( BYPASS_W ),
263   .BypassB   ( BYPASS_B ),
264   .BypassAr  ( BYPASS_AR ),
265   .BypassR   ( BYPASS_R ),
266   .aw_chan_t ( aw_chan_t ),
267   .w_chan_t  ( w_chan_t ),
268   .b_chan_t  ( b_chan_t ),
269   .ar_chan_t ( ar_chan_t ),
270   .r_chan_t  ( r_chan_t ),
271   .axi_req_t ( axi_req_t ),
272   .axi_resp_t ( axi_resp_t )
273 ) i_axi_cut (
274   .clk_i,
275   .rst_ni,
276   .slv_req_i ( slv_req ),
277   .slv_resp_o ( slv_resp ),
278   .mst_req_o  ( mst_req ),
279   .mst_resp_i ( mst_resp )
280 );
281
282 // Check the invariants.
283 // pragma translate_off
284 `ifndef VERILATOR
285   initial begin
286     assert (ADDR_WIDTH > 0) else $fatal(1, "Wrong addr width
parameter");
287     assert (DATA_WIDTH > 0) else $fatal(1, "Wrong data width
parameter");
288     assert (in.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
289     assert (in.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
290     assert (out.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
291     assert (out.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
292   end
293   `endif
294 // pragma translate_on
295 endmodule

```

A.40 pulp-platform_axi/src/axi_err_slv.sv (260줄, 8,461B)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the
Solderpad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR

```

```

8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations
under the License.
10 //
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
13 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
14 // - Matheus Cavalcante <matheusd@iis.ee.ethz.ch>
15
16 // AXI Error Slave: This module always responds with an AXI
error for transactions that are sent to
17 // it. This module optionally supports ATOPs if the `ATOPs`
parameter is set.
18
19 module axi_err_slv #(
20   parameter int unsigned      AxiIdWidth =
0,
21   parameter type              axi_req_t =
logic,
22   parameter type              axi_resp_t =
logic,
23   parameter axi_pkg::resp_t   Resp      =
axi_pkg::RESP_DECERR, // Error generated by this slave.
24   parameter int unsigned      RespWidth =
32'd64,
25   parameter logic [RespWidth-1:0] RespData =
64'hCA11AB1EBADCAB1E, // Hexvalue for data return value
26   parameter bit               ATOPs      =
1'b1,
27   parameter int unsigned      MaxTrans   =
1
28 ) (
29   input logic   clk_i, // Clock
30   input logic   rst_ni, // Asynchronous reset active low
31   // slave port
32   input axi_req_t slv_req_i,
33   output axi_resp_t slv_resp_o
34 );
35 typedef logic [AxiIdWidth-1:0] id_t;
36 typedef struct packed {
37   id_t id;
38   axi_pkg::len_t len;
39 } r_data_t;
40
41 axi_req_t err_req;
42 axi_resp_t err_resp;
43
44 if (ATOPs) begin
45   axi_atop_filter #(
46     .AxiIdWidth      ( AxiIdWidth ),
47     .AxiMaxWriteTxns  ( MaxTrans ),
48     .axi_req_t        ( axi_req_t ),
49     .axi_resp_t        ( axi_resp_t )
50   ) i_atop_filter (
51     .clk_i,
52     .rst_ni,
53     .slv_req_i ( slv_req_i ),
54     .slv_resp_o ( slv_resp_o ),
55     .mst_req_o  ( err_req ),
56     .mst_resp_i ( err_resp )
57   );
58 end else begin
59   assign err_req  = slv_req_i;
60   assign slv_resp_o = err_resp;
61 end
62
63 // w fifo
64 logic w_fifo_full, w_fifo_empty;
65 logic w_fifo_push, w_fifo_pop;
66 id_t w_fifo_data;
67 // b fifo
68 logic b_fifo_full, b_fifo_empty;
69 logic b_fifo_push, b_fifo_pop;
70 id_t b_fifo_data;
71 // r fifo
72 r_data_t r_fifo_inp;
73 logic r_fifo_full, r_fifo_empty;
74 logic r_fifo_push, r_fifo_pop;
75 r_data_t r_fifo_data;
76 // r counter
77 logic r_cnt_clear, r_cnt_en, r_cnt_load;
78 axi_pkg::len_t r_current_beat;
79 // r status
80 logic r_busy_d, r_busy_q, r_busy_load;
81
82 //-----
83 // Write Transactions
84 //-----
85 // push, when there is room in the fifo
86 assign w_fifo_push = err_req.aw_valid & ~w_fifo_full;
87 assign err_resp.aw_ready = ~w_fifo_full;
88
89 cc_fifo #(
90   .FallThrough ( 1'b1 ),
91   .Depth        ( MaxTrans ),
92   .data_t        ( id_t )
93 ) i_w_fifo (

```

```

94   .clk_i      ( clk_i      ),
95   .rst_ni     ( rst_ni     ),
96   .clr_i      ( 1'b0      ),
97   .flush_i    ( 1'b0      ),
98   .full_o     ( w_fifo_full ),
99   .empty_o    ( w_fifo_empty ),
100  .usage_o    (             ),
101  .data_i      ( err_req.aw.id ),
102  .push_i     ( w_fifo_push ),
103  .data_o     ( w_fifo_data ),
104  .pop_i      ( w_fifo_pop  ),
105 );
106
107 always_comb begin : proc_w_channel
108   err_resp.w_ready = 1'b0;
109   w_fifo_pop       = 1'b0;
110   b_fifo_push      = 1'b0;
111   if (!w_fifo_empty && !b_fifo_full) begin
112     // eat the beats
113     err_resp.w_ready = 1'b1;
114     // on the last w transaction
115     if (err_req.w_valid && err_req.w.last) begin
116       w_fifo_pop = 1'b1;
117       b_fifo_push = 1'b1;
118     end
119   end
120 end
121
122 cc_fifo #(
123   .FallThrough ( 1'b0 ),
124   .Depth        ( unsigned'(2) ), // two placed so that w can
eat beats if b is not sent
125   .data_t       ( id_t       )
126 ) i_b_fifo (
127   .clk_i      ( clk_i      ),
128   .rst_ni     ( rst_ni     ),
129   .clr_i      ( 1'b0      ),
130   .flush_i    ( 1'b0      ),
131   .full_o     ( b_fifo_full ),
132   .empty_o    ( b_fifo_empty ),
133   .usage_o    (             ),
134   .data_i     ( w_fifo_data ),
135   .push_i     ( b_fifo_push ),
136   .data_o     ( b_fifo_data ),
137   .pop_i      ( b_fifo_pop  ),
138 );
139
140 always_comb begin : proc_b_channel
141   b_fifo_pop = 1'b0;
142   err_resp.b = '0;
143   err_resp.b.id = b_fifo_data;
144   err_resp.b.resp = Resp;
145   err_resp.b.valid = 1'b0;
146   if (!b_fifo_empty) begin
147     err_resp.b.valid = 1'b1;
148     // b transaction
149     b_fifo_pop = err_req.b_ready;
150   end
151 end
152
153 //-----
154 // Read Transactions
155 //-----
156 // push if there is room in the fifo
157 assign r_fifo_push = err_req.ar_valid & ~r_fifo_full;
158 assign err_resp.ar_ready = ~r_fifo_full;
159
160 // fifo data assignment
161 assign r_fifo_inp.id = err_req.ar.id;
162 assign r_fifo_inp.len = err_req.ar.len;
163
164 cc_fifo #(
165   .FallThrough ( 1'b0 ),
166   .Depth        ( MaxTrans ),
167   .data_t       ( r_data_t   )
168 ) i_r_fifo (
169   .clk_i      ( clk_i      ),
170   .rst_ni     ( rst_ni     ),
171   .clr_i      ( 1'b0      ),
172   .flush_i    ( 1'b0      ),
173   .full_o     ( r_fifo_full ),
174   .empty_o    ( r_fifo_empty ),
175   .usage_o    (             ),
176   .data_i     ( r_fifo_inp  ),
177   .push_i     ( r_fifo_push ),
178   .data_o     ( r_fifo_data ),
179   .pop_i      ( r_fifo_pop  ),
180 );
181
182 always_comb begin : proc_r_channel
183   // default assignments
184   r_busy_d = r_busy_q;
185   r_busy_load = 1'b0;
186   // r fifo signals
187   r_fifo_pop = 1'b0;
188   // r counter signals
189   r_cnt_clear = 1'b0;
190   r_cnt_en = 1'b0;
191   r_cnt_load = 1'b0;
192   // r_channel

```

```

193   err_resp.r      = '0;
194   err_resp.r.id   = r_fifo_data.id;
195   err_resp.r.data = RespData;
196   err_resp.r.resp = Resp;
197   err_resp.r.valid = 1'b0;
198   // control
199   if (r_busy_q) begin
200     err_resp.r.valid = 1'b1;
201     err_resp.r.last = (r_current_beat == '0);
202     // r transaction
203     if (err_req.r_ready) begin
204       r_cnt_en = 1'b1;
205       if (r_current_beat == '0) begin
206         r_busy_d = 1'b0;
207         r_busy_load = 1'b1;
208         r_cnt_clear = 1'b1;
209         r_fifo_pop = 1'b1;
210       end
211     end
212   end else begin
213     // when not busy and fifo not empty, start counter err
214   gen
215     if (!r_fifo_empty) begin
216       r_busy_d = 1'b1;
217       r_busy_load = 1'b1;
218       r_cnt_load = 1'b1;
219     end
220   end
221
222   always_ff @(posedge clk_i, negedge rst_ni) begin
223     if (!rst_ni) begin
224       r_busy_q <= '0;
225     end else if (r_busy_load) begin
226       r_busy_q <= r_busy_d;
227     end
228   end
229
230   cc_counter #(
231     .Width ($bits(axi_pkg::len_t))
232   ) i_r_counter (
233     .clk_i      ( clk_i      ),
234     .rst_ni     ( rst_ni     ),
235     .clr_i      ( r_cnt_clear ),
236     .en_i       ( r_cnt_en   ),
237     .load_i     ( r_cnt_load ),
238     .down_i     ( 1'b1      ),
239     .d_i        ( r_fifo_data.len ),
240     .q_o        ( r_current_beat ),
241     .overflow_o (             ),
242   );
243
244   // pragma translate_off
245   `ifndef VERILATOR
246   initial begin
247     assert (Resp == axi_pkg::RESP_DECERR || Resp ==
axi_pkg::RESP_SLVERR) else
248     $fatal(1, "This module may only generate RESP_DECERR or
RESP_SLVERR responses!");
249   end
250   `ifndef XILINX_SIMULATOR
251   default disable iff (!rst_ni);
252   if (!ATOPs) begin : gen_assert_atops_unsupported
253     assume property( @(posedge clk_i) (slv_req_i.aw_valid |->
slv_req_i.aw.atop == '0)) else
254     $fatal(1, "Got ATOP but not configured to support
ATOPs!");
255   end
256   `endif
257   `endif
258   // pragma translate_on
259
260 endmodule

```

A.41 openhwgroup_core-v-verif/cv32e40p/tb/uvmt/ uvmt_cv32e40p_step_compare.sv (567줄, 22,402B)

```

1 //
2 // Copyright 2020 OpenHW Group
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
5 // you may not use this file except in compliance with the
License.
6 // You may obtain a copy of the License at
7 //
8 // https://solderpad.org/licenses/
9 //
10 // Unless required by applicable law or agreed to in writing,
software
11 // distributed under the License is distributed on an "AS IS"
BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either
express or implied.
13 // See the License for the specific language governing
permissions and
14 // limitations under the License.
15 //
16 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.0
17 //

```

```

18
19
20 `ifndef __UVMT_CV32E40P_STEP_COMPARE_SV__
21 `define __UVMT_CV32E40P_STEP_COMPARE_SV__
22
23 // Step-and-Compare between the CV32E40PE40P and Imperas
OVPsim ISS
24 // Cloned from the Imperas demo at $(IMPERAS_HOME)/
RTL_OVPmodel_step_compare/verilog_testbench/testbench.sv
25
26 /*
27 * Copyright (c) 2005-2020 Imperas Software Ltd.,
www.imperas.com
28 * Copyright (C) Tumbush Enterprises, LLC
29 *
30 * Licensed under the Apache License, Version 2.0 (the
"License");
31 * you may not use this file except in compliance with the
License.
32 * You may obtain a copy of the License at
33 *
34 * http://www.apache.org/licenses/LICENSE-2.0
35 *
36 * Unless required by applicable law or agreed to in writing,
software
37 * distributed under the License is distributed on an "AS IS"
BASIS,
38 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
39 * either express or implied.
40 *
41 * See the License for the specific language governing
permissions and
42 * limitations under the License.
43 *
44 */
45 //
46 //
47 // Execute step and compare of dut OVP instance vs riscv RTL
instance
48 //
49 `ifndef T0_TYPE
50 `define T0_TYPE "RV32IMC"
51 `endif
52
53 import uvm_pkg::*; // needed for the UVM messaging
service (`uvm_info(), etc.)
54
55 `include "uvm_macros.svh"
56 `define CV32E40P_CORE
$root.uvmt_cv32e40p_tb.dut_wrap.cv32e40p_wrapper_i.core_i
57 `define CV32E40P_TRACER
$root.uvmt_cv32e40p_tb.dut_wrap.cv32e40p_wrapper_i.tracer_i
58
59 // TODO change names
60 `define CV32E40P_RM
$root.uvmt_cv32e40p_tb.iss_wrap.cpu
61 `define CV32E40P_RM_RVVI_STATE
$root.uvmt_cv32e40p_tb.iss_wrap.cpu.state
62 `define CV32E40P_RM_RVVI_CONTROL
$root.uvmt_cv32e40p_tb.iss_wrap.cpu.control
63
64 module uvmt_cv32e40p_step_compare
65 (
66     uvma_clknrst_if          clknrst_if,
67     uvmt_cv32e40p_step_compare_if step_compare_if
68 );
69
70
71     bit Clk;
72     bit miscompare;
73     bit is_stall_sim = 0;
74     bit ignore_dpc_check = 0;
75     bit use_iss = 0;
76
77 // FIXME: strichmo: when running random interrupts and random
debug requests it is possible to enter debug mode
78 // (while also acking an interrupt) when the debug program
counter may or may not yet be pointing to the interrupt
79 // vector (mtvec) upon debug entry. However the PC stream
should verify proper operation (the PC will mismatch when
80 // the debug ROM eventually executes a dret). For now we
will skip this comparison if the test requests it
81     initial begin
82         if ($test$plusargs("ignore_dpc_check")) begin
83             ignore_dpc_check = 1;
84             `uvm_info("Step-and-Compare", $sformatf("Requesting
+ignore_dpc_check"), UVM_NONE)
85         end
86     else begin
87         ignore_dpc_check = 0;
88     end
89 end
90
91     initial begin
92         if ($test$plusargs("USE_ISS"))
93             use_iss = 1;
94         end
95
96 // Set the is_stall_sim flag if random stalls are enabled
97 // This will turn off some unpredictable checks:
98 // - CSR wire checks

```

```

99 // - Non-written GPR registers (per instructions)
100 // Note that register writebacks to GPRs will still be
checked during each instruction retirement even with stalls
101
102     initial begin
103         if (!$test$plusargs("rand_stall_obi_disable")) begin
104             is_stall_sim = 1;
105         end
106     end
107
108     always begin
109         wait (clknrst_if.reset_n == 1'b0);
110         wait (clknrst_if.reset_n == 1'b1);
111         if (is_stall_sim) begin
112             `uvm_info("Step-and-Compare", $sformatf("is_stall_sim set
to 1, disabling CSR wire checks and stable GPR register checks"),
UVM_NONE)
113         end
114     end
115
116 // Waiving Verissimo SVTB.
32.2.0: Pass strings by reference unless otherwise needed
117     function void check_32bit(input string compared, input bit
[31:0] expected, input logic [31:0] actual); // @DVT_LINTER_WAIVER
"MT20211228_1" disable SVTB.32.2.0
118         static int now = 0;
119         if (now != $time) begin
120             miscompare = 0;
121             now = $time;
122         end
123         if (expected != actual) begin
124             miscompare = 1;
125             `uvm_error("Step-and-Compare", $sformatf("%s
expected=0x%8h and actual=0x%8h PC=0x%8h", compared, expected,
actual, step_compare_if.ovp_cpu_PCr))
126         end else begin
127             `uvm_info("Step-and-Compare", $sformatf("%s
expected=0x%8h==actual", compared, actual), UVM_DEBUG)
128         end
129     endfunction // check_32bit
130
131     function automatic void compare();
132         int idx;
133         logic [ 5:0] insn_regs_write_addr;
134         logic [31:0] insn_regs_write_value;
135         int insn_regs_write_size;
136         string compared_str;
137         bit ignore;
138
139         logic [31:0] csr_val;
140
141         // Compare PC
142         //
143         check_32bit(.compared("PC"), .expected(step_compare_if.ovp_cpu_PCr),
.actual(step_compare_if.insn_pc));
144
145         check_32bit(.compared("PC"), .expected(`CV32E40P_RM_RVVI_STATE.pc),
.actual(step_compare_if.insn_pc));
146         step_compare_if.num_pc_checks++;
147
148         // Compare GPR's
149         // Assuming that `CV32E40P_TRACER.insn_regs_write size
is never > 1. Check this.
150         // Note that dut_wrap is found 1 level up
151         insn_regs_write_size =
`CV32E40P_TRACER.insn_regs_write.size();
152         if (insn_regs_write_size > 1) begin
153             `uvm_error("Step-and-Compare", $sformatf("Assume
insn_regs_write size is 0 or 1 but is %0d", insn_regs_write_size))
154         end
155         else if (insn_regs_write_size == 1) begin // Get
`CV32E40P_TRACER.insn_regs_write fields if size is 1
156             insn_regs_write_addr =
`CV32E40P_TRACER.insn_regs_write[0].addr;
157             insn_regs_write_value =
`CV32E40P_TRACER.insn_regs_write[0].value;
158             `uvm_info("Step-and-Compare",
$sformatf("insn_regs_write queue[0] addr=0x%0x, value=0x%0x",
insn_regs_write_addr, insn_regs_write_value), UVM_DEBUG)
159         end
160         // Ignore insn_regs_write_addr=0 just like in
riscv_tracer.sv
161         for (idx=0; idx<32; idx++) begin
162             compared_str = $sformatf("GPR[%0d]", idx);
163             if ((idx == insn_regs_write_addr) && (idx != 0) &&
(insn_regs_write_size == 1)) // Use register in insn_regs_write
queue if it exists
164                 check_32bit(.compared(compared_str), .expected(step_compare_if.ovp_
cpu_GPR[idx][31:0]), .actual(insn_regs_write_value));
165                 // FIXME: strichmo: I am removing the static (non-
written) register checks, as they fail in presence of I and D bus
RAM stalls
166                 // It would be highly desirable to find an
alternative for this type of check to ensure unintended writes to do
not
167                 else if (!is_stall_sim && !

```

```

`CV32E40P_TRACER.insn_wb_bypass) // Use actual value from RTL to
compare registers which should have not changed
167
check_32bit(.compared(compared_str), .expected(step_compare_if.ovp_
cpu_GPR[idx][31:0]), .actual(step_compare_if.riscy_GPR[idx]));
168
169     step_compare_if.num_gpr_checks++;
170     end
171     // Compare CSR's
172     foreach('CV32E40P_RM_RVVI_STATE.csr[index]) begin
173         step_compare_if.num_csr_checks++;
174         ignore = 0;
175         csr_val = 0;
176
177         // CSR timing at instruction retirement is not
completely deterministic in this simple model in presence of OBI
stalls
178         if (is_stall_sim)
179             ignore = 1;
180         case (index)
181
182             "marchid" : csr_val =
cv32e40p_pkg::MARCHID; // warning! defined in cv32e40p_pkg
183
184             "mcountinhibit" : csr_val =
`CV32E40P_CORE.cs_registers.i.mcountinhibit_q;
185
186             "mvendroid" : csr_val =
{cv32e40p_pkg::MVENDORID_BANK, cv32e40p_pkg::MVENDORID_OFFSET};
187             "mstatus" : if
(step_compare_if.deferint_prime == 0) ignore = 1;
188                         else csr_val =
{`CV32E40P_CORE.cs_registers.i.mstatus.q.mprv, // Not documented in
Rev 4.5 of user_manual.doc but is in the design
189                                     4'b0,
190
191             "mstatus.q.mpp", 3'b0,
192
193             "mstatus.q.mpie", 2'b0,
194
195             "mstatus.q.upie",
196             "mstatus.q.mie", 2'b0,
197
198             "mstatus.q.ueie";
199             "misa" : csr_val =
`CV32E40P_CORE.cs_registers.i.MISA_VALUE;
200             "mie" : csr_val =
`CV32E40P_CORE.cs_registers.i.mie_q;
201             "mtvec" : csr_val =
{`CV32E40P_CORE.cs_registers.i.mtvec.q, 6'h0,
`CV32E40P_CORE.cs_registers.i.mtvec_mode_q};
202             "mscratch" : csr_val =
`CV32E40P_CORE.cs_registers.i.mscratch_q;
203             "mepc" : if
(step_compare_if.deferint_prime == 0) ignore = 1;
204                         else csr_val =
`CV32E40P_CORE.cs_registers.i.mepc_q;
205             "mcause" : if
(step_compare_if.deferint_prime == 0) ignore = 1;
206                         else csr_val =
{`CV32E40P_CORE.cs_registers.i.mcause.q[5],
207                                     26'b0,
208
209             "mcause.q[4:0]";
210             "mip" : if
(step_compare_if.deferint_prime == 0 || iss_wrap.io.deferint == 0)
ignore = 1;
211             // else csr_val =
`CV32E40P_CORE.cs_registers.i.mip;
212             "mip" : ignore = 1;
213             "mhartid" : csr_val =
`CV32E40P_CORE.cs_registers.i.hart_id_i;
214             // only valid in DEBUG Mode
215             "dcsr" : begin
216                 csr_val =
`CV32E40P_CORE.cs_registers.i.dcsr_q;
217                 if (iss_wrap.io.DM==0) ignore =
1;
218             end
219             "dpc" : begin
220                 csr_val =
`CV32E40P_CORE.cs_registers.i.depc_q;
221                 if (iss_wrap.io.DM==0) ignore =
1;
222                 if (ignore_dpc_check) ignore =
1;
223             end
224             "dscratch0" : csr_val =
`CV32E40P_CORE.cs_registers.i.dscratch0_q;
225             "dscratch1" : csr_val =
`CV32E40P_CORE.cs_registers.i.dscratch1_q;
226             "pmpcfg0" : csr_val =
`CV32E40P_CORE.cs_registers.i.pmp_reg.q.pmpcfg_packed[0];
227             "pmpcfg1" : csr_val =
`CV32E40P_CORE.cs_registers.i.pmp_reg.q.pmpcfg_packed[1];

```

```

228         "pmpcfg2" : csr_val =
`CV32E40P_CORE.cs_registers.i.pmp_reg.q.pmpcfg_packed[2];
229         "pmpcfg3" : csr_val =
`CV32E40P_CORE.cs_registers.i.pmp_reg.q.pmpcfg_packed[3];
230         "tselect" : csr_val = 32'h0000_0000;
231         "tdata1" : csr_val =
`CV32E40P_CORE.cs_registers.i.tmatch_control_rdata;
232         "tdata2" : csr_val =
`CV32E40P_CORE.cs_registers.i.tmatch_value_rdata;
233         "tdata3" : csr_val = 32'h0000_0000;
234         "tinfo" : csr_val =
`CV32E40P_CORE.cs_registers.i.tinfo_types;
235
236         "time" : ignore = 1;
237
238         "mcontext" : ignore = 1;
239         "scontext" : ignore = 1;
240
241         "cycle" : ignore = 1;
242         "cycleh" : ignore = 1;
243         "instret" : ignore = 1;
244         "instreth" : ignore = 1;
245         "minstret" : ignore = 1;
246         "minstreth" : ignore = 1;
247
248         "mimpid" : ignore = 1;
249
250         "hpmcounter3" : ignore = 1;
251         "hpmcounter4" : ignore = 1;
252         "hpmcounter5" : ignore = 1;
253         "hpmcounter6" : ignore = 1;
254         "hpmcounter7" : ignore = 1;
255         "hpmcounter8" : ignore = 1;
256         "hpmcounter9" : ignore = 1;
257         "hpmcounter10" : ignore = 1;
258         "hpmcounter11" : ignore = 1;
259         "hpmcounter12" : ignore = 1;
260         "hpmcounter13" : ignore = 1;
261         "hpmcounter14" : ignore = 1;
262         "hpmcounter15" : ignore = 1;
263         "hpmcounter16" : ignore = 1;
264         "hpmcounter17" : ignore = 1;
265         "hpmcounter18" : ignore = 1;
266         "hpmcounter19" : ignore = 1;
267         "hpmcounter20" : ignore = 1;
268         "hpmcounter21" : ignore = 1;
269         "hpmcounter22" : ignore = 1;
270         "hpmcounter23" : ignore = 1;
271         "hpmcounter24" : ignore = 1;
272         "hpmcounter25" : ignore = 1;
273         "hpmcounter26" : ignore = 1;
274         "hpmcounter27" : ignore = 1;
275         "hpmcounter28" : ignore = 1;
276         "hpmcounter29" : ignore = 1;
277         "hpmcounter30" : ignore = 1;
278         "hpmcounter31" : ignore = 1;
279
280         "hpmcounterh3" : ignore = 1;
281         "hpmcounterh4" : ignore = 1;
282         "hpmcounterh5" : ignore = 1;
283         "hpmcounterh6" : ignore = 1;
284         "hpmcounterh7" : ignore = 1;
285         "hpmcounterh8" : ignore = 1;
286         "hpmcounterh9" : ignore = 1;
287         "hpmcounterh10" : ignore = 1;
288         "hpmcounterh11" : ignore = 1;
289         "hpmcounterh12" : ignore = 1;
290         "hpmcounterh13" : ignore = 1;
291         "hpmcounterh14" : ignore = 1;
292         "hpmcounterh15" : ignore = 1;
293         "hpmcounterh16" : ignore = 1;
294         "hpmcounterh17" : ignore = 1;
295         "hpmcounterh18" : ignore = 1;
296         "hpmcounterh19" : ignore = 1;
297         "hpmcounterh20" : ignore = 1;
298         "hpmcounterh21" : ignore = 1;
299         "hpmcounterh22" : ignore = 1;
300         "hpmcounterh23" : ignore = 1;
301         "hpmcounterh24" : ignore = 1;
302         "hpmcounterh25" : ignore = 1;
303         "hpmcounterh26" : ignore = 1;
304         "hpmcounterh27" : ignore = 1;
305         "hpmcounterh28" : ignore = 1;
306         "hpmcounterh29" : ignore = 1;
307         "hpmcounterh30" : ignore = 1;
308         "hpmcounterh31" : ignore = 1;
309
310         "mhpmcounter3" : ignore = 1;
311         "mhpmcounter4" : ignore = 1;
312         "mhpmcounter5" : ignore = 1;
313         "mhpmcounter6" : ignore = 1;
314         "mhpmcounter7" : ignore = 1;
315         "mhpmcounter8" : ignore = 1;
316         "mhpmcounter9" : ignore = 1;
317         "mhpmcounter10" : ignore = 1;
318         "mhpmcounter11" : ignore = 1;
319         "mhpmcounter12" : ignore = 1;
320         "mhpmcounter13" : ignore = 1;
321         "mhpmcounter14" : ignore = 1;
322         "mhpmcounter15" : ignore = 1;
323         "mhpmcounter16" : ignore = 1;

```

```

324 "mhpcounter17" : ignore = 1;
325 "mhpcounter18" : ignore = 1;
326 "mhpcounter19" : ignore = 1;
327 "mhpcounter20" : ignore = 1;
328 "mhpcounter21" : ignore = 1;
329 "mhpcounter22" : ignore = 1;
330 "mhpcounter23" : ignore = 1;
331 "mhpcounter24" : ignore = 1;
332 "mhpcounter25" : ignore = 1;
333 "mhpcounter26" : ignore = 1;
334 "mhpcounter27" : ignore = 1;
335 "mhpcounter28" : ignore = 1;
336 "mhpcounter29" : ignore = 1;
337 "mhpcounter30" : ignore = 1;
338 "mhpcounter31" : ignore = 1;
339
340 "mhpcounterh3" : ignore = 1;
341 "mhpcounterh4" : ignore = 1;
342 "mhpcounterh5" : ignore = 1;
343 "mhpcounterh6" : ignore = 1;
344 "mhpcounterh7" : ignore = 1;
345 "mhpcounterh8" : ignore = 1;
346 "mhpcounterh9" : ignore = 1;
347 "mhpcounterh10" : ignore = 1;
348 "mhpcounterh11" : ignore = 1;
349 "mhpcounterh12" : ignore = 1;
350 "mhpcounterh13" : ignore = 1;
351 "mhpcounterh14" : ignore = 1;
352 "mhpcounterh15" : ignore = 1;
353 "mhpcounterh16" : ignore = 1;
354 "mhpcounterh17" : ignore = 1;
355 "mhpcounterh18" : ignore = 1;
356 "mhpcounterh19" : ignore = 1;
357 "mhpcounterh20" : ignore = 1;
358 "mhpcounterh21" : ignore = 1;
359 "mhpcounterh22" : ignore = 1;
360 "mhpcounterh23" : ignore = 1;
361 "mhpcounterh24" : ignore = 1;
362 "mhpcounterh25" : ignore = 1;
363 "mhpcounterh26" : ignore = 1;
364 "mhpcounterh27" : ignore = 1;
365 "mhpcounterh28" : ignore = 1;
366 "mhpcounterh29" : ignore = 1;
367 "mhpcounterh30" : ignore = 1;
368 "mhpcounterh31" : ignore = 1;
369
370 default: begin
371   `uvm_error("STEP_COMPARE", $sformatf("index=%s
does not match a CSR name", index))
372   ignore = 1;
373 end
374 endcase // case (index)
375
376 if (!ignore)
377   check_32bit(.compared(index), .expected(`CV32E40P_RM_RVVI_STATE.csr[
index]), .actual(csr_val));
378
379 end // foreach (ovp.cpu.csr[index])
380 endfunction // compare
381
382 int cycles = 0;
383 always @(posedge `CV32E40P_CORE.clk) begin
384   cycles++;
385 end
386
387 // RTL->RM CSR : mcycle, minstret, mcycleh, minstreth
388
389 Waiving Verissimo SVTB.32.2.0: Pass strings by reference unless
otherwise needed
390 function automatic void pushRTL2RM(string message); //
@DVT_LINTER_WAIVER "MT20211228_2" disable SVTB.32.2.0
391   logic [ 5:0] gpr_addr;
392   logic [31:0] gpr_value;
393   if (`CV32E40P_TRACER.insn_regs_write.size()) begin
394     gpr_addr =
`CV32E40P_TRACER.insn_regs_write[0].addr;
395     gpr_value =
`CV32E40P_TRACER.insn_regs_write[0].value;
396     `CV32E40P_RM.state.GPR_rtl[gpr_addr] = gpr_value;
397   end
398   // Pass cycles and reset
399   `CV32E40P_RM.cycles = cycles;
400   cycles = 0;
401 endfunction // pushRTL2RM
402
403 /*
404   The schedule works like this
405   1. Run the RTL for 1 instruction retirement
406   2. if the RTL.RetiredPC == OVP.NextPC
407   then run OVP for 1 instruction retirement
408   3. Compare RTL <-> OVP
409 */
410 event ev_compare;
411 static int instruction_count = 0;
412
413 typedef enum {
414   INIT,
415   IDLE, // Needed to get an event on state so always
block is initially entered

```

```

416
417   RTL_STEP,
418   RTL_VALID,
419   RTL_TRAP,
420   RTL_HALT,
421
422   RM_STEP,
423   RM_VALID,
424   RM_TRAP,
425   RM_HALT,
426
427   CMP
428 } state_e;
429
430 state_e state = INIT;
431 initial state <= IDLE; // cause an event for always @*
432
433 always @(*) begin
434   if (use_iss) begin
435     case (state)
436     IDLE: begin
437       state <= RTL_STEP;
438     end
439     RTL_STEP: begin
440       clknrst_if.start_clk();
441       fork
442       begin
443         @step_compare_if.riscv_retire;
444         clknrst_if.stop_clk();
445         state <= RTL_VALID;
446       end
447       begin
448         @step_compare_if.riscv_trap;
449         state <= RTL_TRAP;
450       end
451       begin
452         @step_compare_if.riscv_halt;
453         state <= RTL_HALT;
454       end
455       join_any
456       disable fork;
457     end
458     RTL_VALID: begin
459       state <= RM_STEP;
460     end
461     RTL_TRAP: begin
462       //state <= RM_STEP; // TODO: RTL/RVVI needs
additional work
463       state <= RTL_STEP;
464     end
465     RTL_HALT: begin
466       state <= RTL_STEP;
467     end
468     RM_STEP: begin
469       pushRTL2RM("ret_rtl");
470       `CV32E40P_RM_RVVI_CONTROL.stepi();
471       fork
472       begin
473         @step_compare_if.ovp_cpu_valid;
474         -> `CV32E40P_TRACER.ovp_retire;
475         state <= RM_VALID;
476       end
477       begin
478         @step_compare_if.ovp_cpu_trap;
479         state <= RM_TRAP;
480       end
481       begin
482         @step_compare_if.ovp_cpu_halt;
483         state <= RM_HALT;
484       end
485       join_any
486       disable fork;
487     end
488     RM_VALID: begin
489       state <= CMP;
490     end
491     RM_TRAP: begin
492       //state <= CMP; // TODO: needs enabling after
RTL/RVVI fix
493       state <= RM_STEP;
494     end
495     RM_HALT: begin
496       state <= RM_STEP;
497     end
498     CMP: begin
499       compare();
500       -> ev_compare;
501       instruction_count += 1;
502       //state <= RTL_STEP;
503       state <= IDLE;
504     end
505   endcase // case (state)

```

```

515     end
516 end
517
518     always @(instruction_count) begin
519         if (!(instruction_count % 10000)) begin
520             `uvm_info("Step-and-Compare", $sformatf("Compared %0d
instructions", instruction_count), UVM_NONE)
521         end
522         if (instruction_count >= 10_000_000) begin
523             `uvm_fatal("Step-and-Compare", $sformatf("Compared
%0d instructions - that's too long!", instruction_count))
524         end
525     end
526
527
528 `ifdef COVERAGE
529     coverage cov1;
530     initial begin
531         cov1 = new();
532     end
533
534     function void split(input string in_s, output string s1,
s2);
535         automatic int i;
536         for (i=0; i<in_s.len(); i++) begin
537             if (in_s.getc(i) == ":")
538                 break;
539         end
540         if (i==0 ) begin
541             `uvm_fatal("STEP COMPARE", $sformatf(": not found
in split '%0s'", in_s))
542         end
543         s1 = in_s.substr(0,i-1);
544         s2 = in_s.substr(i+1,in_s.len()-1);
545     endfunction
546
547     function automatic void sample();
548         string decode = `CV32E40P_RM.Decode;
549         string ins_str, op[4], key, val;
550         int i;
551         ins_t ins;
552         int num = $sscanf (decode, "%s %s %s %s %s", ins_str,
op[0], op[1], op[2], op[3]);
553         ins.ins_str = ins_str;
554         for (i=0; i<num-1; i++) begin
555             split(op[i], key, val);
556             ins.ops[i].key=key;
557             ins.ops[i].val=val;
558         end
559         cov1.sample (ins);
560     endfunction
561 `endif
562
563 endmodule: uvmt_cv32e40p_step_compare
564
565
566 `endif // __UVM CV32E40P_STEP_COMPARE_SV__
567

```

A.42 openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/ uvma_rvfi_instr_mon.sv (299줄, 12,416B)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
7 // you may not use this file except in compliance with the
License.
8 // You may obtain a copy of the License at
9 //
10 //     https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
software
13 // distributed under the License is distributed on an "AS IS"
BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either
express or implied.
15 // See the license for the specific language governing
permissions and
16 // limitations under the License.
17 //
18
19
20 `ifndef __UVM_RVFI_INSTR_MON_SV__
21 `define __UVM_RVFI_INSTR_MON_SV__
22
23
24 /**
25  * Component sampling transactions from a Clock & Reset
virtual interface
26  * (uvma_rvfi_instr_if).
27  */
28 class uvma_rvfi_instr_mon_c#(int ILEN=DEFAULT_ILEN,
int XLEN=DEFAULT_XLEN) extends
uvvm_monitor;

```

```

30
31 // Objects
32 uvma_rvfi_cfg_c   cfg;
33 uvma_rvfi_cntxt_c cntxt;
34
35 // TLM
36 uvvm_analysis_port#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN))
ap;
37
38 // State of monitor
39 bit last_dcsr_nmip = 0;
40 bit [uvma_rvfi_csr_seq_item_c#():XLEN-1:0] dcsr_ret_data;
41
42 longint unsigned supported_csrs_addrs [$];
43 string supported_csrs_names [$];
44
45 string log_tag = "RVFIMONLOG";
46
47 `uvm_component_utils_begin(uvma_rvfi_instr_mon_c)
48     `uvm_field_int(last_dcsr_nmip, UVM_DEFAULT)
49     `uvm_field_int(dcsr_ret_data, UVM_DEFAULT)
50     `uvm_field_object(cfg, UVM_DEFAULT |
UVM_REFERENCE)
51     `uvm_field_object(cntxt, UVM_DEFAULT |
UVM_REFERENCE)
52     `uvm_component_utils_end
53
54 /**
55  * Default constructor.
56  */
57 extern function new(string name="uvma_rvfi_instr_mon",
uvvm_component parent=null);
58
59 /**
60  * 1. Ensures cfg & cntxt handles are not null.
61  * 2. Builds ap.
62  */
63 extern virtual function void build_phase(uvm_phase phase);
64
65 /**
66  * Oversees monitoring via monitor_clk() and
monitor_reset() tasks in parallel
67  * forks.
68  */
69 extern virtual task run_phase(uvm_phase phase);
70
71 /**
72  * Monitor the instruction interface
73  */
74 extern virtual task monitor_rvfi_instr();
75
76 endclass : uvma_rvfi_instr_mon_c
77
78 function uvma_rvfi_instr_mon_c::new(string
name="uvma_rvfi_instr_mon", uvvm_component parent=null);
79
80     super.new(name, parent);
81
82 endfunction : new
83
84
85 function void uvma_rvfi_instr_mon_c::build_phase(uvm_phase
phase);
86
87     super.build_phase(phase);
88
89     if ($test$plusargs("log_rvfi"))
90         set_report_id_verbosity(log_tag, UVM_HIGH);
91
92
93 void'(uvm_config_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::get(this, "",
"cfg", cfg));
94 if (!cfg) begin
95     `uvm_fatal("CFG", "Configuration handle is null")
96 end
97
98 void'(uvm_config_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::get(this, "",
"cntxt", cntxt));
99 if (!cntxt) begin
100     `uvm_fatal("CNTXT", "Context handle is null")
101 end
102
103 ap = new("ap", this);
104
105 endfunction : build_phase
106
107 task uvma_rvfi_instr_mon_c::run_phase(uvm_phase phase);
108
109     super.run_phase(phase);
110
111     cfg.core_cfg.get_supported_csrs_names(supported_csrs_names);
112
113     cfg.core_cfg.get_supported_csrs_addrs(supported_csrs_addrs);
114
115     // TODO Make this prettier
116     if (cfg.enabled) begin
117         while (1) begin
118             wait (cntxt.instr_vif[0].reset_n === 1'b0);
119             wait (cntxt.instr_vif[0].reset_n === 1'b1);

```

```

118
119         fork
120         begin
121             monitor_rvfi_instr();
122         end
123         begin
124             wait (cntxt.instr_vif[0].reset_n ==
1'b0);
125         end
126         join_any
127     end
128 end
129 endtask : run_phase
130
131 task uvma_rvfi_instr_mon_c::monitor_rvfi_instr();
132
133 while(1) begin
134     @(cntxt.instr_vif[0].mon_cb)
135     for(int i = 0; i < cfg.nret; i++)
136     begin
137         int nret_id;
138         bit monitor_condition;
139
140         nret_id = i;
141         monitor_condition = (!cfg.core_cfg.unified_traps) ?
142         (cntxt.instr_vif[nret_id].mon_cb.rvfi_valid ||
143         cntxt.instr_vif[nret_id].mon_cb.rvfi_trap[0]) :
144         (cntxt.instr_vif[nret_id].mon_cb.rvfi_valid);
145
146         if (monitor_condition) begin
147             uvma_rvfi_instr_seq_item_c#(ILEN,XLEN) mon_trn;
148
149             mon_trn =
150             uvma_rvfi_instr_seq_item_c#(ILEN,XLEN)::type_id::create("mon_trn");
151
152             mon_trn.nret_id = nret_id;
153             mon_trn.cycle_cnt =
154             cntxt.instr_vif[nret_id].mon_cb.cycle_cnt;
155             mon_trn.order =
156             cntxt.instr_vif[nret_id].mon_cb.rvfi_order;
157             mon_trn.insn =
158             cntxt.instr_vif[nret_id].mon_cb.rvfi_insn;
159             mon_trn.trap =
160             cntxt.instr_vif[nret_id].mon_cb.rvfi_trap;
161             mon_trn.halt =
162             cntxt.instr_vif[nret_id].mon_cb.rvfi_halt;
163             mon_trn.dbg =
164             cntxt.instr_vif[nret_id].mon_cb.rvfi_dbg;
165             mon_trn.dbg_mode =
166             cntxt.instr_vif[nret_id].mon_cb.rvfi_dbg_mode;
167             mon_trn.nmip =
168             cntxt.instr_vif[nret_id].mon_cb.rvfi_nmip;
169             mon_trn.intr =
170             cntxt.instr_vif[nret_id].mon_cb.rvfi_intr;
171             mon_trn.mode =
172             $cast(mon_trn.mode,
173             cntxt.instr_vif[nret_id].mon_cb.rvfi_mode);
174             mon_trn.ixl =
175             cntxt.instr_vif[nret_id].mon_cb.rvfi_ixl;
176             mon_trn.pc_rdata =
177             cntxt.instr_vif[nret_id].mon_cb.rvfi_pc_rdata;
178             mon_trn.pc_wdata =
179             cntxt.instr_vif[nret_id].mon_cb.rvfi_pc_wdata;
180             mon_trn.rs1_addr =
181             cntxt.instr_vif[nret_id].mon_cb.rvfi_rs1_addr;
182             mon_trn.rs1_rdata =
183             cntxt.instr_vif[nret_id].mon_cb.rvfi_rs1_rdata;
184             mon_trn.rs2_addr =
185             cntxt.instr_vif[nret_id].mon_cb.rvfi_rs2_addr;
186             mon_trn.rs2_rdata =
187             cntxt.instr_vif[nret_id].mon_cb.rvfi_rs2_rdata;
188             mon_trn.rs3_addr =
189             cntxt.instr_vif[nret_id].mon_cb.rvfi_rs3_addr;
190             mon_trn.rs3_rdata =
191             cntxt.instr_vif[nret_id].mon_cb.rvfi_rs3_rdata;
192             mon_trn.rd1_addr =
193             cntxt.instr_vif[nret_id].mon_cb.rvfi_rd1_addr;
194             mon_trn.rd1_wdata =
195             cntxt.instr_vif[nret_id].mon_cb.rvfi_rd1_wdata;
196             mon_trn.rd2_addr =
197             cntxt.instr_vif[nret_id].mon_cb.rvfi_rd2_addr;
198             mon_trn.rd2_wdata =
199             cntxt.instr_vif[nret_id].mon_cb.rvfi_rd2_wdata;
200             mon_trn.mem_addr =
201             cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_addr;
202             mon_trn.mem_rdata =
203             cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_rdata;
204             mon_trn.mem_rmask =
205             cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_rmask;
206             mon_trn.mem_wdata =
207             cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_wdata;
208             mon_trn.mem_wmask =

```

```

cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_wmask;
209
210 // Get the CSRs
211 if (!cfg.core_cfg.disable_all_csr_checks) begin
212     if (!cfg.unified_csr_vif) begin
213         foreach (cntxt.csr_vif[csr]) begin
214             uvma_rvfi_csr_seq_item_c csr_trn =
215             uvma_rvfi_csr_seq_item_c#(XLEN)::type_id::create({csr, "_trn"});
216
217             csr_trn.csr = csr;
218             csr_trn.nret_id = nret_id;
219             csr_trn.rmask = cntxt.csr_vif[csr]
220             [nret_id].mon_cb.rvfi_csr_rmask;
221             csr_trn.wmask = cntxt.csr_vif[csr]
222             [nret_id].mon_cb.rvfi_csr_wmask;
223             csr_trn.rdata = cntxt.csr_vif[csr]
224             [nret_id].mon_cb.rvfi_csr_rdata;
225             csr_trn.wdata = cntxt.csr_vif[csr]
226             [nret_id].mon_cb.rvfi_csr_wdata;
227
228             mon_trn.name_csrs[csr] = csr_trn;
229         end
230     else begin
231         uvma_rvfi_csr_seq_item_c csr_trn;
232         longint unsigned addr;
233
234         csr_trn =
235         uvma_rvfi_csr_seq_item_c#(XLEN)::type_id::create({addr, "_trn"});
236         addr =
237         cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_addr;
238         csr_trn.addr = addr;
239         if
240         (cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wmask != 0) begin
241             csr_trn.wmask =
242             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wmask;
243             csr_trn.wdata =
244             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wdata;
245             csr_trn.rmask =
246             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_rmask;
247             csr_trn.rdata =
248             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_rdata;
249             mon_trn.addr_csrs[addr] = csr_trn;
250         end
251         foreach(supported_csrs_addrs[i]) begin
252             addr = supported_csrs_addrs[i];
253             csr_trn =
254             uvma_rvfi_csr_seq_item_c#(XLEN)::type_id::create({addr, "_trn"});
255             csr_trn.addr = addr;
256             csr_trn.wmask =
257             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_wmask[addr];
258             csr_trn.wdata =
259             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_wdata[addr];
260             csr_trn.rmask =
261             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_rmask[addr];
262             csr_trn.rdata =
263             cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_rdata[addr];
264             mon_trn.addr_csrs[addr] = csr_trn;
265         end
266     end
267
268 // Decode interrupts that need to be
269 // communicated to ISS (external or NMI bus faults)
270 if (mon_trn.intr) begin
271     // The cause of the interrupt should be
272     // in the "rdata" field of the mcause CSR RVFI port
273     bit [XLEN-1:0] csr_mcause;
274
275     if (!cfg.unified_csr_vif)
276         csr_mcause =
277         mon_trn.name_csrs["mcause"].rdata;
278     else
279         csr_mcause =
280         mon_trn.addr_csrs[MCAUSE].rdata;
281
282     // External interrupt
283     if (csr_mcause[31]) begin
284         // NMI - Load fault
285         if (cfg.nmi_load_fault_enabled &&
286         csr_mcause[XLEN-2:0] == cfg.nmi_load_fault_cause) begin
287             mon_trn.insn_nmi_load_fault = 1;
288         end
289         // NMI - Store fault
290         else if (cfg.nmi_store_fault_enabled
291         && csr_mcause[XLEN-2:0] == cfg.nmi_store_fault_cause) begin
292             mon_trn.insn_nmi_store_fault =
293             1;
294         end
295         // External interrupt
296         else begin
297             mon_trn.insn_interrupt = 1;
298             mon_trn.insn_interrupt_id =
299             { 1'b0, csr_mcause[XLEN-2:0] };
300         end
301     end
302 end
303
304 if (!cfg.unified_csr_vif &&
305 mon_trn.name_csrs.exists("dcsr"))
306     dcsr_ret_data =
307     mon_trn.name_csrs["dcsr"].get_csr_retirement_data();

```

```

259         else if (cfg.unified_csr_vif &&
mon_trn.addr_csrs.exists(DCSR))
260             dcsr_ret_data =
mon_trn.addr_csrs[DCSR].get_csr_retirement_data();
261
262             // In debug mode, detect NMIP event for a
data bus error
263             if (mon_trn.dbg_mode &&
264                 !last_dcsr_nmip &&
265                 mon_trn.nmip[0] &&
266                 dcsr_ret_data[3])
267                 begin
268                     `uvm_info("RVFIMON", $sformatf("Debug
NMIP"), UVM_LOW);
269
270                     if (mon_trn.nmip[1] == 0) begin
271                         mon_trn.insn_nmi_load_fault = 1;
272                     end else begin
273                         mon_trn.insn_nmi_store_fault = 1;
274                     end
275                 end
276
277             // Latch the last DCSR NMIP bit to detect
positive assertion of dcsr.nmip
278             last_dcsr_nmip = dcsr_ret_data[3];
279         end
280
281         // Detect instruction bus fault
282         if (cfg.insn_bus_fault_enabled &&
283             mon_trn.is_trap() &&
284             mon_trn.get_trap_cause() ==
cfg.insn_bus_fault_cause) begin
285             mon_trn.insn_bus_fault = 1;
286
287             `uvm_info("RVFIMON", $sformatf("Detected bus
fault"), UVM_LOW)
288         end
289         `uvm_info(log_tag, $sformatf("%s",
mon_trn.convert2string()), UVM_HIGH);
290
291         ap.write(mon_trn);
292     end
293 end
294 end
295
296 endtask : monitor_rvfi_instr
297
298 `endif // __UVMA_RVFI_INSTR_MON_SV__
299

```

A.43 openhwgroup_core-v-verif/lib/uvma_agents/uvma_rvfi/uvma_rvfi_assert.sv (133줄, 4,351B)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the
License.
7 // You may obtain a copy of the License at
8 //
9 //     https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either
express or implied.
14 // See the License for the specific language governing
permissions and
15 // limitations under the License.
16
17 module uvma_rvfi_assert
18     import uvma_pkg::*;
19     #(
20         parameter int ADDR_WIDTH=32,
21         parameter int DATA_WIDTH=32
22     )
23     (
24         input                clk,
25         input                reset_n,
26
27         input                req,
28         input                gnt,
29         input [ADDR_WIDTH-1:0] addr,
30         input                we,
31         input [DATA_WIDTH/8-1:0] be,
32         input [DATA_WIDTH-1:0] wdata,
33         input [DATA_WIDTH-1:0] rdata,
34         input                rvalid,
35         input                rready
36     );
37
38 //

```

```

39 // Local variables
40 //
-----
41 string info_tag = "CV32E40P-RVFI_ASSERT";
42
43 //
-----
44 // Clocking blocks
45 //
-----
46
47 // Single clock, single reset design, use default clocking
48 default clocking @(posedge clk); endclocking
49 default disable iff !(reset_n);
50
51 //
-----
52 // Begin module code
53 //
-----
54
55 // R-3.1.1 : Address phase signals stable during address
phase
56 property p_addr_signal_stable(sig);
57     req ##0 !gnt |>=> $stable(sig);
58 endproperty : p_addr_signal_stable
59
60 a_addr_stable: assert property(p_addr_signal_stable(addr))
61 else
62     `uvm_error(info_tag, "addr signal not stable in address
phase")
63
64 a_we_stable: assert property(p_addr_signal_stable(we))
65 else
66     `uvm_error(info_tag, "we signal not stable in address
phase")
67
68 a_wdata_stable: assert property(p_addr_signal_stable(wdata))
69 else
70     `uvm_error(info_tag, "wdata signal not stable in address
phase")
71
72 a_be_stable: assert property(p_addr_signal_stable(be))
73 else
74     `uvm_error(info_tag, "be signal not stable in address
phase")
75
76 // R-3.1.2 : Req may not deassert until the gnt is asserted
77 property p_req_until_gnt;
78     req ##0 !gnt |>=> req;
79 endproperty : p_req_until_gnt
80 a_req_until_gnt : assert property(p_req_until_gnt)
81 else
82     `uvm_error(info_tag, "req may not deassert until gnt
asserted")
83
84 // R-7 At least one byte enable must be set
85 property p_be_not_zero;
86     req |> be != 0;
87 endproperty : p_be_not_zero
88 a_be_not_zero : assert property(p_be_not_zero)
89 else
90     `uvm_error(info_tag, "be was zero during an address
cycle")
91
92 // R-7 All ones must be contiguous in writes
93 reg[3:0] contiguous_be[] = {
94     4'b0001,
95     4'b0011,
96     4'b0111,
97     4'b1111,
98     4'b0010,
99     4'b0110,
100    4'b1110,
101    4'b0100,
102    4'b1100,
103    4'b1000
104 };
105 bit be_inside_contiguous_be;
106 always_comb begin
107     be_inside_contiguous_be = be inside {contiguous_be};
108 end
109 property p_be_contiguous;
110     req |> be_inside_contiguous_be;
111 endproperty : p_be_contiguous
112 a_be_contiguous : assert property(p_be_contiguous)
113 else
114     `uvm_error(info_tag, $sformatf("be of 0x%x was not
contiguous", $sampled(be)));
115
116 // R-8 Data address LSBs must be consistent with byte
enables on writes
117 function bit [1:0] get_addr_lsb(bit[3:0] be);
118     casex (be)
119         4'b???1: return 0;
120         4'b???0: return 1;

```

```

121     4'b100: return 2;
122     4'b1000: return 3;
123     endcase
124   endfunction : get_addr_lsb
125
126   property p_addr_be_consistent;
127     req |-> addr[1:0] == get_addr_lsb(be);
128   endproperty : p_addr_be_consistent
129   a_addr_be_consistent: assert property(p_addr_be_consistent)
130   else
131     `uvm_error(info_tag, $sformatf("be of 0x%01x not
consistent with addr 0x%08x", $sampled(be), $sampled(addr)));
132
133   endmodule : uvma_rvfi_assert

```

A.44 openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv (2,062줄, 81,567B)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
"License");
5 // you may not use this file except in compliance with the
License, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/
10 //
11 // Unless required by applicable law or agreed to in writing,
any work
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either
express or implied.
14 // See the License for the specific language governing
permissions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
18 //
19 // Contributors: Davide Schiavone, OpenHW Group
<davide@openhwgroup.org> //
20 // Halfdan Bechmann, Silicon Labs
<halfdan.bechmann@silabs.com> //
21 // Yoann Pruvost, Dolphin Design
<yoann.pruvost@dolphin.fr> //
22 //
23 // Description: CV32E40P RVFI
interface //
24 //
25 ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
26 `include "cv32e40p_rvfi_pkg.sv"
27
28 module cv32e40p_rvfi
29   import cv32e40p_pkg::*;
30   import cv32e40p_rvfi_pkg::*;
31
32   #(
33     parameter FPU = 0,
34     parameter ZFINX = 0,
35     parameter NUM_MHPMCOUNTERS = 1
36   ) (
37     input logic clk_i,
38     input logic rst_ni,
39
40     input logic is_decoding_i,
41     input logic is_illegal_i,
42     input logic trigger_match_i,
43     input logic data_misaligned_i,
44     input logic lsu_data_we_ex_i,
45
46     input logic debug_mode_i,
47     input logic [2:0] debug_cause_i,
48     /// INSTR IF Probes ///
49     input logic instr_req_i,
50     input logic instr_grant_i,
51     input logic instr_rvalid_i,
52
53     input logic prefetch_req_i,
54     input logic pc_set_i,
55     /// IF Probes ///
56     input logic if_valid_i,
57     input logic if_ready_i,
58     input logic instr_valid_if_i,
59     input logic [31:0] instr_if_i,
60     input logic [31:0] pc_if_i,
61     input logic instr_pmp_err_if_i,
62
63     input logic instr_valid_id_i,
64     input logic [31:0] instr_rdata_id_i,

```

```

65     input logic is_fetch_failed_id_i,
66     input logic instr_req_int_i,
67     input logic clear_instr_valid_i,
68     /// ID probes ///
69     input logic [31:0] pc_id_i,
70     input logic id_valid_i,
71
72     input logic id_ready_i,
73     input logic [1:0] rf_re_id_i,
74     input logic sys_en_id_i,
75     input logic sys_mret_insn_id_i,
76     input logic jump_in_id_i,
77     input logic [31:0] jump_target_id_i,
78     input logic is_compressed_id_i,
79
80     input logic ebrk_insn_dec_i,
81     input logic ecall_insn_dec_i,
82
83     input logic mret_insn_dec_i,
84     input logic mret_dec_i,
85
86     input logic [5:0] csr_cause_i,
87
88     input logic debug_csr_save_i,
89     /// HWLOOP regs
90     input logic [1:0][31:0] hwlp_start_q_i,
91     input logic [1:0][31:0] hwlp_end_q_i,
92     input logic [1:0][31:0] hwlp_counter_q_i,
93     input logic [1:0][31:0] hwlp_counter_n_i,
94     input logic minstret_i,
95     /// LSU
96     input logic lsu_en_id_i,
97     input logic lsu_we_id_i,
98     input logic [1:0] lsu_size_id_i,
99     /// Register reads
100     input logic [5:0] rs1_addr_id_i,
101     input logic [5:0] rs2_addr_id_i,
102     input logic [5:0] rs3_addr_id_i,
103     input logic [31:0] operand_a_fw_id_i,
104     input logic [31:0] operand_b_fw_id_i,
105     input logic [31:0] operand_c_fw_id_i,
106
107     /// EX probes ///
108
109     /// Register writes in EX
110     input logic ex_ready_i,
111     input logic ex_valid_i,
112
113     input logic ex_reg_we_i,
114     input logic [5:0] ex_reg_addr_i,
115     input logic [31:0] ex_reg_wdata_i,
116
117     input logic apu_en_ex_i,
118     input logic apu_singlecycle_i,
119     input logic apu_multicycle_i,
120     input logic wb_contention_lsu_i,
121     input logic wb_contention_i,
122     input logic regfile_we_lsu_i,
123
124     input logic branch_in_ex_i,
125     input logic branch_decision_ex_i,
126     input logic dret_in_ex_i,
127     /// LSU
128     input logic lsu_en_ex_i,
129     input logic lsu_pmp_err_ex_i,
130     input logic lsu_pma_err_atomic_ex_i,
131
132     input logic [31:0] branch_target_ex_i,
133
134     input logic [31:0] data_addr_ex_i,
135     input logic [31:0] data_wdata_ex_i,
136     input logic lsu_split_q_ex_i,
137
138     input logic mult_ready_i,
139     input logic alu_ready_i,
140
141     /// WB probes ///
142     input logic [31:0] pc_wb_i,
143     input logic wb_ready_i,
144     input logic wb_valid_i,
145     input logic ebreak_in_wb_i,
146     input logic [31:0] instr_rdata_wb_i,
147     input logic csr_en_wb_i,
148     input logic sys_wfi_insn_wb_i,
149     /// Register writes
150     input logic rf_we_wb_i,
151     input logic [5:0] rf_addr_wb_i,
152     input logic [31:0] rf_wdata_wb_i,
153     input logic regfile_alu_we_ex_i,
154     /// LSU
155     input logic [31:0] lsu_rdata_wb_i,
156
157     input logic data_we_ex_i,
158     input logic [5:0] data_atop_ex_i,
159     input logic [1:0] data_type_ex_i,
160     input logic [31:0] alu_operand_c_ex_i,
161     input logic [1:0] data_reg_offset_ex_i,
162     input logic data_load_event_ex_i,
163     input logic [1:0] data_sign_ext_ex_i,
164     input logic [31:0] lsu_rdata_i,
165     input logic data_req_ex_i,

```

```

166 input logic [31:0] alu_operand_a_ex_i,
167 input logic [31:0] alu_operand_b_ex_i,
168 input logic useincr_addr_ex_i,
169 input logic data_misaligned_ex_i,
170 input logic p_elw_start_i,
171 input logic p_elw_finish_i,
172 input logic lsu_ready_ex_i,
173 input logic lsu_ready_wb_i,
174
175 input logic [3:0] lsu_data_be_i,
176
177 input logic data_req_pmp_i,
178 input logic data_gnt_pmp_i,
179 input logic data_rvalid_i,
180 input logic data_err_pmp_i,
181 input logic [31:0] data_addr_pmp_i,
182 input logic data_we_i,
183 input logic [ 5:0] data_atop_i,
184 input logic [ 3:0] data_be_i,
185 input logic [31:0] data_wdata_i,
186 input logic [31:0] data_rdata_i,
187
188 /// APU ///
189 input logic apu_req_i,
190 input logic apu_gnt_i,
191 input logic apu_rvalid_i,
192
193
194 // PC //
195 input logic [31:0] branch_addr_n_i,
196
197 // Controller FSM probes
198 input ctrl_state_e ctrl_fsm_cs_i,
199 input ctrl_state_e ctrl_fsm_ns_i,
200 input logic pending_single_step_i,
201 input logic single_step_allowed_i,
202 input logic pending_debug_i,
203 input logic debug_mode_q_i,
204 input logic [3:0] pc_mux_i,
205 input logic [2:0] exc_pc_mux_i,
206
207 // Interrupt Controller probes
208 input logic [31:0] irq_i,
209 input logic irq_wu_ctrl_i,
210 input logic [ 4:0] irq_id_ctrl_i,
211
212 /// CSR Probes ///
213 input csr_num_e csr_addr_i,
214 input logic csr_we_i,
215 input logic [31:0] csr_wdata_int_i,
216
217 input logic csr_fregs_we_i,
218 input logic csr_jvt_we_i,
219 input Status_t csr_mstatus_n_i,
220 input Status_t csr_mstatus_q_i,
221 input FS_t csr_mstatus_fs_n_i,
222 input FS_t csr_mstatus_fs_q_i,
223
224 input logic csr_mstatush_we_i,
225 input logic [31:0] csr_misa_n_i,
226 input logic [31:0] csr_misa_q_i,
227
228 input logic [31:0] csr_mie_n_i,
229 input logic [31:0] csr_mie_q_i,
230 input logic csr_mie_we_i,
231 input logic [23:0] csr_mtvec_n_i,
232 input logic [23:0] csr_mtvec_q_i,
233 input logic [ 1:0] csr_mtvec_mode_n_i,
234 input logic [ 1:0] csr_mtvec_mode_q_i,
235
236 input logic csr_mtv_twe_i,
237 input logic [31:0] csr_mcountinhibit_n_i,
238 input logic [31:0] csr_mcountinhibit_q_i,
239 input logic csr_mcountinhibit_we_i,
240 input logic [31:0][31:0] csr_mhpmevent_n_i,
241 input logic [31:0][31:0] csr_mhpmevent_q_i,
242 input logic [31:0] csr_mhpmevent_we_i,
243 input logic [31:0] csr_mscratch_n_i,
244 input logic [31:0] csr_mscratch_q_i,
245 input logic [31:0] csr_mepc_n_i,
246 input logic [31:0] csr_mepc_q_i,
247
248 input logic [ 5:0] csr_mcause_n_i,
249 input logic [ 5:0] csr_mcause_q_i,
250 input logic [31:0] csr_mip_n_i,
251 input logic [31:0] csr_mip_q_i,
252 input logic csr_mip_we_i,
253 input logic [31:0] csr_mnxti_n_i,
254 input logic [31:0] csr_mnxti_q_i,
255 input logic csr_mnxti_we_i,
256
257 input logic csr_mintstatus_we_i,
258 input logic [31:0] csr_mintthresh_n_i,
259 input logic [31:0] csr_mintthresh_q_i,
260 input logic csr_mintthresh_we_i,
261 input logic [31:0] csr_mscratchcs_w_n_i,
262 input logic [31:0] csr_mscratchcs_w_q_i,
263 input logic csr_mscratchcs_w_we_i,
264 input logic [31:0] csr_mscratchcs_wl_n_i,
265 input logic [31:0] csr_mscratchcs_wl_q_i,
266 input logic csr_mscratchcs_wl_we_i,
267
268 input logic [31:0] csr_mclibbase_n_i,
269 input logic [31:0] csr_mclibbase_q_i,
270 input logic [31:0] csr_mclibbase_we_i,
271 input logic [31:0] csr_tdata1_n_i,
272 input logic [31:0] csr_tdata1_q_i,
273 input logic [31:0] csr_tdata1_we_i,
274 input logic [31:0] csr_tdata2_n_i,
275 input logic [31:0] csr_tdata2_q_i,
276 input logic [31:0] csr_tdata2_we_i,
277 input logic [31:0] csr_tinfo_n_i,
278 input logic [31:0] csr_tinfo_q_i,
279
280 input Dcsr_t csr_dcsr_n_i,
281 input Dcsr_t csr_dcsr_q_i,
282
283 input logic [31:0] csr_dpc_n_i,
284 input logic [31:0] csr_dpc_q_i,
285 input logic [31:0] csr_dpc_we_i,
286 input logic [31:0] csr_dscratch0_n_i,
287 input logic [31:0] csr_dscratch0_q_i,
288 input logic [31:0] csr_dscratch0_we_i,
289 input logic [31:0] csr_dscratch1_n_i,
290 input logic [31:0] csr_dscratch1_q_i,
291 input logic [31:0] csr_dscratch1_we_i,
292 input logic [31:0] csr_mconfigptr_n_i,
293 input logic [31:0] csr_mconfigptr_q_i,
294 input logic [31:0] csr_mconfigptr_we_i,
295
296 // performance counters
297 // cycle, instret, hpcounter, cycleh, instreth,
298 hpcounterh
299 // mcycle, minstret, mhpcounter, mcycleh, minstreth,
300 mhpcounterh
301 input logic [63:0][MHPMCOUNTER_WIDTH-1:0]
302 csr_mhpcounter_q_i,
303 input logic [31:0]
304 csr_mhpcounter_write_lower_i,
305 input logic [31:0]
306 csr_mhpcounter_write_upper_i,
307
308 input logic [31:0] csr_mvvendorid_i,
309 input logic [31:0] csr_marchid_i,
310 input logic [31:0] csr_mhartid_i,
311 input logic [31:0] csr_mimpid_i,
312
313 input logic [31:0] csr_mcounteren_n_i,
314 input logic [31:0] csr_mcounteren_q_i,
315 input logic [31:0] csr_mcounteren_we_i,
316
317 input logic [ 7:0] csr_pmpcfg_n_i [16],
318 input logic [ 7:0] csr_pmpcfg_q_i [16],
319 input logic [15:0] csr_pmpcfg_we_i,
320 input logic [31:0] csr_pmpaddr_n_i, // PMP address input
321 shared for all pmpaddr registers
322 input logic [31:0] csr_pmpaddr_q_i [16],
323 input logic [15:0] csr_pmpaddr_we_i,
324 input logic [31:0] csr_mseccfg_n_i,
325 input logic [31:0] csr_mseccfg_q_i,
326 input logic [31:0] csr_mseccfg_we_i,
327 input logic [31:0] csr_mseccfgh_n_i,
328 input logic [31:0] csr_mseccfgh_q_i,
329 input logic [31:0] csr_mseccfgh_we_i,
330
331 input logic [4:0] csr_fcsr_fflags_n_i,
332 input logic [4:0] csr_fcsr_fflags_q_i,
333 input logic [4:0] csr_fcsr_fflags_we_i,
334 input logic [2:0] csr_fcsr_frm_n_i,
335 input logic [2:0] csr_fcsr_frm_q_i,
336
337 // RISC-V Formal Interface
338 // Does not comply with the coding standards of _i/_o
339 suffixes, but follow,
340 // the convention of RISC-V Formal Interface
341 Specification.
342 output logic [ 0:0] rvfi_valid,
343 output logic [63:0] rvfi_order,
344 output integer rvfi_start_cycle,
345 output time rvfi_start_time,
346 output integer rvfi_stop_cycle,
347 output time rvfi_stop_time,
348 output logic [31:0] rvfi_insn,
349 output rvfi_trap_t rvfi_trap,
350 output logic [ 0:0] rvfi_halt,
351 output rvfi_intr_t rvfi_intr,
352 output logic [ 1:0] rvfi_mode,
353 output logic [ 1:0] rvfi_ixl,
354 output logic [ 1:0] rvfi_nmip,
355
356 output logic [2:0] rvfi_dbg,
357 output logic [0:0] rvfi_dbg_mode,
358
359 output rvfi_wu_t rvfi_wu,
360 output logic [0:0] rvfi_sleep,
361
362 output logic [ 4:0] rvfi_rd_addr [1:0],
363 output logic [31:0] rvfi_rd_wdata [1:0],
364 output logic rvfi_fr_d_wvalid [1:0],
365 output logic [ 4:0] rvfi_fr_d_addr [1:0],
366 output logic [31:0] rvfi_fr_d_wdata [1:0],
367 output logic rvfi_2_rd,
368 output logic [ 4:0] rvfi_rs1_addr,

```

```

360 output logic [ 4:0] rvfi_rs2_addr,
361 output logic [ 4:0] rvfi_rs3_addr,
362 output logic [31:0] rvfi_rs1_rdata,
363 output logic [31:0] rvfi_rs2_rdata,
364 output logic [31:0] rvfi_rs3_rdata,
365 output logic [ 4:0] rvfi_frs1_addr,
366 output logic [ 4:0] rvfi_frs2_addr,
367 output logic [ 4:0] rvfi_frs3_addr,
368 output logic rvfi_frs1_rvalid,
369 output logic rvfi_frs2_rvalid,
370 output logic rvfi_frs3_rvalid,
371 output logic [31:0] rvfi_frs1_rdata,
372 output logic [31:0] rvfi_frs2_rdata,
373 output logic [31:0] rvfi_frs3_rdata,
374
375 output logic [31:0] rvfi_pc_rdata,
376 output logic [31:0] rvfi_pc_wdata,
377
378 output logic [31:0] rvfi_mem_addr,
379 output logic [31:0] rvfi_mem_rmask,
380 output logic [31:0] rvfi_mem_wmask,
381 output logic [31:0] rvfi_mem_rdata,
382 output logic [31:0] rvfi_mem_wdata,
383
384 // CSRs
385 output logic [31:0] rvfi_csr_fflags_rmask,
386 output logic [31:0] rvfi_csr_fflags_wmask,
387 output logic [31:0] rvfi_csr_fflags_rdata,
388 output logic [31:0] rvfi_csr_fflags_wdata,
389 output logic [31:0] rvfi_csr_frm_rmask,
390 output logic [31:0] rvfi_csr_frm_wmask,
391 output logic [31:0] rvfi_csr_frm_rdata,
392 output logic [31:0] rvfi_csr_frm_wdata,
393 output logic [31:0] rvfi_csr_fcsr_rmask,
394 output logic [31:0] rvfi_csr_fcsr_wmask,
395 output logic [31:0] rvfi_csr_fcsr_rdata,
396 output logic [31:0] rvfi_csr_fcsr_wdata,
397 output logic [31:0] rvfi_csr_jvt_rmask,
398 output logic [31:0] rvfi_csr_jvt_wmask,
399 output logic [31:0] rvfi_csr_jvt_rdata,
400 output logic [31:0] rvfi_csr_jvt_wdata,
401 output logic [31:0] rvfi_csr_mstatus_rmask,
402 output logic [31:0] rvfi_csr_mstatus_wmask,
403 output logic [31:0] rvfi_csr_mstatus_rdata,
404 output logic [31:0] rvfi_csr_mstatus_wdata,
405 output logic [31:0] rvfi_csr_mstatush_rmask,
406 output logic [31:0] rvfi_csr_mstatush_wmask,
407 output logic [31:0] rvfi_csr_mstatush_rdata,
408 output logic [31:0] rvfi_csr_mstatush_wdata,
409 output logic [31:0] rvfi_csr_misa_rmask,
410 output logic [31:0] rvfi_csr_misa_wmask,
411 output logic [31:0] rvfi_csr_misa_rdata,
412 output logic [31:0] rvfi_csr_misa_wdata,
413 output logic [31:0] rvfi_csr_mie_rmask,
414 output logic [31:0] rvfi_csr_mie_wmask,
415 output logic [31:0] rvfi_csr_mie_rdata,
416 output logic [31:0] rvfi_csr_mie_wdata,
417 output logic [31:0] rvfi_csr_mtvec_rmask,
418 output logic [31:0] rvfi_csr_mtvec_wmask,
419 output logic [31:0] rvfi_csr_mtvec_rdata,
420 output logic [31:0] rvfi_csr_mtvec_wdata,
421 output logic [31:0] rvfi_csr_mtv_t_rmask,
422 output logic [31:0] rvfi_csr_mtv_t_wmask,
423 output logic [31:0] rvfi_csr_mtv_t_rdata,
424 output logic [31:0] rvfi_csr_mtv_t_wdata,
425 output logic [31:0] rvfi_csr_mcountinhibit_rmask,
426 output logic [31:0] rvfi_csr_mcountinhibit_wmask,
427 output logic [31:0] rvfi_csr_mcountinhibit_rdata,
428 output logic [31:0] rvfi_csr_mcountinhibit_wdata,
429 output logic [31:0][31:0] rvfi_csr_mhpmevent_rmask, //
3-31 implemented
430 output logic [31:0][31:0] rvfi_csr_mhpmevent_wmask,
431 output logic [31:0][31:0] rvfi_csr_mhpmevent_rdata,
432 output logic [31:0][31:0] rvfi_csr_mhpmevent_wdata,
433 output logic [31:0] rvfi_csr_mscratch_rmask,
434 output logic [31:0] rvfi_csr_mscratch_wmask,
435 output logic [31:0] rvfi_csr_mscratch_rdata,
436 output logic [31:0] rvfi_csr_mscratch_wdata,
437 output logic [31:0] rvfi_csr_mepc_rmask,
438 output logic [31:0] rvfi_csr_mepc_wmask,
439 output logic [31:0] rvfi_csr_mepc_rdata,
440 output logic [31:0] rvfi_csr_mepc_wdata,
441 output logic [31:0] rvfi_csr_mcause_rmask,
442 output logic [31:0] rvfi_csr_mcause_wmask,
443 output logic [31:0] rvfi_csr_mcause_rdata,
444 output logic [31:0] rvfi_csr_mcause_wdata,
445 output logic [31:0] rvfi_csr_mtval_rmask,
446 output logic [31:0] rvfi_csr_mtval_wmask,
447 output logic [31:0] rvfi_csr_mtval_rdata,
448 output logic [31:0] rvfi_csr_mtval_wdata,
449 output logic [31:0] rvfi_csr_mip_rmask,
450 output logic [31:0] rvfi_csr_mip_wmask,
451 output logic [31:0] rvfi_csr_mip_rdata,
452 output logic [31:0] rvfi_csr_mip_wdata,
453 output logic [31:0] rvfi_csr_mnxti_rmask,
454 output logic [31:0] rvfi_csr_mnxti_wmask,
455 output logic [31:0] rvfi_csr_mnxti_rdata,
456 output logic [31:0] rvfi_csr_mnxti_wdata,
457 output logic [31:0] rvfi_csr_mintstatus_rmask,
458 output logic [31:0] rvfi_csr_mintstatus_wmask,
459 output logic [31:0] rvfi_csr_mintstatus_rdata,
460 output logic [31:0] rvfi_csr_mintstatus_wdata,
461 output logic [31:0] rvfi_csr_mintthresh_rmask,
462 output logic [31:0] rvfi_csr_mintthresh_wmask,
463 output logic [31:0] rvfi_csr_mintthresh_rdata,
464 output logic [31:0] rvfi_csr_mintthresh_wdata,
465 output logic [31:0] rvfi_csr_mscratchcs_wmask,
466 output logic [31:0] rvfi_csr_mscratchcs_wmask,
467 output logic [31:0] rvfi_csr_mscratchcs_wdata,
468 output logic [31:0] rvfi_csr_mscratchcs_wdata,
469 output logic [31:0] rvfi_csr_mscratchcs_wmask,
470 output logic [31:0] rvfi_csr_mscratchcs_wdata,
471 output logic [31:0] rvfi_csr_mscratchcs_wdata,
472 output logic [31:0] rvfi_csr_mscratchcs_wdata,
473 output logic [31:0] rvfi_csr_mclibase_rmask,
474 output logic [31:0] rvfi_csr_mclibase_wmask,
475 output logic [31:0] rvfi_csr_mclibase_rdata,
476 output logic [31:0] rvfi_csr_mclibase_wdata,
477 output logic [31:0] rvfi_csr_tselect_rmask,
478 output logic [31:0] rvfi_csr_tselect_wmask,
479 output logic [31:0] rvfi_csr_tselect_rdata,
480 output logic [31:0] rvfi_csr_tselect_wdata,
481 output logic [ 3:0][31:0] rvfi_csr_tdata_rmask, // 1-3
implemented
482 output logic [ 3:0][31:0] rvfi_csr_tdata_wmask,
483 output logic [ 3:0][31:0] rvfi_csr_tdata_rdata,
484 output logic [ 3:0][31:0] rvfi_csr_tdata_wdata,
485 output logic [31:0] rvfi_csr_tinfo_rmask,
486 output logic [31:0] rvfi_csr_tinfo_wmask,
487 output logic [31:0] rvfi_csr_tinfo_rdata,
488 output logic [31:0] rvfi_csr_tinfo_wdata,
489 output logic [31:0] rvfi_csr_mcontext_rmask,
490 output logic [31:0] rvfi_csr_mcontext_wmask,
491 output logic [31:0] rvfi_csr_mcontext_rdata,
492 output logic [31:0] rvfi_csr_mcontext_wdata,
493 output logic [31:0] rvfi_csr_scontext_rmask,
494 output logic [31:0] rvfi_csr_scontext_wmask,
495 output logic [31:0] rvfi_csr_scontext_rdata,
496 output logic [31:0] rvfi_csr_scontext_wdata,
497 output logic [31:0] rvfi_csr_dcsr_rmask,
498 output logic [31:0] rvfi_csr_dcsr_wmask,
499 output logic [31:0] rvfi_csr_dcsr_rdata,
500 output logic [31:0] rvfi_csr_dcsr_wdata,
501 output logic [31:0] rvfi_csr_dpc_rmask,
502 output logic [31:0] rvfi_csr_dpc_wmask,
503 output logic [31:0] rvfi_csr_dpc_rdata,
504 output logic [31:0] rvfi_csr_dpc_wdata,
505 output logic [ 1:0][31:0] rvfi_csr_dscratch_rmask, // 0-1
implemented
506 output logic [ 1:0][31:0] rvfi_csr_dscratch_wmask,
507 output logic [ 1:0][31:0] rvfi_csr_dscratch_rdata,
508 output logic [ 1:0][31:0] rvfi_csr_dscratch_wdata,
509 output logic [31:0] rvfi_csr_mcycle_rmask,
510 output logic [31:0] rvfi_csr_mcycle_wmask,
511 output logic [31:0] rvfi_csr_mcycle_rdata,
512 output logic [31:0] rvfi_csr_mcycle_wdata,
513 output logic [31:0] rvfi_csr_minstret_rmask,
514 output logic [31:0] rvfi_csr_minstret_wmask,
515 output logic [31:0] rvfi_csr_minstret_rdata,
516 output logic [31:0] rvfi_csr_minstret_wdata,
517 output logic [31:0][31:0] rvfi_csr_mhpmcounter_rmask, //
3-31 implemented
518 output logic [31:0][31:0] rvfi_csr_mhpmcounter_wmask,
519 output logic [31:0][31:0] rvfi_csr_mhpmcounter_rdata,
520 output logic [31:0][31:0] rvfi_csr_mhpmcounter_wdata,
521 output logic [31:0] rvfi_csr_mcycleh_rmask,
522 output logic [31:0] rvfi_csr_mcycleh_wmask,
523 output logic [31:0] rvfi_csr_mcycleh_rdata,
524 output logic [31:0] rvfi_csr_mcycleh_wdata,
525 output logic [31:0] rvfi_csr_minstreth_rmask,
526 output logic [31:0] rvfi_csr_minstreth_wmask,
527 output logic [31:0] rvfi_csr_minstreth_rdata,
528 output logic [31:0] rvfi_csr_minstreth_wdata,
529 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_rmask, //
3-31 implemented
530 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_wmask,
531 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_rdata,
532 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_wdata,
533 output logic [31:0] rvfi_csr_cycle_rmask,
534 output logic [31:0] rvfi_csr_cycle_wmask,
535 output logic [31:0] rvfi_csr_cycle_rdata,
536 output logic [31:0] rvfi_csr_cycle_wdata,
537 output logic [31:0] rvfi_csr_instret_rmask,
538 output logic [31:0] rvfi_csr_instret_wmask,
539 output logic [31:0] rvfi_csr_instret_rdata,
540 output logic [31:0] rvfi_csr_instret_wdata,
541 output logic [31:0][31:0] rvfi_csr_hpmcounter_rmask, //
3-31 implemented
542 output logic [31:0][31:0] rvfi_csr_hpmcounter_wmask,
543 output logic [31:0][31:0] rvfi_csr_hpmcounter_rdata,
544 output logic [31:0][31:0] rvfi_csr_hpmcounter_wdata,
545 output logic [31:0] rvfi_csr_cycleh_rmask,
546 output logic [31:0] rvfi_csr_cycleh_wmask,
547 output logic [31:0] rvfi_csr_cycleh_rdata,
548 output logic [31:0] rvfi_csr_cycleh_wdata,
549 output logic [31:0] rvfi_csr_instreth_rmask,
550 output logic [31:0] rvfi_csr_instreth_wmask,
551 output logic [31:0] rvfi_csr_instreth_rdata,
552 output logic [31:0] rvfi_csr_instreth_wdata,
553 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rmask, //

```

3-31 implemented

```

554 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wmask,
555 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rdata,
556 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wdata,
557 output logic [31:0] rvfi_csr_mvendordir_rmask,
558 output logic [31:0] rvfi_csr_mvendordir_wmask,
559 output logic [31:0] rvfi_csr_mvendordir_rdata,
560 output logic [31:0] rvfi_csr_mvendordir_wdata,
561 output logic [31:0] rvfi_csr_marchid_rmask,
562 output logic [31:0] rvfi_csr_marchid_wmask,
563 output logic [31:0] rvfi_csr_marchid_rdata,
564 output logic [31:0] rvfi_csr_marchid_wdata,
565 output logic [31:0] rvfi_csr_mimpid_rmask,
566 output logic [31:0] rvfi_csr_mimpid_wmask,
567 output logic [31:0] rvfi_csr_mimpid_rdata,
568 output logic [31:0] rvfi_csr_mimpid_wdata,
569 output logic [31:0] rvfi_csr_mhartid_rmask,
570 output logic [31:0] rvfi_csr_mhartid_wmask,
571 output logic [31:0] rvfi_csr_mhartid_rdata,
572 output logic [31:0] rvfi_csr_mhartid_wdata,
573
574 output logic [31:0] rvfi_csr_mcounteren_rmask,
575 output logic [31:0] rvfi_csr_mcounteren_wmask,
576 output logic [31:0] rvfi_csr_mcounteren_rdata,
577 output logic [31:0] rvfi_csr_mcounteren_wdata,
578
579 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_rmask,
580 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_wmask,
581 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_rdata,
582 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_wdata,
583 output logic [15:0][31:0] rvfi_csr_pmpaddr_rmask,
584 output logic [15:0][31:0] rvfi_csr_pmpaddr_wmask,
585 output logic [15:0][31:0] rvfi_csr_pmpaddr_rdata,
586 output logic [15:0][31:0] rvfi_csr_pmpaddr_wdata,
587 output logic [31:0] rvfi_csr_mseccfg_rmask,
588 output logic [31:0] rvfi_csr_mseccfg_wmask,
589 output logic [31:0] rvfi_csr_mseccfg_rdata,
590 output logic [31:0] rvfi_csr_mseccfg_wdata,
591 output logic [31:0] rvfi_csr_mseccfgh_rmask,
592 output logic [31:0] rvfi_csr_mseccfgh_wmask,
593 output logic [31:0] rvfi_csr_mseccfgh_rdata,
594 output logic [31:0] rvfi_csr_mseccfgh_wdata,
595
596 output logic [31:0] rvfi_csr_mconfigptr_rmask,
597 output logic [31:0] rvfi_csr_mconfigptr_wmask,
598 output logic [31:0] rvfi_csr_mconfigptr_rdata,
599 output logic [31:0] rvfi_csr_mconfigptr_wdata,
600
601 output logic [31:0] rvfi_csr_lptest0_rmask,
602 output logic [31:0] rvfi_csr_lptest0_wmask,
603 output logic [31:0] rvfi_csr_lptest0_rdata,
604 output logic [31:0] rvfi_csr_lptest0_wdata,
605 output logic [31:0] rvfi_csr_lptest0_rmask,
606 output logic [31:0] rvfi_csr_lptest0_wmask,
607 output logic [31:0] rvfi_csr_lptest0_rdata,
608 output logic [31:0] rvfi_csr_lptest0_wdata,
609 output logic [31:0] rvfi_csr_lptest0_rmask,
610 output logic [31:0] rvfi_csr_lptest0_wmask,
611 output logic [31:0] rvfi_csr_lptest0_rdata,
612 output logic [31:0] rvfi_csr_lptest0_wdata,
613 output logic [31:0] rvfi_csr_lptest1_rmask,
614 output logic [31:0] rvfi_csr_lptest1_wmask,
615 output logic [31:0] rvfi_csr_lptest1_rdata,
616 output logic [31:0] rvfi_csr_lptest1_wdata,
617 output logic [31:0] rvfi_csr_lptest1_rmask,
618 output logic [31:0] rvfi_csr_lptest1_wmask,
619 output logic [31:0] rvfi_csr_lptest1_rdata,
620 output logic [31:0] rvfi_csr_lptest1_wdata,
621 output logic [31:0] rvfi_csr_lptest1_rmask,
622 output logic [31:0] rvfi_csr_lptest1_wmask,
623 output logic [31:0] rvfi_csr_lptest1_rdata,
624 output logic [31:0] rvfi_csr_lptest1_wdata,
625 );
626
627 bit clk_i_d;
628 assign #0.01 clk_i_d = clk_i;
629
630 integer cycles;
631 // cycle counter
632 always_ff @(posedge clk_i_d, negedge rst_ni) begin
633   if (rst_ni == 1'b0) cycles <= 0;
634   else cycles <= cycles + 1;
635 end
636
637 logic pc_mux_debug;
638 logic pc_mux_dret;
639 logic pc_mux_exception;
640 logic pc_mux_debug_exception;
641 logic pc_mux_interrupt;
642 logic pc_mux_nmi;
643
644 localparam logic [31:0] MSTATUS_WRITE_MASK = 32'h0000_6088;
645 localparam logic [31:0] MCAUSEINHIBIT_WRITE_MASK = {{{(29-
NUM_MHPMCOUNTERS){1'b0}}, {(NUM_MHPMCOUNTERS){1'b1}}, 3'b101}};
646 localparam NUM_HPM_EVENTS = 16;
647 localparam logic [31:0] MHPMEVENT_WRITE_MASK = {{{(31-
NUM_HPM_EVENTS){1'b0}}, {(NUM_HPM_EVENTS){1'b1}}}};
648
649 `include "pipe_freeze_trace.sv"
650
651
652 `include "insn_trace.sv"
653
654 insn_trace_t trace_if, trace_id, trace_ex, trace_ex_next,
trace_wb;
655 insn_trace_t tmp_trace_wb;
656 insn_trace_t rvfi_trace_q[$], wb_bypass_trace_q[$];
657
658 bit clear_trace_ex; // = 1'b1;
659 bit clear_trace_wb;
660
661 logic csr_is_irq;
662 logic is_dbg_taken;
663 logic [2:0] saved_debug_cause;
664 integer next_send;
665
666 event e_empty_queue;
667 function void empty_fifo();
668   integer i, trace_q_size;
669   trace_q_size = wb_bypass_trace_q.size();
670   if (trace_q_size > 50) begin
671     $fatal("Reorder queue too long\n");
672   end
673   if (trace_q_size != 0) begin
674     for (i = 0; i < trace_q_size; i++) begin
675       insn_trace_t new_rvfi_trace;
676       new_rvfi_trace = new();
677     end
678     new_rvfi_trace.copy_full(wb_bypass_trace_q.pop_front());
679     if (next_send == new_rvfi_trace.m_order) begin
680       new_rvfi_trace.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
681       rvfi_trace_q.push_back(new_rvfi_trace);
682       next_send = next_send + 1;
683       ->e_empty_queue;
684     end else begin
685       wb_bypass_trace_q.push_back(new_rvfi_trace);
686     end
687   end
688   trace_q_size = wb_bypass_trace_q.size(); //Re-calculate
here for accurate status
689 endfunction
690 /*
691 * Function used to allocate a new insn and send it to the
rvfi driver
692 */
693 event e_add_to_bypass;
694 function void send_rvfi(insn_trace_t m_wb_insn);
695   insn_trace_t new_rvfi_trace;
696   new_rvfi_trace = new();
697   new_rvfi_trace.copy_full(m_wb_insn);
698   if (next_send == new_rvfi_trace.m_order) begin
699     rvfi_trace_q.push_back(new_rvfi_trace);
700     next_send = next_send + 1;
701   end else begin
702     wb_bypass_trace_q.push_back(new_rvfi_trace);
703     ->e_add_to_bypass;
704   end
705   empty_fifo();
706 endfunction
707
708 /*
709 * Assing rvfi signals once the instruction is completed
710 */
711 `define SET_RVFI_CSR_FROM_INSN(CSR_NAME) \
712   rvfi_csr_`CSR_NAME`_rdata =
new_rvfi_trace.m_csr.``CSR_NAME`_rdata; \
713   rvfi_csr_`CSR_NAME`_rmask =
new_rvfi_trace.m_csr.``CSR_NAME`_rmask; \
714   rvfi_csr_`CSR_NAME`_wdata =
new_rvfi_trace.m_csr.``CSR_NAME`_wdata; \
715   rvfi_csr_`CSR_NAME`_wmask =
new_rvfi_trace.m_csr.``CSR_NAME`_wmask;
716
717 logic [31:0] s_fflags_mirror;
718 logic [31:0] s_frm_mirror;
719 logic [31:0] s_fcsr_mirror;
720
721 function void set_rvfi();
722   insn_trace_t new_rvfi_trace;
723   new_rvfi_trace = rvfi_trace_q.pop_front();
724
725   if (new_rvfi_trace.m_is_apu) begin
726     if (new_rvfi_trace.m_csr.fflags_we) begin
727       s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
728     end else begin
729       s_fflags_mirror = new_rvfi_trace.m_csr.fflags_rdata;
730     end
731     if (new_rvfi_trace.m_csr.frm_we) begin
732       s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
733     end else begin
734       s_frm_mirror = new_rvfi_trace.m_csr.frm_rdata;
735     end
736     if (new_rvfi_trace.m_csr.fcsr_we) begin
737       s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
738     end else begin
739       s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_rdata;
740     end
741   end
742   end else begin
743     new_rvfi_trace.m_csr.fflags_rdata = s_fflags_mirror;

```

```

744 new_rvfi_trace.m_csr.frm_rdata = s_frm_mirror;
745 new_rvfi_trace.m_csr.fcsr_rdata = s_fcsr_mirror;
746
747 if (new_rvfi_trace.m_fflags_we_non_apu) begin
748   s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
749   s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
750   new_rvfi_trace.m_csr.fflags_wmask = 32'hFFFF_FFFF;
751   new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
752 end
753 if (new_rvfi_trace.m_frm_we_non_apu) begin
754   s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
755   s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
756   new_rvfi_trace.m_csr.frm_wmask = 32'hFFFF_FFFF;
757   new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
758 end
759 if (new_rvfi_trace.m_fcsr_we_non_apu) begin
760   s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
761   s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
762   s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
763   new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
764   new_rvfi_trace.m_csr.fflags_wmask = 32'hFFFF_FFFF;
765   new_rvfi_trace.m_csr.frm_wmask = 32'hFFFF_FFFF;
766 end
767 end
768
769 rvfi_order = new_rvfi_trace.m_order;
770 rvfi_start_cycle = new_rvfi_trace.m_start_cycle;
771 rvfi_start_time = new_rvfi_trace.m_start_time;
772 rvfi_stop_cycle = new_rvfi_trace.m_stop_cycle;
773 rvfi_stop_time = new_rvfi_trace.m_stop_time;
774 rvfi_pc_rdata = new_rvfi_trace.m_pc_rdata;
775 rvfi_insn = new_rvfi_trace.m_insn;
776
777 rvfi_rs1_addr = '0;
778 rvfi_rs1_rdata = '0;
779 rvfi_rs2_addr = '0;
780 rvfi_rs2_rdata = '0;
781 rvfi_rs3_addr = '0;
782 rvfi_rs3_rdata = '0;
783 rvfi_frs1_rvalid = '0;
784 rvfi_frs1_addr = '0;
785 rvfi_frs1_rdata = '0;
786 rvfi_frs2_rvalid = '0;
787 rvfi_frs2_addr = '0;
788 rvfi_frs2_rdata = '0;
789 rvfi_frs3_rvalid = '0;
790 rvfi_frs3_addr = '0;
791 rvfi_frs3_rdata = '0;
792
793 if (new_rvfi_trace.m_rs1_addr[5]) begin
794   rvfi_frs1_rvalid = '1'b1;
795   rvfi_frs1_addr = new_rvfi_trace.m_rs1_addr[4:0];
796   rvfi_frs1_rdata = new_rvfi_trace.m_rs1_rdata;
797 end else begin
798   rvfi_rs1_addr = new_rvfi_trace.m_rs1_addr[4:0];
799   rvfi_rs1_rdata = new_rvfi_trace.m_rs1_rdata;
800 end
801
802 if (new_rvfi_trace.m_rs2_addr[5]) begin
803   rvfi_frs2_rvalid = '1'b1;
804   rvfi_frs2_addr = new_rvfi_trace.m_rs2_addr[4:0];
805   rvfi_frs2_rdata = new_rvfi_trace.m_rs2_rdata;
806 end else begin
807   rvfi_rs2_addr = new_rvfi_trace.m_rs2_addr[4:0];
808   rvfi_rs2_rdata = new_rvfi_trace.m_rs2_rdata;
809 end
810
811 if (new_rvfi_trace.m_rs3_addr[5]) begin
812   rvfi_frs3_rvalid = '1'b1;
813   rvfi_frs3_addr = new_rvfi_trace.m_rs3_addr[4:0];
814   rvfi_frs3_rdata = new_rvfi_trace.m_rs3_rdata;
815 end else begin
816   rvfi_rs3_addr = new_rvfi_trace.m_rs3_addr[4:0];
817   rvfi_rs3_rdata = new_rvfi_trace.m_rs3_rdata;
818 end
819
820 rvfi_frd_wvalid[0] = '0;
821 rvfi_frd_addr[0] = '0;
822 rvfi_frd_wdata[0] = '0;
823
824 rvfi_frd_wvalid[1] = '0;
825 rvfi_frd_addr[1] = '0;
826 rvfi_frd_wdata[1] = '0;
827
828 rvfi_2_rd = new_rvfi_trace.m_2_rd_insn;
829 if (new_rvfi_trace.m_rd_addr[0][5] == '1'b0) begin
830   rvfi_rd_addr[0] = new_rvfi_trace.m_rd_addr[0][4:0];
831   rvfi_rd_wdata[0] = new_rvfi_trace.m_rd_wdata[0];
832 end else begin
833   rvfi_rd_addr[0] = '0;
834   rvfi_rd_wdata[0] = '0;
835
836   rvfi_frd_wvalid[0] = '1'b1;
837   rvfi_frd_addr[0] = new_rvfi_trace.m_rd_addr[0][4:0];
838   rvfi_frd_wdata[0] = new_rvfi_trace.m_rd_wdata[0];
839 end
840
841 if (new_rvfi_trace.m_2_rd_insn) begin
842   if (new_rvfi_trace.m_rd_addr[1][5] == '1'b0) begin
843     rvfi_rd_addr[1] = new_rvfi_trace.m_rd_addr[1][4:0];
844
845     rvfi_rd_wdata[1] = new_rvfi_trace.m_rd_wdata[1];
846   end else begin
847     rvfi_frd_wvalid[1] = '1'b1;
848     rvfi_frd_addr[1] = new_rvfi_trace.m_rd_addr[1][4:0];
849     rvfi_frd_wdata[1] = new_rvfi_trace.m_rd_wdata[1];
850   end
851
852   rvfi_rd_addr[1] = '0;
853   rvfi_rd_wdata[1] = '0;
854 end else begin
855   rvfi_rd_addr[1] = '0;
856   rvfi_rd_wdata[1] = '0;
857 end
858
859 rvfi_mem_addr = new_rvfi_trace.m_mem_addr;
860 rvfi_mem_rmask = new_rvfi_trace.m_mem_rmask;
861 rvfi_mem_wmask = new_rvfi_trace.m_mem_wmask;
862 rvfi_mem_rdata = new_rvfi_trace.m_mem_rdata;
863 rvfi_mem_wdata = new_rvfi_trace.m_mem_wdata;
864
865 rvfi_dbg = new_rvfi_trace.m_dbg_cause;
866 rvfi_dbg_mode = new_rvfi_trace.m_dbg_taken;
867
868 rvfi_trap.trap = 0;
869 if (new_rvfi_trace.m_is_illegal) begin
870   rvfi_trap.trap = '1'b1;
871 end
872
873 if (new_rvfi_trace.m_trap) begin
874   rvfi_trap.trap = '1'b1;
875 end
876
877 //CSR
878 rvfi_csr_mstatus_rmask =
879   new_rvfi_trace.m_csr.mstatus_rmask |
880   new_rvfi_trace.m_csr.mstatus_fs_rmask;
881 rvfi_csr_mstatus_wmask =
882   new_rvfi_trace.m_csr.mstatus_wmask & MSTATUS_WRITE_MASK;
883 rvfi_csr_mstatus_wmask[31] =
884   new_rvfi_trace.m_csr.mstatus_fs_wmask[31];
885 rvfi_csr_mstatus_wmask[14:13] =
886   new_rvfi_trace.m_csr.mstatus_fs_wmask[14:13];
887
888 if (FPU == 1 && ZFINX == 0) begin
889   rvfi_csr_mstatus_rdata[31] =
890   (new_rvfi_trace.m_csr.mstatus_fs_rdata == FS_DIRTY) ? '1'b1 : '1'b0;
891   end else begin
892     rvfi_csr_mstatus_rdata[31] = '0;
893   end
894   rvfi_csr_mstatus_rdata[30:18] = '0;
895   rvfi_csr_mstatus_rdata[17] =
896   new_rvfi_trace.m_csr.mstatus_rdata.mprv;
897   rvfi_csr_mstatus_rdata[16:15] = '0;
898   if (FPU == 1 && ZFINX == 0) begin
899     rvfi_csr_mstatus_rdata[14:13] =
900     new_rvfi_trace.m_csr.mstatus_fs_rdata;
901   end else begin
902     rvfi_csr_mstatus_rdata[14:13] = '0;
903   end
904   rvfi_csr_mstatus_rdata[12:11] =
905   new_rvfi_trace.m_csr.mstatus_rdata.mpp;
906   rvfi_csr_mstatus_rdata[10:8] = '0;
907   rvfi_csr_mstatus_rdata[7] =
908   new_rvfi_trace.m_csr.mstatus_rdata.mpie;
909   rvfi_csr_mstatus_rdata[6:5] = '0;
910   rvfi_csr_mstatus_rdata[4] =
911   new_rvfi_trace.m_csr.mstatus_rdata.upie;
912   rvfi_csr_mstatus_rdata[3] =
913   new_rvfi_trace.m_csr.mstatus_rdata.mie;
914   rvfi_csr_mstatus_rdata[2:1] = '0;
915   rvfi_csr_mstatus_rdata[0] =
916   new_rvfi_trace.m_csr.mstatus_rdata.uie;
917
918   if (FPU == 1 && ZFINX == 0) begin
919     rvfi_csr_mstatus_wdata[31] =
920     (new_rvfi_trace.m_csr.mstatus_fs_wdata == FS_DIRTY) ? '1'b1 : '1'b0;
921     end else begin
922       rvfi_csr_mstatus_wdata[31] = '0;
923     end
924     rvfi_csr_mstatus_wdata[30:18] = '0;
925     // MPRV is not implemented in the target configuration,
926     writes to it are ignored
927     rvfi_csr_mstatus_wdata[17] = '1'b0; //
928     new_rvfi_trace.m_csr.mstatus_wdata.mprv;
929     rvfi_csr_mstatus_wdata[16:15] = '0;
930     if (FPU == 1 && ZFINX == 0) begin
931       rvfi_csr_mstatus_wdata[14:13] =
932       new_rvfi_trace.m_csr.mstatus_fs_wdata;
933     end else begin
934       rvfi_csr_mstatus_wdata[14:13] = '0;
935     end
936     rvfi_csr_mstatus_wdata[12:11] =
937     new_rvfi_trace.m_csr.mstatus_wdata.mpp;
938     rvfi_csr_mstatus_wdata[10:8] = '0;
939     rvfi_csr_mstatus_wdata[7] =
940     new_rvfi_trace.m_csr.mstatus_wdata.mpie;
941     rvfi_csr_mstatus_wdata[6:5] = '0;
942     // UPIE is not implemented in the target configuration,
943     writes to it are ignored
944     rvfi_csr_mstatus_wdata[4] = '1'b0; //
945     new_rvfi_trace.m_csr.mstatus_wdata.upie;

```

```

925     rvfi_csr_mstatus_wdata[3] =
new_rvfi_trace.m_csr.mstatus_wdata.mie;
926     rvfi_csr_mstatus_wdata[2:1] = '0';
927     // UIE is not implemented in the target configuration,
writes to it are ignored
928     rvfi_csr_mstatus_wdata[0] = 1'b0; //
new_rvfi_trace.m_csr.mstatus_wdata.uiw;
929
930     `SET_RVFI_CSR_FROM_INSN(misa)
931     `SET_RVFI_CSR_FROM_INSN(mie)
932     `SET_RVFI_CSR_FROM_INSN(mtvec)
933
934     rvfi_csr_mcountinhibit_rdata =
new_rvfi_trace.m_csr.mcountinhibit_rdata;
935     rvfi_csr_mcountinhibit_rmask =
new_rvfi_trace.m_csr.mcountinhibit_rmask;
936     rvfi_csr_mcountinhibit_wdata =
new_rvfi_trace.m_csr.mcountinhibit_wdata;
937     rvfi_csr_mcountinhibit_wmask =
new_rvfi_trace.m_csr.mcountinhibit_wmask & MCOUNTINHIBIT_WRITE_MASK;
938
939     `SET_RVFI_CSR_FROM_INSN(mscratch)
940     `SET_RVFI_CSR_FROM_INSN(mepc)
941     `SET_RVFI_CSR_FROM_INSN(mcause)
942     `SET_RVFI_CSR_FROM_INSN(mcycle)
943     `SET_RVFI_CSR_FROM_INSN(minstret)
944     `SET_RVFI_CSR_FROM_INSN(minstreth)
945
946     // `SET_RVFI_CSR_FROM_INSN(cycle)
947     // `SET_RVFI_CSR_FROM_INSN(instret)
948     rvfi_csr_instret_rdata =
new_rvfi_trace.m_csr.minstret_rdata;
949     rvfi_csr_instret_rmask =
new_rvfi_trace.m_csr.minstret_rmask;
950     rvfi_csr_instret_wdata =
new_rvfi_trace.m_csr.minstret_wdata;
951     rvfi_csr_instret_wmask =
new_rvfi_trace.m_csr.minstret_wmask;
952
953     for(int idx=3; idx<32; idx++) begin
954         rvfi_csr_mhpmcounter_rmask[idx] =
new_rvfi_trace.m_csr.mhpmcounter_rmask[idx][31:0];
955         rvfi_csr_mhpmcounter_wmask[idx] =
new_rvfi_trace.m_csr.mhpmcounter_wmask[idx][31:0];
956         rvfi_csr_mhpmcounter_rdata[idx] =
new_rvfi_trace.m_csr.mhpmcounter_rdata[idx][31:0];
957         rvfi_csr_mhpmcounter_wdata[idx] =
new_rvfi_trace.m_csr.mhpmcounter_wdata[idx][31:0];
958
959         rvfi_csr_mhpmcounterh_rmask[idx] =
new_rvfi_trace.m_csr.mhpmcounterh_rmask[idx][63:32];
960         rvfi_csr_mhpmcounterh_wmask[idx] =
new_rvfi_trace.m_csr.mhpmcounterh_wmask[idx][63:32];
961         rvfi_csr_mhpmcounterh_rdata[idx] =
new_rvfi_trace.m_csr.mhpmcounterh_rdata[idx][63:32];
962         rvfi_csr_mhpmcounterh_wdata[idx] =
new_rvfi_trace.m_csr.mhpmcounterh_wdata[idx][63:32];
963
964         rvfi_csr_mhpmevent_rmask[idx] =
new_rvfi_trace.m_csr.mhpmevent_rmask[idx];
965         rvfi_csr_mhpmevent_wmask[idx] =
new_rvfi_trace.m_csr.mhpmevent_wmask[idx] & MHPMEVENT_WRITE_MASK;
966         rvfi_csr_mhpmevent_rdata[idx] =
new_rvfi_trace.m_csr.mhpmevent_rdata[idx];
967         rvfi_csr_mhpmevent_wdata[idx] =
new_rvfi_trace.m_csr.mhpmevent_wdata[idx];
968     end
969     // `SET_RVFI_CSR_FROM_INSN(minstreth)
970     rvfi_csr_minstreth_rdata =
new_rvfi_trace.m_csr.minstreth_rdata;
971     rvfi_csr_minstreth_rmask =
new_rvfi_trace.m_csr.minstreth_rmask;
972     rvfi_csr_minstreth_wdata =
new_rvfi_trace.m_csr.minstreth_wdata;
973     rvfi_csr_minstreth_wmask =
new_rvfi_trace.m_csr.minstreth_wmask;
974
975     `SET_RVFI_CSR_FROM_INSN(mip)
976
977     rvfi_csr_tdata_rdata[0] = 'Z';
978     rvfi_csr_tdata_rmask[0] = '0'; // Does not exist
979     rvfi_csr_tdata_wdata[0] = 'Z'; // Does not exist
980     rvfi_csr_tdata_wmask[0] = '0';
981
982     rvfi_csr_tdata_rdata[1] =
new_rvfi_trace.m_csr.tdata1_rdata;
983     rvfi_csr_tdata_rmask[1] =
new_rvfi_trace.m_csr.tdata1_rmask; // '1
984     rvfi_csr_tdata_wdata[1] =
new_rvfi_trace.m_csr.tdata1_wdata;
985     rvfi_csr_tdata_wmask[1] =
new_rvfi_trace.m_csr.tdata1_wmask;
986
987     rvfi_csr_tdata_rdata[2] =
new_rvfi_trace.m_csr.tdata2_rdata;
988     rvfi_csr_tdata_rmask[2] =
new_rvfi_trace.m_csr.tdata2_rmask; // '1
989     rvfi_csr_tdata_wdata[2] =
new_rvfi_trace.m_csr.tdata2_wdata;
990     rvfi_csr_tdata_wmask[2] =
new_rvfi_trace.m_csr.tdata2_wmask;

```

```

991
992     rvfi_csr_tdata_rmask[3] = '1';
993
994     `SET_RVFI_CSR_FROM_INSN(tinfo)
995     `SET_RVFI_CSR_FROM_INSN(dcsr)
996     `SET_RVFI_CSR_FROM_INSN(dpc)
997     rvfi_csr_dscratch_rmask[0] =
new_rvfi_trace.m_csr.dscratch0_rmask;
998     rvfi_csr_dscratch_wmask[0] =
new_rvfi_trace.m_csr.dscratch0_wmask;
999     rvfi_csr_dscratch_rdata[0] =
new_rvfi_trace.m_csr.dscratch0_rdata;
1000     rvfi_csr_dscratch_wdata[0] =
new_rvfi_trace.m_csr.dscratch0_wdata;
1001     rvfi_csr_dscratch_rmask[1] =
new_rvfi_trace.m_csr.dscratch1_rmask;
1002     rvfi_csr_dscratch_wmask[1] =
new_rvfi_trace.m_csr.dscratch1_wmask;
1003     rvfi_csr_dscratch_rdata[1] =
new_rvfi_trace.m_csr.dscratch1_rdata;
1004     rvfi_csr_dscratch_wdata[1] =
new_rvfi_trace.m_csr.dscratch1_wdata;
1005     // `SET_RVFI_CSR_FROM_INSN(dscratch0)
1006     `SET_RVFI_CSR_FROM_INSN(mvendoid)
1007     `SET_RVFI_CSR_FROM_INSN(marchid)
1008
1009     `SET_RVFI_CSR_FROM_INSN(fflags)
1010     `SET_RVFI_CSR_FROM_INSN(frm)
1011     `SET_RVFI_CSR_FROM_INSN(fcsr)
1012
1013     `SET_RVFI_CSR_FROM_INSN(lpstart0)
1014     `SET_RVFI_CSR_FROM_INSN(lpcount0)
1015     `SET_RVFI_CSR_FROM_INSN(lpstart1)
1016     `SET_RVFI_CSR_FROM_INSN(lpcount1)
1017     `SET_RVFI_CSR_FROM_INSN(lpcount1)
1018
1019 endfunction // set_rvfi
1020
1021 function void sample_perf_counter_to_id(int idx);
1022     trace_id.m_csr.mhpmcounter_rdata[idx][31:0] =
r_pipe_freeze_trace.csr.mhpmcounter_q[idx][31:0];
1023     trace_id.m_csr.mhpmcounter_rmask[idx][31:0] = '1';
1024 endfunction
1025
1026 function void perf_counter_to_id(int idx);
1027     if(!trace_id.m_csr.mhpmcounter_we[idx][0]) begin
1028         trace_id.m_csr.mhpmcounter_wdata[idx][31:0] =
r_pipe_freeze_trace.csr.wdata_int;
1029     end
1030     if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx])
begin
1031         trace_id.m_csr.mhpmcounter_we[idx][0] =
r_pipe_freeze_trace.csr.mhpmcounter_wdata[idx][31:0] =
r_pipe_freeze_trace.csr.wdata_int;
1032     end
1033     sample_perf_counter_to_id(idx);
1034 endfunction
1035
1036 function void sample_perf_event_to_trace(int idx,
insn_trace_t m_trace);
1037     m_trace.m_csr.mhpmevent_rdata[idx] =
r_pipe_freeze_trace.csr.mhpmevent_q[idx];
1038     m_trace.m_csr.mhpmevent_rmask[idx] = '1';
1039 endfunction
1040
1041 function void perf_event_to_trace(int idx, insn_trace_t
m_trace);
1042     if(!m_trace.m_csr.mhpmevent_we[idx]) begin
1043         m_trace.m_csr.mhpmevent_wdata[idx] =
r_pipe_freeze_trace.csr.wdata_int;
1044     end
1045     if(r_pipe_freeze_trace.csr.mhpmevent_we[idx]) begin
1046         m_trace.m_csr.mhpmevent_we[idx] =
r_pipe_freeze_trace.csr.mhpmevent_wdata[idx] =
r_pipe_freeze_trace.csr.wdata_int;
1047     end
1048     sample_perf_event_to_trace(idx, m_trace);
1049 endfunction
1050
1051 function void sample_minstret_to_trace(insn_trace_t
m_trace);
1052     m_trace.m_csr.minstret_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2][31:0];
1053     m_trace.m_csr.minstret_rmask = '1';
1054 endfunction
1055
1056 function void minstret_to_trace(insn_trace_t m_trace);
1057     if(!m_trace.m_csr.minstret_we) begin
1058         m_trace.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1059     end
1060     if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2])
begin
1061         m_trace.m_csr.minstret_we =

```

```

r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2];
1067     m_trace.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1068     m_trace.m_csr.minstret_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2] ? '1' : '0';
1069     end
1070     sample_minstret_to_trace(m_trace);
1071     endfunction
1072
1073     function void sample_perf_counter_h_to_id(int idx);
1074         trace_id.m_csr.mhpmcounter_rdata[idx][63:32] =
r_pipe_freeze_trace.csr.mhpmcounter_q[idx][63:0];
1075         trace_id.m_csr.mhpmcounter_rmask[idx][63:32] = '1';
1076     endfunction
1077
1078     function void perf_counter_h_to_id(int idx);
1079         if(!trace_id.m_csr.mhpmcounter_we[idx][1]) begin
1080             trace_id.m_csr.mhpmcounter_wdata[idx][63:32] =
r_pipe_freeze_trace.csr.wdata_int;
1081         end
1082         if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx])
begin
1083             trace_id.m_csr.mhpmcounter_we[idx][1] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx];
1084             trace_id.m_csr.mhpmcounter_wdata[idx][63:32] =
r_pipe_freeze_trace.csr.wdata_int;
1085             trace_id.m_csr.mhpmcounter_wmask[idx][63:32] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx] ? '1' : '0';
1086         end
1087         sample_perf_counter_h_to_id(idx);
1088     endfunction
1089
1090     function void sample_minstreth_to_trace(insn_trace_t
m_trace);
1091         m_trace.m_csr.minstreth_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2][63:32];
1092         m_trace.m_csr.minstreth_rmask = '1';
1093     endfunction
1094
1095     function void sample_mcycle_to_trace(insn_trace_t m_trace);
1096         m_trace.m_csr.mcycle_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[0];
1097         m_trace.m_csr.mcycle_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[31:0];
1098         m_trace.m_csr.mcycle_rmask = '1';
1099         m_trace.m_csr.mcycle_wdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[31:0];
1100         m_trace.m_csr.mcycle_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[0] ? '1' : '0';
1101     endfunction
1102
1103     function void minstreth_to_trace(insn_trace_t m_trace);
1104         if(!m_trace.m_csr.minstreth_we) begin
1105             m_trace.m_csr.minstreth_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1106         end
1107         if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2])
begin
1108             m_trace.m_csr.minstreth_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2];
1109             m_trace.m_csr.minstreth_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1110             m_trace.m_csr.minstreth_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2] ? '1' : '0';
1111         end
1112         sample_minstreth_to_trace(m_trace);
1113     endfunction
1114
1115     function void sample_perf_counter_to_trace(insn_trace_t
m_trace);
1116         sample_minstret_to_trace(m_trace);
1117         sample_minstreth_to_trace(m_trace);
1118         sample_mcycle_to_trace(m_trace);
1119         for(int idx=3; idx<32; idx++)begin
1120             sample_perf_event_to_trace(idx, m_trace); //TO CHANGE
1121         end
1122     endfunction
1123
1124     function void perf_counter_to_trace(insn_trace_t m_trace);
1125         if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2])
begin
1126             minstret_to_trace(m_trace);
1127         end
1128         if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2])
begin
1129             minstreth_to_trace(m_trace);
1130         end
1131         for(int idx=3; idx<32; idx++) begin
1132             if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx]) begin
1133                 perf_counter_to_id(idx);
1134             end
1135             if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[idx]) begin
1136                 perf_counter_h_to_id(idx);
1137             end
1138             if(r_pipe_freeze_trace.csr.mhpmevent_we[idx]) begin
1139                 perf_event_to_trace(idx, m_trace);
1140             end
1141         end
1142     endfunction

```

```

1143
1144     function void tinfo_to_trace(insn_trace_t m_trace);
1145         m_trace.m_csr.tinfo_we = '0'; // READ ONLY
csr_tinfo_we_i;
1146         m_trace.m_csr.tinfo_rdata =
r_pipe_freeze_trace.csr.tinfo_q;
1147         m_trace.m_csr.tinfo_rmask = '1';
1148         m_trace.m_csr.tinfo_wdata =
r_pipe_freeze_trace.csr.tinfo_n;
1149         m_trace.m_csr.tinfo_wmask = '0';
1150     endfunction
1151
1152     function void mtvec_to_id();
1153         trace_id.m_csr.mtvec_we =
r_pipe_freeze_trace.csr.mtvec_we;
1154         trace_id.m_csr.mtvec_rdata = {
1155             r_pipe_freeze_trace.csr.mtvec_q, 6'h0,
r_pipe_freeze_trace.csr.mtvec_mode_q
1156         };
1157         trace_id.m_csr.mtvec_rmask = '1';
1158         trace_id.m_csr.mtvec_wdata = {
1159             r_pipe_freeze_trace.csr.mtvec_n, 6'h0,
r_pipe_freeze_trace.csr.mtvec_mode_n
1160         };
1161     };
1162     trace_id.m_csr.mtvec_wmask =
r_pipe_freeze_trace.csr.mtvec_we ? '1' : '0';
1163     endfunction
1164
1165     function void dcsr_to_id();
1166         trace_id.m_csr.dcsr_wdata = trace_id.m_csr.dcsr_we ?
trace_id.m_csr.dcsr_wdata : r_pipe_freeze_trace.csr.dcsr_n;
1167         trace_id.m_csr.dcsr_we = r_pipe_freeze_trace.csr.dcsr_we |
trace_id.m_csr.dcsr_we;
1168         trace_id.m_csr.dcsr_rdata =
r_pipe_freeze_trace.csr.dcsr_q;
1169         trace_id.m_csr.dcsr_rmask = '1';
1170         trace_id.m_csr.dcsr_wdata =
r_pipe_freeze_trace.csr.dcsr_n;
1171         trace_id.m_csr.dcsr_wmask =
r_pipe_freeze_trace.csr.dcsr_we ? '1' : '0';
1172     endfunction
1173
1174     function void csrId_to_id();
1175         trace_id.m_csr.mvendorid_we = '0';
1176         trace_id.m_csr.mvendorid_rdata =
r_pipe_freeze_trace.csr.mvendorid;
1177         trace_id.m_csr.mvendorid_rmask = '1';
1178         trace_id.m_csr.mvendorid_wdata = '0';
1179         trace_id.m_csr.mvendorid_wmask = '0';
1180
1181         trace_id.m_csr.marchid_we = '0';
1182         trace_id.m_csr.marchid_rdata =
r_pipe_freeze_trace.csr.marchid;
1183         trace_id.m_csr.marchid_rmask = '0';
1184         trace_id.m_csr.marchid_wdata = '0';
1185         trace_id.m_csr.marchid_wmask = '0';
1186     endfunction
1187
1188     function void lpcount1_to_id();
1189         trace_id.m_csr.lpcount1_we = '0';
1190         trace_id.m_csr.lpcount1_rdata =
r_pipe_freeze_trace.hwloop.counter_q[1];
1191         trace_id.m_csr.lpcount1_rmask = '1';
1192         trace_id.m_csr.lpcount1_wdata = '0';
1193         trace_id.m_csr.lpcount1_wmask = '0';
1194     endfunction
1195
1196     function void lpcount0_to_id();
1197         trace_id.m_csr.lpcount0_we = '0';
1198         trace_id.m_csr.lpcount0_rdata =
r_pipe_freeze_trace.hwloop.counter_q[0];
1199         trace_id.m_csr.lpcount0_rmask = '1';
1200         trace_id.m_csr.lpcount0_wdata = '0';
1201         trace_id.m_csr.lpcount0_wmask = '0';
1202     endfunction
1203
1204     function void lpend0_to_id();
1205         trace_id.m_csr.lpend0_we = '0';
1206         trace_id.m_csr.lpend0_rdata =
r_pipe_freeze_trace.hwloop.end_q[0];
1207         trace_id.m_csr.lpend0_rmask = '1';
1208         trace_id.m_csr.lpend0_wdata = '0';
1209         trace_id.m_csr.lpend0_wmask = '0';
1210     endfunction
1211
1212     function void lpend1_to_id();
1213         trace_id.m_csr.lpend1_we = '0';
1214         trace_id.m_csr.lpend1_rdata =
r_pipe_freeze_trace.hwloop.end_q[1];
1215         trace_id.m_csr.lpend1_rmask = '1';
1216         trace_id.m_csr.lpend1_wdata = '0';
1217         trace_id.m_csr.lpend1_wmask = '0';
1218     endfunction
1219
1220     function void lpstart0_to_id();
1221         trace_id.m_csr.lpstart0_we = '0';
1222         trace_id.m_csr.lpstart0_rdata =
r_pipe_freeze_trace.hwloop.start_q[0];
1223         trace_id.m_csr.lpstart0_rmask = '1';

```

```

1224     trace_id.m_csr.lpstart0_wdata = '0';
1225     trace_id.m_csr.lpstart0_wmask = '0';
1226 endfunction
1227
1228 function void lpstart1_to_id();
1229     trace_id.m_csr.lpstart1_we = '0';
1230     trace_id.m_csr.lpstart1_rdata =
r_pipe_freeze_trace.hwloop.start_q[1];
1231     trace_id.m_csr.lpstart1_rmask = '1';
1232     trace_id.m_csr.lpstart1_wdata = '0';
1233     trace_id.m_csr.lpstart1_wmask = '0';
1234 endfunction
1235
1236 function void hwloop_to_id();
1237     lpcount0_to_id();
1238     lpend0_to_id();
1239     lpstart0_to_id();
1240
1241     lpcount1_to_id();
1242     lpend1_to_id();
1243     lpstart1_to_id();
1244 endfunction
1245
1246 bit s_was_flush; //debug exception is flagged as trap only
if preceed by a flush
1247 //Work around until I find the coreect way to distinguish
trap
1248 function void check_trap();
1249     bit s_dbg_exception, s_exception, s_irq;
1250     s_dbg_exception = '1'b0;
1251     s_exception = '1'b0;
1252     s_irq = '1'b0;
1253
1254     if (r_pipe_freeze_trace.pc_mux == PC_EXCEPTION) begin
1255         if (r_pipe_freeze_trace.exc_pc_mux[1] == '1'b1) begin
1256             if (r_pipe_freeze_trace.ctrl_fsm_cs != XRET_JUMP)
begin
1257                 s_dbg_exception = '1'b1;
1258             end
1259         end
1260         if (r_pipe_freeze_trace.exc_pc_mux == EXC_PC_EXCEPTION)
begin
1261             s_exception = '1'b1;
1262         end
1263         if (r_pipe_freeze_trace.exc_pc_mux == EXC_PC_IRQ) begin
1264             s_irq = '1'b1;
1265             trace_if.m_is_irq = '1'b1;
1266         end
1267     end
1268
1269     if (s_was_flush == '1'b0) begin
1270         s_dbg_exception = '1'b0;
1271     end
1272
1273 endfunction
1274 /*
1275  * This tracer works with three process,
1276  *
1277  * The first one simply stores information from the pipeline
every clock cycles and sends those information to the second process
1278  *
1279  * The second process uses those information to fill the
rvfi interface with the correct values
1280  *
1281  * The third updates the rvfi interface
1282  */
1283 `define CSR_FROM_PIPE(TRACE_NAME, CSR_NAME) \
1284     if(!trace_`TRACE_NAME`.m_csr.`CSR_NAME`_we) begin \
1285         trace_`TRACE_NAME`.m_csr.`CSR_NAME`_wdata =
r_pipe_freeze_trace.csr.`CSR_NAME`_n; \
1286     end \
1287     if (r_pipe_freeze_trace.csr.`CSR_NAME`_we) begin \
1288         trace_`TRACE_NAME`.m_csr.`CSR_NAME`_we =
r_pipe_freeze_trace.csr.`CSR_NAME`_we; \
1289         trace_`TRACE_NAME`.m_csr.`CSR_NAME`_wdata =
r_pipe_freeze_trace.csr.`CSR_NAME`_n; \
1290         trace_`TRACE_NAME`.m_csr.`CSR_NAME`_wmask = '1; \
1291     end \
1292     trace_`TRACE_NAME`.m_csr.`CSR_NAME`_rdata =
r_pipe_freeze_trace.csr.`CSR_NAME`_q; \
1293     trace_`TRACE_NAME`.m_csr.`CSR_NAME`_rmask = '1;
1294
1295 event e_mstatus_to_id;
1296 event e_fregs_dirty_1, e_fregs_dirty_2, e_fregs_dirty_3;
1297 function void mstatus_to_id();
1298     `CSR_FROM_PIPE(id, mstatus)
1299     `CSR_FROM_PIPE(id, mstatus_fs)
1300     if(r_pipe_freeze_trace.csr.fregs_we && !
r_pipe_freeze_trace.csr.mstatus_fs_we && !
(r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we)) begin //writes happening in
ex that needs to be reported to id
1301         trace_id.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1302         ->e_fregs_dirty_2;
1303     end
1304     ->e_mstatus_to_id;
1305 endfunction
1306 //those event are for debug purpose
1307 event e_dev_send_wb_1, e_dev_send_wb_2;
1308 event
1309     e_dev_commit_rf_to_ex_1,
1310     e_dev_commit_rf_to_ex_2,
1311     e_dev_commit_rf_to_ex_3,
1312     e_dev_commit_rf_to_ex_4,
1313     e_dev_commit_rf_to_ex_5;
1314 event e_if_2_id_1, e_if_2_id_2, e_if_2_id_3, e_if_2_id_4;
1315 event e_ex_to_wb_1, e_ex_to_wb_2;
1316 event e_id_to_ex_1, e_id_to_ex_2;
1317 event e_commit_dpc;
1318 event e_csr_in_ex, e_csr_irq;
1319
1320 event e_send_rvfi_trace_apu_resp;
1321 event
1322     e_send_rvfi_trace_ex_1,
1323     e_send_rvfi_trace_ex_2,
1324     e_send_rvfi_trace_ex_3,
1325     e_send_rvfi_trace_ex_4,
1326     e_send_rvfi_trace_ex_5,
1327     e_send_rvfi_trace_ex_6;
1328 event e_send_rvfi_trace_wb_1, e_send_rvfi_trace_wb_2,
e_send_rvfi_trace_wb_3;
1329 event e_send_rvfi_trace_id_1;
1330
1331 //used to match memory response to memory request and
corresponding instruction
1332 integer cnt_data_req, cnt_data_resp;
1333 integer cnt_apu_req, cnt_apu_resp;
1334
1335 insn_trace_t apu_trace_q[$];
1336 insn_trace_t trace_apu_req, trace_apu_resp;
1337
1338 function void csr_to_apu_resp();
1339     `CSR_FROM_PIPE(apu_resp, fcsr)
1340     `CSR_FROM_PIPE(apu_resp, fflags)
1341
1342     `CSR_FROM_PIPE(apu_resp, mstatus_fs)
1343
1344     if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_id.m_order > trace_apu_resp.m_order)) begin
1345         trace_id.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1346     end
1347     if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_ex.m_order > trace_apu_resp.m_order)) begin
1348         trace_ex.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1349     end
1350     if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_wb.m_order > trace_apu_resp.m_order)) begin
1351         trace_wb.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1352     end
1353 endfunction
1354
1355 function void csr_to_apu_req();
1356     `CSR_FROM_PIPE(apu_req, misa)
1357     `CSR_FROM_PIPE(apu_req, tdata1)
1358     `CSR_FROM_PIPE(apu_req, tdata2)
1359     trace_apu_req.m_csr.tinfo_we = '0; // READ ONLY
csr_tinfo_we_i;
1360     trace_apu_req.m_csr.tinfo_rdata =
r_pipe_freeze_trace.csr.tinfo_q;
1361     trace_apu_req.m_csr.tinfo_rmask = '1;
1362     trace_apu_req.m_csr.tinfo_wdata =
r_pipe_freeze_trace.csr.tinfo_n;
1363     trace_apu_req.m_csr.tinfo_wmask = '0;
1364
1365     trace_apu_req.m_csr.minstret_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2];
1366     trace_apu_req.m_csr.minstret_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2];
1367     trace_apu_req.m_csr.minstret_rmask = '1;
1368     trace_apu_req.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.mhpmcounter_q;
1369     trace_apu_req.m_csr.minstret_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2] ? '1 : '0;
1370
1371     trace_apu_req.m_csr.lpcount0_we = '0;
1372     trace_apu_req.m_csr.lpcount0_rdata =
r_pipe_freeze_trace.hwloop.counter_q[0];
1373     trace_apu_req.m_csr.lpcount0_rmask = '1;
1374     trace_apu_req.m_csr.lpcount0_wdata = '0;
1375     trace_apu_req.m_csr.lpcount0_wmask = '0;
1376
1377     trace_apu_req.m_csr.lpcount1_we = '0;
1378     trace_apu_req.m_csr.lpcount1_rdata =
r_pipe_freeze_trace.hwloop.counter_q[1];
1379     trace_apu_req.m_csr.lpcount1_rmask = '1;
1380     trace_apu_req.m_csr.lpcount1_wdata = '0;
1381     trace_apu_req.m_csr.lpcount1_wmask = '0;
1382
1383     `CSR_FROM_PIPE(apu_req, frm)
1384
1385 endfunction
1386
1387 function void fcsr_to_wb();
1388     `CSR_FROM_PIPE(wb, fflags)
1389     `CSR_FROM_PIPE(wb, frm)
1390     `CSR_FROM_PIPE(wb, fcsr)

```

```

1392 trace_wb.m_csr.fflags_wmask = '0';
1393 trace_wb.m_csr.frm_wmask = '0';
1394 trace_wb.m_csr.fcsr_wmask = '0';
1395 endfunction
1396
1397 bit s_apu_to_alu_port;
1398 bit s_apu_to_lsu_port;
1399
1400 function void apu_resp();
1401   if (!r_pipe_freeze_trace.wb_contention_lsu) begin
1402     if (apu_trace_q.size() > 0) begin
1403       trace_apu_resp = apu_trace_q.pop_front();
1404       if (s_apu_to_alu_port) begin
1405         if (r_pipe_freeze_trace.ex_reg_we) begin
1406           trace_apu_resp.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1407           trace_apu_resp.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1408         end
1409       end else if (s_apu_to_lsu_port) begin
1410         if (r_pipe_freeze_trace.rf_we_wb) begin
1411           trace_apu_resp.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1412           trace_apu_resp.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1413         end
1414       end
1415       csr_to_apu_resp();
1416
1417       trace_apu_resp.m_stop_cycle = cycles;
1418       trace_apu_resp.m_stop_time = $time;
1419       send_rvfi(trace_apu_resp);
1420       ->e_send_rvfi_trace_apu_resp;
1421     end
1422   end
1423 endfunction
1424
1425 /*
1426  * Decoding is complete and instruction enters execute stage
1427  * If at that time, minstret is not asserted it means we
have a trap
1428  */
1429 bit s_is_pc_set; //If pc_set, wait until next trace_id to
commit csr changes
1430 bit s_is_irq_start;
1431 bit s_id_done;
1432 function void if_to_id();
1433   if (trace_id.m_valid) begin
1434     `CSR_FROM_PIPE(id, misa)
1435     `CSR_FROM_PIPE(id, tdata1)
1436     `CSR_FROM_PIPE(id, tdata2)
1437     tinfo_to_trace(trace_id);
1438     `CSR_FROM_PIPE(id, mip)
1439     send_rvfi(trace_id);
1440   end
1441   trace_id.init(trace_if);
1442   trace_id.m_trap = ~r_pipe_freeze_trace.minstret;
1443   trace_id.m_is_illegal = trace_id.m_is_illegal |
r_pipe_freeze_trace.is_illegal;
1444   `CSR_FROM_PIPE(id, dpc)
1445   s_is_pc_set = 1'b0;
1446   s_is_irq_start = 1'b0;
1447   trace_if.m_valid = 1'b0;
1448   s_id_done = 1'b0;
1449 endfunction
1450
1451 function logic [31:0] be_to_mask(logic [3:0] be);
1452   logic [31:0] mask;
1453   mask[7:0] = (be[0] == 1'b1) ? 8'hFF : 8'h00;
1454   mask[15:8] = (be[1] == 1'b1) ? 8'hFF : 8'h00;
1455   mask[23:16] = (be[2] == 1'b1) ? 8'hFF : 8'h00;
1456   mask[31:24] = (be[3] == 1'b1) ? 8'hFF : 8'h00;
1457
1458   be_to_mask = mask;
1459   return mask;
1460 endfunction
1461
1462 function void commit_rf_to_trace(insn_trace_t m_trace);
1463   if (m_trace.m_got_ex_reg) begin
1464     m_trace.m_rd_addr[1] = r_pipe_freeze_trace.rf_addr_wb;
1465     m_trace.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1466     m_trace.m_2_rd_insn = 1'b1;
1467     m_trace.m_got_first_data = 1'b1;
1468   end else begin
1469     m_trace.m_rd_addr[0] = r_pipe_freeze_trace.rf_addr_wb;
1470     m_trace.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1471     m_trace.m_got_first_data = 1'b1;
1472   end
1473 endfunction
1474
1475 task compute_pipeline();
1476   bit s_new_valid_insn;
1477   bit s_ex_valid_adjusted;
1478   bit s_wb_valid_adjusted;
1479
1480   bit s_apu_wb_ok;
1481   bit s_apu_0_cycle_reps;
1482
1483   bit s_fflags_we_non_apu;

```

```

1484   bit s_frm_we_non_apu;
1485   bit s_fcsr_we_non_apu;
1486
1487   bit s_skip_wb; // used to skip wb monitoring when apu
resp and not lsu
1488
1489   bit s_core_is_decoding; // For readability, ctrl_fsm is
DECODE or DECODE_HWLOOP
1490
1491   bit s_ex_reg_we_adjusted; //ex_reg_we
1492   bit s_rf_we_wb_adjusted; //
1493
1494   bit s_dont_override_mstatus_fs_id;
1495
1496   trace_if = new();
1497   trace_id = new();
1498   trace_ex = new();
1499   trace_wb = new();
1500   s_new_valid_insn = 1'b0;
1501   s_ex_valid_adjusted = 1'b0;
1502
1503   s_id_done = 1'b0;
1504   s_apu_wb_ok = 1'b0;
1505   s_apu_0_cycle_reps = 1'b0;
1506
1507   next_send = 1;
1508   cnt_data_req = 0;
1509   cnt_data_resp = 0;
1510   cnt_apu_req = 0;
1511   cnt_apu_resp = 0;
1512   csr_is_irq = '0;
1513   is_dbg_taken = '0;
1514   s_was_flush = 1'b0;
1515
1516   s_is_pc_set = 1'b0;
1517   s_is_irq_start = 1'b0;
1518
1519   s_is_pc_set = 1'b0;
1520   s_is_irq_start = 1'b0;
1521   s_skip_wb = 1'b0;
1522
1523   s_core_is_decoding = 1'b0;
1524
1525   s_ex_reg_we_adjusted = 1'b0;
1526   s_rf_we_wb_adjusted = 1'b0;
1527
1528   s_dont_override_mstatus_fs_id = 1'b0;
1529
1530   forever begin
1531     wait(e_pipe_monitor_ok.triggered); // event triggered
#1;
1532
1533     s_core_is_decoding = (r_pipe_freeze_trace.ctrl_fsm_cs ==
DECODE) || (r_pipe_freeze_trace.ctrl_fsm_cs == DECODE_HWLOOP);
1534     check_trap();
1535
1536     pc_mux_interrupt = 1'b0;
1537     if (r_pipe_freeze_trace.pc_mux == 4'b0100) begin
1538       if (r_pipe_freeze_trace.exc_pc_mux == 3'b001) begin
1539         pc_mux_interrupt = 1'b1;
1540         s_is_irq_start = 1'b1;
1541       end
1542     end
1543
1544     if (r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID &&
r_pipe_freeze_trace.ebrk_insn_dec) begin
1545       if (trace_id.m_valid) begin
1546         `CSR_FROM_PIPE(id, misa)
1547         `CSR_FROM_PIPE(id, tdata1)
1548         `CSR_FROM_PIPE(id, tdata2)
1549         tinfo_to_trace(trace_id);
1550         `CSR_FROM_PIPE(id, mip)
1551       end
1552     end
1553
1554     if (r_pipe_freeze_trace.data_rvalid) begin
1555       cnt_data_resp = cnt_data_resp + 1;
1556     end
1557
1558     if (r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt) begin
1559       cnt_apu_req = cnt_apu_req + 1;
1560     end
1561
1562     if (r_pipe_freeze_trace.apu_rvalid) begin
1563       cnt_apu_resp = cnt_apu_resp + 1;
1564       if (r_pipe_freeze_trace.apu_singlecycle |
r_pipe_freeze_trace.apu_multicycle) begin
1565         s_apu_to_alu_port = 1'b1;
1566         s_apu_to_lsu_port = 1'b0;
1567       end else begin
1568         s_apu_to_lsu_port = 1'b1;
1569         s_apu_to_alu_port = 1'b0;
1570       end
1571     end else begin
1572       s_apu_to_lsu_port = 1'b0;
1573       s_apu_to_alu_port = 1'b0;
1574     end
1575
1576     if(r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt && r_pipe_freeze_trace.apu_rvalid &&

```

```

(cnt_apu_resp == cnt_apu_req)) begin
1577     s_apu_0_cycle_reps = 1'b1;
1578 end else begin
1579     s_apu_0_cycle_reps = 1'b0;
1580 end
1581
1582 if (trace_ex.m_valid & s_wb_valid_adjusted) begin
1583     // Used flopped values in case write happened before
wb_valid
1584     sample_perf_counter_to_trace(trace_ex);
1585     trace_ex.m_csr.got_minstret = '1;
1586 end
1587
1588 if (trace_wb.m_valid & trace_wb.m_is_apu &
(trace_wb.m_apu_req_id == cnt_apu_resp)) begin
1589     s_apu_wb_ok = 1'b1;
1590 end else begin
1591     s_apu_wb_ok = 1'b0;
1592 end
1593
1594 s_new_valid_insn = r_pipe_freeze_trace.id_valid &&
r_pipe_freeze_trace.is_decoding; // && !
r_pipe_freeze_trace.apu_rvalid;
1595
1596 s_wb_valid_adjusted = r_pipe_freeze_trace.wb_valid &&
(s_core_is_decoding || (r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX)
|| (r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_WB) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF)); // && !
r_pipe_freeze_trace.apu_rvalid;
1597
s_ex_reg_we_adjusted = r_pipe_freeze_trace.ex_reg_we &&
r_pipe_freeze_trace.mult_ready && r_pipe_freeze_trace.alu_ready &&
r_pipe_freeze_trace.lsu_ready_ex && !s_apu_to_alu_port;
1598
s_rf_we_wb_adjusted = r_pipe_freeze_trace.rf_we_wb &&
(~r_pipe_freeze_trace.data_misaligned_ex &&
r_pipe_freeze_trace.wb_ready) && (!s_apu_to_lsu_port ||
r_pipe_freeze_trace.wb_contention_lsu);
1599
s_fflags_we_non_apu = 1'b0;
1600
if (r_pipe_freeze_trace.csr.fflags_we) begin
1601
1602 if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1603     s_fflags_we_non_apu = 1'b1;
1604 end
1605 end
1606
s_frm_we_non_apu = 1'b0;
1607
if (r_pipe_freeze_trace.csr.frm_we) begin
1608
1609 if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1610     s_frm_we_non_apu = 1'b1;
1611 end
1612 end
1613
s_fcsr_we_non_apu = 1'b0;
1614
if (r_pipe_freeze_trace.csr.fcsr_we) begin
1615
1616 if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1617     s_fcsr_we_non_apu = 1'b1;
1618 end
1619 end
1620
//WB_STAGE
1621
s_skip_wb = 1'b0;
1622
if (r_pipe_freeze_trace.apu_rvalid &&
(apu_trace_q.size() > 0)) begin
1623
1624 apu_resp();
1625 if (!r_pipe_freeze_trace.data_rvalid) begin
1626     s_skip_wb = 1'b1;
1627 end
1628 end
1629
if (trace_wb.m_valid && !s_skip_wb &&
s_rf_we_wb_adjusted) begin
1630
1631 if (trace_wb.m_2_rd_insn) begin
1632     trace_wb.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1633     trace_wb.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1634 end else if (trace_wb.m_ex_fw) begin
1635     trace_wb.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1636     trace_wb.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1637     trace_wb.m_2_rd_insn = 1'b1;
1638 end else begin
1639     trace_wb.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1640     trace_wb.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1641 end
1642
1643 if (r_pipe_freeze_trace.csr.fregs_we &&
r_pipe_freeze_trace.rf_we_wb && r_pipe_freeze_trace.rf_addr_wb[5])
begin //Catching mstatus updates caused by flw
1644     `CSR_FROM_PIPE(wb, mstatus_fs)
1645     trace_wb.m_csr.mstatus_fs_we = 1'b1;
1646     trace_wb.m_csr.mstatus_fs_wmask = '1;
1647     if (r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we) begin //In this specific

```

```

case, two writes to mstatus_fs happen at the same time. We need to
recreate the writes caused by fregs_we
1648     trace_wb.m_csr.mstatus_fs_wdata = FS_DIRTY;
1649 end
1650     ->e_fregs_dirty_1;
1651 end
1652
send_rvfi(trace_wb);
1653
->e_dev_send_wb_1; ->e_send_rvfi_trace_wb_2;
1654
trace_wb.m_valid = 1'b0;
1655
1656 end
1657
1658 if (trace_ex.m_valid) begin
1659     if (trace_ex.m_instret_smample_trigger == 1) begin //
time to sample instret
1660         sample_perf_counter_to_trace(trace_ex);
1661     end
1662     trace_ex.m_instret_smample_trigger =
trace_ex.m_instret_smample_trigger + 1;
1663
1664     `CSR_FROM_PIPE(ex, misa)
1665     `CSR_FROM_PIPE(ex, tdata1)
1666     `CSR_FROM_PIPE(ex, tdata2)
1667     tinfo_to_trace(trace_ex);
1668
1669 if (s_rf_we_wb_adjusted) begin
1670     ->e_dev_commit_rf_to_ex_4;
1671     if (!(trace_ex.m_got_ex_reg) &&
trace_ex.m_mem_req_id_valid[0]) begin
1672         trace_ex.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1673         trace_ex.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1674         trace_ex.m_got_first_data = 1'b1;
1675         trace_ex.m_mem_req_id_valid[0] = 1'b0;
1676     end else if (trace_ex.m_mem_req_id_valid[1]) begin
1677         trace_ex.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1678         trace_ex.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1679         trace_ex.m_got_first_data = 1'b1;
1680         trace_ex.m_mem_req_id_valid[1] = 1'b0;
1681     end
1682 end
1683
1684 if (s_wb_valid_adjusted) begin
1685     if (trace_wb.m_valid) begin
1686         send_rvfi(trace_ex);
1687         trace_ex.m_valid = 1'b0;
1688         ->e_send_rvfi_trace_ex_2;
1689     end else begin
1690
1691         if (s_rf_we_wb_adjusted) begin
1692             ->e_dev_commit_rf_to_ex_1;
1693             commit_rf_to_trace(trace_ex);
1694
1695             if (r_pipe_freeze_trace.csr.fregs_we &&
(r_pipe_freeze_trace.rf_we_wb && r_pipe_freeze_trace.rf_addr_wb[5]))
begin //Catching mstatus updates caused by flw
1696                 `CSR_FROM_PIPE(ex, mstatus_fs)
1697                 trace_ex.m_csr.mstatus_fs_we = 1'b1;
1698                 trace_ex.m_csr.mstatus_fs_wmask = '1;
1699                 if (r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we) begin //In this specific
case, two writes to mstatus_fs happen at the same time. We need to
recreate the writes caused by fregs_we
1700                     trace_ex.m_csr.mstatus_fs_wdata = FS_DIRTY;
1701                     end else begin
1702                         trace_ex.m_csr.mstatus_fs_rdata =
trace_ex.m_csr.mstatus_fs_wdata;
1703                         s_dont_override_mstatus_fs_id = 1'b1;
1704                     end
1705                     ->e_fregs_dirty_3;
1706                 end
1707
1708                 send_rvfi(trace_ex);
1709                 trace_ex.m_valid = 1'b0;
1710
1711             end else begin
1712                 if (trace_ex.m_is_load) begin // only move
relevant instr in wb stage
1713                     ->e_ex_to_wb_1;
1714                     trace_wb.move_down_pipe(trace_ex);
1715                 end else begin
1716                     send_rvfi(trace_ex);
1717                     ->e_send_rvfi_trace_ex_6;
1718                 end
1719                 trace_ex.m_valid = 1'b0;
1720             end
1721             end else if (s_rf_we_wb_adjusted && !s_was_flush)
begin
1722                 ->e_dev_commit_rf_to_ex_2;
1723                 commit_rf_to_trace(trace_ex);
1724             end
1725         end
1726     end
1727 end
1728
1729 // If mret, we need to keep the instruction in Id during
flush_ex because mstatus update happens at that time

```

```

1730     s_ex_valid_adjusted = (r_pipe_freeze_trace.ex_valid &&
r_pipe_freeze_trace.ex_ready) && (s_core_is_decoding ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
((r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX) && !
r_pipe_freeze_trace.mret_insn_dec));
1731     //EX_STAGE
1732
1733     if (trace_id.m_valid) begin
1734         if (trace_id.m_instret_smample_trigger == 1) begin //
time to sample instret
1735             sample_perf_counter_to_trace(trace_id);
1736             for (int idx=3; idx<32; idx++) begin
1737                 sample_perf_counter_to_id(idx);
1738                 sample_perf_counter_h_to_id(idx);
1739                 sample_perf_event_to_trace(idx, trace_id);
1740             end
1741         end
1742         trace_id.m_instret_smample_trigger =
trace_id.m_instret_smample_trigger + 1;
1743
1744         if (trace_id.m_sample_csr_write_in_ex && !csr_is_irq
&& !s_is_irq_start) begin //First cycle after id_ready, csr write is
asserted in this cycle
1745             `CSR_FROM_PIPE(id, mstatus)
1746             if (!s_dont_override_mstatus_fs_id) begin
1747                 `CSR_FROM_PIPE(id, mstatus_fs)
1748             end
1749             `CSR_FROM_PIPE(id, mepc)
1750             `CSR_FROM_PIPE(id, mcause)
1751             `CSR_FROM_PIPE(id, dscratch0)
1752             `CSR_FROM_PIPE(id, dscratch1)
1753             if (r_pipe_freeze_trace.csr.we &&
(r_pipe_freeze_trace.csr.addr == CSR_DPC)) begin
1754                 `CSR_FROM_PIPE(id, dpc)
1755             end
1756             `CSR_FROM_PIPE(id, mcountinhibit)
1757             perf_counter_to_trace(trace_id);
1758             ->e_csr_in_ex;
1759         end
1760
1761         if (r_pipe_freeze_trace.is_decoding) begin
1762             trace_id.m_sample_csr_write_in_ex = 1'b0;
1763         end
1764         mtvec_to_id();
1765
1766         `CSR_FROM_PIPE(id, mip)
1767         `CSR_FROM_PIPE(id, misa)
1768
1769         `CSR_FROM_PIPE(id, mcountinhibit)
1770         `CSR_FROM_PIPE(id, mscratch)
1771         `CSR_FROM_PIPE(id, mie)
1772
1773         `CSR_FROM_PIPE(id, fflags)
1774         `CSR_FROM_PIPE(id, frm)
1775         `CSR_FROM_PIPE(id, fcsr)
1776
1777         if (r_pipe_freeze_trace.csr.dcsr_we) begin
1778             dcsr_to_id();
1779         end
1780
1781         if (s_fflags_we_non_apu) begin
1782             trace_id.m_fflags_we_non_apu = 1'b1;
1783         end
1784
1785         if (s_frm_we_non_apu) begin
1786             trace_id.m_frm_we_non_apu = 1'b1;
1787         end
1788
1789         if (s_fcsr_we_non_apu) begin
1790             trace_id.m_fcsr_we_non_apu = 1'b1;
1791         end
1792
1793         trace_ex.m_csr.fflags_wmask = '0;
1794         trace_ex.m_csr.frm_wmask = '0;
1795         trace_ex.m_csr.fcsr_wmask = '0;
1796
1797         if (r_pipe_freeze_trace.ctrl_fsm_cs == XRET_JUMP)
begin //xret exit pipeline
1800             tinfo_to_trace(trace_id);
1801             `CSR_FROM_PIPE(id, tdata1)
1802             `CSR_FROM_PIPE(id, tdata2)
1803             send_rvfi(trace_id);
1804             trace_id.m_valid = 1'b0;
1805             s_dont_override_mstatus_fs_id = 1'b0;
1806         end
1807
1808         if (r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt) begin
1809             trace_id.m_is_apu = 1'b1;
1810             trace_id.m_apu_req_id = cnt_apu_req;
1811             trace_apu_req = new();
1812             trace_apu_req.copy_full(trace_id);
1813             csr_to_apu_req();
1814             trace_apu_req.set_to_apu();
1815             apu_trace.q.push_back(trace_apu_req);
1816             trace_id.m_valid = 1'b0;
1817             s_dont_override_mstatus_fs_id = 1'b0;

```

```

1818
1819         if (r_pipe_freeze_trace.apu_rvalid && (cnt_apu_req ==
cnt_apu_resp)) begin//APU return in the same cycle
1820             apu_resp();
1821         end
1822     end
1823
1824     if (s_ex_valid_adjusted) begin //A valid instruction
goes from ID to EX
1825         if (trace_ex.m_valid) begin //We need to bypass wb
1826             send_rvfi(trace_ex);
1827             trace_ex.m_valid = 1'b0;
1828             ->e_send_rvfi_trace_ex_3;
1829         end
1830         if (r_pipe_freeze_trace.ex_reg_we && !
s_apu_to_alu_port) begin
1831             trace_id.m_ex_fw = 1'b1;
1832             trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1833             trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1834             trace_id.m_got_ex_reg = 1'b1;
1835         end else if (!trace_ex.m_valid & s_rf_we_wb_adjusted
& !trace_id.m_ex_fw) begin
1836             trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1837             trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1838             end else if (s_rf_we_wb_adjusted) begin
1839                 trace_id.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1840                 trace_id.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1841                 trace_id.m_2_rd_insn = 1'b1;
1842             end
1843
1844             if (r_pipe_freeze_trace.data_req_ex) begin
1845                 cnt_data_req = cnt_data_req + 1;
1846                 trace_id.m_is_memory = 1'b1;
1847                 trace_id.m_mem_req_id[0] = cnt_data_req;
1848                 trace_id.m_mem_req_id_valid[0] = 1'b1;
1849
1850                 trace_id.m_mem.addr =
r_pipe_freeze_trace.data_addr_pmp;
1851                 if (r_pipe_freeze_trace.data_misaligned) begin
1852                     cnt_data_req = cnt_data_req + 1;
1853                     trace_id.m_mem_req_id[0] = cnt_data_req;
1854                 end
1855
1856                 if (!r_pipe_freeze_trace.data_we_ex) begin
1857                     trace_id.m_is_load = 1'b1;
1858                     trace_id.m_mem.wmask =
be_to_mask(r_pipe_freeze_trace.lsu_data_be); //1;
1859                 end else begin
1860                     trace_id.m_mem.rmask =
be_to_mask(r_pipe_freeze_trace.lsu_data_be); //1;
1861                 end
1862
1863                 if (trace_id.m_got_ex_reg) begin // Shift index 0
to 1
1864                     trace_id.m_mem_req_id[1] =
trace_id.m_mem_req_id[0];
1865                     trace_id.m_mem_req_id[0] = 0;
1866                     trace_id.m_mem_req_id_valid[1] = 1'b1;
1867                     trace_id.m_mem_req_id_valid[0] = 1'b0;
1868                 end
1869             end
1870             hwloop_to_id();
1871             trace_ex.move_down_pipe(trace_id); // The
instruction moves forward from ID to EX
1872             trace_id.m_valid = 1'b0;
1873             s_dont_override_mstatus_fs_id = 1'b0;
1874             ->e_id_to_ex_1;
1875
1876             end else if (r_pipe_freeze_trace.ex_reg_we &&
r_pipe_freeze_trace.rf_alu_we_ex) begin
1877                 trace_id.m_ex_fw = 1'b1;
1878                 trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1879                 trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1880                 trace_id.m_got_ex_reg = 1'b1;
1881                 trace_id.m_got_regs_write = 1'b1;
1882                 // mem_req_id[0] already set here indicates req_id
was set before rf write from EX
1883                 // Hence adjust the req_id again here for such cases
1884                 if (trace_id.m_mem_req_id[0] != 0) begin
1885                     trace_id.m_mem_req_id[1] =
trace_id.m_mem_req_id[0];
1886                     trace_id.m_mem_req_id[0] = 0;
1887                     trace_id.m_mem_req_id_valid[0] = 1'b0;
1888                     trace_id.m_mem_req_id_valid[1] = 1'b1;
1889                 end
1890             end
1891         end //trace_id.m_valid
1892
1893         //ID_STAGE
1894         if (s_new_valid_insn) begin // There is a new valid

```

```

instruction
1895     if (trace_id.m_valid) begin
1896         if (trace_ex.m_valid) begin
1897             if (trace_wb.m_valid) begin
1898                 send_rvfi(trace_ex);
1899                 ->e_send_rvfi_trace_ex_4;
1900             end else begin
1901                 if (trace_ex.m_is_load) begin // only move
relevant instr in wb stage
1902                     ->e_ex_to_wb_2;
1903                     trace_wb.move_down_pipe(trace_ex);
1904                 end else begin
1905                     send_rvfi(trace_ex);
1906                     ->e_send_rvfi_trace_ex_5;
1907                 end
1908             end
1909             trace_ex.m_valid = 1'b0;
1910         end
1911         if (r_pipe_freeze_trace.data_req_ex) begin
1912             cnt_data_req = cnt_data_req + 1;
1913             trace_id.m_is_memory = 1'b1;
1914             trace_id.m_mem_req_id[0] = cnt_data_req;
1915             trace_id.m_mem_req_id_valid[0] = 1'b1;
1916             trace_id.m_mem_addr =
r_pipe_freeze_trace.data_addr_pmp;
1917             if (r_pipe_freeze_trace.data_misaligned) begin
1918                 cnt_data_req = cnt_data_req + 1;
1919                 trace_id.m_mem_req_id[0] = cnt_data_req;
1920             end
1921             if (!r_pipe_freeze_trace.data_we_ex) begin
1922                 trace_id.m_is_load = 1'b1;
1923             end
1924             if (trace_id.m_got_ex_reg) begin // Shift index 0
to 1
1925                 trace_id.m_mem_req_id[1] =
trace_id.m_mem_req_id[0];
1926                 trace_id.m_mem_req_id[0] = 0;
1927                 trace_id.m_mem_req_id_valid[0] = 1'b0;
1928                 trace_id.m_mem_req_id_valid[1] = 1'b1;
1929             end
1930             end else if (s_rf_we_wb_adjusted && !
r_pipe_freeze_trace.ex_reg_we) begin
1931                 trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1932                 trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1933             end
1934             hwlloop_to_id();
1935             trace_ex.move_down_pipe(trace_id);
1936             trace_id.m_valid = 1'b0;
1937             s_dont_override_mstatus_fs_id = 1'b0;
1938             ->e_id_to_ex_2;
1939         end
1940         if_to_id();
1941         trace_id.m_is_ebreak = trace_if.m_is_ebreak;
1942         csrId_to_id();
1943         `CSR_FROM_PIPE(id, dscratch0)
1944         `CSR_FROM_PIPE(id, dscratch1)
1945         mstatus_to_id();
1946         ->e_if_2_id_1;
1947     end else begin
1948         if (trace_id.m_valid) begin
1949             `CSR_FROM_PIPE(id, dscratch0)
1950             `CSR_FROM_PIPE(id, dscratch1)
1951             mstatus_to_id();
1952             ->e_if_2_id_1;
1953         end
1954     end
1955     //IF_STAGE
1956     if(trace_if.m_valid) begin
1957         if(r_pipe_freeze_trace.is_illegal &&
r_pipe_freeze_trace.is_decoding) begin
1958             trace_if.m_is_illegal = 1'b1;
1959         end
1960     end
1961     if (r_pipe_freeze_trace.if_valid &&
r_pipe_freeze_trace.if_ready && r_pipe_freeze_trace.instr_valid_if)
begin
1962         if (trace_if.m_valid) begin
1963             if (r_pipe_freeze_trace.id_valid &&
r_pipe_freeze_trace.id_ready && !trace_id.m_valid &&
r_pipe_freeze_trace.ebrk_insn_dec) begin
1964                 if_to_id();
1965                 trace_id.m_is_ebreak = '1; //
trace_if.m_is_ebreak;
1966                 ->e_if_2_id_2;
1967             end else if (trace_if.m_is_illegal) begin
1968                 if_to_id();
1969                 ->e_if_2_id_3;
1970             end else if (r_pipe_freeze_trace.ecall_insn_dec)
begin
1971                 if_to_id();
1972                 ->e_if_2_id_4;
1973             end
1974         end
1975     end
1976     end
1977     trace_if.m_insn = r_pipe_freeze_trace.instr_if; //
Instr comes from if, buffer for one cycle
1979     trace_if.m_pc_rdata = r_pipe_freeze_trace.pc_if;
1980     trace_if.m_dbg_taken = is_dbg_taken;
1981     trace_if.m_dbg_cause = saved_debug_cause;
1982     trace_if.m_is_ebreak = '0;
1983     trace_if.m_trap = 1'b0;
1984     trace_if.m_valid = 1'b1;
1985     end
1986     if (csr_is_irq && !s_is_pc_set) begin
1987         mstatus_to_id();
1988         `CSR_FROM_PIPE(id, mepc)
1989         `CSR_FROM_PIPE(id, mcause)
1990         ->e_csr_irq;
1991     end
1992     if (!s_id_done && r_pipe_freeze_trace.is_decoding) begin
1993         dcsr_to_id();
1994         ->e_commit_dpc;
1995     end
1996     if (r_pipe_freeze_trace.pc_set) begin
1997         s_is_pc_set = 1'b1;
1998     end
1999     csr_is_irq = r_pipe_freeze_trace.csr_cause[5];
2000
2001     is_dbg_taken = ((r_pipe_freeze_trace.ctrl_fsm_cs ==
DBG_TAKEN_ID) | (r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF)) ?
1'b1 : 1'b0;
2002     saved_debug_cause = r_pipe_freeze_trace.debug_cause;
2003     s_id_done = r_pipe_freeze_trace.id_valid;
2004     if((r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_WB)) begin
2005         s_was_flush = 1'b1;
2006     end else begin
2007         s_was_flush = 1'b0;
2008     end
2009     #1;
2010     endtask
2011
2012     task update_rvfi();
2013     rvfi_valid = 1'b0;
2014     rvfi_order = '0;
2015     rvfi_insn = '0;
2016     rvfi_pc_rdata = '0;
2017     rvfi_rd_addr[0] = '0;
2018     rvfi_rd_wdata[0] = '0;
2019     rvfi_rd_addr[1] = '0;
2020     rvfi_rd_wdata[1] = '0;
2021     rvfi_rs1_addr = '0;
2022     rvfi_rs2_addr = '0;
2023     rvfi_rs3_addr = '0;
2024     rvfi_rs1_rdata = '0;
2025     rvfi_rs2_rdata = '0;
2026     rvfi_rs3_rdata = '0;
2027     rvfi_mem_addr = '0;
2028     rvfi_mem_rmask = '0;
2029     rvfi_mem_wmask = '0;
2030     rvfi_mem_rdata = '0;
2031     rvfi_mem_wdata = '0;
2032     rvfi_mode = 2'b11; //priv_lvl_i; //TODO: correct
this if needed
2033     $display("*****Starting update rvfi task*****\n");
2034     wait(clk_i_d == 1'b1);
2035     forever begin
2036         wait(clk_i_d == 1'b1);
2037         if (rvfi_trace.q.size() != 0) begin
2038             set_rvfi();
2039             rvfi_valid = 1'b1;
2040         end else begin
2041             rvfi_valid = 1'b0;
2042         end
2043         wait(clk_i_d == 1'b0);
2044     end
2045     endtask
2046
2047     initial begin
2048         fork
2049             monitor_pipeline();
2050             compute_pipeline();
2051             update_rvfi();
2052         join
2053     end
2054 endmodule // cv32e40p_rvfi

```